Notes

Trevor Vaughn

October 5, 2026

"There are more things in heaven and earth, Horatio,
Than are dreamt of in your philosophy."

—William Shakespeare, *Hamlet*

Contents

1 Foreword **12**

2 Analysis and Asymptotics **12**
2.1 Stolz–Cesàro . . . 12
2.2 Wallis Product . . . 13
2.3 Stirling's Formula . . . 14
2.4 Karamata's Inequality . . . 16
2.5 Polynomial Root Bounds . . . 18
2.5.1 Real Roots from Their Mean and Variance . . . 18
2.5.2 Positive Coefficients and Consecutive Ratios . . . 19
2.5.3 Gauss–Lucas Theorem . . . 19
2.6 Banach Fixed Point Theorem . . . 20
2.7 Inverse Function Theorem . . . 21
2.8 Implicit Function Theorem . . . 23
2.9 Lagrange Multipliers . . . 24
2.10 The Isoperimetric Inequality . . . 25

3 Linear Algebra **33**
3.1 Determinant and Inverse Identities . . . 33
3.1.1 Woodbury Identity . . . 35
3.1.2 Matrix Determinant Lemma . . . 36
3.2 Characteristic Polynomial . . . 37
3.2.1 Determinant of Affine Matrix Polynomial . . . 38
3.3 Frobenius Form . . . 38
3.3.1 Algorithm Description . . . 38
3.3.2 Correctness . . . 39
3.3.3 Failure Probability . . . 39
3.3.4 Polynomial Matrix Composition and Powering . . . 40
3.4 Black Box Linear Algebra . . . 41
3.4.1 Minimal Polynomial . . . 41
3.4.2 Determinant . . . 42
3.4.3 Wiedemann's Algorithm . . . 43
3.5 Perron-Frobenius Theorem . . . 44
3.6 Continuity of Eigenvalues . . . 45
3.7 Gershgorin's Circle Theorem . . . 46

4 Number Theory and Arithmetic **49**
4.1 Bertrand's Postulate and Products of Small Primes . . . 49
4.2 Lucas's Theorem . . . 51
4.3 Quadratic Reciprocity . . . 52
4.4 Values of the Zeta Function at Even Integers . . . 56
4.5 Hensel's Lemma . . . 58
4.6 Arithmetic Algorithms . . . 59
4.6.1 Linear Sieve . . . 59
4.6.2 Extended GCD with Preprocessing . . . 62
4.6.3 Rational Reconstruction . . . 65
4.6.4 Modular Square Roots . . . 67
4.6.5 Generalized Floor Sum . . . 69
4.6.6 Multiplicative Function Prefix Sums . . . 72
4.6.7 LCM and GCD Convolution . . . 74

4.6.8 Erdős–Ginzburg–Ziv . . . 75
4.6.9 Modular Subset Sum . . . 78
4.6.10 Modular Subset Sum using Runs . . . 79
4.7 Fast Factorials and Deterministic Factorization . . . 82
4.7.1 Fast Factorial . . . 83
4.7.2 Deterministic Integer Factorization . . . 85
4.8 Miller–Rabin Primality Test . . . 88

5 Combinatorics **94**
5.1 Entropy Method . . . 94
5.1.1 Shearer's Lemma . . . 94
5.1.2 Entropy Bound for Binomial Sums . . . 94
5.1.3 Irregular Moore Bound . . . 95
5.2 Matchings . . . 98
5.2.1 Hall's Marriage Theorem . . . 98
5.2.2 Gale–Ryser Theorem . . . 98
5.2.3 Cantor–Schröder–Bernstein . . . 99
5.2.4 Brégman's Theorem . . . 100
5.2.5 Kahn–Lovász Theorem . . . 101
5.2.6 The van der Waerden Permanent Theorem . . . 102
5.3 Additive Combinatorics . . . 105
5.3.1 Kneser's Addition Theorem . . . 105
5.3.2 Iterated Modular Sumsets . . . 106
5.3.3 Ruzsa's Triangle Inequality and Covering Lemma . . . 108
5.3.4 Plünnecke–Ruzsa Inequalities . . . 109
5.3.5 Multiway Sumset Inequalities . . . 110
5.3.6 Short Sums and Long Arithmetic Progressions . . . 111
5.3.7 A Subset-Sum Intersection Bound . . . 112
5.3.8 Witnesses for Long Progressions in Repeated Sumsets . . . 114
5.3.9 Witnesses for Long Progressions in Subset Sums . . . 118
5.3.10 Additive Energy . . . 122
5.3.11 Balog–Szemerédi–Gowers . . . 123
5.3.12 Algorithmic BSG . . . 124
5.4 Extremal Set Theory . . . 128
5.4.1 Bollobás Two Families Theorem . . . 128
5.4.2 Exterior Algebra and the Skew Two Families Theorem . . . 129
5.4.3 Sunflower Lemma . . . 131
5.5 Ramsey Theory . . . 135
5.5.1 Hales–Jewett Theorem . . . 135
5.6 Polynomial Method . . . 138
5.6.1 Triangular Independence . . . 138
5.6.2 Combinatorial Nullstellensatz . . . 138
5.6.3 Multiplicity Schwartz–Zippel . . . 144
5.6.4 Zero Patterns of Polynomial Maps . . . 145
5.6.5 Regular Subgraphs from Chevalley–Warning . . . 146
5.6.6 Alon–Tarsi Orientations and List Coloring . . . 147
5.6.7 Restricted Intersections . . . 148
5.6.8 Slice Rank and Low-Degree Tensor Methods . . . 151
5.7 Probabilistic Method . . . 163
5.7.1 Isolation Lemma . . . 163
5.7.2 Random-Order Independent Sets . . . 164
5.7.3 Counting Independent Sets in Regular Graphs . . . 167

5.7.4 Triangle-Free Independent Sets and Ramsey Numbers . . . 168
5.7.5 Moser–Tardos Resampling . . . 171
5.8 Concentration Inequalities . . . 173
5.8.1 Hoeffding . . . 173
5.8.2 Chernoff . . . 175
5.8.3 Bernstein . . . 175
5.9 Martingales . . . 176
5.9.1 Azuma and McDiarmid . . . 177
5.9.2 Freedman's Inequality . . . 179
5.9.3 Generalized Hölder and Read-k Concentration . . . 181
5.10 Enumerative and Algebraic Combinatorics . . . 182
5.10.1 Lindström–Gessel–Viennot Lemma . . . 182
5.10.2 Burnside's Lemma and Pólya Enumeration . . . 184
5.10.3 Q-analogues . . . 190
5.10.4 Jacobi Triple Product . . . 194
5.10.5 Pentagonal Number Theorem . . . 196
5.10.6 MacMahon's Master Theorem . . . 197
5.10.7 Hook-Length Formula . . . 199
5.10.8 The Fundamental Theorem of Symmetric Polynomials . . . 203
5.10.9 Robinson–Schensted Correspondence . . . 205
5.10.10 Ehrhart Polynomials . . . 206
5.11 Möbius Inversion on Posets . . . 210
5.11.1 Incidence Algebra and the Möbius Function . . . 210
5.11.2 The Inversion Formula . . . 211
5.11.3 Products of Posets . . . 211
5.11.4 Differences, Inclusion–Exclusion, and Divisor Sums . . . 212
5.11.5 Subspaces over a Finite Field . . . 213
5.11.6 Set Partitions . . . 213
5.11.7 Weighted Inversion . . . 214

6 Convexity and Duality **214**
6.1 Sion's Minimax Theorem . . . 214
6.2 Linear Programming Duality . . . 217
6.3 Lagrangian Relaxation . . . 220
6.4 Seidel's Algorithm . . . 222
6.5 Majorization . . . 225
6.5.1 Muirhead's Inequality . . . 226
6.6 SMAWK and Totally Monotone Matrices . . . 227
6.7 Concave Min-Plus Convolution . . . 230
6.7.1 Description . . . 230
6.7.2 Crossings . . . 230
6.7.3 Boundary Conditions . . . 230
6.8 Rectangular Monotone Min-Plus Product . . . 231
6.8.1 Column-Monotone Right Matrices . . . 234
6.9 Radon's Theorem . . . 234
6.10 Helly's Theorem . . . 235
6.11 Carathéodory's Theorem . . . 235
6.12 Colorful Carathéodory . . . 236
6.13 Tverberg's Theorem . . . 237
6.14 Matroid Intersection . . . 239
6.15 Symmetric Submodular Minimization . . . 243

7 Complex Analysis **246**
7.1 Cauchy's Theorem . . . 246
7.2 Cauchy's Integral Formula . . . 248
7.3 Laurent Series . . . 249
7.4 Residue Theorem . . . 251
7.5 Cotangent Partial Fractions . . . 252
7.6 Euler's Product for Sine . . . 253

8 Data Structures **254**
8.1 RMQ . . . 254
8.2 DSU . . . 256
8.2.1 Rank Facts . . . 256
8.2.2 Ackermann Levels . . . 257
8.2.3 Potential . . . 257
8.2.4 Why a Find Costs $O(\alpha)$ Amortized . . . 258
8.3 DSU with Potential . . . 259
8.4 Persistent Queue . . . 259
8.5 Online Suffix Max . . . 260
8.6 Mo's Algorithm . . . 261
8.7 A Fast Succinct Hash Table . . . 263
8.7.1 Small Buckets . . . 263
8.7.2 Storing the Full Keys Succinctly . . . 264
8.7.3 Local Fingerprints . . . 264
8.7.4 A Constant-Time Dictionary Inside a Bucket . . . 265
8.7.5 Cost of Fingerprint Rebuilding . . . 266
8.7.6 Deamortizing Rebuilds . . . 266
8.7.7 The Update Buffers Are Succinct . . . 268
8.7.8 Dynamic Resizing . . . 268
8.7.9 Summary . . . 268
8.8 Simulating Fully Random Hashing . . . 268
8.8.1 Private Random Cells . . . 269
8.8.2 Splitting a Large Set into Small Fixed Sets . . . 270
8.8.3 Randomize After Splitting . . . 271
8.9 Soft Heaps . . . 272
8.9.1 A Forest of Heaps . . . 272
8.9.2 Combine Two Representatives, Keep the Larger Key . . . 272
8.9.3 Root Lists and Heap Operations . . . 273
8.9.4 Why Only Few Items Are Corrupted . . . 274
8.9.5 Amortized Cost . . . 275
8.10 Sampling . . . 275
8.10.1 Alias Tables . . . 276
8.10.2 Exponential Races . . . 277
8.10.3 Priority Sampling . . . 277

9 Trees **279**
9.1 LCA . . . 279
9.2 $O(n)$ Binary Lifting . . . 279
9.3 Virtual Tree . . . 282
9.4 XOR-Linked Traversal . . . 283
9.5 DFS . . . 284
9.6 Diameter . . . 284
9.7 Shallowest Decomposition Tree . . . 285

9.8 Static Top Tree . . . 287
9.8.1 The Four Operations . . . 288
9.8.2 Heavy Paths . . . 289
9.8.3 The Expression Tree . . . 289
9.8.4 Weight-Balanced Merging . . . 290
9.8.5 The DP Interface . . . 292
9.8.6 Point Updates . . . 292
9.8.7 Rooted-Subtree Queries . . . 293
9.8.8 Path Queries . . . 293
9.9 Splay Tree . . . 296
9.9.1 Rotation . . . 296
9.9.2 Splaying . . . 296
9.9.3 Access Lemma . . . 297
9.9.4 Augmentation . . . 299
9.10 Link-Cut Tree . . . 299
9.10.1 Preferred Paths . . . 299
9.10.2 How a Preferred Path Is Stored . . . 300
9.10.3 Connecting the Auxiliary Trees . . . 301
9.10.4 Access . . . 301
9.10.5 Why Access Is Fast . . . 302
9.10.6 The Analysis Tree . . . 303
9.10.7 How Many Auxiliary Trees Does Access Cross? . . . 304
9.10.8 Rerooting . . . 305
9.10.9 Dynamic-Forest Operations . . . 305
9.10.10 Path Queries and Updates . . . 305
9.11 Top Tree . . . 306
9.11.1 Real and Fake Nodes . . . 307
9.11.2 The Same Path/Point Interface Reappears . . . 307
9.11.3 Access with Virtual-Child Trees . . . 308
9.11.4 Aggregates . . . 308
9.11.5 Two Kinds of Lazy Tags . . . 310
9.11.6 Push Before Changing the Decomposition . . . 310
9.11.7 Path Operations . . . 311
9.11.8 Subtree Operations . . . 311
9.11.9 Link, Cut, and Reroot . . . 312
9.11.10 Amortized Analysis . . . 312
9.12 Lazy Anti-Monopoly Tree . . . 313

10 Graph Algorithms **316**
10.1 Incremental SCCs . . . 316
10.2 All-Pairs Connectivity . . . 318
10.2.1 A Walk Generating Matrix . . . 319
10.2.2 Determinants Enumerate Edge-Disjoint Walks . . . 319
10.2.3 Connectivity is Matrix Rank . . . 320
10.2.4 Random Evaluation . . . 321
10.2.5 Bounded All-Pairs Connectivity . . . 322
10.3 Flows . . . 326
10.3.1 Highest Label Push-Relabel . . . 326
10.3.2 Flow Rounding . . . 327
10.4 Cut-Matching Game and Sparse Cuts . . . 329
10.5 Arboricity . . . 332
10.6 Directed Minimum Spanning Tree . . . 333

10.6.1 The Two Reductions . . . 334
10.6.2 Meldable Incoming-Edge Heaps . . . 334
10.6.3 Detecting Cycles with DSU . . . 335
10.6.4 Maintaining Contractions . . . 335
10.6.5 Fast Directed MST Algorithm . . . 336
10.6.6 Correctness . . . 336
10.6.7 Rollback Reconstruction . . . 337
10.6.8 Running Time . . . 337
10.7 Matching and Coloring . . . 338
10.7.1 Bipartite Regular Matching . . . 338
10.7.2 Kasteleyn Signing and Planar Perfect Matchings . . . 338
10.7.3 Bipartite Edge Coloring . . . 342
10.7.4 Chromatic Number . . . 347
10.8 Node-Weighted Triangle . . . 349
10.8.1 One Fixed Weight . . . 349
10.8.2 The Square Case . . . 349
10.8.3 Rectangular Version . . . 351
10.8.4 Minimum Node-Weighted Triangle . . . 354
10.9 Undirected Hamiltonicity . . . 355
10.10 An Algebraic Criterion for Bipartite Perfect Matching . . . 359
10.10.1 The Wronskian as a Multiplicity Budget . . . 360
10.10.2 Designing the Polynomial System . . . 361
10.10.3 Hall Deficiency Leaves Too Much Freedom . . . 363
10.10.4 Private Zeros Force the Active Polynomials to be Independent . . . 364
10.10.5 Hall Expansion Exhausts the Remaining Multiplicity Budget . . . 365
10.10.6 The Resulting Algorithm . . . 366
10.11 Shortest Paths . . . 368
10.11.1 Eppstein's Algorithm . . . 368
10.11.2 Bellman–Ford–Dijkstra . . . 370

11 Algebraic and Spectral Graph Theory **372**
11.1 The Matrix–Tree Theorem . . . 372
11.1.1 Counting Hamiltonian Paths . . . 373
11.1.2 Counting k-Matchings in a Bipartite Graph . . . 374
11.1.3 The BEST Theorem . . . 376
11.2 Cheeger Inequality . . . 377
11.2.1 An Explicit Constant-Degree Expander . . . 380
11.3 Expander Products . . . 384
11.3.1 Lifting a Walk to Edge Labels . . . 384
11.3.2 Derandomized Squaring . . . 385
11.3.3 The Zig-Zag Product . . . 386
11.4 Alon–Boppana . . . 387
11.4.1 The Infinite Binary Tree . . . 387
11.4.2 From a Tree to a Graph Ball . . . 388
11.5 Lower Bound on Spectral Sparsification . . . 390

12 Electrical Flows **390**
12.1 Electrical Flows . . . 391
12.1.1 Potentials and Ohm's Law . . . 392
12.1.2 Effective Resistance . . . 392
12.1.3 Maximum Principle . . . 393
12.1.4 Dirichlet's Principle . . . 393

12.1.5 Series and Parallel Laws . . . 395
12.1.6 Rayleigh Monotonicity . . . 395
12.1.7 Effective Resistance is a Metric . . . 396
12.2 Random Walks and Electrical Networks . . . 396
12.2.1 Voltages are Hitting Probabilities . . . 397
12.2.2 Currents are Expected Net Traversals . . . 397
12.2.3 Hitting a Vertex Through a Given Edge . . . 398
12.2.4 Hitting Times . . . 398
12.2.5 Commute Time . . . 399
12.2.6 Tetali's Hitting-Time Formula . . . 399
12.2.7 Cover-Time Bound . . . 400
12.3 Kron Reduction . . . 400
12.4 Infinite Graphs, Recurrence, and Transience . . . 401
12.4.1 Infinite Thomson Principle . . . 402
12.4.2 Escape Probability . . . 403
12.4.3 Nash–Williams Recurrence Criterion . . . 403
12.4.4 Trees . . . 404
12.5 Random Spanning Trees . . . 404
12.5.1 Weighted Random Spanning Trees . . . 405
12.6 Localization of Electrical Flows . . . 407
12.6.1 Replacing L^+ by a Continuous-Time Walk . . . 408
12.6.2 Splitting the Walk at its Midpoint . . . 410
12.6.3 Turning Edge Differences into a Time Derivative . . . 411
12.6.4 Choosing the Stationary Distribution . . . 415
12.6.5 Integrating the Dissipation . . . 415
12.6.6 Completing the Localization Bound . . . 417

13 Matrix Analysis **418**
13.1 Sampling Matrices without Replacement . . . 418
13.2 Schur and Kronecker Product . . . 418
13.3 Differentiating Matrix Functions . . . 419
13.3.1 Product, Inverse, and Power Rules . . . 419
13.3.2 Divided Differences . . . 420
13.3.3 Traces Recover the Scalar Derivative Rule . . . 422
13.3.4 The Matrix Exponential . . . 423
13.4 Integral Representations and Fractional Powers . . . 424
13.4.1 The Logarithm . . . 424
13.4.2 Fractional Powers . . . 425
13.4.3 Operator Convexity of Inversion . . . 427
13.4.4 Löwner–Heinz and Operator Concavity of Powers . . . 427
13.5 Golden–Thompson Inequality . . . 428
13.6 Trace Rearrangement and Schatten Hölder Inequalities . . . 430
13.7 Generalized Klein Inequality . . . 433
13.8 Wielandt Minimax Formula . . . 434
13.9 The Operator Jensen Inequality . . . 436
13.10 Lieb's Concavity Theorem . . . 437
13.10.1 Joint Convexity of Matrix Relative Entropy . . . 439
13.10.2 A Variational Formula for the Trace . . . 440
13.10.3 Lieb's Trace-Exponential Theorem . . . 440
13.11 Lieb–Thirring Inequalities . . . 441
13.12 Matrix Concentration . . . 444

14 Convex Optimization **450**
14.1 Linear Systems and Preconditioned Richardson Iteration . . . 450
14.2 Smooth Gradient Descent . . . 451
14.3 Bregman Proximal Steps and Relative Curvature . . . 452
14.3.1 Relative Descent . . . 453
14.3.2 Where Norms Enter . . . 453
14.4 Mirror Descent, Entropy, and Multiplicative Weights . . . 453
14.4.1 Entropy on the Simplex . . . 454
14.4.2 Soft Maximum and the Same Entropy Weights . . . 455
14.5 Matrix Multiplicative Weights . . . 455
14.6 Sparsification and Sensitivity . . . 456
14.6.1 Leverage Scores . . . 457
14.6.2 The Sampling Algorithm . . . 457
14.6.3 Why Matrix Concentration Gives Uniform Preservation . . . 458
14.6.4 Why Higher Powers Can Require More Rows . . . 459
14.7 Estimating Leverage Scores and Bootstrapping Linear Solvers . . . 459
14.7.1 Sketch First, Then Solve . . . 459
14.7.2 Why Regularization Removes the Circularity . . . 460
14.7.3 Cost and the Final Unregularized Solve . . . 461
14.8 Lewis Weights and ℓ_p Geometry . . . 461
14.8.1 Why This Scaling Matches the p-Norm . . . 462
14.8.2 The Lewis Ellipsoid . . . 462
14.8.3 Computing the Weights by a Fixed Point . . . 463
14.8.4 A Convex Characterization for $p > 2$ . . . 464
14.8.5 A Sensitivity Bound from the Ellipsoid . . . 465
14.9 High-Accuracy ℓ_p Regression . . . 465
14.9.1 Two Scales in the Power Function . . . 465
14.9.2 Why a Local Improvement Reduces the Global Gap . . . 467
14.9.3 The Penalty of an Optimal Direction . . . 468
14.9.4 Replacing the Power Penalty by a Quadratic . . . 468
14.9.5 Choosing the Scale Without Knowing the Optimum . . . 469
14.10 Width Reduction and Iteratively Reweighted Least Squares . . . 471
14.10.1 The Decision Problem and Its Quadratic Relaxation . . . 471
14.10.2 The Algorithm . . . 471
14.10.3 Why Squared Violations Are the Right Multipliers . . . 472
14.10.4 Counting Moderate and Very-Wide Iterations . . . 474
14.10.5 From Approximate Decision to Optimization . . . 475
14.11 Interior-Point Methods . . . 475
14.11.1 The Central Path and Its Optimality Certificate . . . 476
14.11.2 The Path-Following Algorithm . . . 476
14.11.3 Self-Concordance: When Is the Newton Model Reliable? . . . 478
14.11.4 Damped Newton Steps and a Centered Starting Point . . . 479
14.11.5 Quadratic Recentering with Full Newton Steps . . . 480
14.11.6 How Far to Move the Path . . . 481
14.12 Saddle Points and Mirror-Prox . . . 482
14.12.1 Why Prediction Helps . . . 482
14.13 Area Convexity and Fast Box–Simplex Games . . . 483
14.13.1 The Error Created by Prediction and Correction . . . 484
14.13.2 Choosing Curvature to Match the Matrix . . . 485
14.13.3 Area Convexity and a Local Test . . . 485
14.13.4 Deriving the Inner Updates . . . 487

14.13.5 The Complete Finite-Step Implementation . . . 488
14.13.6 Why the Inner Errors Have the Required Form . . . 490
14.13.7 The Outer Telescope and Running Time . . . 491
14.14 Online Discrepancy by the Gaussian Triplet Walk . . . 492
14.14.1 The Full Triplet Algorithm . . . 492
14.14.2 Why the Fractional Walk Preserves a Gaussian . . . 493
14.14.3 Why the Coupling Has Two Branches . . . 494
14.14.4 The Invariant and Its Concentration Consequence . . . 495
14.14.5 Why the Ambient Dimension Disappears . . . 496

15 Polynomial Algorithms **497**
15.1 Formal Power Series Operations . . . 497
15.1.1 Inverse . . . 498
15.1.2 Logarithm . . . 498
15.1.3 Square Root . . . 498
15.1.4 Inverse Square Root . . . 499
15.1.5 Exponential . . . 499
15.1.6 Powers . . . 499
15.2 Polynomial Arithmetic . . . 500
15.2.1 Division . . . 500
15.2.2 Taylor Shift . . . 500
15.2.3 Evaluation Shift . . . 501
15.2.4 Half-GCD and the Euclidean Quotient Sequence . . . 501
15.2.5 Modular Inversion . . . 503
15.2.6 Resultants and Discriminants . . . 503
15.2.7 Root Finding over a Prime Field . . . 504
15.2.8 Minimum Linear Recurrence . . . 505
15.3 Rational Functions and Transposition . . . 506
15.3.1 Bostan–Mori . . . 506
15.3.2 The Transposition Principle . . . 507
15.3.3 Transposed Multiplication and Reversal Pairings . . . 508
15.3.4 Transposed Bostan–Mori and Modular Powers . . . 508
15.4 Composition and Compositional Inversion . . . 509
15.4.1 Power Projection . . . 509
15.4.2 Composition by Transposition . . . 510
15.4.3 Lagrange Inversion . . . 512
15.4.4 Compositional Inversion by One Power Projection . . . 512
15.5 Evaluation and Interpolation . . . 513
15.5.1 Multipoint Evaluation by Transposition . . . 513
15.5.2 Interpolation . . . 514
15.5.3 Chirp-Z: Geometric Evaluation . . . 515
15.5.4 Inverse Chirp-Z . . . 515
15.5.5 Hermite Interpolation . . . 516
15.6 Multivariate Evaluation by Integer Lifting . . . 517
15.6.1 Product Grids and Nilpotent Taylor Expansion . . . 518
15.6.2 Recursive Prime-Power Reduction . . . 518
15.6.3 Extension Fields . . . 520
15.7 Multilinear Polynomials . . . 522
15.7.1 Multilinear-Term Detection . . . 522
15.8 Algebraic Series, D-Finiteness, and Recurrence Algorithms . . . 526
15.8.1 Differential Equations and Coefficient Recurrences . . . 526
15.8.2 Algebraic Series and Closure Properties . . . 528

15.8.3 Root Power Sums and Products of Rational Sequences . . . 529
15.8.4 Sparse Interpolation through Rational Recurrences . . . 530
15.8.5 Distant Polynomial and Geometric Recurrences . . . 532
15.8.6 Batched Recurrences with a Varying Parameter . . . 534
15.8.7 Exponential and Logarithmic Substitution . . . 535
15.8.8 Linear-Time Translation and Single-Coefficient Queries . . . 537
15.8.9 Diagonals, Constant Terms, and Residues . . . 540
15.8.10 Coefficients in Finite Fields by Reading Index Digits . . . 541
15.9 Fast Formal-Series Equations and Transformations . . . 544
15.9.1 Newton Iteration for Scalar Differential Equations . . . 544
15.9.2 CDQ Convolution and Causal Power-Series Equations . . . 545
15.10 Fractional-Linear and Cubic Substitution . . . 547
15.10.1 Homogeneous Fractional-Linear Substitution . . . 548
15.10.2 A Reversible Symmetric Laurent Substitution . . . 549
15.10.3 Chebyshev and Cubic Substitution . . . 550
15.11 Dirichlet Series . . . 551
15.11.1 The Dirichlet Derivative and Triangular Recurrences . . . 552
15.11.2 Compressed Prefix Tables . . . 554
15.11.3 Compressed Multiplication . . . 555
15.11.4 Compressed Logarithms and Exponentials . . . 556
15.11.5 Prime Counting . . . 558
15.11.6 Logarithmic Binning and Boundary Correction . . . 559

16 Set Power Series **561**
16.1 Subset Convolution . . . 561
16.2 Transposed Subset Convolution . . . 563
16.3 Inversion and Quotient . . . 563
16.4 Exponential and Logarithm . . . 565
16.5 Composition . . . 566
16.6 Power Projection . . . 568
16.7 Power . . . 569

1 Foreword

The goal of these notes is primarily to explain interesting things. I do not claim to have invented anything herein, although some I have simplified or generalized. Some of the algorithms contained herein have been implemented in my library.

2 Analysis and Asymptotics

2.1 Stolz–Cesàro

Stolz–Cesàro is the discrete analogue of l'Hôpital's rule. The useful idea is that a ratio of two growing sequences is controlled by the ratios of their increments.

Theorem 1 (Stolz–Cesàro)**.** *Let (a_n) and (b_n) be real sequences.*

(i) *Suppose b_n is strictly increasing and*

$$b_n \to \infty.$$

Then

$$\boxed{\liminf_{n\to\infty} \frac{a_{n+1}-a_n}{b_{n+1}-b_n} \le \liminf_{n\to\infty} \frac{a_n}{b_n} \le \limsup_{n\to\infty} \frac{a_n}{b_n} \le \limsup_{n\to\infty} \frac{a_{n+1}-a_n}{b_{n+1}-b_n}.}$$

(ii) *Suppose*

$$a_n \to 0, \qquad b_n \to 0,$$

and b_n is strictly decreasing. Then the same inequalities hold:

$$\boxed{\liminf_{n\to\infty} \frac{a_{n+1}-a_n}{b_{n+1}-b_n} \le \liminf_{n\to\infty} \frac{a_n}{b_n} \le \limsup_{n\to\infty} \frac{a_n}{b_n} \le \limsup_{n\to\infty} \frac{a_{n+1}-a_n}{b_{n+1}-b_n}.}$$

In either case, if

$$\frac{a_{n+1}-a_n}{b_{n+1}-b_n} \to L,$$

then

$$\boxed{\frac{a_n}{b_n} \to L.}$$

Proof. Write

$$\ell = \liminf_{n\to\infty} \frac{a_{n+1}-a_n}{b_{n+1}-b_n}, \qquad u = \limsup_{n\to\infty} \frac{a_{n+1}-a_n}{b_{n+1}-b_n}.$$

Fix $\varepsilon > 0$.

For the ∞ case, for all sufficiently large k,

$$(\ell-\varepsilon)(b_{k+1}-b_k) \le a_{k+1}-a_k \le (u+\varepsilon)(b_{k+1}-b_k).$$

Summing from a fixed sufficiently large N to $n-1$ gives

$$(\ell-\varepsilon)(b_n-b_N) \le a_n-a_N \le (u+\varepsilon)(b_n-b_N).$$

Divide by b_n and let $n\to\infty$. Since $b_n\to\infty$, the fixed terms disappear, yielding

$$\ell-\varepsilon \le \liminf_{n\to\infty} \frac{a_n}{b_n} \le \limsup_{n\to\infty} \frac{a_n}{b_n} \le u+\varepsilon.$$

For the $0/0$ case, it is more convenient to reverse both differences. Since b_n is strictly decreasing,

$$b_k - b_{k+1} > 0,$$

and

$$\frac{a_k - a_{k+1}}{b_k - b_{k+1}} = \frac{a_{k+1} - a_k}{b_{k+1} - b_k}.$$

Thus, for all sufficiently large k,

$$(\ell - \varepsilon)(b_k - b_{k+1}) \le a_k - a_{k+1} \le (u + \varepsilon)(b_k - b_{k+1}).$$

Sum from $k = n$ to $m - 1$:

$$(\ell - \varepsilon)(b_n - b_m) \le a_n - a_m \le (u + \varepsilon)(b_n - b_m).$$

Now let $m \to \infty$. Since

$$a_m \to 0, \qquad b_m \to 0,$$

we obtain

$$(\ell - \varepsilon)b_n \le a_n \le (u + \varepsilon)b_n.$$

A strictly decreasing sequence converging to 0 is positive, so dividing by b_n gives

$$\ell - \varepsilon \le \frac{a_n}{b_n} \le u + \varepsilon$$

for all sufficiently large n. Hence again

$$\ell - \varepsilon \le \liminf_{n\to\infty} \frac{a_n}{b_n} \le \limsup_{n\to\infty} \frac{a_n}{b_n} \le u + \varepsilon.$$

Letting $\varepsilon \to 0$ proves both statements. □

2.2 Wallis Product

Theorem 2 (Wallis Product)**.**

$$\frac{\pi}{2} = \prod_{k=1}^{\infty} \frac{(2k)^2}{(2k-1)(2k+1)}.$$

Proof. Let

$$I_m = \int_0^{\pi/2} \sin^m x \, dx.$$

Integration by parts gives, for $m \ge 2$,

$$I_m = \frac{m-1}{m} I_{m-2}.$$

Since $I_0 = \pi/2$ and $I_1 = 1$, this gives

$$I_{2n} = \frac{\pi}{2} \prod_{k=1}^{n} \frac{2k-1}{2k}, \qquad I_{2n+1} = \prod_{k=1}^{n} \frac{2k}{2k+1}.$$

Because $0 \le \sin x \le 1$,

$$I_{2n+1} \le I_{2n} \le I_{2n-1}.$$

The recurrence also gives

$$\frac{I_{2n-1}}{I_{2n+1}} = \frac{2n+1}{2n} \to 1.$$

Therefore, by squeezing,

$$\frac{I_{2n}}{I_{2n+1}} \to 1.$$

On the other hand,

$$\frac{I_{2n}}{I_{2n+1}} = \frac{\pi}{2}\frac{(2n-1)!!(2n+1)!!}{((2n)!!)^2} = \frac{\pi/2}{\displaystyle\prod_{k=1}^{n}\frac{(2k)^2}{(2k-1)(2k+1)}}.$$

Thus the product tends to $\pi/2$. □

The same argument immediately gives the asymptotic form of the central binomial coefficient, which will determine the constant in Stirling's formula.

Corollary 3 (Central Binomial Coefficient)**.**

$$\binom{2n}{n} \sim \frac{4^n}{\sqrt{\pi n}}.$$

Proof. Set

$$c_n = \frac{\binom{2n}{n}}{4^n} = \frac{(2n-1)!!}{(2n)!!}.$$

Then

$$I_{2n} = \frac{\pi}{2}c_n, \qquad I_{2n+1} = \frac{1}{(2n+1)c_n}.$$

Since $I_{2n}/I_{2n+1} \to 1$,

$$\frac{\pi}{2}(2n+1)c_n^2 \to 1.$$

Therefore

$$c_n \sim \frac{1}{\sqrt{\pi n}},$$

which is the claim. □

2.3 Stirling's Formula

Theorem 4 (Stirling's Formula)**.**

$$n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n.$$

Proof. We proceed in three steps. The integral approximation

$$\log(n!) = \sum_{k=1}^{n}\log k \approx \int_1^n \log x\,dx = n\log n - n + 1$$

suggests first removing the terms $n\log n - n$. Define

$$s_n = \log(n!) - n\log n + n.$$

We determine the size of the remaining error using Stolz–Cesàro with $b_n = \log n$. We have

$$s_{n+1} - s_n = 1 - n\log\left(1 + \frac{1}{n}\right),$$

and

$$\log(n+1) - \log n = \log\left(1 + \frac{1}{n}\right).$$

Thus

$$\frac{s_{n+1} - s_n}{\log(n+1) - \log n} = \frac{1}{\log(1+1/n)} - n.$$

and hence

$$\frac{s_{n+1} - s_n}{\log(n+1) - \log n} \to \frac{1}{2}.$$

Stolz–Cesàro therefore gives

$$\frac{s_n}{\log n} \to \frac{1}{2}.$$

So the next-order term is forced to be $\frac{1}{2}\log n$. This motivates

$$c_n = \log(n!) - \left(n + \frac{1}{2}\right)\log n + n.$$

We have

$$c_{n+1} - c_n = 1 - \left(n + \frac{1}{2}\right)\log\left(1 + \frac{1}{n}\right).$$

Set

$$m = n + \frac{1}{2}.$$

Then

$$\log\left(1 + \frac{1}{n}\right) = \log\frac{m + 1/2}{m - 1/2} = \int_{-1/2}^{1/2} \frac{dt}{m+t}.$$

Therefore

$$\begin{aligned} m\log\left(1 + \frac{1}{n}\right) - 1 &= \int_{-1/2}^{1/2} \left(\frac{m}{m+t} - 1\right) dt \\ &= \int_0^{1/2} \left(\frac{m}{m+t} + \frac{m}{m-t} - 2\right) dt \\ &= \int_0^{1/2} \frac{2t^2}{m^2 - t^2}\, dt. \end{aligned}$$

In particular,

$$0 \le m\log\left(1 + \frac{1}{n}\right) - 1 = O(n^{-2}).$$

Hence

$$|c_{n+1} - c_n| = O(n^{-2}).$$

Since $\sum_n n^{-2} < \infty$, the sequence (c_n) converges. Write

$$c_n \to \log C.$$

Exponentiating gives

$$n! \sim C\sqrt{n}\left(\frac{n}{e}\right)^n$$

for some constant $C > 0$.

Apply the preceding asymptotic to

$$\binom{2n}{n} = \frac{(2n)!}{(n!)^2}.$$

We obtain

$$\begin{aligned}\binom{2n}{n} &\sim \frac{C\sqrt{2n}(2n/e)^{2n}}{C^2 n(n/e)^{2n}} \\ &= \frac{\sqrt{2}}{C}\frac{4^n}{\sqrt{n}}.\end{aligned}$$

But by Corollary 3,

$$\binom{2n}{n} \sim \frac{4^n}{\sqrt{\pi n}}.$$

Comparing constants gives

$$\frac{\sqrt{2}}{C} = \frac{1}{\sqrt{\pi}},$$

so

$$C = \sqrt{2\pi}.$$

Therefore

$$n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n.$$

□

2.4 Karamata's Inequality

Karamata's inequality is a consequence of two facts:

- majorization says that the partial-sum differences have one sign;
- convexity says that secant slopes increase as their endpoints move to the right.

Summation by parts then combines these two.

Theorem 5 (Karamata's Inequality)**.** *Let*

$$x, y \in \mathbb{R}^n, \qquad x \prec y,$$

and let f be convex on an interval containing all coordinates of x and y. Then

$$\boxed{\sum_{i=1}^{n} f(x_i) \le \sum_{i=1}^{n} f(y_i).}$$

More generally, if

$$x \prec_w y$$

and f is convex and nondecreasing, then the same inequality holds.

Proof. Since both majorization and the desired inequality are invariant under permuting coordinates, assume

$$x_1 \ge \cdots \ge x_n, \qquad y_1 \ge \cdots \ge y_n.$$

Define the partial sums

$$X_k = \sum_{i=1}^{k} x_i, \qquad Y_k = \sum_{i=1}^{k} y_i,$$

and their difference

$$D_k = Y_k - X_k, \qquad D_0 = 0.$$

Since $x \prec y$,

$$D_k \geq 0 \qquad (1 \leq k < n),$$

and

$$D_n = 0.$$

If $x_i = y_i$ for some i, we may delete both coordinates. The two remaining sequences are still decreasing, and their partial-sum inequalities are unchanged after reindexing. Thus we may assume

$$x_i \neq y_i \qquad \text{for every } i.$$

For each i, define the secant slope

$$c_i = \frac{f(y_i) - f(x_i)}{y_i - x_i}.$$

The basic one-dimensional fact about a convex function is that its secant slope is nondecreasing in either endpoint: if

$$a \leq a', \qquad b \leq b',$$

then

$$\frac{f(b) - f(a)}{b - a} \leq \frac{f(b') - f(a')}{b' - a'}$$

whenever the quotients are defined.

Since

$$x_i \geq x_{i+1}, \qquad y_i \geq y_{i+1},$$

both endpoints of the i-th secant lie no farther left than the corresponding endpoints of the $(i+1)$-st secant. Therefore

$$\boxed{c_1 \geq c_2 \geq \cdots \geq c_n.}$$

Now

$$y_i - x_i = D_i - D_{i-1},$$

so

$$\begin{aligned}\sum_{i=1}^{n} \big(f(y_i) - f(x_i)\big) &= \sum_{i=1}^{n} c_i (y_i - x_i) \\ &= \sum_{i=1}^{n} c_i (D_i - D_{i-1}).\end{aligned}$$

Summation by parts gives

$$\sum_{i=1}^{n} c_i (D_i - D_{i-1}) = c_n D_n + \sum_{i=1}^{n-1} (c_i - c_{i+1}) D_i.$$

For ordinary majorization,

$$D_n = 0,$$

while

$$c_i - c_{i+1} \geq 0, \qquad D_i \geq 0.$$

Hence

$$\sum_{i=1}^{n} \big(f(y_i) - f(x_i)\big) \geq 0,$$

which proves

$$\sum_i f(x_i) \le \sum_i f(y_i).$$

For weak majorization $x \prec_w y$, we instead have

$$D_i \ge 0 \qquad (1 \le i \le n).$$

If f is also nondecreasing, every secant slope satisfies

$$c_i \ge 0.$$

Thus the additional endpoint term

$$c_n D_n$$

is also nonnegative, and the same argument proves the weak form. □

2.5 Polynomial Root Bounds

2.5.1 Real Roots from Their Mean and Variance

The first two coefficients determine the sum and sum of squares of the roots. When the roots are real, this controls how far any one root can lie from their mean: the other roots must compensate for its deviation, and doing so consumes part of the same sum-of-squares budget.

Theorem 6 (Real-rooted polynomial bound)**.** *Let*

$$p(x) = x^n + a_{n-1}x^{n-1} + \cdots + a_0, \qquad n \ge 2,$$

have only real roots. Then every root lies in the interval with endpoints

$$\boxed{-\frac{a_{n-1}}{n} \pm \frac{n-1}{n}\sqrt{a_{n-1}^2 - \frac{2n}{n-1}a_{n-2}}.}$$

Proof. Write the roots, counted with multiplicity, as $r_1, \ldots, r_n$. Let $\mu = n^{-1}\sum_i r_i$ and $\sigma^2 = n^{-1}\sum_i (r_i - \mu)^2$. Vieta's formulas give

$$\sum_i r_i = -a_{n-1}, \qquad \sum_i r_i^2 = \left(\sum_i r_i\right)^2 - 2\sum_{i<j} r_i r_j = a_{n-1}^2 - 2a_{n-2}.$$

Consequently,

$$\mu = -\frac{a_{n-1}}{n}, \qquad \sigma^2 = \frac{a_{n-1}^2 - 2a_{n-2}}{n} - \frac{a_{n-1}^2}{n^2} = \frac{n-1}{n^2}a_{n-1}^2 - \frac{2}{n}a_{n-2}.$$

Fix i. Since the deviations sum to zero, Cauchy–Schwarz gives

$$\begin{aligned}(r_i - \mu)^2 &= \left(\sum_{j \ne i}(r_j - \mu)\right)^2 \\ &\le (n-1)\sum_{j \ne i}(r_j - \mu)^2 \\ &= (n-1)\big(n\sigma^2 - (r_i - \mu)^2\big).\end{aligned}$$

Rearranging yields $(r_i - \mu)^2 \le (n-1)\sigma^2$, or

$$|r_i - \mu| \le \sqrt{n-1}\,\sigma = \frac{n-1}{n}\sqrt{a_{n-1}^2 - \frac{2n}{n-1}a_{n-2}}.$$

Substitute the value of μ. □

The bound is sharp: equality for a particular root holds exactly when the other $n-1$ roots are equal. More generally, the proof shows that any n real numbers of mean μ and variance σ^2 all lie within $\sqrt{n-1}\,\sigma$ of μ.

2.5.2 Positive Coefficients and Consecutive Ratios

Positive coefficients give bounds on root magnitudes without requiring the roots to be real.

Theorem 7 (Eneström–Kakeya)**.** *Let $p(z) = \sum_{k=0}^{n} a_k z^k$, where $n \geq 1$ and every $a_k > 0$. Put*

$$m = \min_{0 \leq k < n} \frac{a_k}{a_{k+1}}, \qquad M = \max_{0 \leq k < n} \frac{a_k}{a_{k+1}}.$$

Then every complex root z of p satisfies

$$\boxed{m \leq |z| \leq M.}$$

In particular, every real root belongs to $[-M, -m]$.

Proof. First suppose $q(z) = \sum_{k=0}^{n} b_k z^k$ has $0 < b_0 \leq b_1 \leq \cdots \leq b_n$. Then

$$(1-z)q(z) = b_0 + \sum_{k=1}^{n} (b_k - b_{k-1}) z^k - b_n z^{n+1}.$$

If $q(w) = 0$ and $t = |w| > 1$, the triangle inequality gives

$$\begin{aligned} b_n t^{n+1} &\leq b_0 + \sum_{k=1}^{n} (b_k - b_{k-1}) t^k \\ &\leq t^n \left(b_0 + \sum_{k=1}^{n} (b_k - b_{k-1}) \right) = b_n t^n, \end{aligned}$$

a contradiction. Thus every root of q lies in the closed unit disk.

For the original polynomial, take $q(w) = p(Mw)$. Its coefficients $b_k = a_k M^k$ are nondecreasing because

$$\frac{b_{k+1}}{b_k} = M \frac{a_{k+1}}{a_k} \geq 1.$$

If $p(z) = 0$, then $q(z/M) = 0$, so $|z| \leq M$.

For the lower bound, apply the upper bound to the reversed polynomial

$$p^*(w) = w^n p(1/w) = a_n + a_{n-1} w + \cdots + a_0 w^n.$$

Since $a_0 > 0$, every root z of p is nonzero, and $1/z$ is a root of p^*. Therefore

$$\frac{1}{|z|} \leq \max_{0 \leq k < n} \frac{a_{k+1}}{a_k} = \frac{1}{m},$$

which gives $|z| \geq m$.

Finally, $p(x) > 0$ for $x \geq 0$, so every real root is negative. □

2.5.3 Gauss–Lucas Theorem

Differentiation cannot move roots outside their convex hull.

Theorem 8 (Gauss–Lucas)**.** *Let $p \in \mathbb{C}[z]$ be nonconstant. Every zero of p' lies in the convex hull of the zeros of p.*

Proof. Write $p(z) = c\prod_{j=1}^{n}(z - r_j)$, listing the roots with multiplicity. For z not a root of p, the product rule gives

$$\frac{p'(z)}{p(z)} = \sum_{j=1}^{n} \frac{1}{z - r_j}.$$

Let $p'(w) = 0$. If $p(w) = 0$, there is nothing to prove. Otherwise,

$$0 = \sum_{j=1}^{n} \frac{1}{w - r_j} = \sum_{j=1}^{n} \frac{\overline{w} - \overline{r_j}}{|w - r_j|^2}.$$

Take complex conjugates and collect the terms containing w:

$$w \sum_{j=1}^{n} \frac{1}{|w - r_j|^2} = \sum_{j=1}^{n} \frac{r_j}{|w - r_j|^2}.$$

Consequently,

$$\boxed{w = \sum_{j=1}^{n} \alpha_j r_j, \qquad \alpha_j = \frac{|w - r_j|^{-2}}{\sum_{i=1}^{n} |w - r_i|^{-2}} > 0, \qquad \sum_{j=1}^{n} \alpha_j = 1.}$$

Thus w is a convex combination of the roots. □

2.6 Banach Fixed Point Theorem

A map $T : X \to X$ on a metric space is a *contraction* if there is some $q < 1$ such that

$$d(Tx, Ty) \leq q\, d(x, y)$$

for every $x, y \in X$.

The point is that repeatedly applying a contraction makes successive errors shrink geometrically.

Theorem 9 (Banach Fixed Point Theorem)**.** *Let X be a complete metric space and let $T : X \to X$ be a contraction with constant $q < 1$. Then T has a unique fixed point x^*.*

Moreover, starting from any $x_0 \in X$ and defining $x_{n+1} = T(x_n)$, we have

$$\boxed{d(x_n, x^*) \leq \frac{q^n}{1 - q}\, d(x_1, x_0).}$$

Proof. Since T contracts distances,

$$d(x_{n+1}, x_n) \leq q^n d(x_1, x_0).$$

Thus for $m > n$, the triangle inequality gives

$$d(x_m, x_n) \leq \sum_{j=n}^{m-1} d(x_{j+1}, x_j) \leq d(x_1, x_0) \sum_{j=n}^{\infty} q^j = \frac{q^n}{1 - q} d(x_1, x_0).$$

Hence (x_n) is Cauchy. Completeness gives $x_n \to x^*$ for some $x^* \in X$.

A contraction is continuous, so

$$T(x^*) = T\left(\lim_n x_n\right) = \lim_n T(x_n) = \lim_n x_{n+1} = x^*.$$

Thus x^* is a fixed point. Letting $m \to \infty$ in the preceding estimate gives the claimed error bound.

Finally, if y^* is another fixed point, then

$$d(x^*, y^*) = d(Tx^*, Ty^*) \leq q\, d(x^*, y^*).$$

Since $q < 1$, this forces $x^* = y^*$. □

The theorem is especially useful when an equation can be rewritten as $x = T(x)$. If T is a contraction on a closed region and maps that region into itself, the equation has exactly one solution there.

2.7 Inverse Function Theorem

Suppose $F : \mathbb{R}^n \to \mathbb{R}^n$ is differentiable near a, and let $A = DF(a)$. If A is invertible, then near a the map F should behave like the invertible linear map

$$F(a) + A(x - a).$$

The clean way to make this precise is to solve $F(x) = y$ by fixed-point iteration. Rearranging the equation gives

$$x = x - A^{-1}(F(x) - y).$$

At $x = a$, the derivative of the right-hand side with respect to x is

$$I - A^{-1}DF(a) = 0.$$

Thus, sufficiently close to a, this map is a strong contraction.

Theorem 10 (Inverse Function Theorem)**.** *Let $U \subseteq \mathbb{R}^n$ be open and let $F : U \to \mathbb{R}^n$ be continuously differentiable. Suppose $a \in U$ and $DF(a)$ is invertible.*

Then there are open neighborhoods V of a and W of $F(a)$ such that

$$F : V \to W$$

is a bijection whose inverse $G : W \to V$ is continuously differentiable.

Moreover,

$$\boxed{DG(y) = DF(G(y))^{-1}.}$$

In particular,

$$DG(F(a)) = DF(a)^{-1}.$$

Proof. Set $b = F(a)$ and $A = DF(a)$. For each y near b, define

$$T_y(x) = x - A^{-1}(F(x) - y).$$

A point x is fixed by T_y exactly when $F(x) = y$.

The derivative with respect to x is

$$DT_y(x) = I - A^{-1}DF(x).$$

At $x = a$ this is 0. Since DF is continuous, we may choose a closed ball $\overline{B}(a, r) \subseteq U$ and some $q < 1$ such that

$$\|I - A^{-1}DF(x)\| \leq q \qquad (x \in \overline{B}(a, r)).$$

By the mean value inequality,

$$\|T_y(x) - T_y(x')\| \le q\|x - x'\|$$

throughout this ball.

We next ensure that T_y maps the ball into itself. Since

$$T_y(a) = a + A^{-1}(y - b),$$

for $x \in \overline{B}(a, r)$,

$$|T_y(x) - a| \le |T_y(x) - T_y(a)| + |T_y(a) - a| \le qr + |A^{-1}(y - b)|.$$

Thus if y is close enough to b that

$$\|A^{-1}(y - b)\| < (1 - q)r,$$

then T_y maps $\overline{B}(a, r)$ into itself.

Let

$$W = \{y : \|A^{-1}(y - b)\| < (1 - q)r\}.$$

For every $y \in W$, Banach's theorem gives a unique $x \in \overline{B}(a, r)$ satisfying $F(x) = y$. In fact the strict inequality above shows $x \in B(a, r)$. Define this unique solution to be $G(y)$.

Let

$$V = B(a, r) \cap F^{-1}(W).$$

Then V and W are open, and $F : V \to W$ is bijective with inverse G.

It remains to prove that G is continuously differentiable.

First observe that G is locally Lipschitz. Define $\Psi(x) = x - A^{-1}F(x)$. The derivative bound above says that Ψ is q-Lipschitz on the ball. If $x = G(y)$ and $x' = G(y')$, then

$$x = \Psi(x) + A^{-1}y, \qquad x' = \Psi(x') + A^{-1}y'.$$

Therefore

$$\|x - x'\| \le q\|x - x'\| + \|A^{-1}\|\,\|y - y'\|,$$

so

$$\|G(y) - G(y')\| \le \frac{\|A^{-1}\|}{1 - q}\|y - y'\|.$$

Now fix $y \in W$, put $x = G(y)$, and let

$$G(y + h) = x + k.$$

The Lipschitz estimate gives $k = O(\|h\|)$. Since $F(x + k) = y + h$ and $F(x) = y$,

$$h = F(x + k) - F(x) = DF(x)k + o(\|k\|) = DF(x)k + o(\|h\|).$$

The matrix $DF(x)$ is invertible: indeed $\|I - A^{-1}DF(x)\| \le q < 1$, so $A^{-1}DF(x)$ is invertible by the geometric-series argument. Hence

$$k = DF(x)^{-1}h + o(\|h\|).$$

Thus G is differentiable and

$$DG(y) = DF(G(y))^{-1}.$$

Continuity follows from the continuity of DF and of matrix inversion on the invertible matrices. $\square$

2.8 Implicit Function Theorem

The implicit function theorem says that if a system of equations can be uniquely solved for some variables at the linearized level, then it can also be solved for those variables locally in the nonlinear system.

Theorem 11 (Implicit Function Theorem)**.** *Let*

$$F : U \subseteq \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^m$$

be continuously differentiable, and suppose

$$F(a, b) = 0.$$

If the $m \times m$ matrix

$$D_y F(a, b)$$

is invertible, then there are neighborhoods X of a and Y of b, and a unique continuously differentiable function

$$g : X \to Y$$

such that

$$F(x, g(x)) = 0$$

for every $x \in X$.
Moreover,

$$\boxed{Dg(x) = -D_y F(x, g(x))^{-1} D_x F(x, g(x)).}$$

Proof. The theorem is an immediate application of the inverse function theorem. Define

$$H(x, y) = (x, F(x, y)).$$

Its derivative at (a, b) is the block matrix

$$DH(a, b) = \begin{pmatrix} I & 0 \\ D_x F(a, b) & D_y F(a, b) \end{pmatrix}.$$

This is invertible because $D_y F(a, b)$ is invertible.
By the inverse function theorem, H has a continuously differentiable local inverse near

$$H(a, b) = (a, 0).$$

Since the first component of $H(x, y)$ is exactly x, its inverse must have the form

$$H^{-1}(x, z) = (x, \psi(x, z)).$$

Set

$$g(x) = \psi(x, 0).$$

Then

$$H(x, g(x)) = (x, 0),$$

which means exactly

$$F(x, g(x)) = 0.$$

Local uniqueness follows from the local injectivity of H.
To compute the derivative, differentiate

$$F(x, g(x)) = 0.$$

The chain rule gives

$$D_x F(x, g(x)) + D_y F(x, g(x)) Dg(x) = 0.$$

Solving for $Dg(x)$ yields

$$Dg(x) = -D_y F(x, g(x))^{-1} D_x F(x, g(x)).$$

□

2.9 Lagrange Multipliers

Suppose we want to optimize $f : \mathbb{R}^n \to \mathbb{R}$ subject to m continuously differentiable equality constraints

$$g_1(x) = \cdots = g_m(x) = 0.$$

Write

$$g = (g_1, \ldots, g_m) : \mathbb{R}^n \to \mathbb{R}^m.$$

If the constraint gradients are linearly independent, the implicit function theorem says that the feasible set is locally an $(n - m)$-dimensional surface. At a constrained extremum, f cannot change to first order in any tangent direction to this surface. Its gradient must therefore point entirely in the normal directions, which are exactly the directions spanned by the constraint gradients.

Theorem 12 (Lagrange Multipliers)**.** *Let $f, g_1, \ldots, g_m$ be continuously differentiable near $a \in \mathbb{R}^n$, and suppose*

$$g_1(a) = \cdots = g_m(a) = 0.$$

Assume that

$$\nabla g_1(a), \ldots, \nabla g_m(a)$$

are linearly independent.

If a is a local maximum or minimum of f subject to the constraints $g_1 = \cdots = g_m = 0$, then there are unique scalars $\lambda_1, \ldots, \lambda_m$ such that

$$\boxed{\nabla f(a) = \sum_{i=1}^{m} \lambda_i \nabla g_i(a).}$$

Proof. Since the constraint gradients are linearly independent, the Jacobian $Dg(a)$ has rank m. After permuting the coordinates, write

$$x = (u, v), \qquad u \in \mathbb{R}^{n-m}, \qquad v \in \mathbb{R}^m,$$

so that the square matrix $D_v g(a)$ is invertible.
Write $a = (u_0, v_0)$. By the implicit function theorem, all nearby feasible points are uniquely of the form

$$(u, \phi(u))$$

for a continuously differentiable function ϕ, with

$$g(u, \phi(u)) = 0.$$

Differentiating this identity at u_0 gives

$$D_u g(a) + D_v g(a) D\phi(u_0) = 0,$$

and hence

$$D\phi(u_0) = -D_v g(a)^{-1} D_u g(a).$$

Now define the restriction of f to the feasible surface by

$$h(u) = f(u, \phi(u)).$$

Since a is a constrained local extremum, u_0 is an ordinary local extremum of h. Therefore $Dh(u_0) = 0$. By the chain rule,

$$D_u f(a) + D_v f(a) D\phi(u_0) = 0.$$

Substituting the formula for $D\phi(u_0)$,

$$D_u f(a) = D_v f(a) D_v g(a)^{-1} D_u g(a).$$

Define the row vector

$$\lambda^T = D_v f(a) D_v g(a)^{-1}.$$

Then

$$D_u f(a) = \lambda^T D_u g(a), \qquad D_v f(a) = \lambda^T D_v g(a).$$

Together these say

$$Df(a) = \lambda^T Dg(a).$$

Writing this in terms of gradients gives

$$\nabla f(a) = \sum_{i=1}^{m} \lambda_i \nabla g_i(a).$$

The multipliers are unique because the constraint gradients are linearly independent. □

2.10 The Isoperimetric Inequality

The isoperimetric inequality says that among all simple closed curves of a given length, the circle encloses the largest area:

$$\boxed{4\pi A \leq L^2.}$$

There are two ingredients.

First, the area enclosed by an oriented curve can be written as

$$2A = \int (x\,dy - y\,dx).$$

For polygons, this is just the shoelace formula, and the shoelace formula itself follows immediately by writing the area of each triangle as a determinant and observing that internal edges cancel.

Second, if a periodic function has mean zero, then it cannot be large without changing substantially:

$$\int_0^{2\pi} f^2 \le \int_0^{2\pi} (f')^2.$$

This is the periodic Wirtinger inequality.

Lemma 13 (Shoelace Formula)**.** *Let*

$$p_i = (x_i, y_i), \qquad i = 1, \dots, n,$$

be the vertices of a simple polygon P, listed counterclockwise, with $p_{n+1} = p_1$. Then

$$\boxed{\operatorname{area}(P) = \frac{1}{2}\sum_{i=1}^{n} \det(p_i, p_{i+1}) = \frac{1}{2}\sum_{i=1}^{n} (x_i y_{i+1} - y_i x_{i+1}).}$$

Proof. For three points a, b, c, the signed area of the oriented triangle abc is

$$[\triangle abc] = \frac{1}{2}\det(b - a, c - a).$$

Expanding the determinant gives

$$\begin{aligned}\det(b - a, c - a) &= \det(b, c) - \det(b, a) - \det(a, c) \\ &= \det(a, b) + \det(b, c) + \det(c, a).\end{aligned}$$

Thus

$$\boxed{2[\triangle abc] = \det(a, b) + \det(b, c) + \det(c, a).}$$

In other words, twice the signed area of a triangle is the sum of $\det(u, v)$ over its three oriented boundary edges $u \to v$.

Now triangulate P, and orient every triangle counterclockwise. Since the triangles have disjoint interiors,

$$\operatorname{area}(P) = \sum_{\triangle \in T} \operatorname{area}(\triangle).$$

Hence

$$2\operatorname{area}(P) = \sum_{\triangle \in T} \sum_{(u,v) \in \partial \triangle} \det(u, v).$$

Every internal edge uv occurs in exactly two triangles. One triangle traverses it as $u \to v$, while the other traverses it as $v \to u$, so their contributions cancel:

$$\det(u, v) + \det(v, u) = 0.$$

Thus only the boundary edges remain. These are

$$p_1 \to p_2, \quad p_2 \to p_3, \quad \dots, \quad p_n \to p_1,$$

and therefore

$$2\,\mathrm{area}(P) = \sum_{i=1}^{n} \det(p_i, p_{i+1}).$$

□

Lemma 14 (Signed Area Formula)**.** *Let*

$$\gamma(t) = (x(t), y(t)), \qquad a \le t \le b,$$

be a counterclockwise simple closed piecewise C^1 curve enclosing a region of area A. Then

$$\boxed{2A = \int_a^b \big(x(t)y'(t) - y(t)x'(t)\big)\,dt.}$$

Proof. First suppose that γ is polygonal, with consecutive vertices $p_1, \ldots, p_n$. On an edge from p to q, parameterized by

$$\gamma(t) = (1-t)p + tq, \qquad 0 \le t \le 1,$$

we have

$$\gamma'(t) = q - p,$$

and hence

$$\det(\gamma(t), \gamma'(t)) = \det(p, q).$$

Therefore

$$\int_{\text{edge}} (x\,dy - y\,dx) = \det(p, q).$$

Summing over all edges and applying Lemma 13 gives

$$\int_\gamma (x\,dy - y\,dx) = 2A.$$

For a general piecewise C^1 simple closed curve, choose increasingly fine inscribed polygonal approximations P_k. Since γ is piecewise C^1, the polygonal parameterizations converge uniformly to γ, while their derivatives converge to γ' in L^1 on each C^1 piece. Consequently,

$$\int_{\partial P_k} (x\,dy - y\,dx) \longrightarrow \int_\gamma (x\,dy - y\,dx).$$

The enclosed polygonal regions also converge to the region enclosed by γ, so

$$\mathrm{area}(P_k) \longrightarrow A.$$

Applying the polygonal formula and taking limits gives

$$2A = \int_\gamma (x\,dy - y\,dx).$$

□

We next prove the analytic ingredient. We begin with the version on an interval. Minimizing

$$\lambda \int_0^\pi (u')^2 - \int_0^\pi u^2$$

subject to

$$u(0) = u(\pi) = 0,$$

we guess that $u'u'' \propto uu'$, so $u = C \sin t$.
We then factor the extremizer out:

$$u(t) = v(t) \sin t.$$

The gap in the inequality will measure exactly how far v is from being constant.

Lemma 15 (Dirichlet Wirtinger Inequality)**.** *Let*

$$u : [0, \pi] \to \mathbb{R}$$

be continuously differentiable with

$$u(0) = u(\pi) = 0.$$

Then

$$\boxed{\int_0^\pi u(t)^2 \, dt \leq \int_0^\pi u'(t)^2 \, dt.}$$

Equality holds exactly for

$$u(t) = C \sin t.$$

Proof. For $0 < t < \pi$, write

$$u(t) = v(t) \sin t.$$

Then

$$u' = v' \sin t + v \cos t.$$

Consequently,

$$(u')^2 - u^2 = (v')^2 \sin^2 t + 2vv' \sin t \cos t + v^2(\cos^2 t - \sin^2 t).$$

The last two terms form a derivative:

$$\frac{d}{dt}\big(v^2 \sin t \cos t\big) = 2vv' \sin t \cos t + v^2(\cos^2 t - \sin^2 t).$$

Thus

$$\boxed{(u')^2 - u^2 = (v')^2 \sin^2 t + \frac{d}{dt}\big(v^2 \sin t \cos t\big).}$$

Integrating first over

$$[\varepsilon, \pi - \varepsilon]$$

gives

$$\int_{\varepsilon}^{\pi-\varepsilon} \left((u')^2 - u^2\right) dt = \int_{\varepsilon}^{\pi-\varepsilon} (v')^2 \sin^2 t \, dt + \left[v^2 \sin t \cos t\right]_{\varepsilon}^{\pi-\varepsilon}.$$

Since u is continuously differentiable and vanishes at the endpoints,

$$u(t) = O(t) \qquad (t \to 0)$$

and

$$u(t) = O(\pi - t) \qquad (t \to \pi).$$

Since $\sin t$ has the same behavior, $v = u/\sin t$ remains bounded near both endpoints. Hence

$$v(t)^2 \sin t \cos t \to 0$$

at both endpoints.
Letting $\varepsilon \to 0$ therefore gives

$$\boxed{\int_0^{\pi} \left((u')^2 - u^2\right) dt = \int_0^{\pi} \sin^2 t \left(\frac{d}{dt}\frac{u(t)}{\sin t}\right)^2 dt \geq 0.}$$

This proves the inequality.
Equality holds precisely when

$$v' = 0,$$

so v is constant and

$$u(t) = C \sin t.$$

□

Lemma 16 (Mean-Zero Wirtinger Inequality)**.** *Let*

$$f : [0, \pi] \to \mathbb{R}$$

be continuously differentiable and suppose

$$\int_0^{\pi} f(t) \, dt = 0.$$

Then

$$\boxed{\int_0^{\pi} f(t)^2 \, dt \leq \int_0^{\pi} f'(t)^2 \, dt.}$$

Proof. Define

$$F(x) = \int_0^{x} f(t) \, dt.$$

Then

$$F(0) = 0,$$

while the mean-zero assumption gives

$$F(\pi) = 0.$$

Thus Lemma 15 gives

$$\int_0^\pi F^2 \le \int_0^\pi (F')^2 = \int_0^\pi f^2.$$

Since $F' = f$, integration by parts gives

$$\begin{aligned}\int_0^\pi f^2 &= \int_0^\pi F' f \\ &= [Ff]_0^\pi - \int_0^\pi F f' \\ &= -\int_0^\pi F f'.\end{aligned}$$

Hence Cauchy–Schwarz gives

$$\int_0^\pi f^2 \le \left(\int_0^\pi F^2\right)^{1/2} \left(\int_0^\pi (f')^2\right)^{1/2}.$$

Using

$$\int_0^\pi F^2 \le \int_0^\pi f^2,$$

we obtain

$$\int_0^\pi f^2 \le \left(\int_0^\pi f^2\right)^{1/2} \left(\int_0^\pi (f')^2\right)^{1/2}.$$

Canceling the first factor proves

$$\int_0^\pi f^2 \le \int_0^\pi (f')^2.$$

□

We now pass from an interval to a circle.

The only issue is that the preceding lemma requires an interval of length π on which the integral is zero. If a function has mean zero on the whole circle, such a semicircle always exists.

Indeed, slide a semicircle around the circle and measure the integral of f over it. Rotating the semicircle by π replaces it by its complement, so its integral changes sign. Somewhere in between, it must be zero.

Lemma 17 (Periodic Wirtinger Inequality)**.** *Let*

$$f : \mathbb{R} \to \mathbb{R}$$

be continuously differentiable and 2π-periodic. If

$$\int_0^{2\pi} f(t)\,dt = 0,$$

then

$$\boxed{\int_0^{2\pi} f(t)^2\,dt \le \int_0^{2\pi} f'(t)^2\,dt.}$$

Proof. For $a \in \mathbb{R}$, define

$$H(a) = \int_a^{a+\pi} f(t)\,dt.$$

The function H is continuous.
The intervals

$$[a, a+\pi] \qquad \text{and} \qquad [a+\pi, a+2\pi]$$

together form one full period, so

$$H(a) + H(a+\pi) = \int_a^{a+2\pi} f(t)\,dt = 0.$$

Therefore

$$H(a+\pi) = -H(a).$$

If $H(a) = 0$ for some a, we already have the desired cut. Otherwise, $H(a)$ and $H(a+\pi)$ have opposite signs. By continuity, there is some $b \in [a, a+\pi]$ such that

$$H(b) = 0.$$

Thus

$$\int_b^{b+\pi} f(t)\,dt = 0.$$

Since the integral over the entire period is zero,

$$\int_{b+\pi}^{b+2\pi} f(t)\,dt = 0$$

as well.
Apply Lemma 16 to the first semicircle:

$$\int_b^{b+\pi} f^2 \le \int_b^{b+\pi} (f')^2.$$

Apply it again to the second:

$$\int_{b+\pi}^{b+2\pi} f^2 \le \int_{b+\pi}^{b+2\pi} (f')^2.$$

Adding gives

$$\int_b^{b+2\pi} f^2 \le \int_b^{b+2\pi} (f')^2.$$

By periodicity,

$$\int_0^{2\pi} f^2 \le \int_0^{2\pi} (f')^2.$$

□

Applying Lemma 17 separately to the two coordinates gives the vector form: if

$$\gamma : [0, 2\pi] \to \mathbb{R}^2$$

is periodic and

$$\int_0^{2\pi} \gamma(t)\, dt = 0,$$

then

$$\boxed{\int_0^{2\pi} |\gamma(t)|^2\, dt \le \int_0^{2\pi} |\gamma'(t)|^2\, dt.}$$

Theorem 18 (Isoperimetric Inequality)**.** *Let Γ be a simple closed piecewise C^1 curve of length L, enclosing area A. Then*

$$\boxed{4\pi A \le L^2.}$$

Equality holds exactly for circles.

Proof. Parameterize Γ counterclockwise at constant speed on $[0, 2\pi]$:

$$\gamma(t) = (x(t), y(t)), \qquad |\gamma'(t)| = \frac{L}{2\pi}$$

wherever γ' exists.
Translation does not change length or area, so translate the curve so that

$$\int_0^{2\pi} \gamma(t)\, dt = 0.$$

By Lemma 14,

$$2A = \int_0^{2\pi} \det(\gamma(t), \gamma'(t))\, dt.$$

Using

$$|\det(u, v)| \le |u||v|$$

and then Cauchy–Schwarz,

$$\begin{aligned} 2A &\le \int_0^{2\pi} |\gamma(t)|\, |\gamma'(t)|\, dt \\ &\le \left(\int_0^{2\pi} |\gamma(t)|^2\, dt \right)^{1/2} \left(\int_0^{2\pi} |\gamma'(t)|^2\, dt \right)^{1/2}. \end{aligned}$$

Periodic Wirtinger gives

$$\int_0^{2\pi} |\gamma(t)|^2\, dt \le \int_0^{2\pi} |\gamma'(t)|^2\, dt,$$

so

$$2A \le \int_0^{2\pi} |\gamma'(t)|^2\, dt.$$

Since the speed is constant,

$$\int_0^{2\pi} |\gamma'(t)|^2 \, dt = 2\pi \left(\frac{L}{2\pi}\right)^2 = \frac{L^2}{2\pi}.$$

Therefore

$$2A \le \frac{L^2}{2\pi},$$

or

$$\boxed{4\pi A \le L^2.}$$

Suppose equality holds. Equality in periodic Wirtinger implies

$$\gamma(t) = p\cos t + q\sin t$$

after the translation, for some fixed vectors $p, q \in \mathbb{R}^2$.
Since γ has constant speed,

$$|\gamma'(t)|^2 = |p|^2 \sin^2 t + |q|^2 \cos^2 t - 2\langle p, q\rangle \sin t \cos t$$

is independent of t. Hence

$$|p| = |q|, \qquad \langle p, q\rangle = 0.$$

Thus p and q are perpendicular vectors of equal length, and

$$\gamma(t) = p\cos t + q\sin t$$

parameterizes a circle.
Conversely, a circle of radius r satisfies

$$L = 2\pi r, \qquad A = \pi r^2,$$

so equality holds. □

3 Linear Algebra

3.1 Determinant and Inverse Identities

Lemma 19 (Cauchy–Binet Formula)**.** *Let A be an $m \times n$ matrix and B an $n \times m$ matrix over a field, with $m \le n$. Then*

$$\det(AB) = \sum_{S \in \binom{[n]}{m}} \det(A_{[m],S}) \det(B_{S,[m]}).$$

Proof. Set $P(x) = \det(A \operatorname{diag}(x_1, \ldots, x_n) B)$, which is homogeneous of degree m. For each i, row additions make column i of A supported in at most one row, without changing P. Then x_i occurs in at most one row of the product, so P is affine in x_i.

Thus every monomial of P uses exactly m distinct variables. Writing $\mathbf{1}_S$ for the indicator vector of S, we obtain

$$\det(AB) = P(\mathbf{1}) = \sum_{S \in \binom{[n]}{m}} P(\mathbf{1}_S) = \sum_{S \in \binom{[n]}{m}} \det(A_{[m],S} B_{S,[m]}) = \sum_{S \in \binom{[n]}{m}} \det(A_{[m],S}) \det(B_{S,[m]}).$$

□

Lemma 20.

$$\det{}'(I) = \mathrm{Tr}$$

Proof. We have:

$$\det{}'(I)(T) = \nabla_T \det(I) = \lim_{\varepsilon \to 0} \frac{\det(I + \varepsilon T) - \det(I)}{\varepsilon}$$

$\det(I + \varepsilon T)$ is a polynomial in ε of order n. The constant term is 1, while the linear term is $\mathrm{Tr}(T)$. □

Lemma 21. *For an invertible matrix A, we have:*

$$\det{}'(A)(T) = \det(A)\,\mathrm{Tr}(A^{-1}T)$$

Proof. Consider the following function of X:

$$\det(X) = \det(AA^{-1}X) = \det(A)\det(A^{-1}X)$$

Using Lemma 20:

$$\det{}'(A)(T) = \det(A)\det{}'(I)(A^{-1}T) = \det(A)\,\mathrm{Tr}(A^{-1}T)$$

□

Lemma 22 (Jacobi's Formula)**.**

$$\frac{d}{dt}\det A = \mathrm{Tr}\left(\mathrm{adj}(A)\frac{dA}{dt}\right)$$

Proof. Let $A(t)$ be a differentiable matrix function. We introduce a variable x and consider $A_x(t) = A(t) - xI$, $A_x(t)$ is invertible for infinitely many values of x.

For all x where $A_x(t)$ is invertible, we can apply Lemma 21 with $T = \frac{dA_x}{dt} = \frac{dA}{dt}$:

$$\frac{d}{dt}\det(A_x) = \det(A_x)\,\mathrm{Tr}\left(A_x^{-1}\frac{dA}{dt}\right)$$

Using $\det(A_x)A_x^{-1} = \mathrm{adj}(A_x)$, this becomes:

$$\frac{d}{dt}\det(A(t) - xI) = \mathrm{Tr}\left(\mathrm{adj}(A(t) - xI)\frac{dA}{dt}\right)$$

Since they agree for infinitely many x, they agree as polynomials, substituting $x = 0$ we have:

$$\frac{d}{dt}\det A = \mathrm{Tr}\left(\mathrm{adj}(A)\frac{dA}{dt}\right)$$

□

Lemma 23. *Let A be an $n \times n$ matrix. For any subset of indices $S \subseteq \{1, 2, \ldots, n\}$, let A_S denote the $(n - |S|) \times (n - |S|)$ principal submatrix. Then:*

$$\frac{p_A^{(k)}(x)}{k!} = \sum_{|S|=k} p_{A_S}(x)$$

Proof. We evaluate $p_A(x + y)$ in two different ways.

We can consider the Taylor expansion of $p_A(x + y)$ around x:

$$p_A(x + y) = \sum_{k=0}^{n} \frac{p_A^{(k)}(x)}{k!} y^k$$

Second:

$$p_A(x+y) = \det\big((x+y)I - A\big) = \det\big(yI + (xI - A)\big)$$

The coefficient y here is the sum of the determinant of all $(n-k)\times(n-k)$ sized minors $\det(xI_{n-k} - A_S) = p_{A_S}(x)$.

Equating the coefficients of y^k yields:

$$\frac{p_A^{(k)}(x)}{k!} = \sum_{|S|=k} p_{A_S}(x)$$

□

3.1.1 Woodbury Identity

The Woodbury identity is most naturally viewed as an identity between the two Schur complements of the same block matrix.

Recall that for

$$M = \begin{pmatrix} A & B \\ C & D \end{pmatrix},$$

if A is invertible, the Schur complement of A is

$$M/A = D - CA^{-1}B.$$

The block factorization

$$M = \begin{pmatrix} I & 0 \\ CA^{-1} & I \end{pmatrix} \begin{pmatrix} A & 0 \\ 0 & M/A \end{pmatrix} \begin{pmatrix} I & A^{-1}B \\ 0 & I \end{pmatrix}$$

immediately gives

$$\det M = \det(A)\det(M/A).$$

If M/A is also invertible, inverting the factorization shows that the upper-left block of M^{-1} is

$$\boxed{(M^{-1})_{11} = A^{-1} + A^{-1}B(M/A)^{-1}CA^{-1}.}$$

Similarly, if D is invertible, then

$$M/D = A - BD^{-1}C,$$

and eliminating the D-block first shows that

$$\boxed{(M^{-1})_{11} = (M/D)^{-1}.}$$

These two descriptions of the same block give Woodbury immediately.

Theorem 24 (Woodbury Identity)**.** *Let A and C be invertible, and suppose*

$$C^{-1} + VA^{-1}U$$

is invertible. Then

$$\boxed{(A+UCV)^{-1} = A^{-1} - A^{-1}U\left(C^{-1} + VA^{-1}U\right)^{-1}VA^{-1}.}$$

Proof. Consider

$$M = \begin{pmatrix} A & U \\ -V & C^{-1} \end{pmatrix}.$$

Taking the Schur complement of A gives

$$M/A = C^{-1} + VA^{-1}U.$$

Hence the upper-left block of M^{-1} is

$$(M^{-1})_{11} = A^{-1} - A^{-1}U\left(C^{-1} + VA^{-1}U\right)^{-1}VA^{-1}.$$

On the other hand, taking the Schur complement of C^{-1} gives

$$M/C^{-1} = A - UC(-V) = A + UCV.$$

Therefore

$$(M^{-1})_{11} = (M/C^{-1})^{-1} = (A + UCV)^{-1}.$$

Equating the two expressions for the same block of M^{-1} gives the identity. □

Thus the identity can be remembered from the block matrix

$$\boxed{\begin{pmatrix} A & U \\ -V & C^{-1} \end{pmatrix},}$$

whose two Schur complements are

$$\boxed{M/A = C^{-1} + VA^{-1}U, \qquad M/C^{-1} = A + UCV.}$$

3.1.2 Matrix Determinant Lemma

The determinant analogue of Woodbury follows from exactly the same block matrix.

Theorem 25 (Matrix Determinant Lemma)**.** *Let*

$$A \in \mathbb{F}^{n\times n}, \qquad C \in \mathbb{F}^{k\times k}$$

be invertible, and let

$$U \in \mathbb{F}^{n\times k}, \qquad V \in \mathbb{F}^{k\times n}.$$

Then

$$\boxed{\det(A + UCV) = \det(A)\det(C)\det\left(C^{-1} + VA^{-1}U\right).}$$

Proof. Again take

$$M = \begin{pmatrix} A & U \\ -V & C^{-1} \end{pmatrix}.$$

Computing its determinant using the Schur complement of A,

$$\det M = \det(A)\det(M/A) = \det(A)\det\left(C^{-1} + VA^{-1}U\right).$$

Computing it instead using the Schur complement of C^{-1},

$$\det M = \det(C^{-1}) \det(M/C^{-1}) = \det(C)^{-1} \det(A + UCV).$$

Equating the two expressions and multiplying by $\det(C)$ gives

$$\det(A + UCV) = \det(A) \det(C) \det\left(C^{-1} + VA^{-1}U\right).$$

□

For $C = I$, this becomes

$$\boxed{\det(A + UV) = \det(A) \det(I + VA^{-1}U).}$$

Thus Woodbury and the matrix determinant lemma are the inverse and determinant consequences of the same observation:

$$\boxed{M/A = C^{-1} + VA^{-1}U, \qquad M/C^{-1} = A + UCV.}$$

The large $n \times n$ update $A+UCV$ and the small $k \times k$ matrix $C^{-1}+VA^{-1}U$ are the two Schur complements of one block matrix.

The following is the positive-semidefiniteness version of the determinant lemma.

Lemma 26 (Schur Complement Positivity)**.** *For $B \succ 0$, $\begin{pmatrix} A & X \\ X^* & B \end{pmatrix} \succeq 0 \iff A \succeq XB^{-1}X^*$.*

Proof. Conjugating a block matrix by an invertible matrix preserves positive semidefiniteness. We calculate:

$$\begin{pmatrix} I & -XB^{-1} \\ 0 & I \end{pmatrix} \begin{pmatrix} A & X \\ X^* & B \end{pmatrix} \begin{pmatrix} I & 0 \\ -B^{-1}X^* & I \end{pmatrix} = \begin{pmatrix} A - XB^{-1}X^* & 0 \\ 0 & B \end{pmatrix}$$

Because $B \succ 0$, the block diagonal matrix on the right is positive semidefinite if and only if $A - XB^{-1}X^* \succeq 0$. □

3.2 Characteristic Polynomial

To compute the characteristic polynomial $p(x) = \det(xI - A)$ of an $n \times n$ matrix A, the algorithm first reduces A to an upper Hessenberg matrix H (where $H_{i,j} = 0$ for all $i > j + 1$) in $O(n^3)$ time. It then evaluates the characteristic polynomial using a dynamic programming approach.

Lemma 27 (Hessenberg Characteristic Polynomial Recurrence)**.** *Let H_k be the $k \times k$ leading principal submatrix of an upper Hessenberg matrix H, and let $p_k(x) = \det(xI_k - H_k)$ be its characteristic polynomial. With the base case $p_0(x) = 1$, the polynomials satisfy the recurrence:*

$$p_{k+1}(x) = (x - H_{k,k})p_k(x) - \sum_{i=0}^{k-1} \left(H_{i,k} \prod_{j=i}^{k-1} H_{j+1,j} \right) p_i(x)$$

Proof. We evaluate $p_{k+1}(x) = \det(xI_{k+1} - H_{k+1})$ by performing a Laplace expansion along its last column. The entries in the last column of $xI_{k+1} - H_{k+1}$ are $-H_{0,k}, -H_{1,k}, \ldots, -H_{k-1,k}$, with the diagonal entry being $(x - H_{k,k})$.

The cofactor for the diagonal entry $(x - H_{k,k})$ is the determinant of the top-left $k \times k$ block, which is $p_k(x)$.

For any other entry $-H_{i,k}$ (where $i < k$) in the last column, removing its row and column leaves a block upper-triangular matrix. The top-left block is $(xI_i - H_i)$, which has determinant $p_i(x)$. Because H is upper Hessenberg, the bottom-right block is upper triangular, with its main diagonal consisting entirely of the subdiagonal elements of H: $-H_{i+1,i}, -H_{i+2,i+1}, \ldots, -H_{k,k-1}$. The alternating signs from the cofactor expansion cancels with the negative signs of these subdiagonal elements. □

3.2.1 Determinant of Affine Matrix Polynomial

The algorithm computes the polynomial $P(x) = \det(A + xB)$ in $O(n^3)$ time.

The algorithm samples a random scalar $a \in \mathbb{F}$ and evaluates the matrix at this point, defining $A' = A + aB$. Assuming A' is invertible, we can factor it out of the original polynomial:

$$\begin{aligned} P(x) &= \det(A + aB + (x - a)B) \\ &= \det(A' + (x - a)B) \\ &= \det(A') \det(I + (x - a)A'^{-1}B) \end{aligned}$$

Let $M = -A'^{-1}B$. The polynomial becomes $P(x) = \det(A') \det(I - (x - a)M)$.

Notice that $\det(I - tM) = t^n \det(t^{-1}I - M) = t^n p_M(t^{-1})$, the reversed characteristic polynomial of M. Let $R(t)$ denote this reversed polynomial. The algorithm computes $p_M(t)$ using the Hessenberg DP, reverses its coefficients to obtain $R(t)$, multiplies the polynomial by the scalar $\det(A')$, and performs a Taylor shift $x \mapsto x - a$ to recover $P(x) = \det(A')R(x - a)$.

Lemma 28 (Failure Probability of the Affine Determinant)**.** *If $P(x) = \det(A + xB)$ is not identically the zero polynomial, the probability that the randomized reduction algorithm incorrectly fails (by returning 0) is bounded by $\frac{n}{|\mathbb{F}|}$.*

Proof. The algorithm only fails if the shifted matrix $A' = A + aB$ is not invertible, if and only if $P(a) = 0$.

The algorithm samples a uniformly and out of all possible random choices for a, at most n of them can cause A' to be singular. If $P(x)$ is identically the zero polynomial, the algorithm will correctly find $\det(A') = 0$ for any a. □

3.3 Frobenius Form

3.3.1 Algorithm Description

To compute the Frobenius form of a matrix $A \in \mathbb{F}^{n \times n}$ in $O(n^3)$ time, the algorithm employs a randomized Krylov subspace approach. The Frobenius form decomposes the vector space V into a direct sum of A-cyclic subspaces $V = \bigoplus_{i=1}^{m} W_i$. This yields a block-diagonal matrix where each block is the companion matrix of an invariant factor $P_i(z)$ of A. By the structure theorem of finitely generated modules over a PID, these invariant factors divide each other: $P_m(z) \mid P_{m-1}(z) \mid \cdots \mid P_1(z)$.

1. **State:** Maintain a set of generator vectors $\{w_1, \ldots, w_k\}$ for the resolved cyclic subspaces $W_1, \ldots, W_k$, and a row-reduced basis $\mathcal{B}$ representing their combined span $W = \bigoplus_{i=1}^{k} W_i$.

2. **Random Sampling:** Draw a uniformly random vector $x \in \mathbb{F}^n$ to act as the candidate generator for the next cyclic block, W_{k+1}.

3. **Krylov Sequence:** Generate the Krylov sequence $x, Ax, A^2x, \ldots$ and incrementally reduce each new vector against the existing basis $\mathcal{B}$ using Gaussian elimination. This process halts when a linear dependence is found, yielding the minimal annihilating polynomial $C(z)$ of x modulo the previously resolved space W.

4. **Orthogonalization:** The linear dependence indicates that $C(A)x \in W$. To ensure the newly found cyclic subspace forms a direct summand, x must be updated to $\tilde{x} = x - u$ (for some $u \in W$) such that $C(A)\tilde{x} = 0$. The algorithm extracts the necessary shift by performing a polynomial division over the concatenated dependency arrays, and backtracks to apply it to x. It then recomputes the Krylov sequence.

5. **Iteration:** The vectors $\tilde{x}, A\tilde{x}, \ldots, A^{\deg(C)-1}\tilde{x}$ are added to $\mathcal{B}$. This repeats until $\dim(W) = n$.

3.3.2 Correctness

Theorem 29 (Divisibility of the Residuals)**.** *Let $W = \bigoplus_{i=1}^{k} W_i$ be the resolved space, where each $W_i = \langle w_i \rangle_A$ is a cyclic submodule with minimal polynomial $P_i(z)$. Let x be the random candidate vector, and $C(z)$ be its minimal polynomial modulo W. The Krylov reduction yields a dependency of the form:*

$$C(A)x = \sum_{i=1}^{k} R^{(i)}(A)w_i$$

Then the polynomial $C(z)$ divides $R^{(i)}(z)$ in $\mathbb{F}[z]$ for all $1 \leq i \leq k$.

Proof. As shown in the next section, with good probability, the minimal polynomial of the newly sampled block must divide the minimal polynomials of all previously extracted blocks: $C(z) \mid P_i(z)$ for all $i \leq k$.

Because the true vector space decomposes into a direct sum of cyclic submodules, we can uniquely decompose our random candidate as $x = \tilde{x} + \sum_{i=1}^{k} v_i$, where $\tilde{x}$ belongs to the target complementary subspace (meaning it is exactly annihilated by $C(A)$) and $v_i \in W_i$.

Applying $C(A)$ to this decomposition yields $C(A)x = C(A)\tilde{x} + \sum_{i=1}^{k} C(A)v_i = \sum_{i=1}^{k} C(A)v_i$. Since each $v_i \in \langle w_i \rangle_A$, there exist polynomials $S^{(i)}(z)$ such that $v_i = S^{(i)}(A)w_i$. Thus:

$$C(A)x = \sum_{i=1}^{k} C(A)S^{(i)}(A)w_i$$

Equating this with the relationship obtained from our Krylov reduction, we have:

$$\sum_{i=1}^{k} \big(C(A)S^{(i)}(A) - R^{(i)}(A)\big)w_i = 0$$

Because W is a direct sum, the components must independently evaluate to zero: $\big(C(A)S^{(i)}(A) - R^{(i)}(A)\big)w_i = 0$. This implies that the minimal polynomial of w_i, which is $P_i(z)$, must divide $C(z)S^{(i)}(z) - R^{(i)}(z)$.

Since we previously established that $C(z) \mid P_i(z)$, it follows by transitivity that $C(z)$ must divide $C(z)S^{(i)}(z) - R^{(i)}(z)$. Consequently, $C(z)$ must divide the residual polynomial $R^{(i)}(z)$ in $\mathbb{F}[z]$. □

3.3.3 Failure Probability

Theorem 30. *The probability that the randomized Krylov algorithm fails to extract the Frobenius form of $A \in \mathbb{F}^{n \times n}$ is bounded by $\frac{n}{|\mathbb{F}|}$.*

Proof. The algorithm fails if and only if, at some iteration, the randomly drawn vector x fails to generate the correct cyclic subspace in the quotient V/W. Let $P(z)$ be the true invariant factor of the remaining space. The vector x fails if its minimal polynomial modulo W is a strict divisor of $P(z)$.

We view V/W as a finitely generated torsion module over the principal ideal domain $\mathbb{F}[z]$, where z acts as A. The module decomposes into primary cyclic submodules:

$$V/W \cong \bigoplus_{j=1}^{\ell} \mathbb{F}[z]/\langle Q_j(z)^{e_j} \rangle$$

where $Q_j(z)$ are the distinct monic irreducible factors of $P(z)$, and e_j are their multiplicities.

A random vector $x \in V/W$ fails to have $P(z)$ as its minimal polynomial if and only if it is annihilated by $P(z)/Q_j(z)$ for at least one irreducible factor Q_j. This occurs exactly when the projection of x onto the primary component $\mathbb{F}[z]/\langle Q_j(z)^{e_j} \rangle$ is a multiple of $Q_j(z)$.

Because x is drawn uniformly at random, its projection onto any component is uniformly distributed. The submodule consisting of multiples of $Q_j(z)$ has a vector space codimension equal to $\deg(Q_j)$. Therefore, the probability that x falls into this "bad" submodule is $|\mathbb{F}|^{-\deg(Q_j)}$.

Applying the union bound over all distinct irreducible factors $Q_j \mid P(z)$, the probability that x fails in a given step is bounded by:

$$\mathbb{P}(\text{Failure at step}) \leq \sum_{Q_j|P} \frac{1}{|\mathbb{F}|^{\deg(Q_j)}}$$

Because $\sum \deg(P_i) = n$, the worst-case upper bound occurs when the characteristic polynomial splits entirely into n distinct linear factors. In this scenario:

$$\mathbb{P}(\text{Total Failure}) \leq \frac{n}{|\mathbb{F}|}$$

□

3.3.4 Polynomial Matrix Composition and Powering

With the Frobenius form $A = T^{-1}FT$ explicitly constructed, we can evaluate any polynomial function $f(A)$, such as matrix powers A^k.

Module Isomorphism on Cyclic Blocks Let $W_i = \langle w_i \rangle_A$ be a cyclic subspace of dimension d_i with minimal polynomial $P_i(z)$. The subspace is spanned by the Krylov basis $\mathcal{B}_i = \{w_i, Aw_i, \ldots, A^{d_i-1}w_i\}$.

There is a natural $\mathbb{F}[z]$-module isomorphism between W_i and the polynomial quotient ring $\mathbb{F}[z]/\langle P_i(z) \rangle$, given by the mapping:

$$A^j w_i \longleftrightarrow z^j \pmod{P_i(z)}$$

Under this isomorphism, applying the matrix function $f(A)$ to a basis vector $A^j w_i$ corresponds simply to polynomial multiplication in the quotient ring:

$$f(A)\big(A^j w_i\big) = A^j f(A) w_i \longleftrightarrow z^j f(z) \pmod{P_i(z)}$$

Constructing the Blocks To construct the matrix block $S_i = f(C_i)$ representing the action of $f(A)$ on the subspace W_i, we need to find the coordinates of $f(A)(A^j w_i)$ in the basis $\mathcal{B}_i$. By the isomorphism above, these coordinates are the coefficients of the polynomial:

$$R_{i,j}(z) = z^j f(z) \pmod{P_i(z)}$$

1. First, computes the base polynomial $R_{i,0}(z) = f(z) \pmod{P_i(z)}$.
2. This polynomial forms the 0-th row (or column depending on preference) of the block S_i.
3. For each subsequent basis vector $j \in \{1, \ldots, d_i - 1\}$, the algorithm calculates the next polynomial by shifting the previous one and reducing modulo $P_i(z)$:

   $$R_{i,j}(z) = z \cdot R_{i,j-1}(z) \pmod{P_i(z)}$$

   This populates the $d_i \times d_i$ block S_i in $O(d_i^2)$ time.

Once all blocks S_i are constructed, they are assembled into the block-diagonal matrix $S = \text{diag}(S_1, \ldots, S_m)$, and the final matrix is recovered via the similarity transform $f(A) = T^{-1}ST$.

Theorem 31 (Complexity of Matrix Exponentiation)**.** *Using the Frobenius form, the matrix power A^k can be computed in $O(n^3 + n \log n \log k)$ time.*

Proof. We compute the Frobenius form $A = T^{-1}FT$ in $O(n^3)$ time. For each invariant factor $P_i(z)$, our target function is $f(z) = z^k$. We evaluate the base polynomial $R_{i,0}(z) = z^k \pmod{P_i(z)}$ using binary exponentiation in the polynomial ring $\mathbb{F}[z]/\langle P_i(z) \rangle$ or using specialized algorithms. Because $\deg(P_i) = d_i$, each polynomial multiplication takes $O(d_i \log d_i)$ with NTT. The total time to compute $z^k \pmod{P_i(z)}$ is $O(d_i^2 \log k)$.

Summing this cost over all blocks yields $\sum_i O(d_i \log d_i \log k) \leq O(n \log n \log k)$. Constructing the blocks via shifts takes $\sum_i O(d_i^2) = O(n^2)$ time. Finally, recovering the matrix via $T^{-1}ST$ requires two $O(n^3)$ matrix multiplications. □

3.4 Black Box Linear Algebra

In the following algorithms, we consider a matrix $A \in \mathbb{F}^{n \times n}$ given strictly as a black-box function $f(v) = Av$.

3.4.1 Minimal Polynomial

Algorithm Description To compute the minimal polynomial of A, the algorithm generates two random vectors $u, v \in \mathbb{F}^n$. It then computes the scalar sequence $s_k = u^T A^k v$ for $k = 0, \ldots, 2n$. Finally, it applies an algorithm to find the minimal linear recurrence of the sequence s_k, which corresponds to the minimal polynomial of A.

Proof of Correctness Let $A \in \mathbb{F}^{n \times n}$. We define three polynomials:

1. $m_A(x)$: The minimal polynomial of the matrix A.
2. $m_{A,v}(x)$: The minimal polynomial of A with respect to the vector v, which is the monic polynomial of least degree such that $m_{A,v}(A)v = 0$.
3. $m_{u,v}(x)$: The minimal polynomial of the scalar sequence $s_k = u^T A^k v$, which is the monic polynomial of least degree such that $\sum_{i=0}^{d} c_i s_{k+i} = 0$ for all k.

Divisibility:
Because $m_A(A) = 0$, it follows that $m_A(A)v = 0$, meaning $m_{A,v}(x)$ must divide $m_A(x)$. Similarly, since $m_{A,v}(A)v = 0$, left-multiplying by $u^T A^k$ gives $u^T A^k m_{A,v}(A)v = 0$. This means the sequence s_k satisfies the recurrence defined by $m_{A,v}(x)$, so $m_{u,v}(x)$ must divide $m_{A,v}(x)$. This establishes the chain of divisibility:

$$m_{u,v}(x) \mid m_{A,v}(x) \mid m_A(x)$$

To prove $m_{u,v}(x) = m_A(x)$ with high probability, we bound the probability of a strict division occurring at either step.

Part 1: The Probability that $m_{A,v}(x) = m_A(x)$
Let $m_A(x)$ be factored into its distinct irreducible monic polynomials over $\mathbb{F}$:

$$m_A(x) = p_1(x)^{e_1} p_2(x)^{e_2} \cdots p_r(x)^{e_r}$$

where $r \leq n$. For $m_{A,v}(x)$ to be a proper divisor of $m_A(x)$, it must divide the polynomial $q_i(x) = m_A(x)/p_i(x)$ for at least one index $i \in \{1, \ldots, r\}$. This happens if and only if $q_i(A)v = 0$, which means v resides in the nullspace of the matrix $q_i(A)$.

Let $W_i = \ker(q_i(A))$. Because $m_A(x)$ is strictly the minimal polynomial of A, the matrix $q_i(A)$ cannot be the zero matrix. Therefore, W_i is a proper subspace of $\mathbb{F}^n$, meaning $\dim(W_i) \leq n - 1$. If v is chosen uniformly at random from $\mathbb{F}^n$, the probability that v falls into a specific proper subspace W_i is at most $1/|\mathbb{F}|$. By the union bound over all r irreducible factors, the probability that v falls into any of these kernels is:

$$P(m_{A,v}(x) \neq m_A(x)) = P\left(v \in \bigcup_{i=1}^{r} W_i\right) \leq \frac{r}{|\mathbb{F}|} \leq \frac{n}{|\mathbb{F}|}$$

Part 2: The Probability that $m_{u,v}(x) = m_{A,v}(x)$
Assume we have chosen a "good" vector v such that $m_{A,v}(x) = m_A(x)$. We now pick $u \in \mathbb{F}^n$ uniformly at random. By the same logic, for $m_{u,v}(x)$ to be a proper divisor of $m_A(x)$, it must divide $q_i(x) = m_A(x)/p_i(x)$ for some index i. This happens if and only if the sequence generated by $q_i(x)$ evaluates to zero for all k:

$$u^T A^k q_i(A)v = 0 \quad \text{for all } k \geq 0$$

Let $w_i = q_i(A)v$. Because we assumed $m_{A,v}(x) = m_A(x)$, we know that $w_i \neq 0$. The condition $u^T A^k w_i = 0$ for all k means that the vector u must be orthogonal to the Krylov subspace $\mathcal{K} = \text{span}\{w_i, Aw_i, A^2 w_i, \ldots\}$.

Since $w_i \neq 0$, the subspace $\mathcal{K}$ has a dimension of at least 1. Therefore, its orthogonal complement $\mathcal{K}^\perp$ is a proper subspace of $\mathbb{F}^n$ with dimension at most $n-1$. The probability that a uniformly random u falls into $\mathcal{K}^\perp$ for a specific index i is at most $1/|\mathbb{F}|$. Applying the union bound again across the r factors:

$$P(m_{u,v}(x) \neq m_A(x) \mid v \text{ is good}) \leq \frac{r}{|\mathbb{F}|} \leq \frac{n}{|\mathbb{F}|}$$

Conclusion
Combining the two failure probabilities using the union bound, the total probability that the algorithm fails to find the true minimal polynomial is bounded by:

$$P(\text{failure}) \leq \frac{2n}{|\mathbb{F}|}$$

Therefore, for a sufficiently large field (where $|\mathbb{F}| \gg 2n$), the minimal recurrence of the scalar sequence s_k is exactly $m_A(x)$ with high probability.

3.4.2 Determinant

Description To evaluate $\det(A)$ for a matrix $A \in \mathbb{F}^{n\times n}$, the algorithm generates a random diagonal matrix D (represented by its diagonal entries $c_1, \ldots, c_n$) and a random vector $v \in \mathbb{F}^n$. It defines the linear map $A' = AD$ and computes its minimal polynomial $m_{AD}(x)$ using a minimal recurrence algorithm on the Krylov sequence $v, A'v, (A')^2v, \ldots$. If the degree of $m_{AD}(x)$ is n, the determinant of A is extracted from the polynomial's constant term.

Proof of Correctness The minimal polynomial $m_{AD}(x)$ always divides the characteristic polynomial $p_{AD}(x) = \det(xI - AD)$. If A is singular, AD is singular, and $m_{AD}(x)$ will trivially have a constant term of 0, yielding a correct determinant of 0. Therefore, we assume A is non-singular. If we can prove that AD generically has distinct eigenvalues, then with high probability, its minimal polynomial will have degree n, allowing us to safely assume $m_{AD}(x) = p_{AD}(x)$.

Lemma 32 (Krylov Subspace Dimension)**.** *A matrix $M \in \mathbb{F}^{n\times n}$ is non-derogatory (i.e., its minimal polynomial has degree n) if and only if there exists a vector $y \in \mathbb{F}^n$ such that the Krylov matrix*

$$K_M(y) = \begin{bmatrix} y & My & M^2y & \ldots & M^{n-1}y \end{bmatrix}$$

is non-singular.

Proof. By the rational canonical form, the vector space decomposes into cyclic subspaces governed by the invariant factors of M. M is non-derogatory if and only if its largest invariant factor (the minimal polynomial) has degree n. If so, summing the generators of the coprime cyclic subspaces yields a vector y that generates an n-dimensional cyclic subspace, making $K_M(y)$ full rank. Conversely, if $K_M(y)$ is full rank, y generates an n-dimensional cyclic subspace, forcing $\deg(m_M) = n$. □

Let $\mathbf{x} = (x_1, \ldots, x_n)$ and $\mathbf{y} = (y_1, \ldots, y_n)$ be formal variables, and let $D_\mathbf{x} = \operatorname{diag}(x_1, \ldots, x_n)$. We construct the formal Krylov matrix $K_{AD_\mathbf{x}}(\mathbf{y})$, whose determinant $F(\mathbf{x}, \mathbf{y}) = \det(K_{AD_\mathbf{x}}(\mathbf{y}))$ is a multivariate polynomial.

Lemma 33 (Generic Distinct Eigenvalues)**.** *If A is non-singular, the formal polynomial $F(\mathbf{x}, \mathbf{y})$ is not identically zero.*

Proof. Let $B = A^{-1}$. Since $\det(A)$ is a non-zero constant, the roots of $\det(\lambda I - AD_\mathbf{x}) = 0$ are distinct if and only if the roots of $Q(\lambda, \mathbf{x}) = \det(\lambda B - D_\mathbf{x})$ are distinct. We proceed by induction on n.

Base Case ($n = 1$): $Q(\lambda, x_1) = \lambda B_{11} - x_1$. Since A is invertible, $B_{11} \neq 0$, yielding exactly one root. The discriminant is trivially non-zero.

Inductive Step: Assume the claim holds for $n-1$. Because B is non-singular, we may permute the coordinates such that the upper-left $(n-1) \times (n-1)$ submatrix, B_{n-1}, is invertible. Expanding $Q(\lambda, \mathbf{x})$ along the last diagonal entry yields:

$$Q(\lambda, \mathbf{x}) = Q_0(\lambda, x_1, \ldots, x_{n-1}) - x_n Q_1(\lambda, x_1, \ldots, x_{n-1})$$

where $Q_1 = \det(\lambda B_{n-1} - D_{n-1})$. By the inductive hypothesis, there exists an assignment $\mathbf{c} \in \mathbb{F}^{n-1}$ such that $Q_1(\lambda, \mathbf{c})$ has $n-1$ distinct, non-zero roots. Fixing these values, we study the two-variable polynomial $Q(\lambda, x_n) = Q_0(\lambda) - x_n Q_1(\lambda)$.

We construct the reverse polynomial. Let $w = 1/\lambda$, and define:

$$R(w, x_n) = w^n Q(1/w, x_n) = R_0(w) - x_n w R_1(w)$$

Substituting $z = 1/x_n$ and multiplying by z yields:

$$H(w, z) = zR(w, 1/z) = zR_0(w) - wR_1(w)$$

Evaluating at $z = 0$ leaves $H(w, 0) = -wR_1(w)$. Because $Q_1(\lambda)$ has $n-1$ distinct non-zero roots, its reverse polynomial $R_1(w)$ has $n-1$ distinct finite roots, none of which are zero. Therefore, $H(w, 0)$ has exactly n distinct roots.

Thus, there exists an assignment for $\mathbf{x}$ yielding n distinct roots, meaning the generic $AD_{\mathbf{x}}$ is non-derogatory and $F(\mathbf{x}, \mathbf{y}) \not\equiv 0$. □

Lemma 34 (Failure Probability)**.** *For random assignments from a finite field $\mathbb{F}$, the probability that the algorithm fails is bounded by $\frac{n(n+1)}{2|\mathbb{F}|}$.*

Proof. To bound the probability of failure, we calculate the total degree of $F(\mathbf{x}, \mathbf{y})$. The k-th column of the Krylov matrix (for $k = 0, \ldots, n-1$) is $(AD_{\mathbf{x}})^k \mathbf{y}$. The entries in this column are homogeneous polynomials of degree k in $\mathbf{x}$ and degree 1 in $\mathbf{y}$. Because the determinant is a sum of products containing exactly one entry from each column, the total degree of F is:

$$\deg(F) = \sum_{k=0}^{n-1} k + \sum_{k=0}^{n-1} 1 = \frac{n(n-1)}{2} + n = \frac{n(n+1)}{2}$$

By the Schwartz-Zippel Lemma, if we draw the entries for D and v uniformly at random from a finite field $\mathbb{F}$, the probability that they form a root of F is bounded by:

$$\mathbb{P}(F(\mathbf{c}, \mathbf{v}) = 0) \leq \frac{n(n+1)}{2|\mathbb{F}|}$$

Finally, the algorithm extracts the constant term c_0 of the minimal polynomial. Since $p_{AD}(0) = \det(-AD) = (-1)^n \det(A) \det(D)$, we correctly compute the determinant as:

$$\det(A) = \frac{(-1)^n c_0}{\prod_{i=1}^{n} c_i}$$

□

3.4.3 Wiedemann's Algorithm

Description Wiedemann's algorithm solves the linear system $Ax = b$ for x. It generates a random vector u and computes the scalar sequence $s_k = u^T A^k b$. It finds the minimal polynomial of this sequence, $p(x) = \sum_{i=0}^{d} p_i x^i$. Assuming A is non-singular, $p_0 \neq 0$. The algorithm then computes the solution vector using Horner's method: $x = -\frac{1}{p_0} \sum_{i=1}^{d} p_i A^{i-1} b$.

Proof of Correctness Let $m_{A,b}(x)$ be the minimal polynomial of A with respect to the vector b, meaning it is the polynomial of minimal degree such that $m_{A,b}(A)b = 0$. With high probability, the minimal polynomial $p(x)$ of the sequence $u^T A^k b$ (found via linear recurrence finding) is exactly $m_{A,b}(x)$.

By definition, $p(A)b = 0$. Expanding this polynomial expression yields:

$$\sum_{i=0}^{d} p_i A^i b = 0$$

Because A is non-singular, the constant term of its minimal polynomial is non-zero, so $p_0 \neq 0$. We can isolate the $i = 0$ term:

$$p_0 b + \sum_{i=1}^{d} p_i A^i b = 0$$

$$p_0 b + A\left(\sum_{i=1}^{d} p_i A^{i-1} b\right) = 0$$

Rearranging to solve for b, we get:

$$A\left[-\frac{1}{p_0}\sum_{i=1}^{d} p_i A^{i-1} b\right] = b$$

Since $Ax = b$, the unique solution x is the term inside the brackets. To compute x efficiently without storing $O(d)$ dense vectors, the algorithm uses Horner's method, evaluating the sum iteratively from $i = d$ down to 1 via $R \leftarrow A(R) + p_i b$, initializing with $R = p_d b$. After d iterations, R equals the summation, giving the correct solution x using only $O(d)$ matrix-vector multiplications and $O(n)$ space.

3.5 Perron-Frobenius Theorem

The Perron-Frobenius theorem establishes the spectral properties of non-negative matrices. A matrix $A \geq 0$ is irreducible if its directed graph is strongly connected; for any i, j, there exists an integer $k > 0$ such that $(A^k)_{ij} > 0$.

Theorem 35 (Perron-Frobenius for Irreducible Matrices)**.** *Let A be an $n \times n$ non-negative, irreducible matrix. Then:*

1. *A has a real, strictly positive eigenvalue $r > 0$ (the Perron root).*
2. *There exists a strictly positive eigenvector $v > 0$ corresponding to r.*
3. *For any other eigenvalue $\lambda \in \mathbb{C}$ of A, $|\lambda| \leq r$.*
4. *The eigenvalue r is algebraically (and geometrically) simple.*

Proof. Let $\Delta = \{x \in \mathbb{R}^n : x \geq 0, \sum_{i=1}^{n} x_i = 1\}$ be the simplex. For any non-zero $x \geq 0$, define the function:

$$r(x) = \max\{\rho \geq 0 \mid Ax \geq \rho x\} = \min_{x_i > 0} \frac{(Ax)_i}{x_i}$$

Let $r = \sup_{x \in \Delta} r(x)$.

Because A is irreducible, the matrix $B = (I + A)^{n-1}$ is positive ($B > 0$). Furthermore, B commutes with A. For any $x \in \Delta$, if $Ax \geq \rho x$, then $A(Bx) = B(Ax) \geq \rho(Bx)$, which implies $r(Bx) \geq r(x)$. The map $x \mapsto Bx$ sends Δ into $\mathbb{R}^n_{>0}$. In this interior, $r(x)$ is continuous. Since the image $B(\Delta)$ is compact and contained in the interior, the supremum r is finite and achieved at some positive vector $v = Bx > 0$. Thus, $Av \geq rv$.

$Av = rv$. Suppose for contradiction that $Av \geq rv$ but $Av \neq rv$. This means the vector $z = Av - rv$ is non-negative and non-zero ($z \gneq 0$). Because $B > 0$, multiplying by B yields a positive vector: $Bz > 0$.

Thus, $B(Av - rv) > 0 \implies A(Bv) > r(Bv)$. Since strict inequality holds for all coordinates of Bv, we can find some small $\epsilon > 0$ such that $A(Bv) \geq (r + \epsilon)(Bv)$. This implies $r(Bv) \geq r + \epsilon$, which contradicts the maximality of r. Therefore, we must have $Av = rv$. Since $v > 0$, r must be strictly positive (as $A \neq 0$).

Strict positivity of the eigenvector. Suppose $u \geq 0$ is any non-zero eigenvector for r, meaning $Au = ru$. Then $Bu = (1 + r)^{n-1}u$. Since $u \gneq 0$ and $B > 0$, we have $Bu > 0$, forcing $(1 + r)^{n-1}u > 0$, which means u must be strictly positive.

Maximality of r. Let λ be any eigenvalue of A with eigenvector $y \in \mathbb{C}^n$, so $Ay = \lambda y$. Taking absolute values component-wise, and using the triangle inequality alongside the non-negativity of A, we get:

$$|\lambda||y| = |\lambda y| = |Ay| \leq A|y|$$

This implies that the non-negative, non-zero vector $|y|$ satisfies $A|y| \geq |\lambda||y|$. By the definition of our function, $r(|y|) \geq |\lambda|$. Since r is the supremum over all such vectors, $r \geq |\lambda|$.

Simplicity of r. By applying the exact same argument to A^T (which is also irreducible), there exists a strictly positive left eigenvector $w^T > 0$ such that $w^T A = rw^T$.

To show the geometric multiplicity is 1, suppose v and y are two linearly independent eigenvectors for r. Since $v > 0$, we can choose a real scalar c such that $z = v - cy$ has at least one coordinate equal to zero, but z is not the zero vector. However, $Az = rz$. Taking absolute values, $A|z| \geq r|z|$. This forces $A|z| = r|z|$, making $|z|$ a non-negative eigenvector for r. Then $|z|$ must be strictly positive, contradicting that it has a zero coordinate. Thus, the geometric multiplicity is 1.

To prove the algebraic multiplicity is 1, we utilize the strictly positive left eigenvector w^T. If the algebraic multiplicity were strictly greater than 1, there would exist a generalized eigenvector $g \in \ker(A - rI)^2$ such that $(A - rI)g = v$. Multiplying both sides on the left by w^T yields:

$$w^T(A - rI)g = w^T v$$

Because w^T is a left eigenvector associated with r, we have $w^T A = rw^T$, meaning $w^T(A - rI) = 0$. Thus, the left side is 0. However, because both $w > 0$ and $v > 0$, their inner product is strictly positive ($w^T v > 0$). Therefore, r must be an algebraically simple root. □

3.6 Continuity of Eigenvalues

The basic fact is that roots of a polynomial with continuously varying coefficients cannot suddenly appear far away, and any limit of roots of nearby polynomials must still be a root of the limiting polynomial. It is convenient to formulate this for the unordered multiset of roots.

For two unordered n-tuples $[a_1, \ldots, a_n]$ and $[b_1, \ldots, b_n]$, define

$$d\big([a_1, \ldots, a_n], [b_1, \ldots, b_n]\big) = \min_{\pi \in S_n} \max_i |a_i - b_{\pi(i)}|.$$

This is the natural metric on $\mathbb{C}^n/S_n$.

Theorem 36 (Continuity of Roots)**.** *The unordered multiset of roots of a monic complex polynomial depends continuously on its coefficients.*

Proof. Let

$$p_k(z) = z^n + c_{n-1}^{(k)} z^{n-1} + \cdots + c_0^{(k)}$$

converge coefficientwise to

$$p(z) = z^n + c_{n-1} z^{n-1} + \cdots + c_0.$$

We prove that the root multisets of p_k converge to the root multiset of p.

First, all roots stay in one fixed compact disk. Indeed, if

$$M_k = \max_{0 \leq j < n} |c_j^{(k)}|,$$

then every root z of p_k satisfies the Cauchy bound

$$|z| \le 1 + M_k.$$

Since the coefficients converge, the numbers M_k are bounded, so one disk contains every root of every sufficiently large p_k.

Write the roots of p_k, in an arbitrary order, as

$$a_1^{(k)}, \dots, a_n^{(k)}.$$

Consider any subsequence. Because all roots lie in a fixed compact disk, after passing to a further subsequence we may assume

$$a_i^{(k)} \longrightarrow a_i \qquad (i = 1, \dots, n).$$

Then for every z,

$$p_k(z) = \prod_{i=1}^{n} (z - a_i^{(k)}) \longrightarrow \prod_{i=1}^{n} (z - a_i).$$

On the other hand coefficientwise convergence gives $p_k(z) \to p(z)$. Hence

$$p(z) = \prod_{i=1}^{n} (z - a_i).$$

Thus every subsequential limit of the root multiset is exactly the root multiset of p, with multiplicity.

If the root multisets did not converge to the roots of p, there would be an $\varepsilon > 0$ and a subsequence remaining at distance at least ε from that multiset. The preceding compactness argument would produce a further subsequence converging to the root multiset of p, a contradiction. Therefore the roots depend continuously on the coefficients. □

Corollary 37 (Continuity of Eigenvalues)**.** *Let $A \in M_n(\mathbb{C})$. The unordered multiset of eigenvalues of A depends continuously on the entries of A.*

Proof. The coefficients of

$$p_A(z) = \det(zI - A)$$

are polynomials in the entries of A, hence depend continuously on A. The eigenvalues are precisely the roots of p_A, so the result follows from the preceding theorem. □

3.7 Gershgorin's Circle Theorem

Gershgorin's theorem localizes the eigenvalues of a matrix using only its entries.

The basic idea is extremely simple. If

$$Ax = \lambda x,$$

choose a coordinate of the eigenvector which is largest in magnitude. In that coordinate, the diagonal term must almost account for the eigenvalue, because all other coordinates are no larger.

It is essentially no harder to allow a different scale for each coordinate. This gives a useful weighted version of the theorem.

Theorem 38 (Weighted Gershgorin Circle Theorem)**.** *Let*

$$A = (a_{ij}) \in \mathbb{C}^{n \times n},$$

and let

$$w_1, \dots, w_n > 0.$$

For each i, define the weighted row radius

$$R_i(w) = \sum_{j \neq i} |a_{ij}| \frac{w_j}{w_i}$$

and the disk

$$D_i(w) = \{z \in \mathbb{C} : |z - a_{ii}| \leq R_i(w)\}.$$

Then every eigenvalue of A lies in

$$\boxed{\sigma(A) \subseteq \bigcup_{i=1}^{n} D_i(w).}$$

Moreover, suppose $I \subseteq [n]$ is such that

$$\bigcup_{i \in I} D_i(w)$$

is disjoint from

$$\bigcup_{i \notin I} D_i(w).$$

Then the first union contains exactly

$$\boxed{|I|}$$

eigenvalues of A, counted with algebraic multiplicity.

Proof. We first prove the localization statement.

Let

$$Ax = \lambda x, \qquad x \neq 0.$$

Choose i maximizing

$$\frac{|x_i|}{w_i}.$$

In particular, $x_i \neq 0$, and for every j,

$$|x_j| \leq \frac{w_j}{w_i} |x_i|.$$

The i-th coordinate of $Ax = \lambda x$ gives

$$(\lambda - a_{ii}) x_i = \sum_{j \neq i} a_{ij} x_j.$$

Taking absolute values,

$$|\lambda - a_{ii}| \, |x_i| \leq \sum_{j \neq i} |a_{ij}| \, |x_j| \leq \sum_{j \neq i} |a_{ij}| \frac{w_j}{w_i} |x_i|.$$

Canceling $|x_i|$,

$$|\lambda - a_{ii}| \leq R_i(w).$$

Thus

$$\lambda \in D_i(w).$$

For the counting statement, write

$$A = D + E,$$

where

$$D = \operatorname{diag}(a_{11}, \ldots, a_{nn})$$

and E has zero diagonal. Consider the (continuous) family

$$A(t) = D + tE, \qquad 0 \le t \le 1.$$

Its weighted Gershgorin disks are

$$D_i(t) = \{z : |z - a_{ii}| \le tR_i(w)\}.$$

Since

$$D_i(t) \subseteq D_i(1),$$

the two unions

$$\mathcal{D}_I(t) = \bigcup_{i \in I} D_i(t), \qquad \mathcal{D}_{\overline{I}}(t) = \bigcup_{i \notin I} D_i(t)$$

remain disjoint for every $t \in [0, 1]$.

By the first part, every eigenvalue of $A(t)$ belongs to one of these two sets.

Because the two original unions are compact and disjoint, they are separated by some positive distance. The eigenvalues depend continuously on the entries of the matrix by Corollary 37, so as t changes, no eigenvalue can pass from one union to the other without crossing the gap between them. Consequently, the number of eigenvalues in $\mathcal{D}_I(t)$, counted with algebraic multiplicity, is constant in t.

At $t = 0$,

$$A(0) = D,$$

whose eigenvalues are exactly

$$a_{11}, \ldots, a_{nn}.$$

The union $\mathcal{D}_I(0)$ contains precisely the diagonal entries

$$a_{ii}, \qquad i \in I,$$

and hence exactly $|I|$ eigenvalues, with multiplicity.

The same is therefore true at $t = 1$, where $A(1) = A$. □

The usual Gershgorin theorem is the special case

$$w_1 = \cdots = w_n = 1.$$

Thus, writing

$$R_i = \sum_{j \ne i} |a_{ij}|,$$

every eigenvalue lies in one of the ordinary Gershgorin disks

$$\boxed{D_i = \{z : |z - a_{ii}| \le R_i\}.}$$

The weighted form can also be viewed as ordinary Gershgorin after a diagonal change of coordinates. Indeed, if

$$W = \operatorname{diag}(w_1, \ldots, w_n),$$

then

$$W^{-1}AW$$

is similar to A, while its ij-entry is

$$a_{ij}\frac{w_j}{w_i}.$$

Thus the weights simply choose the coordinate scaling in which we apply Gershgorin.

Corollary 39 (Second Gershgorin Theorem)**.** *Every connected component of*

$$\bigcup_{i=1}^{n} D_i$$

which consists of k Gershgorin disks contains exactly k eigenvalues, counted with algebraic multiplicity.

Proof. Each disk is connected, so every disk lies entirely within one connected component of the union. A connected component is therefore the union of some subset I of the disks.

Since there are finitely many disks, distinct components are compact and disjoint. Apply the counting part of Theorem 38. □

There is an analogous column version. Define

$$C_i = \sum_{j \neq i} |a_{ji}|.$$

Since A and A^T have the same characteristic polynomial, applying Gershgorin to A^T gives

$$\boxed{\sigma(A) \subseteq \bigcup_i \{z : |z - a_{ii}| \leq C_i\}.}$$

Hence, using both row and column information,

$$\boxed{\sigma(A) \subseteq \left(\bigcup_i D_i^{\text{row}}\right) \cap \left(\bigcup_i D_i^{\text{col}}\right).}$$

For Hermitian matrices all eigenvalues are real, so Gershgorin becomes the interval bound

$$\boxed{\sigma(A) \subseteq \bigcup_i [a_{ii} - R_i, a_{ii} + R_i].}$$

In particular, if A is Hermitian and

$$a_{ii} > R_i \qquad \text{for every } i,$$

then every eigenvalue is positive, and hence

$$A \succ 0.$$

More generally, if

$$|a_{ii}| > R_i \qquad \text{for every } i,$$

then 0 belongs to no Gershgorin disk, so A is nonsingular.

4 Number Theory and Arithmetic

4.1 Bertrand's Postulate and Products of Small Primes

The central binomial coefficient is exponentially large, but small prime powers account for only a limited part of its factorization. Counting all the remaining prime factors gives a lower bound on the product of small primes. Isolating those in a dyadic interval gives Bertrand's postulate and, more strongly, $\Omega(n/\log n)$ primes in $(n, 2n)$.

For real $x \geq 1$, put

$$P(x) = \prod_{p \leq x} p, \qquad \vartheta(x) = \log P(x) = \sum_{p \leq x} \log p.$$

Lemma 40. *For every real $x \geq 1$, $P(x) \leq 4^x$.*

Proof. It suffices to prove the bound for integers, since $P(x) = P(\lfloor x \rfloor)$. Induct on n, with $n = 1, 2$ immediate. If $n \geq 4$ is even, it is composite, so $P(n) = P(n-1) \leq 4^{n-1} < 4^n$.

If $n = 2m+1$ is odd, every prime $m+1 < p \leq 2m+1$ divides $\binom{2m+1}{m}$. The two middle coefficients in the binomial expansion are equal and sum to at most 2^{2m+1}, so $\binom{2m+1}{m} \leq 2^{2m}$. Therefore

$$P(2m+1) \leq P(m+1)\binom{2m+1}{m} \leq 4^{m+1}2^{2m} = 4^{2m+1}.$$

□

Lemma 41 (Central binomial coefficient bounds)**.** *For every even integer $m \geq 4$,*

$$\boxed{\frac{2^m}{m+1} \leq \binom{m}{m/2} \leq m^{\sqrt{m}} P(m/3) \frac{P(m)}{P(m/2)}.}$$

Moreover, every prime power dividing $\binom{m}{m/2}$ is at most m.

Proof. The central coefficient is the largest of the $m+1$ coefficients whose sum is 2^m, proving the lower bound.

For a prime p, Legendre's formula gives

$$v_p\binom{m}{m/2} = \sum_{j \geq 1} \left(\left\lfloor \frac{m}{p^j} \right\rfloor - 2 \left\lfloor \frac{m/2}{p^j} \right\rfloor \right).$$

Every summand is zero or one and vanishes once $p^j > m$. Thus the exponent is at most $\lfloor \log_p m \rfloor$, and the entire prime-power contribution is at most m.

There are at most $\sqrt{m}$ primes at most $\sqrt{m}$, so together they contribute at most $m^{\sqrt{m}}$. Every larger prime occurs with exponent at most one. Among these larger primes, those in $(m/3, m/2]$ do not occur at all, because

$$\left\lfloor \frac{m}{p} \right\rfloor - 2 \left\lfloor \frac{m/2}{p} \right\rfloor = 2 - 2 = 0.$$

The remaining larger primes are at most $m/3$ or belong to $(m/2, m]$. Bounding their products by $P(m/3)$ and $P(m)/P(m/2)$ proves the upper bound. □

The estimate now gives both desired lower bounds. To count all primes, use $P(m/3) \leq P(m/2)$. To isolate the primes in $(m/2, m]$, instead use $P(m/3) \leq 4^{m/3}$. Define the relative error

$$E(x) = \frac{\log x}{\sqrt{x}} + \frac{\log(2x)}{x}.$$

Taking logarithms in the lemma and using $\log(m+1) \leq \log(2m)$ gives, for every even $m \geq 4$,

$$\boxed{\begin{aligned} \vartheta(m) &\geq m\big(\log 2 - E(m)\big), \\ \vartheta(m) - \vartheta(m/2) &\geq m\big(\tfrac{1}{3}\log 2 - E(m)\big). \end{aligned}} \tag{1}$$

Both summands of $E(x)$ decrease for $x \geq 8$: their derivatives have the signs of $1 - \frac{1}{2}\log x$ and $1 - \log(2x)$, respectively. We will use the two explicit values

$$E(64) = \frac{55}{64}\log 2, \qquad E(1024) = \frac{331}{1024}\log 2 < \frac{1}{3}\log 2.$$

Theorem 42 (Bertrand's postulate, quantitatively)**.** *There is an absolute constant $c > 0$ such that, for every integer $n \geq 2$,*

$$\boxed{\#\{p \text{ prime} : n < p < 2n\} \geq c\frac{n}{\log n}.}$$

In particular, every such interval contains a prime.

Proof. For $n \geq 512$, apply the second bound in (1) with $m = 2n$:

$$\vartheta(2n) - \vartheta(n) \geq c_0 n, \qquad c_0 = 2\log 2\left(\frac{1}{3} - \frac{331}{1024}\right) > 0.$$

Since $2n$ is composite, the left side sums $\log p$ over $n < p < 2n$. Each summand is at most $\log(2n) \leq 2\log n$, so

$$\#\{p : n < p < 2n\} \geq \frac{\vartheta(2n) - \vartheta(n)}{\log(2n)} \geq \frac{c_0}{2}\frac{n}{\log n}.$$

For $2 \leq n < 512$, use the prime sequence

$$2, 3, 5, 7, 13, 23, 43, 83, 163, 317, 631.$$

Each term is less than twice its predecessor. If q is the first term greater than n and q' its predecessor, then $q' \leq n < q < 2q' \leq 2n$. Thus these intervals also contain a prime. Decreasing $c > 0$ if necessary makes the quantitative bound hold for these finitely many n as well. □

Corollary 43 (Product of small primes)**.** *For every integer $N \geq 2$,*

$$\boxed{\prod_{p \leq 16\log N} p > N.}$$

Proof. Put $x = 16\log N$. If $x < 64$, then $N < e^4 < 81$, while $x \geq 16\log 2 > 7$. Hence $P(x) \geq 2 \cdot 3 \cdot 5 \cdot 7 = 210 > N$.

Otherwise, let $m = 2\lfloor x/2 \rfloor$, so $m \geq 64$ and $m > x - 2$. The first bound in (1) gives

$$\begin{aligned}\vartheta(x) &\geq \vartheta(m) \geq m\big(\log 2 - E(64)\big)\\ &= \frac{9m}{64}\log 2 > \frac{3m}{32} > \frac{3(x-2)}{32} > \frac{x}{16} = \log N,\end{aligned}$$

using $\log 2 > 2/3$ and $x \geq 64$. Exponentiating proves the claim. □

4.2 Lucas's Theorem

Theorem 44 (Lucas's Theorem)**.** *Let p be prime, and write the base-p expansions*

$$n = n_0 + n_1 p + \cdots + n_r p^r, \qquad k = k_0 + k_1 p + \cdots + k_r p^r,$$

where $0 \leq n_i, k_i < p$. Then

$$\binom{n}{k} \equiv \prod_{i=0}^{r} \binom{n_i}{k_i} \pmod{p}.$$

Proof. Over $\mathbb{F}_p$, the binomial theorem gives

$$(1+x)^p = \sum_{j=0}^{p} \binom{p}{j} x^j \equiv 1 + x^p \pmod{p},$$

since $p \mid \binom{p}{j}$ for $0 < j < p$. Iterating,

$$(1+x)^{p^i} \equiv 1 + x^{p^i} \pmod{p}.$$

Therefore

$$\begin{aligned}(1+x)^n &= \prod_i (1+x)^{n_i p^i} \\ &\equiv \prod_i (1+x^{p^i})^{n_i} \pmod{p}.\end{aligned}$$

Expanding the right-hand side,

$$\prod_i \left(\sum_{j=0}^{n_i} \binom{n_i}{j} x^{jp^i} \right).$$

Since every $j < p$, the exponent

$$k = \sum_i k_i p^i$$

has a unique base-p representation. Thus the only way to obtain x^k is to choose $x^{k_i p^i}$ from the i-th factor. Hence

$$[x^k](1+x)^n \equiv \prod_i \binom{n_i}{k_i} \pmod{p}.$$

Since $[x^k](1+x)^n = \binom{n}{k}$, the result follows. □

Corollary 45. *With the same notation,*

$$\binom{n}{k} \not\equiv 0 \pmod{p} \iff k_i \le n_i \quad \textit{for every } i.$$

Consequently, the number of entries in the n-th row of Pascal's triangle that are non-zero modulo p is

$$\prod_i (n_i + 1).$$

Proof. Because $0 \le n_i, k_i < p$,

$$\binom{n_i}{k_i} \not\equiv 0 \pmod{p}$$

exactly when $k_i \le n_i$. Lucas's theorem gives the first claim. For the second, each digit k_i may independently be chosen from

$$0, 1, \ldots, n_i.$$

□

4.3 Quadratic Reciprocity

For an odd prime p, define the Legendre symbol by

$$\left(\frac{a}{p}\right) = \begin{cases} 0, & p \mid a, \\ 1, & a \not\equiv 0 \pmod{p} \text{ is a square}, \\ -1, & a \text{ is not a square.} \end{cases}$$

We first prove the fact that will give both quadratic reciprocity and its supplementary laws.

Lemma 46 (Euler's Criterion)**.** *For p an odd prime,*

$$a^{(p-1)/2} \equiv \left(\frac{a}{p}\right) \pmod{p}.$$

Proof. Assume $a \not\equiv 0 \pmod{p}$. There are exactly $(p-1)/2$ non-zero squares modulo p, since x and $-x$ have the same square.

Every square $a = x^2$ satisfies

$$a^{(p-1)/2} = x^{p-1} \equiv 1 \pmod{p}.$$

Thus the $(p-1)/2$ non-zero squares are exactly the roots of

$$x^{(p-1)/2} - 1.$$

If a is not a square, then $a^{(p-1)/2} \neq 1$, while Fermat's little theorem gives

$$\left(a^{(p-1)/2}\right)^2 = a^{p-1} \equiv 1.$$

Hence $a^{(p-1)/2} \equiv -1$. □

Lemma 47 (Gauss's Lemma)**.** *Let p be an odd prime, $p \nmid a$, and set $m = (p-1)/2$. Consider*

$$a, 2a, \ldots, ma$$

modulo p, represented by integers in $\{1, \ldots, p-1\}$. Let N be the number of these residues that lie in $(p/2, p)$. Then

$$\left(\frac{a}{p}\right) = (-1)^N.$$

Proof. For each $1 \leq j \leq m$, replace the residue of aj by its representative up to sign in

$$\{1, \ldots, m\}.$$

These absolute representatives are all distinct: if

$$aj \equiv \pm ak \pmod{p},$$

then

$$j \equiv \pm k \pmod{p},$$

and for $1 \leq j, k \leq m$ this forces $j = k$.

Thus they are exactly a permutation of $1, \ldots, m$. A residue contributes a minus sign precisely when it lies in $(p/2, p)$, so

$$a^m m! \equiv (-1)^N m! \pmod{p}.$$

Canceling $m!$ gives

$$a^{(p-1)/2} \equiv (-1)^N \pmod{p}.$$

Now apply Lemma 46. □

Corollary 48 (First Supplement)**.** *For every odd prime p,*

$$\left(\frac{-1}{p}\right) = (-1)^{(p-1)/2}.$$

Equivalently, -1 is a quadratic residue modulo p if and only if

$$p \equiv 1 \pmod{4}.$$

Proof. Multiplication by -1 sends every

$$1, \dots, \frac{p-1}{2}$$

to a residue greater than $p/2$. Thus $N = (p-1)/2$ in Gauss's lemma. □

Corollary 49 (Second Supplement)**.** *For every odd prime p,*

$$\left(\frac{2}{p}\right) = (-1)^{(p^2-1)/8}.$$

Equivalently, 2 is a quadratic residue modulo p exactly when

$$p \equiv \pm 1 \pmod 8.$$

Proof. For

$$1 \le j \le \frac{p-1}{2},$$

the numbers $2j$ already lie in $\{1, \dots, p-1\}$. They lie above $p/2$ exactly when

$$j > \frac{p}{4}.$$

Hence

$$N = \frac{p-1}{2} - \left\lfloor \frac{p}{4} \right\rfloor.$$

Checking the four possibilities $p \equiv 1, 3, 5, 7 \pmod 8$, this number is even for $p \equiv 1, 7 \pmod 8$ and odd for $p \equiv 3, 5 \pmod 8$. Thus

$$(-1)^N = (-1)^{(p^2-1)/8}.$$

Now apply Gauss's lemma. □

For quadratic reciprocity, it is useful to rewrite Gauss's lemma as a lattice-point count.

Lemma 50 (Eisenstein's Form of Gauss's Lemma)**.** *Let p be an odd prime and let a be odd with $p \nmid a$. Then*

$$\left(\frac{a}{p}\right) = (-1)^{\sum_{j=1}^{(p-1)/2} \left\lfloor \frac{aj}{p} \right\rfloor}.$$

Proof. Set $m = (p-1)/2$ and write

$$aj = pq_j + r_j, \qquad 1 \le r_j \le p-1.$$

Let

$$s_j = \min(r_j, p - r_j) \in \{1, \dots, m\}.$$

As in the proof of Gauss's lemma, the s_j are a permutation of $1, \dots, m$.

Let $\epsilon_j = 0$ if $r_j \le m$ and $\epsilon_j = 1$ otherwise. Since a and p are odd, reducing

$$aj = pq_j + r_j$$

modulo 2 gives

$$j \equiv q_j + s_j + \epsilon_j \pmod 2.$$

Summing over j, the sums of the j's and the s_j's cancel modulo 2, since (s_j) is a permutation of (j). Therefore

$$\sum_j q_j \equiv \sum_j \epsilon_j = N \pmod 2.$$

Since $q_j = \lfloor aj/p \rfloor$, Gauss's lemma gives the result. □

Theorem 51 (Quadratic Reciprocity)**.** *For distinct odd primes p and q,*

$$\left(\frac{p}{q}\right)\left(\frac{q}{p}\right) = (-1)^{\frac{(p-1)(q-1)}{4}}.$$

Equivalently,

$$\left(\frac{p}{q}\right) = \left(\frac{q}{p}\right)$$

unless

$$p \equiv q \equiv 3 \pmod 4,$$

in which case the two symbols have opposite signs.

Proof. Set

$$m = \frac{p-1}{2}, \qquad n = \frac{q-1}{2}.$$

By Lemma 50,

$$\left(\frac{q}{p}\right) = (-1)^A, \qquad A = \sum_{x=1}^{m} \left\lfloor \frac{qx}{p} \right\rfloor,$$

and

$$\left(\frac{p}{q}\right) = (-1)^B, \qquad B = \sum_{y=1}^{n} \left\lfloor \frac{py}{q} \right\rfloor.$$

Consider the mn lattice points

$$(x, y), \qquad 1 \le x \le m, \quad 1 \le y \le n.$$

The number below the line

$$py = qx$$

is exactly

$$A,$$

because for a fixed x there are

$$\left\lfloor \frac{qx}{p} \right\rfloor$$

possible positive y.

Similarly, the number above the line is exactly B.

No lattice point lies on the line: $py = qx$ would imply $p \mid x$, impossible since $1 \le x < p$. Thus every point is either above or below, and

$$A + B = mn = \frac{(p-1)(q-1)}{4}.$$

Consequently,

$$\left(\frac{q}{p}\right)\left(\frac{p}{q}\right) = (-1)^{A+B} = (-1)^{\frac{(p-1)(q-1)}{4}}.$$

□

4.4 Values of the Zeta Function at Even Integers

The formula for

$$\zeta(2k) = \sum_{n=1}^{\infty} \frac{1}{n^{2k}}$$

comes from expanding the same function in two different ways.

Expanding the partial fractions of $\cot(\pi z)$ produces the sums $\zeta(2k)$.

On the other hand, $\cot z$ can be written using the exponential function, whose expansion is naturally encoded by the Bernoulli numbers.

Define the Bernoulli numbers by

$$\boxed{\frac{t}{e^t - 1} = \sum_{m=0}^{\infty} B_m \frac{t^m}{m!}.}$$

Thus

$$B_0 = 1, \qquad B_1 = -\frac{1}{2}.$$

Theorem 52 (Euler)**.** *For every integer $k \geq 1$,*

$$\boxed{\zeta(2k) = (-1)^{k+1} \frac{B_{2k}(2\pi)^{2k}}{2(2k)!}.}$$

Proof. We compute the power series of

$$\pi z \cot(\pi z)$$

in two ways.

From the poles of cotangent. The partial-fraction expansion

$$\pi \cot(\pi z) = \frac{1}{z} + \sum_{n=1}^{\infty} \frac{2z}{z^2 - n^2}$$

gives

$$\pi z \cot(\pi z) = 1 - 2\sum_{n=1}^{\infty} \frac{z^2}{n^2 - z^2}.$$

For $|z| < 1$,

$$\frac{z^2}{n^2 - z^2} = \frac{z^2/n^2}{1 - z^2/n^2} = \sum_{k=1}^{\infty} \frac{z^{2k}}{n^{2k}}.$$

Hence

$$\begin{aligned} \pi z \cot(\pi z) &= 1 - 2\sum_{n=1}^{\infty}\sum_{k=1}^{\infty} \frac{z^{2k}}{n^{2k}} \\ &= 1 - 2\sum_{k=1}^{\infty} \left(\sum_{n=1}^{\infty} \frac{1}{n^{2k}} \right) z^{2k}. \end{aligned}$$

Therefore

$$\boxed{\pi z \cot(\pi z) = 1 - 2\sum_{k=1}^{\infty} \zeta(2k) z^{2k}.} \tag{1}$$

The interchange of the sums is justified for $|z| < 1$, where the series is absolutely convergent.

From the Bernoulli generating function. Using

$$\cot x = i\frac{e^{2ix}+1}{e^{2ix}-1} = i + \frac{2i}{e^{2ix}-1},$$

we obtain

$$x\cot x = ix + \frac{2ix}{e^{2ix}-1}.$$

Set

$$t = 2ix.$$

By the Bernoulli generating function,

$$\frac{2ix}{e^{2ix}-1} = \sum_{m=0}^{\infty} B_m \frac{(2ix)^m}{m!}.$$

Thus

$$x\cot x = ix + \sum_{m=0}^{\infty} B_m \frac{(2ix)^m}{m!}.$$

The $m = 1$ term is

$$B_1(2ix) = -\frac{1}{2}(2ix) = -ix,$$

so it cancels the initial ix.

Moreover, $x\cot x$ is even, so all remaining odd coefficients vanish. Thus

$$x\cot x = \sum_{k=0}^{\infty} B_{2k} \frac{(2ix)^{2k}}{(2k)!}.$$

Since

$$i^{2k} = (-1)^k,$$

this becomes

$$\boxed{x\cot x = \sum_{k=0}^{\infty} (-1)^k \frac{2^{2k} B_{2k}}{(2k)!} x^{2k}.}$$

Substituting $x = \pi z$,

$$\boxed{\pi z \cot(\pi z) = \sum_{k=0}^{\infty} (-1)^k \frac{B_{2k}(2\pi)^{2k}}{(2k)!} z^{2k}.} \tag{2}$$

Finally compare the coefficient of z^{2k} in (1) and (2). For $k \geq 1$,

$$-2\zeta(2k) = (-1)^k \frac{B_{2k}(2\pi)^{2k}}{(2k)!}.$$

Therefore

$$\boxed{\zeta(2k) = (-1)^{k+1} \frac{B_{2k}(2\pi)^{2k}}{2(2k)!}.}$$

□

4.5 Hensel's Lemma

A root modulo p^k already determines the first k base-p digits of a possible root modulo a higher power. To find the next digits, change a to $a + p^k t$. Modulo p^{2k}, every nonlinear term in this correction vanishes, leaving one linear equation for t. We generalize this to commutative rings.

Throughout, R is a commutative ring with identity, and congruence modulo an ideal means that the difference belongs to that ideal.

Theorem 53 (Hensel lifting modulo an ideal)**.** *Let $I \subseteq R$ be an ideal, $f \in R[X]$, and $a \in R$. Suppose $f(a) \in I$ and $f'(a)$ is invertible modulo I. Choose $u \in R$ with $uf'(a) \equiv 1 \pmod{I}$. Then there is a unique residue class $b \pmod{I^2}$ satisfying*

$$f(b) \equiv 0 \pmod{I^2}, \qquad b \equiv a \pmod{I}.$$

It is given by

$$\boxed{b \equiv a - uf(a) \pmod{I^2}.}$$

Proof. For any correction $h \in I$, the binomial theorem gives

$$f(a+h) \equiv f(a) + f'(a)h \pmod{I^2},$$

because every remaining term contains at least two factors of h.

Take $h = -uf(a) \in I$. Then

$$f(a+h) \equiv (1 - uf'(a))f(a) \equiv 0 \pmod{I^2},$$

since both factors belong to I. This proves existence.

Conversely, if $h \in I$ gives a root modulo I^2, multiply its linearized equation by u. Since $(uf'(a)-1)h \in I^2$, we obtain

$$0 \equiv uf(a) + uf'(a)h \equiv uf(a) + h \pmod{I^2}.$$

Thus $h \equiv -uf(a) \pmod{I^2}$, proving uniqueness. □

Lift the inverse at the same time. The next step needs an inverse of $f'(b)$ modulo I^2. There is no need to compute it from scratch. Since $b \equiv a \pmod{I}$, the old u remains an inverse modulo I. Put $e = 1 - uf'(b) \in I$ and set

$$\boxed{u^+ = u(2 - uf'(b)) = u(1+e).}$$

Its new error is

$$1 - u^+ f'(b) = 1 - (1+e)(1-e) = e^2 \in I^2.$$

Thus both the root and its inverse derivative have been lifted.

Starting with $f(a_0) \in I$ and $1 - u_0 f'(a_0) \in I$, iterate

$$\boxed{\begin{aligned} a_{j+1} &= a_j - u_j f(a_j), \\ u_{j+1} &= u_j\big(2 - u_j f'(a_{j+1})\big). \end{aligned}}$$

Applying the theorem with I^{2^j} in place of I proves inductively that

$$f(a_j) \in I^{2^j}, \qquad 1 - u_j f'(a_j) \in I^{2^j}, \qquad a_{j+1} - a_j \in I^{2^j}.$$

Each iteration therefore doubles the ideal-adic precision. For computations modulo I^N, reduce every operation modulo the next required precision $I^{\min\{2^{j+1}, N\}}$. Starting modulo I, at most $\lceil \log_2 N \rceil$ iterations suffice.

Prime powers. For $R = \mathbb{Z}$ and $I = (p)$ with p prime, invertibility of $f'(a)$ modulo I is exactly $p \nmid f'(a)$, recovering the usual prime-power statement.

For $R = \mathbb{F}[[t]]$ and $I = (t)$, congruence modulo I^k means agreement of coefficients below degree k. The same update doubles the number of known coefficients. More generally, for $R = \mathbb{F}[[t_1, \ldots, t_r]]$ and $I = (t_1, \ldots, t_r)$, it doubles precision in total degree.

4.6 Arithmetic Algorithms

4.6.1 Linear Sieve

The linear sieve is distinguished from the ordinary Eratosthenes sieve by the fact that every composite integer is generated exactly once.

This also makes it convenient for evaluating arbitrary multiplicative functions.

Suppose f is multiplicative. We assume access to two operations:

- PrimePower(p, e) computes $f(p^e)$ at cost G;
- Combine$(f(a), f(b))$ computes $f(ab)$ when $\gcd(a, b) = 1$, at cost F.

For each integer i, let

$$\mathrm{lp}[i]$$

be its least prime factor. We also maintain

$$e[i] = v_{\mathrm{lp}[i]}(i), \qquad q[i] = \mathrm{lp}[i]^{e[i]}.$$

Thus

$$i = q[i] \cdot r \qquad \text{with} \qquad \gcd(q[i], r) = 1.$$

Algorithm 1 Linear Sieve for Multiplicative Functions

Input: Integer n
Output: Primes below n, least prime factors, and $f(1), \ldots, f(n-1)$

```
f[1] ← f(1)
primes ← ∅
Initialize lp, e, q to zero
for i = 2, ..., n − 1 do
    if lp[i] = 0 then
        lp[i] ← i
        e[i] ← 1
        q[i] ← i
        append i to primes
        f[i] ← PrimePower(i, 1)
    for p in primes do
        if ip ≥ n then
            break
        lp[ip] ← p
        if p = lp[i] then
            e[ip] ← e[i] + 1
            q[ip] ← q[i]p
            if q[ip] = ip then
                f[ip] ← PrimePower(p, e[ip])
            else
                f[ip] ← Combine(f[ip/q[ip]], f[q[ip]])
            break
        else
            e[ip] ← 1
            q[ip] ← p
            f[ip] ← Combine(f[i], f[p])
```

Lemma 54. *Every composite integer $x < n$ is generated exactly once by Algorithm 1.*

Proof. Let

$$p = \mathrm{lp}[x], \qquad i = x/p.$$

Then every prime factor of i is at least p, so

$$p \le \mathrm{lp}[i].$$

Therefore the inner loop for i reaches p before it terminates, and generates $x = ip$.

Conversely, the sieve only generates ip while

$$p \le \mathrm{lp}[i].$$

Thus p is the least prime factor of ip. Hence if x is generated as ip, necessarily

$$p = \mathrm{lp}[x], \qquad i = x/p,$$

which is unique. □

Theorem 55. *Algorithm 1 correctly computes any multiplicative function f on $[1, n)$ in*

$$O\left(n + nF + \frac{n}{\log n} G\right)$$

time.

Proof. By the preceding lemma, the total number of inner-loop iterations is $O(n)$.

Whenever $p < \mathrm{lp}[i]$, we have $\gcd(i, p) = 1$, so

$$f(ip) = \mathrm{Combine}(f(i), f(p)).$$

When $p = \mathrm{lp}[i]$, write

$$i = p^e r, \qquad \gcd(p, r) = 1.$$

Then

$$ip = p^{e+1} r,$$

and the algorithm computes

$$f(ip) = \mathrm{Combine}\big(f(r), f(p^{e+1})\big).$$

The value $f(p^{e+1})$ has already been computed unless $ip = p^{e+1}$ itself is a pure prime power; in that case it is computed directly by PrimePower.

Thus PrimePower is called exactly once for each prime power below n.

There are

$$\pi(n) + O(\sqrt{n}\log n) = O\left(\frac{n}{\log n}\right)$$

prime powers below n. Indeed, the powers with exponent at least two contribute $O(\sqrt{n}\log n)$, while

$$\pi(n) = O\left(\frac{n}{\log n}\right)$$

follows already from Lemma 40: since

$$\sum_{p \le n} \log p \le n \log 4,$$

the primes larger than $\sqrt{n}$ number at most

$$\frac{2n \log 4}{\log n},$$

and there are at most $\sqrt{n}$ smaller primes.

Hence there are $O(n)$ calls to Combine and $O(n/\log n)$ calls to PrimePower, giving

$$O\left(n + nF + \frac{n}{\log n} G\right).$$

□

For ordinary multiplicative functions, Combine is simply multiplication. The only function-specific information needed is therefore the prime-power formula. For example,

$$\varphi(p^e) = p^e - p^{e-1}, \qquad \mu(p^e) = \begin{cases} -1, & e = 1, \\ 0, & e \ge 2, \end{cases}$$

and

$$\sigma_k(p^e) = 1 + p^k + p^{2k} + \cdots + p^{ek}.$$

4.6.2 Extended GCD with Preprocessing

Given integers a, b, we want integers x, y satisfying

$$ax + by = \gcd(a, b).$$

Ordinary extended Euclid takes $O(\log N)$ arithmetic operations for $|a|, |b| \le N$. With preprocessing, we can batch many Euclidean steps.

The basic observation is that after scaling a pair (u, v) to a fixed box, we only have to consider $O(T)$ reductions: one coordinate lies on the boundary of the box.

For an integer $T \ge 2$, define

$$\mathcal{B}_T = \{(p, q) \in \mathbb{Z}^2 : \max(|p|, |q|) = T\}.$$

Then

$$|\mathcal{B}_T| = O(T).$$

Lemma 56 (Short boundary reduction)**.** *For every $(p, q) \in \mathcal{B}_T$, there is a unimodular matrix $R \in \mathrm{SL}_2(\mathbb{Z})$ whose entries have magnitude $O(\sqrt{T})$ and such that the first coordinate of*

$$R\begin{pmatrix} p \\ q \end{pmatrix}$$

has magnitude $O(\sqrt{T})$.

Proof. After swapping coordinates and changing signs, assume $|q| = T$.

Set

$$Q = \lceil \sqrt{T} \rceil.$$

Consider the $Q + 1$ fractional parts

$$0, \frac{p}{q}, \frac{2p}{q}, \ldots, \frac{Qp}{q}.$$

Partition $[0, 1)$ into Q intervals of length $1/Q$. Two fractional parts lie in the same interval, so for some $1 \le r \le Q$ and some integer s,

$$|rp - sq| \le \frac{|q|}{Q} = O(\sqrt{T}).$$

Also,

$$|s| \le \left| \frac{rp}{q} \right| + 1 = O(\sqrt{T}).$$

Dividing (r, s) by $\gcd(r, s)$ only makes these bounds smaller, so assume $\gcd(r, s) = 1$. Then the row vector $(r, -s)$ can be completed to a unimodular matrix

$$R = \begin{pmatrix} r & -s \\ u & v \end{pmatrix}.$$

The Bézout coefficients u, v may be chosen with magnitude $O(\max(|r|, |s|)) = O(\sqrt{T})$.

Its first output coordinate is

$$rp - sq = O(\sqrt{T}),$$

as required. □

Precompute one such matrix $R_{p,q}$ for every $(p, q) \in \mathcal{B}_T$. The table contains only $O(T)$ matrices.

Lemma 57 (One lookup shrinks the pair)**.** *Let*

$$z = \begin{pmatrix} u \\ v \end{pmatrix} \neq 0, \qquad B = \max(|u|, |v|).$$

Using one lookup in the boundary table followed by one Euclidean division, we can apply a unimodular transformation sending z to z' with

$$\|z'\|_\infty = O\left(\frac{B}{\sqrt{T}}\right).$$

Proof. Round the normalized pair to the boundary:

$$p = \left\lfloor \frac{Tu}{B} \right\rceil, \qquad q = \left\lfloor \frac{Tv}{B} \right\rceil.$$

Because one of $|u|, |v|$ equals B, one of $|p|, |q|$ equals T, so

$$(p, q) \in \mathcal{B}_T.$$

Moreover,

$$z = \frac{B}{T}\begin{pmatrix} p \\ q \end{pmatrix} + e, \qquad \|e\|_\infty = O(B/T).$$

Let $R = R_{p,q}$. By Lemma 56, the first coordinate of $R(p, q)^T$ is $O(\sqrt{T})$ and every entry of R is $O(\sqrt{T})$. Thus the first coordinate of Rz is

$$O\left(\frac{B}{T}\sqrt{T}\right) + O\left(\sqrt{T}\frac{B}{T}\right) = O\left(\frac{B}{\sqrt{T}}\right).$$

Call this coordinate w_1 and the other coordinate w_2. If $w_1 = 0$, the gcd has already been computed. Otherwise choose

$$Q = \left\lfloor \frac{w_2}{w_1} \right\rceil$$

and apply

$$\begin{pmatrix} 1 & 0 \\ -Q & 1 \end{pmatrix}.$$

The second coordinate becomes

$$w_2 - Qw_1$$

and has magnitude at most $|w_1|/2$. Hence both coordinates are

$$O(B/\sqrt{T}).$$

□

The extended coefficients are maintained simply by accumulating the unimodular transformations.

Algorithm 2 Preprocessed Extended GCD

Input: Integers a, b, boundary table for parameter T
Output: (x, y) satisfying $ax + by = \gcd(a, b)$

$z \leftarrow (a, b)^T$
$P \leftarrow I$
while $z_1 \neq 0$ and $z_2 \neq 0$ **do**
  $B \leftarrow \max(|z_1|, |z_2|)$
  $p \leftarrow \lfloor Tz_1/B \rceil$
  $q \leftarrow \lfloor Tz_2/B \rceil$
  $R \leftarrow R_{p,q}$
  $z \leftarrow Rz$
  $P \leftarrow RP$
  if $z_1 = 0$ or $z_2 = 0$ **then**
    break
  $Q \leftarrow \lfloor z_2/z_1 \rceil$
  $E \leftarrow \begin{pmatrix} 1 & 0 \\ -Q & 1 \end{pmatrix}$
  $z \leftarrow Ez$
  $P \leftarrow EP$
if $z_1 \neq 0$ **then**
  return $\operatorname{sgn}(z_1)$ times the first row of P
else
  return $\operatorname{sgn}(z_2)$ times the second row of P

Theorem 58. *Suppose*

$$0 < \max(|a|, |b|) \leq N.$$

After storing an $O(T)$-entry boundary table, Algorithm 2 uses

$$O\left(\frac{\log N}{\log T}\right)$$

operations.

Proof. At all times,

$$z = P(a, b)^T$$

with P unimodular, so

$$\gcd(z_1, z_2) = \gcd(a, b).$$

By Lemma 57, every nonterminal iteration decreases

$$B = \|z\|_\infty$$

by a factor $\Theta(\sqrt{T})$. Hence after s iterations,

$$B \leq N \cdot O(T^{-s/2}).$$

Since the coordinates are integers, after

$$s = O(\log N / \log T)$$

iterations one coordinate must become zero.

If

$$z = (g, 0)^T,$$

then the first row (x, y) of P satisfies

$$ax + by = g,$$

and $|g| = \gcd(a, b)$. The other case is identical. □

Thus, for example,

$$T = N^{\varepsilon}$$

gives $O(N^{\varepsilon})$ stored table entries and

$$O(1/\varepsilon)$$

query iterations, while

$$T = (\log N)^c$$

gives polylogarithmic space and

$$O\left(\frac{\log N}{\log \log N}\right)$$

query iterations.

4.6.3 Rational Reconstruction

Chinese remaindering or Hensel lifting may give a rational answer only through its residue modulo a large integer. Small numerator and denominator bounds make that residue sufficient to recover the exact fraction. Extended Euclid performs the recovery: its remainder is a candidate numerator, and the corresponding coefficient of the residue is a candidate denominator.

Given $M \geq 2$ and a residue $x \pmod M$, seek integers a, b satisfying

$$|a| \leq A, \qquad 1 \leq b \leq B, \qquad \gcd(a, b) = 1, \qquad a \equiv bx \pmod M,$$

where $A \geq 0$, $B \geq 1$, and $2AB < M$. The congruence and coprimality already imply $\gcd(b, M) = 1$: any common divisor of b and M also divides a. Thus the congruence really says that a/b has residue x.

Lemma 59 (Uniqueness)**.** *Under $2AB < M$, there is at most one such reduced fraction.*

Proof. If a/b and c/d both qualify, then $M \mid ad - bc$, whereas

$$|ad - bc| \leq |a|d + |c|b \leq 2AB < M.$$

Therefore $ad = bc$. Reducedness and positive denominators make the two pairs identical. □

Reduce x to $0 \leq x < M$. Start from $(r_0, t_0) = (M, 0)$ and $(r_1, t_1) = (x, 1)$, and apply the same Euclidean update to both coordinates:

$$(r_0, r_1) \leftarrow (r_1, r_0 - qr_1), \qquad (t_0, t_1) \leftarrow (t_1, t_0 - qt_1), \qquad q = \lfloor r_0/r_1 \rfloor.$$

Every pair satisfies $r_i \equiv t_i x \pmod M$. The nonnegative remainders decrease while the nonzero t_i alternate in sign and increase in absolute value. Stop as soon as the current remainder fits the numerator bound, and check the denominator and coprimality bounds.

Algorithm 3 Rational reconstruction by stopping extended Euclid early

Input: $M \geq 2$, integer x, $A \geq 0$, $B \geq 1$, and $2AB < M$
Output: The unique reduced pair (a, b) in the stated bounds, or Fail

$x \leftarrow x \bmod M$, with $0 \leq x < M$
if $x = 0$ **then**
  return $(0, 1)$
if $A = 0$ **then**
  return Fail
$(r_0, r_1) \leftarrow (M, x)$; $(t_0, t_1) \leftarrow (0, 1)$
while $r_1 > A$ **do**
  $q \leftarrow \lfloor r_0 / r_1 \rfloor$
  $(r_0, r_1) \leftarrow (r_1, r_0 - q r_1)$
  $(t_0, t_1) \leftarrow (t_1, t_0 - q t_1)$
if $|t_1| > B$ or $\gcd(r_1, |t_1|) \neq 1$ **then**
  return Fail
return $(\mathrm{sgn}(t_1) r_1, |t_1|)$

The loop never divides by zero: it performs a step only while $r_1 > A \geq 0$. The coefficient t_1 is always nonzero. Correctness of an accepted pair is immediate from the invariants. The real question is why stopping at the *first* small remainder cannot miss a solution.

A smaller denominator cannot yet have a small remainder. The relevant congruence pairs form the integer lattice

$$\mathcal{L} = \{(u, v) \in \mathbb{Z}^2 : u \equiv vx \pmod{M}\}.$$

Initially $(M, 0), (x, 1)$ are a basis: every pair is $((u - vx)/M)(M, 0) + v(x, 1)$. A Euclidean update replaces a basis by another integer basis, since subtracting an integer multiple and swapping are reversible over $\mathbb{Z}$.

Lemma 60 (Euclidean exclusion bound)**.** *For consecutive pairs $(r_0, t_0), (r_1, t_1)$ in the algorithm, if $(u, v) \in \mathcal{L}$ and $0 < |v| < |t_1|$, then $|u| \geq r_0$.*

Proof. Express $(u, v) = \alpha(r_0, t_0) + \beta(r_1, t_1)$ with integers α, β. We cannot have $\alpha = 0$, because then a nonzero v would have magnitude at least $|t_1|$. Negate the pair if necessary so that $\alpha > 0$; the hypotheses and desired conclusion use absolute values and are unchanged.

The coefficients t_0, t_1 have opposite signs, except that initially $t_0 = 0$. If $\beta < 0$, then αt_0 and βt_1 have the same sign, or the first is zero. Their sum has magnitude at least $|t_1|$, a contradiction. Hence $\beta \geq 0$, and $u = \alpha r_0 + \beta r_1 \geq r_0$. □

Theorem 61 (Rational reconstruction)**.** *Algorithm 3 returns the unique reduced fraction with $|a| \leq A$, $1 \leq b \leq B$, and $a \equiv bx \pmod{M}$, or correctly reports that none exists. It uses $O(\log M)$ Euclidean steps and a constant number of $O(\log M)$-bit working integers.*

Proof. The cases $x = 0$ and $A = 0$ follow from uniqueness and the fact that a reduced fraction with numerator zero has denominator one. Otherwise, at the stopping point $r_0 > A \geq r_1 \geq 0$. Suppose a valid (a, b) exists. The exclusion bound implies $|t_1| \leq b \leq B$, since $b < |t_1|$ would force $|a| \geq r_0 > A$.

Both pairs satisfy the congruence, so $M \mid at_1 - br_1$. Their bounds give

$$|at_1 - br_1| \leq A|t_1| + br_1 \leq 2AB < M.$$

Thus $at_1 = br_1$. Since (a, b) is primitive, this implies $(r_1, t_1) = \lambda(a, b)$ for an integer λ. It is nonzero because $t_1 \neq 0$, and $|t_1| \leq b$ forces $|\lambda| = 1$. Consequently the stopping pair passes both checks and normalization returns exactly (a, b). If either check fails, no valid pair exists.

For the cost, remainders at least halve every two Euclidean divisions: if $r_1 \le r_0/2$ this is immediate, and otherwise the next remainder is $r_0 - r_1 < r_0/2$. Hence the loop has $O(\log M)$ iterations. The final gcd test has the same step bound. Consecutive pairs also satisfy

$$|r_0 t_1 - r_1 t_0| = M, \qquad M = r_0|t_1| + r_1|t_0|,$$

because their determinant is preserved up to sign and the coefficients alternate in sign. Since $r_0 \ge 1$, all stored t_i have magnitude at most M, as do the remainders. This proves the storage claim. □

Bounds and certification. A symmetric choice satisfying the *strict* uniqueness condition is

$$A = B = \left\lfloor \sqrt{\frac{M-1}{2}} \right\rfloor \qquad (M \ge 3).$$

The denominator check must not be omitted. For example, with $M = 5$, $x = 2$, and $A = B = 1$, Euclid stops at $(r_1, t_1) = (1, -2)$. Its fraction $-1/2$ has the correct residue, but its denominator is outside the prescribed range, so the answer is Fail.

When numerator and denominator bounds are known, accumulate CRT moduli or precision using Hensel lifting until $M > 2AB$, then reconstruct.

4.6.4 Modular Square Roots

Let p be an odd prime. Given $a \in \mathbb{F}_p$, we want to find

$$x^2 = a,$$

or determine that no solution exists.

If $p \equiv 3 \pmod 4$, Euler's criterion immediately gives the particularly simple formula

$$x = a^{(p+1)/4},$$

since

$$x^2 = a^{(p+1)/2} = a \cdot a^{(p-1)/2} = a.$$

For general odd primes, Cipolla's algorithm gives an equally simple $O(\log p)$ expected-time algorithm.

Lemma 62. *Let d be a quadratic non-residue modulo p, and let*

$$K = \mathbb{F}_p[\omega]/(\omega^2 - d).$$

Then

$$\omega^p = -\omega.$$

Consequently,

$$(u + v\omega)^p = u - v\omega.$$

Proof. By Euler's criterion,

$$d^{(p-1)/2} = -1.$$

Thus

$$\omega^p = \omega(\omega^2)^{(p-1)/2} = \omega d^{(p-1)/2} = -\omega.$$

Also, in characteristic p,

$$(u + v\omega)^p = u^p + v^p \omega^p = u - v\omega.$$

□

Theorem 63 (Cipolla's Algorithm)**.** *Suppose $a \neq 0$ is a quadratic residue modulo the odd prime p. Choose $t \in \mathbb{F}_p$ such that*

$$d = t^2 - a$$

is a quadratic non-residue. In

$$K = \mathbb{F}_p[\omega]/(\omega^2 - d),$$

define

$$x = (t + \omega)^{(p+1)/2}.$$

Then $x \in \mathbb{F}_p$ and

$$x^2 = a.$$

Proof. By Lemma 62,

$$(t + \omega)^p = t - \omega.$$

Therefore

$$x^2 = (t + \omega)^{p+1} = (t + \omega)(t - \omega) = t^2 - \omega^2 = t^2 - d = a.$$

Furthermore,

$$x^{p-1} = (x^2)^{(p-1)/2} = a^{(p-1)/2} = 1$$

by Euler's criterion. Hence

$$x^p = x,$$

so x is fixed by Frobenius and therefore lies in $\mathbb{F}_p$. □

Arithmetic in the quadratic extension is easily computable:

$$(u + v\omega)(r + s\omega) = (ur + dvs) + (us + vr)\omega.$$

Thus the exponentiation in Theorem 63 is ordinary binary exponentiation, using only $O(\log p)$ operations in $\mathbb{F}_p$.

A random choice of t succeeds with constant probability. In fact, exactly $(p-1)/2$ values of t make $t^2 - a$ a non-residue.

Proof. Let χ denote the Legendre symbol. The number of pairs (t, y) satisfying

$$y^2 = t^2 - a$$

is $p - 1$: indeed,

$$(t - y)(t + y) = a,$$

and every choice of nonzero $u = t - y$ uniquely determines

$$t + y = a/u.$$

On the other hand, for fixed t, the number of choices of y is

$$1 + \chi(t^2 - a).$$

Hence

$$\sum_{t \in \mathbb{F}_p} \chi(t^2 - a) = -1.$$

Since a is a nonzero square, exactly two choices of t give $t^2 - a = 0$. If R and S denote the numbers giving a nonzero residue and a non-residue, then

$$R + S = p - 2, \qquad R - S = -1.$$

Thus

$$S = \frac{p-1}{2}.$$

□

Algorithm 4 Modular Square Root

Input: Odd prime p, $a \in \mathbb{F}_p$
Output: x with $x^2 = a$, or None if no root exists

if $a = 0$ **then**
  return 0
if $a^{(p-1)/2} \neq 1$ **then**
  return None
if $p \equiv 3 \pmod 4$ **then**
  return $a^{(p+1)/4}$
repeat
  Choose $t \in \mathbb{F}_p$ uniformly at random
  $d \leftarrow t^2 - a$
  if $d = 0$ **then**
    return t
until $d^{(p-1)/2} = -1$
Work in $\mathbb{F}_p[\omega]/(\omega^2 - d)$
$x \leftarrow (t + \omega)^{(p+1)/2}$
return x

The expected number of choices of t is $O(1)$, and every test and the final exponentiation take $O(\log p)$ field operations. Thus the expected running time is $O(\log p)$.

4.6.5 Generalized Floor Sum

For nonnegative integers a, b, n, r, s and $m > 0$, define

$$F_{r,s}(a, b; m, n) = \sum_{x=0}^{n-1} x^r \left\lfloor \frac{ax + b}{m} \right\rfloor^s.$$

The ordinary floor sum is

$$F_{0,1}(a, b; m, n).$$

We use

$$P_r(t) = \sum_{x=0}^{t-1} x^r,$$

viewed as the usual degree-$(r + 1)$ power-sum polynomial.

The algorithm follows the same idea as Euclid's algorithm. First remove integer parts of a/m and b/m. Once

$$0 \leq a, b < m,$$

transpose the lattice points underneath the line

$$y = \frac{ax + b}{m}.$$

The useful point is that after the transposition, the new slope is m/a, so the pair (a, m) undergoes an ordinary Euclidean step.

Removing Integer Parts Write

$$a = qm + a_0, \qquad b = tm + b_0,$$

with

$$0 \le a_0, b_0 < m.$$

Then

$$\left\lfloor \frac{ax+b}{m} \right\rfloor = qx + t + \left\lfloor \frac{a_0 x + b_0}{m} \right\rfloor.$$

Expanding the power gives

$$\boxed{F_{r,s}(a,b;m,n) = \sum_{j=0}^{s} \binom{s}{j} \sum_{\ell=0}^{s-j} \binom{s-j}{\ell} q^{\ell} t^{s-j-\ell} F_{r+\ell,j}(a_0, b_0; m, n).}$$

Thus we may reduce immediately to

$$0 \le a, b < m.$$

The Reciprocal Step Assume now

$$0 < a < m, \qquad 0 \le b < m.$$

Set

$$h = \left\lfloor \frac{an+b}{m} \right\rfloor, \qquad c = (an+b) \bmod m.$$

Thus

$$an + b = mh + c.$$

The geometric motivation is to regard

$$q_x = \left\lfloor \frac{ax+b}{m} \right\rfloor$$

as the height of the x-th column and then transpose the resulting lattice diagram. Algebraically, write

$$q_x^s = \sum_{t=1}^{q_x} \left(t^s - (t-1)^s \right).$$

For $t = h - y$, the condition $q_x \ge t$ is

$$ax + b \ge m(h-y).$$

Using $an + b = mh + c$,

$$\begin{aligned} x &\ge \left\lceil \frac{m(h-y) - b}{a} \right\rceil \\ &= n - \left\lfloor \frac{my + c}{a} \right\rfloor. \end{aligned}$$

Define

$$u_y = \left\lfloor \frac{my+c}{a} \right\rfloor.$$

Then the contributing x's are exactly

$$n - u_y, \ldots, n-1.$$

Now define the two polynomials

$$D_s(Y) = (h-Y)^s - (h-Y-1)^s = \sum_{i=0}^{s-1} d_i Y^i,$$

and

$$R_r(U) = \sum_{j=1}^{U} (n-j)^r = P_r(n) - P_r(n-U) = \sum_{j=1}^{r+1} e_j U^j.$$

Theorem 64 (Reciprocal Floor-Sum Recurrence)**.** *For $s > 0$,*

$$\boxed{F_{r,s}(a,b;m,n) = \sum_{i=0}^{s-1} \sum_{j=1}^{r+1} d_i e_j F_{i,j}(m,c;a,h).}$$

Proof. Using the finite-difference expansion of q_x^s and swapping the order of summation,

$$\begin{aligned} F_{r,s}(a,b;m,n) &= \sum_{x=0}^{n-1} x^r \sum_{t=1}^{q_x} (t^s - (t-1)^s) \\ &= \sum_{t=1}^{h} (t^s - (t-1)^s) \sum_{\substack{0 \le x < n \\ q_x \ge t}} x^r. \end{aligned}$$

Substitute $t = h - y$. By the preceding calculation,

$$q_x \ge h - y \iff n - u_y \le x < n,$$

where

$$u_y = \left\lfloor \frac{my + c}{a} \right\rfloor.$$

Hence

$$F_{r,s} = \sum_{y=0}^{h-1} D_s(y) R_r(u_y).$$

Expanding the two polynomials,

$$D_s(y) = \sum_i d_i y^i, \qquad R_r(u_y) = \sum_j e_j u_y^j,$$

gives exactly

$$\sum_{i,j} d_i e_j \sum_{y=0}^{h-1} y^i \left\lfloor \frac{my + c}{a} \right\rfloor^j.$$

The inner sum is

$$F_{i,j}(m,c;a,h).$$

□

The base case is simply

$$F_{r,0}(a,b;m,n) = P_r(n).$$

If $a = 0$ after reduction, then the floor is constant and the answer is immediate.

For the ordinary floor sum, $r = 0, s = 1$,

$$D_1(Y) = 1, \qquad R_0(U) = U,$$

so Theorem 64 reduces to the particularly simple identity

$$\boxed{F_{0,1}(a,b;m,n) = F_{0,1}(m,c;a,h).}$$

After the reciprocal step, reducing the new numerator m modulo the new denominator a performs exactly a Euclidean-algorithm step on (m, a). Thus there are only

$$O(\log m)$$

levels.

For any fixed powers r, s, the running time is therefore $O(\log m)$. If all

$$F_{i,j}, \qquad i + j \le K,$$

are computed together, the straightforward implementation takes

$$O(K^4 \log m)$$

arithmetic operations. The transformations are polynomial convolutions, so this dependence on K can be reduced further with fast polynomial multiplication.

4.6.6 Multiplicative Function Prefix Sums

Let f, g be arithmetic functions and let

$$c = f * g$$

be their Dirichlet convolution:

$$c(n) = \sum_{d|n} f(n/d)g(d).$$

Write their prefix sums as

$$F(x) = \sum_{n \le x} f(n), \qquad G(x) = \sum_{n \le x} g(n), \qquad C(x) = \sum_{n \le x} c(n).$$

Suppose $G(x)$ and $C(x)$ can be evaluated quickly and $g(1)$ is invertible. Then $F(x)$ can be recovered much faster than evaluating every value of f up to x.

Lemma 65 (Prefix Sum from Dirichlet Convolution)**.** *For every x,*

$$C(x) = \sum_{d=1}^{x} g(d) F\left(\left\lfloor \frac{x}{d} \right\rfloor\right).$$

Consequently,

$$\boxed{F(x) = \frac{C(x) - \displaystyle\sum_{d=2}^{x} g(d) F\left(\left\lfloor \frac{x}{d} \right\rfloor\right)}{g(1)}.}$$

Proof. Expand and swap the order of summation:

$$\begin{aligned} C(x) &= \sum_{n \le x} \sum_{d|n} f(n/d)g(d) \\ &= \sum_{d \le x} g(d) \sum_{k \le x/d} f(k) \\ &= \sum_{d \le x} g(d) F\left(\left\lfloor \frac{x}{d} \right\rfloor\right). \end{aligned}$$

Separating the $d = 1$ term gives the recurrence. □

The crucial optimization is that

$$\left\lfloor \frac{x}{d} \right\rfloor$$

takes only $O(\sqrt{x})$ distinct values. If

$$q = \left\lfloor \frac{x}{d} \right\rfloor,$$

then the largest index having the same quotient is

$$r = \left\lfloor \frac{x}{q} \right\rfloor.$$

Therefore the whole interval $d, \dots, r$ contributes

$$\big(G(r) - G(d-1)\big)F(q).$$

Precompute $F(x)$ for

$$x \le B$$

using the linear sieve, and memoize all larger recursive values.

Algorithm 5 Multiplicative Function Prefix Sum

Input: x, prefix functions G, C, table of $F(t)$ for $t \le B$
Output: $F(x)$

```
function PrefixF(x)
    if x ≤ B then
        return F_small[x]
    if x is memoized then
        return memo[x]
    A ← C(x)
    l ← 2
    while l ≤ x do
        q ← ⌊x/l⌋
        r ← ⌊x/q⌋
        A ← A − (G(r) − G(l − 1)) · PrefixF(q)
        l ← r + 1
    A ← A/g(1)
    memo[x] ← A
    return A
```

Theorem 66. *If $G(x)$ and $C(x)$ are evaluable in $O(1)$ arithmetic operations and $F(1), \dots, F(B)$ are precomputed, then Algorithm 5 computes $F(n)$ in*

$$O\left(B + \frac{n}{\sqrt{B}}\right)$$

arithmetic operations. Taking

$$B = n^{2/3}$$

gives

$$O(n^{2/3}).$$

Proof. All recursive arguments larger than B are of the form

$$\left\lfloor \frac{n}{k} \right\rfloor > B,$$

so there are at most

$$\frac{n}{B}$$

such memoized states.

For

$$x = \left\lfloor \frac{n}{k} \right\rfloor,$$

one call performs $O(\sqrt{x})$ quotient blocks. Hence the total work on large states is

$$\begin{aligned}\sum_{k \le n/B} O\left(\sqrt{\frac{n}{k}}\right) &= O\left(\sqrt{n} \sum_{k \le n/B} k^{-1/2}\right) \\ &= O\left(\sqrt{n}\sqrt{\frac{n}{B}}\right) \\ &= O\left(\frac{n}{\sqrt{B}}\right).\end{aligned}$$

Adding the $O(B)$ preprocessing gives the claim. □

Two standard examples are immediate.

Since

$$\varphi * 1 = \mathrm{id},$$

we may take

$$g(n) = 1, \qquad G(x) = x, \qquad C(x) = \sum_{n \le x} n = \frac{x(x+1)}{2}$$

to compute

$$\sum_{n \le x} \varphi(n)$$

in $O(x^{2/3})$ time.

Similarly,

$$\mu * 1 = \varepsilon$$

gives

$$\sum_{n \le x} \mu(n)$$

from

$$G(x) = x, \qquad C(x) = 1.$$

4.6.7 LCM and GCD Convolution

Now take

$$P = \{1, \dots, N\}$$

ordered by divisibility:

$$a \le b \iff a \mid b.$$

Then

$$a \vee b = \mathrm{lcm}(a, b), \qquad a \wedge b = \gcd(a, b).$$

Hence

$$(f *_{\mathrm{LCM}} g)(k) = \sum_{\mathrm{lcm}(a,b)=k} f(a)g(b)$$

is join convolution and is diagonalized by the divisor zeta transform

$$\boxed{(\zeta_{\text{div}} f)(k) = \sum_{d|k} f(d).}$$

Likewise

$$(f *_{\text{GCD}} g)(k) = \sum_{\gcd(a,b)=k} f(a)g(b)$$

is meet convolution and is diagonalized by the multiple zeta transform

$$\boxed{(\zeta_{\text{mult}} f)(k) = \sum_{\substack{j \le N \\ k|j}} f(j).}$$

Both transforms can be implemented by processing one prime (this can be viewed as a dimension) at a time.

For divisor zeta, for every prime p, process multiples x of p in increasing order:

$$\boxed{F(x) \leftarrow F(x) + F(x/p).}$$

Möbius inversion processes in the reverse direction with subtraction.

The total number of updates is

$$\sum_{p \le N} \left\lfloor \frac{N}{p} \right\rfloor = O(N \log\log N).$$

For the multiple transform, process the same prime chains in the opposite direction:

$$\boxed{F(x) \leftarrow F(x) + F(px).}$$

Again inversion reverses the order and changes addition to subtraction.

Thus GCD and LCM convolution take

$$\boxed{O(N \log\log N)}$$

ring operations.

4.6.8 Erdős–Ginzburg–Ziv

The theorem asks for both a cardinality and a congruence: among $2n-1$ integers, choose exactly n whose sum is divisible by n. For a prime modulus, pairing the input enforces the cardinality automatically. The remaining task looks like subset sum, but we need much less than its full reachable set: one new representable residue per pair suffices. A binary search finds that new residue without assuming any monotonicity of the membership pattern.

Theorem 67 (Constructive Erdős–Ginzburg–Ziv)**.** *Given $2n-1$ integers, one can deterministically find n distinct input positions whose values sum to zero modulo n in $O(n \log(2n))$ arithmetic operations and $O(n)$ space.*

Reduce the input values modulo n first.

Lemma 68 (Finding a new translate)**.** *Let p be prime, let $S \subseteq \mathbb{Z}_p$ contain 0 but not t, and let $d \neq 0$ modulo p. With constant-time membership queries to S, one can find $z \notin S$ with $z - d \in S$ in $O(\log p)$ time.*

Proof. Since d is invertible, the progression $0, d, 2d, \ldots$ visits every residue. Put $b = td^{-1} \bmod p$, so $1 \le b < p$. At the endpoints of the integer interval (with respect to the AP) $[0, b]$, the membership answers are respectively yes and no. Bisect the interval, retaining a left endpoint whose answer is yes and a right endpoint whose answer is no. When they become consecutive, the right endpoint times d is the desired z.

Each query halves the interval length. The modular inverse is computed by the extended Euclidean algorithm in $O(\log p)$ time. □

Algorithm 6 A boundary along a cyclic progression

```
function NewResidue(p, S, d, t)
    a ← 0; b ← (td^{-1} mod p)
    while b − a > 1 do
        c ← ⌊(a + b)/2⌋
        if cd mod p ∈ S then
            a ← c
        else
            b ← c
    return bd mod p
```

Sort the $2p - 1$ residues, writing their values as $b_1 \le \cdots \le b_{2p-1}$. If any p consecutive values are equal, their sum is already divisible by p. Otherwise, keep b_p fixed and choose one item from each pair (b_i, b_{p+i}), $1 \le i < p$. Every such choice uses exactly p positions. Start by taking $b_1, \ldots, b_p$, whose sum is s. Switching pair i increases the sum by $d_i = b_{p+i} - b_i \not\equiv 0 \pmod p$: equality would give at least p consecutive equal residues. We therefore need a subset of the $p - 1$ nonzero differences with sum $t = -s \pmod p$.

Maintain only a set S of known representable residues, initially $\{0\}$. After processing i differences, every member of S must be representable using those differences. Unless t is already present, apply Lemma 68 to d_i. It supplies a new residue $z = (z - d_i) + d_i$ whose predecessor $z - d_i$ was already representable. Add just z, recording that predecessor and the index i. We do not add the rest of $S + d_i$.

Each unsuccessful step adds one new residue. After $p - 1$ steps we would have all p residues, so t must be found. This proves the prime case and explains the speedup: the theorem only needs guaranteed growth, not every sum that became reachable at each step.

Algorithm 7 Finding a zero-sum p-subset for prime p

function PRIMEEGZ($p, a_1, \ldots, a_{2p-1}$)
  Sort the position labels $q_1, \ldots, q_{2p-1}$ by $a_{q_i} \bmod p$
  $b_i \leftarrow a_{q_i} \bmod p$ for every i
  for $i = 1, \ldots, p$ **do**
    if $b_i = b_{i+p-1}$ **then**
      return $\{q_i, \ldots, q_{i+p-1}\}$
  $t \leftarrow -\sum_{i=1}^{p} b_i \bmod p$; $S \leftarrow \{0\}$, stored as a Boolean array
  for $i = 1, \ldots, p-1$ **do**
    if $t \in S$ **then**
      break
    $d_i \leftarrow (b_{p+i} - b_i) \bmod p$
    $z \leftarrow$ NEWRESIDUE(p, S, d_i, t)
    prev$[z] \leftarrow (z - d_i) \bmod p$; pair$[z] \leftarrow i$
    Insert z into S
  $I \leftarrow \{q_1, \ldots, q_p\}$; $z \leftarrow t$
  while $z \neq 0$ **do**
    $i \leftarrow$ pair$[z]$
    Replace q_i by q_{p+i} in I
    $z \leftarrow$ prev$[z]$
  return I

Every predecessor was present strictly earlier, so pair indices decrease along the reconstruction path. No pair is switched twice. The switches sum to t, giving total sum $s + t = 0 \pmod p$. Sorting and the at most $p - 1$ boundary searches cost $O(p \log p)$; the Boolean and predecessor arrays use $O(p)$ space.

Composite Moduli. Suppose $n = pq$, with p prime. Repeatedly take a pool of $2p - 1$ unused positions, use PrimeEGZ to remove a group G_j of p positions whose sum is divisible by p, and retain the $p - 1$ unused positions in the pool. Adding p fresh positions prepares the next call. Starting with $p - 1$ positions and making $2q - 1$ such calls uses exactly $(p - 1) + p(2q - 1) = 2pq - 1$ input positions.

The groups are disjoint. Their integer quotients $c_j = p^{-1} \sum_{i \in G_j} a_i$, $1 \le j \le 2q - 1$, form an EGZ instance for modulus q. Select q of these quotients with sum divisible by q; the union of the corresponding groups has pq positions and sum divisible by pq.

Algorithm 8 EGZ for an arbitrary modulus

function EGZ($n, a_1, \ldots, a_{2n-1}$)
  if $n = 1$ **then**
    return $\{1\}$
  Find the least divisor $p \ge 2$ of n by trial division up to $\sqrt{n}$
  if no divisor was found **then**
    return PRIMEEGZ($n, a_1, \ldots, a_{2n-1}$)
  $q \leftarrow n/p$; $P \leftarrow$ the first $p - 1$ position labels
  for $j = 1, \ldots, 2q - 1$ **do**
    Append the next p fresh position labels to P
    Apply PrimeEGZ to the values at P, obtaining position set $G_j \subseteq P$
    $P \leftarrow P \setminus G_j$; $c_j \leftarrow \left(\sum_{i \in G_j} a_i\right)/p$
  $J \leftarrow$ EGZ($q, c_1, \ldots, c_{2q-1}$)
  return $\bigcup_{j \in J} G_j$

Proof of Theorem 67. The prime construction proves existence and correctness for prime moduli. The

disjoint-group construction reduces a composite modulus to a smaller one, and the modulus-one case is immediate. Strong induction proves correctness for every n.

The least proper divisor is prime. At a composite call the $2q-1$ prime subproblems therefore take $O(qp\log p) = O(n\log p)$ time in total, and assembling the groups and quotients takes $O(n)$. Only the final call at modulus $q = n/p \le n/2$ is recursive. Every level costs at most its current modulus times $O(\log(2n))$, while the successive moduli form a geometrically decreasing sequence. Their sum is at most $2n$, proving $O(n\log(2n))$ total time. Trial division is smaller than this bound. The stored group lists across the recursion also have geometrically decreasing total size, so the space usage is $O(n)$. □

4.6.9 Modular Subset Sum

Given residues $a_1, \ldots, a_n \in \mathbb{Z}_m$, we want every residue obtained by a subset of the input positions. The ordinary recurrence repeatedly adds a translate of the reachable set. Its cost comes from scanning all m residues at every step, even though most of them have not changed. Hashing lets us skip unchanged intervals.

Theorem 69 (Modular subset sum)**.** *For any fixed $c > 0$, all subset sums modulo m can be computed in $O(n + m\log^2(2m))$ word operations with probability at least $1-(m+2)^{-c}$. The algorithm uses $O(m)$ working space besides storage for input-position labels. It also stores predecessors from which a subset representing any reported sum can be recovered. Hash collisions can omit reachable sums, but never create an invalid witness.*

The word size is $O(\log(n+m))$ bits for fixed c, and the input residues are already reduced modulo m. We include the empty subset, of sum zero. The case $m = 1$ is immediate, so assume $m \ge 2$ below. Multiplicities can be stored in an array of length m after an $O(n+m)$ input pass.

Let S be the reachable set before processing an item of value x. Bellman's recurrence is

$$S \leftarrow S \cup (S + x).$$

The genuinely new residues are $C = (S+x) \setminus S$. Comparing the two characteristic strings naturally reports the symmetric difference $D = S \triangle (S+x)$, including both directions of disagreement. This costs only a factor of two: translation preserves cardinality, so $|D| = 2|C|$. Every successful update permanently adds at least one residue. Across the whole execution, $\sum |C| \le m-1$.

We must also bound steps with no new residues, especially when $n \gg m$. Process equal values consecutively. As soon as $S \cup (S+x) = S$, we have $S + x = S$ and every remaining copy of x can be skipped. Thus there are at most $m-1$ successful updates and at most one final unsuccessful update per value. Reordering the input does not change its subset sums.

Two implementation rules are essential. First, find the entire difference against the *old* S before setting any new bits; updating during the comparison could use the same input copy more than once. Second, for each new residue r, record its old predecessor $(r-x) \bmod m$ and the current input-position label. Every predecessor was reached at an earlier step, so reconstruction terminates at zero and never reuses an input position.

Write $s[0], \ldots, s[m-1]$ for the characteristic string of S. The characteristic string of $S+x$ has entry $s[(i-x) \bmod m]$ at position i. To compare these strings on an interval, store the doubled string ss and use a polynomial fingerprint. For a base $\alpha \neq 0$ in a large prime field, the normalized hash of $D[l:r]$ is

$$\operatorname{hash}(D[l:r]) = \alpha^{-l} \sum_{i=l}^{r-1} D[i]\alpha^i.$$

The normalization makes hashes of equally long strings comparable even when their starting positions differ. Precompute powers and inverse powers of α, and keep the weighted values $D[i]\alpha^i$ in a Fenwick tree or Segtree. A bit insertion updates its two copies in $O(\log m)$ time; an interval hash takes two prefix-sum queries and therefore $O(\log m)$ time.

Compare $s[l:r]$ with the length-$(r-l)$ interval of ss beginning at $(l-x) \bmod m$. If their hashes agree, skip the interval. Otherwise bisect it and recurse; at a singleton, inspect the two actual bits. The visited

nodes lie on paths to disagreements, together with their siblings, so d disagreements cost $O((d+1)\log m)$ hash comparisons. This is $O((d+1)\log^2 m)$ time. By $|D| = 2|C|$ and the bound on the number of updates, the total is $O(m \log^2 m)$ after reading the input, provided no unequal compared strings collide.

4.6.10 Modular Subset Sum using Runs

Let c_x be the number of available copies of residue $x \in \mathbb{Z}_m$. We want every attainable subset sum, including zero from the empty subset. Bellman's update for one copy of x is $S \leftarrow S \cup (S + x)$. The important freedom is to change the order in which we represent S: if consecutive positions differ by x, this update just extends each occupied run by one position. The boundaries needed to construct this new representation are exactly the differences between S and $S + x$. Thus discovering the update also prepares a compact representation for the next one.

Theorem 70 (Modular subset sum by runs)**.** *From the multiplicities $c_0, \ldots, c_{m-1}$, all subset sums modulo m can be computed deterministically in $O(m \log(2m))$ word operations and $O(m)$ working space. Predecessors give a witness for any attainable target. For an explicit list of n items, forming the multiplicities adds $O(n + m)$ time.*

The case $m = 1$ is immediate. Henceforth $m \geq 2$, and zero-valued items are discarded.

Runs along an addition cycle. Fix a nonzero direction d. The permutation $z \mapsto z + d$ splits $\mathbb{Z}_m$ into $g = \gcd(d, m)$ cycles of length $L = m/g$. Store the occupied positions of each cycle as maximal cyclic runs, and keep the ordinary membership array of S. Full and empty cycles have separate flags and no stored runs. A partial cycle is neither full nor empty; maintain a list of partial cycles so that the other cycles can be skipped.

For example, when m is prime there is just one cycle, $0, d, 2d, \ldots, (m-1)d$, and the representation is an ordinary sorted list of occupied intervals in this cyclic order. For general m, we use one such list per partial cycle. A wrapping run may be split into two ordinary half-open intervals at coordinate zero.

Write $\rho_d(S)$ for the total number of cyclic runs in partial cycles. Every run has a distinct first unoccupied position after it, so

$$\boxed{\rho_d(S) = |(S + d) \setminus S|.} \tag{2}$$

This identity links the size of the representation to growth under this iteration.

Lemma 71 (Growth pays for the next representation)**.** *For $S' = S \cup (S + x)$ and $\Delta = |S' \setminus S|$,*

$$|S \triangle (S + x)| = 2\Delta, \qquad \rho_x(S') \leq \Delta.$$

Proof. The two sets S and $S + x$ have equal cardinality, giving the first identity. In direction x, the update extends each old run by one; runs may merge or fill a cycle, but cannot split. Thus $\rho_x(S') \leq \rho_x(S) = \Delta$. □

The next update will scan only these runs. Even if it uses a different direction, their number is already paid for by the previous update's new residues.

Change direction using only the boundaries. Suppose S is currently stored in direction d, but the next item is x. We first compare S and $S + x$ in the *old* order, where both are readily available as interval lists. Then we use their differences to reconstruct S in the *new* order.

Indeed, in the x-order a position z is a run start exactly when $z \in S$ and $z - x \notin S$. It is an exclusive run end exactly when $z \notin S$ and $z - x \in S$. Hence the tagged boundaries are

$$P = S \setminus (S + x) \quad \text{(starts)}, \qquad Q = (S + x) \setminus S \quad \text{(ends)}.$$

These sets both have size Δ. Sort their 2Δ elements by their positions in the new cycles. Within a partial cycle, starts and ends alternate and uniquely determine its occupied intervals. A cycle with no boundary is either full or empty, distinguished by one membership lookup. We have reconstructed the entire set from its boundaries, not by sorting all its elements.

Comparisons. Here are explicit formulas implementing the change of coordinates. With $g = \gcd(d, m)$, $L = m/g$, and $u = d/g$, represent cycle $c \in [0, g)$ by

$$\phi_d(c, t) = c + g(ut \bmod L), \qquad 0 \le t < L.$$

The inverse coordinate map is $c = z \bmod g$ and $t = ((z - c)/g)u^{-1} \bmod L$. The inverse exists because $\gcd(u, L) = 1$ and is computed once for each input direction by the extended Euclidean algorithm.

Translation by x sends an old cycle to another old cycle, rotating its interval list:

$$\boxed{c' = (c + x) \bmod g, \qquad t' = t + \frac{c + x - c'}{g} u^{-1} \pmod L.} \tag{3}$$

To check this, substitute in $\phi_d(c', t')$ and use $uu^{-1} \equiv 1 \pmod L$; the result is $\phi_d(c, t) + x \pmod m$. A cyclic shift preserves the order of an interval list up to one cut. Split an interval at zero if necessary and concatenate the wrapped and unwrapped portions. This takes linear time in the list length, without sorting.

For each destination cycle, merge-sweep its old interval list and its translated source list. Between successive endpoints, membership in both sets is constant. Skip equal stretches; on an unequal stretch, emit each position with its start or end tag. The work is linear in the number of scanned intervals plus the number of emitted positions.

If $h = \gcd(x, m)$ equals g, then $g \mid x$ and every old cycle stays in place. Its full and empty states cannot change, so only partial cycles need scanning or rebuilding. Moreover, x/g generates each cycle, so every partial cycle has boundaries in the new order. If $h \ne g$, scan all g old cycles and initialize all h new cycle records.

Lemma 72 (One update)**.** *Given the run representation of S in direction d, one Bellman update with x takes*

$$O(1 + \rho_d(S) + \Delta \log(2 + \Delta) + \mathbf{1}_{g \ne h}(g + h))$$

word operations, where $g = \gcd(d, m)$, $h = \gcd(x, m)$, and $\Delta = |(S + x) \setminus S|$. The resulting set is stored in direction x with at most Δ partial runs.

Proof. The old-order sweep costs $O(1 + \rho_d(S) + \Delta)$ if $g = h$, and an additional $O(g)$ otherwise. Each source and destination list is scanned a constant number of times. Sorting the 2Δ new-coordinate boundary keys costs $O(\Delta \log(2 + \Delta))$; pairing them reconstructs all partial cycles in $O(\Delta)$ time. When $g \ne h$, boundary-free cycles are initialized from their representative membership bits in $O(h)$ time. When $g = h$, retain the existing full/empty flags.

Now extend each reconstructed run by one position and merge touching runs. This computes $S \cup (S + x)$ in $O(\Delta)$ time, since the number of old runs in direction x is exactly Δ. Full and empty cycles are unchanged. The new membership bits are precisely Q; record their old predecessors before setting any of them. The preceding lemma bounds the resulting number of runs. □

Algorithm 9 One Bellman update using addition-cycle runs

function RunUpdate($S, \mathcal{I}_d, x, \text{copy}$)
  $g \leftarrow \gcd(d, m)$; $h \leftarrow \gcd(x, m)$
  $P, Q \leftarrow \emptyset$
  for each old source cycle c (only partial cycles if $g = h$) **do**
    Compute its destination c' and rotation by (3)
    Rotate its sorted interval list by splitting at zero
    Merge-sweep it against the old list in cycle c'
    Emit old-only positions into P and translated-only positions into Q
  Convert tagged P, Q to x-cycle coordinates and sort by cycle and position
  if $g \neq h$ **then**
    Initialize each new cycle c as full or empty from the old bit $S[c]$
  else
    Retain full and empty cycle records
  for each cycle containing boundary events **do**
    Pair consecutive cyclic starts and ends to reconstruct its old runs
    Extend every run by one and merge; flag a cycle if it becomes full
  for each $z \in Q$ **do**
    parent$[z] \leftarrow z - x \pmod m$; item$[z] \leftarrow (x, \text{copy})$
  Set all bits $S[z]$ for $z \in Q$; replace d by x
  return $|Q|$

In the reconstruction, a final start pairs with an initial end across zero; store the wrapping interval as a suffix and a prefix.

Group equal gcds. The potentially expensive cycle initialization happens only when the gcd changes. Process all values with the same gcd consecutively. Each possible gcd is a divisor of m, so their total contribution is bounded by

$$\sum_{g|m} g = m \sum_{q|m} \frac{1}{q} \leq m \sum_{q=1}^{m} \frac{1}{q} = O(m \log(2m)). \tag{4}$$

If an update by x makes no change, then $S + x = S$. Iterating translation shows that S is a union of the cycles generated by x, namely the residue classes modulo $h = \gcd(x, m)$. Every other value with this same gcd preserves those classes. We may therefore skip the rest of the entire gcd class, not only the remaining copies of x.

Algorithm 10 Deterministic modular subset sum

Input: Modulus $m \geq 2$ and multiplicities c_x for $0 \leq x < m$

Group nonzero values with positive multiplicity by $\gcd(x, m)$
Precompute $(x/g)^{-1} \bmod (m/g)$ for every such value, where $g = \gcd(x, m)$
$S \leftarrow \{0\}$; $d \leftarrow 1$; initialize the one run $[0, 1)$
for each nonempty gcd class g **do**
    finished $\leftarrow$ false
    for each value x in this class **do**
        for $r = 1, \ldots, c_x$ **do**
            $\Delta \leftarrow \textsc{RunUpdate}(S, \mathcal{I}_d, x, r)$
            if $\Delta = 0$ or $|S| = m$ **then**
                finished $\leftarrow$ true; **break**
        if finished **then**
            break
    if $|S| = m$ **then**
        break
return the membership and predecessor arrays

Proof of Theorem 70. Every executed update is Bellman's recurrence, and the preceding argument justifies all skipped items. This proves correctness. Let Δ_i be the number of residues first reached by update i. Since each nonzero residue is discovered at most once, $\sum_i \Delta_i \leq m - 1$. There are at most $m - 1$ productive updates and at most one unproductive update per gcd class, hence $O(m)$ updates.

The initial representation has one run. After update i, it has at most Δ_i partial runs, so the sum of the run counts scanned at the next updates is at most $1 + \sum_i \Delta_i = O(m)$. There are exactly $2\Delta_i$ boundary keys in update i, giving total sorting cost

$$\sum_i O\big(\Delta_i \log(2 + \Delta_i)\big) \leq O\left(\log(2m) \sum_i \Delta_i\right) = O(m \log(2m)).$$

Each gcd class is entered once, so its gcd contributes at most once as an old value and once as a new value in the cycle-initialization costs. Equation (4) bounds their sum by $O(m \log(2m))$. Computing gcds and modular inverses for at most $m - 1$ values by the Euclidean algorithm takes $O(m \log(2m))$ additional word operations. All other costs are covered by Lemma 72.

Cycle records, intervals, boundary events, and membership and predecessor arrays each use $O(m)$ space. For a target $z \neq 0$, follow parent$[z]$ while recording item$[z]$ until reaching zero. Every predecessor was reachable strictly before the corresponding item copy was used, because all new bits were set only after recording the whole update. Thus the chain terminates, never reuses an input copy, and its item values sum to the target modulo m. □

4.7 Fast Factorials and Deterministic Factorization

To compute the factorial we will obtain values of

$$F_v(x) = (x + 1)(x + 2) \cdots (x + v)$$

at the arithmetic progression

$$0, v, 2v, \ldots.$$

Using a product tree followed by multipoint evaluation gives two logarithmic factors. The special structure lets us do better: we never construct the coefficients of F_v at all. Instead, we recursively construct its values on the arithmetic progression.

The main tool is fast shifting of equally spaced polynomial evaluations.

4.7.1 Fast Factorial

Let p be prime and suppose

$$0 \le n < p.$$

We compute

$$n! \pmod p$$

in

$$O(\sqrt{n}\log n)$$

field operations.

Choose a power of two

$$v = \Theta(\sqrt{n}), \qquad v \ge \sqrt{n}.$$

For

$$d = 1, 2, 4, \ldots, v,$$

define

$$F_d(x) = \prod_{j=1}^{d}(x+j).$$

Rather than storing the coefficients of F_d, store the $d+1$ evaluations

$$\boxed{S_d = \big(F_d(0), F_d(v), F_d(2v), \ldots, F_d(dv)\big).}$$

The base case is

$$S_1 = (1, v+1).$$

The doubling identity is

$$\boxed{F_{2d}(x) = F_d(x)F_d(x+d).}$$

Suppose S_d is known. Using Lemma 502, compute

$$A_i = F_d(iv+d), \qquad 0 \le i \le d.$$

Then

$$F_{2d}(iv) = F_d(iv)F_d(iv+d) = S_d[i]A_i$$

gives the first $d+1$ entries of S_{2d}.

For the remaining entries, compute by two more evaluation shifts

$$B_j = F_d((d+1+j)v),$$

and

$$C_j = F_d((d+1+j)v+d), \qquad 0 \le j \le d.$$

Then

$$B_jC_j = F_{2d}((d+1+j)v).$$

Keeping

$$j = 0, \ldots, d-1$$

gives exactly the remaining d entries.

Thus

$$\boxed{S_d \longrightarrow S_{2d}}$$

uses three evaluation shifts and $O(d)$ pointwise multiplications.

Algorithm 11 Fast Rising Factorial Evaluations

Input: Power of two v

Output:

$$S_v[i] = F_v(iv) \qquad (0 \le i \le v)$$

```
S ← (1, v + 1)
d ← 1
while d < v do
    A ← EvalShift(S, d)                          ▷ A_i = F_d(iv + d)
    B ← EvalShift(S, (d + 1)v)
    C ← EvalShift(S, (d + 1)v + d)
    S' ← empty array
    for i = 0, ..., d do
        append S[i]A[i] to S'
    for j = 0, ..., d − 1 do
        append B[j]C[j] to S'
    S ← S'
    d ← 2d
return S
```

Theorem 73 (Fast Factorial)**.** *For prime p and $n < p = n^{O(1)}$, one can compute*

$$n! \pmod p$$

in

$$\boxed{O(\sqrt{n}\log n)}$$

time.

Proof. At a doubling step of size d, the work is

$$O(d \log d).$$

Therefore

$$T(v) = T(v/2) + v\log v = O(v\log v).$$

Since

$$v = \Theta(\sqrt{n}),$$

this is

$$O(\sqrt{n}\log n).$$

At the end we know

$$F_v(0), F_v(v), \ldots, F_v(v^2).$$

Since

$$q = \left\lfloor \frac{n}{v} \right\rfloor \le v,$$

the complete blocks give

$$(qv)! = \prod_{i=0}^{q-1} F_v(iv).$$

Finally multiply the fewer than v remaining terms

$$qv + 1, \ldots, n.$$

□

4.7.2 Deterministic Integer Factorization

The same construction gives a deterministic $\widetilde{O}(N^{1/4})$-time factorization algorithm.

Let N be composite. If N is even, we immediately find the factor 2, so assume that N is odd.

Let v be the smallest power of two satisfying

$$v \geq N^{1/4}.$$

Thus

$$N^{1/4} \leq v < 2N^{1/4}.$$

Define

$$F_v(x) = \prod_{j=1}^{v}(x+j).$$

Then

$$F_v(iv) = \prod_{j=1}^{v}(iv+j)$$

is the product of the i-th block of v consecutive integers.

Using the fast evaluation construction from Theorem 73, compute

$$\boxed{F_v(0), F_v(v), \ldots, F_v((v-1)v) \pmod N}$$

in

$$O(v \log v)$$

arithmetic operations over $\mathbb{Z}/N\mathbb{Z}$.

The only additional issue is that $\mathbb{Z}/N\mathbb{Z}$ is not a field: the evaluation-shift algorithm may try to divide by a nonunit. For factorization, this causes no difficulty.

Batched Inversions Suppose one stage of the algorithm requires the inverses of

$$a_1, \ldots, a_t \in \mathbb{Z}/N\mathbb{Z}.$$

Compute

$$A = \prod_{i=1}^{t} a_i \pmod N$$

and

$$g = \gcd(A, N).$$

If

$$g = 1,$$

then every a_i is invertible. One extended gcd computes A^{-1}, after which prefix and suffix products recover all

$$a_i^{-1}$$

using $O(t)$ modular multiplications.
If

$$g > 1,$$

split the product into two halves and compute the gcd of each half-product with N. Repeating this isolates a nonunit denominator. Its gcd with N reveals a factor of N.

Thus every required inversion either succeeds or produces a factor, while the batching adds only linear work to each evaluation-shift stage.

Theorem 74 (Strassen–Pollard Factorization)**.** *A nontrivial factor of a composite integer N can be found deterministically in*

$$\boxed{O\left(N^{1/4}(\log N)^2 \log\log N\right)}$$

bit operations, assuming

$$M(\ell) = O(\ell \log \ell)$$

for multiplication of ℓ-bit integers.

Proof. Let p be the smallest prime divisor of N. Since N is composite,

$$p \le \sqrt{N}.$$

By the choice of v,

$$\sqrt{N} \le v^2.$$

Hence p lies among the integers

$$1, 2, \ldots, v^2.$$

Therefore there are

$$0 \le i < v, \qquad 1 \le j \le v$$

such that

$$p = iv + j.$$

For this i,

$$F_v(iv) = \prod_{t=1}^{v} (iv + t)$$

contains p as a factor, and consequently

$$\gcd(F_v(iv), N) > 1.$$

After computing the v block products, calculate

$$g_i = \gcd(F_v(iv), N) \qquad (0 \le i < v).$$

At least one satisfies

$$g_i > 1.$$

If

$$1 < g_i < N,$$

then g_i is already a nontrivial factor.
The remaining possibility is

$$g_i = N.$$

In this case inspect the v integers in that block:

$$iv + 1, \dots, iv + v.$$

At least one is divisible by the smallest prime factor p. Since every one of these integers is at most

$$v^2 = O(\sqrt{N}) < N,$$

its gcd with N is a proper nontrivial factor.
It remains to bound the running time. Put

$$L = \lceil \log_2 N \rceil.$$

All arithmetic takes place modulo N, so every ring element has $O(L)$ bits.
The fast evaluation construction uses

$$O(v \log v)$$

ring operations. A modular multiplication costs

$$O(M(L))$$

bit operations, so the sampling phase costs

$$O\big(v \log v \, M(L)\big).$$

Since

$$v = \Theta(N^{1/4})$$

and

$$\log v = \Theta(\log N) = \Theta(L),$$

this is

$$O\big(N^{1/4} L \, M(L)\big).$$

The final $O(v)$ gcd computations cost

$$O\big(v \, M(L) \log L\big),$$

which is smaller than the sampling cost. The batched inversions likewise contribute only lower-order terms.

Therefore the total bit complexity is

$$O\big(N^{1/4} L \, M(L)\big).$$

Substituting

$$M(L) = O(L \log L)$$

gives

$$\boxed{O\left(N^{1/4}(\log N)^2 \log\log N\right).}$$

□

4.8 Miller–Rabin Primality Test

Let $n > 2$ be odd, and write

$$n - 1 = 2^s d, \qquad d \text{ odd}.$$

The Miller–Rabin test studies the squaring sequence

$$a^d, \quad a^{2d}, \quad a^{4d}, \quad \ldots, \quad a^{2^s d} = a^{n-1} \pmod{n}.$$

For a prime modulus, this sequence has a very restricted form.

Lemma 75. *Let p be an odd prime and write*

$$p - 1 = 2^s d$$

with d odd. For every $a \not\equiv 0 \pmod{p}$, either

$$a^d \equiv 1 \pmod{p},$$

or

$$a^{2^r d} \equiv -1 \pmod{p}$$

for some $0 \le r < s$.

Proof. By Fermat's theorem,

$$a^{2^s d} = a^{p-1} \equiv 1 \pmod{p}.$$

If $a^d \equiv 1$, we are done.

Otherwise, let $j \ge 1$ be the smallest index such that

$$a^{2^j d} \equiv 1 \pmod{p}.$$

Then

$$x := a^{2^{j-1} d}$$

satisfies

$$x^2 \equiv 1 \pmod{p} \qquad \text{but} \qquad x \not\equiv 1 \pmod{p}.$$

Since

$$(x - 1)(x + 1) \equiv 0 \pmod{p}$$

and p is prime,

$$x \equiv -1 \pmod{p}.$$

□

For composite n, a base a satisfying the conclusion of Lemma 75 is called a *strong liar*; otherwise it is a *strong witness* for the compositeness of n.

Algorithm 12 Miller–Rabin Primality Test

Input: Integer $n \geq 2$, number of rounds t
Output: Composite or Probably Prime

```
if n ∈ {2, 3} then
    return Probably Prime
if n < 2 or n is even then
    return Composite
Write n − 1 = 2^s d with d odd
for k = 1, . . . , t do
    Choose a uniformly from {2, . . . , n − 2}
    if gcd(a, n) > 1 then
        return Composite
    x ← a^d mod n
    if x = 1 or x = n − 1 then
        continue
    pass ← false
    for r = 1, . . . , s − 1 do
        x ← x^2 mod n
        if x = n − 1 then
            pass ← true
            break
        if x = 1 then
            return Composite
    if not pass then
        return Composite
return Probably Prime
```

By Lemma 75, a prime is always accepted. The usefulness of the test comes from the fact that very few bases can fool it.

Theorem 76 (Rabin–Monier)**.** *Let n be an odd composite integer. At most one quarter of the nonzero residues modulo n are strong liars.*

Consequently, t independent rounds of the Miller–Rabin test accept a composite n with probability at most

$$\boxed{4^{-t}}.$$

The running time is

$$O\big(t\, M(\log n) \log n\big)$$

bit operations, where $M(\ell)$ is the cost of multiplying two ℓ-bit integers. Using

$$M(\ell) = O(\ell \log \ell),$$

this is

$$\boxed{O\left(t \log^2 n \log\log n\right).}$$

Proof. Write

$$n-1=2^s d, \qquad d \text{ odd},$$

and factor

$$n=\prod_{i=1}^{k} p_i^{e_i}.$$

For each prime divisor p_i, write

$$p_i-1=2^{s_i}u_i, \qquad u_i \text{ odd},$$

and set

$$\nu=\min_i s_i.$$

We first place every strong liar inside a set that is easy to count.
Suppose a is a strong liar. If

$$a^d \equiv 1 \pmod{n},$$

then certainly

$$a^{2^{\nu-1}d} \equiv 1 \pmod{n}.$$

Otherwise, for some $0 \le r < s$,

$$a^{2^r d} \equiv -1 \pmod{n}.$$

Reducing modulo any p_i, the order of a modulo p_i has 2-adic valuation exactly $r+1$. Since this order divides p_i-1,

$$r+1 \le s_i$$

for every i, and hence

$$r+1 \le \nu.$$

Therefore, after squaring if necessary,

$$a^{2^{\nu-1}d} \equiv \pm 1 \pmod{n}.$$

Thus every strong liar belongs to

$$S=\left\{a \in (\mathbb{Z}/n\mathbb{Z})^\times : a^{2^{\nu-1}d} \equiv \pm 1 \pmod{n}\right\}.$$

We now count S.
By the Chinese remainder theorem,

$$(\mathbb{Z}/n\mathbb{Z})^\times \cong \prod_{i=1}^{k} (\mathbb{Z}/p_i^{e_i}\mathbb{Z})^\times.$$

For odd p_i, the i-th group is cyclic of order

$$N_i = p_i^{e_i-1}(p_i-1) = p_i^{e_i-1}2^{s_i}u_i.$$

Set

$$m = 2^{\nu-1} d.$$

Since $p_i \nmid d$,

$$\gcd(m, N_i) = 2^{\nu-1} \gcd(d, u_i).$$

In a cyclic group of order N_i, the equation

$$x^m = 1$$

has exactly $\gcd(m, N_i)$ solutions.
The equation

$$x^m = -1$$

has the same number of solutions. Indeed,

$$\gcd(m, N_i) \mid \frac{N_i}{2},$$

because $s_i \geq \nu$, so -1 lies in the image of the m-th power map.
Hence, writing

$$g_i = \gcd(d, u_i),$$

the CRT gives

$$|S| = 2 \prod_{i=1}^{k} 2^{\nu-1} g_i.$$

Since

$$\varphi(n) = \prod_{i=1}^{k} p_i^{e_i-1} 2^{s_i} u_i,$$

we obtain

$$\frac{\varphi(n)}{|S|} = \frac{1}{2} \prod_{i=1}^{k} p_i^{e_i-1} \left(2^{s_i-\nu+1} \frac{u_i}{g_i} \right).$$

Define

$$R_i = 2^{s_i-\nu+1} \frac{u_i}{g_i}.$$

Each R_i is an even integer, so in particular

$$R_i \geq 2.$$

It remains only to check that

$$\frac{\varphi(n)}{|S|} \geq 4.$$

If $k \geq 3$, then immediately

$$\frac{\varphi(n)}{|S|} \geq \frac{1}{2} 2^k \geq 4.$$

Suppose $k = 2$. If n is not squarefree, some $e_i \geq 2$, and therefore

$$p_i^{e_i - 1} \geq 3.$$

Hence

$$\frac{\varphi(n)}{|S|} \geq \frac{1}{2} \cdot 3 \cdot 2 \cdot 2 > 4.$$

It remains to consider

$$n = pq, \qquad p < q.$$

We always have $R_p \geq 2$.
If

$$s_q > \nu,$$

then

$$R_q \geq 4.$$

Otherwise $s_q = \nu$, so

$$q - 1 = 2^\nu u_q.$$

We claim that $u_q \nmid d$. Indeed, since $s \geq \nu$, if $u_q \mid d$ then

$$q - 1 = 2^\nu u_q \mid 2^s d = n - 1.$$

But

$$n - 1 = pq - 1 \equiv p - 1 \pmod{q - 1},$$

and

$$0 < p - 1 < q - 1,$$

so $q - 1 \nmid n - 1$, a contradiction.
Thus

$$\frac{u_q}{\gcd(u_q, d)}$$

is an odd integer greater than 1, hence at least 3. Therefore

$$R_q \geq 6.$$

So in either case,

$$\frac{\varphi(n)}{|S|} \geq \frac{1}{2} \cdot 2 \cdot 4 = 4.$$

Finally, suppose $k = 1$, so

$$n = p^e, \qquad e \geq 2.$$

Since

$$p - 1 \mid p^e - 1 = n - 1,$$

the odd part u_1 divides d. Hence

$$R_1 = 2,$$

and therefore

$$\frac{\varphi(n)}{|S|} = p^{e-1}.$$

This is at least 4 except when

$$n = 3^2 = 9.$$

For $n = 9$, the only strong liars are

$$\pm 1,$$

so there are exactly

$$2 = \frac{9-1}{4}.$$

Thus in every case the number of strong liars is at most

$$\frac{n-1}{4}.$$

Therefore a random Miller–Rabin round accepts a composite number with probability at most $1/4$, and t independent rounds fail with probability at most

$$4^{-t}.$$

For the running time, let

$$\ell = \Theta(\log n).$$

Computing $a^d \bmod n$ by repeated squaring and then following the Miller–Rabin squaring chain uses

$$O(\log n)$$

modular multiplications. Each operates on $O(\log n)$-bit integers, so one round takes

$$O\big(M(\log n)\log n\big)$$

bit operations. Therefore t rounds take

$$O\big(t\, M(\log n)\log n\big).$$

Using

$$M(\ell) = O(\ell \log \ell)$$

gives

$$O\left(t \log^2 n \log\log n\right).$$

□

5 Combinatorics

5.1 Entropy Method

Throughout this section, entropy is measured in bits.

5.1.1 Shearer's Lemma

Lemma 77 (Shearer's Lemma)**.** *Let*

$$X = (X_1, \dots, X_n)$$

be a random vector, and let

$$\mathcal{A} = \{A_1, \dots, A_m\}$$

be a collection of subsets of $[n]$ *such that every coordinate* $i \in [n]$ *belongs to at least* k *members of* $\mathcal{A}$*. Then*

$$\boxed{kH(X) \le \sum_{j=1}^{m} H(X_{A_j}).}$$

Proof. Order the coordinates as $1, \dots, n$. For every $A \subseteq [n]$, the chain rule gives

$$H(X_A) = \sum_{i \in A} H\left(X_i \mid X_j : j \in A,\ j < i\right).$$

Conditioning on more information can only decrease entropy, so

$$H(X_A) \ge \sum_{i \in A} H(X_i \mid X_1, \dots, X_{i-1}).$$

Summing over $A \in \mathcal{A}$,

$$\begin{aligned}\sum_{A \in \mathcal{A}} H(X_A) &\ge \sum_{A \in \mathcal{A}} \sum_{i \in A} H(X_i \mid X_1, \dots, X_{i-1}) \\ &\ge k \sum_{i=1}^{n} H(X_i \mid X_1, \dots, X_{i-1}) \\ &= kH(X).\end{aligned}$$

□

The interpretation is that if several overlapping projections together cover every coordinate k times, then together they contain at least k copies of the information in the whole object.

5.1.2 Entropy Bound for Binomial Sums

Define the binary entropy function

$$H(p) = -p \log_2 p - (1-p) \log_2 (1-p).$$

Theorem 78. *For*

$$0 \le k \le \frac{n}{2},$$

$$\boxed{\sum_{j=0}^{k} \binom{n}{j} \le 2^{\,nH(k/n)}.}$$

In particular,

$$\binom{n}{k} \le 2^{\,nH(k/n)}.$$

Proof. Let $X \subseteq [n]$ be chosen uniformly from all subsets of size at most k, and let

$$X_i = \mathbf{1}[i \in X].$$

Then

$$H(X_1, \dots, X_n) = \log_2 \sum_{j=0}^{k} \binom{n}{j}.$$

By subadditivity of entropy,

$$H(X_1, \dots, X_n) \le \sum_{i=1}^{n} H(X_i).$$

By symmetry, every X_i has the same probability p of being 1. Moreover,

$$np = \mathbb{E}|X| \le k,$$

so

$$p \le \frac{k}{n} \le \frac{1}{2}.$$

Since the binary entropy function is increasing on $[0, 1/2]$,

$$H(X_i) = H(p) \le H(k/n).$$

Therefore

$$\log_2 \sum_{j=0}^{k} \binom{n}{j} \le nH(k/n),$$

which gives the result. □

5.1.3 Irregular Moore Bound

For a directed edge

$$\vec{e} = (u, v),$$

a *non-returning walk* is a walk that never immediately traverses an edge and then its reverse.

Let $N_i(v)$ be the number of non-returning walks of length i starting at v, and let $N_i(\vec{e})$ be the number of non-returning walks of length $i + 1$ whose first edge is $\vec{e}$.

The entropy argument says that even though the degrees are irregular, the average number of such walks grows as though every vertex had the average degree.

Lemma 79 (Non-Returning Walk Growth)**.** *Let G be a graph of minimum degree at least 2 and average degree*

$$\bar{d}.$$

Define the stationary vertex distribution

$$\pi(v) = \frac{d_v}{\bar{d}\, n}.$$

Then for $i \ge 1$,

$$\boxed{\mathbb{E}_{v \sim \pi} N_i(v) \ge \bar{d}(\bar{d} - 1)^{i-1},}$$

and if $\vec{e}$ is a uniformly random directed edge, then for $i \ge 0$,

$$\boxed{\mathbb{E}_{\vec{e}} N_i(\vec{e}) \ge (\bar{d} - 1)^i.}$$

Proof. We first prove the edge version.

Choose $\vec{e}_0$ uniformly among all directed edges, and then generate

$$\vec{e}_0, \vec{e}_1, \ldots, \vec{e}_i$$

by choosing each next edge uniformly among all non-returning continuations.

Every $\vec{e}_j$ is again uniformly distributed among directed edges, so the head vertex of each $\vec{e}_j$ has distribution π.

Conditioned on $\vec{e}_0 = \vec{e}$, the random continuation

$$(\vec{e}_1, \ldots, \vec{e}_i)$$

has support of size at most $N_i(\vec{e})$. Hence

$$H(\vec{e}_1, \ldots, \vec{e}_i \mid \vec{e}_0 = \vec{e}) \leq \log_2 N_i(\vec{e}).$$

Averaging and using Jensen,

$$\begin{aligned} \log_2 \mathbb{E}_{\vec{e}} N_i(\vec{e}) &\geq \mathbb{E}_{\vec{e}} \log_2 N_i(\vec{e}) \\ &\geq H(\vec{e}_1, \ldots, \vec{e}_i \mid \vec{e}_0). \end{aligned}$$

By the chain rule and the Markov property,

$$H(\vec{e}_1, \ldots, \vec{e}_i \mid \vec{e}_0) = i\, \mathbb{E}_{v \sim \pi} \log_2(d_v - 1).$$

Now

$$\mathbb{E}_{v \sim \pi} \log_2(d_v - 1) = \frac{1}{\bar{d}} \mathbb{E}_{v \text{ uniform}} \left[d_v \log_2(d_v - 1) \right].$$

The function

$$x \longmapsto x \log(x - 1)$$

is convex for $x \geq 2$, so Jensen gives

$$\mathbb{E}_{v \sim \pi} \log_2(d_v - 1) \geq \log_2(\bar{d} - 1).$$

Thus

$$\mathbb{E}_{\vec{e}} N_i(\vec{e}) \geq (\bar{d} - 1)^i.$$

For the vertex version, choose $v \sim \pi$, then the first edge uniformly from the d_v choices, and continue non-returningly. Exactly the same argument gives

$$\log_2 \mathbb{E}_{v \sim \pi} N_i(v) \geq \mathbb{E}_{v \sim \pi} \log_2 d_v + (i - 1) \mathbb{E}_{v \sim \pi} \log_2(d_v - 1).$$

Since

$$x \longmapsto x \log x$$

is convex,

$$\mathbb{E}_{v \sim \pi} \log_2 d_v = \frac{1}{\bar{d}} \mathbb{E}[d_v \log_2 d_v] \geq \log_2 \bar{d}.$$

Combining the bounds gives

$$\mathbb{E}_{v \sim \pi} N_i(v) \geq \bar{d}(\bar{d} - 1)^{i-1}.$$

□

Theorem 80 (Alon–Hoory–Linial Moore Bound)**.** *Let G be a simple graph with n vertices, minimum degree at least 2, average degree $\bar{d}$, and girth g.*

If

$$g = 2r + 1,$$

then

$$\boxed{n \geq 1 + \bar{d} \sum_{i=0}^{r-1} (\bar{d} - 1)^i.}$$

If

$$g = 2r,$$

then

$$\boxed{n \geq 2 \sum_{i=0}^{r-1} (\bar{d} - 1)^i.}$$

Proof. Suppose first that

$$g = 2r + 1.$$

Fix a vertex v. The endpoints of all non-returning walks of lengths

$$0, 1, \ldots, r$$

starting at v are distinct: two such walks ending at the same vertex would produce a cycle of length at most $2r$.

Hence

$$n \geq 1 + \sum_{i=1}^{r} N_i(v).$$

Average over $v \sim \pi$ and apply Lemma 79:

$$n \geq 1 + \sum_{i=1}^{r} \bar{d}(\bar{d} - 1)^{i-1}.$$

Now suppose

$$g = 2r.$$

Take an edge uv, and orient it both ways:

$$\vec{e} = (u, v), \qquad \overleftarrow{e} = (v, u).$$

The endpoints of the non-returning walks of lengths up to r extending from the two sides of this edge are all distinct; otherwise they would form a cycle of length less than $2r$.

Therefore

$$n \geq \sum_{i=0}^{r-1} N_i(\vec{e}) + \sum_{i=0}^{r-1} N_i(\overleftarrow{e}).$$

Choose $\vec{e}$ uniformly among directed edges. Its reverse is also uniform, so taking expectations gives

$$n \geq 2 \sum_{i=0}^{r-1} (\bar{d} - 1)^i.$$

□

5.2 Matchings

5.2.1 Hall's Marriage Theorem

Theorem 81 (Hall)**.** *Let $G = (L \sqcup R, E)$ be bipartite. There is a matching saturating L if and only if*

$$\boxed{|N(S)| \geq |S| \qquad \text{for every } S \subseteq L.}$$

Proof. Necessity is immediate: if a matching saturates S, its matched neighbors are $|S|$ distinct elements of $N(S)$.

For sufficiency, let M be a maximum matching. Suppose some $x_0 \in L$ is unmatched.

Let $S \subseteq L$ and $T \subseteq R$ be the vertices reachable from x_0 by alternating paths, starting with an edge outside M.

No vertex of T is unmatched, since otherwise the alternating path to it would be an augmenting path, contradicting maximality of M.

Moreover,

$$\boxed{T = N(S).}$$

Indeed, every reached right vertex is clearly a neighbor of S. Conversely, from any $x \in S$, every nonmatching edge can extend the alternating path to its other endpoint; the unique matching edge incident to x, if any, already lies on the alternating path.

Finally, M gives a bijection

$$T \longleftrightarrow S \setminus \{x_0\}.$$

Every $y \in T$ is matched, and its matched partner is reachable and hence lies in $S \setminus \{x_0\}$; conversely every $x \in S \setminus \{x_0\}$ was reached through its matching edge.

Therefore

$$|N(S)| = |T| = |S| - 1,$$

contradicting Hall's condition. Hence M saturates L. □

5.2.2 Gale–Ryser Theorem

Hall's theorem generalizes naturally from choosing one neighbor for every left vertex to prescribing all degrees on both sides.

Theorem 82 (Bipartite Degree Realization)**.** *Let $G = (L \sqcup R, E)$ be bipartite, and let*

$$a : L \to \mathbb{Z}_{\geq 0}, \qquad b : R \to \mathbb{Z}_{\geq 0}$$

satisfy

$$\sum_{u \in L} a_u = \sum_{v \in R} b_v = D.$$

There is a subgraph $H \subseteq G$ with

$$\deg_H(u) = a_u \quad (u \in L), \qquad \deg_H(v) = b_v \quad (v \in R)$$

if and only if for every $X \subseteq L$ and $Y \subseteq R$,

$$\boxed{a(X) \leq b(Y) + |E(X, R \setminus Y)|.}$$

Proof. Construct the network

$$s \longrightarrow L \longrightarrow R \longrightarrow t$$

with capacities

$$c(s,u) = a_u, \qquad c(u,v) = 1 \quad (uv \in E), \qquad c(v,t) = b_v.$$

An integral flow of value D chooses exactly a_u edges incident to every $u \in L$ and exactly b_v edges incident to every $v \in R$. Thus such a flow is exactly the desired subgraph.

By max-flow min-cut, it remains to characterize when every cut has capacity at least D.

Any cut is determined by sets $X \subseteq L$ and $Y \subseteq R$ placed on the source side. Its capacity is

$$a(L \setminus X) + |E(X, R \setminus Y)| + b(Y).$$

Since $a(L) = D$, this is at least D exactly when

$$a(X) \le b(Y) + |E(X, R \setminus Y)|.$$

Thus the stated inequalities are precisely the min-cut conditions. Integrality of max flow completes the proof. □

5.2.3 Cantor–Schröder–Bernstein

An injection in each direction should let us pair two sets perfectly, but neither injection need be surjective. View the injections as two matchings in a bipartite graph. One already covers the right side; repair its unmatched left vertices by switching matchings along the alternating chains they start. The chains may be infinite, but the switch can be specified explicitly.

Theorem 83 (Cantor–Schröder–Bernstein)**.** *If there are injections $f : A \to B$ and $g : B \to A$, then there is a bijection $h : A \to B$. Explicitly, put*

$$A_0 = A \setminus g(B), \qquad A_{j+1} = g(f(A_j)), \qquad C = \bigcup_{j \ge 0} A_j.$$

Then a bijection is

$$\boxed{h(a) = \begin{cases} f(a), & a \in C, \\ g^{-1}(a), & a \notin C. \end{cases}}$$

Here g^{-1} is defined on $g(B)$.

Proof. The definition of C gives $C = A_0 \sqcup g(f(C))$: its initial part is outside $g(B)$, and all later parts are inside. Consequently, using injectivity of g,

$$\begin{aligned} g(B \setminus f(C)) &= g(B) \setminus g(f(C)) \\ &= (A \setminus A_0) \setminus g(f(C)) = A \setminus C. \end{aligned}$$

Thus f maps C bijectively onto $f(C)$, while g^{-1} maps $A \setminus C$ bijectively onto $B \setminus f(C)$. These two maps have disjoint domains covering A and disjoint images covering B, so their union is the stated bijection. □

Consider disjoint copies of A and B, with edges $a \to f(a)$ and $b \to g(b)$. The g-edges already pair every vertex of B with one of A, leaving exactly A_0 unmatched. From $a_0 \in A_0$, follow

$$a_0 \xrightarrow{f} b_0 \xrightarrow{g} a_1 \xrightarrow{f} b_1 \xrightarrow{g} a_2 \longrightarrow \cdots .$$

Every vertex has at most one incoming edge and exactly one outgoing edge in this directed graph. These chains therefore cannot branch, merge, or repeat a vertex: tracing a repetition backward would give an incoming edge at their starting vertex in A_0. Switch from the g-edges to the f-edges along all these chains. Every affected B vertex stays matched, the starting A vertex becomes matched, and every later A vertex is rematched rather than lost. The affected A vertices are exactly C.

Outside these chains, retain the g-edges. This handles cycles, doubly infinite chains, and chains starting on the B side without choosing an orientation separately for each component. The formula for C performs all the switches at once.

Corollary 84 (Combining two saturating matchings)**.** *In any bipartite graph with sides A, B, possibly infinite, if one matching covers A and another covers B, their union contains a perfect matching.*

Proof. Let $f(a)$ be the partner of a in the first matching and $g(b)$ the partner of b in the second. Both are injections. The bijection above uses either the edge $af(a)$ or the edge $g(b)b$ with $g(b) = a$, so all its matched pairs belong to the union of the two matchings. □

5.2.4 Brégman's Theorem

For a 0–1 matrix A, its permanent counts the perfect matchings in the corresponding bipartite graph.

Theorem 85 (Brégman's Theorem)**.** *Let*

$$A \in \{0,1\}^{n\times n}$$

have row sums

$$r_1, \dots, r_n.$$

Then

$$\boxed{\operatorname{per}(A) \le \prod_{i=1}^{n} (r_i!)^{1/r_i}.}$$

Proof. Let $\mathcal{M}$ be the set of perfect matchings represented by A, and choose a uniformly random

$$\sigma \in \mathcal{M}.$$

Then

$$H(\sigma) = \log_2 |\mathcal{M}| = \log_2 \operatorname{per}(A).$$

Independently choose a uniformly random ordering of the rows. Reveal the values $\sigma(i)$ in this random order.

When row i is revealed, let R_i be the number of currently unused columns j such that

$$A_{ij} = 1.$$

Conditioned on everything already revealed, $\sigma(i)$ has at most R_i possible values. Thus

$$H(\sigma(i) \mid \text{earlier rows}) \le \mathbb{E}[\log_2 R_i].$$

By the chain rule,

$$H(\sigma) \le \sum_{i=1}^{n} \mathbb{E}[\log_2 R_i].$$

Now fix both i and the matching σ. For every allowed column

$$j \in N(i),$$

there is a unique row

$$\sigma^{-1}(j)$$

matched to j. These give exactly r_i rows, one of which is i itself.

A column $j \in N(i)$ is still available when row i is exposed exactly when the row $\sigma^{-1}(j)$ has not appeared earlier. Therefore R_i is the number of these r_i distinguished rows appearing at or after i.

Since the row order is uniformly random, the rank of i among these r_i rows is uniform. Consequently,

$$R_i \quad \text{is uniform on} \quad \{1, 2, \dots, r_i\}.$$

Hence

$$\mathbb{E}[\log_2 R_i] = \frac{1}{r_i}\sum_{j=1}^{r_i}\log_2 j = \frac{1}{r_i}\log_2(r_i!).$$

Therefore

$$\log_2 \operatorname{per}(A) \le \sum_{i=1}^{n}\frac{\log_2(r_i!)}{r_i}.$$

Exponentiating gives

$$\operatorname{per}(A) \le \prod_{i=1}^{n}(r_i!)^{1/r_i}.$$

□

5.2.5 Kahn–Lovász Theorem

Brégman's inequality bounds perfect matchings in bipartite graphs. The same degree bound extends to arbitrary simple graphs, with each vertex contributing half the exponent. The reason for this square root is combinatorial: two perfect matchings superimpose to form even cycles, and directed cycle covers are exactly what the permanent counts.

Write $\operatorname{pm}(G)$ for the number of perfect matchings of G.

Lemma 86 (Squaring the matching count)**.** *Let G be a finite simple graph on an even number of vertices, and let A be its full adjacency matrix. Then*

$$\boxed{\operatorname{pm}(G)^2 \le \operatorname{per}(A).}$$

More precisely, $\operatorname{pm}(G)^2$ *equals the number of permutations counted by* $\operatorname{per}(A)$ *whose cycles all have even length.*

Proof. The square $\operatorname{pm}(G)^2$ counts ordered pairs (M_1, M_2) of perfect matchings. Retaining two copies of every common edge, their union is a spanning multigraph whose components are doubled edges and even cycles of length at least four.

Fix such a union H, and let $c(H)$ count its cycles of length at least four. Each of these cycles has exactly two ways to assign its edges alternately to M_1 and M_2. A doubled edge must occur in both matchings. Thus exactly $2^{c(H)}$ ordered pairs produce H.

On the other hand,

$$\operatorname{per}(A) = \sum_{\sigma\in S_{|V(G)|}}\prod_{v\in V(G)} A_{v,\sigma(v)}$$

counts permutations for which every v is adjacent to $\sigma(v)$. Such a permutation is a directed cycle cover: a two-cycle uses one undirected edge in both directions, while every longer cycle has two possible orientations.

Consequently, the permutations having only even cycles produce exactly the same unions H, each with the same multiplicity $2^{c(H)}$. Their number is therefore $\operatorname{pm}(G)^2$. The permanent may additionally count cycle covers containing odd cycles, which proves the inequality. □

The key correspondence is that an even cycle has two alternating assignments to the matchings and two orientations. These choices account for the same multiplicity.

Theorem 87 (Kahn–Lovász)**.** *Let G be a finite simple graph on $2n$ vertices, with no isolated vertices, and write $d_v = \deg_G(v)$. Then*

$$\boxed{\operatorname{pm}(G) \le \prod_{v\in V(G)}(d_v!)^{1/(2d_v)}.}$$

The bound is attained by every disjoint union of balanced complete bipartite graphs.

Proof. The row sums of the full adjacency matrix A are the vertex degrees d_v. By Lemma 86 and Brégman's inequality,

$$\mathrm{pm}(G)^2 \le \mathrm{per}(A) \le \prod_{v \in V(G)} (d_v!)^{1/d_v}.$$

Take square roots.

For sharpness, $K_{d,d}$ has $d!$ perfect matchings, while its $2d$ vertices contribute

$$\prod_{v \in V(K_{d,d})} (d!)^{1/(2d)} = d!.$$

Both the matching count and the degree product multiply over disjoint components. □

If G has an isolated vertex, then $\mathrm{pm}(G) = 0$, so that case is immediate.

Corollary 88 (Regular graphs)**.** *Every simple d-regular graph on $2n$ vertices, where $d \ge 1$, satisfies*

$$\boxed{\mathrm{pm}(G) \le (d!)^{n/d}.}$$

When $d \mid n$, equality is attained by the disjoint union of n/d copies of $K_{d,d}$.

Proof. There are $2n$ identical factors in the Kahn–Lovász bound. □

5.2.6 The van der Waerden Permanent Theorem

The permanent of a nonnegative matrix is a weighted sum of perfect matchings, with no signs to cancel them as in the determinant. A doubly stochastic matrix has every row and column sum equal to one. The van der Waerden theorem says that even the least favorable such weights leave at least as much total matching weight as the uniform matrix.

Theorem 89 (van der Waerden)**.** *For every doubly stochastic $A = (a_{ij}) \in \mathbb{R}_{\ge 0}^{n \times n}$, with $n \ge 1$,*

$$\boxed{\mathrm{per}\, A := \sum_{\sigma \in S_n} \prod_{i=1}^{n} a_{i,\sigma(i)} \ge \frac{n!}{n^n}.}$$

The bound is attained when every entry is $1/n$.

The useful object is the polynomial

$$p_A(x) = \prod_{i=1}^{n} \left(\sum_{j=1}^{n} a_{ij} x_j \right), \qquad [x_1 \cdots x_n] p_A = \mathrm{per}\, A.$$

Selecting one term from each row uses every variable exactly once precisely when the chosen columns form a permutation. To extract this coefficient, we can repeatedly differentiate in one variable and then set it to zero. After the first step the polynomial need no longer be a product of linear forms. We therefore need a larger class that survives coefficient extraction, and a quantity that does not decrease too much at each step.

Capacity For a nonzero homogeneous polynomial p of degree d in d variables with nonnegative real coefficients, define

$$\mathrm{Cap}(p) = \inf_{x_1, \ldots, x_d > 0} \frac{p(x)}{x_1 \cdots x_d} = \inf_{\substack{x_i > 0 \\ x_1 \cdots x_d = 1}} p(x).$$

The equality follows by scaling all coordinates by the same positive factor. Nonnegativity immediately gives $[x_1 \cdots x_d]p \le \mathrm{Cap}(p)$: the desired monomial is one of the nonnegative terms in the quotient. We need the reverse inequality up to a controlled factor.

For the polynomial p_A, weighted AM–GM gives, for every positive x,

$$p_A(x) \ge \prod_{i=1}^{n}\prod_{j=1}^{n} x_j^{a_{ij}} = \prod_{j=1}^{n} x_j^{\sum_i a_{ij}} = x_1 \cdots x_n.$$

Since $p_A(1,\dots,1) = 1$, we have $\mathrm{Cap}(p_A) = 1$. Thus it suffices to retain a fraction $n!/n^n$ of capacity while extracting all variables.

The following one-variable inequality gives exactly the loss needed at one extraction. Its hypothesis on the roots will be justified afterward.

Lemma 90 (The cost of extracting a linear coefficient)**.** *Let F be a real polynomial of degree $d \ge 2$, with $F(0) > 0$ and all roots strictly negative and real. Then*

$$\boxed{F'(0) \ge \left(\frac{d-1}{d}\right)^{d-1} \inf_{t>0} \frac{F(t)}{t}.}$$

Proof. Write $F(t) = F(0)\prod_{j=1}^{d}(1 + b_j t)$ with $b_j > 0$, and put $s = \sum_j b_j = F'(0)/F(0)$. AM–GM bounds the product from above:

$$F(t) \le F(0)\left(1 + \frac{st}{d}\right)^d.$$

Evaluate this bound at $t = d/((d-1)s)$, the minimizer of its right side divided by t. This yields

$$\inf_{t>0} \frac{F(t)}{t} \le F(0)\frac{(d-1)s}{d}\left(\frac{d}{d-1}\right)^d = F'(0)\left(\frac{d}{d-1}\right)^{d-1}.$$

Rearrange. The loss comes from replacing the individual b_j by their average; equality in that replacement holds when they are all equal. □

Root Condition A homogeneous real polynomial is *half-plane stable* here if $p(z) \ne 0$ whenever every $\mathrm{Re}\, z_i > 0$. A product of nonzero linear forms with nonnegative coefficients has this property: each form has strictly positive real part. In particular, p_A is stable.

For the inductive proof, temporarily impose two stronger conditions: every degree-d monomial has a positive coefficient, and

$$p(z) \ne 0 \quad \text{if all } \mathrm{Re}\, z_i \ge 0 \text{ and } \sum_i \mathrm{Re}\, z_i > 0. \tag{5}$$

Call the latter condition *strict half-plane stability*. We will remove both extra assumptions by a simple substitution.

Lemma 91 (Real roots and closure under extraction)**.** *Suppose p has degree $d \ge 2$ in d variables and satisfies these two stronger conditions. Then, for every $y \in \mathbb{R}_{>0}^{d-1}$, $F(t) = p(y,t)$ has degree d and all its roots are strictly negative and real. Moreover,*

$$q(x_1,\dots,x_{d-1}) = \left.\frac{\partial p}{\partial x_d}\right|_{x_d=0}$$

is homogeneous of degree $d-1$, has every degree-$(d-1)$ coefficient positive, and is strictly half-plane stable.

Proof. The coefficient of t^d is the positive coefficient of x_d^d in p, independent of the other coordinates. Also $F(0) > 0$. If a nonzero root t were not on the negative real axis, write $t = |t|e^{i\theta}$ with $-\pi < \theta < \pi$. Multiplication by $\lambda = e^{-i\theta/2}$ puts both $\lambda y_1, \dots, \lambda y_{d-1}$ and λt in the open right half-plane. Homogeneity would give $p(\lambda y, \lambda t) = \lambda^d p(y,t) = 0$, contradicting stability. Thus all roots are negative and real.

To prove closure, now fix a possibly complex vector z with $\operatorname{Re} z_i \geq 0$ and $\sum_i \operatorname{Re} z_i > 0$. The polynomial $F(t) = p(z,t)$ still has degree d, and the strict condition makes it nonzero whenever $\operatorname{Re} t \geq 0$. All its roots $\lambda_1, \dots, \lambda_d$ therefore have negative real part. Its logarithmic derivative at zero is

$$\frac{q(z)}{p(z,0)} = \frac{F'(0)}{F(0)} = \sum_{j=1}^{d} \frac{1}{-\lambda_j}.$$

Every summand has strictly positive real part, so the sum is nonzero. This proves $q(z) \neq 0$. Its degree and positive coefficients follow because the operation selects exactly the monomials with exponent one in x_d. □

Capacity Losses Telescope Put $G(d) = ((d-1)/d)^{d-1}$ for $d \geq 2$. For p, q as in the preceding lemma, fix $y > 0$ and apply the one-variable bound:

$$q(y) \geq G(d) \inf_{t>0} \frac{p(y,t)}{t} \geq G(d) \operatorname{Cap}(p) \prod_{i=1}^{d-1} y_i.$$

Divide by the product of the coordinates and take the infimum. Thus

$$\boxed{\operatorname{Cap}(q) \geq G(d) \operatorname{Cap}(p).} \tag{6}$$

The polynomial class and the capacity bound both survive the same operation, so we can repeat it.

Theorem 92 (Gurvits's capacity bound)**.** *If p is a nonzero homogeneous half-plane-stable polynomial of degree n in n variables with nonnegative real coefficients, then*

$$\boxed{[x_1 \cdots x_n]p \geq \frac{n!}{n^n} \operatorname{Cap}(p).}$$

Proof. First assume all degree-n coefficients are positive and p is strictly half-plane stable. Repeatedly differentiate in the last variable and set it to zero. The resulting polynomials satisfy the same conditions by Lemma 91. After extracting $x_n, \dots, x_2$, the last polynomial is ax_1, where $a = [x_1 \cdots x_n]p$, and its capacity is a. Iterating (6) gives

$$a \geq \operatorname{Cap}(p) \prod_{d=2}^{n} \left(\frac{d-1}{d}\right)^{d-1} = \operatorname{Cap}(p) \frac{(n-1)!}{n^{n-1}} = \operatorname{Cap}(p) \frac{n!}{n^n}.$$

For $n = 1$ the equality is immediate and the product is empty.

For the general case, set $S = x_1 + \cdots + x_n$ and, for $\varepsilon > 0$,

$$p_\varepsilon(x) = p(x_1 + \varepsilon S, \dots, x_n + \varepsilon S).$$

If all real parts of z are nonnegative and their sum is positive, every $z_i + \varepsilon \sum_j z_j$ has positive real part. Thus p_ε is strictly half-plane stable. Expanding any nonzero monomial of p and choosing εS in all its n factors shows that every degree-n coefficient of p_ε is positive. The same expansion retains all original terms with nonnegative additions, so $p_\varepsilon \geq p$ coefficientwise and $\operatorname{Cap}(p_\varepsilon) \geq \operatorname{Cap}(p)$. The proved case therefore gives

$$[x_1 \cdots x_n]p_\varepsilon \geq \frac{n!}{n^n} \operatorname{Cap}(p).$$

Let $\varepsilon \downarrow 0$. Coefficients converge to those of p, proving the bound. □

Proof of Theorem 89. The product of row forms p_A is homogeneous, nonnegative, and half-plane stable. The AM–GM calculation gives $\operatorname{Cap}(p_A) = 1$, and its $x_1 \cdots x_n$ coefficient is $\operatorname{per} A$. Apply Theorem 92. For the uniform matrix, each of the $n!$ permutation terms is n^{-n}, so the bound is attained. □

Corollary 93 (Perfect matchings in regular bipartite graphs)**.** *An r-regular bipartite graph with n vertices on each side and $r \geq 1$ has at least $r^n n!/n^n$ perfect matchings.*

Proof. If B is its bipartite adjacency matrix, B/r is doubly stochastic and $\operatorname{per}(B/r) = r^{-n} \operatorname{per} B$. Apply the theorem. The same statement holds for a multigraph when parallel edges are counted as distinct choices. □

5.3 Additive Combinatorics

Unless stated otherwise, sets are finite subsets of an abelian group. Write $A \pm B = \{a \pm b : a \in A, b \in B\}$ and $hA = \{a_1 + \cdots + a_h : a_i \in A\}$, with $0A = \{0\}$. Repetition is allowed in hA.

For a multiset $X = \{x_1, \ldots, x_n\}$, occurrences are indexed even when their values agree, and $|X| = n$. Write $\Sigma(X) = \sum_{i=1}^{n} x_i$ for its total weight and $\mathcal{S}(X) = \{\sum_{i \in I} x_i : I \subseteq [n]\}$ for its subset sums, including zero. Thus $\mathcal{S}(X) = \{0, x_1\} + \cdots + \{0, x_n\}$, and each occurrence may be used at most once. Subsets of multisets select occurrences; $X \uplus Y$ adds their multiplicities, so $\mathcal{S}(X \uplus Y) = \mathcal{S}(X) + \mathcal{S}(Y)$ and $\Sigma(X \uplus Y) = \Sigma(X) + \Sigma(Y)$. All sets in denominators are nonempty; $[M] = \{1, \ldots, M\}$, and log denotes the natural logarithm.

Addition either produces growth or exposes structure that prevents it. Kneser's theorem identifies the exact obstruction to elementary growth: periodicity. Ruzsa's and Plünnecke's inequalities propagate small growth, while additive energy detects it inside a large subset. Fourier analysis and almost-periodicity then turn small growth into subspace structure. For Subset Sum and Knapsack, a complementary objective is to find structure *inside the attainable sums*: long progressions allow us to fill targets and force exchanges of equal weight.

5.3.1 Kneser's Addition Theorem

The *stabilizer* of a finite nonempty set S is the finite subgroup $H(S) = \{g : S + g = S\}$. It is a subgroup because translations compose and invert; it is finite because $g \mapsto s + g$ injects it into S for any fixed $s \in S$.

The expected bound $|A + B| \geq |A| + |B| - 1$ fails when, for example, $A = B = H$ is a nontrivial finite subgroup. Here $A + B = H$: adding elements merely moves within an existing coset. Kneser's theorem says that this is the only obstruction, once we pass to the stabilizer of the *sumset*.

Theorem 94 (Kneser's addition theorem)**.** *For finite nonempty subsets A, B of an abelian group, put $H = H(A + B)$. Then*

$$\boxed{|A + B| \geq |A + H| + |B + H| - |H|.}$$

All three sets are unions of H-cosets. Dividing by $|H|$ gives the ordinary sum-of-sizes-minus-one inequality for their images in G/H, where the sumset has trivial stabilizer. The proof has two parts: a union cannot lose too much periodicity without gaining elements, and moving elements between the summands eventually forces periodicity.

Lemma 95 (Union of periodic sets)**.** *If $S = S_1 \cup \cdots \cup S_m$, where the S_i are finite and nonempty, then $|S| + |H(S)| \geq \min_i \big(|S_i| + |H(S_i)|\big)$.*

Proof. It suffices to prove the two-set case and then induct. Put $H_i = H(S_i)$ and $H = H(S_1 \cup S_2)$. Quotient by $H_1 \cap H_2$: this subgroup stabilizes both sets and their union, so every cardinality in the inequality is divided by its order. Thus assume $H_1 \cap H_2 = \{0\}$, and write $h_i = |H_i|$.

Each coset of $K = H_1 + H_2$ is an h_2-by-h_1 grid: its rows are H_1-cosets and its columns are H_2-cosets, with one point at each intersection. Within such a coset, S_1 consists of r whole rows and S_2 of c whole columns. Their local contributions to $S_2 \setminus S_1$ and $S_1 \setminus S_2$ are $c(h_2 - r)$ and $r(h_1 - c)$, respectively.

If some coset has $0 < r < h_2$ and $0 < c < h_1$, the product of these two gains is $r(h_2 - r)c(h_1 - c) \geq (h_2 - 1)(h_1 - 1)$. Consequently either $|S_2 \setminus S_1| \geq h_1 - 1$ or $|S_1 \setminus S_2| \geq h_2 - 1$. Since $|H| \geq 1$, the first inequality gives $|S| + |H| \geq |S_1| + h_1$, and the second gives $|S| + |H| \geq |S_2| + h_2$.

Otherwise, every nonempty coset not completely filled by S meets only one of the two sets. If one such coset meets only S_1 and another only S_2, then $|S_1 \setminus S_2| \geq h_1$ and $|S_2 \setminus S_1| \geq h_2$. If $h_1 \geq h_2$, use the first gain to compensate for the stabilizer of S_2; if $h_2 \geq h_1$, use the second for S_1.

In the remaining case, all partially filled cosets come from the same set, say S_1. Both these cosets and the completely filled cosets are unions of H_1-cosets. Thus $S + H_1 = S$, which means $H_1 \leq H$, and $|S| + |H| \geq |S_1| + |H_1|$. □

Proof of Theorem 94. First we prove the weaker-looking inequality

$$|A + B| + |H(A + B)| \geq |A| + |B|. \tag{7}$$

Fix $b \in B$. Among finite pairs (A', B') with $A \subseteq A'$, $b \in B'$, $A' + B' \subseteq A + B$, and $|A'| + |B'| = |A| + |B|$, choose (A_b, B_b) minimizing $|B_b|$. The original pair is admissible. Put $S_b = A_b + B_b$.

For $t \in A_b - b$, consider the transfusion $A' = A_b \cup (B_b + t)$, $B' = B_b \cap (A_b - t)$. Each removed $y \in B_b$ corresponds to the newly inserted $y + t \in A'$, so total cardinality is preserved, and $b \in B'$. Moreover $A' + B' \subseteq A_b + B_b$: the only new type of sum is $(z + t) + y = z + (y + t)$, where $z \in B_b$ and $y + t \in A_b$. The transformed pair is therefore admissible. Minimality forces $B' = B_b$, or

$$B_b + t \subseteq A_b \qquad (t \in A_b - b). \tag{8}$$

For $c \in B_b - b$ and $a \in A_b$, take $t = a - b$ to obtain $a + c \in A_b$. Hence $A_b + c \subseteq A_b$, and finiteness gives equality. Thus $B_b - b \subseteq H(A_b)$: if such a difference were not a period, the transfusion could still move another element. In particular $S_b = A_b + b$ and $|H(A_b)| \geq |B_b|$, so $|S_b| + |H(S_b)| \geq |A_b| + |B_b| = |A| + |B|$. As b ranges over B, the inclusions $A + b \subseteq S_b \subseteq A + B$ show that $A + B = \bigcup_{b \in B} S_b$. The union lemma proves (7).

Finally put $H = H(A + B)$. The sets $A + H$ and $B + H$ still sum to $A + B$, whose stabilizer is H. Applying (7) to these enlarged summands yields $|A + B| + |H| \geq |A + H| + |B + H|$, as required. □

Corollary 96 (Cauchy–Davenport)**.** *If p is prime and $A, B \subseteq \mathbb{Z}/p\mathbb{Z}$ are nonempty, then $|A + B| \geq \min\{p, |A| + |B| - 1\}$. More generally, for nonempty $A_1, \ldots, A_h \subseteq \mathbb{Z}/p\mathbb{Z}$,*

$$|A_1 + \cdots + A_h| \geq \min\left\{p, \sum_{i=1}^{h} |A_i| - h + 1\right\}.$$

Proof. If $A + B$ is proper, its stabilizer is a proper subgroup of a prime-order group, hence trivial. Apply Kneser; the iterated statement follows by induction. □

Corollary 97 (Growth of modular subset sums)**.** *Let p be prime. For nonzero $a_1, \ldots, a_t \in \mathbb{Z}/p\mathbb{Z}$, let $\mathcal{S}_j = \{\sum_{i \in I} a_i : I \subseteq [j]\}$. Then $|\mathcal{S}_j| \geq \min\{p, j + 1\}$. In particular, any $p - 1$ nonzero input occurrences have subset sums covering the whole group.*

Proof. Use $\mathcal{S}_j = \mathcal{S}_{j-1} + \{0, a_j\}$, starting from $|\mathcal{S}_0| = 1$. Cauchy–Davenport increases the size by at least one at each step until the group is filled. □

5.3.2 Iterated Modular Sumsets

Sparse convolution computes one sumset quickly, but repeatedly using it can still be expensive: many intermediate sets may be large even when their sum is no larger. In a cyclic group, this failure of growth reveals a period. We quotient out that period immediately. Processing a balanced merge tree depth first keeps only a logarithmic number of completed subtrees waiting to be merged, which makes the total work easy to charge.

We count operations on words of logarithmic bit length, and use the usual fast algorithm for dense integer convolution. All randomized routines are Las Vegas: their answers are exact and their running-time bounds are in expectation.

Theorem 98 (Iterated modular sumsets)**.** *Let $m, r \geq 1$. Given nonempty sets $A_1, \ldots, A_r \subseteq \mathbb{Z}/m\mathbb{Z}$, put $L = \sum_i |A_i|$ and $K = |A_1 + \cdots + A_r|$. Their sumset can be listed in expected $\widetilde{O}(L + K)$ time.*

An empty input set makes the answer empty. The two-set routine used below is ModAdd(X, Y, m): compute the ordinary integer sumset of the representatives in $[0, m-1]$, reduce modulo m, and deduplicate. Every modular sum has at most two ordinary lifts, so a near-linear sparse sumset algorithm implements this routine in expected $\widetilde{O}(|X + Y|)$ time. Sets returned by the routines are sorted.

Finding a period from the gaps. For $C \subseteq \mathbb{Z}/m\mathbb{Z}$, a period is a residue h such that $C + h = C$. The periods form a subgroup. If $C \neq \varnothing$ and its smallest positive integer period is d, allowing $d = m$, then $d \mid m$ and $C = (C \bmod d) + \{0, d, \ldots, m-d\}$.

Lemma 99 (Cyclic periods)**.** *The smallest positive period of a sorted nonempty set $C = \{c_0 < \cdots < c_{s-1}\} \subseteq [0, m-1]$ can be found in $O(s)$ time.*

Proof. Form the cyclic gaps $g_i = c_{i+1} - c_i$ for $i < s-1$, and $g_{s-1} = m + c_0 - c_{s-1}$. A translation preserving C preserves its cyclic order, hence rotates this gap word without changing it. Conversely, if the word consists of repetitions of a block of length $q \mid s$, translation by $d = g_0 + \cdots + g_{q-1}$ preserves C. Thus the shortest repeating block gives the desired period.

The prefix-function computation below finds all possible block lengths. Here π_i is the length of the longest proper prefix of $g_0 \cdots g_i$ that is also a suffix thereof. On a mismatch, the next possible match is a border of the current matched prefix, so replacing j by π_{j-1} skips no candidate. Successful extensions increase j by one, whereas every failed comparison decreases it; there are only $O(s)$ comparisons. Following the same border links from π_{s-1} enumerates every proper border. A border of length b gives a repeating block precisely when $s - b$ divides s. This proves correctness and the linear time bound. □

Algorithm 13 CyclicPeriod(C, m)

```
Form the cyclic gap word g_0, ..., g_{s-1}; set π_0 ← 0
for i = 1, ..., s − 1 do
    j ← π_{i−1}
    while j > 0 and g_i ≠ g_j do
        j ← π_{j−1}
    If g_i = g_j, set j ← j + 1
    π_i ← j
q ← s, b ← π_{s−1}
while b > 0 do
    If s − b divides s, set q ← min{q, s − b}
    b ← π_{b−1}
return g_0 + ··· + g_{q−1}
```

Merge until growth fails, then quotient. Kneser's theorem implies that if $|X + Y| < |X| + |Y| - 1$, the sumset has a nontrivial period. Moreover, every period of a partial sumset is a period of the final sumset: $(X + Y + h) + Z = (X + Y) + Z$. Consequently a period found anywhere in the merge tree can be used to reduce the entire problem.

The recursive helper returns either an ordinary set or a tagged value Period(d). Such a tag is propagated immediately, before processing any further subtree. The helper uses the input family and modulus of the current ModularSum call.

Algorithm 14 Computation of an iterated modular sumset

```
function ModularSum(A_1, ..., A_r, m)
    If some A_i is empty, return ∅
    If m = 1, return {0}
    Sort and deduplicate the sets; pad the family with {0} to a power of two
    Z ← Merge(1, r), using the padded value of r
    If Z is a set, return Z
    Let Z = Period(d); form B_i ← A_i mod d
    B ← ModularSum(B_1, ..., B_r, d)
    return B + {0, d, ..., m − d}, listed in sorted order
function Merge(ℓ, u)
    If ℓ = u, return A_ℓ
    v ← ⌊(ℓ + u)/2⌋
    X ← Merge(ℓ, v); if it is a period tag, return it
    Y ← Merge(v + 1, u); if it is a period tag, return it
    C ← ModAdd(X, Y, m)
    if |C| < |X| + |Y| − 1 then
        return Period(CyclicPeriod(C, m))
    return C
```

Proof of Theorem 98. Padding at most doubles the input size; below, L includes the dummy singleton sets.

Correctness. Every intermediate set is the sum of a subfamily. If no period is reported, the root is the required sumset. Otherwise Kneser and Lemma 99 give a proper divisor $d \mid m$ which is a period of the full answer S. Writing π for reduction modulo d, we have $\pi(S) = \pi(A_1) + \cdots + \pi(A_r)$ and $S = \pi^{-1}(\pi(S))$. The recursive answer and its lift are therefore correct. The modulus decreases by at least a factor of two.

Work before the first period. Write $h = \log_2 r$ after padding. Every partial sumset has size at most K: choose one element from each omitted set and translate the partial sumset into the final one. For every completed merge before the first failure of growth, $|C| - 1 \geq (|X| - 1) + (|Y| - 1)$. Hence, in a completed subtree with root set F, the sum of $|C| - 1$ over any one level is at most $|F| - 1$. The total size of all sets in that subtree is at most $(h + 1)(|F| - 1)$ plus its number of nodes.

Just before the first failed merge, the visited leaves are partitioned into at most $h + 1$ completed subtrees: they are the sets on the depth-first stack, including the two operands of that merge. Each root has size at most K, and together these subtrees have $O(r)$ nodes. The sum of all intermediate sizes computed so far is thus $O(K(h + 1)^2 + r)$; the failed merge adds at most another K. If no merge fails, use the single completed root instead. Sparse convolution and period detection therefore cost $\widetilde{O}(L + K)$ at this modulus.

Quotienting. The total projected input size never exceeds the original L, and there are at most $1 + \log_2 m$ moduli. At each proper quotient the answer size drops by at least a factor of two, since the answer is a union of full fibers. Thus the answer sizes over all recursive calls sum to at most $2K$. Projection, sorting, all merges, and all final lifts together take expected $\widetilde{O}(L + K)$ time. □

For ordinary integer sumsets, every merge already satisfies $|X + Y| - 1 \geq (|X| - 1) + (|Y| - 1)$. The same argument then bounds a balanced merge tree directly, without quotienting. The cyclic case needs the period step precisely because this inequality can fail.

5.3.3 Ruzsa's Triangle Inequality and Covering Lemma

Two elementary operations underlie much of sumset calculus: an intermediate set compares two difference sets, and a maximal disjoint family of translates turns a cardinality bound into a covering.

Theorem 100 (Ruzsa's triangle inequality)**.** *For finite nonempty subsets A, B, C of an abelian group,*

$$\boxed{|A - C|\,|B| \le |A - B|\,|B - C|.}$$

Proof. For each $d \in A - C$, fix $d = a_d - c_d$. The map $(d, b) \mapsto (a_d - b, b - c_d)$ into $(A - B) \times (B - C)$ is injective: adding the output coordinates recovers d, hence the chosen a_d, and then the first coordinate recovers b. □

Taking $-A$ as the intermediate set gives $|A - A| \le |A + A|^2/|A|$, so small doubling also controls subtraction.

Theorem 101 (Ruzsa's covering lemma)**.** *For nonempty A, B, if $|A + B| \le K|B|$, some $T \subseteq A$ satisfies $|T| \le K$ and $A \subseteq T + (B - B)$.*

Proof. Choose $T \subseteq A$ maximal with the translates $t + B$ pairwise disjoint. They lie in $A + B$, so $|T||B| \le |A + B|$. For each $a \in A$, maximality gives $a + b = t + b'$ for some $t \in T$ and $b, b' \in B$, which places a in $t + (B - B)$. □

5.3.4 Plünnecke–Ruzsa Inequalities

From $|A + B| \le K|A|$, we would like each further addition of B to cost at most another factor K. The hypothesis says nothing directly about $A + B$, so it cannot simply be iterated. Instead, choose a subset of A with minimum relative growth; minimality then controls every part of that subset.

Lemma 102 (Minimal growth)**.** *Let X, B be nonempty. Suppose $\kappa = |X + B|/|X| \le |Y + B|/|Y|$ for every nonempty $Y \subseteq X$. Then $|X + B + C| \le \kappa|X + C|$ for every finite set C.*

Proof. List $C = \{c_1, \dots, c_m\}$, and let $X_i \subseteq X$ consist of those x for which $x + c_i$ first appears in translate i. Thus the $X_i + c_i$ partition $X + C$, and $|X + C| = \sum_i |X_i|$.

Now add the translates $X + B + c_i$ in the same order. Every point of $(X \setminus X_i) + c_i$ was already covered by earlier translates of X, so every point of $(X \setminus X_i) + B + c_i$ was already covered by earlier translates of $X + B$. Hence translate i contributes at most

$$|X + B| - |(X \setminus X_i) + B| \le \kappa|X| - \kappa|X \setminus X_i| = \kappa|X_i|$$

new points. The inequality also holds when $X \setminus X_i$ is empty. Sum over i. □

The overlaps are useful: minimality ensures that an already covered portion of X accounts for at least its proportional share of $X + B$.

The signs in Ruzsa's triangle inequality can in fact be chosen independently. Reflecting sets in the original inequality gives four of the eight cases; the other four follow from the all-plus case, using the preceding lemma.

Theorem 103 (Ruzsa's triangle inequalities, all sign choices)**.** *For finite nonempty subsets A, B, C of an abelian group and $\varepsilon, \eta, \theta \in \{-1, 1\}$,*

$$|A|\,|B + \varepsilon C| \le |A + \eta B|\,|A + \theta C|.$$

Proof. First prove the all-plus case. Choose a nonempty $X \subseteq A$ minimizing $\kappa = |X + B|/|X|$, so $\kappa \le |A + B|/|A|$. Since $X \neq \varnothing$, the set $X + B + C$ contains a translate of $B + C$. By Lemma 102 and $X \subseteq A$,

$$|B + C| \le |X + B + C| \le \kappa|X + C| \le \frac{|A + B|\,|A + C|}{|A|}.$$

The original triangle inequality, with intermediate set $-A$, also gives $|A|\,|B - C| \le |A + B|\,|A + C|$. Thus either sign is allowed on the left when both signs on the right are plus.

Now replace B, C by $\eta B, \theta C$. The right side becomes $|A + \eta B|\,|A + \theta C|$, while the two possible left-hand sumsets have sizes $|\eta B \pm \theta C| = |B \pm \eta\theta C|$. These give both choices of ε, independently of η and θ, covering all eight cases. □

Theorem 104 (Plünnecke–Ruzsa)**.** *For nonempty A, B, if $|A+B| \le K|A|$, there is a nonempty $X \subseteq A$ with $|X+hB| \le K^h|X|$ for every integer $h \ge 0$. Moreover,*

$$\boxed{|rB - sB| \le K^{r+s}|A| \qquad (r, s \in \mathbb{Z}_{\ge 0}).}$$

In particular, $|A+A| \le K|A|$ implies $|rA - sA| \le K^{r+s}|A|$.

Proof. Choose $X \subseteq A$ minimizing $\kappa = |X+B|/|X|$, so $\kappa \le K$. Iterating the minimal-growth lemma with $C = (h-1)B$ gives $|X+hB| \le \kappa^h|X|$. Ruzsa's triangle inequality, with intermediate set $-X$, then gives

$$|rB - sB|\,|X| \le |rB + X|\,|X + sB| \le \kappa^{r+s}|X|^2.$$

Divide by $|X|$ and use $|X| \le |A|$. □

Corollary 105 (Plünnecke–Ruzsa with either sign)**.** *Let A, B be finite and nonempty, and suppose $|A + \delta B| \le K|A|$ for some $\delta \in \{-1, 1\}$. Then, for either $\varepsilon \in \{-1, 1\}$ and all integers $r, s \ge 0$, we have $|rB + \varepsilon sB| \le K^{r+s}|A|$.*

Proof. This follows from the original theorem: $rB + sB = (r+s)B$, so take its parameters to be $(r+s, 0)$ and reflect B if $\delta = -1$. □

A finite symmetric set S containing zero is an *L-approximate subgroup* if $S + S \subseteq T + S$ for some $|T| \le L$. If $|A+A| \le K|A|$, then $S = A - A$ has $|S| \le K^2|A|$ and $|S+S+A| = |3A - 2A| \le K^5|A|$. Ruzsa's covering lemma, applied to $S+S$ and A, yields $S+S \subseteq T+S$ with $|T| \le K^5$. Thus $A - A$ is a K^5-approximate subgroup.

5.3.5 Multiway Sumset Inequalities

When an algorithm merges many sumsets, we want both to charge the input sizes to the output and to control the full sum from smaller partial sums. The following inequalities serve these two purposes.

Lemma 106 (Growth of a multiway sumset)**.** *For finite nonempty $A_1, \dots, A_k \subseteq \mathbb{Z}^d$,*

$$\sum_{i=1}^{k} (|A_i| - 1) \le |A_1 + \cdots + A_k| - 1.$$

Proof. Order each A_i lexicographically. Start with the sum of the least elements, advance the first summand through its list, then the second, and so on. Each advance strictly increases the total lexicographically, producing $1 + \sum_i (|A_i| - 1)$ distinct sums. □

Theorem 107 (Sumset submultiplicativity)**.** *For finite nonempty subsets $A_1, \dots, A_k$ of a commutative semigroup, where $k \ge 2$, put $B_i = \sum_{j \ne i} A_j$. Then*

$$\boxed{|A_1 + \cdots + A_k|^{k-1} \le \prod_{i=1}^{k} |B_i|.}$$

Proof. Order each A_i arbitrarily, and let V contain the lexicographically first representing tuple of each element of $A_1 + \cdots + A_k$. Thus $|V| = |A_1 + \cdots + A_k|$.

For each i, the partial-sum map on $\pi_{-i}V$ is injective. Indeed, if two distinct projected tuples had the same sum, replace the larger projection in its canonical full tuple by the smaller one, keeping coordinate i fixed. This preserves the full sum but decreases its representing tuple, a contradiction. Consequently, $|\pi_{-i}V| \le |B_i|$.

Each coordinate belongs to exactly $k-1$ of these projections. Shearer's lemma, in its projection form, therefore gives

$$|A_1 + \cdots + A_k|^{k-1} = |V|^{k-1} \le \prod_{i=1}^{k} |\pi_{-i}V| \le \prod_{i=1}^{k} |B_i|.$$

□

Corollary 108 (Controlling a multiway merge)**.** *If every leave-one-out sum has size at most M, then $|A_1 + \cdots + A_k| \le M^{k/(k-1)}$. If also $k - 1 \ge \log_2 M$, this is at most $2M$.*

Proof. Apply sumset submultiplicativity. For the second claim, write $M^{k/(k-1)} = M \cdot M^{1/(k-1)} \le 2M$. □

5.3.6 Short Sums and Long Arithmetic Progressions

For integer sums, a useful conclusion is often that attainable sums *contain* a long progression. The basic construction first reaches every residue modulo an available number, then adds copies of that number to fill the rest of each residue class. With two generators, the residue graph gives an exact answer.

Theorem 109 (Two-generator Frobenius theorem)**.** *For coprime integers $a, b \ge 2$, the largest integer not of the form $ax + by$, with $x, y \in \mathbb{Z}_{\ge 0}$, is $ab - a - b$. Thus every integer at least $(a-1)(b-1)$ is representable, and this threshold is sharp.*

Proof. Make a directed graph on $\mathbb{Z}/a\mathbb{Z}$, with an edge $r \to r + b$ of weight b from each vertex. Since $\gcd(a, b) = 1$, these edges form one directed cycle, visiting $0, b, 2b, \ldots, (a-1)b \pmod a$ in that order. Adding an a-coin is a loop of weight a at the current vertex.

For $0 \le j < a$, any walk from zero to residue jb uses $j + ka$ of the b-edges for some $k \ge 0$. Its weight is therefore at least jb, and the direct j-edge path attains that weight. The a-loops then attain exactly $jb, jb + a, jb + 2a, \ldots$ in this residue class. Its largest unattainable integer is consequently $jb - a$. Taking the maximum over the cycle gives $(a-1)b - a = ab - a - b$. □

The same graph describes the exact threshold for any finite $A \subseteq \mathbb{Z}_{>0}$ with $\gcd(A) = 1$. Put $M = \max A$, give each edge $r \to r + a \pmod M$ weight a, and let w_r be the least weight of a walk from zero to r. Every residue is reachable because A generates $\mathbb{Z}/M\mathbb{Z}$. Minimality and the available M-loops show that the attainable integers in residue r are exactly $w_r + qM$, $q \ge 0$. Thus the sharp threshold is $1 + \max_r w_r - M$. To bound it without finding the shortest weights, Kneser's theorem will bound the number of edges needed to reach each residue; every edge weighs at most M.

Lemma 110 (A diameter bound for a cyclic sumset)**.** *Let $C \subseteq \mathbb{Z}/m\mathbb{Z}$ contain zero and generate the group. If $b = |C|$ and $h = \lceil 2m/b \rceil$, then $hC = \mathbb{Z}/m\mathbb{Z}$.*

Proof. The case $m = 1$ is immediate. If $(j+1)C$ is proper, its stabilizer H is proper. Because $0 \in C$ and C generates the group, C meets at least two H-cosets. Thus $|C + H| \ge \max\{b, 2|H|\}$, and Kneser gives

$$|(j+1)C| \ge |jC| + |C + H| - |H| \ge |jC| + \max\{b - |H|, |H|\} \ge |jC| + b/2.$$

Since zero belongs to C, the sets jC are nested. If hC were proper, each of the first h steps would gain at least $b/2$ elements, giving $|hC| \ge 1 + hb/2 > m$, a contradiction. □

Theorem 111 (A progression of short repeated sums)**.** *Let $\varnothing \ne A \subseteq [M]$, put $b = |A|$, and let $d = \gcd(A)$. Then $\bigcup_{1 \le j \le \lfloor 8M/b \rfloor} jA$ contains M consecutive positive multiples of d.*

Proof. Divide by d, obtaining $D = A/d$, and put $m = \max D$. The b residues of D modulo m are distinct, contain zero, and generate $\mathbb{Z}/m\mathbb{Z}$. By the diameter bound, every residue has a representative s_r which is a sum of at most $h = \lceil 2m/b \rceil$ elements of D, so $0 \le s_r \le hm$.

For any integer $t \ge hm$, choose the representative of its residue. Then $t = s_r + qm$ for an integer $q \ge 0$; append q copies of $m \in D$. In particular, every integer from hm to $hm + M - 1$ uses at most

$$h + q \le 2h + \left\lceil \frac{M}{m} \right\rceil \le \frac{4m}{b} + \frac{M}{m} + 3 \le \frac{5M}{b} + 3 \le \frac{8M}{b}$$

summands, using $b \le m \le M$. The number of summands is an integer, so it is at most $\lfloor 8M/b \rfloor$. Multiply this positive interval by d. □

Corollary 112 (Unbounded Subset Sum threshold)**.** *If $A \subseteq \mathbb{Z}_{>0}$, $M = \max A$, $b = |A|$, and $\gcd(A) = 1$, every integer $t \geq M\lceil 2M/b \rceil$ is a nonnegative integer combination of elements of A. In particular, $2M^2/b + M$ is a sufficient threshold.*

Proof. The preceding residue representatives have value at most $M\lceil 2M/b \rceil$. Append copies of M to the representative of t mod M. □

The argument also recovers a representation. Breadth-first search in the directed graph with edges $r \to r + a \pmod M$, for $a \in A$, finds a path with the fewest edges to each residue. Its weight is at most M times its length, so the diameter bound makes every representative light enough to pad with copies of M.

Algorithm 15 Unbounded Subset Sum above the elementary threshold

Input: $A \subseteq \mathbb{Z}_{>0}$, $\gcd(A) = 1$, $M = \max A$, $t \geq M\lceil 2M/|A| \rceil$
Output: Nonnegative integer multiplicities $(c_a)_{a \in A}$ with $\sum_a c_a a = t$

```
Initialize seen[0 ... M − 1] ← false
seen[0] ← true; initialize a queue with 0
while the queue is nonempty do
    Remove r from the front
    for each a ∈ A do
        u ← (r + a) mod M
        if seen[u] is false then
            seen[u] ← true
            parent[u] ← r, item[u] ← a
            Append u to the queue
Initialize all c_a ← 0; set r ← t mod M, s ← 0
while r ≠ 0 do
    a ← item[r]
    c_a ← c_a + 1, s ← s + a
    r ← parent[r]
c_M ← c_M + (t − s)/M
return (c_a)_{a∈A}
```

Preprocessing uses $O(M|A|)$ arithmetic operations and $O(M)$ space. Reconstruction follows at most $\lceil 2M/|A| \rceil$ parent edges, in addition to initializing the $|A|$ output counts. The counts provide a compact answer even when t is large.

5.3.7 A Subset-Sum Intersection Bound

In a Knapsack exchange argument, bounding the number of changed items does not by itself bound their total weight in another part. The next theorem supplies this second bound: a multiset with sufficiently many occurrences and bounded multiplicity shares a positive subset sum with every multiset of sufficiently large total weight.

Theorem 113 (Subset-sum intersection)**.** *Let X, Y be multisets of integers in $[W]$, with $r = |X| \geq 1$, and suppose every value occurs at most μ times in X. If*

$$r^2 \geq 256\mu W\lceil \log(2r) \rceil, \qquad \Sigma(Y) \geq \frac{128\mu W^2}{r},$$

then $\mathcal{S}(X) \cap \mathcal{S}(Y)$ contains a positive integer.

The useful contrapositive is that, under the density condition on X, having no common positive subset sum forces $\Sigma(Y) < 128\mu W^2/r$. Size and bounded multiplicity produce a progression in $\mathcal{S}(X)$, and the total weight of Y forces it to hit that progression.

Why a progression suffices. Suppose $\mathcal{S}(X)$ contains consecutive positive multiples of d. Group occurrences of Y into small disjoint blocks whose weights are divisible by d. Their running sums stay aligned with the progression; if the progression is long enough, the first running sum to reach its beginning cannot jump past its end.

Lemma 114 (Hitting a progression)**.** *Let Y be a multiset in $[W]$, and let a, d be positive integers with $d \mid a$. If $\Sigma(Y) \geq a + (d-1)W$, then $\mathcal{S}(Y)$ intersects $\{a, a+d, \ldots, a+(W-1)d\}$.*

Proof. Order any d remaining occurrences. Among their $d+1$ prefix sums, two agree modulo d; the intervening nonempty block has weight divisible by d and at most dW. Repeatedly extract such a block until fewer than d occurrences remain.

The resulting disjoint block weights $q_1, \ldots, q_k$ are positive multiples of d, each at most dW, and their total is at least $\Sigma(Y) - (d-1)W \geq a$. Take the shortest prefix whose weight q is at least a. Its predecessor is below a, and its last block weighs at most dW, so $a \leq q < a + dW$. As $d \mid q$, this is one of the desired progression points; disjointness makes it a subset sum of Y. □

From repeated sums to subset sums. The short-sums theorem allows repetition, whereas a subset sum may use each occurrence only once. To bridge the gap, use a set of *popular pair sums*: each available summand has many disjoint representations by two occurrences of X. A short representation then has enough alternatives to avoid every occurrence already used.

Proof of Theorem 113. Regard occurrences as distinct objects, and let m_u be the multiplicity of value u in X. For each integer z, let t_z be the maximum number of pairwise disjoint occurrence pairs with sum z. Thus $t_z \leq \lfloor r/2 \rfloor$. Disjointness is required within each family for a fixed sum; families for different sums may overlap.

Count the available pair representations. For distinct values u, v, we can form $\min\{m_u, m_v\} \geq m_u m_v/\mu$ disjoint pairs. For a single value u, we can form $\lfloor m_u/2 \rfloor \geq m_u(m_u-1)/(2\mu)$ pairs. For a fixed sum z, complementary value classes do not overlap: value u can pair only with $z-u$. We can therefore take all these pairs simultaneously for that sum. Summing over sums gives

$$\sum_z t_z \geq \frac{1}{\mu} \sum_{u<v} m_u m_v + \frac{1}{2\mu} \sum_u m_u(m_u-1) = \frac{r(r-1)}{2\mu} \geq \frac{r^2}{4\mu}. \tag{9}$$

Here $r \geq 2$ follows from the density hypothesis. Bounded multiplicity has converted the $\binom{r}{2}$ occurrence pairs into at least $\binom{r}{2}/\mu$ disjoint representations.

Choose a threshold that provides both variety and alternatives. For $1 \leq v \leq \lfloor r/2 \rfloor$, put $B_v = \{z : t_z \geq v\}$ and $b_v = |B_v|$. We want b_v large, so the short-sums theorem uses few summands, and vb_v large, so those few choices cannot exhaust the v available pairs for any sum.

Set $L_r = \lceil \log(2r) \rceil$. Some threshold satisfies

$$b := b_v \geq \frac{r}{4\mu} + \frac{r^2}{8\mu L_r v}. \tag{10}$$

Indeed, counting each z once for every $v \leq t_z$ gives $\sum_v b_v = \sum_z t_z$. If (10) failed for all v, then

$$\sum_z t_z < \frac{r}{4\mu} \left\lfloor \frac{r}{2} \right\rfloor + \frac{r^2}{8\mu L_r} \sum_{v=1}^{\lfloor r/2 \rfloor} \frac{1}{v} \leq \frac{r^2}{8\mu} + \frac{r^2}{8\mu} = \frac{r^2}{4\mu},$$

contradicting (9). We used $\sum_{v \leq r/2} 1/v \leq 1 + \log(r/2) \leq L_r$. Fix this v. The two useful consequences, together with density, are $b \geq r/(4\mu)$ and $vb \geq r^2/(8\mu L_r) \geq 32W$.

Realize a long progression without reusing occurrences. Apply Theorem 111 to $B_v \subseteq [2W]$. It gives $2W$ consecutive positive multiples of $d = \gcd(B_v)$, forming a progression P, each represented using at most $H = \lfloor 16W/b \rfloor$ elements of B_v. Our threshold gives $H \le 64\mu W/r$ and $2H \le v$.

Fix a representation $z_1 + \cdots + z_h$ of a point of P, where $h \le H$. For each z_i, fix v disjoint occurrence pairs of sum z_i, and choose one pair from each family in order. Before choice i, exactly $2(i-1)$ occurrences have been used. Because the candidates within this family are disjoint, each used occurrence forbids at most one candidate. At most $2(i-1) < 2H \le v$ candidates are forbidden, so an unused pair remains. This works even if some z_i's coincide. Consequently $P \subseteq \mathcal{S}(X)$.

Check that the progression starts early enough. Put $a = \min P$. Every point uses at most H pair sums, each at most $2W$, so $\max P \le 2WH$. Also, B_v contains b distinct positive multiples of d in $[2W]$, whence $bd \le 2W$. Therefore

$$a \le \max P \le \frac{128\mu W^2}{r}, \qquad d \le \frac{2W}{b} \le \frac{8\mu W}{r}. \tag{11}$$

Since P has $2W$ terms, its endpoint is $\max P = a + (2W-1)d \ge a + Wd$. The weight hypothesis therefore gives $\Sigma(Y) \ge \max P \ge a + (d-1)W$. The hitting lemma now finds a positive subset sum of Y among the first W points of P, all of which belong to $\mathcal{S}(X)$. □

5.3.8 Witnesses for Long Progressions in Repeated Sumsets

An existential progression is insufficient when an algorithm must produce the summands of a requested term. The construction below first finds a progression with a small step d, then fills its missing residue classes using a table modulo d. This table is cheap either in preprocessing or in the query: if $d > |A|$, then $d = O(k)$, so it can be deferred to a query whose answer already permits $O(k)$ summands.

Theorem 115 (Constructive repeated-sum progression)**.** *Let $A \subseteq \{0, \dots, M\}$, where $M \ge 1$, $0 \in A$, $\gcd(A) = 1$, and $n = |A| \ge 2$. Put $k = \lceil (M+1)/n \rceil$. In expected $\widetilde{O}(n)$ preprocessing time, one can find an integer T and a data structure certifying $\{T, T+1, \dots, T+M\} \subseteq 328kA$. For any requested term, it returns an explicit list of $328k$ summands in expected $\widetilde{O}(k)$ time. Preprocessing space is $\widetilde{O}(n)$, and query workspace is $\widetilde{O}(n+k)$.*

The constants merely leave room between the estimates. The query bound is for an explicit list of summands. Zeros can instead be recorded as one multiplicity. Under the theorem's normalization, zero padding places the same interval in every hA with $h \ge 328k$. For a general nonsingleton integer set, subtract its minimum and divide by the gcd of the differences; a representation with a fixed number of terms can then be translated back.

A greedy representation succeeds on most targets. For $B \subseteq [0, D]$, define its prefix density up to D by $\rho_D(B) = \min_{1 \le x \le D} |B \cap [1, x]|/x$. Given a target, greedily subtract the largest available element not exceeding the remainder. Greedy need not always finish quickly, but positive prefix density makes it finish quickly on most targets in every prefix. Splitting a target randomly into two such easy targets then gives a Las Vegas witness.

Lemma 116 (Greedy splitting)**.** *Suppose $\{0, 1\} \subseteq B \subseteq [0, D]$ and $\rho_D(B) \ge \rho > 0$. For an integer $h \ge 2/\rho$, every $t \in [0, D]$ can be represented by $2h$ elements of B in expected $O(h \log(2|B|))$ time after sorting B.*

Proof. Let G_j be the targets in $[0, D]$ that greedy represents in at most j steps; zero belongs to every G_j. Fix $x \le D$, list $B \cap [1, x] = \{b_1 < \cdots < b_q\}$, and put $b_{q+1} = x + 1$. Since $b_1 = 1$, the intervals $[b_i, b_{i+1} - 1]$ partition $[1, x]$. In the interval beginning at b_i, greedy first takes b_i, and succeeds precisely when the remaining offset belongs to G_j. Writing $\gamma_j = \rho_D(G_j)$, this gives at least $q + \gamma_j(x - q)$ successful targets. Consequently $\gamma_{j+1} \ge \rho + (1-\rho)\gamma_j$, and induction gives $\gamma_h \ge 1 - (1-\rho)^h \ge 1 - e^{-2} > 3/4$.

Choose u uniformly in $\{0, \dots, t\}$. Each of the tests $u \in G_h$ and $t - u \in G_h$ fails on at most $t/4$ choices, so both succeed with probability at least $1/2$. Run greedy on both parts and retry if either needs more than h steps. Each trial costs $O(h \log(2|B|))$, and a successful trial returns the desired representation, padded with zeros. □

Algorithm 16 Greedy witnesses and random splitting

```
function Greedy(B, t, h)
    Q ← [ ]
    for j = 1, ..., h do
        Find b = max(B ∩ [0, t]) by binary search
        Append b to Q; set t ← t − b
    If t = 0, return Q; otherwise return ⊥
function Split(B, t, h)
    repeat
        Choose u uniformly from {0, ..., t}
        Q_1 ← Greedy(B, u, h), Q_2 ← Greedy(B, t − u, h)
    until neither answer is ⊥
    return the concatenation of Q_1, Q_2
```

From total density to prefix density. A set can be large while leaving a long initial gap. A prefix-count function locates an anchor from which it is dense in one direction. The available elements $0, 1$ let us translate the resulting witnesses back without subtracting any input elements.

Lemma 117 (An interval when zero and one are available)**.** *Suppose $\{0, 1\} \subseteq B \subseteq [0, D]$ and $|B| \geq (D+1)/q$, where $q \geq 1$ is an integer. In $O(|B| \log(2|B|))$ time one can construct an integer s and a witness for $[s, s + D] \subseteq 32qB$, with expected $O(q \log(2|B|))$ time per query.*

Proof. The obstacle is that a large set need not have dense initial prefixes. We first find a point from which B is dense in one direction. Measuring distances from that point produces a set E to which the greedy-splitting lemma applies. The elements $0, 1 \in B$ then let us replace each E-summand by two B-summands.

Find a dense direction. Define $f(x) = 2q|B \cap [0, x]| - (x + 1)$ for integer $-1 \leq x \leq D$. This measures the surplus of points over density $1/(2q)$: for $x < y$, $f(y) - f(x) = 2q|B \cap [x + 1, y]| - (y - x)$. Thus an interval on which f does not decrease has the desired density. Also, f decreases by at most one per step.

Let u minimize f and v maximize f. Since $f(-1) = 0$ and $f(D) \geq D + 1$, we have $f(u) \leq 0$ and $f(v) \geq D + 1$. Either $u \leq D/2$ or $v > D/2$. Indeed, otherwise $0 \leq v \leq D/2 < u \leq D$, so f would drop by at least $D + 1$ in at most D steps.

If $u \leq D/2$, look to the right of u. Minimality gives $|B \cap [u + 1, u + x]| \geq x/(2q)$ whenever $1 \leq x \leq D - u$. Set $E = \{0\} \cup ((B - u) \cap [1, D])$.

Otherwise $v > D/2$, and we look to the left of $v + 1$. Maximality gives $|B \cap [v + 1 - x, v]| \geq x/(2q)$ whenever $1 \leq x \leq v + 1$. Set $E = \{0\} \cup ((v + 1 - B) \cap [1, D])$.

In either case, the available direction has length at least $D/2$. Up to that length, every prefix of E has density at least $1/(2q)$. A longer prefix still contains at least $D/(4q)$ positive elements, so its density is at least $1/(4q)$. Consequently $\rho_D(E) \geq 1/(4q)$; the length-one prefix also shows that $1 \in E$. By Lemma 116, every integer in $[0, D]$ has a witness using exactly $h = 16q$ elements of E.

Convert the witnesses back to B. In the right-facing case, $\{u + 1\} + E \subseteq B + B$. For $e > 0$, use $u + 1 + e = 1 + (u + e)$; for $e = 0$, use $u + 1 = 0 + (u + 1)$. Both are valid because $u + e \in B$ for positive $e \in E$, and $1 \in E$ implies $u + 1 \in B$. Thus a representation of $j \in [0, D]$ by h elements of E becomes a representation of $h(u + 1) + j$ by $2h$ elements of B. Take $s = h(u + 1)$.

In the left-facing case, $\{v+1\} - E \subseteq B + B$. For $e > 0$, use $v + 1 - e = 0 + (v + 1 - e)$; for $e = 0$, use $v + 1 = 1 + v$. Here $1 \in E$ ensures $v \in B$. To represent $s + j$, where $s = h(v + 1) - D$, represent $D - j$ by h elements of E and apply these replacements. Their sum is $h(v + 1) - (D - j) = s + j$. Again the result uses exactly $2h = 32q$ elements of B.

Construction and query time. Sort B. Between consecutive elements of B, the function f decreases by one per step, and it jumps upward at each element. Its extrema therefore occur among $-1, D$, the elements $b \in B$, and their predecessors $b - 1$. Evaluate these $O(|B|)$ candidates using their sorted ranks; there is no need to scan $[0, D]$. Constructing and sorting E takes $O(|B| \log(2|B|))$ time. Each query makes one greedy-splitting call, taking expected $O(q \log(2|B|))$ time, followed by $O(q)$ replacements. □

Algorithm 17 The interval witness $\textsc{Seed}(B, D, q)$

```
function Seed(B, D, q)
    Sort B; evaluate f(x) = 2q|B ∩ [0, x]| − (x + 1) at the extrema candidates in the proof
    Choose a minimizing position u
    if u ≤ D/2 then
        σ ← +1, s ← 16q(u + 1)
        E ← {0} ∪ {b − u : b ∈ B, 1 ≤ b − u ≤ D}
    else
        Choose a maximizing position v; set u ← v + 1
        σ ← −1, s ← 16qu − D
        E ← {0} ∪ {u − b : b ∈ B, 1 ≤ u − b ≤ D}
    return the record (E, D, q, u, σ, s), with E sorted
function SeedQuery(W, j)                    ▷ Returns a witness for s + j, 0 ≤ j ≤ D
    Read (E, D, q, u, σ, s) from W
    z ← j if σ = +1, and z ← D − j otherwise
    Q ← Split(E, z, 8q)
    Replace each entry by its pair from the proof of Lemma 117
    return the resulting list of 32q elements of B
```

A small step, followed by a residue table. The closest pair in A supplies a small positive difference d. Adding that pair to a large residue class creates a normalized set containing zero and one. The preceding lemma builds a progression in that class. Its length is chosen to leave enough room for the residue representatives used to fill its gaps: if there are t occupied residues, a representative needs $O(d/t)$ summands, so a progression spanning $\Theta(Md/t)$ suffices.

For the residue table we use the following elementary witness routine. Its dense convolution operations have length equal to the small modulus, not the original numerical range.

Lemma 118 (Dense modular convolution with witnesses)**.** *Given $X, Y \subseteq \mathbb{Z}/d\mathbb{Z}$, one can compute $X + Y$ and one pair $(x_z, y_z) \in X \times Y$ with $x_z + y_z \equiv z \pmod d$ for every output residue z, in expected $\widetilde{O}(d)$ time.*

Proof. A dense cyclic convolution of the indicator arrays gives the support. To obtain witnesses, independently retain each element of X with probability 2^{-j}, for each $j = 0, \ldots, \lceil \log_2 d \rceil$. If X' is the retained set, compute the two cyclic convolutions $c = 1_{X'} * 1_Y$ and $w = (x 1_{X'}(x)) * 1_Y$. Whenever $c_z = 1$, its unique representation is $(w_z, (z - w_z) \bmod d)$.

For a residue with $a \geq 1$ representations, the corresponding first coordinates are distinct. One of the probabilities lies in $[1/(2a), 1/a]$. If $a > 1$, it isolates exactly one representation with probability at least $\frac{1}{2}(1 - 1/a)^{a-1} \geq 1/(2e)$; if $a = 1$, retaining everything succeeds. A full pass over the probabilities therefore finds each missing witness with constant probability. Repeat passes until all known output residues have witnesses. A union bound gives $O(\log(2d))$ expected passes. Every discovered witness is exact, and the total cost is $\widetilde{O}(d)$. □

Algorithm 18 Dense modular pair witnesses

function PairWitness(X, Y, d)
  Compute $Z = \mathrm{supp}(1_X * 1_Y)$ by cyclic convolution modulo d
  Mark the parent pair of every $z \in Z$ as missing
  while some parent pair is missing **do**
    for $j = 0, \ldots, \lceil \log_2 d \rceil$ **do**
      Retain each $x \in X$ independently with probability 2^{-j}, forming X'
      Compute $c = 1_{X'} * 1_Y$ and $w = (x 1_{X'}(x)) * 1_Y$
      For every z with $c_z = 1$, store the pair $(w_z, (z - w_z) \bmod d)$
  return Z and all stored pairs

Lemma 119 (Short residue representatives)**.** *Let $d \geq 1$, $0 \in A \subseteq [0, M]$, $\gcd(A \cup \{d\}) = 1$, and let $C = A \bmod d$ have size t. Let H be the smallest power of two at least $2d/t$. In expected $\widetilde{O}(|A| + d)$ time one can store, for every residue r, a value $w_r \leq HM$ and a representation of $w_r \equiv r \pmod d$ by exactly H elements of A. Each representation can be expanded in $O(H)$ time.*

Proof. First, $HC = \mathbb{Z}/d\mathbb{Z}$. For completeness, if $(j+1)C$ is proper with stabilizer J, then C meets at least two J-cosets, since it contains zero and generates the cyclic group. Kneser gives $|(j+1)C| \geq |jC| + |C + J| - |J| \geq |jC| + t/2$, using $|C + J| \geq \max\{t, 2|J|\}$. Since $0 \in C$, the repeated sumsets are nested. If HC were proper, the first H additions would therefore give more than d elements. The case $d = 1$ is immediate.

Choose one original representative $a_r \in A$ of every $r \in C$. Starting with C, repeatedly square modulo d, using Lemma 118 to store parent pairs at each level. At level zero store weight a_r; at higher levels store the sum of the weights of the two chosen parents. At the final level every residue is present and its weight is at most HM. Recursively following parents produces H leaves and takes $O(H)$ time. There are $\log_2 H = O(\log(2d))$ levels, so the construction takes expected $\widetilde{O}(|A| + d)$ time. □

Algorithm 19 Building and reading the residue table

function ResidueTable(A, d, H)
  $C_0 \leftarrow A \bmod d$; choose a representative $a_r \in A$ for each $r \in C_0$
  Set $w_0(r) \leftarrow a_r$ and $h \leftarrow \log_2 H$
  for $j = 1, \ldots, h$ **do**
    $(C_j, P_j) \leftarrow$ PairWitness(C_{j-1}, C_{j-1}, d)
    For $r \in C_j$ with $P_j(r) = (u, v)$, set $w_j(r) \leftarrow w_{j-1}(u) + w_{j-1}(v)$
  return $(a_r, P_1, \ldots, P_h, w_h)$
function Expand(j, r, Q) ▷ *Append to one shared output list*
  if $j = 0$ **then**
    Append a_r to Q; return
  Let $(u, v) = P_j(r)$
  Expand($j - 1, u, Q$); Expand($j - 1, v, Q$)

Proof of Theorem 115.

Build a progression with a small step. Sort A and find a closest pair $a_*, a_* + d$. Then $1 \leq d \leq M/(n-1) \leq 2M/n \leq 2k$. Put $t = |A \bmod d|$, choose a largest residue class $A' \subseteq A$, and let $a_0 = \min A'$. The set $B = ((A' - a_0)/d) + \{0, 1\}$ contains zero and one, and every $b \in B$ has a stored pair of original elements summing to $a_0 + a_* + db$. Also $|B| \geq |A'| + 1 \geq n/t + 1$.

Set $\ell = \lceil 5M/t \rceil$. Since $t \leq d$, we have $\max B \leq M/d + 1 \leq \ell$, and $|B| \geq n/t + 1 \geq (\ell + 1)/(5k)$. The last inequality uses $\ell + 1 \leq 5M/t + 2$ and $kn \geq M + 1$. Apply Lemma 117 to B, with range ℓ and parameter $5k$. It supplies an interval $[s_B, s_B + \ell] \subseteq 160kB$. Replacing each summand by its stored pair gives

$P = \{s, s+d, \ldots, s+\ell d\} \subseteq 320kA$, where $s = 160k(a_0 + a_*) + ds_B$. All this takes $O(n \log(2n))$ time, and a requested term of P has a witness in expected $O(k \log(2n))$ time.

Fill the residue classes. Let H be the smallest power of two at least $2d/t$. Then $H \le 4d/t \le 8k$, and $\ell d \ge 5Md/t \ge HM + M$. The residue table provides $w_r \in HA$, with $0 \le w_r \le HM$ and $w_r \equiv r \pmod d$. Take $T = s + HM$. For $x \in [T, T + M]$, let $r = (x - s) \bmod d$. Then $x - w_r \equiv s \pmod d$, while $s \le x - w_r \le s + \ell d$. Thus $x - w_r \in P$. Combine its $320k$-term witness with the H-term witness for w_r, and pad with zeros to obtain exactly $328k$ terms.

When to build the table. If $d \le n$, build it during preprocessing, in expected $\widetilde{O}(n)$ time. If $d > n$, store the residue lists but build the table inside a query. In that case $n < d \le 2k$, so its expected cost is $\widetilde{O}(k)$. Expansion costs $O(H) = O(k)$, as does handling the output list. These observations give all the claimed time and space bounds. □

Algorithm 20 A repeated-sum progression and its witnesses

```
1:  function ProgressionQuery(D, x)                                  ▷ T ≤ x ≤ T + M
2:      Read the construction records from D
3:      Use the stored table R, or build it now if d > n
4:      r ← (x − s) mod d, w ← w_{log2 H}(r)
5:      j ← (x − s − w)/d, Q ← SeedQuery(W, j)
6:      Replace every b ∈ Q by its stored pair of original elements
7:      Expand(log2 H, r, Q); append zeros to reach 328k entries
8:      return the resulting list
```

5.3.9 Witnesses for Long Progressions in Subset Sums

Write $\mathcal{S}(X) = \{\sum_{x \in Y} x : Y \subseteq X\}$ and $\Sigma_X = \sum_{x \in X} x$. Repeated-sum witnesses may reuse an element; subset-sum witnesses must remember which items remain available. Disjoint pairs provide a convenient way to enforce this. Start by taking the smaller endpoint of every pair. Using a gap means switching that pair to its larger endpoint. Several pairs with the same gap therefore provide several independent copies of that gap.

This gives a short initial progression using the repeated-sum theorem. To extend it, partition the unused numbers into short intervals and pair numbers with different residues modulo the current step d. Either there are enough such pairs to fill a smaller lattice, or most numbers remain in groups having the same residue. Pairing the extremes of those groups then lengthens the progression without changing d. Each step uses a fresh collection of disjoint pairs.

Theorem 120 (Constructive subset-sum progression)**.** *Let $X \subseteq \{1, \ldots, M\}$ have n distinct elements, put $\Lambda = \lceil \log_2(2n) \rceil$, and let ℓ be an integer with $M \le \ell \le n^2/(C\Lambda)$, where $C = 2^{30}$ suffices. In expected $\widetilde{O}(n)$*

time, one can find $X_* \subseteq X$, integers $s, d > 0$, and a witness for $\{s, s+d, \dots, s+\ell d\} \subseteq \mathcal{S}(X_*)$, with $d \le 5M/n$ and

$$|X_*| = O\left(\frac{\ell + M\log(2n)}{n}\right), \qquad \textit{expected query time } \widetilde{O}\left(\frac{\ell + M\log(2n)}{n}\right).$$

Every query returns distinct original items. In fact, all answers choose exactly one endpoint from each pair in a fixed matching on X_*.

The constant is only for the bookkeeping below. Taking $\ell = M$ gives the usual density requirement $n^2 \ge CM\Lambda$. The hypotheses imply $n \ge C\Lambda$ and $\ell/n \ge 1$; in particular, all rounding estimates below have ample slack. The progression is stored by its three parameters; its terms are never enumerated. For a multiset, the theorem may be applied to one chosen copy of each distinct value, with n counting those distinct values. Its density hypothesis cannot use the total multiplicity in place of n.

Turning repeated gaps into distinct items. All pairs below are oriented as (a, b) with $a < b$, and their endpoints are distinct across the entire collection. A pair carries its original item indices as well as its values. For a collection T, put $B_T = \sum_{(a,b)\in T} a$. If a gap value g has q available pairs, it may be used at most q times: switch that many different pairs. A list of requested gaps then gives a subset of weight $B_T + \sum g$. Unused pairs contribute their smaller endpoints, rather than disappearing.

We need one elementary way to obtain uniform multiplicities. Suppose $p \le n$ objects have labels, with label frequencies f_v. There is a threshold $u \ge 1$ such that, for $F = \{v : f_v \ge u\}$, we have $u|F| \ge p/\Lambda$. Indeed, writing $N_j = |\{v : f_v \ge j\}|$, we have $p = \sum_{j=1}^{p} N_j \le (\max_j jN_j)\sum_{j=1}^{p} 1/j \le \Lambda \max_j jN_j$. Sorting the frequencies finds a maximizing threshold in $O(p\log(2p))$ time. We may then keep any prescribed number at most u of objects of each retained label.

Algorithm 21 Uniform labels and independent endpoint choices

```
function Uniform(T, label)
    Group the objects by label into lists T_v; set f_v ← |T_v|
    Sort the frequencies as f_1 ≤ ··· ≤ f_t
    Choose i maximizing f_i(t − i + 1); set u ← f_i
    return u and the lists T_v with f_v ≥ u
function ChooseEndpoints((T_v)_v, Q)        ▷ Each nonzero label occurs in Q at most |T_v| times
    Set a counter to zero for each list; select every pair's smaller endpoint
    w ← 0
    for each nonzero label v in Q do
        Increment the counter of T_v; let (a, b) be that pair
        Switch its selected endpoint to b; set w ← w + b − a
    return w and the list of selected original items
```

The initial progression. Set $L_0 = \lceil 512\ell/n \rceil$ and $G = \lceil 4M/n \rceil$. Sort X, pair consecutive elements as $(x_1, x_2), (x_3, x_4), \dots$, and retain pairs whose gap is at most G. All the original paired gaps sum to at most M, so fewer than M/G pairs are discarded. At least $n/8$ pairs remain. Uniformize their gap labels, obtaining t distinct gaps, each available at least u times, with $ut \ge n/(8\Lambda)$.

Let d_0 be the gcd of the retained gaps and $A = \{0\} \cup \{g/d_0 : g \text{ is a retained gap}\}$. Then $A \subseteq [0, L_0]$, $|A| = t + 1$, and $\gcd(A) = 1$. Apply Theorem 115 with ambient range L_0. It supplies an interval $[T, T + L_0] \subseteq qA$, where $q = 328\lceil (L_0 + 1)/(t + 1) \rceil$. Since $1 \le t \le G \le L_0$, we have $qt \le 656L_0$. The density hypothesis gives $656L_0 \le n/(8\Lambda) \le ut$; thus $q \le u$.

Keep q pairs of each retained gap, and let B be the sum of their smaller endpoints. Multiplying a repeated-sum witness by d_0 and switching distinct pairs gives a witness for $\{B + d_0(T + j) : 0 \le j \le L_0\} \subseteq \mathcal{S}(X_0)$, with step $d_0 \le G \le 5M/n$. There are at most $656L_0$ pairs, so $|X_0| \le 1312L_0 < 2^{20}\ell/n$. The construction takes expected $\widetilde{O}(n)$ time and its query takes expected $\widetilde{O}(L_0)$ time. Only these retained pairs are removed from the reservoir of unused items.

Algorithm 22 A subset-sum seed

function SUBSETSEED(X, M, L_0)
  Sort X; form disjoint consecutive pairs with gap at most $G = \lceil 4M/|X| \rceil$
  Apply UNIFORM with label $b - a$, obtaining t retained labels
  $d_0 \leftarrow \gcd\{g : g \text{ is a retained label}\}$, $q \leftarrow 328\lceil (L_0+1)/(t+1) \rceil$
  Keep q pairs per label; relabel their lists by g/d_0
  $B \leftarrow \sum a$, over all kept pairs; $A \leftarrow \{0\} \cup \{g/d_0\}$
  $(T, \mathcal{W}) \leftarrow$ PROGRESSION(A, L_0)
  Store the lists, $B, d_0, \mathcal{W}$; return the progression $(B + d_0 T, d_0, L_0)$ and its record
function SUBSETSEEDQUERY(record, z)
  Read the pair lists, $B, d_0, \mathcal{W}$ from the record
  $Q \leftarrow$ PROGRESSIONQUERY($\mathcal{W}, (z - B)/d_0$)
  $(w, Y) \leftarrow$ CHOOSEENDPOINTS($(T_v)_v, Q$)
  return Y

A batch of pairs either changes the step or extends the length. Suppose the current progression is $P = \{s, s+d, \ldots, s+Ld\}$, where $L_0 \le L < \ell$ and $d \le 5M/n$. Let U be the unused items; for now assume $|U| \ge n/2$. We will verify this throughout the construction. Put $\Delta = \lfloor L/32 \rfloor$ and partition $[1, M]$ into consecutive intervals of $\Delta + 1$ integers. There are at most $h \le M/\Delta + 1 \le 64M/L + 1$ intervals, and two numbers in the same interval differ by at most Δ. This choice lets $O(d)$ gap switches move a sum by only a fixed fraction of the current span Ld. If too few switches change the residue, density instead forces many d-spacings inside the intervals.

Within each interval, repeatedly pair two numbers of different residues modulo d. This can be done by grouping by residue and maintaining a list of nonempty groups. Remove the two chosen endpoints from that interval's temporary lists. Stop immediately if $4d\Lambda$ pairs have been found. Otherwise finish all intervals; the unpaired numbers in each interval then have one common residue. These are temporary pairings: only pairs ultimately selected for the witness will be removed from U.

Many different-residue pairs: reduce the step. Uniformize the $4d\Lambda$ pairs by their nonzero gap residues. There are t retained residues, each available u times, with $ut \ge 4d$. Put $d' = \gcd(d, \text{retained residues})$ and $D = d/d'$. Since every residue is nonzero, $d' < d$. The set $C = \{0\} \cup \{r/d' : r \text{ retained}\}$ generates $\mathbb{Z}/D\mathbb{Z}$.

Let H be the smallest power of two at least $2D/(t+1)$. Then $Ht < 4D \le 4d \le ut$, so we can keep H pairs of each retained residue. By Lemma 119, in expected $\widetilde{O}(D)$ time the table RESIDUETABLE(C, D, H) gives an H-term representation of every residue modulo D. Replace each nonzero term r/d' by a different kept pair having gap congruent to r modulo d. For every residue class in $d'\mathbb{Z}/d\mathbb{Z}$, this gives an actual gap sum w in that class, with $0 \le w \le H\Delta \le 4d\Delta$. No pair is reused, even when the same residue occurs repeatedly.

Let B be the sum of the smaller endpoints of all kept pairs. The new progression has start $s' = s + B + 4d\Delta$, step d', and length $L' = (L - 4\Delta)d/d' \ge 7L/4$. For any term z of this progression, choose the table residue $(z - s - B)/d' \bmod D$, expand it, and perform the corresponding switches. Then $z - B - w \equiv s \pmod d$ and $s \le z - B - w \le s + Ld$, so it is a term of P. This growth step consumes fewer than $8d$ items. Its additional query work is $O(Ht) = O(d)$.

Few different-residue pairs: extend with equal residues. There are fewer than $4d\Lambda$ temporary pairs. Thus the number R of unpaired items is at least $n/2 - 8d\Lambda \ge n/3$. Since $L \ge 512M/n$, we have $h \le n/8 + 1 \le n/6$, hence $R \ge 2h$. Within each interval, pair the smallest remaining number with the largest, the second smallest with the second largest, and so on. Every resulting gap is a positive multiple of d and at most Δ.

An interval with q_i remaining numbers supplies total gap at least $d\lfloor q_i^2/4 \rfloor$: the j-th extreme pair spans at least $q_i + 1 - 2j$ gaps of size d. Consequently the total gap W of all these pairs satisfies $W \ge d(R^2/h - h)/4 \ge dR^2/(8h)$, by Cauchy–Schwarz and $R \ge 2h$. In particular $W \ge Ld$, since $72hL \le 4608M + 72L \le 4680\ell \le n^2$.

There are at most $R/2$ pairs, so their average gap is at least $dR/(4h)$. Take pairs in decreasing gap order until their gap sum first reaches Ld. An initial segment has average at least the overall average, so at most $\lceil 4Lh/R \rceil \le 1 + (1024M + 16L)/n$ pairs suffice. Write their gaps as $d\delta_1, \dots, d\delta_q$, and put $B = \sum a_i$. Each $1 \le \delta_i \le L$, so the two intervals $[0, L]$ and $\delta_i + [0, L]$ overlap or touch. Successively choosing the lower or upper endpoints therefore gives the progression with start $s + B$, step d, and length $L' = L + \sum_i \delta_i \ge 2L$.

The witness is equally simple. Starting with the requested progression index, undo the pairs in reverse order. If the index exceeds the length before that pair was added, choose the upper endpoint and subtract δ_i; otherwise choose the lower endpoint. The index then lies in the preceding interval. After all pairs have been undone, query the original progression. The step consumes at most $2 + (2048M + 32L)/n$ items.

Algorithm 23 One growth step using short intervals

function IntervalPairs(U, d, Δ, p)
  Group U by $\lfloor (x-1)/(\Delta+1) \rfloor$, then by residue modulo d
  $T \leftarrow \varnothing$
  for each interval **do**
    Keep a list of its nonempty residue groups
    while at least two groups remain **do**
      Pop one item from each of two groups; orient and append their pair to T
      Remove empty groups from the list
      If $|T| = p$, return (`residues`, T)
  In each remaining group, sort the items and pair its extremes
  return (`stretch`, all these extreme pairs)
function Grow(U, s, d, L, n, Λ)
  $\Delta \leftarrow \lfloor L/32 \rfloor$
  $(\tau, T) \leftarrow$ IntervalPairs($U, d, \Delta, 4d\Lambda$)
  if $\tau =$ `residues` **then**
    Uniformize T with label $(b - a) \bmod d$; let t be the number of retained labels
    $d' \leftarrow \gcd(d, \text{retained labels})$, $D \leftarrow d/d'$
    $H \leftarrow$ the smallest power of two at least $2D/(t+1)$
    Keep H pairs per retained label; relabel their lists by r/d'
    $C \leftarrow \{0\} \cup \{r/d'\}$, $\mathcal{R} \leftarrow$ ResidueTable(C, D, H)
    $B \leftarrow \sum a$ over kept pairs
    $s' \leftarrow s + B + 4d\Delta$, $L' \leftarrow (L - 4\Delta)D$
    Store a residue record with $s, d, L, B, d', D, H, \mathcal{R}$ and the pair lists
  else
    Sort T by decreasing gap; keep the shortest prefix of gap sum at least Ld
    $\delta_i \leftarrow (b_i - a_i)/d$ for the kept pairs, $B \leftarrow \sum_i a_i$
    $s' \leftarrow s + B$, $d' \leftarrow d$, $L' \leftarrow L + \sum_i \delta_i$
    Store a stretch record with s, d, L, B and the kept pairs
  return (s', d', L'), the record, and the endpoints of the kept pairs

Proof of Theorem 120. Build the seed, remove its retained endpoints, and repeatedly apply the growth step until the length reaches ℓ. Truncate the last progression to length ℓ. Every new collection of pairs uses unused items, so recursive reconstruction produces distinct items. Each step retains one endpoint of each of its pairs; the same is true of the seed.

We check that the reservoir always has at least $n/2$ items. Each growth factor is at least $7/4$, so there are at most 2Λ steps. Moreover, the sum of their starting lengths is at most $7\ell/3$. The step d stays unchanged in a stretch step and at least halves in a residue step. Thus the values of d at the residue steps sum to at most $2d_0 \le 10M/n$. The seed uses fewer than $2^{20}\ell/n$ items. Summing the two growth bounds, including the additive constant in each stretch step, gives $|X_*| \le 2^{21}(\ell + M\Lambda)/n \le n/256$. These estimates apply to every prefix of the construction. In particular, a first step leaving fewer than $n/2$ unused items would contradict

the same bound. Every growth step is therefore valid.

Each step sorts or scans at most n items. All residue tables together have moduli summing to $O(d_0)$, and the seed construction costs expected $\widetilde{O}(n)$. Hence the total preprocessing takes expected $\widetilde{O}(n)$ time and space. A query traverses one chain of growth records, reads each retained pair once, and makes one repeated-sum query at the seed. Its expected time is $\widetilde{O}(|X_*|)$. The sharper size bound in the statement follows by summing the geometrically increasing lengths, rather than charging ℓ separately at every step. □

Algorithm 24 Building and querying the subset-sum progression

function SubsetProgression(X, M, ℓ)
  $n \leftarrow |X|$, $\Lambda \leftarrow \lceil \log_2(2n) \rceil$, $L_0 \leftarrow \lceil 512\ell/n \rceil$
  Build the seed record and its progression (s, d, L_0)
  Remove its kept endpoints from X, leaving U; set $L \leftarrow L_0$
  while $L < \ell$ **do**
    $((s, d, L), \mathcal{W}, Y) \leftarrow$ Grow(U, s, d, L, n, Λ)
    Append $\mathcal{W}$ to the record list; remove Y from U
  return (s, d, ℓ), all records, and the union X_* of kept endpoints
function UndoGrowth($\mathcal{W}, z, Q$) ▷ *Append new items to one shared list Q*
  Read the stored parameters and pairs from $\mathcal{W}$
  if $\mathcal{W}$ is a residue record **then**
    $r \leftarrow ((z - s - B)/d') \bmod D$; expand residue r in $\mathcal{R}$ into an H-term list V
    $(w, Y) \leftarrow$ ChooseEndpoints($(T_v)_v, V$)
    Append Y to Q; return $z - B - w$
  else
    $j \leftarrow (z - s - B)/d$, $K \leftarrow L + \sum_i \delta_i$
    for kept pairs (a_i, b_i) in reverse order **do**
      $K \leftarrow K - \delta_i$
      if $j > K$ **then**
        Append b_i to Q; set $j \leftarrow j - \delta_i$
      else
        Append a_i to Q
    return $s + dj$
function SubsetProgressionQuery(z)
  $Q \leftarrow$ an empty output list
  For growth records in reverse order, set $z \leftarrow$ UndoGrowth($\mathcal{W}, z, Q$)
  Append SubsetSeedQuery(seed record, z) to Q
  return Q

5.3.10 Additive Energy

Small sumsets force many pairs to produce the same sum. Additive energy measures these collisions even when the sumset is not known to be small. Define $r_{A+B}(x) = |\{(a, b) \in A \times B : a + b = x\}|$ and $E(A, B) = \sum_x r_{A+B}(x)^2$; write $E(A) = E(A, A)$. Equivalently, energy counts quadruples with $a + b = a' + b'$.

Proposition 121 (Energy bounds)**.** *For finite nonempty sets A, B,*

$$\frac{|A|^2|B|^2}{|A + B|} \le E(A, B) \le |A||B| \min\{|A|, |B|\}.$$

Also $E(A, B) = \sum_x r_{A-A}(x) r_{B-B}(x)$.

Proof. The multiplicities sum to $|A||B|$, so Cauchy–Schwarz gives the lower bound. Each multiplicity is at most $\min\{|A|, |B|\}$, giving the upper bound. Finally, rewrite $a + b = a' + b'$ as $a - a' = b' - b$ and count by this common difference. □

Thus $|A+A| \le K|A|$ implies $E(A) \ge |A|^3/K$. A converse cannot control all of A, since a structured part may contribute most of the energy while the remaining elements make the sumset large. Balog–Szemerédi–Gowers finds such a structured part.

5.3.11 Balog–Szemerédi–Gowers

Large additive energy means that many pairs reuse the same sums. We retain a suitable set of sum labels and view their representations as edges of a bipartite graph. Four-edge walks then express endpoint differences as alternating sums of edge labels. If every difference has many such representations, but there are few possible label tuples, the difference set must be small.

Lemma 122 (Many four-edge walks)**.** *Let Γ be bipartite with parts U, V of size $N \ge 1$, and at least δN^2 edges, where $0 < \delta \le 1$. There is $U' \subseteq U$, of size at least $\delta N/2$, such that every ordered $u, u' \in U'$ is joined by at least $\delta^5 N^3/2^{12}$ four-edge walks u, v_1, w, v_2, u'. Vertices may repeat.*

Proof. Call an ordered pair $(u, w) \in U^2$ bad if it has fewer than $\theta = \delta^2 N/32$ common neighbors. We seek a large set in which each vertex has few bad partners. Then any two vertices have many common good partners, each supplying many four-edge walks.

Choose $v \in V$ uniformly and put $W = N_\Gamma(v)$. A pair with c common neighbors belongs to W^2 with probability c/N, so sampling a neighborhood favors pairs with many common neighbors. In particular, $\mathbb{E}|W| \ge \delta N$, hence $\mathbb{E}|W|^2 \ge \delta^2 N^2$. If $b(W)$ counts bad ordered pairs in W^2, then $\mathbb{E}b(W) \le N^2(\theta/N) = \delta^2 N^2/32$.

Consequently, some neighborhood satisfies

$$|W|^2 - 16b(W) \ge \frac{\delta^2 N^2}{2}.$$

Thus $|W| \ge \delta N/\sqrt{2}$ and $b(W) \le |W|^2/16$. Delete vertices with more than $|W|/4$ bad partners in W. At most $4b(W)/|W| \le |W|/4$ vertices are deleted, leaving

$$|U'| \ge \frac{3|W|}{4} \ge \frac{3\delta N}{4\sqrt{2}} \ge \frac{\delta N}{2}.$$

For any $u, u' \in U'$, at least $|W|/2$ vertices $w \in W$ are good partners of both endpoints. Each gives at least θ^2 choices of (v_1, v_2), so the number of walks is at least

$$\frac{|W|}{2}\theta^2 \ge \frac{\delta N}{4}\left(\frac{\delta^2 N}{32}\right)^2 = \frac{\delta^5 N^3}{2^{12}}.$$

□

Lemma 123 (Restricted sums give a small difference set)**.** *Let A, B be subsets of an abelian group with $|A| = |B| = N \ge 1$, and let $\Gamma \subseteq A \times B$ have at least δN^2 edges, where $0 < \delta \le 1$. If $S = \{a + b : (a, b) \in \Gamma\}$ has size at most LN, some $A' \subseteq A$ satisfies*

$$|A'| \ge \frac{\delta N}{2}, \qquad |A' - A'| \le 2^{12} L^4 \delta^{-5} N.$$

Proof. Choose A' using Lemma 122. For each $d \in A' - A'$, fix endpoints a, a' with $d = a - a'$. Every walk a, b_1, c, b_2, a' supplies four labels in S with

$$d = (a + b_1) - (c + b_1) + (c + b_2) - (a' + b_2).$$

For the fixed endpoints, these labels recover b_1, then c, then b_2, so distinct walks give distinct label tuples. Their alternating sum recovers d, so tuples counted for different differences are disjoint. There are therefore at least $|A' - A'|\delta^5 N^3/2^{12}$ distinct tuples in S^4. Since $|S|^4 \le L^4 N^4$, the claimed bound follows. □

The remaining choice is which sums to retain. The restricted-sum lemma gives

$$\frac{|A'-A'|}{|A'|} \le 2^{13}\frac{L^4}{\delta^6} = 2^{13}\left(\frac{L}{\delta^{3/2}}\right)^4.$$

Thus we want few sum labels relative to the density of their representations. We optimize this ratio over popularity thresholds.

Theorem 124 (Balog–Szemerédi–Gowers, energy form)**.** *Let A be a finite nonempty subset of an abelian group. If $|A| = N$ and $E(A) \ge N^3/K$, where $K \ge 1$, some $A' \subseteq A$ satisfies*

$$\boxed{|A'| \ge \frac{N}{4K}, \qquad |A'-A'| \le 2^{21}K^4|A'|.}$$

Proof. Put $r(x) = r_{A+A}(x)$. For $0 < t \le 1$, define $S_t = \{x : r(x) \ge tN\}$, $L_t = |S_t|/N$, and $\delta_t = N^{-2}\sum_{x\in S_t} r(x)$. Thus δ_t is the density of the graph retaining precisely the pairs whose sums lie in S_t. Since every retained sum has at least tN representations, $tL_t \le \delta_t \le 1$.

Integrating over thresholds recovers the normalized energy: a sum with $r(x)$ representations contributes for exactly the thresholds $0 < t \le r(x)/N$. Hence

$$\int_0^1 \delta_t\,dt = \frac{1}{N^2}\sum_x r(x)\int_0^{r(x)/N} dt = \frac{E(A)}{N^3} \ge \frac{1}{K}.$$

Among thresholds with $\delta_t \ge 1/(2K)$, choose one maximizing $\delta_t^{3/2}/L_t$, and denote the maximum by Q. Such thresholds exist because $\delta_t = 1$ for sufficiently small positive t. Only finitely many sets S_t arise, so the maximum is attained.

We claim that $Q \ge 1/(4K)$. For every threshold satisfying the density condition, maximality gives $\delta_t^{3/2} \le QL_t \le Q\delta_t/t$, and therefore $\delta_t \le Q^2/t^2$. At the other thresholds, $\delta_t < 1/(2K)$. Together with $\delta_t \le 1$, these bounds imply

$$\frac{1}{K} \le \int_0^1 \delta_t\,dt \le \frac{1}{2K} + \int_0^\infty \min\left\{1, \frac{Q^2}{t^2}\right\} dt = \frac{1}{2K} + 2Q.$$

The last integral is evaluated by splitting at $t = Q$. This proves the claim.

At the chosen threshold, write $S = S_t$, $L = L_t$, and $\delta = \delta_t$. We have $\delta \ge 1/(2K)$ and $L \le 4K\delta^{3/2}$. Join two copies of A when their sum lies in S. By Lemma 123, some $A' \subseteq A$ satisfies $|A'| \ge \delta N/2 \ge N/(4K)$ and

$$|A'-A'| \le 2^{12}L^4\delta^{-5}N \le 2^{20}K^4\delta N \le 2^{21}K^4|A'|.$$

□

If $|A'-A'| \le D|A'|$, Plünnecke–Ruzsa with base set A' and summand set $-A'$ gives $|A'+A'| \le D^2|A'|$. Here this yields $|A'+A'| \le 2^{42}K^8|A'|$.

The proof also gives a deterministic polynomial-time extraction. Compute the representation counts and scan their distinct values to maximize δ_t^3/L_t^2 subject to $\delta_t \ge 1/(2K)$. For the resulting graph, compute common-neighbor counts, choose a neighborhood with the required score, and prune it as in Lemma 122.

5.3.12 Algorithmic BSG

For algorithms, it is useful to cover most relevant pairs by a few Cartesian products whose full sumsets are small. Each product can then be handled as a batch; the uncovered pairs can be inspected individually.

We count group operations and comparisons of group elements as constant-time operations. Sampling uses independent uniform draws with replacement. Write $C_0 = 2^{22}$.

Theorem 125 (BSG decomposition)**.** *Let A, B, S be subsets of an abelian group, each of size at most $N \geq 1$, and let $0 < \varepsilon < 1$. There is a Las Vegas algorithm with expected running time $O(N^2 \log^2(2N))$ which outputs sets $A_i \subseteq A$, $B_i \subseteq B$, their full sumsets $T_i = A_i + B_i$, and the residual pair list*

$$R = \{(a, b) \in A \times B : a + b \in S\} \setminus \bigcup_{i=1}^{k} (A_i \times B_i).$$

The bounds are $k \leq \lceil 32/\varepsilon \rceil$, $|T_i| \leq C_0 \varepsilon^{-5} N$, and $|R| \leq \varepsilon N^2$. Every returned decomposition is correct; only the running-time bound is in expectation.

The products may overlap, and their sumsets may contain elements outside S. The important guarantees are that they cover the relevant pairs and have few possible sums. Three observations make the construction fast: estimate only the graph statistics we need, charge work to the edges removed, and stop using the delicate construction whenever the claimed sumset bound is already trivial.

A sampling test with a gap. We repeatedly need to recognize small probabilities. A constant-factor gap around the threshold makes the sample count proportional to its reciprocal.

Lemma 126 (Testing a probability threshold)**.** *Let $P : \Omega \to \{0, 1\}$, let $p = \mathbb{E}_{x \in \Omega} P(x)$, and let $0 < \tau \leq 1/2$, $0 < \zeta < 1$. The procedure* Low(Ω, P, τ, ζ) *samples $s = \lceil 8\tau^{-1} \log(2/\zeta) \rceil$ independent points and returns true if their mean is at most τ. Except with probability at most ζ, it returns true whenever $p \leq \tau/2$, and false whenever $p \geq 2\tau$. Thus an accurate true answer implies $p < 2\tau$, and an accurate false answer implies $p > \tau/2$.*

Proof. Let X be the number of sampled successes. If $p \leq \tau/2$, exponential Markov and independence give $\Pr(X > s\tau) \leq (1+p)^s 2^{-s\tau} \leq e^{-s\tau/8}$. If $p \geq 2\tau$, applying Markov to 2^{-X} gives $\Pr(X \leq s\tau) \leq (1-p/2)^s 2^{s\tau} \leq e^{-s\tau/4}$. Here we used $1 + x \leq e^x$, $\log 2 - 1/2 \geq 1/8$, and $1 - \log 2 \geq 1/4$. The chosen sample count makes each relevant error probability at most ζ. For $\tau/2 < p < 2\tau$, either answer is allowed. □

Finding one structured product. Represent the relevant pairs by a bipartite graph Γ on parts U, V of size N; pad smaller parts with isolated vertices. Let $e_\Gamma(X, Y)$ count edges between X and Y. Suppose Γ has at least δN^2 edges, where $0 < \delta \leq 1$. We will find a large U' and a set V' such that every pair in $U' \times V'$ has many three-edge walks between its endpoints. When edges carry sum labels, these walks will bound $|U' + V'|$.

Keep only left vertices of degree at least $\delta N/2$, forming U_0. Choose a right vertex v and examine $W = U_0 \cap N_\Gamma(v)$. Pairs with very few common neighbors rarely occur together in this random neighborhood. We seek a W with few such pairs, remove vertices having too many bad partners, and take right vertices with many remaining neighbors.

The expensive operation would be computing all common-neighbor counts. Instead, choose one random list Q of right vertices and define

$$\theta = \frac{\delta^3}{2^{16}}, \qquad \mathrm{bad}(u, w) = \left[\frac{1}{|Q|} \sum_{z \in Q} \Gamma_{u,z} \Gamma_{w,z} \leq 2\theta \right],$$

where $\Gamma_{u,z}$ is the adjacency-matrix entry. This predicate is evaluated only when requested. The list Q is fixed throughout one call, so it defines an actual symmetric relation; fresh samples are used to estimate densities in that relation. The cutoff $\Theta(\delta^3 N)$ comes from the following balance: $\mathbb{E}|W|^2 = \Omega(\delta^2 N^2)$, and we can tolerate an $O(\delta)$ fraction of bad pairs before pruning becomes costly.

Sampling from $W \times W$ means drawing its two coordinates independently. Each call to Low follows the explicit sampling procedure in Lemma 126.

Lemma 127 (Sampled three-walk lemma)**.** *Whenever Algorithm 25 returns sets, they satisfy $|U'| \geq \delta N/8$ and $e_\Gamma(U', V') \geq \delta N |U'|/4$. With probability at least $1 - \eta$, it returns sets and every $(u, b) \in U' \times V'$ has at least $\delta^5 N^2 / 2^{22}$ three-edge walks u, z, w, b. Vertices may repeat. Writing $\ell = \log(2N/(\delta\eta))$, the running time is $O(N\delta^{-4}\ell^2 + \delta^{-6}\ell^3 + N|U'|)$. The last term is omitted if the procedure returns $\perp$.*

Proof.

Algorithm 25 BsgPiece(Γ, δ, η): sampling a structured product

Input: Adjacency matrix and left degrees of Γ, density lower bound $0 < \delta \le 1$, failure allowance $0 < \eta < 1$
Output: A pair (U', V'), or $\perp$

$U_0 \leftarrow \{u : \deg_\Gamma(u) \ge \delta N/2\}$, $\theta \leftarrow \delta^3/2^{16}$
Sample a list Q of $\lceil 4\theta^{-1} \log(8N^2/\eta) \rceil$ points of V
Define bad by the displayed formula above
$M \leftarrow \lceil 16\delta^{-2} \log(4/\eta) \rceil$, $\zeta \leftarrow \eta/(4(M+N+1))$
for $j = 1, \dots, M$ **do**
    Choose $v \in V$; form the list $W \leftarrow U_0 \cap N_\Gamma(v)$
    If $|W| < \delta N/4$, continue
    If Low$(W \times W, \text{bad}, \delta/256, \zeta)$ is false, continue
    $U' \leftarrow \{u \in W : \text{Low}(W, \text{bad}(u, \cdot), \delta/32, \zeta)\}$
    if $|U'| < |W|/2$ **then**
        return $\perp$
    Compute $d(b) = \sum_{u \in U'} \Gamma_{u,b}$ for all $b \in V$, by scanning $U' \times V$
    $V' \leftarrow \{b : d(b) \ge \delta |U'|/4\}$
    return (U', V')
return $\perp$

The size and edge bounds are deterministic. Any returned U' passes the check $|U'| \ge |W|/2 \ge \delta N/8$. Every $u \in U' \subseteq U_0$ has degree at least $\delta N/2$; discarding right vertices outside V' loses at most $\delta N|U'|/4$ edges. Thus $e_\Gamma(U', V') \ge \delta N|U'|/4$. In particular, neither returned set contains an isolated padding vertex.

The sampled relation is reliable. Apply Lemma 126 with threshold 2θ to each of the N^2 pairs, using the shared list Q. A union bound shows that, with probability at least $1 - \eta/4$, every flagged pair has at most $4\theta N$ common neighbors, and every unflagged pair has at least θN. Condition on such a list Q.

A useful neighborhood occurs often enough. Deleting the low-degree left vertices removes at most $\delta N^2/2$ edges. For uniform $v \in V$, therefore, $\mathbb{E}|W| \ge \delta N/2$ and $\mathbb{E}|W|^2 \ge \delta^2 N^2/4$. Let $b(W)$ count flagged ordered pairs in W^2. Each flagged pair appears in W^2 with probability at most 4θ, so $\mathbb{E}b(W) \le 4\theta N^2$. Consequently the score $Z = |W|^2 - 512b(W)/\delta$ satisfies $\mathbb{E}Z \ge 7\delta^2 N^2/32$, while $Z \le N^2$. Since $\mathbb{E}Z \le \delta^2 N^2/8 + N^2 \Pr(Z \ge \delta^2 N^2/8)$, a neighborhood with $Z \ge \delta^2 N^2/8$ occurs with probability at least $\delta^2/16$. Such a neighborhood has $|W| \ge \delta N/4$ and $b(W)/|W|^2 \le \delta/512$, so its first Low test accepts whenever accurate. The probability that none of the M trials produces one is at most $\eta/4$.

Pruning makes the conclusion hold for every endpoint. There are at most $M + N$ further sampling tests. Conditional on their inputs and all previous randomness, each uses fresh samples and fails with probability at most ζ. Their total failure probability is at most $\eta/4$. Assume all are accurate. An accepted W has $b(W)/|W|^2 \le \delta/128$. At least half its vertices have at most $\delta|W|/64$ bad partners and hence survive. Every survivor has at most $\delta|W|/16$ bad partners. Thus $|U'| \ge |W|/2 \ge \delta N/8$.

For $u \in U'$ and $b \in V'$, there are at least $\delta|U'|/4 \ge \delta|W|/8$ neighbors $w \in U'$ of b. At least $\delta|W|/16$ of these are unflagged partners of u. Each supplies at least θN choices of z, giving $(\delta|W|/16)\theta N \ge \delta^5 N^2/2^{22}$ walks. The three failure allowances sum to at most $3\eta/4$.

Cost. One bad-pair query scans Q, costing $O(\delta^{-3}\ell)$. A threshold test on W^2 or on one vertex's partners uses $O(\delta^{-1}\ell)$ such queries. There are $O(\delta^{-2}\ell)$ neighborhood trials and at most N partner tests after one is accepted. Forming the trial neighborhoods costs $O(N\delta^{-2}\ell)$; the final exact degree scan costs $O(N|U'|)$. These bounds give the stated time. □

From walks to sums. Suppose $U = A$, $V = B$, and every edge of Γ has label $a + b \in S$, where $|S| \le N$. For each $t \in U' + V'$, fix one pair (a, b) with $a + b = t$. A walk a, z, w, b gives the three labels $(a + z, w + z, w + b) \in S^3$, whose alternating sum is t. For the fixed endpoints, these labels recover z and w. Thus distinct walks give distinct triples, and triples used for different t's are disjoint. The walk bound proves $|U' + V'|\delta^5 N^2/2^{22} \le |S|^3 \le N^3$, or $|U' + V'| \le C_0\delta^{-5}N$.

We do not need to verify the walk counts. Enumerating $U' \times V'$ and sorting its sums verifies the desired conclusion directly.

Peeling and exact verification. Build the graph $\Gamma_0 = \{(a, b) : a + b \in S\}$, using sorted S for membership tests. Store its adjacency matrix, all left degrees, and its edge count. During a pass, remove the edges covered by each returned product, and update these quantities as the removed entries are scanned. A restart begins again from Γ_0.

Algorithm 26 Near-quadratic BSG decomposition

Input: A, B, S, each of size at most N, and $0 < \varepsilon < 1$
Output: The decomposition in Theorem 125

if $C_0\varepsilon^{-5} \ge N$ **then**
  Enumerate $A \times B$; sort and deduplicate its sums to form $T = A + B$
  return the single triple (A, B, T) and $R = \varnothing$
Build Γ_0, padding the parts to size N with isolated vertices
repeat
  $\Gamma \leftarrow \Gamma_0$, $m \leftarrow$ its edge count, $\mathcal{L} \leftarrow [\,]$, *failed* $\leftarrow$ false
  while $m > \varepsilon N^2$ **do**
    $\delta \leftarrow m/N^2$
    $(X, Y) \leftarrow$ BsgPiece$(\Gamma, \delta, \varepsilon/1024)$
    if the call returns $\perp$ **then**
      failed $\leftarrow$ true; **break**
    Enumerate $X \times Y$; sort and deduplicate its sums to form $T = X + Y$
    if $|T| > C_0\delta^{-5}N$ **then**
      failed $\leftarrow$ true; **break**
    Append (X, Y, T) to $\mathcal{L}$
    Delete the edges in $X \times Y$, updating degrees and m
until *failed* = false
return $\mathcal{L}$ and the list R of remaining edges

Proof of Theorem 125. If $C_0\varepsilon^{-5} \ge N$, the one-product answer is valid because $|A + B| \le N^2 \le C_0\varepsilon^{-5}N$. Assume henceforth $C_0\varepsilon^{-5} < N$. This is also what makes the sampling costs affordable.

Correctness and number of products. Only products passing exact verification are kept. Every kept sumset has size at most $C_0\delta^{-5}N \le C_0\varepsilon^{-5}N$, and the remaining edges are exactly the required set R. Let m_i be the edge count before a successful iteration and put $\delta_i = m_i/N^2$. The deterministic graph bounds ensure that at least $\delta_i N|A_i|/4 \ge \delta_i^2 N^2/32$ edges are removed. Thus, while the next density is positive, $\delta_{i+1} \le \delta_i(1 - \delta_i/32)$, giving $1/\delta_{i+1} \ge 1/\delta_i + 1/32$. Before each iteration $1/\delta_i < 1/\varepsilon$, so there are at most $\lceil 32/\varepsilon \rceil$ kept products. An aborted pass has at most one additional, failed call.

Why exact scans do not accumulate a factor $1/\varepsilon$. The stronger edge-removal guarantee gives $m_{i+1} \le m_i(1 - |A_i|/(4N)) \le m_i e^{-|A_i|/(4N)}$. Multiplying these inequalities over all but the final kept product, then using $|A_i| \le N$ for that final product, shows $\sum_i |A_i| \le N + 4N\log(1/\varepsilon)$. Indeed, before that final iteration the graph still has more than εN^2 edges. This argument also bounds every verified prefix of an aborted pass; its final failed candidate adds at most one N-vertex scan.

Consequently all exact degree scans, product enumerations, and edge deletions in one pass inspect $O(N^2(1+\log(1/\varepsilon)))$ pairs in total. Sorting the resulting sum lists adds a factor $O(\log(2N))$. This also accounts for explicitly producing every T_i.

Sampling time. There are $O(1/\varepsilon)$ calls, each with $\delta \geq \varepsilon$ and $\eta = \varepsilon/1024$. Apart from the exact scans already charged, their total time is

$$O\big(N\varepsilon^{-5}\ell^2 + \varepsilon^{-7}\ell^3\big), \qquad \ell = \log(2N/\varepsilon).$$

The nontrivial-case assumption implies $\varepsilon^{-5} < N$, hence $N\varepsilon^{-5} < N^2$, $\varepsilon^{-7} < N^{7/5}$, and $\ell = O(\log(2N))$. The smaller term $N^{7/5}\log^3(2N)$ is $O(N^2\log^2(2N))$, since $\log(2N) = O(N^{3/5})$. Thus every pass costs $O(N^2\log^2(2N))$, including initialization and a possible final failed candidate.

Restarts. Conditional on any preceding history, a fresh call has failure probability at most $\varepsilon/1024$, by Lemma 127 and the label-counting argument. There are fewer than $34/\varepsilon$ calls in a pass, so a union bound makes its failure probability less than $1/16$. Independent passes therefore finish after fewer than two attempts in expectation. Exact verification guarantees correctness even when some sampling tests err, proving the Las Vegas claim. □

The useful distinction from a purely existential BSG statement is that the full lists T_i are produced within the same bound.

5.4 Extremal Set Theory

5.4.1 Bollobás Two Families Theorem

Theorem 128 (Bollobás Two Families Theorem)**.** *Let*

$$(A_1, B_1), \ldots, (A_m, B_m)$$

be pairs of finite sets such that

$$A_i \cap B_i = \emptyset$$

for every i, while

$$A_i \cap B_j \neq \emptyset$$

whenever

$$i \neq j.$$

Then

$$\boxed{\sum_{i=1}^{m} \binom{|A_i| + |B_i|}{|A_i|}^{-1} \leq 1.}$$

Proof. Choose a uniformly random ordering of all elements appearing in the sets.

For each i, let E_i be the event that every element of A_i appears before every element of B_i.

Among the

$$|A_i| + |B_i|$$

elements of $A_i \cup B_i$, every relative order is equally likely. The event E_i occurs exactly when the first $|A_i|$ positions are occupied by A_i, so

$$\Pr[E_i] = \binom{|A_i| + |B_i|}{|A_i|}^{-1}.$$

We claim that the events E_i are pairwise disjoint. Indeed, suppose both E_i and E_j occurred for

$$i \neq j.$$

Choose

$$x \in A_i \cap B_j, \qquad y \in A_j \cap B_i.$$

Since E_i occurs,

$$x \text{ appears before } y.$$

Since E_j occurs,

$$y \text{ appears before } x,$$

a contradiction.

Therefore

$$\sum_i \Pr[E_i] \le 1,$$

which is exactly the desired inequality. □

Corollary 129. *If*

$$|A_i| = a, \qquad |B_i| = b$$

for every i, then

$$\boxed{m \le \binom{a+b}{a}.}$$

5.4.2 Exterior Algebra and the Skew Two Families Theorem

The uniform bound has an algebraic explanation: represent each first member by a vector in a space of dimension $\binom{a+b}{a}$, and use the second members to certify linear independence. Exterior products turn intersections into zeros. The proof needs intersections in only one direction of the index order, giving the *skew* version of the theorem.

Exterior products as intersection tests. If V has basis $e_1, \dots, e_d$, its ath exterior power $\bigwedge^a V$ has basis $e_I = e_{i_1} \wedge \cdots \wedge e_{i_a}$ indexed by $I = \{i_1 < \cdots < i_a\} \subseteq [d]$. Thus

$$\dim \bigwedge^a V = \binom{d}{a}.$$

The product is bilinear: $e_I \wedge e_J = 0$ when $I \cap J \neq \emptyset$, and otherwise it is $e_{I \cup J}$ multiplied by the sign of the permutation sorting the concatenated indices. In particular, the product is alternating on vectors, including in characteristic 2. Multilinear expansion gives

$$v_1 \wedge \cdots \wedge v_a = \sum_{I \in \binom{[d]}{a}} \det([v_1 \ \cdots \ v_a]_I) e_I.$$

Here the subscript selects the rows in I. The product is nonzero exactly when the vectors are linearly independent, because its coordinates are precisely their maximal minors.

If $U, W \subseteq V$ have dimensions a, b, choose bases and write $\alpha = u_1 \wedge \cdots \wedge u_a$ and $\beta = w_1 \wedge \cdots \wedge w_b$. Their concatenated bases are independent exactly when $U \cap W = \{0\}$, so

$$\boxed{\alpha \wedge \beta \neq 0 \quad \iff \quad U \cap W = \{0\}.}$$

Theorem 130 (Exterior two families bound)**.** *Let $\dim V = a + b$, and suppose $\alpha_1, \dots, \alpha_m \in \bigwedge^a V$ and $\beta_1, \dots, \beta_m \in \bigwedge^b V$ satisfy*

$$\alpha_i \wedge \beta_i \neq 0, \qquad \alpha_i \wedge \beta_j = 0 \quad (i < j).$$

Then the α_i are linearly independent, and hence

$$\boxed{m \le \binom{a+b}{a}.}$$

The exterior elements need not themselves be products of vectors.

Proof. In a nontrivial dependence $\sum_i c_i\alpha_i = 0$, choose the largest j with $c_j \neq 0$ and wedge with β_j on the right. Every term with $i < j$ vanishes by the hypothesis, while the later coefficients are zero. This leaves $c_j\alpha_j \wedge \beta_j = 0$, a contradiction. The dimension of $\bigwedge^a V$ gives the bound. □

Since $\bigwedge^{a+b} V$ is one-dimensional, taking the coefficient of $e_1 \wedge \cdots \wedge e_{a+b}$ in $\alpha \wedge \beta_j$ defines a scalar linear functional of α. Thus this is exactly the triangular-independence principle, with the index order reversed.

Theorem 131 (Skew subspace two families)**.** *Let $(U_1, W_1), \ldots, (U_m, W_m)$ be pairs of subspaces of a finite-dimensional vector space over any field, with $\dim U_i = a$ and $\dim W_i = b$. Suppose*

$$U_i \cap W_i = \{0\}, \qquad U_i \cap W_j \neq \{0\} \quad (i < j).$$

Then $m \le \binom{a+b}{a}$.

Proof. Put $d = a + b$. If the ambient dimension is d, wedge the bases of the subspaces and apply Theorem 130. We reduce a larger ambient space to this case without destroying any required intersection or disjointness.

Choose coordinates in the original ambient space $\mathbb{F}^D$. Let T be a $d \times D$ matrix of independent indeterminates, and work over the rational-function field $K = \mathbb{F}(T)$. For each i, concatenate bases of U_i, W_i into a $D \times d$ matrix C_i. Its columns are independent, so there exists a linear map sending them to the standard basis of $\mathbb{F}^d$. Consequently $\det(TC_i)$ is a nonzero polynomial: it takes the value 1 at that map. Over K, all these determinants are nonzero.

Thus T is injective on each diagonal sum $U_i + W_i$ after extending scalars to K. The image subspaces have dimensions a, b and trivial diagonal intersection. A nonzero vector in $U_i \cap W_j$ remains nonzero under T, since it belongs to $U_i \subseteq U_i + W_i$. Hence the required cross-intersections also persist. The image configuration lies in K^d, where the first part applies. The case $d = 0$ is immediate: the cross-intersection condition permits at most one pair. □

The rational-function extension is only a device for choosing one projection that works for every pair.

Corollary 132 (Skew set-pair bound)**.** *Let $(A_1, B_1), \ldots, (A_m, B_m)$ be pairs of finite sets with $|A_i| = a$, $|B_i| = b$, $A_i \cap B_i = \emptyset$, and $A_i \cap B_j \neq \emptyset$ whenever $i < j$. Then*

$$\boxed{m \le \binom{a+b}{a}.}$$

Proof. Give each ground element u a coordinate basis vector e_u. The coordinate subspaces $U_i = \operatorname{span}\{e_u : u \in A_i\}$ and $W_i = \operatorname{span}\{e_u : u \in B_i\}$ have the required intersection pattern. Apply Theorem 131. □

For sets with $a+b > 0$, there is also an explicit way to start in dimension $a+b$: assign distinct real numbers t_u to the ground elements and use $v_u = (1, t_u, \ldots, t_u^{a+b-1})$. Every $a + b$ distinct vectors are independent by the Vandermonde determinant. Wedge the vectors from A_i and B_i; diagonal products are nonzero, and a shared ground element makes every required cross-product zero.

The bound is sharp: take all pairs $(A, [a + b] \setminus A)$ with $|A| = a$. Distinct a-subsets intersect one another's complements. This exterior argument proves the uniform bound under a weaker, one-sided intersection hypothesis.

5.4.3 Sunflower Lemma

A family $S_1, \dots, S_r$ is an *r-sunflower* if there is a set C, called the *core*, such that

$$S_i \cap S_j = C \qquad (i \neq j).$$

Equivalently, after deleting the common core, the petals $S_1 \setminus C, \dots, S_r \setminus C$ are pairwise disjoint.

Let $\mathrm{Sun}(k, r)$ be the least N such that every family of N distinct sets of size at most k contains an r-sunflower. The classical Erdős–Rado lemma gives

$$\mathrm{Sun}(k, r) \leq (r-1)^k k! + 1.$$

A better bound replaces the factorial by only a logarithmic factor in the base:

$$\boxed{\mathrm{Sun}(k, r) \leq (Cr \log(2k))^k}$$

for a universal constant C.

The proof passes through a stronger probabilistic notion.

Definition 133 (Robust Sunflower)**.** Let $\mathcal{R}$ be a family of at least two finite sets, with common intersection $C = \bigcap_{S \in \mathcal{R}} S$. Let W_γ contain each element of the universe independently with probability γ.

We call $\mathcal{R}$ a (γ, ε)-*robust sunflower* if

$$\boxed{\Pr\left[\exists S \in \mathcal{R} : S \subseteq C \cup W_\gamma\right] \geq 1 - \varepsilon.}$$

After removing the common core, this says that a random set of density γ contains an entire petal with probability at least $1 - \varepsilon$.

The useful point is that a robust sunflower gives many alternative ways for a random set to succeed. No particular petal needs to be likely to appear; the family as a whole is rich enough that a sparse random sample almost always contains *some* petal. This will eventually let us put different petals into disjoint random parts and obtain an ordinary sunflower.

Theorem 134 (Robust Sunflower Lemma)**.** *There is a universal constant C_0 such that, for $k \geq 1$, $0 < \gamma \leq 1$, and $0 < \varepsilon \leq 1/2$, every family of at least*

$$\boxed{\left(\frac{C_0 \log(2k/\varepsilon)}{\gamma}\right)^k}$$

distinct sets of size at most k contains a (γ, ε)-robust sunflower.

The proof first removes a core so that no nonempty set occurs too frequently. A random sample then repeatedly halves the number of elements still needed to complete a member of the remaining family.

Spread families and covering. A nonempty family $\mathcal{F}$ is *R-spread* if

$$\#\{S \in \mathcal{F} : Z \subseteq S\} \leq |\mathcal{F}| R^{-|Z|} \qquad \text{for every nonempty } Z. \tag{12}$$

Say that $S \in \mathcal{F}$ is *m-covered by* T if some $Q \in \mathcal{F}$ satisfies

$$Q \subseteq T \cup S, \qquad |Q \setminus T| \leq m.$$

Thus S supplies at most m missing elements of a possible completion. Initially every set is k-covered by $\emptyset$. If any set is 0-covered by T, then T contains a member of $\mathcal{F}$.

Enlarging T preserves every covering relation. All witnesses are taken from the fixed family $\mathcal{F}$, even when we later track only a subfamily of sets.

Tracking the missing elements of each original set is too restrictive: a sparse random subset is unlikely to supply half of them. One alternative is considering the global closest member to being completed, but this forgets the information of where the requirements are, and how many subsets are providing the bound. Instead, let every S have a target member, which can change, and ask how few elements from an original set S together with the exposed set T would suffice to complete some member. Keeping the remaining requirements inside S lets us use spreadness: any large requirement is supported by only a small fraction of the original sets.

Lemma 135 (Cover Halving)**.** *Let $\mathcal{F}$ be R-spread, put $M = |\mathcal{F}|$, and fix an exposed set T and an integer $m \geq 1$. Include each unused element in B independently with probability $p \in (0, 1)$.*

If $pR \geq 256$, then with probability at least $1/2$, all but at most $M/4^m$ of the m-covered sets become $\lfloor m/2 \rfloor$-covered by $T \cup B$.

Proof. Put $d = \lfloor m/2 \rfloor$. Collect all short completions before exposing B:

$$\mathcal{H} = \{Q \setminus T : Q \in \mathcal{F},\ |Q \setminus T| \leq m\}.$$

A set is m-covered by T exactly when it contains a member of $\mathcal{H}$.

Let $\mathcal{M}(B)$ be the inclusion-minimal members of $\{H \setminus B : H \in \mathcal{H}\}$. These are the minimal remainders of the short completions.

Call an m-covered set S *bad* if it is not d-covered by $T \cup B$. Choose $H \in \mathcal{H}$ contained in S, and then choose $Z \in \mathcal{M}(B)$ contained in $H \setminus B$. Necessarily $|Z| > d$: otherwise the member $Q \in \mathcal{F}$ corresponding to this remainder would satisfy

$$Q \subseteq T \cup B \cup Z \subseteq T \cup B \cup S, \qquad |Q \setminus (T \cup B)| = |Z| \leq d,$$

contradicting badness.

Thus every bad set contains a minimal remainder of size greater than d. If $X_b(B)$ counts members of $\mathcal{M}(B)$ of size b and $D(B)$ counts bad sets, spreadness gives

$$D(B) \leq M \sum_{b=d+1}^{m} R^{-b} X_b(B). \tag{13}$$

We now bound X_b by encoding a pair (B, Z) through its union $Y = B \sqcup Z$. For each possible Y, fix any $H_Y \in \mathcal{H}$ contained in Y. Such a member exists whenever Y arises from a counted pair. Then $H_Y \setminus B$ is another remainder contained in Z, so minimality forces

$$H_Y \setminus B = Z.$$

In particular, $Z \subseteq H_Y$. Since $|H_Y| \leq m$, there are at most $\binom{m}{b}$ possible b-element sets Z for this fixed Y. Each determines $B = Y \setminus Z$ uniquely.

To convert this count into an expectation, write $V = U \setminus T$ and let $\pi(A) = p^{|A|}(1-p)^{|V|-|A|}$ be the probability that $B = A$. For a counted pair with $|Z| = b$,

$$\pi(B) = \left(\frac{1-p}{p}\right)^b \pi(Y).$$

Summing first over pairs with the same union, and then over all $Y \subseteq V$, gives

$$\begin{aligned} \mathbb{E}[X_b(B)] &= \sum_{(B,Z)} \pi(B) \\ &\leq \binom{m}{b} \left(\frac{1-p}{p}\right)^b \sum_{Y \subseteq V} \pi(Y) \\ &= \binom{m}{b} \left(\frac{1-p}{p}\right)^b. \end{aligned} \tag{14}$$

Substitute this into (13). Because every index in the sum satisfies $b > m/2$,

$$\begin{aligned}\mathbb{E}[D(B)] &\le M \sum_{b>m/2}^{m} \binom{m}{b} \left(\frac{1-p}{pR}\right)^b \\ &\le M(pR)^{-m/2} \sum_{b=0}^{m} \binom{m}{b} \\ &= M\left(\frac{2}{\sqrt{pR}}\right)^m \le \frac{M}{8^m}.\end{aligned}$$

Finally, Markov's inequality gives

$$\Pr\left[D(B) > \frac{M}{4^m}\right] \le \frac{M/8^m}{M/4^m} = 2^{-m} \le \frac{1}{2}.$$

□

Proof of Theorem 134. Set

$$L = \log\frac{2k}{\varepsilon}, \qquad \ell = \lceil 4L \rceil, \qquad p = \frac{\gamma}{\ell}, \qquad R = \frac{256}{p}.$$

Since $\ell = O(L)$, choosing C_0 sufficiently large ensures that the given family $\mathcal{F}$ has at least $(2R)^k$ members.

Extracting a spread family. For $Z \subseteq U$, write $\mathcal{F}_Z = \{S \in \mathcal{F} : Z \subseteq S\}$. Choose C maximizing

$$R^{|C|}|\mathcal{F}_C|.$$

The choice $C = \emptyset$ shows that the maximum is at least $|\mathcal{F}|$. In particular, $\mathcal{F}_C$ is nonempty and $|C| \le k$.

Delete C from these sets and put

$$\mathcal{A} = \{S \setminus C : S \in \mathcal{F}_C\}, \qquad M = |\mathcal{A}| = |\mathcal{F}_C|.$$

For every nonempty $Z \subseteq U \setminus C$, maximality gives

$$\#\{A \in \mathcal{A} : Z \subseteq A\} = |\mathcal{F}_{C\cup Z}| \le R^{-|Z|}|\mathcal{F}_C| = MR^{-|Z|}.$$

Thus $\mathcal{A}$ is R-spread. Moreover,

$$M \ge \frac{|\mathcal{F}|}{R^{|C|}} \ge \frac{(2R)^k}{R^k} = 2^k \ge 2.$$

Since $R > 1$, spreadness applied to singletons implies $\bigcap_{A\in\mathcal{A}} A = \emptyset$. Consequently the original subfamily $\mathcal{F}_C$ has common core C. It remains to show that W_γ contains some member of $\mathcal{A}$ with probability at least $1 - \varepsilon$.

Repeated halving. Choose independent random sets $B_1, \dots, B_\ell$, each including every element of $U \setminus C$ independently with probability p. Let $T_i = B_1 \cup \cdots \cup B_i$.

Initially track all M members of $\mathcal{A}$, with covering parameter $m_0 = k$. At a round with current parameter $m > 0$, call the round successful if all but at most $M/4^m$ tracked sets become $\lfloor m/2 \rfloor$-covered. On success, discard those exceptions and replace m by $\lfloor m/2 \rfloor$. On failure, keep the tracked sets and the parameter unchanged. In either case, expose the new block. Once the parameter reaches zero, keep it there.

Conditioned on the past, the new elements $B_{i+1} \setminus T_i$ form an independent density-p sample of the unused universe. Since $pR = 256$, the cover-halving lemma makes each round successful with conditional probability at least $1/2$. Failed rounds cannot destroy existing covering relations.

Some tracked sets survive throughout the process. Indeed, the positive values of m at successful rounds are distinct, so the total number discarded is at most

$$M \sum_{j \geq 1} 4^{-j} = \frac{M}{3}.$$

Thus at least $2M/3 > 0$ sets remain at every stage. If $m_\ell = 0$, a surviving set has a witness contained in T_ℓ, so T_ℓ contains a member of $\mathcal{A}$.

At every round,

$$\mathbb{E}[m_{i+1} \mid \text{past}] \leq \frac{1}{2} \left\lfloor \frac{m_i}{2} \right\rfloor + \frac{1}{2} m_i \leq \frac{3}{4} m_i.$$

Iteration gives $\mathbb{E}[m_\ell] \leq k(3/4)^\ell$. Since m_ℓ is a nonnegative integer, Markov's inequality yields

$$\Pr[m_\ell > 0] \leq \mathbb{E}[m_\ell] \leq k(3/4)^\ell \leq k e^{-\ell/4} \leq \varepsilon.$$

Hence T_ℓ contains a member of $\mathcal{A}$ with probability at least $1 - \varepsilon$.

Finally, T_ℓ includes each element independently with probability

$$q = 1 - (1-p)^\ell \leq \ell p = \gamma.$$

It can therefore be coupled as a subset of W_γ. Containing a member of $\mathcal{A}$ is preserved when elements are added, so

$$\Pr[\exists A \in \mathcal{A} : A \subseteq W_\gamma] \geq 1 - \varepsilon.$$

Restoring the common core C proves that $\mathcal{F}_C$ is a (γ, ε)-robust sunflower. □

Corollary 136 (Sunflower Lemma)**.** *There is a universal constant C such that*

$$\boxed{\mathrm{Sun}(k, r) \leq (Cr \log(2k))^k.}$$

Proof. By increasing C, it is enough to consider a uniform family. Indeed, among sets of size at most k, one of the $k+1$ possible sizes occurs in at least a $1/(k+1)$ fraction of the family, and this polynomial loss is absorbed by increasing the constant in the base.

Thus let $\mathcal{F}$ be a sufficiently large q-uniform family for some $q \leq k$. Apply Theorem 134 with

$$\gamma = \frac{1}{2r}, \qquad \varepsilon = \frac{1}{2}.$$

We obtain a robust sunflower $\mathcal{R} \subseteq \mathcal{F}$ with core $C = \bigcap_{S \in \mathcal{R}} S$.

Since the members of $\mathcal{R}$ are distinct and all have the same size, no member equals C. Thus every petal $S \setminus C$ is nonempty.

Now assign every element outside C, independently and uniformly, to one of $2r$ bins $B_1, \ldots, B_{2r}$. For each fixed j, the marginal distribution of B_j is exactly $W_{1/(2r)}$. Robustness therefore gives

$$\Pr[\exists S \in \mathcal{R} : S \setminus C \subseteq B_j] \geq \frac{1}{2}.$$

By linearity of expectation, the expected number of bins containing a complete petal is at least $2r \cdot \frac{1}{2} = r$. Hence some partition has at least r such bins.

Choose one petal from each of r distinct good bins. Since the bins are disjoint, these petals are pairwise disjoint. Adding back the common core C gives r members of $\mathcal{R}$ forming an r-sunflower. □

Lower Bound

Proposition 137. *For every $k, r \geq 2$,*

$$\boxed{\mathrm{Sun}(k, r) > (r-1)^k.}$$

Proof. Partition the universe into k disjoint blocks $X_1, \ldots, X_k$, each of size $r-1$, and let $\mathcal{F}$ contain every transversal

$$S = \{x_1, \ldots, x_k\}, \qquad x_i \in X_i.$$

Then $|\mathcal{F}| = (r-1)^k$.

Suppose r distinct members of $\mathcal{F}$ formed a sunflower. Fix a block X_i. The r sets choose r elements from only $r-1$ possibilities, so two of them choose the same element $x \in X_i$.

In a sunflower, any element lying in two members belongs to the core and therefore to every member. Thus all r sets choose the same element from X_i.

The same argument applies independently to every block X_i. Hence the r sets make the same choice in every block and are therefore identical, contradicting their distinctness. Thus $\mathcal{F}$ contains no r-sunflower. □

5.5 Ramsey Theory

5.5.1 Hales–Jewett Theorem

Let

$$[m] = \{0, 1, \ldots, m-1\}.$$

A *combinatorial line* in $[m]^N$ is obtained from a word $\tau(*) \in ([m] \cup \{*\})^N$ containing at least one wildcard. For $a \in [m]$, let $\tau(a)$ be the word obtained by replacing every occurrence of $*$ by a. The corresponding line is

$$\{\tau(0), \ldots, \tau(m-1)\}.$$

Let $HJ(c, m)$ denote the least N such that every c-coloring of $[m]^N$ contains a monochromatic combinatorial line.

For the quantitative bound, define the hierarchy

$$h_1(n) = 2n, \qquad h_{r+1}(n) = h_r^n(1),$$

where h_r^n denotes n-fold composition. Thus $h_2(n) = 2^n$, $h_3(n)$ is a tower of 2's of height n, and h_4 iterates the tower function.

Theorem 138 (Hales–Jewett)**.** *For every $c, m \geq 1$,*

$$\boxed{HJ(c, m) \leq \frac{1}{cm} h_4(c + m + 2).}$$

In particular, $HJ(c, m)$ is finite and has a primitive-recursive upper bound.

We induct on the alphabet size and force the symbols 0 and 1 to become indistinguishable. We may then merge them into a single symbol and invoke Hales–Jewett for an alphabet of size $m-1$.

Proof of the reduction. We induct on m. The case $m = 1$ is trivial.

Assume the theorem for an alphabet of size $m-1$, and set

$$\ell = HJ(c, m-1).$$

We shall construct ℓ blocks

$$\tau_1(*), \ldots, \tau_\ell(*)$$

with the following property:

$$\chi\big(\tau_1(\nu_1)\cdots\tau_\ell(\nu_\ell)\big) \tag{15}$$

is unchanged if any collection of coordinates ν_i is switched between 0 and 1.

Once this is achieved, 0 and 1 can be regarded as the same letter, leaving an alphabet of size $m-1$. The induction hypothesis will then finish the proof.

Constructing the Blocks We choose the block lengths large enough for a repeated pigeonhole argument. Set $L_0 = 0$, and for $i = 1, \ldots, \ell$ define

$$n_i = c^{\,m^{L_{i-1}+\ell-i}}, \qquad L_i = L_{i-1} + n_i.$$

We claim that

$$HJ(c, m) \le L_\ell.$$

Let

$$\chi : [m]^{L_\ell} \to [c]$$

be arbitrary, and partition the coordinates into consecutive blocks of lengths $n_1, \ldots, n_\ell$.

We construct the block roots backwards, starting with $\tau_\ell(*)$ and ending with $\tau_1(*)$. Suppose $\tau_{i+1}, \ldots, \tau_\ell$ have already been chosen.

For $0 \le t \le n_i$, let

$$W_t = 0^t 1^{\,n_i - t}.$$

Think of W_t as placing a boundary between a string of 0's and a string of 1's. We shall find two positions of this boundary that are indistinguishable to the coloring.

For each t, consider the function

$$\Phi_t : [m]^{L_{i-1}} \times [m]^{\ell-i} \longrightarrow [c]$$

defined by

$$\Phi_t(z, \nu_{i+1}, \ldots, \nu_\ell) = \chi\big(z\, W_t\, \tau_{i+1}(\nu_{i+1})\cdots\tau_\ell(\nu_\ell)\big).$$

The arguments of Φ_t describe everything outside the i-th block that can still affect its color: z is an arbitrary assignment to all earlier coordinates, while $\nu_{i+1}, \ldots, \nu_\ell$ specify the already constructed later blocks.

There are

$$m^{L_{i-1}+\ell-i}$$

such contexts. A function assigning one of c colors to every context can therefore take at most

$$c^{\,m^{L_{i-1}+\ell-i}} = n_i$$

different values.

But we have $n_i + 1$ words

$$W_0, W_1, \ldots, W_{n_i}.$$

By the pigeonhole principle, two of them, say W_t and W_u with $t < u$, induce exactly the same function:

$$\Phi_t = \Phi_u.$$

Define

$$\tau_i(*) = 0^t *^{u-t} 1^{n_i-u}.$$

Then

$$\tau_i(0) = W_u, \qquad \tau_i(1) = W_t.$$

Thus changing the parameter of the i-th block from 0 to 1, or back again, does not change the color, no matter how the earlier coordinates or later block parameters are chosen.

Proceeding backwards through all ℓ blocks proves Equation (15).

Merging 0 and 1 For $\nu = (\nu_1, \ldots, \nu_\ell) \in [m]^\ell$, write

$$\tau(\nu) = \tau_1(\nu_1) \cdots \tau_\ell(\nu_\ell).$$

By construction, $\chi(\tau(\nu))$ is unchanged whenever any coordinates of ν are switched between 0 and 1. We may therefore identify these two symbols. Let

$$\overline{[m]} = \{\star, 2, \ldots, m-1\},$$

where $\star$ represents either 0 or 1. Define

$$\overline{\chi}(\bar{\nu}) = \chi(\tau(\nu)),$$

where each occurrence of $\star$ in $\bar{\nu}$ may be represented by either 0 or 1 in ν.

The invariance just proved makes $\overline{\chi}$ well defined. It is a c-coloring of

$$\overline{[m]}^\ell,$$

whose alphabet has size $m-1$.

Since $\ell = HJ(c, m-1)$, this coloring contains a monochromatic combinatorial line. Let

$$\sigma(*) \in (\overline{[m]} \cup \{*\})^\ell$$

be a root for such a line.

Lifting the Line We now turn $\sigma(*)$ into a root in the original space $[m]^{L_\ell}$, block by block.

For coordinate i,

- if $\sigma_i = *$, use the entire root $\tau_i(*)$;
- if $\sigma_i = j \in \{2, \ldots, m-1\}$, use the fixed block $\tau_i(j)$;
- if $\sigma_i = \star$, use the fixed block $\tau_i(0)$.

Call the resulting root $\rho(*)$.

For $a = 2, \ldots, m-1$, the word $\rho(a)$ maps exactly to the a-point of the monochromatic line for $\overline{\chi}$. The word $\rho(0)$ maps to its $\star$-point.

The word $\rho(1)$ differs from $\rho(0)$ only by changing some block parameters from 0 to 1. By Equation (15), this does not change its color. Therefore

$$\chi(\rho(0)) = \chi(\rho(1)) = \cdots = \chi(\rho(m-1)).$$

Thus $\rho(*)$ gives a monochromatic combinatorial line in $[m]^{L_\ell}$. □

Quantitative Bound The preceding argument gives an explicit recursive bound: if $B(c,1)=1$, and for $m \geq 2$ we set $\ell = B(c, m-1)$ and recursively

$$L_0 = 0, \qquad L_i = L_{i-1} + c^{m^{L_{i-1}+\ell-i}}, \qquad B(c,m) = L_\ell,$$

then

$$HJ(c,m) \leq B(c,m).$$

5.6 Polynomial Method

5.6.1 Triangular Independence

To bound a family of objects, represent its members by functions in a small vector space. If each function has an evaluation that detects it but annihilates every later function, the functions are linearly independent. The bound is then the dimension of the space.

Lemma 139 (Triangular independence)**.** *Let $v_1, \dots, v_m$ belong to a vector space V over a field $\mathbb{F}$. Suppose there are linear functionals $\lambda_1, \dots, \lambda_m \in V^*$ such that*

$$\lambda_i(v_i) \neq 0, \qquad \lambda_i(v_j) = 0 \quad \text{whenever } i < j.$$

Then $v_1, \dots, v_m$ are linearly independent. In particular, $m \leq \dim V$ when V is finite-dimensional.

Proof. Suppose $\sum_j c_j v_j = 0$ is a nontrivial dependence, and choose the smallest i with $c_i \neq 0$. Applying λ_i kills all terms with $j > i$, while all coefficients with $j < i$ are zero. Thus $0 = c_i \lambda_i(v_i)$, a contradiction. □

Equivalently, the matrix $M_{ij} = \lambda_i(v_j)$ is lower triangular with nonzero diagonal. Reversing the order of the indices gives the same conclusion when the vanishing condition is $i > j$ instead. We thus do not need every off-diagonal entry to vanish.

5.6.2 Combinatorial Nullstellensatz

For

$$\alpha = (\alpha_1, \dots, \alpha_n) \in \mathbb{N}^n,$$

write

$$x^\alpha = x_1^{\alpha_1} \cdots x_n^{\alpha_n}.$$

We say that x^α is *maximal in the support* of f if

$$[x^\alpha] f \neq 0$$

and there is no different monomial x^β in the support of f with

$$\beta_i \geq \alpha_i \qquad \text{for every } i.$$

We give an upgraded version of the Combinatorial Nullstellensatz is an exact formula for a coefficient.

Lemma 140 (Univariate Coefficient Formula)**.** *Let*

$$A \subseteq \mathbb{F}, \qquad |A| = d+1,$$

and define

$$\phi_A(t) = \prod_{a \in A} (t - a).$$

Then for every polynomial g of degree at most d,

$$\boxed{[t^d]g(t) = \sum_{a\in A} \frac{g(a)}{\phi'_A(a)}.}$$

In particular,

$$\sum_{a\in A} \frac{a^j}{\phi'_A(a)} = \begin{cases} 0, & j < d, \\ 1, & j = d. \end{cases}$$

Proof. Lagrange interpolation gives

$$g(t) = \sum_{a\in A} g(a) \frac{\phi_A(t)}{(t-a)\phi'_A(a)}.$$

Each polynomial

$$\frac{\phi_A(t)}{t-a}$$

is monic of degree d. Taking the coefficient of t^d gives the result. □

Theorem 141 (Coefficient Formula)**.** *Let*

$$f \in \mathbb{F}[x_1, \ldots, x_n],$$

and fix

$$d = (d_1, \ldots, d_n).$$

For each i, choose

$$A_i \subseteq \mathbb{F}, \qquad |A_i| = d_i + 1,$$

and define

$$\phi_i(t) = \prod_{a\in A_i} (t-a).$$

Suppose either

1. *$x^d = x_1^{d_1} \cdots x_n^{d_n}$ is maximal in the support of f, or*
2. $$\deg f \le d_1 + \cdots + d_n.$$

Then

$$\boxed{[x^d]f = \sum_{a\in A_1\times\cdots\times A_n} \frac{f(a)}{\prod_{i=1}^n \phi'_i(a_i)}.}$$

In the second case the coefficient on the left is allowed to be zero.

Proof. By linearity, evaluate the right-hand side on a monomial

$$x^\alpha.$$

The sum factors:

$$\sum_a \frac{a^\alpha}{\prod_i \phi'_i(a_i)} = \prod_{i=1}^n \left(\sum_{a_i\in A_i} \frac{a_i^{\alpha_i}}{\phi'_i(a_i)} \right).$$

For $\alpha = d$, every factor equals 1 by Lemma 140.

Now consider any other monomial x^α appearing in f. Under the first hypothesis, maximality of x^d implies that

$$\alpha_i < d_i$$

for some i.

Under the second hypothesis, if $\alpha_i \geq d_i$ for every i, then

$$|\alpha| \geq \sum_i d_i.$$

Since $|\alpha| \leq \deg f \leq \sum_i d_i$, equality would hold everywhere and $\alpha = d$. Thus again, for every $\alpha \neq d$, some coordinate satisfies

$$\alpha_i < d_i.$$

The corresponding factor is 0 by Lemma 140. Hence every monomial except x^d is annihilated, while x^d contributes exactly its coefficient. □

Theorem 142 (Combinatorial Nullstellensatz)**.** *Suppose*

$$x_1^{d_1} \cdots x_n^{d_n}$$

is a maximal monomial in the support of

$$f \in \mathbb{F}[x_1, \ldots, x_n].$$

If

$$|A_i| > d_i$$

for every i, then there exists

$$a \in A_1 \times \cdots \times A_n$$

such that

$$f(a) \neq 0.$$

Proof. Restrict each A_i to any $d_i + 1$ elements. By Theorem 141,

$$0 \neq [x^d]f = \sum_a \frac{f(a)}{\prod_i \phi_i'(a_i)}.$$

Therefore not every evaluation $f(a)$ can be zero. □

The usual formulation, which assumes

$$\deg f = d_1 + \cdots + d_n \qquad \text{and} \qquad [x^d]f \neq 0,$$

is an immediate special case: the total-degree condition automatically makes x^d maximal.

Alon–Füredi Combinatorial Nullstellensatz guarantees one nonzero grid point. Alon–Füredi gives the sharp minimum number of them.

For positive integers $a_1, \ldots, a_n$ define

$$\mu(a_1, \ldots, a_n; d) = \min_{\substack{0 \leq d_i \leq a_i - 1 \\ \sum_i d_i \leq d}} \prod_{i=1}^{n} (a_i - d_i).$$

Theorem 143 (Alon–Füredi)**.** *Let*

$$A = A_1 \times \cdots \times A_n, \qquad |A_i| = a_i,$$

and suppose

$$f \in \mathbb{F}[x_1, \ldots, x_n]$$

is not identically zero on A. If

$$\deg f \leq d,$$

then

$$\boxed{|\{x \in A : f(x) \neq 0\}| \geq \mu(a_1, \ldots, a_n; d).}$$

Proof. Reduce f modulo

$$\prod_{a \in A_i} (x_i - a)$$

for every variable x_i. This preserves all values on A, does not increase total degree, and allows us to assume

$$\deg_{x_i} f < a_i.$$

We induct on n.

For $n = 1$, a nonzero polynomial of degree at most d has at most d roots, so it is nonzero on at least

$$a_1 - \min(d, a_1 - 1) = \mu(a_1; d)$$

points.

Now suppose $n > 1$. Let

$$S = \{c \in A_n : f(x_1, \ldots, x_{n-1}, c) \equiv 0\}, \qquad s = |S|.$$

Since f is nonzero somewhere on A,

$$s < a_n.$$

Every $x_n - c$ for $c \in S$ divides f, so

$$f(x) = \left(\prod_{c \in S} (x_n - c) \right) g(x)$$

with

$$\deg g \leq d - s.$$

For every

$$c \in A_n \setminus S,$$

the slice

$$g_c(x_1, \ldots, x_{n-1}) = g(x_1, \ldots, x_{n-1}, c)$$

is nonzero. By induction, it is nonzero on at least

$$\mu(a_1, \ldots, a_{n-1}; d - s)$$

points.

Therefore

$$|\{f \neq 0\}| \geq (a_n - s)\mu(a_1, \ldots, a_{n-1}; d - s).$$

If

$$d_1, \ldots, d_{n-1}$$

achieve the latter minimum, set

$$d_n = s.$$

Then

$$0 \le d_n \le a_n - 1, \qquad \sum_i d_i \le d,$$

so this choice is feasible in the definition of

$$\mu(a_1, \ldots, a_n; d).$$

Thus

$$\mu(a_1, \ldots, a_n; d) \le (a_n - s)\mu(a_1, \ldots, a_{n-1}; d - s),$$

which proves the theorem. □

The bound is sharp. For any feasible $d_1, \ldots, d_n$, choose

$$S_i \subseteq A_i, \qquad |S_i| = d_i,$$

and take

$$f(x) = \prod_{i=1}^{n} \prod_{a \in S_i} (x_i - a).$$

Then

$$\deg f = \sum_i d_i,$$

and f is nonzero on exactly

$$\prod_i (a_i - d_i)$$

grid points.

Ryser's Permanent Formula The coefficient formula can be significantly more useful than the ordinary version, as is shown in the following.

Theorem 144 (Ryser's Formula)**.** *For*

$$A = (a_{ij}) \in \mathbb{F}^{n \times n},$$

$$\boxed{\operatorname{per}(A) = \sum_{S \subseteq [n]} (-1)^{n-|S|} \prod_{i=1}^{n} \left(\sum_{j \in S} a_{ij} \right).}$$

Consequently the permanent can be computed in

$$O(n2^n)$$

arithmetic operations.

Proof. Define

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} \left(\sum_{j=1}^{n} a_{ij} x_j \right).$$

Choosing one variable from each factor shows

$$[x_1 x_2 \cdots x_n] P = \operatorname{per}(A).$$

Apply Theorem 141 with

$$A_j = \{0, 1\}, \qquad d_j = 1.$$

For

$$\phi(t) = t(t-1),$$
$$\phi'(0) = -1, \qquad \phi'(1) = 1.$$

Thus

$$\operatorname{per}(A) = \sum_{\varepsilon \in \{0,1\}^n} (-1)^{n - \sum_j \varepsilon_j} P(\varepsilon).$$

Writing

$$S = \{j : \varepsilon_j = 1\}$$

gives exactly

$$\operatorname{per}(A) = \sum_{S \subseteq [n]} (-1)^{n-|S|} \prod_i \sum_{j \in S} a_{ij}.$$

□

Chevalley–Warning The same coefficient identity can turn a degree bound into a congruence for the number of solutions of a polynomial system.

Theorem 145 (Chevalley–Warning)**.** *Let*

$$f_1, \dots, f_m \in \mathbb{F}_q[x_1, \dots, x_n]$$

satisfy

$$\sum_{j=1}^{m} \deg f_j < n.$$

Then the number N of common zeros of

$$f_1, \dots, f_m$$

in $\mathbb{F}_q^n$ is divisible by the characteristic p of $\mathbb{F}_q$.

Proof. Define

$$P(x) = \prod_{j=1}^{m} \left(1 - f_j(x)^{q-1}\right).$$

For every

$$a \in \mathbb{F}_q^n,$$

Fermat's theorem gives

$$P(a) = \begin{cases} 1, & f_1(a) = \cdots = f_m(a) = 0, \\ 0, & \text{otherwise.} \end{cases}$$

Hence in $\mathbb{F}_q$,

$$\sum_{a \in \mathbb{F}_q^n} P(a) = N.$$

Also,

$$\deg P \le (q-1) \sum_j \deg f_j < n(q-1).$$

Therefore

$$[x_1^{q-1} \cdots x_n^{q-1}] P = 0.$$

Use the degree-bounded form of Theorem 141 with

$$A_i = \mathbb{F}_q, \qquad d_i = q - 1.$$

Since

$$\phi_i(t) = \prod_{a \in \mathbb{F}_q} (t - a) = t^q - t,$$

we have

$$\phi_i'(a) = -1$$

for every $a \in \mathbb{F}_q$. Thus

$$0 = [x_1^{q-1} \cdots x_n^{q-1}]P = (-1)^n \sum_{a \in \mathbb{F}_q^n} P(a) = (-1)^n N$$

inside $\mathbb{F}_q$.

Hence

$$N \equiv 0 \pmod{p}.$$

□

5.6.3 Multiplicity Schwartz–Zippel

Ordinary Schwartz–Zippel counts zeros. The multiplicity version counts a zero several times according to how many derivatives vanish there.

For a multi-index

$$\alpha \in \mathbb{N}^n,$$

define the Hasse derivative $D^\alpha f$ by

$$f(x + z) = \sum_{\alpha \in \mathbb{N}^n} D^\alpha f(x) z^\alpha.$$

Definition 146 (Multiplicity)**.** For a nonzero polynomial f and a point a, define

$$\operatorname{mult}(f, a) = \min\{|\alpha| : D^\alpha f(a) \neq 0\}.$$

Thus

$$\operatorname{mult}(f, a) = 0$$

exactly when $f(a) \neq 0$.

We use the elementary fact

$$\operatorname{mult}(D^\alpha f, a) \geq \operatorname{mult}(f, a) - |\alpha|.$$

This follows immediately from the Taylor expansion: taking a Hasse derivative of order $|\alpha|$ can lower the smallest nonzero total degree by at most $|\alpha|$.

Theorem 147 (Multiplicity Schwartz–Zippel)**.** *Let*

$$0 \neq f \in \mathbb{F}[x_1, \ldots, x_n]$$

have total degree at most d, and let

$$S \subseteq \mathbb{F}$$

be finite. Then

$$\boxed{\sum_{a \in S^n} \operatorname{mult}(f, a) \leq d|S|^{n-1}.}$$

Consequently, for every $r \geq 1$,

$$\boxed{|\{a \in S^n : \operatorname{mult}(f, a) \geq r\}| \leq \frac{d}{r}|S|^{n-1}.}$$

Proof. We induct on n.

For $n = 1$, this is the usual fact that the sum of the multiplicities of the roots of a nonzero degree-d polynomial is at most d.

Now write

$$f(x_1, \dots, x_n) = \sum_{j=0}^{t} f_j(x_1, \dots, x_{n-1}) x_n^j,$$

where

$$f_t \neq 0.$$

Then

$$\deg f_t \leq d - t.$$

Fix

$$a' = (a_1, \dots, a_{n-1}) \in S^{n-1}$$

and put

$$r(a') = \mathrm{mult}(f_t, a').$$

Choose a multi-index α of size $r(a')$ such that

$$D^\alpha f_t(a') \neq 0.$$

Then the univariate polynomial

$$g(T) = D^{(\alpha,0)} f(a', T)$$

is nonzero and has degree exactly t.

For every $b \in S$,

$$\mathrm{mult}(f, (a', b)) \leq r(a') + \mathrm{mult}(g, b).$$

Summing over b and using the univariate multiplicity bound,

$$\sum_{b \in S} \mathrm{mult}(f, (a', b)) \leq |S| r(a') + t.$$

Now sum over a':

$$\begin{aligned} \sum_{a \in S^n} \mathrm{mult}(f, a) &\leq |S| \sum_{a' \in S^{n-1}} \mathrm{mult}(f_t, a') + t|S|^{n-1} \\ &\leq |S|(d-t)|S|^{n-2} + t|S|^{n-1} \\ &= d|S|^{n-1}. \end{aligned}$$

The second claim follows by Markov's inequality. □

5.6.4 Zero Patterns of Polynomial Maps

Given polynomials $f_1, \dots, f_m$, their zero pattern at a point records which of them vanish. We equivalently record the nonzero set

$$S(x) = \{i \in [m] : f_i(x) \neq 0\}.$$

Although the point x may range over an infinite field, the number of patterns is controlled by the dimension of a polynomial space.

Theorem 148 (Elementary zero-pattern bound)**.** *Let $f_1, \dots, f_m \in K[x_1, \dots, x_n]$, and suppose* $\deg f_i \le d_i$*, where $d_i \ge 0$. Writing $D = \sum_i d_i$, the number of realized zero patterns is at most*

$$\boxed{\binom{n+D}{n}.}$$

If every degree is at most d, then the number of realized patterns having at most s nonzero entries is at most

$$\boxed{\binom{n+ds}{n}.}$$

Both conclusions remain valid when the points are restricted to an arbitrary subset of K^n.

Proof. For every realized nonzero set S, choose a representative point x_S and define

$$P_S(x) = \prod_{i \in S} f_i(x).$$

Then

$$P_S(x_T) \neq 0 \quad \Longleftrightarrow \quad S \subseteq T.$$

Order the realized patterns by nondecreasing cardinality. The resulting evaluation matrix is triangular with nonzero diagonal, so the P_S's are linearly independent.

Every P_S has degree at most D, and the space of polynomials of total degree at most D in n variables has dimension $\binom{n+D}{n}$.

For the second claim, retain only patterns with $|S| \le s$. Their product polynomials have degree at most ds, and the same argument applies. □

Corollary 149 (Support patterns of low-rank matrices)**.** *Over any field, the number of support patterns of $a \times b$ matrices of rank at most $r \ge 1$ is at most*

$$\boxed{\binom{2ab + r(a+b)}{r(a+b)}.}$$

For $n \times n$ matrices and $1 \le r \le n$, this is at most

$$\left(\frac{2en}{r}\right)^{2nr}.$$

Proof. Every rank-at-most-r matrix can be written as UV, where $U \in K^{a \times r}$ and $V \in K^{r \times b}$. Its ab entries are degree-two polynomials in the $r(a+b)$ entries of U, V. Apply the zero-pattern bound.

For the square case, use $\binom{N}{k} \le (eN/k)^k$ and $r \le n$. □

5.6.5 Regular Subgraphs from Chevalley–Warning

Over $\mathbb{F}_p$, the expression x^{p-1} records whether x is nonzero. This lets a polynomial system select a subset of objects and impose linear constraints on its incidence vector.

Lemma 150 (A nonempty zero-sum subsequence)**.** *Let p be prime and $a_1, \dots, a_m \in \mathbb{F}_p^r$. If $m > r(p-1)$, there is a nonempty $S \subseteq [m]$ with*

$$\sum_{i \in S} a_i = 0.$$

Proof. For each coordinate j, impose

$$\sum_{i=1}^{m} a_{i,j} x_i^{p-1} = 0.$$

The sum of the polynomial degrees is at most $r(p-1) < m$. Chevalley–Warning says that the number of common zeros is divisible by p. The all-zero vector is one solution, so another solution exists.

For such a solution, let $S = \{i : x_i \neq 0\}$. Fermat's theorem gives $x_i^{p-1} = 1$ on S and zero elsewhere, so the equations say precisely that $\sum_{i \in S} a_i = 0$. □

The threshold is sharp: $p-1$ copies of each standard basis vector have no nonempty zero-sum subsequence. One may replace r by the dimension of the span of the a_i's.

Theorem 151 (A regular subgraph)**.** *Let p be prime, and let G be a finite loopless graph, allowing parallel edges. If*

$$|E(G)| > (p-1)|V(G)|, \qquad \Delta(G) \leq 2p-1,$$

then G contains a nonempty p-regular subgraph.

Proof. Associate to each edge its endpoint-incidence vector in $\mathbb{F}_p^{V(G)}$. The zero-sum lemma gives a nonempty edge set whose degree at every vertex is divisible by p.

Each selected degree lies between zero and $2p-1$, so it is either zero or p. Delete the isolated vertices. □

Corollary 152 (A cubic subgraph in an almost 4-regular graph)**.** *Adding one edge between distinct vertices of a finite 4-regular graph produces a graph containing a nonempty 3-regular subgraph.*

Proof. On n vertices the new graph has $2n+1$ edges and maximum degree at most five. Apply the theorem with $p = 3$. □

5.6.6 Alon–Tarsi Orientations and List Coloring

A proper coloring is a nonzero evaluation of the graph polynomial $\prod_{uv \in E}(x_u - x_v)$. An orientation identifies a particular coefficient of this polynomial: its exponent vector is the outdegree sequence, and its coefficient counts balanced edge subsets with signs.

Lemma 153 (The orientation coefficient)**.** *Let D orient a finite loopless graph G, and put*

$$P_D(x) = \prod_{u \to v \in E(D)} (x_u - x_v).$$

Call $F \subseteq E(D)$ balanced *if $d_F^+(v) = d_F^-(v)$ at every vertex. Then*

$$\boxed{\left[\prod_v x_v^{d_D^+(v)}\right] P_D = \sum_{\substack{F \subseteq E(D) \\ F \text{ balanced}}} (-1)^{|F|}.}$$

Proof. Choosing x_u from every factor $x_u - x_v$ gives the exponent vector d_D^+. Let F contain the edges on which we instead choose $-x_v$. The exponent of x_v becomes

$$d_D^+(v) - d_F^+(v) + d_F^-(v).$$

This equals $d_D^+(v)$ at every vertex exactly when F is balanced, and the term has sign $(-1)^{|F|}$. □

Theorem 154 (Alon–Tarsi list coloring)**.** *Suppose an orientation D of G has different numbers of even-sized and odd-sized balanced edge subsets. For any lists $L(v)$ satisfying*

$$|L(v)| \geq d_D^+(v) + 1,$$

there is a proper coloring c with $c(v) \in L(v)$ for every v.

Proof. Choose $d_D^+(v)+1$ elements from each list and identify their union with distinct real numbers. The preceding coefficient is a nonzero integer, and $\deg P_D = |E| = \sum_v d_D^+(v)$. Combinatorial Nullstellensatz supplies a list assignment where $P_D \neq 0$. Every factor is then nonzero, so adjacent vertices receive different colors. □

Corollary 155 (Orientations with no odd directed cycle)**.** *If D has no odd directed cycle, lists of size $d_D^+(v)+1$ suffice.*

Proof. Every nonempty balanced edge set decomposes into edge-disjoint directed cycles. To obtain the decomposition, follow edges until a directed cycle appears, remove that cycle, and repeat; balance is preserved.

All these cycles have even length, so every balanced edge set has even size. The signed sum in Lemma 153 is therefore positive, since it includes the empty set. □

Obtaining the orientation from a density bound.

Lemma 156 (Prescribed outdegree upper bounds)**.** *Let $G=(V,E)$ be a finite loopless graph, and let b_v be nonnegative integers. An orientation with $d^+(v) \le b_v$ exists if and only if*

$$\boxed{|E(S)| \le \sum_{v\in S} b_v \qquad (S \subseteq V).}$$

Proof. Necessity holds because each edge internal to S has one tail in S.

For sufficiency, make a bipartite graph with one left vertex for each edge of G and b_v right slots for each $v \in V$. Connect an edge to every slot of either endpoint.

For a set F of left vertices, let S be its set of endpoints. Its neighborhood has size $\sum_{v\in S} b_v$, which is at least $|E(S)| \ge |F|$. Hall's theorem supplies a matching covering all edges. Orient each edge away from the vertex whose slot receives it. □

Corollary 157 (List coloring bipartite graphs from density)**.** *Let G be bipartite, and suppose*

$$|E(S)| \le \sum_{v\in S} b_v \qquad (S \subseteq V).$$

Then arbitrary lists of size at least b_v+1 at vertex v admit a proper coloring.

Proof. Choose the bounded-outdegree orientation from the lemma. Every directed cycle is even because the underlying graph is bipartite. Apply Corollary 155. □

Corollary 158 (Bipartite planar graphs are 3-choosable)**.** *Every finite simple bipartite planar graph admits a proper coloring from arbitrary lists of size three.*

Proof. Every connected simple bipartite plane graph with at least three vertices has facial boundary walks of length at least four. Thus $2e \ge 4f$, and Euler's formula gives $e \le 2v-4$. Components with one or two vertices also satisfy $e \le 2v$.

The same applies to every induced subgraph, so $|E(S)| \le 2|S|$ for all $S \subseteq V$. Use $b_v = 2$ in the preceding corollary. □

5.6.7 Restricted Intersections

Intersection restrictions produce low-degree polynomials that separate the members of a set family. The resulting bound depends on the dimension of the polynomial functions on the domain where they are evaluated. On one uniform layer, that dimension is smaller than on the whole Boolean cube.

Low-degree functions on a uniform layer. For $S \subseteq [n]$, write $x_S = \prod_{i \in S} x_i$. On Boolean inputs, replacing every positive power of x_i by x_i preserves values and does not increase degree. Thus polynomial functions of degree at most s on the Boolean cube have dimension at most

$$\sum_{j=0}^{\min\{s,n\}} \binom{n}{j}.$$

Lemma 159 (Dimension on a uniform layer)**.** *Let $0 \le s \le k \le n$, and let K be any field. The restrictions of degree-at-most-s polynomials to incidence vectors of k-subsets of $[n]$ span a space of dimension at most*

$$\boxed{\binom{n}{s}.}$$

Proof. First work over $\mathbb{Q}$. For $|S| = j \le s$, on the k-th layer we have

$$\sum_{\substack{T \supseteq S \\ |T|=s}} x_T = \binom{k-j}{s-j} x_S.$$

Indeed, if the input set contains S, the left side counts its s-subsets containing S; otherwise both sides vanish. The binomial coefficient is positive, so every monomial of degree at most s is in the span of the degree-s monomials. There are $\binom{n}{s}$ of these.

Now consider the evaluation matrix whose rows are k-subsets and whose columns are squarefree monomials of degree at most s. Its entries are integers zero and one. Every minor larger than $\binom{n}{s}$ vanishes over $\mathbb{Q}$, hence vanishes as an integer and therefore over every field. Its rank over K is consequently at most $\binom{n}{s}$. □

Lemma 160 (Intersection-polynomial bound)**.** *Let $A_1, \dots, A_m$ be distinct subsets of $[n]$. Suppose there are polynomials $q_i(t) \in K[t]$, each of degree at most s, such that*

$$q_i(|A_i|) \neq 0, \qquad q_i(|A_i \cap A_j|) = 0 \quad (j < i).$$

Integers in these evaluations are interpreted in K. Then

$$\boxed{m \le \sum_{j=0}^{\min\{s,n\}} \binom{n}{j}.}$$

If every A_i has size k and $s \le k$, then

$$\boxed{m \le \binom{n}{s}.}$$

Proof. Associate to A_i the polynomial function

$$P_i(x) = q_i\left(\sum_{v \in A_i} x_v\right).$$

At the incidence vector of A_j, its value is $q_i(|A_i \cap A_j|)$. The evaluation matrix is triangular with nonzero diagonal, so the functions P_i are linearly independent.

Their Boolean reductions have degree at most s. Use the Boolean-cube dimension bound, or Lemma 159 in the uniform case. □

Theorem 161 (Restricted intersections over the integers)**.** *Let L be a set of s nonnegative integers, and let $\mathcal{F} \subseteq 2^{[n]}$ satisfy*

$$|A \cap B| \in L \qquad (A, B \in \mathcal{F},\ A \neq B).$$

Then

$$\boxed{|\mathcal{F}| \leq \sum_{j=0}^{\min\{s,n\}} \binom{n}{j}.}$$

If $\mathcal{F}$ is k-uniform and $s \leq k$, then

$$\boxed{|\mathcal{F}| \leq \binom{n}{s}.}$$

Proof. Order the sets by nondecreasing cardinality and work over $\mathbb{Q}$. For each A_i, put

$$q_i(t) = \prod_{\substack{\ell \in L \\ \ell < |A_i|}} (t - \ell).$$

Its diagonal value $q_i(|A_i|)$ is nonzero.

If $j < i$, then $|A_j| \leq |A_i|$, and distinctness gives $|A_i \cap A_j| < |A_i|$. This intersection size belongs to L, so it is one of the roots of q_i. Apply Lemma 160. □

Discarding roots at least $|A_i|$ is necessary in the nonuniform case: a set size may itself belong to L, but only the smaller roots are needed to annihilate earlier sets.

The nonuniform bound is sharp for $L = \{0, \ldots, s-1\}$ and the family of all subsets of size at most s, when $s \leq n$. For a uniform family, $s = 1$ gives the bound n when all distinct pairs have the same intersection size.

Theorem 162 (Frankl–Wilson modular intersection bound)**.** *Let p be prime and $L \subseteq \mathbb{F}_p$, with $|L| = s$. Suppose $\mathcal{F} \subseteq 2^{[n]}$ satisfies*

$$|A| \bmod p \notin L \quad (A \in \mathcal{F}),$$

and

$$|A \cap B| \bmod p \in L \quad (A, B \in \mathcal{F},\ A \neq B).$$

Then

$$\boxed{|\mathcal{F}| \leq \sum_{j=0}^{\min\{s,n\}} \binom{n}{j}.}$$

If $\mathcal{F}$ is k-uniform and $s \leq k$, then

$$\boxed{|\mathcal{F}| \leq \binom{n}{s}.}$$

Proof. Work over $\mathbb{F}_p$ and take

$$q_i(t) = \prod_{\ell \in L} (t - \ell)$$

for every i. The size condition makes every diagonal evaluation nonzero, and the intersection condition makes every off-diagonal evaluation zero. Apply Lemma 160. □

5.6.8 Slice Rank and Low-Degree Tensor Methods

The polynomial method becomes particularly effective when a forbidden configuration can be encoded by a tensor that is diagonal on the set of interest. The reason is a simple relationship:

diagonal support forces large slice rank, low degree forces small slice rank.

We first isolate this mechanism once. The cap-set theorem, tri-colored sum-free theorem, bounds for general systems of linear equations, and the three-sunflower bound will then be short consequences. The tri-colored theorem also supplies the main input to the sharp arithmetic removal lemma.

Slice rank Let $X_1, \dots, X_k$ be finite sets and $T : X_1 \times \cdots \times X_k \to \mathbb{F}$.

A *slice* is a function of the form

$$f(x_i)g(x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_k)$$

for some distinguished coordinate i. The *slice rank* $\operatorname{srank}(T)$ is the minimum number of slices whose sum is T.

For $k = 2$, this is ordinary matrix rank. For larger k, one factor may depend arbitrarily on all but one coordinate, which makes slice rank much more flexible than ordinary tensor rank.

We need one elementary support lemma.

Lemma 163 (Large support in a subspace)**.** *If $V \le \mathbb{F}^X$ has dimension d, then some $f \in V$ is nonzero on at least d elements of X.*

Proof. Choose $D \subseteq X$, $|D| = d$, such that restriction $V \to \mathbb{F}^D$ has rank d. It is therefore onto. Choose $f \in V$ whose restriction to D is identically one. □

The analogue of "a nonsingular diagonal matrix has full rank" is the central fact behind the method.

Theorem 164 (Diagonal tensor lemma)**.** *Let A be finite, let $k \ge 2$, and suppose $T : A^k \to \mathbb{F}$ satisfies*

$$T(a_1, \dots, a_k) \neq 0 \quad \Longleftrightarrow \quad a_1 = \cdots = a_k.$$

Then

$$\boxed{\operatorname{srank}(T) = |A|.}$$

Proof. The upper bound follows by slicing in the first coordinate:

$$T(x_1, \dots, x_k) = \sum_{a \in A} 1_{x_1 = a} T(a, x_2, \dots, x_k).$$

For the lower bound, induct on k. The case $k = 2$ is ordinary matrix rank.

Suppose

$$T = \sum_{i=1}^{k} \sum_{j=1}^{r_i} f_{i,j}(x_i) g_{i,j}(x_1, \dots, \widehat{x_i}, \dots, x_k),$$

and put $r = \sum_i r_i$.

Let W be the span of $f_{1,1}, \dots, f_{1,r_1}$ in $\mathbb{F}^A$. Its orthogonal complement has dimension at least $|A| - r_1$, so by Lemma 163 there is $h \in W^{\perp}$ with

$$|\operatorname{supp} h| \ge |A| - r_1.$$

Contract the first coordinate against h:

$$T_h(x_2, \dots, x_k) = \sum_{x_1 \in A} h(x_1) T(x_1, x_2, \dots, x_k).$$

All slices distinguished by the first coordinate vanish. Every other slice remains a slice, so

$$\operatorname{srank}(T_h) \le r - r_1.$$

The tensor T_h is diagonal, with nonzero diagonal entries precisely on $\operatorname{supp} h$. By induction,

$$r - r_1 \ge \operatorname{srank}(T_h) = |\operatorname{supp} h| \ge |A| - r_1.$$

Thus $r \ge |A|$. □

Low degree gives few slices The second ingredient is trivial. A monomial of small total degree must have small degree in at least one variable block.

For a prime p, write

$$M_p(n,d) = \left|\{\alpha \in \{0,\dots,p-1\}^n : |\alpha| \le d\}\right|.$$

This is the number of reduced monomials in n variables of total degree at most d.

Lemma 165 (Polynomial slice-rank bound)**.** *Let*

$$P(x^{(1)},\dots,x^{(k)})$$

be a polynomial function on $(\mathbb{F}_p^n)^k$, represented with degree at most $p-1$ in every scalar variable and total degree at most D. Then

$$\boxed{\operatorname{srank}(P) \le kM_p\left(n, \left\lfloor \frac{D}{k} \right\rfloor\right).}$$

Proof. Expand P into monomials. For every monomial, at least one of its k blocks has degree at most D/k. Assign the monomial to one such block.

For a fixed block i, group together all assigned monomials having the same monomial in $x^{(i)}$. Such a group is one slice: the chosen monomial is the one-variable-block factor, and everything else is absorbed into the other factor.

There are at most $M_p(n, \lfloor D/k \rfloor)$ choices for the low-degree monomial in each block. □

We will also use the identical Boolean-cube specialization.

Corollary 166 (Multilinear slice-rank bound)**.** *Suppose P is multilinear in each scalar variable, with k blocks of n variables and total degree at most D. Then*

$$\operatorname{srank}(P) \le k \sum_{j=0}^{\lfloor D/k \rfloor} \binom{n}{j}.$$

Combining low slice rank with the diagonal tensor lemma gives the general method.

Corollary 167 (Diagonal-polynomial principle)**.** *Let $\phi_i : [m] \to \mathbb{F}_p^n$ for $i = 1,\dots,k$, and let P be as in Lemma 165. Suppose*

$$P(\phi_1(j_1),\dots,\phi_k(j_k)) \ne 0 \quad \Longleftrightarrow \quad j_1 = \cdots = j_k.$$

Then

$$\boxed{m \le kM_p\left(n, \left\lfloor \frac{D}{k} \right\rfloor\right).}$$

Proof. Restrict P to the k indexed lists. The resulting tensor is diagonal, so its slice rank is m by Theorem 164. Restriction cannot increase the displayed slice decomposition, and Lemma 165 supplies the upper bound. □

Counting the low-degree monomials For $0 < \rho < 1$,

$$\begin{aligned} M_p(n,d)\rho^d &\le \sum_{\alpha \in \{0,\dots,p-1\}^n} \rho^{|\alpha|} \\ &= (1+\rho+\cdots+\rho^{p-1})^n. \end{aligned}$$

Thus

$$\boxed{M_p(n,d) \le \rho^{-d}(1+\rho+\cdots+\rho^{p-1})^n.} \tag{16}$$

Define

$$\boxed{\theta_p = \min_{0<\rho<1} (1+\rho+\cdots+\rho^{p-1})\rho^{-(p-1)/3}.}$$

It is convenient to write

$$\theta_p = p^{1-c_p}, \qquad c_p = 1 - \log_p \theta_p.$$

Then $0 < c_p < 1$.

Tri-colored sum-free sets A *tri-colored sum-free set* is a collection

$$\{(x_i, y_i, z_i) : 1 \le i \le m\}$$

such that

$$x_i + y_j + z_k = 0 \quad \Longleftrightarrow \quad i = j = k.$$

This is exactly a diagonal zero-set problem.

Theorem 168 (Tri-colored sum-free bound)**.** *If*

$$\{(x_i, y_i, z_i) : 1 \le i \le m\} \subseteq (\mathbb{F}_p^n)^3$$

is tri-colored sum-free, then

$$\boxed{m \le \theta_p^n = p^{(1-c_p)n}.}$$

Proof. Use the zero-sum indicator

$$P(x,y,z) = \prod_{\ell=1}^{n} \left(1 - (x_\ell + y_\ell + z_\ell)^{p-1}\right).$$

Fermat's theorem gives $P(x,y,z) = 1$ exactly when $x+y+z = 0$. The total degree is $(p-1)n$, so Corollary 167 gives

$$m \le 3M_p\left(n, \left\lfloor \frac{(p-1)n}{3} \right\rfloor\right) \le 3\theta_p^n. \tag{1}$$

The factor 3 disappears by tensor powering. The r-fold Cartesian power of the collection is tri-colored sum-free of size m^r in $\mathbb{F}_p^{nr}$. Hence

$$m^r \le 3\theta_p^{nr}.$$

Take r-th roots and let $r \to \infty$. □

The exponential rate is sharp.

Theorem 169 (Sharpness of the tri-colored bound; external)**.** *For every fixed prime p, there exist tri-colored sum-free collections in $\mathbb{F}_p^n$ of size*

$$\boxed{p^{(1-c_p)n - o(n)} = \theta_p^{n-o(n)}.}$$

General translation-invariant linear systems The same diagonal-polynomial principle handles many equations at once.

Fix r homogeneous equations in k vector variables,

$$\sum_{i=1}^{k} a_{j,i} x_i = 0, \qquad j = 1, \dots, r,$$

where $a_{j,i} \in \mathbb{F}_p$. Assume

$$\sum_{i=1}^{k} a_{j,i} = 0 \qquad (j = 1, \dots, r),$$

so every diagonal tuple is a solution.

Define

$$\Gamma_{p,r,k} = \min_{0<\rho<1} (1 + \rho + \cdots + \rho^{p-1}) \rho^{-r(p-1)/k}.$$

Theorem 170 (Diagonal solutions to linear systems)**.** *Suppose $A \subseteq \mathbb{F}_p^n$ has the property that every solution in A^k to the preceding system is diagonal. Then*

$$\boxed{|A| \le \Gamma_{p,r,k}^n.}$$

If $k > 2r$, then $\Gamma_{p,r,k} < p$.

Proof. The system indicator is

$$P(x_1, \dots, x_k) = \prod_{j=1}^{r} \prod_{\ell=1}^{n} \left[1 - \left(\sum_{i=1}^{k} a_{j,i} x_{i,\ell} \right)^{p-1} \right].$$

Reduce modulo $u^p - u$ in every scalar variable. This preserves the function on $\mathbb{F}_p$, keeps every scalar degree at most $p-1$, and does not increase total degree. Thus the total degree is at most $r(p-1)n$.

On A^k, the polynomial is nonzero exactly on the diagonal. Hence

$$|A| \le k M_p \left(n, \left\lfloor \frac{r(p-1)n}{k} \right\rfloor \right) \le k \Gamma_{p,r,k}^n.$$

Apply this to the s-fold Cartesian power A^s, take s-th roots, and let $s \to \infty$.

For the last claim, put $\rho = e^{-t}$. At $t = 0$, the expression defining $\Gamma_{p,r,k}$ equals p. The derivative of its logarithm at $t = 0$ is

$$-\frac{p-1}{2} + \frac{r(p-1)}{k},$$

which is negative when $k > 2r$. Hence a sufficiently small positive t makes the value strictly less than p. □

The cap-set theorem is now immediate.

Corollary 171 (Cap-set bound)**.** *If $A \subseteq \mathbb{F}_3^n$ contains no nontrivial solution to $x + y + z = 0$, equivalently no nontrivial three-term arithmetic progression, then*

$$\boxed{|A| \le \theta_3^n,}$$

where

$$\theta_3 = \min_{0<\rho<1} (1 + \rho + \rho^2) \rho^{-2/3} \approx 2.7551046.$$

The minimizer is

$$\rho = \frac{\sqrt{33} - 1}{8}.$$

Proof. Apply Theorem 170 with $p = 3$, $r = 1$, $k = 3$, and equation $x + y + z = 0$.

For the minimizer, differentiating the logarithm gives

$$\frac{1 + 2\rho}{1 + \rho + \rho^2} = \frac{2}{3\rho},$$

or $4\rho^2 + \rho - 2 = 0$. □

For odd p, the same argument applied to $x - 2y + z = 0$ gives the corresponding exponential bound for sets with no nontrivial three-term progression in $\mathbb{F}_p^n$.

Three-petal sunflowers The Boolean-cube specialization gives another application.

Theorem 172 (Slice-rank sunflower bound)**.** *Let $\mathcal{F} \subseteq 2^{[n]}$ contain no three distinct sets forming a sunflower. Then*

$$\boxed{|\mathcal{F}| \le 3(n+1) \sum_{j=0}^{\lfloor n/3 \rfloor} \binom{n}{j} \le 3(n+1) \left(\frac{3}{2^{2/3}}\right)^n.}$$

Proof. Partition $\mathcal{F}$ by cardinality. Fix one uniform layer $\mathcal{A} \subseteq \binom{[n]}{s}$, represented by its indicator vectors in $\{0,1\}^n$.

Over a field of characteristic zero, define

$$P(x, y, z) = \prod_{i=1}^{n} (2 - x_i - y_i - z_i).$$

For Boolean inputs, a factor vanishes exactly when its coordinate has membership pattern 110, 101, or 011, i.e. exactly two of the three sets contain that element.

Suppose first that $x, y, z \in \mathcal{A}$ are three distinct sets. If $P(x, y, z) \neq 0$, then no coordinate lies in exactly two sets. Consequently every pairwise intersection consists exactly of the coordinates of type 111, so the three pairwise intersections are equal. Thus x, y, z form a sunflower, a contradiction.

If exactly two of x, y, z coincide, say $x = y \neq z$, then $|x| = |z|$ implies $x \setminus z \neq \varnothing$. At such a coordinate the pattern is 110, so again $P = 0$.

On the diagonal, every factor equals 2 or -1, so $P \neq 0$. Thus P restricts to a diagonal tensor on $\mathcal{A}^3$.

The polynomial is multilinear and has total degree n. By Corollary 166 and Theorem 164,

$$|\mathcal{A}| \le 3 \sum_{j \le n/3} \binom{n}{j}.$$

There are $n + 1$ cardinality layers.

Finally, for $\rho = 1/2$,

$$\begin{aligned} \sum_{j \le n/3} \binom{n}{j} &\le \rho^{-n/3} \sum_{j=0}^{n} \binom{n}{j} \rho^j \\ &= 2^{n/3} \left(\frac{3}{2}\right)^n = \left(\frac{3}{2^{2/3}}\right)^n. \end{aligned}$$

□

Arithmetic triangle removal We now use the tri-colored theorem in a different way. Instead of assuming that the only triangles are already isolated, we start with an arbitrary collection of interacting triangles and randomly pass to a small subspace. Bounded local triangle degree then makes a positive fraction of the sampled triangles isolated, at which point Theorem 168 applies.

Let $N = p^n$. For $X, Y, Z \subseteq \mathbb{F}_p^n$, a *triangle* is a triple $(x, y, z) \in X \times Y \times Z$ satisfying $x + y + z = 0$. We regard the three sets as colored copies, so vertices in different color classes are distinct even if they represent the same vector.

Set

$$\boxed{C_p = 1 + \frac{1}{c_p}.}$$

Lemma 173 (Bounded triangle degree)**.** *Suppose $X, Y, Z \subseteq \mathbb{F}_p^n \setminus \{0\}$ are pairwise disjoint and satisfy:*

1. *every two distinct vectors in $X \cup Y \cup Z$ are linearly independent;*
2. *every two-dimensional subspace contains at most one triangle from $X \times Y \times Z$.*

Suppose every point lies in at most ρN triangles, where $0 < \rho \le 1/(5p^3)$, and suppose there are δN^2 triangles in total. Then

$$\boxed{\delta \le 125 p^2 \rho^{1+c_p}.}$$

Proof. Choose

$$d = \left\lfloor \log_p \frac{1}{5\rho} \right\rfloor.$$

Then $d \ge 3$ and

$$\frac{1}{5p} < \rho p^d \le \frac{1}{5}. \tag{2}$$

Choose a uniformly random d-dimensional subspace $U \le \mathbb{F}_p^n$. Call a sampled triangle *good* if it is the only sampled triangle containing any of its three vertices.

Fix a triangle (x, y, z), and let $W = \operatorname{span}\{x, y\}$. The assumptions give $\dim W = 2$, and $z = -x - y \in W$.

Condition on $W \subseteq U$. Then U/W is a uniformly random $(d-2)$-subspace of the $(n-2)$-dimensional quotient $\mathbb{F}_p^n/W$.

Consider another triangle containing x, say (x, y', z'). By the second genericity condition, $y' \notin W$. Thus $y' + W$ is a fixed nonzero vector of the quotient. By symmetry,

$$\Pr[y' \in U \mid W \subseteq U] = \frac{p^{d-2} - 1}{p^{n-2} - 1}.$$

Moreover

$$\frac{p^{d-2} - 1}{p^{n-2} - 1} \le \frac{p^{d-2}}{p^{n-2}} = p^{d-n};$$

after cross-multiplication this inequality is simply $p^{d-2} \le p^{n-2}$.

There are at most ρN triangles containing x, so conditioned on $W \subseteq U$, the probability that x lies in another sampled triangle is at most

$$\rho N p^{d-n} = \rho p^d \le \frac{1}{5}.$$

The same estimate holds for y and z. A union bound therefore shows that, conditioned on $W \subseteq U$, the triangle is good with probability at least $2/5$.

It remains to sample W. A random d-subspace contains a fixed two-dimensional subspace with probability

$$\begin{aligned} q &:= \Pr[W \subseteq U] \\ &= \frac{(p^d - 1)(p^{d-1} - 1)}{(p^n - 1)(p^{n-1} - 1)}. \end{aligned}$$

Factoring out the leading powers,

$$q = p^{2(d-n)} \frac{(1-p^{-d})(1-p^{-(d-1)})}{(1-p^{-n})(1-p^{-(n-1)})}.$$

Since $d \geq 3$ and $p \geq 2$, the fraction is at least $1/2$. Hence

$$q \geq \frac{1}{2} p^{2(d-n)}.$$

Therefore a fixed triangle is good with probability at least

$$\frac{2}{5} q \geq \frac{1}{5} p^{2(d-n)}.$$

From the strict inequality in (2), $p^d > 1/(5p\rho)$, so

$$\Pr[\text{fixed triangle is good}] > \frac{1}{125 p^2 \rho^2 N^2}.$$

If T is the number of good triangles, summing over the δN^2 original triangles gives

$$\mathbb{E}T \geq \frac{\delta}{125 p^2 \rho^2}. \tag{3}$$

The good triangles form a tri-colored sum-free collection: a mixed zero-sum triple would be a second sampled triangle containing each of its chosen vertices. Thus

$$T \leq \theta_p^d = p^{(1-c_p)d}.$$

Together with (3),

$$\frac{\delta}{125 p^2 \rho^2} \leq p^{(1-c_p)d}.$$

Since $p^d \leq 1/(5\rho) \leq 1/\rho$,

$$p^{(1-c_p)d} \leq \rho^{-(1-c_p)}.$$

Rearranging gives

$$\delta \leq 125 p^2 \rho^{1+c_p}.$$

□

The genericity assumptions can be enforced without altering the triangle pattern.

Lemma 174 (Generic padding)**.** *Replace colored points $x_i, y_i, z_i \in \mathbb{F}_p^n$ by*

$$x_i' = (x_i, 1, 0), \qquad y_i' = (y_i, -1, 1), \qquad z_i' = (z_i, 0, -1)$$

in $\mathbb{F}_p^{n+2}$.

Then triangles are preserved exactly, the three color classes become disjoint, every two distinct padded points are linearly independent, and every two-dimensional subspace contains at most one triangle.

Proof. The final two coordinates sum to zero, so

$$x_i' + y_j' + z_k' = 0 \quad \Longleftrightarrow \quad x_i + y_j + z_k = 0.$$

They also distinguish the three colors.

If two padded points are scalar multiples, their final two coordinates force the scalar to equal one and the points to coincide.

Finally fix a triangle (x_i', y_j', z_k'), and let W be its two-dimensional span. If another Y-point $y_{j'}'$ belonged to W, then

$$y_{j'}' = \alpha x_i' + \beta y_j'.$$

The last two coordinates force $\alpha = 0$ and $\beta = 1$, hence $y_{j'}' = y_j'$. The same argument applies to the X- and Z-points. Thus this is the unique triangle contained in W. □

We next remove the bounded-degree hypothesis. Only the exponent matters at this stage.

The preceding lemma says that if every point lies in at most ρN triangles, then the total triangle density is at most $O_p(\rho^{1+c_p})$. By itself, this is only an upper bound. The key is to make ρ proportional to the triangle density itself.

A family of ηN vertex-disjoint triangles makes the configuration robust under deletions: deleting one point destroys at most one of these distinguished triangles. Thus fewer than ηN point deletions cannot destroy every triangle.

We exploit this by deleting points that belong to disproportionately many triangles. Such a deletion removes many triangles while sacrificing at most one distinguished triangle. When the current triangle density is β, use a degree threshold $g(\beta)\beta N/\eta$, where g is a slowly growing function. During one scale at which the density stays between β and 2β, only $O(\beta N^2)$ triangles are available to be destroyed, so at most $O(\eta N/g(\beta))$ points can be deleted. Taking $g(\beta)$ to be a sufficiently large constant times $\log^2(e/\beta)$ makes these deletion costs summable over all scales.

Consequently, the process stops before half the distinguished triangles are destroyed. The remaining density β is positive, but every point now has degree at most ρN, where $\rho = g(\beta)\beta/\eta$. In the small-density regime, the preceding lemma gives

$$\beta \lesssim_p \left(\frac{g(\beta)\beta}{\eta}\right)^{1+c_p} \quad \Longrightarrow \quad \beta \gtrsim_p \left(\frac{\eta}{g(\beta)}\right)^{C_p}, \qquad C_p = 1 + \frac{1}{c_p}.$$

The upper bound contains β^{1+c_p}, which decreases faster than β. If the remaining triangle count were too small, the upper bound would be smaller than that count itself. Pruning rules out excessive concentration at individual points, while disjointness rules out the empty configuration; together they force many triangles. The logarithmic overhead produces the intermediate exponent $C_p + o(1)$, and Cartesian powering later removes that loss.

Lemma 175 (Disjoint triangles force many triangles, asymptotically)**.** *Suppose $X, Y, Z \subseteq \mathbb{F}_p^n$ contain ηN pairwise vertex-disjoint triangles. Then, for fixed p and $\eta \to 0$,*

$$\boxed{\#\{\textit{triangles in } X \times Y \times Z\} \geq \eta^{C_p + o(1)} N^2.}$$

Proof. By Lemma 174, after adjoining two coordinates we may assume the genericity hypotheses of Lemma 173. This changes N, the density of the distinguished triangles, and the total triangle density only by fixed powers of p, which are absorbed into the $o(1)$ exponent. We therefore work directly in the generic situation.

Let the current triangle density be β, so there are βN^2 triangles. Choose

$$g(\beta) = K \log^2(1/\beta),$$

with K a sufficiently large absolute constant. For sufficiently small β, both $g(\beta)$ and the monotonicity properties used below behave in the evident direction.

Repeatedly delete any point lying in at least

$$g(\beta)\frac{\beta}{\eta}N$$

current triangles, recomputing β after each deletion.

We first bound the number of deleted points. During a dyadic scale in which the triangle density decreases from at most 2β to below β, every deletion destroys at least $g(\beta)\beta N/\eta$ triangles, while fewer than $2\beta N^2$ triangles are available. Thus this scale causes at most

$$\frac{2\eta N}{g(\beta)}$$

deletions. Summing over dyadic scales gives fewer than $\eta N/2$ deletions when K is sufficiently large, because

$$\sum_{j\geq 0} \frac{1}{\log^2(2^j/\beta_0)} < \infty.$$

Since the distinguished triangles are vertex-disjoint, fewer than $\eta N/2$ deletions cannot destroy all of them. Thus the process terminates with positive triangle density β, and every remaining point belongs to at most ρN triangles, where

$$\rho = g(\beta)\frac{\beta}{\eta}.$$

If $\rho > 1/(5p^3)$, then

$$\beta > \frac{\eta}{5p^3 g(\beta)}.$$

Taking logarithms shows $\log(1/\beta) \leq (1+o(1))\log(1/\eta)$, hence

$$\beta \geq \eta^{1+o(1)} \geq \eta^{C_p+o(1)},$$

since $C_p > 1$.

Otherwise Lemma 173 applies:

$$\beta \leq 125p^2 \left(g(\beta)\frac{\beta}{\eta}\right)^{1+c_p}.$$

Rearranging,

$$\beta^{c_p} \geq \frac{\eta^{1+c_p}}{125p^2 g(\beta)^{1+c_p}}. \tag{4}$$

Put

$$u = \log\frac{1}{\beta}, \qquad v = \log\frac{1}{\eta}.$$

Since $g(\beta) = Ku^2$, taking logarithms of (4) gives

$$c_p u \leq (1+c_p)v + O(\log u).$$

This first implies $u = O(v)$. Hence $O(\log u) = o(v)$, and therefore

$$u \leq \left(\frac{1+c_p}{c_p} + o(1)\right) v = (C_p + o(1))v.$$

Equivalently,

$$\beta \geq \eta^{C_p+o(1)}.$$

The original configuration had at least as many triangles. □

A standard product argument removes the $o(1)$ completely.

Theorem 176 (Disjoint-triangle supersaturation)**.** *If $X, Y, Z \subseteq \mathbb{F}_p^n$ contain ηN pairwise vertex-disjoint triangles, then*

$$\boxed{\#\{\text{triangles in } X\times Y\times Z\} \geq \eta^{C_p} N^2.}$$

Proof. Suppose the triangle density is $\delta < \eta^{C_p}$. Then for some $\alpha > 0$,

$$\delta = \eta^{(1+\alpha)C_p}.$$

Take the r-fold Cartesian power. The new space has size N^r, contains $\eta^r N^r$ distinguished disjoint triangles, and has total triangle density δ^r. By Lemma 175,

$$\delta^r \geq (\eta^r)^{C_p+o(1)}$$

as $r \to \infty$, because $\eta^r \to 0$.

But

$$\delta^r = (\eta^r)^{(1+\alpha)C_p},$$

contradicting the preceding inequality for sufficiently large r. □

Theorem 177 (Arithmetic triangle removal in $\mathbb{F}_p^n$)**.** *Let $0 < \varepsilon < 1$ and*

$$\boxed{\delta = \left(\frac{\varepsilon}{3}\right)^{C_p}.}$$

If $X, Y, Z \subseteq \mathbb{F}_p^n$ contain fewer than δN^2 triangles, then fewer than εN colored points can be deleted so that no triangle remains.

Proof. Greedily choose a maximal family of vertex-disjoint triangles.

If it contains fewer than $\varepsilon N/3$ triangles, delete the three vertices of every chosen triangle. This removes fewer than εN colored points, and maximality ensures that no triangle remains.

Therefore, if εN deletions are necessary, there are at least $\varepsilon N/3$ disjoint triangles. By Theorem 176, there are at least

$$\left(\frac{\varepsilon}{3}\right)^{C_p} N^2$$

triangles in total. This is the contrapositive. □

The exponent C_p is essentially optimal.

Theorem 178 (Sharpness of arithmetic removal)**.** *For every fixed prime p, there are examples with $\varepsilon \to 0$ for which at least εN deletions are necessary, while the triangle density satisfies*

$$\boxed{\delta = \varepsilon^{C_p - o(1)}.}$$

Proof. Take a tri-colored sum-free collection

$$\{(x_i, y_i, z_i) : 1 \le i \le m\} \subseteq (\mathbb{F}_p^n)^3$$

with

$$m = p^{(1-c_p)n - o(n)}$$

from Theorem 169.

For $\ell \ge 1$, replace each point by a fiber:

$$X' = X \times \mathbb{F}_p^\ell,$$
$$Y' = Y \times \mathbb{F}_p^\ell,$$
$$Z' = Z \times \mathbb{F}_p^\ell.$$

Tri-colored sum-freeness forces every triangle to use the same index i. For fixed i, there are $p^{2\ell}$ triangles, because two fiber coordinates determine the third. Hence there are $mp^{2\ell}$ triangles in total.

For a fixed i, at least p^ℓ points must be deleted from its three fibers to remove all triangles. Indeed, if fewer are deleted, the remaining fiber sets $U, V, W \subseteq \mathbb{F}_p^\ell$ satisfy

$$|U| + |V| + |W| > 2p^\ell.$$

Choose $w \in W$. Then $|U| + |V| > p^\ell$, so the sets U and $-w - V$ intersect, giving $u + v + w = 0$.

Thus at least mp^ℓ deletions are necessary. Relative to the ambient size $p^{n+\ell}$,

$$\varepsilon = \frac{m}{p^n}, \qquad \delta = \frac{m}{p^{2n}}.$$

Since $m = p^{(1-c_p)n - o(n)}$,

$$\varepsilon = p^{-c_p n - o(n)}, \qquad \delta = p^{-(1+c_p)n - o(n)} = \varepsilon^{C_p - o(1)}.$$

□

Compressing a sumset The preceding applications used a polynomial whose restriction became diagonal. There is another useful consequence of exactly the same low-degree decomposition: matrices of the form

$$P(s+t)$$

have low ordinary rank. This yields a strong compression theorem for sumsets.

For a prime power q, define

$$M_q(n,d) = |\{\alpha \in \{0,\dots,q-1\}^n : |\alpha| \le d\}|\,.$$

We need one matrix-space lemma.

Lemma 179 (Pivot positions and matrix rank)**.** *Let $\mathcal{V}$ be a linear space of matrices over a field. Order the matrix entries row by row, and choose an echelon basis whose first nonzero entries, or* pivots*, are all distinct.*

If every matrix in $\mathcal{V}$ has rank at most r, then all pivot positions can be covered by at most r rows and columns.

Proof. Form the bipartite graph whose left vertices are matrix rows, whose right vertices are columns, and whose edges are the pivot positions. By König's theorem, it is enough to show that a matching of t pivot positions forces some matrix in $\mathcal{V}$ to have rank at least t.

Let the matched pivots be

$$p_i = (r_i, c_i), \qquad r_1 < \cdots < r_t,$$

and let B_i be the corresponding basis matrix. Scale B_i so that $B_i(p_i) = 1$.

Let $R = \{r_1,\dots,r_t\}$, $C = \{c_1,\dots,c_t\}$, and define

$$Q(\lambda_1,\dots,\lambda_t) = \det\left(\sum_{i=1}^{t} \lambda_i B_i\Big|_{R\times C}\right).$$

We claim

$$[\lambda_1 \cdots \lambda_t]Q = \pm 1. \tag{5}$$

Consider a determinant contribution to this coefficient. For every i, let q_i be the cell from which the factor λ_i is selected. The cells $q_1,\dots,q_t$ form a transversal of the selected rows and columns.

Since p_i is the first nonzero entry of B_i and $B_i(q_i) \neq 0$, we have $q_i \ge p_i$ in row-major order. Hence the row of q_i is at least r_i. But the rows of the q_i's are exactly $\{r_1,\dots,r_t\}$, so equality of their sums forces

$$\mathrm{row}(q_i) = r_i \qquad \text{for every } i.$$

Now q_i and p_i lie in the same row, so $q_i \ge p_i$ implies

$$\mathrm{col}(q_i) \ge c_i.$$

Again the columns of the q_i's are exactly $\{c_1,\dots,c_t\}$, and equality of their sums forces $\mathrm{col}(q_i) = c_i$ for every i.

Thus the only contribution to $\lambda_1\cdots\lambda_t$ uses the matched pivots themselves. Its coefficient is $\pm\prod_i B_i(p_i) = \pm 1$, proving (5).

The determinant polynomial has total degree at most t. Therefore inclusion–exclusion on the Boolean cube gives

$$[\lambda_1 \cdots \lambda_t]Q = \sum_{S\subseteq[t]} (-1)^{t-|S|} Q(1_S).$$

Indeed every monomial missing some variable cancels, while a monomial containing every variable must equal $\lambda_1\cdots\lambda_t$ because its total degree is at most t.

The left side is nonzero, so $Q(1_S) \neq 0$ for some S. Hence $\sum_{i\in S} B_i$ has a nonsingular $t\times t$ minor and therefore rank at least t. □

Define

$$\Lambda_q(n) = \min_{0\le d\le (q-1)n} \left(2M_q\left(n, \left\lfloor \frac{d}{2} \right\rfloor\right) + q^n - M_q(n,d)\right).$$

Theorem 180 (Sumset compression in $\mathbb{F}_q^n$)**.** *For arbitrary $S, T \subseteq \mathbb{F}_q^n$, there are $S_0 \subseteq S$ and $T_0 \subseteq T$ such that*

$$\boxed{|S_0| + |T_0| \le \Lambda_q(n),}$$

and

$$\boxed{S + T = (S_0 + T) \cup (S + T_0).}$$

If

$$D = \left\lfloor \frac{(q-1)n}{3} \right\rfloor,$$

then

$$\boxed{\Lambda_q(n) \le 3M_q(n, D).}$$

Consequently $\Lambda_q(n) \le 3\vartheta_q^n$ for

$$\vartheta_q = \min_{0<\rho<1} (1 + \rho + \cdots + \rho^{q-1})\rho^{-(q-1)/3} < q.$$

Proof. Fix $d \le (q-1)n$, and let $\mathcal{P}_d$ be the space of reduced polynomials on $\mathbb{F}_q^n$ of total degree at most d. Its dimension is $M_q(n,d)$.

Let $V \le \mathcal{P}_d$ consist of polynomials vanishing outside $S + T$. Vanishing at each point outside $S + T$ gives at most one linear condition, so

$$\dim V \ge M_q(n,d) - q^n + |S + T|. \tag{6}$$

For $P \in V$, form the matrix

$$A_P(s,t) = P(s+t), \qquad s \in S,\ t \in T.$$

The map $P \mapsto A_P$ is injective: if $A_P = 0$, then P vanishes on $S + T$, while by definition it vanishes outside $S + T$, so it vanishes on all of $\mathbb{F}_q^n$. A reduced polynomial that vanishes everywhere is zero.

Expand $P(s + t)$. Every term is a monomial in s times a monomial in t, and their two degrees sum to at most d. Hence one side has degree at most $d/2$. Assign each term to such a side and group by its low-degree monomial. Exactly as in the slice-rank argument, this writes A_P as a sum of at most

$$2M_q\left(n, \left\lfloor \frac{d}{2} \right\rfloor\right)$$

rank-one matrices.

Thus every matrix in the image of V has rank at most this number. Choose an echelon basis of that matrix space. By Lemma 179, all of its pivots can be covered by rows $S_1 \subseteq S$ and columns $T_1 \subseteq T$ with

$$|S_1| + |T_1| \le 2M_q\left(n, \left\lfloor \frac{d}{2} \right\rfloor\right). \tag{7}$$

For each $u \in S + T$, let $\phi(u)$ be the lexicographically first pair (s,t) with $s + t = u$. Every pivot is some $\phi(u)$: if a matrix has a nonzero entry at one representation of u, it has the same nonzero entry at every representation of u, so its first such entry is $\phi(u)$.

Therefore distinct pivots correspond to distinct sums. Every pivot covered by a row in S_1 or a column in T_1 corresponds to a sum in

$$(S_1 + T) \cup (S + T_1).$$

Hence this union contains at least $\dim V$ distinct elements of $S + T$.

By (6), at most $q^n - M_q(n,d)$ sums remain uncovered. For each uncovered sum, choose one representation $s+t$ and add its s-coordinate to S_1. This yields S_0, T_0 satisfying

$$S + T = (S_0 + T) \cup (S + T_0)$$

and

$$|S_0| + |T_0| \le 2M_q\left(n, \left\lfloor \frac{d}{2} \right\rfloor\right) + q^n - M_q(n,d).$$

Minimizing over d proves the first bound.

For the simpler bound, put $L = (q-1)n = 3D + r$ with $r \in \{0,1,2\}$. Choose

$$d = \begin{cases} 2D, & r = 0,1, \\ 2D+1, & r = 2. \end{cases}$$

Then $\lfloor d/2 \rfloor = D$.

The involution

$$\alpha_i \longmapsto q - 1 - \alpha_i$$

sends a reduced monomial of degree j to one of degree $L - j$. For the chosen d, every monomial of degree $> d$ is therefore paired with a monomial of degree at most D. Hence

$$q^n - M_q(n,d) \le M_q(n,D),$$

and

$$\Lambda_q(n) \le 3M_q(n,D).$$

The final exponential estimate follows from (16), with p replaced by q. □

Taking $S = T$ gives a useful one-sided form: some $S_0 \subseteq S$ with $|S_0| \le \Lambda_q(n)$ satisfies

$$S_0 + S = S + S.$$

For a three-term-progression-free set in odd characteristic, this gives another proof of an exponential cap-set bound: if $s \in S \setminus S_0$, then $2s \notin S_0 + S$, since an equality $2s = s_0 + s'$ would be a nontrivial three-term progression unless $s_0 = s' = s$. Thus necessarily $S_0 = S$.

5.7 Probabilistic Method

5.7.1 Isolation Lemma

Let

$$\mathcal{F} \subseteq 2^{[n]}$$

be any nonempty family of feasible sets. Assign independently chosen random integer weights

$$w_i \in [M]$$

to the ground elements, and define

$$w(S) = \sum_{i \in S} w_i.$$

Theorem 181 (Isolation Lemma)**.** *With probability at least*

$$\boxed{1 - \frac{n}{M},}$$

there is a unique minimum-weight set in $\mathcal{F}$.

In particular, choosing

$$M = 2n$$

gives uniqueness with probability at least $1/2$.

Proof. For each $i \in [n]$, fix all weights except w_i.

Let

$$A_i = \min_{\substack{S \in \mathcal{F} \\ i \in S}} \sum_{j \in S \setminus \{i\}} w_j$$

and

$$B_i = \min_{\substack{S \in \mathcal{F} \\ i \notin S}} w(S),$$

with the convention that a minimum over an empty family is $+\infty$.

Call i *confused* if

$$A_i + w_i = B_i.$$

Once all the other weights have been fixed, there is at most one value of w_i that can satisfy this equality. Therefore

$$\Pr[i \text{ is confused}] \leq \frac{1}{M}.$$

Now suppose the minimum-weight feasible set is not unique. Let S, T be two distinct minimum-weight members of $\mathcal{F}$, and choose

$$i \in S \triangle T.$$

Assume without loss of generality that

$$i \in S \setminus T.$$

Then

$$A_i + w_i \leq w(S), \qquad B_i \leq w(T).$$

But S and T already have globally minimum weight, so equality must hold:

$$A_i + w_i = B_i.$$

Thus some element is confused.

Hence

$$\Pr[\text{minimum is not unique}] \leq \Pr[\exists i \text{ confused}] \leq \sum_i \frac{1}{M} = \frac{n}{M}.$$

□

5.7.2 Random-Order Independent Sets

An independent set can be obtained by ordering the vertices and excluding the last vertex of every edge. The resulting events are dependent, but their dependence has the right sign: conditioned on one vertex's position, all the requirements for retaining it are increasing events in independent choices.

Lemma 182 (Harris's inequality)**.** *Let $X_1, \ldots, X_m$ be independent Bernoulli variables. If $f, g : \{0,1\}^m \to \mathbb{R}$ are both coordinatewise increasing, or both coordinatewise decreasing, then*

$$\mathbb{E}[fg] \geq \mathbb{E}[f]\mathbb{E}[g].$$

Consequently, for increasing events $E_1, \ldots, E_r$,

$$\Pr(E_1 \cap \cdots \cap E_r) \geq \prod_{i=1}^{r} \Pr(E_i).$$

Proof. Induct on m. Condition on X_m, and write $F_a = \mathbb{E}[f \mid X_m = a]$, $G_a = \mathbb{E}[g \mid X_m = a]$, and $p = \Pr(X_m = 1)$. By induction,

$$\mathbb{E}[fg] \geq (1-p)F_0G_0 + pF_1G_1.$$

The difference between this expression and $\mathbb{E}[f]\mathbb{E}[g]$ is $p(1-p)(F_1 - F_0)(G_1 - G_0)$, which is nonnegative. The case $m = 0$ is immediate.

For events, apply the inequality repeatedly to their indicators. The product of increasing indicators is again increasing. □

Theorem 183 (Weighted nonuniform independence bound)**.** *Let $H = (V, \mathcal{E})$ be a finite hypergraph with no empty edge. There is a random independent set I such that, for every $v \in V$,*

$$\boxed{\Pr(v \in I) \geq \int_0^1 \prod_{\substack{e \in \mathcal{E} \\ v \in e}} \bigl(1 - t^{|e|-1}\bigr)\, dt.}$$

In particular, for arbitrary weights $w_v \geq 0$,

$$\boxed{\alpha_w(H) \geq \sum_{v \in V} w_v \int_0^1 \prod_{\substack{e \in \mathcal{E} \\ v \in e}} \bigl(1 - t^{|e|-1}\bigr)\, dt,}$$

where $\alpha_w(H)$ is the maximum weight of an independent set.

Proof. Give the vertices independent uniform priorities $U_v \in [0,1]$. Retain v precisely when it is not the last vertex of any edge containing it. Every edge loses its last vertex, so the retained set is independent.

Condition on $U_v = t$. For $u \neq v$, the indicators $X_u = \mathbf{1}_{\{U_u > t\}}$ are independent Bernoulli variables. For each incident edge e, retaining v requires

$$\sum_{u \in e \setminus \{v\}} X_u > 0.$$

This is an increasing event of probability $1 - t^{|e|-1}$. Harris's inequality bounds the probability that all these requirements hold by the product of their probabilities. Integrate over t.

Finally, $\alpha_w(H) \geq \mathbb{E} \sum_{v \in I} w_v = \sum_v w_v \Pr(v \in I)$. □

An empty product is one. A singleton edge makes the corresponding factor zero, correctly excluding its vertex.

Corollary 184 (Caro–Tuza)**.** *If H is r-uniform, with $r \geq 2$, then*

$$\boxed{\alpha_w(H) \geq \sum_{v \in V} w_v \binom{d(v) + 1/(r-1)}{d(v)}^{-1}.}$$

Here

$$\binom{d + 1/(r-1)}{d}^{-1} = \prod_{j=1}^{d} \frac{(r-1)j}{(r-1)j + 1},$$

with the empty product equal to one.

Proof. Put $q = r - 1$ and

$$J_d = \int_0^1 (1 - t^q)^d\, dt.$$

Then $J_0 = 1$. For $d \geq 1$, integrate by parts with

$$u = (1-t^q)^d, \qquad dv = dt.$$

Since

$$du = -qd\, t^{q-1}(1-t^q)^{d-1}\, dt, \qquad v = t,$$

and the boundary term $t(1-t^q)^d$ vanishes at both 0 and 1,

$$\begin{aligned} J_d &= qd \int_0^1 t^q (1-t^q)^{d-1}\, dt \\ &= qd \int_0^1 \big(1-(1-t^q)\big)(1-t^q)^{d-1}\, dt \\ &= qd(J_{d-1} - J_d). \end{aligned}$$

Therefore

$$(qd+1)J_d = qdJ_{d-1}, \qquad J_d = \frac{qd}{qd+1} J_{d-1}.$$

Iterating from $J_0 = 1$ gives

$$J_d = \prod_{j=1}^{d} \frac{qj}{qj+1}.$$

Finally,

$$\binom{d+1/q}{d} = \prod_{j=1}^{d} \frac{j+1/q}{j} = \prod_{j=1}^{d} \frac{qj+1}{qj},$$

so

$$J_d = \binom{d+1/q}{d}^{-1} = \binom{d+1/(r-1)}{d}^{-1}.$$

Substitute this value into Theorem 183. □

Corollary 185 (Caro–Wei)**.** *Every finite simple graph satisfies*

$$\boxed{\alpha_w(G) \geq \sum_{v \in V(G)} \frac{w_v}{d(v)+1}.}$$

In particular, if G has n vertices and m edges, then

$$\alpha(G) \geq \sum_v \frac{1}{d(v)+1} \geq \frac{n^2}{n+2m}.$$

Proof. Set $r = 2$. The last inequality is Cauchy–Schwarz. □

Weighted priorities for graphs. For graphs, the priorities can also be chosen to favor particular vertices. Give v an independent exponential priority of rate $a_v > 0$, and retain the local minima. This is an independent set, and

$$\Pr(v \in I) = \int_0^\infty a_v e^{-a_v t} \prod_{u \in N(v)} e^{-a_u t}\, dt = \frac{a_v}{\sum_{u \in N[v]} a_u}.$$

Consequently,

$$\boxed{\alpha_w(G) \geq \sum_v \frac{w_v a_v}{\sum_{u \in N[v]} a_u}.}$$

Taking $a_v = w_v + \varepsilon$ and letting $\varepsilon \downarrow 0$ gives

$$\alpha_w(G) \geq \sum_{v:\, w_v > 0} \frac{w_v^2}{\sum_{u \in N[v]} w_u}.$$

5.7.3 Counting Independent Sets in Regular Graphs

For a graph G, let $\mathcal{I}(G)$ be its independent sets and define

$$Z_G(\lambda) = \sum_{I \in \mathcal{I}(G)} \lambda^{|I|}.$$

Thus $Z_G(1)$ counts independent sets, while other values of λ give different weights to their sizes.

Theorem 186 (Kahn–Zhao)**.** *Let G be a finite simple d-regular graph on n vertices, where $d \geq 1$. For every $\lambda \geq 0$,*

$$\boxed{Z_G(\lambda) \leq \left(2(1+\lambda)^d - 1\right)^{n/(2d)}.}$$

In particular,

$$|\mathcal{I}(G)| \leq (2^{d+1} - 1)^{n/(2d)}.$$

Equality holds for disjoint unions of $K_{d,d}$.

Proof. The case $\lambda = 0$ is immediate, so assume $\lambda > 0$.

Bipartite graphs. Suppose G has parts L, R. Counting edges from either side gives $d|L| = d|R|$, so both parts have size $n/2$.

Fix the part $S \subseteq L$ of an independent set. The vertices that can still be chosen on the right are

$$U(S) = \{v \in R : N(v) \cap S = \varnothing\}.$$

Every subset of $U(S)$ is allowed, since there are no edges within R. Summing the weights of all completions gives

$$\begin{aligned} Z_G(\lambda) &= \sum_{S \subseteq L} \lambda^{|S|} \sum_{T \subseteq U(S)} \lambda^{|T|} \\ &= \sum_{S \subseteq L} \lambda^{|S|} (1+\lambda)^{|U(S)|}. \end{aligned}$$

We now express this sum as an expectation under independent choices. Choose each vertex of L with probability $\lambda/(1+\lambda)$. A particular subset S then has probability

$$\Pr(S) = \left(\frac{\lambda}{1+\lambda}\right)^{|S|} \left(\frac{1}{1+\lambda}\right)^{|L|-|S|} = \frac{\lambda^{|S|}}{(1+\lambda)^{|L|}}.$$

This absorbs the weight of the left part into the sampling distribution. The weight of its right-side completions remains inside the expectation:

$$Z_G(\lambda) = (1+\lambda)^{|L|} \mathbb{E}(1+\lambda)^{|U(S)|}.$$

For $v \in R$, put $f_v(S) = 1 + \lambda \mathbf{1}_{\{v \in U(S)\}}$. Then $(1+\lambda)^{|U(S)|} = \prod_{v \in R} f_v(S)$. The factor f_v depends only on the choices at the d neighbors of v, and each left-vertex choice appears in exactly d factors. By Corollary 207,

$$\mathbb{E} \prod_{v \in R} f_v \leq \prod_{v \in R} \left(\mathbb{E} f_v^d\right)^{1/d}.$$

All neighbors of v are absent with probability $(1+\lambda)^{-d}$. Hence

$$\begin{aligned} \mathbb{E} f_v^d &= \left(1 - (1+\lambda)^{-d}\right) + (1+\lambda)^{-d}(1+\lambda)^d \\ &= 2 - (1+\lambda)^{-d}. \end{aligned}$$

Consequently,

$$\begin{aligned} Z_G(\lambda) &\leq (1+\lambda)^{n/2} \left(2 - (1+\lambda)^{-d}\right)^{n/(2d)} \\ &= \left(2(1+\lambda)^d - 1\right)^{n/(2d)}. \end{aligned}$$

Reduction to the bipartite case. Let $\widetilde{G}$ be the bipartite double cover of G. Its vertices are $V(G) \times \{0,1\}$; every edge uv of G gives the two edges $(u,0)(v,1)$ and $(v,0)(u,1)$. We construct a size-preserving injection

$$\mathcal{I}(G) \times \mathcal{I}(G) \longrightarrow \mathcal{I}(\widetilde{G}).$$

For each $W \subseteq V(G)$ such that $G[W]$ is bipartite, fix a bipartition $W = W_0 \sqcup W_1$, depending only on W. Given independent sets A, B, their union $W = A \cup B$ induces a bipartite graph: every edge joins $A \setminus B$ to $B \setminus A$, and vertices of $A \cap B$ are isolated.

Swap the two memberships at every vertex of W_1:

$$A' = (A \cap W_0) \cup (B \cap W_1), \qquad B' = (B \cap W_0) \cup (A \cap W_1).$$

Each edge of $G[W]$ originally has one endpoint in $A \setminus B$ and the other in $B \setminus A$. Exactly one endpoint is swapped, so afterward both endpoints belong to the same one of A', B'. Thus no edge joins A' to B', and

$$(A' \times \{0\}) \cup (B' \times \{1\})$$

is independent in $\widetilde{G}$.

The union remains W, so the output determines the fixed bipartition used in the construction. Swapping again recovers A, B, proving injectivity. The swap also preserves $|A| + |B|$, and therefore preserves weights. Summing over all input pairs gives

$$\boxed{Z_G(\lambda)^2 \le Z_{\widetilde{G}}(\lambda).}$$

The double cover is d-regular on $2n$ vertices. Apply the bipartite bound and take square roots.

Finally, every independent set of $K_{d,d}$ lies entirely in one side. Each side contributes $(1+\lambda)^d$, with the empty set counted twice, so

$$Z_{K_{d,d}}(\lambda) = 2(1+\lambda)^d - 1.$$

Independence polynomials multiply over disjoint unions, proving the equality statement. □

5.7.4 Triangle-Free Independent Sets and Ramsey Numbers

For lower bounds, we sample an independent set itself, with probability proportional to its weight:

$$\Pr_\lambda(I) = \frac{\lambda^{|I|}}{Z_G(\lambda)}, \qquad a_G(\lambda) = \frac{1}{n}\mathbb{E}_\lambda|I|, \qquad \lambda > 0.$$

Triangle-freeness will let us describe the conditional distribution inside a neighborhood exactly. The same calculation relates the probability of selecting its center to the expected number of selected neighbors.

Available vertices. Call v *available* when $N(v) \cap I = \varnothing$. Condition on the choices at all vertices except v. If v is unavailable, it is forced to be absent. Otherwise both choices are allowed, with respective weights 1 and λ, so its conditional inclusion probability is $\lambda/(1+\lambda)$. Averaging gives

$$\boxed{\Pr_\lambda(v \in I) = \frac{\lambda}{1+\lambda}\Pr_\lambda(v \text{ is available}).} \tag{17}$$

For $x \ge 0$, let $W(x)$ be the unique nonnegative solution of $we^w = x$. Uniqueness follows because $w \mapsto we^w$ is strictly increasing on $[0,\infty)$.

Theorem 187 (Triangle-free occupancy bound)**.** *Let G be a finite simple triangle-free graph on $n \ge 1$ vertices, with maximum degree at most d, where $d \ge 1$. For every $\lambda > 0$,*

$$\boxed{a_G(\lambda) \ge \frac{\lambda}{1+\lambda}\frac{W(d\ln(1+\lambda))}{d\ln(1+\lambda)}.}$$

Proof. Put $r = \lambda/(1+\lambda)$ and $L = \ln(1+\lambda)$.

Fix a vertex v, and expose $J = I \cap (V \setminus N(v))$. This exposes v itself but leaves its neighbors unexposed. Since G is triangle-free, $N(v)$ is independent, and every neighbor of a vertex $u \in N(v)$ lies outside $N(v)$. Thus J determines which vertices in $N(v)$ are available:

$$A_v(J) = \{u \in N(v) : N(u) \cap J = \varnothing\}.$$

Write $U_v = |A_v(J)|$.

The possible completions are exactly $I = J \cup T$ with $T \subseteq A_v(J)$. Their weights are $\lambda^{|J|}\lambda^{|T|}$, so, conditionally on J,

$$\Pr_\lambda(I \cap N(v) = T \mid J) = \frac{\lambda^{|T|}}{(1+\lambda)^{U_v}} \qquad (T \subseteq A_v(J)).$$

In particular, the available neighbors are selected independently with probability r.

The center v is available exactly when $T = \varnothing$. The conditional probability of this event is $(1+\lambda)^{-U_v}$, while the conditional expected number of selected neighbors is rU_v. Averaging over J,

$$\Pr_\lambda(v \text{ is available}) = \mathbb{E}_\lambda e^{-LU_v}, \qquad \mathbb{E}_\lambda|I \cap N(v)| = r\,\mathbb{E}_\lambda U_v.$$

Now average also over a uniformly chosen vertex v, independently of I. Write $U = U_v$ and $a = a_G(\lambda)$. The first identity and (17) give

$$\frac{a}{r} = \mathbb{E}e^{-LU} \geq e^{-L\mathbb{E}U},$$

by Jensen's inequality. The second identity gives

$$\begin{aligned} r\,\mathbb{E}U &= \frac{1}{n}\sum_v \mathbb{E}_\lambda|I \cap N(v)| \\ &= \frac{1}{n}\sum_u d(u)\Pr_\lambda(u \in I) \leq da. \end{aligned}$$

Combining the two bounds,

$$\frac{a}{r} \geq e^{-dLa/r}.$$

Set $z = dLa/r$. The inequality becomes $ze^z \geq dL$, so $z \geq W(dL)$. Rearranging proves the theorem. □

Expected size and the independence polynomial. Differentiating the finite sums defining Z_G and $\mathbb{E}_\lambda|I|$ gives

$$\begin{aligned} \frac{d}{d\lambda}\ln Z_G(\lambda) &= \frac{\mathbb{E}_\lambda|I|}{\lambda} = \frac{n\,a_G(\lambda)}{\lambda}, \\ \frac{d}{d\lambda}\mathbb{E}_\lambda|I| &= \frac{\mathbb{E}_\lambda|I|^2 - (\mathbb{E}_\lambda|I|)^2}{\lambda} = \frac{\mathrm{Var}_\lambda(|I|)}{\lambda} \geq 0. \end{aligned} \tag{18}$$

Thus increasing λ increases the expected size, and integrating a lower bound on that expectation gives a lower bound on Z_G.

Corollary 188 (Asymptotic independence bound)**.** *As $d \to \infty$, every finite simple triangle-free graph on n vertices with maximum degree at most d satisfies*

$$\boxed{\alpha(G) \geq \mathbb{E}_{I \text{ uniform in } \mathcal{I}(G)}|I| \geq (1-o(1))\frac{n\ln d}{d}.}$$

Proof. For $x \geq e$, the identity $W(x) + \ln W(x) = \ln x$ gives

$$\ln x - \ln\ln x \leq W(x) \leq \ln x.$$

Indeed, $W(x) \geq 1$, so first $W(x) \leq \ln x$; substituting this upper bound into the logarithm gives the lower bound.

Choose $\lambda = 1/\ln d$. With $r = \lambda/(1+\lambda)$ and $L = \ln(1+\lambda)$,

$$\frac{r}{L} = 1 - O\left(\frac{1}{\ln d}\right), \qquad dL = (1+o(1))\frac{d}{\ln d}.$$

Hence $W(dL) \geq \ln d - 2\ln\ln d - O(1)$, and the occupancy bound yields

$$a_G(\lambda) \geq \frac{\ln d - 2\ln\ln d - O(1)}{d} = (1-o(1))\frac{\ln d}{d}.$$

For sufficiently large d, $\lambda \leq 1$. By (18), the expectation at $\lambda = 1$, which is the uniform distribution on independent sets, is at least the expectation at this λ. Finally, the maximum size is at least the expected size. □

Corollary 189 (Triangle Ramsey upper bound)**.** *As $t \to \infty$,*

$$\boxed{R(3,t) \leq (1+o(1))\frac{t^2}{\ln t}.}$$

Proof. Let G be a triangle-free graph with no independent set of size t. Every neighborhood is independent, so $\Delta(G) \leq t-1$. Applying Corollary 188 with degree bound $d = t$,

$$t > \alpha(G) \geq (1-o(1))\frac{|V(G)|\ln t}{t}.$$

Thus $|V(G)| \leq (1+o(1))t^2/\ln t$. Any graph with more vertices must contain a triangle or an independent set of size t, proving the claim. □

Corollary 190 (Independent-set count in triangle-free graphs)**.** *Under the hypotheses of Theorem 187, put $w = W(d\ln(1+\lambda))$. Then*

$$\boxed{Z_G(\lambda) \geq \exp\left(\frac{n}{2d}(w^2+2w)\right).}$$

In particular,

$$|\mathcal{I}(G)| \geq \exp\left(\left(\frac{1}{2} - o(1)\right)\frac{n\ln^2 d}{d}\right) \qquad (d \to \infty).$$

Proof. Use (18), $Z_G(0) = 1$, and the occupancy bound:

$$\begin{aligned}
\ln Z_G(\lambda) &= n\int_0^\lambda \frac{a_G(t)}{t}\,dt \\
&\geq \frac{n}{d}\int_0^\lambda \frac{W(d\ln(1+t))}{(1+t)\ln(1+t)}\,dt \\
&= \frac{n}{d}\int_0^{d\ln(1+\lambda)} \frac{W(x)}{x}\,dx,
\end{aligned}$$

where the last step substitutes $x = d\ln(1+t)$. The integrands at zero are interpreted by their continuous limits.

Now substitute $u = W(x)$, so $x = ue^u$ and $dx/x = (1+u)\,du/u$. Therefore

$$\ln Z_G(\lambda) \geq \frac{n}{d}\int_0^w (1+u)\,du = \frac{n}{2d}(w^2+2w).$$

Exponentiating proves the first bound. At $\lambda = 1$, $w = (1-o(1))\ln d$, giving the second. □

5.7.5 Moser–Tardos Resampling

Suppose an object is described by independent random choices, and we want none of a given collection of bad events to occur. Resampling the whole object may discard almost everything that was already correct. Instead, repair a violated event by resampling only the variables on which it depends. This can create other violations, but only nearby ones. The Moser–Tardos theorem shows that sufficiently rare, sparsely dependent bad events cannot sustain this chain of repairs for very long.

Let $X_1, \dots, X_N$ be mutually independent random variables and let $\mathcal{A}$ be a finite collection of bad events. Each $A \in \mathcal{A}$ is determined by a specified set of variable indices $\mathrm{vbl}(A)$. Write $p_A = \Pr[A]$ for its probability under the original product distribution, and define

$$\Gamma(A) = \{B \in \mathcal{A} \setminus \{A\} : \mathrm{vbl}(A) \cap \mathrm{vbl}(B) \neq \emptyset\}, \qquad \Gamma^+(A) = \Gamma(A) \cup \{A\}.$$

The open neighborhood excludes A; the closed neighborhood includes it.

Repair only what changed. Maintain the set Q of currently violated events. Resampling A can change the truth value only of events in $\Gamma^+(A)$, so only these need to be checked again. The dependency lists are assumed available.

Algorithm 27 Moser–Tardos resampling

Input: Independent-variable samplers, bad-event tests, and dependency lists
Output: An assignment in which no bad event occurs, upon termination

```
Sample X_1, ..., X_N independently
Q ← {A ∈ 𝒜 : A occurs}
while Q ≠ ∅ do
    Choose any A ∈ Q
    for each i ∈ vbl(A) do
        Replace X_i by a fresh independent sample from its distribution
    for each B ∈ Γ⁺(A) do
        if B occurs then
            Insert B into Q
        else
            Remove B from Q if present
return (X_1, ..., X_N)
```

Theorem 191 (Moser–Tardos)**.** *Suppose there are numbers $x_A \in (0,1)$ satisfying*

$$\boxed{p_A \le x_A \prod_{B \in \Gamma(A)} (1 - x_B) \qquad (A \in \mathcal{A}).}$$

Then Algorithm 27 terminates with probability one. If N_A is the number of times it resamples A, then

$$\boxed{\mathbb{E}N_A \le \frac{x_A}{1 - x_A}, \qquad \mathbb{E}\left[\sum_{A \in \mathcal{A}} N_A\right] \le \sum_{A \in \mathcal{A}} \frac{x_A}{1 - x_A}.}$$

Any rule for choosing a currently violated event is allowed, provided it does not inspect unused random samples. The numbers x_A certify the guarantee but are not needed by the algorithm.

The assignment at a resampling time is conditioned on previous repairs, so we cannot simply charge p_A for each occurrence of A in the log. Instead, encode a resampling by the earlier repairs relevant to it. This certificate will impose bad-event tests on disjoint random samples.

Witness tree. Let $A_1, A_2, \ldots$ be the resampling log, listing an event immediately before its variables are resampled. For step t, start a tree T_t with root labelled A_t. Scan $A_{t-1}, \ldots, A_1$ backward. If the current event is in the closed neighborhood of any label already in the tree, attach a new node beneath a *deepest* such node; otherwise skip it. Fix any rule for breaking depth ties.

Every child label belongs to its parent's closed neighborhood. More importantly, if two tree nodes use a common variable, the earlier resampling has greater depth: when that earlier node is inserted, the later node is already present, so the chosen parent is at least as deep. Consequently nodes using a common variable have distinct depths. The same argument for equal labels shows that siblings have distinct labels. Call a rooted labelled tree *proper* if every child label is in its parent's closed neighborhood and siblings have distinct labels. Thus every tree produced by the log is proper.

Lemma 192 (Witness-tree probability)**.** *For a fixed finite labelled tree T, the probability that $T = T_t$ for some resampling step t is at most*

$$\prod_{v \in V(T)} p_{\ell(v)},$$

where $\ell(v)$ denotes the event labelling node v.

Proof. Presample an infinite table $R(i,0), R(i,1), \ldots$ for each variable X_i, with all table entries independent and each $R(i,j)$ distributed as X_i. Initialize $X_i = R(i,0)$; each resampling of X_i advances its index by one. Generating entries only when needed has exactly the same law, so these tables describe the algorithm's randomness.

If two nodes of T use the same variable at the same depth, the tree cannot occur. Otherwise, for a node v and $i \in \mathrm{vbl}(\ell(v))$, put

$$k_T(v,i) = \#\{w \in V(T) : \mathrm{depth}(w) > \mathrm{depth}(v),\ i \in \mathrm{vbl}(\ell(w))\}.$$

Test the event $\ell(v)$ using the values $R(i, k_T(v,i))$ for its variables. For each fixed i, distinct depths give distinct indices, so different nodes use disjoint table entries. The tests are therefore independent, and all pass with probability $\prod_v p_{\ell(v)}$.

It remains to show that occurrence of T forces all these tests to pass. Suppose node v came from step s of the log. Every earlier resampling of a variable used by $\ell(v)$ is included in the tree: the backward scan already contains v when it reaches that resampling. Among nodes using X_i, chronological order is the reverse of depth order. Thus $k_T(v,i)$ is exactly the number of times X_i was resampled before step s. The table value used in the test is consequently the value present when the algorithm selected $\ell(v)$, and that event was violated. This holds for every node, proving the probability bound. □

The table is read in *decreasing depth*, not from the root downward. Nodes in different branches may share variables; the table index counts all deeper nodes using that variable, not merely descendants.

Sum all certificates. For an isolated event, proper witness trees are chains, representing repeated failures of the same test. With dependencies, each node may also have one child for each neighboring label. The factor $\prod_{B \in \Gamma(A)}(1 - x_B)$ will pay for these possible branches.

Lemma 193 (Total witness weight)**.** *Under the theorem's hypothesis, the total weight of all finite proper trees rooted at A satisfies*

$$\sum_{T \text{ proper, rooted at } A} \prod_{v \in V(T)} p_{\ell(v)} \le \frac{x_A}{1 - x_A}.$$

Proof. Let $W_h(A)$ be the same sum restricted to trees with at most h levels, so $W_0(A) = 0$. Children are not ordered, and at most one has any given label. For each $B \in \Gamma^+(A)$, either omit that child, contributing 1, or choose its subtree, contributing $W_h(B)$. Therefore

$$W_{h+1}(A) = p_A \prod_{B \in \Gamma^+(A)} (1 + W_h(B)).$$

Put $K_A = x_A/(1-x_A)$, so $1 + K_A = 1/(1-x_A)$. If $W_h(B) \le K_B$ for all B, the hypothesis gives

$$W_{h+1}(A) \le p_A \prod_{B \in \Gamma^+(A)} \frac{1}{1-x_B} \le \frac{x_A}{1-x_A} = K_A.$$

Induction proves this for every h. Every finite proper tree has finite height, so taking the increasing limit proves the claim. □

Proof of Theorem 191. Different resamplings of the same event produce different witness trees. Indeed, the tree of the jth resampling of A contains exactly j nodes labelled A: every earlier occurrence of A is included because it is in the root's closed neighborhood. Hence, on each execution,

$$N_A \le \sum_{T \text{ proper, rooted at } A} \mathbf{1}\{T \text{ occurs in the log}\}.$$

Take expectations and apply the two lemmas. Nonnegative sums allow passage from finite truncations to the entire log, even before termination has been established. We obtain $\mathbb{E}N_A \le x_A/(1-x_A)$ and the stated bound on total resamplings. A nonnegative integer-valued count with finite expectation is finite with probability one. Upon termination Q is empty, so no bad event occurs. □

The symmetric criterion.

Corollary 194 (Symmetric algorithmic local lemma)**.** *Suppose every event has probability at most p and overlaps at most $D \ge 1$ other events. If $ep(D+1) \le 1$, then the algorithm terminates with probability one and uses at most $|\mathcal{A}|/D$ expected resamplings.*

Proof. Choose $x_A = 1/(D+1)$ for every A. Since $(1+1/D)^D \le e$,

$$x_A \prod_{B \in \Gamma(A)} (1-x_B) \ge \frac{1}{D+1}\left(\frac{D}{D+1}\right)^D \ge \frac{1}{e(D+1)} \ge p.$$

The theorem applies, and $x_A/(1-x_A) = 1/D$. □

5.8 Concentration Inequalities

Most concentration inequalities come from the same idea:

$$\Pr[X \ge t] = \Pr[e^{\theta X} \ge e^{\theta t}] \le e^{-\theta t}\mathbb{E}e^{\theta X},$$

followed by a bound on the moment generating function.

5.8.1 Hoeffding

Lemma 195 (Hoeffding's Lemma)**.** *If $\mathbb{E}X = 0$ and $a \le X \le b$, then for every $\theta \in \mathbb{R}$,*

$$\boxed{\mathbb{E}e^{\theta X} \le \exp\left(\frac{\theta^2(b-a)^2}{8}\right).}$$

Proof. Let

$$\psi(\theta) = \log \mathbb{E}e^{\theta X}$$

be the cumulant generating function. Define the modified expectation

$$\mathbb{E}_\theta[g(X)] = \frac{\mathbb{E}[g(X)e^{\theta X}]}{\mathbb{E}e^{\theta X}}.$$

Differentiating gives

$$\psi'(\theta) = \mathbb{E}_\theta X$$

and

$$\boxed{\psi''(\theta) = \mathbb{E}_\theta X^2 - (\mathbb{E}_\theta X)^2 = \mathrm{Var}_\theta(X).}$$

The modification changes the distribution of X, but not its support, so under the modified distribution we still have $a \le X \le b$. Any random variable $Y \in [a, b]$ satisfies

$$\mathrm{Var}(Y) = \min_c \mathbb{E}(Y - c)^2 \le \mathbb{E}\left(Y - \frac{a+b}{2}\right)^2 \le \frac{(b-a)^2}{4}.$$

Therefore, for every θ,

$$\psi''(\theta) \le \frac{(b-a)^2}{4}.$$

Finally,

$$\psi(0) = 0, \qquad \psi'(0) = \mathbb{E}X = 0.$$

By Taylor's theorem, for some ξ between 0 and θ,

$$\psi(\theta) = \psi(0) + \theta\psi'(0) + \frac{\theta^2}{2}\psi''(\xi).$$

Therefore

$$\psi(\theta) = \frac{\theta^2}{2}\psi''(\xi) \le \frac{\theta^2}{2} \cdot \frac{(b-a)^2}{4} = \frac{\theta^2(b-a)^2}{8}.$$

Exponentiating gives

$$\mathbb{E}e^{\theta X} \le \exp\left(\frac{\theta^2(b-a)^2}{8}\right).$$

□

Theorem 196 (Hoeffding's Inequality)**.** *Let $X_1, \ldots, X_n$ be independent with*

$$a_i \le X_i \le b_i.$$

Then

$$\boxed{\Pr\left[\sum_i (X_i - \mathbb{E}X_i) \ge t\right] \le \exp\left(-\frac{2t^2}{\sum_i (b_i - a_i)^2}\right).}$$

The same bound holds for the lower tail.

Proof. Apply Lemma 195 independently:

$$\mathbb{E}e^{\theta \sum_i (X_i - \mathbb{E}X_i)} \le \exp\left(\frac{\theta^2}{8}\sum_i (b_i - a_i)^2\right).$$

The exponential-moment bound gives

$$\Pr[\cdots] \le \exp\left(-\theta t + \frac{\theta^2}{8}\sum_i (b_i - a_i)^2\right).$$

Optimize over θ. □

5.8.2 Chernoff

Theorem 197 (Chernoff Bounds)**.** *Let*

$$X = \sum_i X_i$$

where the X_i are independent Bernoulli variables, and let

$$\mu = \mathbb{E}X.$$

Then for $\delta \ge 0$,

$$\boxed{\Pr[X \ge (1+\delta)\mu] \le \left(\frac{e^\delta}{(1+\delta)^{1+\delta}}\right)^\mu \le \exp\left(-\frac{\delta^2}{2+\delta}\mu\right).}$$

For $0 \le \delta \le 1$,

$$\boxed{\Pr[X \le (1-\delta)\mu] \le e^{-\delta^2\mu/2}.}$$

Proof. For a Bernoulli variable of mean p,

$$\mathbb{E}e^{\theta X_i} = 1 + p(e^\theta - 1) \le \exp(p(e^\theta - 1)).$$

Hence

$$\mathbb{E}e^{\theta X} \le \exp(\mu(e^\theta - 1)).$$

Therefore

$$\Pr[X \ge (1+\delta)\mu] \le \exp(-\theta(1+\delta)\mu + \mu(e^\theta - 1)).$$

Set

$$e^\theta = 1 + \delta.$$

This gives the exact first bound. The simpler forms follow from elementary calculus.

The lower tail is identical with $\theta < 0$. □

5.8.3 Bernstein

Lemma 198 (Bernstein MGF Bound)**.** *If*

$$\mathbb{E}X = 0, \qquad |X| \le R,$$

then for

$$0 \le \theta < 3/R,$$

$$\boxed{\log \mathbb{E}e^{\theta X} \le \frac{\theta^2 \mathbb{E}X^2}{2(1 - \theta R/3)}.}$$

Proof. For $k \geq 2$,

$$k! \geq 2 \cdot 3^{k-2},$$

and

$$|X|^k \leq R^{k-2} X^2.$$

Thus

$$\begin{aligned} e^{\theta X} &\leq 1 + \theta X + X^2 \sum_{k \geq 2} \frac{\theta^k R^{k-2}}{k!} \\ &\leq 1 + \theta X + \frac{\theta^2 X^2}{2(1 - \theta R/3)}. \end{aligned}$$

Take expectations, use $\mathbb{E}X = 0$, and use

$$1 + x \leq e^x.$$

□

Theorem 199 (Bernstein's Inequality)**.** *Let $X_1, \ldots, X_n$ be independent, mean zero, and satisfy*

$$|X_i| \leq R.$$

Set

$$\sigma^2 = \sum_i \mathbb{E}X_i^2.$$

Then

$$\boxed{\Pr\left[\sum_i X_i \geq t\right] \leq \exp\left(-\frac{t^2}{2(\sigma^2 + Rt/3)}\right).}$$

Proof. By Lemma 198,

$$\Pr\left[\sum_i X_i \geq t\right] \leq \exp\left(-\theta t + \frac{\theta^2 \sigma^2}{2(1 - \theta R/3)}\right).$$

Take

$$\theta = \frac{t}{\sigma^2 + Rt/3}.$$

□

5.9 Martingales

The exponential-moment method extends almost unchanged to martingales. The main difference is that moment-generating-function bounds are applied conditionally at each step.

Lemma 200 (Bounded Optional Stopping)**.** *Let $(M_k)_{k=0}^n$ be a supermartingale and let $\tau \leq n$ be a stopping time. Then*

$$\boxed{\mathbb{E}M_\tau \leq \mathbb{E}M_0.}$$

For a martingale, equality holds.

Proof. Write

$$M_\tau - M_0 = \sum_{i=1}^n \mathbf{1}_{\{\tau \geq i\}}(M_i - M_{i-1}).$$

Since

$$\{\tau \geq i\} \in \mathcal{F}_{i-1},$$

we have

$$\begin{aligned}\mathbb{E}[M_\tau - M_0] &= \sum_{i=1}^{n} \mathbb{E}\left[\mathbf{1}_{\{\tau \geq i\}} \mathbb{E}[M_i - M_{i-1} \mid \mathcal{F}_{i-1}]\right] \\ &\leq 0.\end{aligned}$$

For a martingale, every term is zero. □

A useful consequence is the martingale analogue of the maximal form of Markov's inequality.

Lemma 201 (Ville's Inequality)**.** *Let $(M_k)_{k=0}^n$ be a nonnegative supermartingale. Then for every $a > 0$,*

$$\boxed{\Pr\left[\max_{0 \leq k \leq n} M_k \geq a\right] \leq \frac{\mathbb{E}M_0}{a}.}$$

Proof. Let τ be the first k such that $M_k \geq a$, taking $\tau = n$ if this never occurs. By bounded optional stopping,

$$\mathbb{E}M_\tau \leq \mathbb{E}M_0.$$

On the event

$$\max_{k \leq n} M_k \geq a$$

we have $M_\tau \geq a$, while $M_\tau \geq 0$ always. Hence

$$a \Pr\left[\max_{k \leq n} M_k \geq a\right] \leq \mathbb{E}M_\tau \leq \mathbb{E}M_0.$$

□

5.9.1 Azuma and McDiarmid

Azuma's inequality is Hoeffding's inequality with independence replaced by the martingale property.

Theorem 202 (Maximal Azuma–Hoeffding)**.** *Let*

$$Y_k = \sum_{i=1}^{k} X_i, \qquad Y_0 = 0,$$

where

$$\mathbb{E}[X_i \mid \mathcal{F}_{i-1}] = 0$$

and

$$a_i \leq X_i \leq b_i.$$

Then

$$\boxed{\Pr\left[\max_{0 \leq k \leq n} Y_k \geq t\right] \leq \exp\left(-\frac{2t^2}{\sum_{i=1}^n (b_i - a_i)^2}\right).}$$

The same bound holds for

$$\Pr\left[\min_{0 \leq k \leq n} Y_k \leq -t\right].$$

Consequently,

$$\Pr\left[\max_{0 \leq k \leq n} |Y_k| \geq t\right] \leq 2\exp\left(-\frac{2t^2}{\sum_{i=1}^n (b_i - a_i)^2}\right).$$

Proof. Set

$$c_i = b_i - a_i.$$

Conditionally on $\mathcal{F}_{i-1}$, Hoeffding's lemma gives

$$\mathbb{E}[e^{\theta X_i} \mid \mathcal{F}_{i-1}] \le e^{\theta^2 c_i^2/8}.$$

Therefore

$$M_k = \exp\left(\theta Y_k - \frac{\theta^2}{8}\sum_{i=1}^{k} c_i^2\right)$$

is a nonnegative supermartingale with $M_0 = 1$.

If $Y_k \ge t$ for some $k \le n$, then

$$M_k \ge \exp\left(\theta t - \frac{\theta^2}{8}\sum_{i=1}^{n} c_i^2\right).$$

Thus Ville's inequality gives

$$\Pr\left[\max_{k\le n} Y_k \ge t\right] \le \exp\left(-\theta t + \frac{\theta^2}{8}\sum_{i=1}^{n} c_i^2\right).$$

Optimizing at

$$\theta = \frac{4t}{\sum_i c_i^2}$$

gives the claimed bound.

Apply the same argument to $-Y_k$ for the lower tail, and union bound the two tails for the final statement. □

Theorem 203 (McDiarmid's Inequality)**.** *Let $Z_1, \dots, Z_n$ be independent, and suppose*

$$|f(z) - f(z')| \le c_i$$

whenever z, z' differ only in coordinate i. Then

$$\boxed{\Pr[f(Z) - \mathbb{E}f(Z) \ge t] \le \exp\left(-\frac{2t^2}{\sum_i c_i^2}\right).}$$

Proof. Consider the Doob martingale

$$M_i = \mathbb{E}[f(Z) \mid Z_1, \dots, Z_i].$$

Fixing $Z_1, \dots, Z_{i-1}$, regard M_i as a function of Z_i. For any two possible values z_i, z_i',

$$|\mathbb{E}[f(Z) \mid Z_i = z_i, Z_1, \dots, Z_{i-1}] - \mathbb{E}[f(Z) \mid Z_i = z_i', Z_1, \dots, Z_{i-1}]| \le c_i,$$

because the two expectations can be coupled using the same values of $Z_{i+1}, \dots, Z_n$.

Thus, conditioned on $\mathcal{F}_{i-1}$, M_i ranges over an interval of length at most c_i. Since

$$M_{i-1} = \mathbb{E}[M_i \mid \mathcal{F}_{i-1}],$$

the martingale difference $M_i - M_{i-1}$ also lies in an interval of length at most c_i.

Now

$$M_n - M_0 = f(Z) - \mathbb{E}f(Z),$$

so Theorem 202 gives the result. □

5.9.2 Freedman's Inequality

Azuma only uses a worst-case bound on each martingale increment. When the increments usually have much smaller conditional variance, Freedman's inequality gives the martingale analogue of Bernstein's inequality.

Let

$$Y_k = \sum_{i=1}^{k} X_i, \qquad V_k = \sum_{i=1}^{k} \mathbb{E}[X_i^2 \mid \mathcal{F}_{i-1}],$$

where

$$\mathbb{E}[X_i \mid \mathcal{F}_{i-1}] = 0.$$

The quantity V_k is the accumulated conditional variance.

We first record the conditional version of the Bernstein MGF bound.

Lemma 204 (Conditional Bernstein MGF Bound)**.** *Suppose*

$$\mathbb{E}[X \mid \mathcal{F}] = 0, \qquad X \le R$$

almost surely, where $R > 0$. Then for

$$0 \le \theta < 3/R,$$

$$\boxed{\mathbb{E}[e^{\theta X} \mid \mathcal{F}] \le \exp\left(\frac{\theta^2 \mathbb{E}[X^2 \mid \mathcal{F}]}{2(1 - \theta R/3)}\right).}$$

Proof. For $\theta \ge 0$ and $x \le R$,

$$e^{\theta x} \le 1 + \theta x + \frac{e^{\theta R} - \theta R - 1}{R^2} x^2.$$

Taking conditional expectation and using

$$\mathbb{E}[X \mid \mathcal{F}] = 0$$

gives

$$\mathbb{E}[e^{\theta X} \mid \mathcal{F}] \le 1 + \frac{e^{\theta R} - \theta R - 1}{R^2} \mathbb{E}[X^2 \mid \mathcal{F}].$$

Since

$$1 + x \le e^x$$

and, for $\theta R < 3$,

$$e^{\theta R} - \theta R - 1 \le \frac{(\theta R)^2}{2(1 - \theta R/3)},$$

the result follows. □

Theorem 205 (Freedman's Inequality)**.** *Suppose*

$$\mathbb{E}[X_i \mid \mathcal{F}_{i-1}] = 0$$

and

$$X_i \le R$$

almost surely. Then

$$\boxed{\Pr\left[\exists k \le n : Y_k \ge t \text{ and } V_k \le \sigma^2\right] \le \exp\left(-\frac{t^2}{2(\sigma^2 + Rt/3)}\right).}$$

Proof. Fix

$$0 \le \theta < 3/R$$

and write

$$g(\theta) = \frac{\theta^2}{2(1 - \theta R/3)}.$$

By Lemma 204,

$$\mathbb{E}[e^{\theta X_i} \mid \mathcal{F}_{i-1}] \le \exp\left(g(\theta)\mathbb{E}[X_i^2 \mid \mathcal{F}_{i-1}]\right).$$

Hence

$$M_k = \exp\left(\theta Y_k - g(\theta)V_k\right)$$

is a nonnegative supermartingale with $M_0 = 1$.

If for some $k \le n$,

$$Y_k \ge t, \qquad V_k \le \sigma^2,$$

then

$$M_k \ge \exp\left(\theta t - g(\theta)\sigma^2\right).$$

Ville's inequality therefore gives

$$\Pr\left[\exists k \le n : Y_k \ge t,\ V_k \le \sigma^2\right] \le \exp\left(-\theta t + g(\theta)\sigma^2\right).$$

Take

$$\theta = \frac{t}{\sigma^2 + Rt/3}.$$

When $\sigma^2 > 0$, this satisfies $\theta R < 3$, and

$$1 - \frac{\theta R}{3} = \frac{\sigma^2}{\sigma^2 + Rt/3}.$$

Therefore

$$\begin{aligned} -\theta t + g(\theta)\sigma^2 &= -\frac{t^2}{\sigma^2 + Rt/3} + \frac{t^2}{2(\sigma^2 + Rt/3)} \\ &= -\frac{t^2}{2(\sigma^2 + Rt/3)}. \end{aligned}$$

The case $\sigma^2 = 0$ follows by taking a limit. □

Thus Freedman has the same basic form as Bernstein:

$$\exp\left(-\Theta\left(\min\left\{\frac{t^2}{\sigma^2}, \frac{t}{R}\right\}\right)\right),$$

but it simultaneously controls every time k at which the accumulated conditional variance is at most σ^2.

If also

$$X_i \ge -R,$$

then applying the theorem to $-Y_k$ gives the corresponding lower-tail bound.

5.9.3 Generalized Hölder and Read-k Concentration

Functions of independent variables need not themselves be independent. When each underlying variable appears in only a bounded number of functions, Hölder's inequality controls the resulting dependence.

Theorem 206 (Generalized Hölder inequality)**.** *Let $X_1, \dots, X_m$ be independent random variables, and let $f_i = f_i(X_{S_i}) \geq 0$, where $S_i \subseteq [m]$. Suppose $0 < \theta_i \leq 1$ and*

$$\sum_{i:\, j \in S_i} \theta_i \leq 1 \qquad (j \in [m]).$$

Then

$$\boxed{\mathbb{E} \prod_i f_i^{\theta_i} \leq \prod_i (\mathbb{E} f_i)^{\theta_i}.}$$

Proof. Induct on the number of underlying variables. Fix all variables except X_m. Apply Hölder with exponents $1/\theta_i$ to the factors whose sets S_i contain m, adding a constant factor one if their weights sum to less than one. This gives

$$\mathbb{E}_{X_m} \prod_i f_i^{\theta_i} \leq \prod_{i:\, m \notin S_i} f_i^{\theta_i} \prod_{i:\, m \in S_i} (\mathbb{E}_{X_m} f_i)^{\theta_i}.$$

The new factors depend only on the remaining variables and satisfy the same weight constraints. Apply induction and use independence to identify the iterated expectations with $\mathbb{E} f_i$. The case with no underlying variables is equality. □

Corollary 207 (Bounded variable reuse)**.** *Suppose $f_1, \dots, f_r \geq 0$ are functions of independent variables, and each underlying variable occurs in at most k functions. Then*

$$\boxed{\mathbb{E} \prod_{i=1}^{r} f_i \leq \prod_{i=1}^{r} \left(\mathbb{E} f_i^k\right)^{1/k}.}$$

Proof. Apply Theorem 206 to f_i^k with $\theta_i = 1/k$. □

Concentration. A family $Y_1, \dots, Y_r$ is *read-k* if each Y_i is a function of independent underlying variables and each underlying variable appears in at most k of these functions.

For $a, p \in [0, 1]$, write

$$D(a \| p) = a \ln \frac{a}{p} + (1 - a) \ln \frac{1 - a}{1 - p},$$

with the usual limiting conventions.

Theorem 208 (Read-k Chernoff bound)**.** *Let $Y_1, \dots, Y_r \in [0, 1]$ form a read-k family, and put $S = \sum_i Y_i$ and $p = \mathbb{E} S / r$. Then*

$$\boxed{\Pr(S \geq ra) \leq \exp\left(-\frac{r}{k} D(a \| p)\right) \qquad (p \leq a \leq 1),}$$

and

$$\boxed{\Pr(S \leq ra) \leq \exp\left(-\frac{r}{k} D(a \| p)\right) \qquad (0 \leq a \leq p).}$$

In particular, for every $\varepsilon > 0$,

$$\Pr\left(|S - \mathbb{E} S| \geq \varepsilon r\right) \leq 2 \exp\left(-\frac{2\varepsilon^2 r}{k}\right).$$

Proof. The cases $p = 0, 1$ are deterministic, so assume $0 < p < 1$. Write $p_i = \mathbb{E}Y_i$. For every real t, Corollary 207 gives

$$\mathbb{E}e^{tS} \le \prod_i \left(\mathbb{E}e^{ktY_i}\right)^{1/k}.$$

Convexity on $[0, 1]$ gives $e^{ktu} \le 1 - u + ue^{kt}$. Therefore

$$\mathbb{E}e^{tS} \le \prod_i (1 - p_i + p_i e^{kt})^{1/k} \le (1 - p + pe^{kt})^{r/k},$$

where the last step uses concavity of $u \mapsto \ln(1 - u + ue^{kt})$.

Exponential Markov with $t \ge 0$ bounds the upper tail, and with $t \le 0$ bounds the lower tail. For $0 < a < 1$, the optimizing value satisfies

$$e^{kt} = \frac{a(1-p)}{p(1-a)}.$$

Substitution yields the relative-entropy bounds; the endpoints follow by limits.

Finally, as a function of a,

$$\frac{\partial^2}{\partial a^2} D(a\|p) = \frac{1}{a(1-a)} \ge 4,$$

while $D(p\|p) = 0$ and its first derivative there is zero. Hence $D(a\|p) \ge 2(a-p)^2$. Apply this to both tails. □

Writing $\mu = \mathbb{E}S$, the same calculation also gives the usual multiplicative forms with an extra factor k in the denominator of the exponent:

$$\Pr(S \ge (1+\delta)\mu) \le \left(\frac{e^\delta}{(1+\delta)^{1+\delta}}\right)^{\mu/k} \qquad (\delta \ge 0),$$

and

$$\Pr(S \le (1-\delta)\mu) \le \exp\left(-\frac{\delta^2 \mu}{2k}\right) \qquad (0 \le \delta \le 1).$$

Indeed, before averaging the p_i's, use $1 + u \le e^u$ to obtain $\mathbb{E}e^{tS} \le \exp((\mu/k)(e^{kt} - 1))$, and optimize.

The factor k is necessary in general. Repeat each of several independent Bernoulli variables exactly k times. Their sum is k times a sum with only r/k independent terms.

5.10 Enumerative and Algebraic Combinatorics

5.10.1 Lindström–Gessel–Viennot Lemma

The LGV lemma is fundamentally a sign-reversing involution. The usual edge-weighted version is a special case of the following slightly more general statement.

Let G be a finite directed acyclic graph and let R be a commutative ring. Assign to every directed path P a weight

$$w(P) \in R.$$

We only require the following *tail-switching property.* Suppose two paths P, Q meet at a vertex v, and write

$$P = P^- P^+, \qquad Q = Q^- Q^+$$

at v. Let

$$P' = P^- Q^+, \qquad Q' = Q^- P^+.$$

Then require

$$\boxed{w(P)w(Q) = w(P')w(Q').}$$

Ordinary multiplicative edge weights

$$w(P) = \prod_{e \in P} w(e)$$

automatically satisfy this condition.

Let

$$a_1, \dots, a_k, \qquad b_1, \dots, b_k$$

be ordered vertices. Define the path matrix

$$M_{ij} = \sum_{P : a_i \to b_j} w(P).$$

For a permutation $\sigma \in S_k$, let $\mathcal{N}_\sigma$ be the set of k-tuples

$$(P_1, \dots, P_k)$$

of pairwise vertex-disjoint directed paths satisfying

$$P_i : a_i \to b_{\sigma(i)}.$$

Define

$$w(P_1, \dots, P_k) = \prod_i w(P_i).$$

Theorem 209 (Lindström–Gessel–Viennot)**.** *With the notation above,*

$$\boxed{\det M = \sum_{\sigma \in S_k} \operatorname{sgn}(\sigma) \sum_{\mathbf{P} \in \mathcal{N}_\sigma} w(\mathbf{P}).}$$

Proof. Expand the determinant:

$$\det M = \sum_{\sigma \in S_k} \operatorname{sgn}(\sigma) \prod_{i=1}^{k} \left(\sum_{P_i : a_i \to b_{\sigma(i)}} w(P_i) \right).$$

Thus $\det M$ is the signed weighted sum over *all* path families

$$(P_1, \dots, P_k),$$

including intersecting ones.

We cancel the intersecting families by an involution.

Fix a topological ordering of the vertices of G. For an intersecting family, choose the earliest vertex v in this order that lies on at least two paths. Among the paths containing v, choose the first pair

$$P_i, P_j$$

according to some fixed ordering of the indices.

Swap their tails after v:

$$P_i = P_i^- P_i^+, \qquad P_j = P_j^- P_j^+$$

becomes

$$P_i' = P_i^- P_j^+, \qquad P_j' = P_j^- P_i^+.$$

Because G is acyclic, the switched objects are still directed paths. The endpoints $b_{\sigma(i)}$ and $b_{\sigma(j)}$ are exchanged, so the new permutation is

$$\sigma \circ (i\, j),$$

which has the opposite sign.

By the tail-switching assumption,

$$w(P_i)w(P_j) = w(P_i')w(P_j'),$$

so the total weight is unchanged.

The chosen first intersection and pair are unchanged by performing the swap again, so this operation is an involution.

Hence every intersecting family cancels with another family of equal weight and opposite sign. The only uncancelled terms are the pairwise vertex-disjoint families, giving the stated formula. □

A particularly common special case occurs when the sources and sinks are *compatible*: no vertex-disjoint path family can connect

$$a_i \to b_{\sigma(i)}$$

for any nonidentity permutation σ. Then

$$\boxed{\det M = \sum_{\substack{P_i : a_i \to b_i \\ P_1,\dots,P_k \text{ pairwise disjoint}}} \prod_i w(P_i).}$$

Thus a determinant directly enumerates nonintersecting path families.

5.10.2 Burnside's Lemma and Pólya Enumeration

Suppose a finite group acts on a family of objects. Objects in the same orbit are regarded as equivalent. Simply dividing the number of objects by the group size is generally wrong, because different objects may have different numbers of symmetries. Burnside's lemma corrects this by counting each orbit through the symmetries that fix its elements.

Pólya enumeration applies this principle when a group permutes a finite set of positions and each position receives an object from some weighted family. A decoration fixed by a permutation must be constant on each cycle. Thus a cycle of length j contributes the inventory of one object with every weight multiplied by j. The cycle index records exactly how these contributions combine.

Burnside's lemma Let a finite group G act on a set Ω. For $x \in \Omega$, write

$$G_x = \{g \in G : gx = x\}$$

for its stabilizer, and for $g \in G$, write

$$\mathrm{Fix}(g) = \{x \in \Omega : gx = x\}.$$

For every orbit $O = Gx$, the orbit–stabilizer theorem gives

$$|O|\,|G_x| = |G|.$$

Theorem 210 (Weighted Burnside lemma)**.** *Let a finite group G act on a set Ω, and let*

$$w : \Omega \to R$$

be constant on orbits, where R is a commutative $\mathbb{Q}$-algebra. Assume all sums below are defined. Then

$$\boxed{\sum_{O \in \Omega/G} w(O) = \frac{1}{|G|} \sum_{g \in G} \sum_{x \in \mathrm{Fix}(g)} w(x).}$$

In particular,

$$\boxed{|\Omega/G| = \frac{1}{|G|} \sum_{g \in G} |\,\mathrm{Fix}(g)|.}$$

Proof. Group the fixed pairs by the orbit of x. If O is one orbit, then every $x \in O$ has the same weight $w(O)$ and stabilizers of the same size. Its total contribution to the double sum is therefore

$$\sum_{x \in O} |G_x| w(x) = |O|\,|G_x|\,w(O) = |G| w(O).$$

Summing over all orbits and dividing by $|G|$ proves the result. □

For generating functions, it is useful to allow Ω to be infinite but graded. If every coefficient receives contributions from only finitely many objects, the same proof applies coefficientwise.

Cycle indices and weighted decorations Let G act on a finite set D of n positions. For $g \in G$, let $c_j(g)$ be the number of j-cycles in its permutation of D. The *cycle index* of the action is

$$\boxed{Z_G(u_1, \dots, u_n) = \frac{1}{|G|} \sum_{g \in G} \prod_{j=1}^{n} u_j^{c_j(g)}.}$$

Now let $\mathcal{A}$ be a weighted family. Give each $a \in \mathcal{A}$ a weight vector

$$\mathrm{wt}(a) = (\alpha_1(a), \dots, \alpha_r(a)) \in \mathbb{N}^r,$$

and define its multivariate generating function

$$A(\mathbf{x}) = A(x_1, \dots, x_r) = \sum_{a \in \mathcal{A}} x_1^{\alpha_1(a)} \cdots x_r^{\alpha_r(a)}.$$

Assume each coefficient of A is finite.

A decoration of D is a function

$$f : D \to \mathcal{A}.$$

Its weight is the product of the weights placed at its positions:

$$\mathrm{wt}(f) = \prod_{v \in D} \mathbf{x}^{\mathrm{wt}(f(v))} = \mathbf{x}^{\sum_{v \in D} \mathrm{wt}(f(v))}.$$

The action of G is

$$(g \cdot f)(v) = f(g^{-1} v).$$

For $j \geq 1$, write

$$\mathbf{x}^j = (x_1^j, \dots, x_r^j).$$

Theorem 211 (Pólya enumeration)**.** *With the preceding notation,*

$$\boxed{\sum_{[f] \in \mathcal{A}^D/G} \mathrm{wt}(f) = Z_G\big(A(\mathbf{x}), A(\mathbf{x}^2), \dots, A(\mathbf{x}^n)\big).}$$

Equivalently,

$$\sum_{[f] \in \mathcal{A}^D/G} \mathrm{wt}(f) = \frac{1}{|G|} \sum_{g \in G} \prod_{j=1}^{n} A(\mathbf{x}^j)^{c_j(g)}.$$

Proof. Fix $g \in G$. A decoration f is fixed by g exactly when it is constant on every cycle of g.

Consider one cycle of length j. If it receives the object $a \in \mathcal{A}$, then a appears at all j positions, so its contribution to the total weight is

$$\left(\mathbf{x}^{\mathrm{wt}(a)}\right)^j = \mathbf{x}^{j\,\mathrm{wt}(a)}.$$

Summing over the possible objects placed on this cycle gives

$$\sum_{a \in \mathcal{A}} \mathbf{x}^{j \operatorname{wt}(a)} = A(\mathbf{x}^j).$$

Different cycles may be decorated independently. Hence the weighted inventory of decorations fixed by g is

$$\prod_{j=1}^{n} A(\mathbf{x}^j)^{c_j(g)}.$$

Weighted Burnside now gives the stated average. □

For example, if

$$\mathcal{A} = \{a_1, \ldots, a_r\}, \qquad \operatorname{wt}(a_i) = e_i,$$

then

$$A(x_1, \ldots, x_r) = x_1 + \cdots + x_r,$$

and the coefficient of $x_1^{n_1} \cdots x_r^{n_r}$ counts orbits in which object type a_i occurs exactly n_i times.

If instead $\mathcal{A}$ has two objects of weights 0 and 1, then $A(z) = 1 + z$, and

$$Z_G(1 + z, 1 + z^2, \ldots, 1 + z^n)$$

counts G-orbits of subsets of D by cardinality.

Necklaces Let the cyclic group C_n act by rotation on n cyclically ordered positions. A rotation by s positions has $\gcd(n, s)$ cycles, each of length

$$\frac{n}{\gcd(n, s)}.$$

Equivalently, a rotation of order $d \mid n$ consists of n/d cycles of length d, and there are $\varphi(d)$ such rotations.

Theorem 212 (Necklace cycle index)**.** *For the cyclic action on n positions,*

$$\boxed{Z_{C_n}(u_1, \ldots, u_n) = \frac{1}{n} \sum_{d \mid n} \varphi(d) u_d^{n/d}.}$$

Consequently, for a weighted family with generating function $A(\mathbf{x})$, the generating function of cyclic sequences of n objects from $\mathcal{A}$, modulo rotation, is

$$\boxed{N_n(\mathbf{x}) = \frac{1}{n} \sum_{d \mid n} \varphi(d) A(\mathbf{x}^d)^{n/d}.}$$

Proof. Group the rotations according to their order d. There are $\varphi(d)$ rotations of order d, and each has n/d cycles of length d. This gives the cycle index. The second formula is the substitution

$$u_d \mapsto A(\mathbf{x}^d).$$

□

If $\mathcal{A}$ consists of r unweighted object types, then $A = r$, and the number of necklaces is

$$\boxed{\frac{1}{n} \sum_{d \mid n} \varphi(d) r^{n/d}.}$$

If instead the r types have independent weights $x_1, \dots, x_r$, then

$$A(\mathbf{x}) = x_1 + \cdots + x_r.$$

For prescribed multiplicities

$$n_1 + \cdots + n_r = n,$$

the coefficient of $x_1^{n_1} \cdots x_r^{n_r}$ is

$$\boxed{\frac{1}{n} \sum_{d \mid \gcd(n_1, \dots, n_r)} \varphi(d) \frac{(n/d)!}{(n_1/d)! \cdots (n_r/d)!}.}$$

Indeed,

$$[x_1^{n_1} \cdots x_r^{n_r}](x_1^d + \cdots + x_r^d)^{n/d}$$

vanishes unless d divides every n_i, and otherwise is the displayed multinomial coefficient.

The symmetric group and unlabelled multisets For S_n acting on n positions, suppose a permutation has m_j cycles of length j. Then

$$\sum_{j \geq 1} j m_j = n,$$

and the number of permutations with these cycle counts is

$$\frac{n!}{\prod_{j \geq 1} j^{m_j} m_j!}.$$

Therefore

$$\boxed{Z_{S_n} = \sum_{\substack{m_1, m_2, \dots \geq 0 \\ \sum_j j m_j = n}} \prod_{j \geq 1} \frac{u_j^{m_j}}{j^{m_j} m_j!}.}$$

Summing over all n,

$$\begin{aligned} \sum_{n \geq 0} Z_{S_n}(u_1, u_2, \dots) t^n &= \prod_{j \geq 1} \sum_{m \geq 0} \frac{1}{m!} \left(\frac{u_j t^j}{j} \right)^m \\ &= \boxed{\exp\left(\sum_{j \geq 1} \frac{u_j t^j}{j} \right).} \end{aligned} \tag{1}$$

Now let

$$A(\mathbf{x}) = \sum_{\alpha \in \mathbb{N}^r \setminus \{0\}} a_\alpha \mathbf{x}^\alpha$$

be the generating function of a family of nonempty unlabelled object types. An unordered multiset of n such objects is precisely an orbit of an ordered n-tuple under S_n. Substituting

$$u_j = A(\mathbf{x}^j)$$

into (1), and then setting $t = 1$, gives the inventory of arbitrary finite multisets:

$$\boxed{M(\mathbf{x}) = \exp\left(\sum_{j \geq 1} \frac{A(\mathbf{x}^j)}{j} \right).}$$

Expanding A,

$$\sum_{j\geq 1}\frac{A(\mathbf{x}^j)}{j} = \sum_{\alpha} a_\alpha \sum_{j\geq 1}\frac{\mathbf{x}^{j\alpha}}{j}$$
$$= -\sum_{\alpha} a_\alpha \log(1-\mathbf{x}^\alpha).$$

Hence

$$\boxed{M(\mathbf{x}) = \prod_{\alpha}(1-\mathbf{x}^\alpha)^{-a_\alpha}.}$$

In one variable, if

$$A(z) = \sum_{m\geq 1} a_m z^m,$$

this becomes

$$\boxed{M(z) = \exp\left(\sum_{j\geq 1}\frac{A(z^j)}{j}\right) = \prod_{m\geq 1}(1-z^m)^{-a_m}.}$$

The condition $A(\mathbf{0}) = 0$ ensures that these infinite sums and products are well-defined coefficientwise.

This is the unlabelled analogue of the exponential formula. For labelled sets of components one obtains $\exp(A(z))$; for unlabelled multisets, repeated identical components produce the extra terms

$$\frac{A(z^2)}{2}, \frac{A(z^3)}{3}, \ldots.$$

Application: unlabelled graphs A simple graph on $[n]$ is determined by a subset of

$$E_n = \binom{[n]}{2}.$$

The symmetric group S_n acts on vertices and therefore induces an action on E_n. Two graphs are isomorphic exactly when their edge sets lie in the same orbit of this induced action.

Burnside's lemma reduces the problem to counting the orbits of each vertex permutation on E_n. If a permutation has vertex-cycle lengths

$$\lambda = (\lambda_1, \ldots, \lambda_r) \vdash n,$$

let m_j denote the number of parts of λ equal to j, and put

$$z_\lambda = \prod_{j\geq 1} j^{m_j} m_j!.$$

There are

$$\frac{n!}{z_\lambda}$$

permutations of cycle type λ.

We first count the induced cycles on unordered pairs.

Between two distinct vertex cycles of lengths a and b, the orbit of a pair has length

$$\operatorname{lcm}(a,b).$$

Since there are ab such pairs, the number of pair-orbits between the two cycles is

$$\frac{ab}{\operatorname{lcm}(a,b)} = \gcd(a,b).$$

Within a vertex cycle of length a, an unordered pair is determined up to rotation by its cyclic distance

$$1, \dots, \left\lfloor \frac{a}{2} \right\rfloor.$$

Thus there are

$$\left\lfloor \frac{a}{2} \right\rfloor$$

pair-orbits inside that cycle.

Define

$$e(\lambda) = \sum_{i=1}^{r} \left\lfloor \frac{\lambda_i}{2} \right\rfloor + \sum_{1 \le i < j \le r} \gcd(\lambda_i, \lambda_j).$$

Theorem 213 (Unlabelled graph count)**.** *The number g_n of isomorphism classes of simple graphs on n vertices is*

$$\boxed{g_n = \sum_{\lambda \vdash n} \frac{2^{e(\lambda)}}{z_\lambda}.}$$

Proof. Fix a permutation of cycle type λ. By the preceding count, its induced action on E_n has exactly $e(\lambda)$ orbits.

An edge set is fixed by the permutation exactly when it is a union of these edge-orbits. Each orbit may independently be included or omitted, so the number of fixed graphs is

$$2^{e(\lambda)}.$$

There are $n!/z_\lambda$ permutations of this cycle type. Burnside's lemma therefore gives

$$\begin{aligned} g_n &= \frac{1}{n!} \sum_{\lambda \vdash n} \frac{n!}{z_\lambda} 2^{e(\lambda)} \\ &= \sum_{\lambda \vdash n} \frac{2^{e(\lambda)}}{z_\lambda}. \end{aligned}$$

□

The same calculation can retain the number of edges. If the induced edge-orbits have lengths

$$\ell_1, \dots, \ell_m,$$

then the fixed graphs contribute

$$\prod_{i=1}^{m} (1 + z^{\ell_i}),$$

because including the i-th orbit contributes all ℓ_i of its edges.

For two distinct vertex cycles of lengths a, b, there are $\gcd(a, b)$ edge-orbits, each of length

$$\operatorname{lcm}(a, b).$$

Inside a vertex cycle of length a, each distance

$$d < \frac{a}{2}$$

gives one orbit of length a; when a is even, the opposite pairs $d = a/2$ form one orbit of length $a/2$. Thus the same Burnside calculation gives the ordinary generating polynomial for unlabelled graphs on n vertices by number of edges.

5.10.3 Q-analogues

A useful q-analogue replaces subsets by subspaces: cardinality becomes dimension, intersection becomes intersection of subspaces, and disjointness means intersection $\{0\}$. The extra powers of q count free choices of linear maps. The same Gaussian coefficients also enumerate partitions in a rectangle when q is a formal variable.

In statements about $\mathbb{F}_q$, q is a prime power. In generating identities, q is an indeterminate. The finite identities are polynomial identities and hence hold in both interpretations; the infinite products below are interpreted as formal series, or for $|q| < 1$ where appropriate.

Count Bases. Define $[r]_q = 1 + q + \cdots + q^{r-1}$ and $[r]_q! = \prod_{j=1}^{r} [j]_q$, with $[0]_q! = 1$. The Gaussian coefficient is

$$\binom{n}{k}_q = \frac{[n]_q!}{[k]_q![n-k]_q!} = \prod_{i=0}^{k-1} \frac{q^{n-i}-1}{q^{k-i}-1} \qquad (0 \le k \le n).$$

Set it to zero for $k < 0$ or $k > n$. At $q = 1$, its value is $\binom{n}{k}$.

Theorem 214 (Subspace count)**.** *An n-dimensional space over $\mathbb{F}_q$ has exactly $\binom{n}{k}_q$ subspaces of dimension k.*

Proof. An ordered independent k-tuple in $\mathbb{F}_q^n$ has $\prod_{i=0}^{k-1}(q^n - q^i)$ choices: at stage i, exclude the q^i vectors already in the span. Each k-space has $\prod_{i=0}^{k-1}(q^k - q^i)$ ordered bases. Divide these two products and cancel their powers of q. □

Orthogonal complementation gives $\binom{n}{k}_q = \binom{n}{n-k}_q$. Splitting according to containment in a fixed hyperplane gives the two Pascal recurrences

$$\boxed{\binom{n}{k}_q = \binom{n-1}{k}_q + q^{n-k}\binom{n-1}{k-1}_q = \binom{n-1}{k-1}_q + q^{k}\binom{n-1}{k}_q.} \tag{19}$$

Indeed, a k-space not contained in the hyperplane meets it in a $(k-1)$-space S. In the quotient by S, its remaining choice is a line outside a hyperplane of an $(n-k+1)$-space. There are $[n-k+1]_q - [n-k]_q = q^{n-k}$ such lines. Symmetry gives the other form.

How large is the Gaussian coefficient? For a numerical prime power $q \ge 2$,

$$\boxed{q^{k(n-k)} \le \binom{n}{k}_q \le 4q^{k(n-k)}.} \tag{20}$$

Factor out $q^{k(n-k)}$ from the product formula. Each resulting numerator factor is at least its corresponding denominator factor, giving the lower bound. For the upper bound, discard the numerator factors and use

$$\prod_{j=1}^{\infty}(1-q^{-j}) \ge \frac{1}{2}\cdot\frac{3}{4}\left(1 - \sum_{j=3}^{\infty} 2^{-j}\right) = \frac{9}{32} > \frac{1}{4}.$$

Here $\prod(1-a_j) \ge 1 - \sum a_j$ follows by induction for finite products and then by limits. Thus a k-space has roughly $k(n-k)$ freely chosen field coordinates.

Lemma 215 (Prescribed intersection dimension)**.** *Fix an a-dimensional subspace $H \le V = \mathbb{F}_q^n$. The number of k-spaces W with $\dim(W \cap H) = r$ is*

$$\boxed{\binom{a}{r}_q \binom{n-a}{k-r}_q q^{(a-r)(k-r)}.}$$

The expression is understood as zero outside the feasible dimensions.

Proof. Choose $S = W \cap H$, giving $\binom{a}{r}_q$ choices, and fix a complement $V = H \oplus K$. Projection to K sends W to a $(k-r)$-space L, with $\binom{n-a}{k-r}_q$ choices. In $V/S = (H/S) \oplus K$, the image of W is disjoint from H/S and projects bijectively onto L. It is therefore the graph of a unique linear map $L \to H/S$. There are $q^{(a-r)(k-r)}$ such maps, each giving exactly one W. □

Summing over r gives a q-Vandermonde identity:

$$\boxed{\binom{a+b}{k}_q = \sum_r q^{(a-r)(k-r)} \binom{a}{r}_q \binom{b}{k-r}_q.}$$

In particular, there are $q^{ak}\binom{n-a}{k}_q$ k-spaces disjoint from a fixed a-space, and $q^{a(n-a)}$ complements to that space.

Flags and a second q-factorial interpretation. If $d_1 + \cdots + d_s = n$, a flag whose successive dimension increases are $d_1, \ldots, d_s$ can be chosen in successive quotients. Its count is

$$\binom{n}{d_1}_q \binom{n-d_1}{d_2}_q \cdots = \frac{[n]_q!}{\prod_i [d_i]_q!}.$$

In particular, the number of complete flags is $[n]_q!$. The same polynomial counts permutations by inversions: $\sum_{\sigma \in S_n} q^{\mathrm{inv}(\sigma)} = [n]_q!$. To prove this, insert the largest label n into a permutation of $[n-1]$. Its position creates each possible number $0, \ldots, n-1$ of new inversions once, multiplying the generating polynomial by $[n]_q$.

Partitions. For a partition λ fitting inside a $k \times (n-k)$ rectangle, write $|\lambda|$ for its number of cells. Then

$$\boxed{\binom{n}{k}_q = \sum_{\lambda \subseteq k \times (n-k)} q^{|\lambda|}.} \tag{21}$$

For a direct proof, split these partitions into those with fewer than k nonzero rows and those with exactly k. The first fit in a $(k-1) \times (n-k)$ rectangle. From each of the second kind, remove its first column of height k, leaving a $k \times (n-k-1)$ rectangle. Thus their generating polynomials satisfy the second recurrence in (19), with the same boundary values. This proves the identity and also shows that Gaussian coefficients are polynomials with nonnegative integer coefficients. The product and recurrence identities, initially justified for prime powers, hold formally because they agree at infinitely many values of q after denominators are cleared.

Theorem 216 (Finite q-binomial theorem)**.** *For $n \geq 0$,*

$$\boxed{\prod_{i=0}^{n-1} (1 + zq^i) = \sum_{k=0}^{n} q^{\binom{k}{2}} \binom{n}{k}_q z^k.}$$

Proof. The coefficient of z^k chooses $0 \leq i_1 < \cdots < i_k \leq n-1$, with weight $q^{i_1 + \cdots + i_k}$. Subtract $0, 1, \ldots, k-1$ from these indices. The resulting weakly increasing entries lie between zero and $n-k$ and, on reversal, describe a partition in the rectangle of (21). The removed staircase has size $\binom{k}{2}$. □

Taking $z = -1$ gives the cancellation identity

$$\sum_{j=0}^{d} (-1)^j q^{\binom{j}{2}} \binom{d}{j}_q = \begin{cases} 1, & d = 0, \\ 0, & d > 0. \end{cases} \tag{22}$$

This is the subspace-lattice Möbius cancellation identity.

Durfee square identity. For ordinary partitions, marking each row by z and each cell by q gives

$$\boxed{\prod_{j\geq 1}\frac{1}{1-zq^j}=\sum_{r\geq 0}\frac{z^r q^{r^2}}{\prod_{j=1}^{r}(1-q^j)\prod_{j=1}^{r}(1-zq^j)}.} \tag{23}$$

The left side chooses the multiplicity of each positive part. On the right, cut at the largest upper-left square, of side r. The square contributes $z^r q^{r^2}$. The part to its right has at most r rows and contributes $\prod_{j=1}^{r}(1-q^j)^{-1}$: these cells create no new rows. The part below has parts at most r and contributes $\prod_{j=1}^{r}(1-zq^j)^{-1}$. Conversely, attaching any such two parts gives a partition with this largest square. This proves the formula.

Random Flags Prove the q-Sperner Theorem. Order all subspaces of $V=\mathbb{F}_q^n$ by inclusion. A complete flag is a chain $\{0\}=V_0<V_1<\cdots<V_n=V$ with $\dim V_j=j$. The linear group acts transitively on each dimension level and permutes complete flags. Hence a uniform complete flag contains any given j-space with probability $1/\binom{n}{j}_q$: it contains exactly one of them, and all have the same probability.

Theorem 217 (q-LYM and q-Sperner)**.** *If $\mathcal{A}$ is an antichain of subspaces of $\mathbb{F}_q^n$, then*

$$\boxed{\sum_{U\in\mathcal{A}}\binom{n}{\dim U}_q^{-1}\leq 1,\qquad |\mathcal{A}|\leq\binom{n}{\lfloor n/2\rfloor}_q.}$$

More generally, a family with no chain of $s+1$ members has size at most the sum of the s largest dimension levels, for $1\leq s\leq n+1$. All these size bounds are attained by the corresponding full levels.

Proof. An antichain meets a complete flag at most once. Taking expectations proves the first bound. More generally, the same calculation gives a weighted sum at most s when every flag contains at most s members. Let $N_j=\binom{n}{j}_q$ and a_j be the number of family members of dimension j. Thus $0\leq a_j/N_j\leq 1$ and $\sum_j a_j/N_j\leq s$. The quantity $\sum_j a_j=\sum_j N_j(a_j/N_j)$ is maximized by assigning this budget first to the s largest N_j: moving any available budget from a smaller coefficient to a larger one cannot decrease the sum. This proves the general size bound.

Finally, $N_{j+1}/N_j=(q^{n-j}-1)/(q^{j+1}-1)$, so the levels increase up to the middle and then decrease, and are symmetric by complementation. The largest is therefore $N_{\lfloor n/2\rfloor}$. A union of s levels has no chain of $s+1$ members, proving sharpness. □

A shadow bound. If $\mathcal{F}$ consists of k-spaces, let $\partial\mathcal{F}$ contain all their $(k-1)$-subspaces. Double-counting containment pairs gives

$$|\partial\mathcal{F}|[n-k+1]_q\geq|\mathcal{F}|[k]_q,\qquad\boxed{\frac{|\partial\mathcal{F}|}{\binom{n}{k-1}_q}\geq\frac{|\mathcal{F}|}{\binom{n}{k}_q}.}$$

Each k-space has $[k]_q$ hyperplanes, and each $(k-1)$-space is in $[n-k+1]_q$ k-spaces.

Intersecting Subspaces and the q-Erdős–Ko–Rado Theorem. An intersecting family of k-spaces has $U\cap W\neq\{0\}$ for every two distinct members. A natural example consists of all k-spaces containing one fixed line. There are $\binom{n-1}{k-1}_q$ of them, by passing to the quotient by that line. We prove this is optimal for $n\geq 2k$. Unlike the antichain condition, intersection is not visible along one random flag; instead, use the graph whose edges represent forbidden disjoint pairs.

Lemma 218 (Spectral bound for an independent set)**.** *Let a simple D-regular graph on N vertices have adjacency matrix A, where $D>0$. If every eigenvalue of A is at least $-\beta$, with $\beta>0$, then every independent set has size at most $N\beta/(D+\beta)$.*

Proof. Write its indicator as $z = \alpha\mathbf{1} + h$, where $\alpha = |S|/N$ and $h \perp \mathbf{1}$. Then $\|h\|_2^2 = N\alpha(1-\alpha)$, while independence gives

$$0 = z^\top A z = DN\alpha^2 + h^\top A h \geq DN\alpha^2 - \beta N\alpha(1-\alpha).$$

For $\alpha > 0$, divide by $N\alpha$ and rearrange; the empty set is immediate. □

Let Ω be the set of k-spaces, with $n \geq 2k \geq 2$. The q-Kneser graph joins $W, Z \in \Omega$ when $W \cap Z = \{0\}$. Its degree, by Lemma 215, is $D = q^{k^2}\binom{n-k}{k}_q$. We need its least eigenvalue.

For a j-space J, define the containment indicator $f_J(W) = \mathbf{1}_{J\subseteq W}$, and let U_j be the real span of all such indicators, with $U_{-1} = \{0\}$. These spaces are nested: $U_0 \subseteq U_1 \subseteq \cdots \subseteq U_k = \mathbb{R}^\Omega$. Indeed, summing f_J over all j-spaces containing a fixed $(j-1)$-space L gives $[k-j+1]_q f_L$; at level k the indicators are the individual coordinate vectors.

For fixed J, W, the quantity $(Af_J)(W)$ counts k-spaces containing J and disjoint from W. It is zero if $J \cap W \neq \{0\}$. Otherwise quotienting by J and applying the disjoint-subspace count gives

$$(Af_J)(W) = c_j \mathbf{1}_{J\cap W=\{0\}}, \qquad c_j = q^{k(k-j)}\binom{n-k-j}{k-j}_q.$$

The cancellation identity (22) expands this disjointness indicator into containment indicators:

$$Af_J = c_j \sum_{L\leq J} (-1)^{\dim L} q^{\binom{\dim L}{2}} f_L.$$

Only the term $L = J$ is at level j; every other term lies in U_{j-1}. Consequently A preserves U_j and acts on it modulo U_{j-1} by

$$\lambda_j = (-1)^j q^{k(k-j)+\binom{j}{2}}\binom{n-k-j}{k-j}_q.$$

Because A is symmetric, the orthogonal complement of U_{j-1} inside U_j is also invariant. On this complement, $(A - \lambda_j I)v$ lies both in U_{j-1} and in its orthogonal complement, so it is zero. These complements exhaust $\mathbb{R}^\Omega$, proving that every eigenvalue is among the λ_j.

Their absolute values strictly decrease, since for $0 \leq j < k$,

$$\frac{|\lambda_{j+1}|}{|\lambda_j|} = q^{j-k}\frac{q^{k-j}-1}{q^{n-k-j}-1} \leq q^{j-k} < 1.$$

Thus all eigenvalues are at least $\lambda_1 = -q^{k(k-1)}\binom{n-k-1}{k-1}_q$.

Theorem 219 (q-Erdős–Ko–Rado)**.** *For $n \geq 2k$ and $k \geq 1$, an intersecting family $\mathcal{F}$ of k-dimensional subspaces of $\mathbb{F}_q^n$ satisfies*

$$\boxed{|\mathcal{F}| \leq \binom{n-1}{k-1}_q.}$$

The family of all k-spaces containing a fixed line attains equality.

Proof. The family is an independent set of the q-Kneser graph. Apply the spectral bound with $N = \binom{n}{k}_q$, the degree D above, and $\beta = -\lambda_1$. The formula gives

$$\frac{D}{\beta} = q^k\frac{q^{n-k}-1}{q^k-1}, \qquad \frac{\beta}{D+\beta} = \frac{q^k-1}{q^n-1}.$$

Therefore $|\mathcal{F}| \leq \binom{n}{k}_q(q^k-1)/(q^n-1) = \binom{n-1}{k-1}_q$. The fixed-line construction gives the stated equality example. □

When $n < 2k$, all k-spaces intersect because $\dim(U \cap W) \geq 2k - n > 0$, so the whole level is allowed.

Uniform random subspaces. To sample a uniform k-space, choose an $n \times k$ matrix uniformly over $\mathbb{F}_q$, reject it unless its columns are independent, and return their span. Every k-space has the same number $|\operatorname{GL}_k(q)|$ of ordered bases, so the accepted span is uniform. The acceptance probability is

$$\prod_{i=0}^{k-1}(1-q^{i-n}) \geq \prod_{j=1}^{\infty}(1-q^{-j}) > \frac{1}{4}.$$

Thus fewer than four trials are needed in expectation. Gaussian elimination uses $O(nk^2)$ field operations per trial for $k \geq 1$.

5.10.4 Jacobi Triple Product

We seek a product expansion for the bilateral series

$$\Theta(x,y) = \sum_{k\in\mathbb{Z}} x^{k^2} y^k.$$

The derivation starts from the square in the exponent. A rectangular version of the Durfee-square decomposition will separate the signed index into two independent nonnegative indices.

Why rectangles appear. Writing $k = r - s$ with $r, s \geq 0$ gives

$$k^2 = r^2 + s^2 - 2rs.$$

The square terms separate into one depending on r and one depending on s. The obstruction is the mixed term $-2rs$, which is minus twice the area of an $r \times s$ rectangle.

This suggests attaching an ordinary partition to each k, giving every cell weight x^2, and cutting the partition around a rectangle with $r - s = k$. The rectangle contributes x^{2rs}, exactly canceling the mixed term. To avoid counting the same partition repeatedly, choose the *largest* rectangle with this difference.

The rectangular Durfee identity. Write $|\lambda|$ for the size of a partition and $\ell(\lambda)$ for its number of nonzero parts. Define

$$P(q) = \sum_{\lambda} q^{|\lambda|} = \prod_{j\geq 1} \frac{1}{1-q^j}, \qquad P_r(q) = \sum_{\ell(\lambda)\leq r} q^{|\lambda|} = \prod_{j=1}^{r} \frac{1}{1-q^j},$$

with $P_0(q) = 1$. The first product chooses the multiplicity of each possible part. For the second, transpose the diagram: partitions with at most r rows correspond to partitions whose parts are at most r.

Lemma 220 (Rectangular Durfee identity)**.** *For every integer k,*

$$\boxed{P(q) = \sum_{\substack{r,s\geq 0 \\ r-s=k}} q^{rs} P_r(q) P_s(q).}$$

Proof. In a partition λ, choose the largest upper-left rectangle of height r and width s satisfying $r - s = k$. Zero dimensions are allowed, and we append zero rows when necessary. The candidate rectangles have dimensions

$$\big(\max\{k,0\} + t\big) \times \big(\max\{-k,0\} + t\big), \qquad t = 0, 1, 2, \ldots.$$

They are nested, the first always fits, and sufficiently large ones do not. Thus there is a unique largest one.

Remove this rectangle. The piece ρ to its right has at most r rows. The piece below it has at most s columns: otherwise $\lambda_{r+1} \geq s + 1$, so the next rectangle of dimensions $(r+1) \times (s+1)$ would fit. Transposing the lower piece gives a partition σ with at most s rows. Their sizes satisfy $|\lambda| = rs + |\rho| + |\sigma|$.

Conversely, choose any such ρ and σ. Attach ρ to the right of the rectangle and $\sigma^\top$ below it. After padding ρ with zeros, the first r row lengths are $s+\rho_i$, while every lower row has length at most s. Hence this is a partition, and the next rectangle cannot fit. The construction therefore recovers exactly the prescribed maximal rectangle.

The rectangle contributes q^{rs}, and the two remaining pieces are independent, contributing $P_r(q)$ and $P_s(q)$. Summing over its possible dimensions proves the identity. □

For $k=0$, the rectangles are squares, and we recover the usual Durfee-square identity

$$P(q)=\sum_{r\geq 0} q^{r^2}P_r(q)^2.$$

Separating the two indices. Multiply $\Theta(x,y)$ by $P(x^2)$, thereby attaching a partition whose cells have weight x^2. Apply the rectangular identity separately for each k:

$$\begin{aligned}P(x^2)\Theta(x,y)&=\sum_{k\in\mathbb{Z}} x^{k^2}y^k \sum_{\substack{r,s\geq 0\\ r-s=k}} x^{2rs}P_r(x^2)P_s(x^2)\\ &=\sum_{r,s\geq 0} x^{(r-s)^2+2rs}y^{r-s}P_r(x^2)P_s(x^2)\\ &=\sum_{r,s\geq 0}\big(x^{r^2}y^rP_r(x^2)\big)\big(x^{s^2}y^{-s}P_s(x^2)\big).\end{aligned}$$

The rectangle has canceled the mixed term: $(r-s)^2+2rs=r^2+s^2$. The indices are now independent. Thus, defining $A(x,z)=\sum_{r\geq 0}x^{r^2}z^rP_r(x^2)$, we obtain

$$\boxed{P(x^2)\Theta(x,y)=A(x,y)A(x,y^{-1}).} \tag{24}$$

It remains to recognize the series A.

Odd staircases. The factor $P_r(x^2)$ counts partitions with at most r rows, with their sizes doubled. The remaining factor x^{r^2} suggests adding a fixed diagram of size r^2. The useful choice comes from

$$r^2=1+3+\cdots+(2r-1):$$

add an odd staircase.

Lemma 221 (Odd-staircase identity)**.** *For every $r\geq 0$,*

$$\sum_{\substack{\alpha \text{ has exactly } r\\ \text{distinct positive odd parts}}} x^{|\alpha|}=x^{r^2}P_r(x^2)=\frac{x^{r^2}}{\prod_{j=1}^{r}(1-x^{2j})}.$$

Consequently,

$$A(x,z)=\prod_{m\geq 1}(1+zx^{2m-1}).$$

Proof. Take a partition ρ with at most r rows and pad it with zeros to have r entries. Double its entries and add the descending odd staircase:

$$\alpha_i=2\rho_i+2(r-i)+1 \qquad (1\leq i\leq r).$$

The entries are positive and odd, and $\alpha_i - \alpha_{i+1} = 2(\rho_i - \rho_{i+1}) + 2 \geq 2$, so they are distinct. Moreover,

$$|\alpha| = 2|\rho| + \sum_{i=1}^{r} \big(2(r-i)+1\big) = 2|\rho| + r^2.$$

Conversely, distinct positive odd parts satisfy $\alpha_i \geq 2(r-i)+1$. Subtracting the staircase and halving therefore gives nonnegative, nonincreasing integers, recovering ρ. This proves the first identity, including the empty partition when $r = 0$.

Multiplying by z^r and summing over r counts all partitions into distinct odd parts, with z recording their number of parts. Each positive odd integer may independently occur once or not at all, so the generating function is $\prod_{m\geq 1}(1 + zx^{2m-1})$. □

Substituting these two factors into (24) now produces the product expansion.

Theorem 222 (Jacobi triple product)**.** *As a formal power series in x with Laurent-polynomial coefficients in y, and analytically for $|x| < 1$ and $y \neq 0$,*

$$\boxed{\sum_{k\in\mathbb{Z}} x^{k^2} y^k = \prod_{m=1}^{\infty} (1-x^{2m})(1+yx^{2m-1})(1+y^{-1}x^{2m-1}).}$$

Proof. The factorization and odd-staircase identity give

$$P(x^2)\Theta(x,y) = \prod_{m\geq 1} (1+yx^{2m-1})(1+y^{-1}x^{2m-1}).$$

Multiply by $P(x^2)^{-1} = \prod_{m\geq 1}(1-x^{2m})$.

All preceding rearrangements are valid coefficientwise. In particular, the smallest power of x contributed by indices r, s in the double sum is $r^2 + s^2$, so only finitely many pairs contribute to any fixed coefficient. The partition series have constant term one and are therefore invertible as formal power series.

For $|x| < 1$ and $y \neq 0$, the series converges absolutely because the quadratic exponent k^2 dominates the linear growth from y^k. The product also converges absolutely, since

$$\sum_{m\geq 1} \big(|x|^{2m} + (|y| + |y|^{-1})|x|^{2m-1}\big) < \infty.$$

Thus the formal identity also gives the stated analytic identity. □

5.10.5 Pentagonal Number Theorem

To obtain $\prod_{m\geq 1}(1-q^m)$ from the Jacobi triple product, choose its three families of factors to cover the three residue classes modulo 3. The corresponding substitution on the series side produces the pentagonal exponents.

Theorem 223 (Pentagonal number theorem)**.** *As an identity of formal power series, and analytically for $|q| < 1$,*

$$\boxed{\prod_{m=1}^{\infty} (1-q^m) = \sum_{k\in\mathbb{Z}} (-1)^k q^{k(3k-1)/2}.}$$

Equivalently,

$$\prod_{m=1}^{\infty} (1-q^m) = 1 + \sum_{k=1}^{\infty} (-1)^k \left(q^{k(3k-1)/2} + q^{k(3k+1)/2}\right).$$

Proof. To avoid fractional powers in the substitution, write $q = t^2$. For $0 < |t| < 1$, apply Theorem 222 with $x = t^3$ and $y = -t^{-1}$. This gives

$$\begin{aligned}\sum_{k\in\mathbb{Z}}(-1)^k t^{3k^2-k} &= \prod_{m=1}^{\infty}(1-t^{6m})(1-t^{6m-4})(1-t^{6m-2}) \\ &= \prod_{m=1}^{\infty}(1-q^{3m})(1-q^{3m-2})(1-q^{3m-1}) \\ &= \prod_{m=1}^{\infty}(1-q^m).\end{aligned}$$

The three families cover each positive integer exponent of q exactly once. Since $k(3k-1)$ is even, $t^{3k^2-k} = q^{k(3k-1)/2}$, proving the first identity. The case $q = 0$ is immediate, and equality of the convergent power series also gives the formal identity. Pairing the terms indexed by k and $-k$ gives the second form. □

The expansion starts $1 - q - q^2 + q^5 + q^7 - q^{12} - q^{15} + \cdots$. Its sparsity gives a recurrence for partition numbers.

Corollary 224 (Partition recurrence)**.** *Let $p(n)$ count partitions of n, with $p(0) = 1$ and $p(n) = 0$ for $n < 0$. For every $n \geq 1$,*

$$\boxed{p(n) = \sum_{k\geq 1}(-1)^{k-1}\left(p\left(n - \frac{k(3k-1)}{2}\right) + p\left(n - \frac{k(3k+1)}{2}\right)\right).}$$

Proof. Choosing the multiplicity of each possible part gives $\sum_{n\geq 0} p(n)q^n = \prod_{m\geq 1}(1-q^m)^{-1}$. Multiply by the pentagonal expansion. The product is 1, so its coefficient of q^n vanishes for $n \geq 1$:

$$p(n) + \sum_{k\geq 1}(-1)^k\left(p\left(n - \frac{k(3k-1)}{2}\right) + p\left(n - \frac{k(3k+1)}{2}\right)\right) = 0.$$

Isolating $p(n)$ gives the recurrence. □

Only $O(\sqrt{n})$ summands can be nonzero for a fixed n, so the recurrence computes $p(0), \ldots, p(N)$ using $O(N^{3/2})$ arithmetic operations. Note that this is not as efficient as just inverting the expansion using FPS inversion, which is $O(N \log N)$, but this may be independently useful.

5.10.6 MacMahon's Master Theorem

Theorem 225. *Let $A = (a_{ij})$ be an $n \times n$ matrix, and let $X = \operatorname{diag}(x_1, \ldots, x_n)$ be a diagonal matrix of formal variables. For any tuple $k = (k_1, \ldots, k_n) \in \mathbb{N}^n$, let $x^k = x_1^{k_1} \cdots x_n^{k_n}$. Then*

$$[x^k]\prod_{i=1}^{n}\left(\sum_{j=1}^{n} a_{ij}x_j\right)^{k_i} = [x^k]\frac{1}{\det(I - AX)}.$$

Proof. Let $R = \mathbb{C}[a_{11}, \ldots, a_{nn}]$ be the polynomial ring representing the generic matrix A. We work in the ring of formal power series $R[[x_1, \ldots, x_n]]$.

Let V be a free module of rank n over R with basis $v_1, \ldots, v_n$. Let $M = AX$, meaning $M_{ij} = a_{ij}x_j$. We define a linear operator $T : V \to V$ by its action on the standard basis:

$$T(v_i) = \sum_{j=1}^{n} M_{ij}v_j.$$

The operator T extends naturally to the symmetric algebra $\mathrm{Sym}(V) \cong R[v_1, \ldots, v_n]$. The subspace $S^m(V)$ of homogeneous polynomials of degree m has a basis of monomials $v^k = v_1^{k_1} \cdots v_n^{k_n}$ where $|k| = \sum_{i=1}^n k_i = m$. The action of T on this basis is:

$$T(v^k) = \prod_{i=1}^n T(v_i)^{k_i} = \prod_{i=1}^n \left(\sum_{j=1}^n a_{ij} x_j v_j \right)^{k_i}.$$

The trace of T acting on $S^m(V)$ is the sum of its diagonal matrix entries in this standard basis:

$$\mathrm{Tr}(S^m(T)) = \sum_{|k|=m} [v^k] T(v^k) = \sum_{|k|=m} [v^k] \prod_{i=1}^n \left(\sum_{j=1}^n a_{ij} x_j v_j \right)^{k_i}.$$

In the polynomial expansion of $\prod_{i=1}^n (\sum_{j=1}^n a_{ij} x_j v_j)^{k_i}$, the variables x_j and v_j are strictly coupled; every term chosen from the inner sum contributes exactly one x_j and one v_j. Therefore, the formal coefficient of v^k is exactly x^k multiplied by the scalar coefficient we would obtain if we evaluated at $v_j = 1$. Thus:

$$[v^k] T(v^k) = x^k \cdot [x^k] \prod_{i=1}^n \left(\sum_{j=1}^n a_{ij} x_j \right)^{k_i}.$$

Summing this trace over all degrees m yields a formal power series:

$$\sum_{m=0}^\infty \mathrm{Tr}(S^m(T)) = \sum_{k \in \mathbb{N}^n} x^k \left([x^k] \prod_{i=1}^n \left(\sum_{j=1}^n a_{ij} x_j \right)^{k_i} \right).$$

We must now prove that this sum evaluates to $\det(I - AX)^{-1}$. Let W be an $n \times n$ matrix of independent formal variables w_{ij}. We will establish the generic trace identity $\sum_{m=0}^\infty \mathrm{Tr}(S^m(W)) = \det(I - W)^{-1}$ in $\mathbb{C}[[w_{11}, \ldots, w_{nn}]]$.

By comparing the homogeneous components of degree m, we define a sequence of polynomials:

$$F_m(W) = \mathrm{Tr}(S^m(W)) - [\text{degree } m \text{ of } \det(I - W)^{-1}].$$

We must prove $F_m(W)$ is identically zero in $\mathbb{C}[w_{11}, \ldots, w_{nn}]$.

Let $\Delta(W)$ be the discriminant of the characteristic polynomial of W with respect to λ. Because there exist matrices with distinct eigenvalues, Δ is a non-zero polynomial.

Let $C \in \mathbb{C}^{n \times n}$ be any specific complex matrix such that $\Delta(C) \neq 0$. This non-vanishing discriminant guarantees that C has n distinct roots, meaning C is diagonalizable. Let its eigenvalues be $\lambda_1, \ldots, \lambda_n$, with a basis of eigenvectors $u_1, \ldots, u_n$.

Because C acts diagonally on V, the induced basis of monomials $u^k = u_1^{k_1} \cdots u_n^{k_n}$ is an eigenbasis for $S^m(C)$, with corresponding eigenvalues $\lambda_1^{k_1} \cdots \lambda_n^{k_n}$. Computing the trace in this eigenbasis yields:

$$\sum_{m=0}^\infty \mathrm{Tr}(S^m(C)) = \sum_{m=0}^\infty \sum_{|k|=m} \lambda_1^{k_1} \cdots \lambda_n^{k_n} = \prod_{i=1}^n \left(\sum_{k_i=0}^\infty \lambda_i^{k_i} \right).$$

By formally evaluating the independent geometric series, we obtain:

$$\prod_{i=1}^n \frac{1}{1 - \lambda_i} = \frac{1}{\det(I - C)}.$$

Thus, $F_m(C) = 0$ for all strictly diagonalizable matrices C. We have established that the polynomial F_m evaluates to zero wherever the non-zero polynomial Δ is non-zero, thus $F_m\Delta = 0$, so $F_m = 0$ as $\Delta \neq 0$ and we're over $\mathbb{C}$.

By substituting the variables $w_{ij} = a_{ij}x_j$ (which corresponds to $W = AX$) back into the identity for W, we obtain:

$$\sum_{m=0}^{\infty} \mathrm{Tr}(S^m(T)) = \frac{1}{\det(I - AX)}.$$

□

5.10.7 Hook-Length Formula

Let

$$\lambda = (\lambda_1 \geq \lambda_2 \geq \cdots) \vdash n$$

be a Young diagram. For a cell $u = (i, j)$, its hook consists of u, the cells to its right, and the cells below it; let $h(u)$ denote its size.

Let f^λ be the number of standard Young tableaux of shape λ.

Theorem 226 (Hook-Length Formula)**.**

$$\boxed{f^\lambda = \frac{n!}{\prod_{u\in\lambda} h(u)}.}$$

Why should hooks appear? At each cell, the row and column conditions combine into one requirement: its entry must be the smallest in its hook. A filling is standard exactly when this holds at every cell.

Among the $n!$ uniformly random fillings, a fixed cell u is smallest in its hook with probability $1/h(u)$. Multiplying these local probabilities suggests

$$f^\lambda = n! \prod_{u\in\lambda} \frac{1}{h(u)}.$$

For a single row, the hook lengths are $n, n-1, \ldots, 1$, and this simply recovers the familiar $1/n!$ probability of being sorted. In a general diagram, the hooks overlap and their minimum-entry conditions are *not independent*. The remarkable content of the formula is that the probability of satisfying all of them nevertheless equals the product of their individual probabilities.

Let r be the number of rows of the original diagram. Throughout the branching argument, we regard every smaller diagram as also having r rows by appending zero rows if necessary. This is only bookkeeping: it lets us use the same coordinates even when the last nonempty row disappears.

Define the shifted row lengths

$$\ell_i = \lambda_i + r - i.$$

Because $\lambda_i \geq \lambda_{i+1}$,

$$\ell_i - \ell_{i+1} = \lambda_i - \lambda_{i+1} + 1 > 0,$$

so

$$\ell_1 > \ell_2 > \cdots > \ell_r.$$

The point of this shift is that it turns the hook lengths in each row into almost all the integers from 1 up to ℓ_i.

Lemma 227 (Hook Product Identity)**.**

$$\boxed{\prod_{u\in\lambda} h(u) = \frac{\prod_{i=1}^{r} \ell_i!}{\prod_{1\le i<j\le r}(\ell_i - \ell_j)}.}$$

Proof. Fix a row i. Read its cells from right to left.

If there were no lower rows, the hook lengths would simply be

$$1, 2, \ldots, \lambda_i.$$

Each lower row changes this sequence once. When we pass the last column occupied by row $j > i$, the next cell gains not only one new cell to its right but also the cell in row j below it. Thus the hook length jumps by 2 instead of 1, skipping one integer.

There are $r - i$ lower rows, so the λ_i hook lengths in row i are obtained from the $\ell_i = \lambda_i + r - i$ integers

$$1, 2, \ldots, \ell_i$$

by deleting exactly $r - i$ values.

The value skipped when crossing the end of row j is

$$\lambda_i - \lambda_j + j - i = \ell_i - \ell_j.$$

Hence

$$\{h(i,1), \ldots, h(i,\lambda_i)\} = \{1, \ldots, \ell_i\} \setminus \{\ell_i - \ell_j : j > i\},$$

and therefore

$$\prod_{q=1}^{\lambda_i} h(i,q) = \frac{\ell_i!}{\prod_{j>i}(\ell_i - \ell_j)}.$$

Multiplying over i proves the identity. □

Define

$$F(\lambda) = n! \, \frac{\prod_{i<j}(\ell_i - \ell_j)}{\prod_i \ell_i!}.$$

By the preceding lemma,

$$F(\lambda) = \frac{n!}{\prod_{u\in\lambda} h(u)}.$$

The remainder of the proof will show that F satisfies the same recurrence as f^λ. The benefit of the padded coordinates now becomes apparent: removing a corner from row i just changes some value,

$$\ell_i \longmapsto \ell_i - 1,$$

even when that removal makes an entire row disappear.

The number of standard tableaux satisfies the branching recurrence

$$f^\lambda = \sum_{\mu \lessdot \lambda} f^\mu,$$

because the largest entry n must lie in a removable corner. We will show that F satisfies the same recurrence.

Lemma 228. *For distinct numbers $\ell_1, \dots, \ell_r$,*

$$\boxed{\sum_{i=1}^{r} \ell_i \prod_{j \neq i} \frac{\ell_i - \ell_j - 1}{\ell_i - \ell_j} = \sum_{i=1}^{r} \ell_i - \binom{r}{2}.}$$

Proof. Let

$$P(x) = \prod_{j=1}^{r} (x - \ell_j)$$

and define

$$Q(x) = x\big(P(x-1) - P(x)\big) + rP(x).$$

Since both $P(x-1)$ and $P(x)$ have leading term x^r, their difference has degree at most $r-1$. The x^r-term in $x(P(x-1) - P(x))$ is $-rx^r$, which is cancelled by $rP(x)$, so

$$\deg Q \leq r - 1.$$

At $x = \ell_i$, we have $P(\ell_i) = 0$, and hence

$$Q(\ell_i) = \ell_i P(\ell_i - 1) = -\ell_i \prod_{j \neq i} (\ell_i - \ell_j - 1).$$

Since Q has degree at most $r-1$, Lagrange interpolation says that its leading coefficient is

$$[x^{r-1}]Q = \sum_{i=1}^{r} \frac{Q(\ell_i)}{P'(\ell_i)}.$$

But

$$P'(\ell_i) = \prod_{j \neq i} (\ell_i - \ell_j),$$

so

$$[x^{r-1}]Q = -\sum_{i=1}^{r} \ell_i \prod_{j \neq i} \frac{\ell_i - \ell_j - 1}{\ell_i - \ell_j}.$$

It remains to compute the same coefficient directly. Write

$$s_1 = \sum_i \ell_i.$$

Then

$$P(x) = x^r - s_1 x^{r-1} + \cdots.$$

Expanding $P(x-1)$ to the first two terms,

$$P(x-1) = x^r - (r + s_1)x^{r-1} + \left(\binom{r}{2} + (r-1)s_1 + \cdots\right) x^{r-2} + \cdots.$$

Hence

$$[x^{r-1}]\left(x(P(x-1) - P(x))\right) = \binom{r}{2} + (r-1)s_1,$$

while

$$[x^{r-1}]\, rP(x) = -rs_1.$$

Therefore

$$[x^{r-1}]Q = \binom{r}{2} - s_1.$$

Comparing the two expressions for $[x^{r-1}]Q$ gives

$$\sum_{i=1}^{r} \ell_i \prod_{j \neq i} \frac{\ell_i - \ell_j - 1}{\ell_i - \ell_j} = s_1 - \binom{r}{2},$$

as claimed. □

Now suppose a removable corner is deleted from row i. Then n becomes $n-1$, and among the shifted row lengths only

$$\ell_i \longmapsto \ell_i - 1$$

changes. Therefore

$$\frac{F(\lambda - \square_i)}{F(\lambda)} = \frac{\ell_i}{n} \prod_{j \neq i} \frac{\ell_i - \ell_j - 1}{\ell_i - \ell_j}.$$

This formula also automatically detects rows from which no corner may be removed. For $i < r$, row i is not removable exactly when $\lambda_i = \lambda_{i+1}$. In that case

$$\ell_i - \ell_{i+1} = \lambda_i - \lambda_{i+1} + 1 = 1,$$

so the factor corresponding to $j = i+1$ is

$$\frac{\ell_i - \ell_{i+1} - 1}{\ell_i - \ell_{i+1}} = 0.$$

For the last padded row, if $\lambda_r = 0$, then $\ell_r = 0$, so the prefactor ℓ_r vanishes. Thus we may sum the ratio over all $i = 1, \ldots, r$; non-removable rows simply contribute 0.

Finally,

$$n = \sum_{i=1}^{r} \lambda_i = \sum_{i=1}^{r} (\ell_i - r + i) = \sum_{i=1}^{r} \ell_i - \binom{r}{2}.$$

Hence Lemma 228 gives

$$\sum_{\mu \lessdot \lambda} F(\mu) = F(\lambda) \sum_{i=1}^{r} \frac{\ell_i}{n} \prod_{j \neq i} \frac{\ell_i - \ell_j - 1}{\ell_i - \ell_j} \qquad = F(\lambda), \frac{1}{n} \left(\sum_i \ell_i - \binom{r}{2} \right) \; = F(\lambda).$$

Thus F satisfies exactly the same branching recurrence as f^λ.

For the empty diagram, $f^\varnothing = 1$. Also

$$F(\varnothing) = \frac{\prod_{i<j}(j-i)}{\prod_{i=1}^{r}(r-i)!} = 1,$$

since

$$\prod_{i<j}(j-i) = \prod_{j=1}^{r}(j-1)!.$$

Therefore, by induction on the number of cells,

$$f^\lambda = F(\lambda).$$

Using Lemma 227,

$$\boxed{f^\lambda = \frac{n!}{\prod_{u\in\lambda} h(u)}.}$$

This proves the hook-length formula.

5.10.8 The Fundamental Theorem of Symmetric Polynomials

A symmetric polynomial depends on a collection of values without depending on their order. The coefficients of $\prod_i (T - x_i)$ are the natural coordinates for such information. The fundamental theorem says that they capture every symmetric polynomial uniquely. Its proof is an elimination algorithm: cancel the largest monomial using a product of elementary symmetric polynomials, then repeat.

Let R be a commutative ring with identity. A polynomial $f \in R[x_1, \dots, x_n]$ is *symmetric* if $f(x_{\sigma(1)}, \dots, x_{\sigma(n)}) = f(x_1, \dots, x_n)$ for every $\sigma \in S_n$. Define the elementary symmetric polynomials by

$$e_0 = 1, \qquad e_k = \sum_{1\le i_1<\cdots<i_k\le n} x_{i_1}\cdots x_{i_k} \quad (1 \le k \le n).$$

Thus $e_1 = x_1 + \cdots + x_n$, $e_n = x_1 \cdots x_n$, and

$$\prod_{i=1}^{n}(T - x_i) = T^n - e_1T^{n-1} + e_2T^{n-2} - \cdots + (-1)^n e_n.$$

Theorem 229 (Fundamental theorem of symmetric polynomials)**.** *For every symmetric $f \in R[x_1, \dots, x_n]$, there is a unique $F \in R[Y_1, \dots, Y_n]$ such that*

$$\boxed{f(x_1, \dots, x_n) = F(e_1, \dots, e_n).}$$

If $\deg f \le D$, every monomial $Y_1^{b_1}\cdots Y_n^{b_n}$ in F has weighted degree $\sum_{k=1}^n kb_k \le D$. If f is homogeneous of degree D, every such monomial has weighted degree exactly D.

Elimination. Order monomials first by total degree and then lexicographically with $x_1 > \cdots > x_n$. Thus among monomials of the same degree, compare exponents from left to right and prefer the larger exponent at the first difference. This order is preserved by multiplication by any fixed monomial.

Suppose $cx_1^{\alpha_1}\cdots x_n^{\alpha_n}$ is the leading term of a nonzero symmetric polynomial. Then $\alpha_1 \ge \cdots \ge \alpha_n$: if $\alpha_i < \alpha_{i+1}$, swapping those two variables produces a strictly larger monomial with the same nonzero coefficient. That contradicts the choice of leading term.

The leading monomial of e_k is $x_1 \cdots x_k$, with coefficient one. Consequently, setting $\alpha_{n+1} = 0$, the product

$$E_\alpha = e_1^{\alpha_1-\alpha_2} e_2^{\alpha_2-\alpha_3} \cdots e_n^{\alpha_n} \tag{25}$$

has leading monomial exactly x^α, again with coefficient one. Indeed, the exponent of x_i in the product of leading monomials is $\sum_{j=i}^n (\alpha_j - \alpha_{j+1}) = \alpha_i$. All other choices of terms give smaller monomials.

Proof of Theorem 229. For existence, subtract cE_α from the current remainder. This cancels its leading term, preserves symmetry, and introduces no monomial of greater total degree: E_α is homogeneous of degree $\sum_i \alpha_i$. If the remainder is nonzero, its leading monomial strictly decreases. There are only finitely many monomials of total degree at most $D = \deg f$, so repetition reaches zero. Recording $cY_1^{\alpha_1-\alpha_2}\cdots Y_n^{\alpha_n}$ at every step produces F. Its weighted degree is the degree of the canceled term, proving both degree assertions. The zero input gives $F = 0$.

For uniqueness, a monomial Y^b becomes a product e^b with leading exponent vector

$$\alpha_i = b_i + b_{i+1} + \cdots + b_n.$$

Different b give different vectors, because $b_i = \alpha_i - \alpha_{i+1}$. In a nonzero polynomial F, choose the term whose substituted leading monomial is largest. No other substituted term can cancel that monomial, and its coefficient stays nonzero because the leading coefficient of e^b is one. Thus $F(e_1, \ldots, e_n) \neq 0$. The substitution map is injective, proving uniqueness even if R has zero divisors. □

Algorithm 28 Rewrite a symmetric polynomial in elementary symmetric coordinates

Input: A symmetric polynomial $f \in R[x_1, \ldots, x_n]$ with exact coefficient operations and zero tests
Output: The unique F satisfying $f = F(e_1, \ldots, e_n)$

$r \leftarrow f$; $F \leftarrow 0$; form $e_1, \ldots, e_n$
while $r \neq 0$ **do**
  Let cx^α be the leading term of r in degree-lexicographic order
  $\alpha_{n+1} \leftarrow 0$; $b_i \leftarrow \alpha_i - \alpha_{i+1}$ for $1 \le i \le n$
  $F \leftarrow F + c\prod_{i=1}^n Y_i^{b_i}$
  $r \leftarrow r - c\prod_{i=1}^n e_i^{b_i}$, expanding and collecting coefficients
return F

Power Sums and Newton's Identities. The power sums $p_r = \sum_{i=1}^n x_i^r$ are another natural description of an unordered collection. Newton's identities convert between these and the e_k. The logarithmic derivative performs the conversion at once: it turns the product defining the e_k into a sum involving the p_r.

Theorem 230 (Newton's identities)**.** *Put $e_k = 0$ for $k > n$. Over any commutative ring, for every $k \ge 1$,*

$$\boxed{ke_k = \sum_{r=1}^{k} (-1)^{r-1} e_{k-r} p_r.}$$

If $n!$ is invertible in R, every symmetric polynomial is uniquely a polynomial in $p_1, \ldots, p_n$ as well.

Proof. Work in the formal power-series ring over $R[x_1, \ldots, x_n]$ and set $E(t) = \prod_i (1 + x_i t) = \sum_{k=0}^n e_k t^k$. Since $E(0) = 1$, this series is invertible. The product rule and geometric series give

$$\frac{E'(t)}{E(t)} = \sum_{i=1}^{n} \frac{x_i}{1 + x_i t} = \sum_{r \ge 1} (-1)^{r-1} p_r t^{r-1}.$$

Multiply by $E(t)$ and compare coefficients of t^{k-1} to obtain the identity.

If $n!$ is invertible, so is every $1 \le k \le n$, and the identity recursively expresses e_k using $p_1, \ldots, p_k$. Conversely, solving for p_k expresses it using $e_1, \ldots, e_k$, with leading term $(-1)^{k-1} k e_k$. These are inverse triangular polynomial changes of coordinates with invertible diagonal coefficients. The uniqueness in Theorem 229 therefore transfers to the power-sum coordinates. □

5.10.9 Robinson–Schensted Correspondence

The Robinson–Schensted correspondence gives a bijection

$$\boxed{S_n \longleftrightarrow \bigsqcup_{\lambda \vdash n} \mathrm{SYT}(\lambda) \times \mathrm{SYT}(\lambda).}$$

Its basic operation is row insertion.

Row Insertion To insert x into a standard tableau P:

1. In the first row, find the leftmost entry $y > x$.
2. Replace y by x and insert y into the next row.
3. If there is no entry larger than x, append x to the row and stop.

Lemma 231. *Row insertion preserves the tableau property and adds exactly one corner cell.*

Proof. Replacing the first entry larger than x preserves increasing order within the row.

Suppose an entry y is bumped from column c into the next row. Because the tableau was column-increasing, the entry directly below y, if it exists, is larger than y. Thus the first entry larger than y in the next row occurs in some column $c' \leq c$.

Hence the bumping path moves weakly to the left as it descends. The entry directly above the newly inserted y is therefore smaller than y, so column order is also preserved.

Eventually an entry is appended at the end of some row. The bumping-path property ensures that this new cell is an outer corner. □

The Correspondence Given

$$\pi = \pi_1 \pi_2 \cdots \pi_n \in S_n,$$

start with empty tableaux P, Q.

For $t = 1, \ldots, n$:

- row-insert π_t into P;
- exactly one new cell is created;
- put t into that cell of Q.

The insertion tableau P and recording tableau Q always have the same shape.

Since later cells receive larger labels and new cells are always outer corners, Q is also standard.

Reverse Insertion Suppose a corner cell of P contains x. Delete that cell. Moving upward one row at a time, replace the rightmost entry $y < x$ by x and set

$$x \leftarrow y.$$

When the first row has been processed, output the final value of x.

Lemma 232. *Reverse insertion is the inverse of row insertion.*

Proof. Suppose forward insertion placed x into a row and bumped y. By definition, y was the leftmost entry larger than x. Equivalently, after the replacement, x is the rightmost entry smaller than y. Thus the reverse rule exactly undoes this step. Apply this successively up the bumping path. □

Theorem 233 (Robinson–Schensted)**.** *The map*

$$\pi \longmapsto (P, Q)$$

is a bijection between permutations of $[n]$ *and pairs of standard Young tableaux of the same shape.*

Proof. Given (P, Q), find the cell of Q containing n. It is a corner. Remove it and reverse-insert from the corresponding corner of P. The output is π_n.

Repeat with

$$n-1, n-2, \ldots, 1.$$

By the preceding lemma this exactly reverses the forward algorithm. □

One immediate consequence is

$$\boxed{n! = \sum_{\lambda \vdash n} (f^\lambda)^2.}$$

Longest Increasing Subsequences

Theorem 234 (Schensted)**.** *If the insertion tableau of* π *has shape* λ*, then*

$$\boxed{\lambda_1 = \mathrm{LIS}(\pi), \qquad \lambda_1' = \mathrm{LDS}(\pi).}$$

Proof. Consider the first row after processing a prefix of the permutation:

$$r_1 < r_2 < \cdots < r_t.$$

We claim that

$$r_j$$

is the smallest possible final value of an increasing subsequence of length j in the processed prefix.

This is true initially. When a new value x is inserted, it replaces the first $r_j > x$, or is appended if none exists. This is exactly the standard update of the minimum possible tail of an increasing subsequence of each length.

Thus the first row has an entry in position j exactly when an increasing subsequence of length j exists. Therefore

$$\lambda_1 = \mathrm{LIS}(\pi).$$

For decreasing subsequences, reverse the order on the alphabet. Row insertion in the reversed order is column insertion in the original order, so it transposes the shape. An increasing subsequence in the reversed order is exactly a decreasing subsequence in the original order. Hence

$$\lambda_1' = \mathrm{LDS}(\pi).$$

□

5.10.10 Ehrhart Polynomials

For a lattice polytope $P \subseteq \mathbb{R}^d$, let

$$L_P(t) = |tP \cap \mathbb{Z}^d|, \qquad t = 0, 1, 2, \ldots.$$

Ehrhart's theorem says that this is actually a polynomial in the dilation factor.

The essential reason is already visible for a simplex. After adding one homogenizing coordinate, dilations of the simplex become horizontal slices of a simplicial cone. Every lattice point of that cone is a unique lattice point in one bounded fundamental parallelepiped plus nonnegative integer combinations of the cone generators. The infinite count therefore contributes only a factor $(1-z)^{-d-1}$.

Theorem 235 (Ehrhart)**.** *Let $P \subseteq \mathbb{R}^d$ be a d-dimensional lattice polytope. Then there is a polynomial $L_P(t)$ of degree d such that*

$$L_P(t) = |tP \cap \mathbb{Z}^d|$$

for every integer $t \geq 0$. Its constant term is 1, and its leading coefficient is the Euclidean volume:

$$\boxed{L_P(t) = \mathrm{vol}(P)t^d + O(t^{d-1}).}$$

Proof. We first treat a lattice simplex

$$S = \mathrm{conv}(v_0, \ldots, v_d).$$

Homogenize its vertices to

$$w_i = (v_i, 1) \in \mathbb{Z}^{d+1}.$$

The w_i are linearly independent, and

$$(x, t) \in \mathbb{Z}^{d+1}$$

satisfies $x \in tS$ exactly when

$$(x, t) = \sum_{i=0}^{d} \lambda_i w_i \qquad \text{for some } \lambda_i \geq 0.$$

Consider the half-open fundamental parallelepiped

$$\Pi = \left\{ \sum_{i=0}^{d} \theta_i w_i : 0 \leq \theta_i < 1 \right\}.$$

Every lattice point y in the cone has a unique representation

$$y = p + \sum_{i=0}^{d} n_i w_i,$$

where $n_i \in \mathbb{Z}_{\geq 0}$ and $p \in \Pi \cap \mathbb{Z}^{d+1}$: simply write $\lambda_i = n_i + \theta_i$ with $n_i = \lfloor \lambda_i \rfloor$.

If $p \in \Pi \cap \mathbb{Z}^{d+1}$, its last coordinate is $\sum_i \theta_i$, hence is an integer in $\{0, \ldots, d\}$. Let h_j^* be the number of such points whose last coordinate is j. The generating function of the horizontal lattice-point counts is therefore

$$\boxed{\sum_{t \geq 0} L_S(t) z^t = \frac{h_0^* + h_1^* z + \cdots + h_d^* z^d}{(1-z)^{d+1}}.} \tag{26}$$

Indeed, each added generator w_i raises the final coordinate by one, giving one geometric factor $1 + z + z^2 + \cdots$.

Since

$$[z^r](1-z)^{-d-1} = \binom{r+d}{d},$$

we obtain

$$\boxed{L_S(t) = \sum_{j=0}^{d} h_j^* \binom{t+d-j}{d}.}$$

Each term is a polynomial in t of degree d, so L_S is a polynomial.

The sum $\sum_j h_j^*$ is the number of lattice points in a fundamental parallelepiped for the lattice generated by $w_0, \ldots, w_d$, hence

$$\sum_j h_j^* = |\det(w_0, \ldots, w_d)|.$$

Subtracting the first column from the others shows

$$|\det(w_0, \dots, w_d)| = |\det(v_1 - v_0, \dots, v_d - v_0)| = d! \operatorname{vol}(S).$$

Since every binomial polynomial above has leading coefficient $1/d!$, the leading coefficient of L_S is $\operatorname{vol}(S)$.

Now triangulate P into lattice simplices $S_1, \dots, S_m$ using vertices of P. Any nonempty intersection of simplices in a triangulation is a common face, hence is itself a lattice simplex. Inclusion–exclusion gives

$$L_P(t) = \sum_{\emptyset \neq I \subseteq [m]} (-1)^{|I|+1} L_{\bigcap_{i \in I} S_i}(t).$$

The same cone argument applies to lower-dimensional faces, so every term is polynomial. Hence L_P is polynomial.

Distinct d-simplices intersect only in lower-dimensional faces, so only the maximal simplices contribute to the coefficient of t^d. That coefficient is

$$\sum_{i=1}^{m} \operatorname{vol}(S_i) = \operatorname{vol}(P).$$

Finally $0P = \{0\}$, so $L_P(0) = 1$. □

Rational polytopes. The same proof explains the rational case essentially for free.

Corollary 236 (Rational Ehrhart theorem)**.** *Suppose every vertex of P lies in $\frac{1}{q}\mathbb{Z}^d$. Then $L_P(t) = |tP \cap \mathbb{Z}^d|$ is a quasipolynomial of period dividing q: for each residue $r \bmod q$, there is a polynomial p_r such that*

$$L_P(t) = p_r(t) \qquad \text{whenever } t \equiv r \pmod{q}.$$

Its degree is $\dim P$*, and its leading coefficient is again* $\operatorname{vol}(P)$*.*

Proof. For a rational simplex, replace the homogenized generators $(v_i, 1)$ by the integral vectors

$$(qv_i, q).$$

The same fundamental-parallelepiped decomposition now gives a generating function of the form

$$\frac{B(z)}{(1 - z^q)^{d+1}}$$

for some polynomial $B(z)$. On each fixed residue class modulo q, its coefficients are polynomial in t. Triangulating a rational polytope into rational simplices completes the proof. □

Ehrhart reciprocity and Pick's theorem. The same cone picture also counts interior lattice points. For a polytope P, write

$$L_{P^\circ}(t) = |tP^\circ \cap \mathbb{Z}^d|.$$

Theorem 237 (Ehrhart reciprocity)**.** *For every d-dimensional lattice polytope P and every integer $t \geq 1$,*

$$\boxed{L_{P^\circ}(t) = (-1)^d L_P(-t).}$$

Proof. For a lattice simplex $S = \operatorname{conv}(v_0, \dots, v_d)$, recall the homogenized generators $w_i = (v_i, 1)$ and the half-open parallelepiped

$$\Pi = \left\{ \sum_{i=0}^{d} \theta_i w_i : 0 \leq \theta_i < 1 \right\}.$$

If h_j^* counts its lattice points of height j, then

$$L_S(t) = \sum_{j=0}^{d} h_j^* \binom{t+d-j}{d}.$$

For interior points of the cone, use instead

$$\Pi^\circ = \left\{ \sum_{i=0}^{d} \theta_i w_i : 0 < \theta_i \leq 1 \right\}.$$

The map

$$p \longmapsto \sum_{i=0}^{d} w_i - p$$

bijects lattice points of Π of height j with lattice points of Π° of height $d+1-j$. Hence

$$L_{S^\circ}(t) = \sum_{j=0}^{d} h_j^* \binom{t+j-1}{d}.$$

Using

$$\binom{t+j-1}{d} = (-1)^d \binom{-t+d-j}{d},$$

we obtain

$$L_{S^\circ}(t) = (-1)^d L_S(-t).$$

As in the proof of Ehrhart's theorem, a lattice triangulation and inclusion–exclusion extend the identity from simplices to arbitrary lattice polytopes. □

In dimension two, reciprocity immediately determines the middle coefficient of the Ehrhart polynomial.

Corollary 238 (Pick's theorem)**.** *Let P be a lattice polygon of area A, with I lattice points in its interior and B lattice points on its boundary. Then*

$$\boxed{A = I + \frac{B}{2} - 1.}$$

Moreover,

$$\boxed{L_P(t) = At^2 + \frac{B}{2}t + 1.}$$

Proof. By Ehrhart's theorem,

$$L_P(t) = At^2 + ct + 1$$

for some c. Evaluating at $t = 1$ counts all lattice points of P, so

$$I + B = A + c + 1.$$

On the other hand, Ehrhart reciprocity in dimension two gives

$$I = L_{P^\circ}(1) = L_P(-1) = A - c + 1.$$

Subtracting the second identity from the first gives $B = 2c$, hence $c = B/2$. Substituting back yields

$$A = I + \frac{B}{2} - 1.$$

□

5.11 Möbius Inversion on Posets

Often an object has an exact type x, but it is easier to count objects whose type is at most y. If $f(x)$ counts exact types, the accessible quantity is $g(y) = \sum_{x \le y} f(x)$. Möbius inversion recovers the exact counts by coefficients depending only on the partial order.

5.11.1 Incidence Algebra and the Möbius Function

A poset P is *locally finite* if every interval $[x, y] = \{z : x \le z \le y\}$ is finite.

An *incidence function* assigns an integer $\alpha(x, y)$ to every comparable pair $x \le y$. Define convolution by

$$(\alpha * \beta)(x, y) = \sum_{x \le z \le y} \alpha(x, z)\beta(z, y).$$

This is matrix multiplication with indices restricted by the order. Local finiteness makes every entry a finite sum. Associativity follows because both parenthesizations expand to

$$\sum_{x \le u \le v \le y} \alpha(x, u)\beta(u, v)\gamma(v, y).$$

The identity is $\delta(x, y) = 1$ when $x = y$ and 0 otherwise. The *zeta function* is $\zeta(x, y) = 1$ for every $x \le y$. These functions, with pointwise addition and convolution, form the incidence algebra $I(P; \mathbb{Z})$.

Theorem 239 (Möbius function)**.** *Every locally finite poset has a unique integer-valued incidence function μ satisfying*

$$\mu * \zeta = \zeta * \mu = \delta.$$

Equivalently,

$$\sum_{x \le z \le y} \mu(x, z) = \sum_{x \le z \le y} \mu(z, y) = \delta(x, y).$$

It is determined recursively by

$$\boxed{\mu(x, x) = 1, \qquad \mu(x, y) = -\sum_{x \le z < y} \mu(x, z) \quad (x < y).}$$

It also has the chain formula

$$\boxed{\mu(x, y) = \sum_{k \ge 0} (-1)^k \#\{x = x_0 < x_1 < \cdots < x_k = y\}.}$$

The length-zero chain is counted only when $x = y$.

Proof. Put $N = \zeta - \delta$, so $N(x, y) = 1$ for $x < y$ and is zero on the diagonal. Repeated convolution shows that $N^{*k}(x, y)$ counts strict chains with k steps from x to y; here $N^{*0} = \delta$. Such a chain uses $k + 1$ distinct elements of $[x, y]$, so $N^{*k}(x, y) = 0$ for $k \ge |[x, y]|$.

Consequently the geometric series

$$\mu = \delta - N + N^{*2} - N^{*3} + \cdots$$

is a finite sum at each interval. Multiplying on either side by $\zeta = \delta + N$ cancels all terms except δ. This proves both inverse identities and the chain formula. An inverse is unique: if ν were another, then $\nu = \nu * (\zeta * \mu) = (\nu * \zeta) * \mu = \mu$. Finally, separating the $z = y$ term in $\mu * \zeta = \delta$ gives the recursion. □

The chains in the formula need not be saturated: a step may skip intermediate elements. The alternating signs undo repeated counting along chains of intermediate types. In a finite poset, after ordering the elements compatibly with $\le$, this is simply the inverse of an upper triangular matrix with ones on the diagonal.

5.11.2 The Inversion Formula

Theorem 240 (Möbius inversion)**.** *Let P be locally finite and let $f, g : P \to A$, where A is an abelian group. Suppose that, below each fixed element, f and g have only finitely many nonzero values. Then*

$$\boxed{g(y) = \sum_{x \le y} f(x) \quad \Longleftrightarrow \quad f(y) = \sum_{x \le y} \mu(x,y) g(x).}$$

Dually, if f and g have only finitely many nonzero values above each fixed element, then

$$\boxed{g(x) = \sum_{y \ge x} f(y) \quad \Longleftrightarrow \quad f(x) = \sum_{y \ge x} \mu(x,y) g(y).}$$

Proof. Suppose $g(y) = \sum_{x \le y} f(x)$. Substitution and interchange of the finite sums give

$$\begin{aligned} \sum_{x \le y} \mu(x,y) g(x) &= \sum_{x \le y} \mu(x,y) \sum_{z \le x} f(z) \\ &= \sum_{z \le y} f(z) \sum_{z \le x \le y} \mu(x,y) \\ &= \sum_{z \le y} f(z) \delta(z,y) = f(y). \end{aligned}$$

Conversely, if $f(y) = \sum_{x \le y} \mu(x,y) g(x)$, then

$$\begin{aligned} \sum_{x \le y} f(x) &= \sum_{x \le y} \sum_{z \le x} \mu(z,x) g(z) \\ &= \sum_{z \le y} g(z) \sum_{z \le x \le y} \mu(z,x) = g(y). \end{aligned}$$

The rearrangements are finite: only finitely many starting points z contribute, and for each one the intermediate points lie in the finite interval $[z, y]$. The upper-sum version follows by reversing the order, whose Möbius function is $\mu_{P^{\mathrm{op}}}(y,x) = \mu_P(x,y)$ by the chain formula. □

On a finite poset the support conditions are automatic. More generally, one may always apply the theorem inside any finite interval $[a, b]$, using sums over $a \le x \le y$ or $x \le y \le b$.

5.11.3 Products of Posets

The recursion shows that $\mu(x,y)$ depends only on the order structure of $[x, y]$. Intervals of the product thus give a useful factorization.

Lemma 241 (Product formula)**.** *For locally finite posets P, Q, ordered coordinatewise on $P \times Q$,*

$$\mu_{P \times Q}\big((x,u),(y,v)\big) = \mu_P(x,y) \mu_Q(u,v).$$

Proof. The proposed function satisfies the defining cancellation identity:

$$\begin{aligned} \sum_{\substack{x \le z \le y \\ u \le w \le v}} \mu_P(x,z) \mu_Q(u,w) &= \left(\sum_{x \le z \le y} \mu_P(x,z) \right) \left(\sum_{u \le w \le v} \mu_Q(u,w) \right) \\ &= \delta(x,y) \delta(u,v). \end{aligned}$$

The recursion uniquely determines the Möbius function, proving the claim. Repetition gives the formula for any finite product. □

5.11.4 Differences, Inclusion–Exclusion, and Divisor Sums

Chains. On the chain $\{0, 1, \dots, n\}$,

$$\mu(i,j) = \begin{cases} 1, & j = i, \\ -1, & j = i+1, \\ 0, & j \geq i+2. \end{cases}$$

Indeed, after the first two values, the preceding values sum to zero, so the recursion gives zero at every longer interval. Thus $g(j) = \sum_{i=0}^{j} f(i)$ is inverted by $f(j) = g(j) - g(j-1)$, with $g(-1) = 0$.

Subsets. For subsets ordered by inclusion, $[S, T]$ is a product of $|T \setminus S|$ two-element chains. Hence

$$\mu(S,T) = (-1)^{|T \setminus S|}, \qquad g(T) = \sum_{S \subseteq T} f(S) \iff f(T) = \sum_{S \subseteq T} (-1)^{|T \setminus S|} g(S).$$

For inclusion–exclusion, let $A_1, \dots, A_n$ be subsets of a finite universe Ω. Let $E(T)$ count elements belonging to exactly the sets indexed by T, and put $C(S) = |\bigcap_{i \in S} A_i|$, with $C(\varnothing) = |\Omega|$. Since $C(S) = \sum_{T \supseteq S} E(T)$, upper-sum inversion gives

$$\left| \Omega \setminus \bigcup_{i=1}^{n} A_i \right| = E(\varnothing) = \sum_{S \subseteq \{1,\dots,n\}} (-1)^{|S|} C(S).$$

Taking $f(S) = a_{|S|}$ similarly gives binomial inversion:

$$b_n = \sum_{k=0}^{n} \binom{n}{k} a_k \quad \iff \quad a_n = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} b_k.$$

Divisibility. Order the positive integers by divisibility. If $d \mid n$ and $n/d = \prod_i p_i^{e_i}$, the interval $[d, n]$ is a product of chains $\{0, 1, \dots, e_i\}$, recording prime exponents. Therefore $\mu(d,n) = \mu_{\text{arith}}(n/d)$, where

$$\mu_{\text{arith}}(m) = \begin{cases} 1, & m = 1, \\ (-1)^r, & m \text{ is a product of } r \text{ distinct primes}, \\ 0, & m \text{ is divisible by a prime square}. \end{cases}$$

Poset inversion becomes the arithmetic Möbius inversion formula

$$\boxed{g(n) = \sum_{d \mid n} f(d) \quad \iff \quad f(n) = \sum_{d \mid n} \mu_{\text{arith}}(n/d) g(d).}$$

The cancellation identity says $\sum_{d \mid m} \mu_{\text{arith}}(d) = 1$ for $m = 1$ and 0 otherwise. In particular,

$$\mathbf{1}_{\gcd(a,b)=1} = \sum_{d \mid \gcd(a,b)} \mu_{\text{arith}}(d).$$

Summing this indicator over $1 \leq a \leq N$ gives the useful counting identity

$$\#\{1 \leq a \leq N : \gcd(a,b) = 1\} = \sum_{d \mid b} \mu_{\text{arith}}(d) \left\lfloor \frac{N}{d} \right\rfloor.$$

5.11.5 Subspaces over a Finite Field

Let V be finite-dimensional over $\mathbb{F}_q$, and order its subspaces by inclusion. For $U \subseteq W$, the map $X \mapsto X/U$ identifies $[U, W]$ with the subspace lattice of W/U. Thus its Möbius function depends only on $d = \dim W - \dim U$.

Theorem 242 (Subspace Möbius function)**.** *If $d = \dim W - \dim U$, then*

$$\boxed{\mu(U, W) = (-1)^d q^{\binom{d}{2}}.}$$

Proof. The number of j-dimensional subspaces of a d-dimensional space is

$$\binom{d}{j}_q = \prod_{i=0}^{j-1} \frac{q^d - q^i}{q^j - q^i}.$$

The numerator counts ordered independent j-tuples, and the denominator counts the ordered bases of each j-dimensional subspace. The finite q-binomial theorem gives

$$\sum_{j=0}^{d} \binom{d}{j}_q q^{\binom{j}{2}} t^j = \prod_{i=0}^{d-1} (1 + tq^i).$$

At $t = -1$, the right side is zero for $d > 0$ and one for $d = 0$. Consequently the proposed values satisfy

$$\sum_{U \subseteq X \subseteq W} (-1)^{\dim X - \dim U} q^{\binom{\dim X - \dim U}{2}} = \sum_{j=0}^{d} \binom{d}{j}_q (-1)^j q^{\binom{j}{2}} = \delta(U, W).$$

This is the defining cancellation identity. □

5.11.6 Set Partitions

For $n \geq 1$, let Π_n be the set of partitions of $\{1, \ldots, n\}$, ordered by refinement: $\pi \leq \sigma$ means that every block of π is contained in a block of σ. Write $\widehat{0}$ for the partition into singletons and $\widehat{1}$ for the one-block partition.

Theorem 243 (Partition-lattice Möbius function)**.** *For $n \geq 1$,*

$$\mu_{\Pi_n}(\widehat{0}, \widehat{1}) = (-1)^{n-1}(n-1)!.$$

More generally, if $\pi \leq \sigma$ and b_B is the number of blocks of π contained in $B \in \sigma$, then

$$\boxed{\mu_{\Pi_n}(\pi, \sigma) = \prod_{B \in \sigma} (-1)^{b_B - 1}(b_B - 1)!.}$$

Proof. An intermediate partition between π and σ independently partitions the b_B blocks of π inside each $B \in \sigma$. Thus $[\pi, \sigma] \cong \prod_{B \in \sigma} \Pi_{b_B}$, so the product formula reduces the claim to the value from bottom to top.

Put $a_j = (-1)^{j-1}(j-1)!$ for $j \geq 1$. For a fixed partition ρ, $\prod_{B \in \rho}(|B| - 1)!$ counts permutations whose cycle blocks are exactly ρ: each block can be made into a cycle in $(|B| - 1)!$ ways. The sign of each such permutation is $\prod_{B \in \rho}(-1)^{|B|-1}$. Therefore

$$\sum_{\rho \in \Pi_n} \prod_{B \in \rho} a_{|B|} = \sum_{\tau \in S_n} \operatorname{sgn}(\tau) = \begin{cases} 1, & n = 1, \\ 0, & n \geq 2. \end{cases}$$

For $n \geq 2$, the last sum vanishes by pairing τ with $(1\,2)\tau$, which has opposite sign.

Now induct on n. The assertion holds for $n = 1$. For $\rho < \widehat{1}$, all its blocks have size less than n, so the interval product and induction give $\mu(\widehat{0}, \rho) = \prod_{B \in \rho} a_{|B|}$. The displayed zero-sum identity then forces $\mu(\widehat{0}, \widehat{1}) = a_n$ by the Möbius recursion. The general interval formula follows from the product decomposition. □

5.11.7 Weighted Inversion

The same algebra also inverts weighted cumulative sums. Allow incidence functions to take values in any ring R with identity, using the same convolution. Multiplication in R need not be commutative.

Theorem 244 (Invertibility in the incidence algebra)**.** *An incidence function $\alpha \in I(P;R)$ has a two-sided convolution inverse if and only if every diagonal value $\alpha(x,x)$ is a unit of R.*

Proof. Necessity follows by evaluating both inverse identities at (x,x). For sufficiency, let D agree with α on the diagonal and be zero elsewhere. Its inverse D^{-1} has diagonal entries $\alpha(x,x)^{-1}$. Put $K = D^{-1} * \alpha - \delta$. Then K vanishes on the diagonal and $\alpha = D * (\delta + K)$. As before, $K^{*j}(x,y) = 0$ for $j \geq |[x,y]|$, because every nonzero term requires a strict chain. Hence

$$\alpha^{-1} = \big(\delta - K + K^{*2} - K^{*3} + \cdots\big) * D^{-1}$$

is well-defined entrywise and is a two-sided inverse. □

For R-valued functions with finite support below each fixed element, associativity now gives

$$g(y) = \sum_{x \leq y} f(x)\alpha(x,y) \quad \Longleftrightarrow \quad f(y) = \sum_{x \leq y} g(x)\alpha^{-1}(x,y).$$

The order of the factors is part of the formula. Ordinary Möbius inversion is the special case $\alpha = \zeta$.

6 Convexity and Duality

6.1 Sion's Minimax Theorem

A function g on a convex set is *quasiconvex* if

$$g((1-t)x + ty) \leq \max\{g(x), g(y)\} \qquad (t \in [0,1]),$$

equivalently, if every sublevel set $\{x : g(x) \leq a\}$ is convex.

It is *quasiconcave* if

$$g((1-t)x + ty) \geq \min\{g(x), g(y)\},$$

equivalently, if every superlevel set $\{x : g(x) \geq a\}$ is convex.

Sion's theorem says that quasiconvexity in the minimizing variable and quasiconcavity in the maximizing variable are enough for minimax equality.

The proof is easiest to understand at a fixed threshold a. For a choice y, call x *good for* y if $f(x,y) \leq a$. Quasiconvexity in x says that the good points for each fixed y form a convex set.

The key fact is that if two choices y_0, y_1 have no common good point, then some convex combination of them is bad for *every* x. Quasiconcavity in y is exactly what makes this possible.

Lemma 245 (Two-Point Reduction)**.** *Let X be compact and convex, let $y_0, y_1 \in Y$, and set*

$$y_t = (1-t)y_0 + ty_1.$$

Suppose that $f(\cdot, y)$ is quasiconvex on X for every y, that $f(x,\cdot)$ is quasiconcave on Y for every x, and that f is continuous.

Set

$$M = \min_{x \in X} \max\{f(x,y_0), f(x,y_1)\}.$$

Then for some $t \in [0,1]$,

$$\boxed{\min_{x \in X} f(x,y_t) \geq M.}$$

Proof. Define

$$U_i = \{x \in X : f(x, y_i) < M\}, \qquad i \in \{0, 1\},$$

and

$$D_t = \{x \in X : f(x, y_t) < M\}.$$

By the definition of M,

$$U_0 \cap U_1 = \emptyset.$$

Indeed, if some x belonged to both, then

$$\max\{f(x, y_0), f(x, y_1)\} < M,$$

contradicting the definition of M.

Now quasiconcavity in y gives

$$f(x, y_t) \geq \min\{f(x, y_0), f(x, y_1)\}.$$

Hence if $f(x, y_t) < M$, then at least one of $f(x, y_0), f(x, y_1)$ is less than M. Therefore

$$D_t \subseteq U_0 \cup U_1.$$

On the other hand, D_t is convex, since it is a strict sublevel set of the quasiconvex function $f(\cdot, y_t)$. In particular, D_t is connected. Since U_0 and U_1 are disjoint open subsets of X, every nonempty D_t must therefore lie entirely in one of them:

$$D_t \subseteq U_0 \qquad \text{or} \qquad D_t \subseteq U_1.$$

If some D_t is empty, then $f(x, y_t) \geq M$ for every x, and we are done. Suppose therefore that D_t is nonempty for every $t \in [0, 1]$.

Define

$$I = \{t : D_t \subseteq U_0\}, \qquad J = \{t : D_t \subseteq U_1\}.$$

Then I and J are disjoint and cover $[0, 1]$. Moreover, $D_0 = U_0$ and $D_1 = U_1$, so $0 \in I$ and $1 \in J$.

We claim that I is closed. Let $t_k \in I$ with $t_k \to t$. Choose $x \in D_t$, which is possible by assumption. Since $f(x, y_t) < M$, continuity gives

$$f(x, y_{t_k}) < M$$

for all sufficiently large k. Hence

$$x \in D_{t_k} \subseteq U_0,$$

so D_t meets U_0. But D_t lies entirely in either U_0 or U_1, and therefore $D_t \subseteq U_0$. Thus $t \in I$.

The same argument shows that J is closed. This writes the connected interval $[0, 1]$ as a disjoint union of two nonempty closed sets, a contradiction.

Hence $D_t = \emptyset$ for some t, and consequently

$$\min_x f(x, y_t) \geq M.$$

□

The lemma says that two incompatible choices of y can be replaced by a single convex combination that is at least as bad for the minimizer as the better of the two choices was pointwise.

Lemma 246 (Finite Reduction)**.** *Let $y_1, \dots, y_m \in Y$ and $a \in \mathbb{R}$. If*

$$\bigcap_{i=1}^{m} \{x \in X : f(x, y_i) \le a\} = \emptyset,$$

then there is

$$\bar{y} \in \operatorname{conv}\{y_1, \dots, y_m\}$$

such that

$$\boxed{\min_{x \in X} f(x, \bar{y}) > a.}$$

Proof. We induct on m.

For $m = 1$, the hypothesis says $f(x, y_1) > a$ for every $x \in X$. Compactness and continuity give

$$\min_x f(x, y_1) > a.$$

Now suppose $m > 1$, and let

$$X' = \{x \in X : f(x, y_m) \le a\}.$$

This is compact and convex: it is a closed sublevel set of the continuous quasiconvex function $f(\cdot, y_m)$.

If $X' = \emptyset$, then $\bar{y} = y_m$ works.

Otherwise, within X' the first $m - 1$ sublevel sets have empty intersection. By the inductive hypothesis, there is

$$y' \in \operatorname{conv}\{y_1, \dots, y_{m-1}\}$$

such that

$$\min_{x \in X'} f(x, y') > a.$$

Thus for every $x \in X$,

$$\max\{f(x, y'), f(x, y_m)\} > a.$$

Indeed, if $x \in X'$, then $f(x, y') > a$; while if $x \notin X'$, then $f(x, y_m) > a$.

Since X is compact,

$$M := \min_{x \in X} \max\{f(x, y'), f(x, y_m)\} > a.$$

Applying Lemma 245 to y' and y_m, there is some point $\bar{y}$ on the segment joining them such that

$$\min_x f(x, \bar{y}) \ge M > a.$$

Finally,

$$\bar{y} \in \operatorname{conv}\{y_1, \dots, y_m\}.$$

□

Theorem 247 (Sion's Minimax Theorem)**.** *Let X be a nonempty compact convex set and let Y be a nonempty convex set, both contained in finite-dimensional real vector spaces. Let*

$$f : X \times Y \to \mathbb{R}$$

be continuous.

Suppose that for every $y \in Y$, the function $f(\cdot, y)$ is quasiconvex on X, and for every $x \in X$, the function $f(x, \cdot)$ is quasiconcave on Y. Then

$$\boxed{\min_{x \in X} \sup_{y \in Y} f(x, y) = \sup_{y \in Y} \min_{x \in X} f(x, y).}$$

If Y is also compact, the supremum is attained, and hence

$$\boxed{\min_{x \in X} \max_{y \in Y} f(x, y) = \max_{y \in Y} \min_{x \in X} f(x, y).}$$

Proof. Set

$$\alpha = \sup_{y \in Y} \min_{x \in X} f(x, y).$$

Weak minimax always gives

$$\alpha \leq \min_{x} \sup_{y} f(x, y).$$

We prove the reverse inequality.

For every $y \in Y$, define

$$C_y = \{x \in X : f(x, y) \leq \alpha\}.$$

Each C_y is closed and convex.

We claim that the family $\{C_y : y \in Y\}$ has the finite intersection property. Suppose instead that for some $y_1, \ldots, y_m \in Y$,

$$C_{y_1} \cap \cdots \cap C_{y_m} = \emptyset.$$

By Lemma 246, there would be some

$$\bar{y} \in \operatorname{conv}\{y_1, \ldots, y_m\} \subseteq Y$$

such that

$$\min_{x} f(x, \bar{y}) > \alpha.$$

This contradicts the definition

$$\alpha = \sup_{y} \min_{x} f(x, y).$$

Thus every finite subfamily has nonempty intersection. Since X is compact,

$$\bigcap_{y \in Y} C_y \neq \emptyset.$$

Choose x^* in this intersection. Then

$$f(x^*, y) \leq \alpha \qquad \text{for every } y \in Y,$$

so

$$\sup_{y} f(x^*, y) \leq \alpha.$$

Consequently,

$$\min_{x} \sup_{y} f(x, y) \leq \alpha.$$

Together with weak minimax,

$$\sup_{y} \min_{x} f(x, y) \leq \min_{x} \sup_{y} f(x, y) \leq \alpha = \sup_{y} \min_{x} f(x, y),$$

and therefore equality holds. □

6.2 Linear Programming Duality

Sion's theorem makes LP duality into a direct calculation.

Consider the standard-form primal LP

minimize	$c^T x$
subject to	$Ax = b,$
	$x \geq 0.$

Its dual is

$$\boxed{\begin{array}{ll}\text{maximize} & b^T y \\ \text{subject to} & A^T y \le c.\end{array}}$$

The variable y is unrestricted.

Theorem 248 (LP Strong Duality)**.** *If the primal LP is feasible and has a finite optimum, then the dual is feasible, attains its optimum, and*

$$\min_{\substack{Ax=b \\ x\ge 0}} c^T x = \max_{A^T y \le c} b^T y.$$

Proof. Let x^* be an optimal primal solution, and choose M so large that

$$0 \le x_i^* < M \qquad \text{for every } i.$$

Set

$$X = [0, M]^n.$$

Define the Lagrangian

$$L(x, y) = c^T x + y^T (b - Ax).$$

For fixed x,

$$\sup_{y \in \mathbb{R}^m} L(x, y) = \begin{cases} c^T x, & Ax = b, \\ +\infty, & Ax \ne b. \end{cases}$$

Therefore

$$\min_{x \in X} \sup_y L(x, y) = c^T x^*.$$

The set X is compact and convex, and L is linear in each variable, so Sion's theorem gives

$$c^T x^* = \sup_y \min_{x \in X} L(x, y).$$

For fixed y, write

$$r = c - A^T y.$$

Then

$$L(x, y) = b^T y + r^T x,$$

and hence

$$q(y) := \min_{x \in X} L(x, y) = b^T y + M \sum_{i=1}^{n} \min(0, r_i).$$

This is a concave polyhedral function of y. Since its supremum is finite, it attains that supremum; let y^* be a maximizer. Thus

$$q(y^*) = c^T x^*.$$

Because x^* is primal feasible,

$$L(x^*, y^*) = c^T x^*.$$

But

$$q(y^*) = \min_{x \in X} L(x, y^*) = c^T x^*,$$

so x^* minimizes $L(\cdot, y^*)$ over the box X.

Now every coordinate satisfies

$$x_i^* < M.$$

If some

$$r_i = c_i - (A^T y^*)_i$$

were negative, increasing x_i^* slightly would remain inside X and would strictly decrease $L(x, y^*)$, contradicting minimality. Therefore

$$r_i \geq 0 \qquad \text{for every } i,$$

or equivalently,

$$A^T y^* \leq c.$$

Thus y^* is dual feasible.

Since $r \geq 0$,

$$q(y^*) = \min_{0 \leq x \leq M} \left(b^T y^* + r^T x \right) = b^T y^*.$$

Consequently,

$$b^T y^* = q(y^*) = c^T x^*,$$

which proves strong duality. □

Weak duality is even simpler: if x is primal feasible and y is dual feasible, then

$$b^T y = x^T A^T y \leq x^T c = c^T x.$$

The signs in the other standard forms can be recovered without memorization.

Free primal variables. If x_j is unrestricted, write

$$x_j = x_j^+ - x_j^-, \qquad x_j^+, x_j^- \geq 0.$$

The two corresponding dual inequalities are

$$(A^T y)_j \leq c_j \qquad \text{and} \qquad -(A^T y)_j \leq -c_j,$$

so together they give

$$(A^T y)_j = c_j.$$

Primal inequality constraints. For a minimization problem,

$$Ax \geq b$$

can be written

$$Ax - s = b, \qquad s \geq 0.$$

The dual constraint corresponding to s is

$$-y \leq 0,$$

hence

$$y \geq 0.$$

Similarly,

$$Ax \leq b$$

gives

$$y \leq 0.$$

Thus the standard correspondences for a minimization primal are

Primal	Dual
$x_j \geq 0$	$(A^T y)_j \leq c_j$
x_j free	$(A^T y)_j = c_j$
$(Ax)_i = b_i$	y_i free
$(Ax)_i \geq b_i$	$y_i \geq 0$
$(Ax)_i \leq b_i$	$y_i \leq 0$.

Negating the objective gives the familiar maximization form:

$$\boxed{\begin{aligned} &\text{maximize} && c^T x \\ &\text{subject to} && Ax \leq b, \\ & && x \geq 0 \end{aligned}} \quad \Longleftrightarrow \quad \boxed{\begin{aligned} &\text{minimize} && b^T y \\ &\text{subject to} && A^T y \geq c, \\ & && y \geq 0. \end{aligned}}$$

6.3 Lagrangian Relaxation

Let $F(k)$ be the minimum cost of a feasible object using exactly k units of a resource. Instead of enforcing the count, assign each unit a price λ and compute $G(\lambda) = \min_k (F(k) + \lambda k)$. This can remove a cardinality dimension from an algorithm. Discrete convexity of F is what makes the constrained value recoverable from the penalized values.

Theorem 249 (Discrete Lagrangian duality)**.** *Let $F : I \to \mathbb{R}$, where I is a finite interval of integers, and put $G(\lambda) = \min_{k \in I}(F(k) + \lambda k)$. For every $K \in I$ and $\lambda \in \mathbb{R}$,*

$$G(\lambda) - \lambda K \leq F(K).$$

Equality can be attained for every $K \in I$ if and only if F is discretely convex: its differences $\Delta_k = F(k) - F(k-1)$ are nondecreasing.

In that case,

$$\boxed{F(K) = \max_{\lambda \in \mathbb{R}} \big(G(\lambda) - \lambda K\big),}$$

and a price is optimal exactly when

$$\boxed{-\Delta_{K+1} \leq \lambda \leq -\Delta_K,}$$

with inequalities involving missing endpoint differences omitted.

Proof. The lower bound follows by using $k = K$ in the minimum defining G. Equality holds precisely when K minimizes $F(k) + \lambda k$.

Suppose F is discretely convex. For $j > K$,

$$F(j) + \lambda j - F(K) - \lambda K = \sum_{i=K+1}^{j} (\Delta_i + \lambda) \geq 0$$

when $\lambda \geq -\Delta_{K+1}$. For $j < K$, the same difference is $-\sum_{i=j+1}^{K} (\Delta_i + \lambda) \geq 0$ when $\lambda \leq -\Delta_K$. Thus the stated prices are sufficient; comparison with the adjacent counts shows they are necessary.

Conversely, if $2F(K) > F(K-1) + F(K+1)$, then for every λ,

$$G(\lambda) - \lambda K \leq \frac{F(K-1) + F(K+1)}{2} < F(K).$$

So exactness at every count requires discrete convexity. □

The optimal price is the negative slope of a supporting line to F at K. At a fixed price, the minimizing counts form an integer interval: the discretely convex sequence $F(k) + \lambda k$ cannot rise between two minima. Thus a solver may skip K when breaking ties while still returning the value needed to recover $F(K)$. Recovering a witness of count exactly K is a separate task.

Corollary 250 (Integer Lagrangian search)**.** *Suppose F is integer-valued and discretely convex, with $|F(k)| \le B$, where $B \ge 0$ is an integer. If $G(\lambda)$ can be evaluated in time T, then $F(K)$ can be computed in*

$$\boxed{O(T \log(B+2))}$$

time using only integer prices.

Proof. Every marginal is an integer in $[-2B, 2B]$. The supporting interval in Theorem 249 therefore contains an integer price in $[-2B-1, 2B+1]$.

The function $D(\lambda) = G(\lambda) - \lambda K$ is concave, since G is a minimum of affine functions. Its integer first differences are consequently nonincreasing.

Initialize $\ell = -2B-1$, $r = 2B+1$. While $\ell < r$, set $m = \lfloor (\ell + r)/2 \rfloor$. If $D(m) < D(m+1)$, replace ℓ by $m+1$; otherwise replace r by m. The first case discards no maximizer, and the second retains a leftmost maximizer. Return $D(\ell)$ when the interval becomes a singleton. □

Certifying convexity by minimum-cost flow.

Theorem 251 (Convexity of minimum-cost flow)**.** *Fix a directed network with distinguished vertices s, t, finite integral lower and upper capacities, and real edge costs. Let $C(k)$ be the minimum cost of an integral s-t flow of value k.*

The feasible integer values form an interval, and C is discretely convex on that interval:

$$\boxed{2C(k) \le C(k-1) + C(k+1).}$$

Consequently, if another problem has exact-count value $F(k) = C(k) + \alpha k + \beta$, then F is discretely convex.

Proof. A convex combination of flows of values $a < b$ gives a fractional flow of every intermediate value. At an integer value, flow integrality gives an integral feasible flow. Thus the feasible integer values form an interval.

Now average optimal flows of values $k-1$ and $k+1$. The average is a feasible fractional flow of value k, with cost $(C(k-1) + C(k+1))/2$. By integrality of the fixed-value flow polytope, an integral optimum costs no more. This proves the inequality.

Adding an affine function does not change second differences. □

A reduction to minimum-cost k-flow is therefore useful even when flow is not used to solve the problem: it certifies the convexity needed by the faster penalized dynamic program. The reduction must preserve the optimum for every relevant k using one fixed network.

Several constraints by repeated one-dimensional relaxation. For several integer resources, the same search can be nested. The required condition is that convexity survives each preceding relaxation, not merely that the original value function is convex along its coordinate lines.

Theorem 252 (Repeated Lagrangian relaxation)**.** *Let $\mathcal{X}$ be finite, with integer costs $c(X)$, integer resource counts $k_1(X), \dots, k_d(X)$, and a feasible target $K = (K_1, \dots, K_d)$.*

For each j and each choice of integer prices $\lambda_1, \dots, \lambda_{j-1}$, define the partially relaxed one-dimensional value function

$$H_j(t; \lambda_{<j}) = \min_{\substack{X \in \mathcal{X} \\ k_j(X) = t \\ k_i(X) = K_i \ (i > j)}} \left(c(X) + \sum_{i<j} \lambda_i (k_i(X) - K_i) \right).$$

Assume its attainable values of t form an integer interval and $H_j(\,\cdot\,;\lambda_{<j})$ is discretely convex on that interval. Then

$$\boxed{\min_{\substack{X\in\mathcal{X}\\ k(X)=K}} c(X) = \max_{\lambda_1\in\mathbb{Z}}\cdots\max_{\lambda_d\in\mathbb{Z}}\min_{X\in\mathcal{X}}\left(c(X)+\sum_{i=1}^{d}\lambda_i(k_i(X)-K_i)\right).}$$

Suppose additionally that $|c(X)|\le B$ and $|k_i(X)-K_i|\le R$, where $B\ge 0$, $R\ge 1$ are integers. At search depth j, it suffices to use

$$L_j=(2B+1)(2R+1)^{j-1},\qquad -L_j\le\lambda_j\le L_j.$$

If the fully penalized minimum takes time T, nested searches compute the constrained optimum in

$$\boxed{O_d\left(T\prod_{j=1}^{d}\log(L_j+2)\right)}$$

time.

Proof. Let

$$V_j(\lambda_1,\dots,\lambda_j)=\min_{\substack{X\in\mathcal{X}\\ k_i(X)=K_i\ (i>j)}}\left(c(X)+\sum_{i=1}^{j}\lambda_i(k_i(X)-K_i)\right).$$

Thus V_0 is the requested optimum and V_d is the fully penalized problem.

Fix the first $j-1$ prices. By definition,

$$V_{j-1}=H_j(K_j;\lambda_{<j}),\qquad V_j=\min_t\big(H_j(t;\lambda_{<j})+\lambda_j(t-K_j)\big).$$

The profile is integer-valued and discretely convex, so Theorem 249 gives

$$\boxed{V_{j-1}(\lambda_{<j})=\max_{\lambda_j\in\mathbb{Z}}V_j(\lambda_{<j},\lambda_j).}$$

Applying these identities successively proves the nested formula.

For the search bounds, suppose the previous prices satisfy $|\lambda_i|\le L_i$. Every finite value of H_j then has absolute value at most

$$B_j=B+R\sum_{i<j}L_i.$$

The stated geometric choice satisfies $L_j=2B_j+1$. Hence Corollary 250 permits the j-th search inside $[-L_j,L_j]$.

To evaluate a search value V_j, recursively perform the remaining searches; at depth d, call the fully penalized solver. Each depth uses $O(\log(L_j+2))$ evaluations of the next depth, giving the product bound. □

To verify the hypothesis, it is enough to reduce each profile $H_j(t;\lambda_{<j})$, with the earlier prices fixed, to minimum-cost t-flow up to an affine term in t. The network and its costs may depend on the fixed earlier prices, but must remain fixed as t varies.

6.4 Seidel's Algorithm

We consider low-dimensional linear programming. A linear program may have many constraints, but in fixed dimension only a few are needed to determine its optimum. Seidel's algorithm exploits this by inserting the constraints in random order. Most insertions leave the current answer unchanged. When a constraint does invalidate it, the new optimum lies on that constraint's boundary, reducing the dimension by one. The analysis runs this observation backward: an insertion can matter only if deleting that constraint changes the answer.

Consider minimizing $c^\top x$ subject to m inequalities $a_i^\top x \le b_i$ in $\mathbb{R}^d$. First impose a fixed box $[-R, R]^d$, where $R > 0$. Every nonempty intermediate feasible region is then compact. Among its objective minimizers, choose the lexicographically smallest coordinate vector. Equivalently, minimize

$$(c^\top x, x_1, \ldots, x_d) \quad \text{lexicographically.}$$

This gives a unique answer. We remove the artificial box at the end.

Lemma 253 (Boundary reduction)**.** *Let x be the canonical optimum before adding $a^\top z \le b$, and suppose $a^\top x > b$. If the new problem is feasible, its canonical optimum lies on $a^\top z = b$. Moreover, the new problem is infeasible exactly when its restriction to that boundary is infeasible.*

Proof. Let y be the new optimum. It is feasible for the old problem and differs from x, so its objective-and-coordinate tuple is lexicographically larger. If $a^\top y < b$, moving a sufficiently small positive distance from y toward x preserves every constraint, including the new one, and strictly decreases that tuple. This contradicts optimality.

If any new feasible point exists, the segment from it to x crosses the boundary while remaining feasible for all old constraints. Thus a feasible boundary point exists exactly when a new feasible point exists. □

The recursive problem therefore uses the old constraints restricted to one hyperplane.

Eliminate one coordinate. Suppose $a \neq 0$, and choose the *last* index j with $a_j \neq 0$. On the new boundary,

$$x_j = \frac{b - \sum_{\ell \neq j} a_\ell x_\ell}{a_j}.$$

Substituting in an inequality $u^\top x \le v$ replaces it by

$$\widetilde{u}_\ell = u_\ell - \frac{u_j}{a_j} a_\ell \quad (\ell \neq j), \qquad \widetilde{v} = v - \frac{u_j}{a_j} b. \tag{27}$$

The reduced objective has coefficients $\widetilde{c}_\ell = c_\ell - c_j a_\ell / a_j$; its omitted constant does not affect optimization. The remaining coordinates retain their box bounds. The two eliminated bounds $x_j \le R$ and $-x_j \le R$ must also be substituted and included as inequalities.

Choosing the last nonzero coordinate preserves the coordinate tie rule. The eliminated x_j depends only on earlier coordinates, since $a_\ell = 0$ for $\ell > j$. If two reduced points first differ before j, that is also their first difference after lifting. If they first differ after j, all earlier coordinates, including the determined x_j, agree. Thus lexicographic order on the remaining coordinates is exactly the order inherited from the original space.

Algorithm 29 Restriction and randomized incremental linear programming

```
function RESTRICT((u, v), (a, b), j)
    λ ← u_j / a_j
    return (u − λa with coordinate j deleted,  v − λb)

function SEIDEL(c, H, R)
    d ← dim c
    if d = 0 then
        return the empty vector if every (u, v) ∈ H has v ≥ 0; otherwise return Infeasible
    x_ℓ ← −R if c_ℓ ≥ 0, and x_ℓ ← R otherwise, for ℓ = 1, ..., d
    Randomly permute H as ((a_1, b_1), ..., (a_m, b_m))
    for i = 1, ..., m do
        if a_i^⊤ x > b_i then
            if a_i = 0 then
                return Infeasible
            j ← max{ℓ : (a_i)_ℓ ≠ 0}
            H' ← {RESTRICT((a_t, b_t), (a_i, b_i), j) : 1 ≤ t < i}
            Append RESTRICT((e_j, R), (a_i, b_i), j) and RESTRICT((−e_j, R), (a_i, b_i), j) to H'
            c' ← c − (c_j/(a_i)_j) a_i, with coordinate j deleted
            y ← SEIDEL(c', H', R)
            if y = Infeasible then
                return Infeasible
            Insert coordinate x_j = (b_i − Σ_{ℓ≠j} (a_i)_ℓ y_ℓ)/(a_i)_j into y to obtain x
    return x
```

In the lifting line, coordinates of y are indexed by the surviving indices $\ell \neq j$. Every recursive call uses a fresh uniform permutation, including the two substituted box inequalities. Initialization is the canonical optimum of the box alone: a zero objective coefficient chooses $-R$ by the tie rule.

Theorem 254 (Seidel's algorithm)**.** *For fixed d, Algorithm 29 solves the boxed linear program, or reports infeasibility, in expected $O_d(m+1)$ arithmetic operations. It is correct for every sequence of random choices. The same bound holds for an unrestricted linear program using the symbolic-box implementation below.*

Why only a few constraints can matter. The following consequence of Helly's theorem replaces a general-position assumption that exactly d constraints meet at the optimum.

Lemma 255 (Small certificates)**.** *In dimension q, with a fixed compact box, a feasible system has at most q inequalities that already determine its canonical optimum. An infeasible system has at most $q+1$ inequalities that are infeasible together with the box.*

Proof. Let x be the canonical optimum, and let U_x consist of points whose objective-and-coordinate tuple is lexicographically smaller than that of x. This set is convex: a positive convex combination of two lexicographically negative difference vectors is still lexicographically negative. The box, all input halfspaces, and U_x have empty intersection. Helly gives an empty subfamily of at most $q+1$ sets. That subfamily must include U_x, since x belongs to every other set. Thus at most q input halfspaces, together with the box, already exclude every better point. They contain x, so they determine exactly its canonical optimum.

For an infeasible system, apply Helly directly to the box and input halfspaces. At most $q+1$ input halfspaces suffice, whether or not the box occurs in the empty subfamily. □

These certificates are used only for analysis; the algorithm does not compute them.

Proof of Theorem 254 for a fixed box. Correctness follows by induction on dimension and on the processed prefix. A satisfied inequality preserves the old optimum. A violated nonconstant inequality reduces to its

boundary by Lemma 253, and (27) describes that problem exactly. Its canonical solution lifts to the new canonical optimum. A violated constant inequality, or an infeasible recursive problem, certifies infeasibility.

For the expected work, condition on the first i inequalities as an *unordered* set. The last of them is uniform among these i choices. If the prefix is feasible, fix a certificate of at most q inequalities. Removing an inequality outside it preserves the optimum, so that inequality cannot trigger recursion when inserted last. If the prefix is infeasible, fix an infeasibility certificate of at most $q+1$ inequalities. Removing anything outside it leaves an infeasible prefix, in which case the algorithm would have stopped before reaching this insertion. Therefore

$$\boxed{\Pr[\text{insertion } i \text{ triggers recursion}] \le \min\{1, (q+1)/i\}.}$$

For a feasible prefix the sharper bound is q/i.

Let $T_q(m)$ bound expected work in dimension q. Scanning and shuffling cost $O(q(m+1))$. At insertion i, restriction costs $O(q(i+1))$ and creates a $(q-1)$-dimensional problem with $i+1$ inequalities. Summing over insertions,

$$T_q(m) \le O(q^2(m+1)) + \sum_{i=1}^{m} \frac{q+1}{i} T_{q-1}(i+1), \qquad T_0(m) = O(m+1).$$

If $T_{q-1}(s) \le K_{q-1}(s+1)$, then $\sum_{i=1}^{m}(i+2)/i \le 3m$ gives $T_q(m) \le K_q(m+1)$ with $K_q = 3(q+1)K_{q-1} + O(q^2)$. This proves linear dependence on m for fixed dimension; for example, it gives the explicit bound $O(3^d(d+1)!(m+1))$. □

Remove the box. Use a formal parameter $R = \Lambda$ and interpret comparisons as holding for all sufficiently large positive Λ. Every right-hand side and every point coordinate remains affine in Λ: normal vectors and objective coefficients never depend on it, so restriction only multiplies affine expressions by ordinary scalars. Store an expression $u + \Lambda v$ as the pair (u, v) and compare two expressions by their v coefficients first, then their u coefficients. Thus every symbolic operation has constant arithmetic overhead.

An Infeasible result means the boxed system is infeasible for all sufficiently large boxes, hence the original system is infeasible. Otherwise write the returned point as $x(\Lambda) = u + \Lambda v$. It is feasible and optimal in the box for all sufficiently large Λ. Consequently $a_i^\top v \le 0$ for every original inequality, and if $a_i^\top v = 0$, then $a_i^\top u \le b_i$. Choose

$$\Lambda_0 = \max\left(\{1\} \cup \left\{ \frac{a_i^\top u - b_i}{-a_i^\top v} : a_i^\top v < 0 \right\}\right).$$

Then $x_0 = u + \Lambda_0 v$ satisfies every original inequality. The objective slope determines the outcome:

$$\begin{cases} c^\top v < 0: & x_0 + tv \ (t \ge 0) \text{ certifies unboundedness below,} \\ c^\top v = 0: & x_0 \text{ is an optimum of the original problem.} \end{cases}$$

Indeed, every fixed feasible point lies in the box for all sufficiently large Λ, so its objective is at least $c^\top u + \Lambda c^\top v$. This rules out a positive slope; for zero slope it proves global optimality, and for negative slope the displayed ray is feasible and improving.

For fixed input there are only finitely many possible recursive executions, and every comparison in them is eventually constant as Λ grows.

6.5 Majorization

Theorem 256 (Hardy-Littlewood-Pólya)**.** *Let $x, y \in \mathbb{R}^n$. If $x \prec_w y$, then there exists a doubly stochastic matrix M such that $x \le My$. If $x \prec y$ then $x = My$.*

Proof. Let $\mathcal{M}$ be the compact, convex set of $n \times n$ doubly stochastic matrices, and let $\mathcal{V} = [0,1]^n$, which is also compact and convex. Consider the bilinear function $f(M, v) = v^T(My - x)$.

For any fixed $v \in \mathcal{V}$, let $P \in \mathcal{M}$ be a permutation matrix that orders the components of y comonotonically with v. Because $x \prec y$, an application of summation by parts yields $v^T x \le v^T P y$. Consequently, we can always choose $M = P$ to see that $\max_{M \in \mathcal{M}} f(M, v) \ge f(P, v) \ge 0$.

By Sion's Minimax Theorem:

$$\max_{M\in\mathcal{M}}\min_{v\in\mathcal{V}} v^T(My-x) = \min_{v\in\mathcal{V}}\max_{M\in\mathcal{M}} v^T(My-x) \geq 0$$

This guarantees the existence of some $M_0 \in \mathcal{M}$ such that $v^T(M_0y - x) \geq 0$ for all possible vectors $v \in \mathcal{V}$.

Applying this inequality with the standard basis vectors $v = e_i \in \mathcal{V}$ implies that $M_0y \geq x$ component-wise. However, because M_0 is doubly stochastic, the sum of the components of M_0y exactly equals the sum of the components of y. By the definition of majorization, $\sum y_i = \sum x_i$ if $x \prec y$, thus $M_0y = x$. □

6.5.1 Muirhead's Inequality

For

$$\alpha = (\alpha_1, \ldots, \alpha_n) \in \mathbb{R}^n_{\geq 0}$$

and

$$x = (x_1, \ldots, x_n) \in \mathbb{R}^n_{\geq 0},$$

define the symmetric monomial mean

$$\boxed{[\alpha](x) = \frac{1}{n!}\sum_{\pi\in S_n} x_1^{\alpha_{\pi(1)}} \cdots x_n^{\alpha_{\pi(n)}}.}$$

We average over all permutations, including repetitions when some α_i's are equal.

Muirhead's inequality says that making the exponent vector more unequal increases its symmetric monomial mean.

Theorem 257 (Muirhead's Inequality)**.** *Let*

$$\alpha, \beta \in \mathbb{R}^n_{\geq 0}$$

satisfy

$$\beta \prec \alpha.$$

Then for every

$$x_1, \ldots, x_n \geq 0,$$

$$\boxed{[\beta](x) \leq [\alpha](x).}$$

Proof. We first assume

$$x_i > 0 \qquad (i = 1, \ldots, n);$$

the general case follows by continuity.

By Hardy–Littlewood–Pólya,

$$\beta = M\alpha$$

for some doubly stochastic matrix M.

By the Birkhoff–von Neumann theorem, every doubly stochastic matrix is a convex combination of permutation matrices, so

$$M = \sum_{k=1}^{N} \theta_k P_k, \qquad \theta_k \geq 0, \qquad \sum_k \theta_k = 1.$$

Therefore

$$\beta = \sum_k \theta_k P_k \alpha.$$

Fix a permutation $Q \in S_n$. Then

$$Q\beta = \sum_k \theta_k Q P_k \alpha.$$

Writing

$$x^\gamma = \prod_{i=1}^{n} x_i^{\gamma_i},$$

we obtain

$$\begin{aligned} x^{Q\beta} &= x^{\sum_k \theta_k Q P_k \alpha} \\ &= \prod_k \left(x^{Q P_k \alpha}\right)^{\theta_k}. \end{aligned}$$

Weighted AM–GM gives

$$\boxed{x^{Q\beta} \le \sum_k \theta_k x^{Q P_k \alpha}.}$$

Now average this inequality over all $Q \in S_n$. The left-hand side gives

$$\frac{1}{n!} \sum_Q x^{Q\beta} = [\beta](x).$$

For each fixed P_k, as Q ranges over S_n, so does QP_k. Hence

$$\frac{1}{n!} \sum_Q x^{Q P_k \alpha} = [\alpha](x).$$

Therefore

$$\begin{aligned} [\beta](x) &\le \sum_k \theta_k \left(\frac{1}{n!} \sum_Q x^{Q P_k \alpha} \right) \\ &= \sum_k \theta_k [\alpha](x) \\ &= [\alpha](x). \end{aligned}$$

This proves Muirhead for positive x_i, and continuity gives the result when some $x_i = 0$. □

6.6 SMAWK and Totally Monotone Matrices

We want the leftmost minimum of every row of an implicit $m \times n$ matrix A, where any entry can be evaluated in constant time. Scanning every row costs $O(mn)$. If optimal columns move only to the right as rows increase, solved rows constrain the search in nearby rows. SMAWK adds a second idea: first discard columns until only as many as there are rows remain. Alternating column reduction with recursion on half the rows gives $O(m + n)$ time.

Total monotonicity. Call A *totally monotone for leftmost row minima* if

$$\boxed{A_{i,a} > A_{i,b} \implies A_{j,a} > A_{j,b} \qquad (i < j,\ a < b).} \tag{28}$$

Thus once a later column becomes strictly better than an earlier one, it stays strictly better in every lower row. The contrapositive says that if the earlier column is at least as good in a lower row, it is at least as good in every upper row.

Lemma 258. *Condition (28) holds exactly when, in every submatrix obtained by selecting rows and columns in their original orders, the leftmost row-minimum columns are nondecreasing.*

Proof. Suppose two rows of such a submatrix had leftmost minimum columns b and a, respectively, with $a < b$. In the upper row i, leftmostness gives $A_{i,a} > A_{i,b}$. Total monotonicity then gives $A_{j,a} > A_{j,b}$ in the lower row, contradicting its minimum at a. Conversely, a failure of (28) gives a 2×2 submatrix whose leftmost minima are in the upper-right and lower-left corners, violating the stated monotonicity. □

Total monotonicity, not merely monotonicity of the original row minima, is needed because the algorithm repeatedly deletes columns.

Monge matrices. A real matrix is *Monge* if

$$A_{i,a} + A_{j,b} \leq A_{i,b} + A_{j,a} \qquad (i < j,\ a < b).$$

Rearranging gives $A_{i,a} - A_{i,b} \leq A_{j,a} - A_{j,b}$, which immediately implies (28).

Column reduction. Consider ordered row and column lists $R = (r_0, \ldots, r_{q-1})$ and $C = (c_0, \ldots, c_{s-1})$, with $q, s > 0$. Indices in these lists start at zero. Scan the columns from left to right and maintain a stack S of at most q candidate columns. Its key invariant is

the column in stack position h cannot be a leftmost minimum in rows $r_0, \ldots, r_{h-1}$.

The stack position counts how many upper rows have already ruled out that column.

If the stack top is in position h, compare it with the new column at row r_h. If the new column is strictly better, the old top loses in every lower row as well, and the invariant has already ruled it out above: discard it. Otherwise, the new column loses, including by ties, in every row through r_h. It can be pushed into position $h + 1$, or discarded if all rows have now been ruled out.

Algorithm 30 Remove columns

```
1:  function REDUCE(R, C)
2:      q ← |R|; S ← an empty stack
3:      for each c ∈ C, in increasing order do
4:          while S ≠ ∅ do
5:              h ← |S| − 1
6:              if A_{r_h,c} < A_{r_h,S[h]} then
7:                  Pop the last column from S
8:              else
9:                  break
10:         if |S| < q then
11:             Push c onto S
12:     return S
```

Lemma 259 (Column reduction)**.** *Algorithm 30 takes $O(|C|)$ time and returns at most $|R|$ columns containing the leftmost minimum of every row of $A[R, C]$.*

Proof. We verify the invariant and show that every discarded column is incapable of being a leftmost minimum anywhere in the original submatrix $A[R, C]$.

Suppose a column d is popped from position h because a later column c is strictly better in row r_h. By total monotonicity, c is strictly better in every row r_i with $i \geq h$. For $i < h$, the invariant already excludes d. Thus popping it is safe.

Suppose instead the new column c is pushed into position $h > 0$. The loop stopped because the preceding stack column d satisfies $A_{r_{h-1},d} \leq A_{r_{h-1},c}$, with $d < c$. The contrapositive of total monotonicity gives $A_{r_i,d} \leq A_{r_i,c}$ for all $i < h$. Because d is earlier, c cannot be the leftmost minimum in those rows, establishing

the invariant. At position zero the invariant is empty. If the stack is already full, the same comparison at its last row rules out c in all q rows, so omitting it is safe.

Every omitted column is therefore excluded from all row optima. Each input column is pushed at most once and popped at most once. Every successful while-loop comparison pops a column, and each input column causes at most one unsuccessful comparison. Hence the total number of comparisons and stack operations is $O(|C|)$. □

Solve half the rows. After reduction, call the remaining columns C'. Recursively solve rows $r_1, r_3, r_5, \ldots$ using only C'. Total monotonicity places the minimum of an unsolved row between the minima of its two solved neighbors. At the first or last row, the missing neighbor is replaced by the first or last column of C'.

Scan each such interval from left to right. The intervals for successive unsolved rows overlap only at their shared endpoint, so their total length, counting repetitions, is $O(|C'| + |R|)$. The crucial point is that reduction has made $|C'| \le |R|$: each recursive level now pays only for its shrinking number of rows.

Use a global answer array p, with $p[r]$ the original column index of row r's minimum, and one workspace array pos indexed by original column indices. The latter translates answers back into positions in the current reduced list.

Algorithm 31 SMAWK for all leftmost row minima

Input: Totally monotone matrix A accessed by entry evaluations; answer array p and workspace pos allocated once

```
1:  function SMAWK(R, C)
2:      if R = ∅ then
3:          return
4:      C ← REDUCE(R, C); q ← |R|; s ← |C|
5:      if q = 1 then
6:          p[r_0] ← c_0; return
7:      SMAWK((r_1, r_3, r_5, ...), C)
8:      for j = 0, ..., s − 1 do
9:          pos[c_j] ← j
10:     for i = 0, 2, 4, ... < q do
11:         L ← 0 if i = 0, otherwise pos[p[r_{i−1}]]
12:         U ← s − 1 if i + 1 = q, otherwise pos[p[r_{i+1}]]
13:         p[r_i] ← c_L
14:         for j = L + 1, ..., U do
15:             if A_{r_i,c_j} < A_{r_i,p[r_i]} then
16:                 p[r_i] ← c_j
```

Call the routine with $R = (0, \ldots, m-1)$ and $C = (0, \ldots, n-1)$, where $n \ge 1$.

Theorem 260 (SMAWK)**.** *For a totally monotone $m \times n$ matrix with $n \ge 1$, Algorithm 31 finds every leftmost row minimum using $O(m+n)$ entry evaluations, $O(m+n)$ additional operations, and $O(m+n)$ space. In particular, constant-time entry access gives linear time.*

Proof. Induct on the number of rows. Reduction preserves every leftmost minimum. With one row, it leaves exactly its minimum column. Otherwise, the recursive call is on a submatrix and hence is still totally monotone. Induction gives the correct minima for its rows. By Lemma 258, each remaining minimum is inside its scanned interval. Scanning left to right and changing the answer only on strict improvement finds its leftmost occurrence.

For the cost, write $T(q, s)$ for one call. Reduction takes $O(s)$, and interpolation plus the position map takes $O(q+s)$, so

$$T(q, s) \le T(\lfloor q/2 \rfloor, \min\{q, s\}) + O(q+s).$$

The first call costs $O(m+n)$. At depth $h \geq 1$, there are at most $m/2^h$ rows and $m/2^{h-1}$ columns. Summing this geometric sequence gives total time $O(m+n)$. The stored row and column lists satisfy the same bounds, and the answer and position arrays use $O(m+n)$ space, proving the space claim. □

6.7 Concave Min-Plus Convolution

6.7.1 Description

Given an array a of length n (which is concave, meaning its forward differences are monotonically decreasing) and an arbitrary array b of length m, the concave min-plus convolution computes:

$$c[p] = \min_{0 \leq x < n,\, 0 \leq y < m,\, x+y=p} (a[x] + b[y])$$

We give an $O(m \log n + n)$ solution by chunking the array b and using a modified Li Chao Tree.

The algorithm processes b in chunks of size up to n. For a chunk starting at index s, we compute its contributions to a window of length $2n$ in the output array c using two passes:

- **Forward Pass:** Evaluates pairs where the relative index in a is greater than or equal to the relative index in b. It inserts functions $f_k(i) = a[i-k] + b[s+k]$ into a Li Chao tree and queries them for $i \geq k$.
- **Reverse Pass:** Symmetrically handles the remaining cross-terms where the relative index in a is strictly less than the relative index in b, sweeping backwards.

6.7.2 Crossings

Lemma 261 (Single Crossing Property)**.** *For a concave array a, define the extended functions for $k \in [0, n)$:*

$$f_k(i) = \begin{cases} -\infty - k & \text{if } i < k \\ a[i-k] + b[s+k] & \text{if } i \geq k \end{cases}$$

For any $j < k$, the difference $D(i) = f_k(i) - f_j(i)$ transitions from negative to positive at most once.

Proof. We evaluate the difference $D(i)$ across three domains:

1. **For $i < j$:** Both are artificial. $D(i) = (-\infty - k) - (-\infty - j) = j - k < 0$.
2. **For $j \leq i < k$:** f_j is finite, but f_k is artificial. $D(i) = -\infty < 0$.
3. **For $i \geq k$:** Both are finite. $D(i) = a[i-k] - a[i-j] + C$. Because a is concave, its slope is decreasing. Since $i - k < i - j$, the difference $a[i-k] - a[i-j]$ strictly monotonically increases with i.

Because $D(i)$ is strictly negative for all $i < k$ and monotonically increases for all $i \geq k$, the functions f_j and f_k intersect at most once globally. □

6.7.3 Boundary Conditions

While Lemma 261 proves the functions cross at most once, their shapes are highly irregular due to the jump at $i = k$.

A standard Li Chao Tree assumes that if a function is better at the left endpoint L and worse at the midpoint M, the single continuous crossing must lie within $[L, M]$, and recursively routes the function leftward.

A function f_k might be artificially better at L (because it's about $-\infty$), worse at M, but then become the true minimum on the right side of the interval.

For a node spanning the discrete interval $[L, R)$, let z be the current resident function and x be the newly inserted function.

- If $x(L) \geq z(L)$ and $x(R-1) \geq z(R-1)$, then x is dominated at both strict boundaries. By Lemma 261, it must be dominated everywhere inside the interval. We safely discard x.
- If $x(L) < z(L)$ and $x(R-1) < z(R-1)$, then x dominates z everywhere in the interval. We replace z with x and discard z.
- Otherwise, a crossing exists within the interval. We swap x and z if necessary to maintain the best function at the midpoint, and recursively push the loser to the appropriate child.

6.8 Rectangular Monotone Min-Plus Product

Let A be $n \times k$ and B be $k \times n$, with nondecreasing rows in B. We want $C_{ij} = \min_t(A_{it} + B_{tj})$. Encoding a weight a as X^a lets ordinary matrix multiplication count candidate sums, but large weights make the polynomials expensive. We will reduce the exponents modulo a small random prime. The product then counts both correct sums and false matches due to the reduction; monotonicity lets us find and subtract the false matches in entire intervals.

Theorem 262 (Row-monotone rectangular min-plus product)**.** *Suppose $B \in \{0, \ldots, U\}^{k \times n}$ has nondecreasing rows, where $U \geq 1$, and A has integer entries or $+\infty$. Write $\mathsf{M} = \mathsf{MM}(n, k, n)$ for an available ordinary matrix multiplication bound, including input/output work. Then $A \star B$ can be computed exactly in expected time*

$$\boxed{\widetilde{O}\left(n^2 + nk + \sqrt{nkU\,\mathsf{M}}\right).}$$

The random choice affects running time, not correctness. The tilde suppresses logarithms in the dimensions and integer magnitudes; word-operation bounds assume polynomially bounded input integers.

Normalize. Subtract the smallest finite entry from each row of A and save it for restoration at the end. The normalized answer is in $[0, U]$, since each row now has a zero entry. Entries of A greater than U cannot improve that answer, so replace them by $+\infty$. Entirely infinite rows are answered separately. Henceforth all finite entries and all answers lie in $[0, U]$, and every row of A has a zero.

Rounding. For $\ell \geq 0$, round the entries:

$$A^{(\ell)} = \left\lfloor A/2^\ell \right\rfloor, \qquad B^{(\ell)} = \left\lfloor B/2^\ell \right\rfloor, \qquad C^{(\ell)} = A^{(\ell)} \star B^{(\ell)}.$$

Infinite entries stay infinite. Every $C^{(\ell)}$ has nondecreasing rows and lies in $[0, \lfloor U/2^\ell \rfloor]$. For $h = \lceil \log_2(U+1) \rceil$, all finite rounded inputs are zero, so $C^{(h)} = 0$. Starting there, recover $C^{(0)}$ one level at a time.

Each finite rounded entry satisfies $A^{(\ell)}_{it} = 2A^{(\ell+1)}_{it} + \alpha_{it}$ with $\alpha_{it} \in \{0, 1\}$, and similarly for B. Every candidate sum therefore doubles and increases by zero, one, or two. Taking minima gives

$$\boxed{2C^{(\ell+1)}_{ij} \leq C^{(\ell)}_{ij} \leq 2C^{(\ell+1)}_{ij} + 2.}$$

The lower bound holds for every candidate; the upper bound uses a minimizer at the preceding level. Thus a refinement needs to determine which of three consecutive integers is the smallest attainable sum.

Count Modulo a Prime, Then Subtract False Matches. Fix a prime p. During one refinement abbreviate $a = A^{(\ell)}$, $b = B^{(\ell)}$, and $D = C^{(\ell+1)}$. Form polynomial matrices

$$\mathcal{A}_{it} = X^{a_{it} \bmod p}, \qquad \mathcal{B}_{tj} = X^{b_{tj} \bmod p}, \qquad \mathcal{H} = \mathcal{A}\mathcal{B} \pmod{X^p - 1},$$

using zero at infinite entries. Reduction modulo $X^p - 1$ folds exponents with the same residue together. Consequently

$$[X^{r \bmod p}]\mathcal{H}_{ij} = \#\{t : a_{it} + b_{tj} \equiv r \pmod p\}.$$

Compute the ordinary polynomial product first and then fold its coefficients. Its degree is at most $2p-2$, so this costs $\widetilde{O}(p\mathsf{M})$. The prime p reduces weights, not coefficient counts.

For $\delta \in \{0,1,2\}$, let $r_{ij}^{\delta} = 2D_{ij} + \delta$. The coefficient at this residue counts the desired exact matches plus indices whose discrepancy is a *nonzero* multiple of p:

$$E_{ij}^{\delta} = \#\{t : a_{it} + b_{tj} - r_{ij}^{\delta} \in p\mathbb{Z} \setminus \{0\}\}.$$

After subtracting them, put $N_{ij}^{\delta} = [X^{r_{ij}^{\delta} \bmod p}]\mathcal{H}_{ij} - E_{ij}^{\delta}$. The refinement is simply

$$\boxed{C_{ij}^{(\ell)} = 2D_{ij} + \min\{\delta \in \{0,1,2\} : N_{ij}^{\delta} \neq 0\}.} \tag{29}$$

At least one of the three counts is nonzero by the rounding bound. The only remaining problem is to subtract false matches without examining all n^2k triples.

False Match Search. For fixed i, t, δ, the discrepancy is $a_{it} + b_{tj} - (2D_{ij} + \delta)$. It is a difference of two nondecreasing sequences. The difference itself need not be monotone, but the endpoints of a column interval bound every value inside it.

More generally, take an integer a and nondecreasing integer sequences $u_1, \ldots, u_N$ and $v_1, \ldots, v_N$. On $[L, R]$, every $a + u_j - v_j$ belongs to

$$[\, a + u_L - v_R,\ a + u_R - v_L \,].$$

If this interval contains no nonzero multiple of p, discard it. If both sequences are constant on the interval, report the entire interval or nothing. Otherwise bisect and repeat.

Algorithm 32 Report intervals of false matches

```
function REPORT(a, u, v, p, L, R)
    q_- ← ⌈(a + u_L − v_R)/p⌉; q_+ ← ⌊(a + u_R − v_L)/p⌋
    if q_- > q_+ or q_- = q_+ = 0 then
        return
    if u_L = u_R and v_L = v_R then
        Emit [L, R]; return
    M ← ⌊(L + R)/2⌋
    REPORT(a, u, v, p, L, M)
    REPORT(a, u, v, p, M + 1, R)
```

Here $1 \le L \le R \le N$. Monotonicity makes equal endpoint values certify constancy, so the emitted intervals are disjoint and cover exactly the desired indices.

Lemma 263 (Expected cost of endpoint pruning)**.** *Fix a, u, v independently of p, and write $\mathcal{V} = (u_N - u_1) + (v_N - v_1)$ for their total variation. For a uniformly chosen prime $p \in [P, 2P]$, with P above an absolute constant, Algorithm 32 runs in expected time*

$$\boxed{\widetilde{O}(1 + \mathcal{V}/P),}$$

including the time to emit its intervals.

Proof. Consider the full balanced interval tree, fixed before choosing p. A nonconstant node $I = [L, R]$ has bounding-interval width

$$w_I = (u_R - u_L) + (v_R - v_L) \ge 1.$$

It can recurse only if p divides some nonzero integer in that bounding interval. There are at most $w_I + 1 \le 2w_I$ such integers. If their magnitudes are bounded by $W \ge 2$, each has at most $\log W / \log P$ prime divisors in $[P, 2P]$. There are $\Omega(P/\log P)$ primes in this range, so a union bound gives

$$\Pr[I \text{ recurses}] = O(w_I \log W / P),$$

with the bound capped at one.

At a fixed depth the intervals are disjoint. By monotonicity, $\sum_I w_I \le \mathcal{V}$: their variations are disjoint portions of the total variation of the two sequences. Summing over the $O(\log(2N))$ depths bounds the expected number of nodes that recurse by $O(\mathcal{V} \log W \log(2N)/P)$. Every visited node except the root is a child of a node that recurses, so the number of calls is at most one plus twice this number. Constant intervals never recurse, even when they contain many matches. Each call and each emitted interval take constant time. □

Apply the reporter to an output row. For each finite (i, t) and each δ, call the reporter with

$$a = a_{it}, \qquad u_j = b_{tj}, \qquad v_j = 2D_{ij} + \delta.$$

A reported interval contributes one false match to every output column in it. Accumulate these contributions in three difference arrays per output row: an interval $[L, R]$ adds 1 at L and -1 at $R + 1$. Prefix sums then give all E^{δ}_{ij} in $O(n^2)$ additional work.

Algorithm 33 Row-monotone min-plus product

Input: Normalized A, B as above and a prime p

$h \leftarrow \lceil \log_2(U + 1) \rceil$; $D \leftarrow 0$
for $\ell = h - 1, h - 2, \ldots, 0$ **do**
  $a \leftarrow \lfloor A/2^{\ell} \rfloor$; $b \leftarrow \lfloor B/2^{\ell} \rfloor$
  Compute $\mathcal{H} = (X^{a \bmod p})(X^{b \bmod p}) \bmod (X^p - 1)$
  Initialize three false-count difference arrays per output row
  for each finite a_{it} and $\delta \in \{0, 1, 2\}$ **do**
    for each $[L, R]$ emitted by Report$(a_{it}, b_{t,*}, 2D_{i,*} + \delta, p, 1, n)$ **do**
      Add 1 at L and -1 at $R + 1$ in array (i, δ)
  Prefix-sum the arrays to obtain E^{δ}_{ij}
  Subtract from the three queried coefficients to obtain N^{δ}_{ij}
  $D'_{ij} \leftarrow 2D_{ij} + \min\{\delta : N^{\delta}_{ij} \ne 0\}$ for all i, j
  $D \leftarrow D'$
return D, restoring the saved row offsets

Correctness and the Running Time.

Proof of Theorem 262. Initially $D = C^{(h)} = 0$. If $D = C^{(\ell+1)}$, the polynomial product counts modular matches and the reporter subtracts exactly those that are not exact matches. The three-value rounding bound then proves (29). Induction gives $D = C^{(0)}$ at termination, for *any* prime p.

Choose one prime uniformly from $[P, 2P]$ and reuse it at every level. All rounded inputs and all exact products $C^{(\ell)}$ are determined by the original matrices, independently of p. Thus the reporting lemma applies directly.

At level ℓ, the reporter's two sequences have total variation at most $2U/2^{\ell}$. Summing its expected cost over i, t, δ gives $\widetilde{O}(nk + nkU/(2^{\ell}P))$. There are $O(\log(U + 1))$ levels, and $\sum_{\ell \ge 0} 2^{-\ell} \le 2$. Together with the polynomial products and output work, the total is

$$\widetilde{O}\left(n^2 + nk + \frac{nkU}{P} + P\mathsf{M}\right).$$

Choosing P within a constant factor of $\sqrt{nkU/\mathsf{M}}$ balances the last two terms. Primes up to $2P$ can be enumerated by a sieve in $\widetilde{O}(P)$ time and a prime in the interval sampled uniformly.

If the balanced scale is below the absolute constant required by the lemma, use the direct monotone sweep instead. Each row of B has at most $U+1$ constant intervals; for every i,t, each interval supplies one range-minimum update $C_{ij} \leftarrow \min\{C_{ij}, A_{it}+B_{tj}\}$. Store each update on the $O(\log n)$ canonical nodes of a segment tree, then propagate minimum tags to its leaves. This costs $\widetilde{O}(n^2+nk+nkU)$, which is within the claimed bound when $nkU/\mathsf{M} = O(1)$. This also covers small U. □

For $k = n^\beta$, $U = O(n^\mu)$, and an admissible exponent $\omega_\beta = \omega(1,\beta,1)$, the bound is $\widetilde{O}(n^2 + n^{1+\beta} + n^{(1+\beta+\mu+\omega_\beta)/2})$. One may always take the better of this bound and the direct $O(n^2k)$ algorithm.

6.8.1 Column-Monotone Right Matrices

If B is instead nondecreasing down its columns, replace each normalized row of A by prefix minima $\widehat{A}_{it} = \min_{s\le t} A_{is}$. This preserves the product: $\widehat{A} \le A$ gives one inequality, and a minimizing $s \le t$ gives

$$\widehat{A}_{it} + B_{tj} = A_{is} + B_{tj} \ge A_{is} + B_{sj} \ge C_{ij}.$$

Thus rows of $\widehat{A}$ are nonincreasing; an initial infinite prefix is skipped. Rounding preserves this order.

Use the same three-value refinement and polynomial products. For each output (i,j) and offset δ, now report *middle-index* intervals, taking

$$a_0 = -(2D_{ij}+\delta), \qquad u_t = b_{tj}, \qquad v_t = -a_{it}.$$

Both sequences are nondecreasing on the finite suffix of row i. A reported interval contributes its length to the single false count E^δ_{ij}, rather than a range update to an output row. Their total variation is again at most $2U/2^\ell$, but there are n^2 reporter calls per offset instead of nk. The same analysis, with the analogous direct sweep over middle-index constant intervals when the prime scale is small, gives

$$\boxed{\widetilde{O}\Big(n^2 + nk + \sqrt{n^2U\,\mathsf{M}}\Big).}$$

Transposition gives the corresponding bounds for a monotone left matrix.

6.9 Radon's Theorem

Theorem 264 (Radon's Theorem)**.** *Any $d+2$ points*

$$x_1, \dots, x_{d+2} \in \mathbb{R}^d$$

can be partitioned into two nonempty sets whose convex hulls intersect.

Proof. The lifted vectors

$$(x_1,1), \dots, (x_{d+2},1) \in \mathbb{R}^{d+1}$$

are linearly dependent. Thus there are coefficients λ_i, not all zero, such that

$$\sum_i \lambda_i x_i = 0, \qquad \sum_i \lambda_i = 0.$$

Let

$$P = \{i : \lambda_i > 0\}, \qquad N = \{i : \lambda_i < 0\}.$$

Both are nonempty. Set

$$L = \sum_{i\in P} \lambda_i = -\sum_{i\in N} \lambda_i > 0.$$

Then

$$\sum_{i \in P} \frac{\lambda_i}{L} x_i = \sum_{i \in N} \frac{-\lambda_i}{L} x_i.$$

Each side is a convex combination, so this common point lies in both convex hulls. □

6.10 Helly's Theorem

Theorem 265 (Helly's Theorem)**.** *Let*

$$C_1, \ldots, C_m \subseteq \mathbb{R}^d$$

be convex sets, with $m \geq d+1$. If every $d+1$ of them have nonempty intersection, then

$$\bigcap_{i=1}^{m} C_i \neq \emptyset.$$

Proof. We induct on m. The case $m = d+1$ is exactly the hypothesis.

Suppose $m > d+1$. By induction, deleting any one of the sets leaves a family with nonempty intersection. For

$$i = 1, \ldots, d+2,$$

choose

$$x_i \in \bigcap_{j \neq i} C_j.$$

Apply Radon's theorem to

$$x_1, \ldots, x_{d+2}.$$

There is a partition

$$P \sqcup N = [d+2]$$

and a point

$$x \in \operatorname{conv}\{x_i : i \in P\} \cap \operatorname{conv}\{x_i : i \in N\}.$$

We claim $x \in C_j$ for every j.

If $j \leq d+2$, one of P, N does not contain j. Every x_i in that part belongs to C_j, so by convexity its convex hull, and hence x, lies in C_j.

If $j > d+2$, then every x_i lies in C_j, so again $x \in C_j$.

Thus

$$x \in \bigcap_{j=1}^{m} C_j.$$

□

6.11 Carathéodory's Theorem

Theorem 266 (Carathéodory's Theorem)**.** *If*

$$x \in \operatorname{conv}(S) \subseteq \mathbb{R}^d,$$

then x lies in the convex hull of at most $d+1$ points of S.

Proof. Choose a representation

$$x = \sum_{i=1}^{m} \alpha_i x_i, \qquad \alpha_i > 0, \qquad \sum_i \alpha_i = 1,$$

using as few points as possible.

Suppose

$$m > d + 1.$$

Then the lifted vectors

$$(x_i, 1) \in \mathbb{R}^{d+1}$$

are linearly dependent, so there are λ_i, not all zero, with

$$\sum_i \lambda_i x_i = 0, \qquad \sum_i \lambda_i = 0.$$

Replace

$$\alpha_i$$

by

$$\alpha_i - t\lambda_i.$$

This does not change either

$$\sum_i \alpha_i \quad \text{or} \quad \sum_i \alpha_i x_i.$$

Choose

$$t = \min_{\lambda_i > 0} \frac{\alpha_i}{\lambda_i}.$$

Then all new coefficients remain nonnegative and at least one becomes zero. This gives a convex representation of x using fewer points, a contradiction.

Hence

$$m \leq d + 1.$$

□

6.12 Colorful Carathéodory

Theorem 267 (Colorful Carathéodory)**.** *Let*

$$C_1, \ldots, C_{d+1} \subseteq \mathbb{R}^d$$

satisfy

$$0 \in \operatorname{conv}(C_i) \qquad \textit{for every } i.$$

Then one can choose

$$c_i \in C_i$$

such that

$$0 \in \operatorname{conv}\{c_1, \ldots, c_{d+1}\}.$$

Proof. By Theorem 266, we may first replace each C_i by a finite subset whose convex hull still contains 0.

Among all colorful choices

$$c_i \in C_i,$$

choose one whose convex hull

$$K = \operatorname{conv}\{c_1, \ldots, c_{d+1}\}$$

has minimum distance from 0. Let $z \in K$ be the closest point.

Suppose

$$z \neq 0.$$

The closest-point property implies

$$z^T x \geq \|z\|^2 \qquad \text{for every } x \in K.$$

Indeed, if some $x \in K$ had

$$z^T x < \|z\|^2,$$

then moving slightly from z toward x would decrease the distance to 0.

The minimal face of K containing z therefore lies in the hyperplane

$$H = \{x : z^T x = \|z\|^2\},$$

which has dimension $d-1$. Hence z is a convex combination of at most d of the selected points. Thus some color, say C_j, is absent from this representation.

Since

$$0 \in \operatorname{conv}(C_j),$$

not every $y \in C_j$ can satisfy $z^T y > 0$. Choose

$$y \in C_j$$

with

$$z^T y \leq 0.$$

Replace the selected point of color j by y. The new colorful convex hull contains both z and y, and for small $\varepsilon > 0$,

$$\|(1-\varepsilon)z + \varepsilon y\|^2 = \|z\|^2 + 2\varepsilon z^T(y-z) + O(\varepsilon^2) < \|z\|^2.$$

This contradicts the minimal choice of K.

Therefore

$$z = 0.$$

□

6.13 Tverberg's Theorem

Tverberg's theorem generalizes Radon's theorem from two parts to an arbitrary number of parts.

Theorem 268 (Tverberg's Theorem)**.** *Let $r \geq 2$. Any*

$$(r-1)(d+1)+1$$

points in $\mathbb{R}^d$ can be partitioned into r nonempty sets

$$S_1, \ldots, S_r$$

such that

$$\bigcap_{j=1}^{r} \operatorname{conv}(S_j) \neq \emptyset.$$

Proof. Set

$$D = (r-1)(d+1).$$

Let

$$u_1, \ldots, u_r \in \mathbb{R}^{r-1}$$

be the vertices of a simplex centered at the origin. Thus

$$u_1 + \cdots + u_r = 0,$$

and this is, up to scaling, their only linear dependence.

Let the given points be

$$x_1, \dots, x_{D+1} \in \mathbb{R}^d,$$

and lift them to

$$\bar{x}_i = (x_i, 1) \in \mathbb{R}^{d+1}.$$

For every i, define a color class in

$$\mathbb{R}^{r-1} \otimes \mathbb{R}^{d+1} \cong \mathbb{R}^D$$

by

$$C_i = \{u_1 \otimes \bar{x}_i, \dots, u_r \otimes \bar{x}_i\}.$$

Because

$$u_1 + \cdots + u_r = 0,$$

we have

$$0 \in \operatorname{conv}(C_i)$$

for every i.

There are exactly

$$D + 1$$

color classes in $\mathbb{R}^D$. By colorful Carathéodory, for every i we may choose an index

$$j(i) \in [r]$$

such that

$$0 \in \operatorname{conv}\{u_{j(i)} \otimes \bar{x}_i : i = 1, \dots, D+1\}.$$

Thus there are coefficients

$$\lambda_i \geq 0, \qquad \sum_i \lambda_i = 1,$$

such that

$$\sum_i \lambda_i u_{j(i)} \otimes \bar{x}_i = 0.$$

Define

$$S_\ell = \{i : j(i) = \ell\}$$

and

$$z_j = \sum_{i \in S_j} \lambda_i \bar{x}_i.$$

Then

$$\sum_{j=1}^{r} u_j \otimes z_j = 0.$$

Choose

$$u_1, \dots, u_{r-1}$$

as a basis and write

$$u_r = -\sum_{j=1}^{r-1} u_j.$$

Then

$$0 = \sum_{j=1}^{r-1} u_j \otimes (z_j - z_r).$$

Linear independence of $u_1, \dots, u_{r-1}$ implies

$$z_1 = \cdots = z_r =: z.$$

The last coordinate of z_j is

$$\alpha_j = \sum_{i \in S_j} \lambda_i.$$

Since all the z_j are equal,

$$\alpha_1 = \cdots = \alpha_r.$$

But

$$\sum_{j=1}^{r} \alpha_j = \sum_i \lambda_i = 1,$$

so

$$\alpha_j = \frac{1}{r}.$$

In particular, every S_j is nonempty.

Let y be r times the first d coordinates of z. For every j,

$$y = \sum_{i \in S_j} \frac{\lambda_i}{\alpha_j} x_i,$$

and the coefficients are nonnegative and sum to 1. Therefore

$$y \in \operatorname{conv}\{x_i : i \in S_j\}$$

for every j.

Hence

$$y \in \bigcap_{j=1}^{r} \operatorname{conv}(S_j).$$

□

For more than $(r-1)(d+1)+1$ points, apply the theorem to any subset of the required size and distribute the remaining points arbitrarily among the parts.

6.14 Matroid Intersection

Let

$$M_1 = (E, \mathcal{I}_1), \qquad M_2 = (E, \mathcal{I}_2)$$

be two matroids on the same ground set, with rank functions r_1, r_2. We want a largest set

$$I \in \mathcal{I}_1 \cap \mathcal{I}_2.$$

The basic example is bipartite matching: take one partition matroid requiring at most one edge incident to each left vertex, and another requiring at most one edge incident to each right vertex.

Theorem 269 (Matroid Intersection)**.** *For any two matroids on the same ground set,*

$$\boxed{\max_{I \in \mathcal{I}_1 \cap \mathcal{I}_2} |I| = \min_{U \subseteq E} \big(r_1(U) + r_2(E \setminus U)\big).}$$

The easy direction is immediate. For every common independent set I and every $U \subseteq E$,

$$|I| = |I \cap U| + |I \setminus U| \le r_1(U) + r_2(E \setminus U).$$

We prove the reverse inequality algorithmically.

Exchange Graph Fix a common independent set

$$I \in \mathcal{I}_1 \cap \mathcal{I}_2.$$

Its exchange graph D_I is the directed bipartite graph with sides I and $E \setminus I$.

For $y \in I$ and $x \notin I$, add

$$y \longrightarrow x$$

if

$$I - y + x \in \mathcal{I}_1,$$

and add

$$x \longrightarrow y$$

if

$$I - y + x \in \mathcal{I}_2.$$

Also define

$$X_1 = \{x \notin I : I + x \in \mathcal{I}_1\},$$

and

$$X_2 = \{x \notin I : I + x \in \mathcal{I}_2\}.$$

Thus an augmenting path has the form

$$x_0, y_1, x_1, \ldots, y_k, x_k,$$

where

$$x_0 \in X_1, \qquad x_k \in X_2.$$

If P is such a path, define

$$I' = I \triangle V(P) = I - \{y_1, \ldots, y_k\} + \{x_0, \ldots, x_k\}.$$

Then

$$|I'| = |I| + 1.$$

We use a shortest augmenting path, so it has no directed shortcut.

Lemma 270 (Unique Exchange Matching)**.** *Let $M = (E, \mathcal{I})$ be a matroid, let $I \in \mathcal{I}$, and let J satisfy*

$$|J| = |I|.$$

Construct the bipartite exchange graph between $I \setminus J$ and $J \setminus I$, joining $y \in I \setminus J$ to $x \in J \setminus I$ whenever

$$I - y + x \in \mathcal{I}.$$

If this graph has a unique perfect matching, then

$$J \in \mathcal{I}.$$

Proof. Let the unique perfect matching be

$$(y_1, x_1), \ldots, (y_k, x_k).$$

Orient the matching edges from x_i to y_i and all other exchange edges from y_i to x_j. There can be no directed cycle, since such a cycle would be an alternating cycle and would give a second perfect matching.

Thus we may order the pairs so that

$$y_i x_j$$

is never an exchange edge when $i < j$.

Suppose for contradiction that J is dependent, and let $C \subseteq J$ be a circuit. Choose the smallest i such that

$$x_i \in C.$$

For every other $x_j \in C \setminus I$, we have $j > i$, so

$$I - y_i + x_j$$

is dependent. Hence

$$x_j \in \mathrm{cl}(I - y_i).$$

Every element of $C \cap I$ also lies in $I - y_i$. Therefore

$$C - x_i \subseteq \mathrm{cl}(I - y_i).$$

Since C is a circuit,

$$x_i \in \mathrm{cl}(C - x_i) \subseteq \mathrm{cl}(I - y_i).$$

But (y_i, x_i) is an exchange edge, so

$$I - y_i + x_i$$

is independent, a contradiction. □

Lemma 271 (Augmenting Path)**.** *If P is a shortest directed path from X_1 to X_2 in D_I, then*

$$I \triangle V(P) \in \mathcal{I}_1 \cap \mathcal{I}_2.$$

Proof. Write

$$P = x_0, y_1, x_1, \ldots, y_k, x_k.$$

We prove independence in M_1. Add a new element t which is a coloop, so that

$$I + t$$

is independent.

Between

$$(I \cap V(P)) \cup \{t\}$$

and

$$V(P) \setminus I,$$

the M_1 exchange graph has the perfect matching

$$tx_0, \quad y_1x_1, \quad \ldots, \quad y_kx_k.$$

Because P is shortest, it has no shortcut, so this perfect matching is unique. By Lemma 270,

$$I \triangle V(P)$$

is independent in M_1.

For M_2, use instead the perfect matching

$$y_1x_0, \quad y_2x_1, \quad \ldots, \quad y_kx_{k-1}, \quad tx_k.$$

The same argument proves independence in M_2. □

Algorithm 34 Matroid Intersection

Input: Independence oracles for M_1, M_2
Output: A maximum common independent set I

```
I ← ∅
while true do
    Construct the exchange digraph D_I
    Construct source set X_1 and sink set X_2
    Find a shortest directed path P from X_1 to X_2
    if no such path exists then
        return I
    I ← I△V(P)
```

It remains to prove that failure to find an augmenting path certifies optimality.

Proof of Theorem 269. The upper bound was already proved.

Run Algorithm 34. Every augmentation increases $|I|$ by one and preserves common independence by Lemma 271.

Suppose eventually there is no path from X_1 to X_2. Let

$$U = \{e \in E : e \text{ can reach } X_2 \text{ in } D_I\}.$$

Then

$$X_2 \subseteq U, \qquad X_1 \cap U = \emptyset.$$

We claim

$$r_1(U) = |I \cap U|.$$

If not, some

$$x \in U \setminus I$$

could be added to $I \cap U$ while preserving independence in M_1. Since $x \notin X_1$, the set $I + x$ is dependent. Let C be its fundamental circuit.

Because

$$(I \cap U) + x$$

is independent, C must contain some

$$y \in I \setminus U.$$

Removing any element of the fundamental circuit gives an independent set, so

$$I - y + x \in \mathcal{I}_1.$$

Thus

$$y \longrightarrow x$$

is an arc. But $x \in U$ can reach X_2, hence so can y, contradicting $y \notin U$.

Thus

$$r_1(U) = |I \cap U|.$$

Similarly,

$$r_2(E \setminus U) = |I \setminus U|.$$

Indeed, otherwise there is

$$x \in (E \setminus U) \setminus I$$

such that

$$(I \setminus U) + x$$

is independent in M_2. Since $X_2 \subseteq U$, we have $x \notin X_2$, so $I + x$ has a fundamental circuit containing some

$$y \in I \cap U.$$

Then

$$I - y + x \in \mathcal{I}_2,$$

so

$$x \longrightarrow y$$

is an arc. Since y can reach X_2, so can x, contradicting $x \notin U$.

Therefore

$$r_1(U) + r_2(E \setminus U) = |I \cap U| + |I \setminus U| = |I|.$$

This matches the general upper bound, so I is maximum and proves the min–max theorem. □

With independence oracles, the direct implementation uses $O(|E|^2)$ oracle queries per augmentation and at most $|E|$ augmentations, hence $O(|E|^3)$ oracle queries.

6.15 Symmetric Submodular Minimization

Let $f : 2^V \to \mathbb{R}$ be submodular and symmetric:

$$f(A) + f(B) \geq f(A \cup B) + f(A \cap B), \qquad f(S) = f(V \setminus S).$$

For $n = |V| \geq 2$, we want a nonempty proper set minimizing f. Excluding $\emptyset$ and V is essential: symmetry and submodularity give $2f(S) \geq f(V) + f(\emptyset) = 2f(\emptyset)$, so they are already unconstrained minimizers.

Queyranne's algorithm uses a greedy ordering and contraction. In each phase it finds a pair s, t whose cheapest separating set is the singleton $\{t\}$. Record that singleton and merge the pair. Either we just recorded an optimum, or some optimum does not separate them and survives the merger. The proof uses only submodularity for the ordering and symmetry at its final step.

The Greedy Rule and the Certificate. For disjoint sets, define their interaction by

$$\operatorname{adj}(A, B) = f(A) + f(B) - f(A \cup B) - f(\emptyset) \geq 0.$$

For an undirected cut function with nonnegative edge weights, this is $2w(E(A, B))$: adding the two boundaries counts edges between the sets twice, while the boundary of their union counts them zero times. Thus the natural ordering starts anywhere and repeatedly adds the element most strongly attached to the current prefix.

Write $W_i = \{v_1, \ldots, v_i\}$ and $W_0 = \emptyset$. A *maximum-adjacency ordering* satisfies

$$\boxed{v_{i+1} \in \arg\min_{v \notin W_i} \big(f(W_i \cup \{v\}) - f(\{v\})\big).} \tag{30}$$

The omitted terms in the interaction are constant over the candidates. At $i = 0$ all scores are zero, so the first element is arbitrary; ties at later steps can also be broken arbitrarily.

A pair (s, t) is *pendent* if

$$f(\{t\}) = \min_{t \in X,\ s \notin X} f(X).$$

The point of the ordering is that its *last two* elements form such a pair. The following prefix inequality will certify this:

$$f(W_i) + f(\{u\}) \leq f(W_i \setminus S) + f(S \cup \{u\}), \qquad u \notin W_i, \quad S \subseteq W_{i-1}.$$

It says that moving any part of the earlier prefix over to a future element cannot improve the total value of the resulting two sides. The restriction $S \subseteq W_{i-1}$ keeps the last chosen element v_i on its original side.

Subtracting singleton values will convert (30) into the simpler rule of minimizing the value of the next prefix. We first prove the inequality for that simpler rule. This intermediate function need not be symmetric or nonnegative.

Lemma 272 (Greedy splitting inequality)**.** *Let $g : 2^U \to \mathbb{R}$ be submodular, and order $U = \{u_1, \dots, u_m\}$ by choosing each next element to minimize $g(U_{i-1} \cup \{u\})$, where $U_i = \{u_1, \dots, u_i\}$. For $m \geq 2$, every set X separating u_{m-1} and u_m satisfies*

$$\boxed{g(X) + g(U \setminus X) \geq g(U_{m-1}) + \min_{x \in X} g(\{x\}).}$$

Proof. Induct on m. For $m = 2$, the two possible sets X are singletons, and the assertion follows from $g(\{u_1\}) \leq g(\{u_2\})$. Suppose $m > 2$ and put $a = u_1$.

First assume $a \notin X$. On $U' = U \setminus \{a\}$ define $h(Y) = g(Y \cup \{a\})$. This is submodular, and $u_2, \dots, u_m$ is a greedy ordering for h: every comparison is exactly the corresponding comparison for g after choosing a. Choose $x \in X$ minimizing $h(\{x\})$. Induction gives

$$g(X \cup \{a\}) + g(U \setminus X) \geq g(U_{m-1}) + g(\{a, x\}).$$

Submodularity applied to X and $\{a, x\}$ gives

$$g(X) + g(\{a, x\}) \geq g(X \cup \{a\}) + g(\{x\}).$$

Adding cancels the two terms involving a, leaving $g(X) + g(U \setminus X) \geq g(U_{m-1}) + g(\{x\})$, which is at least the asserted bound.

If $a \in X$, apply the case just proved to $U \setminus X$. The first greedy choice minimizes singleton values, so

$$\min_{x \in U \setminus X} g(\{x\}) \geq g(\{a\}) \geq \min_{x \in X} g(\{x\}).$$

Substituting this in that case's conclusion proves the remaining case. □

Corollary 273 (Greedy-prefix inequality)**.** *For any submodular f and maximum-adjacency ordering, every $1 \leq i < n$, $u \notin W_i$, and $S \subseteq W_{i-1}$ satisfy*

$$\boxed{f(W_i) + f(\{u\}) \leq f(W_i \setminus S) + f(S \cup \{u\}).}$$

Proof. Restrict to $U = W_i \cup \{u\}$ and subtract the modular function of singleton values:

$$g(Y) = f(Y) - \sum_{v \in Y} f(\{v\}) \qquad (Y \subseteq U).$$

A modular function has equality in the submodular inequality, so g is submodular; also $g(\{v\}) = 0$ for every v. The order $v_1, \dots, v_i, u$ greedily minimizes g of the next prefix, because its candidate score is

$$g(W \cup \{v\}) = f(W \cup \{v\}) - f(\{v\}) - \sum_{w \in W} f(\{w\}).$$

The last sum is independent of v, and restricting the candidates preserves every greedy choice. The set $S \cup \{u\}$ separates the last two elements v_i, u. Apply Lemma 272; its singleton minimum is zero, so it gives $g(S \cup \{u\}) + g(W_i \setminus S) \geq g(W_i)$. The modular terms on the left cover $W_i \cup \{u\}$ once, while those on the right cover W_i once. Cancel them to obtain the claim. □

Theorem 274 (Pendent pair)**.** *For symmetric submodular f, the last two elements $s = v_{n-1}$ and $t = v_n$ of a maximum-adjacency ordering satisfy*

$$\boxed{f(\{t\}) = \min_{t \in X,\ s \notin X} f(X).}$$

Proof. Fix X with $t \in X$ and $s \notin X$. Take $S = X \setminus \{t\} \subseteq W_{n-2}$ in the prefix inequality at $i = n - 1$, with future element $u = t$. Since $W_{n-1} = V \setminus \{t\}$ and $W_{n-1} \setminus S = V \setminus X$, it gives

$$f(V \setminus \{t\}) + f(\{t\}) \leq f(V \setminus X) + f(X).$$

Symmetry turns this into $2f(\{t\}) \leq 2f(X)$. The singleton $\{t\}$ itself is allowed, so the lower bound is attained. □

Contract the Pair. Maintain a partition $\mathcal{P}$ of the original ground set into nonempty blocks. A current element is one block $B \in \mathcal{P}$. The contracted function is

$$f_{\mathcal{P}}(\mathcal{X}) = f\left(\bigcup_{B \in \mathcal{X}} B\right) \qquad (\mathcal{X} \subseteq \mathcal{P}).$$

It remains symmetric and submodular: unions, intersections, and complements of block families correspond to the same operations on their unions, because the blocks partition V.

Algorithm 35 A pendent pair of current blocks

```
function PendentPair(P, f)
    for each B ∈ P do
        d_B ← f(B)
    Choose any B_1 ∈ P; L ← [B_1]; W ← B_1
    while |L| < |P| do
        for each B ∈ P not yet in L do
            q_B ← f(W ∪ B) − d_B
        Choose B with smallest q_B; append B to L; W ← W ∪ B
    return the last two blocks (S, T) of L, together with d_T
```

Algorithm 36 Queyranne's symmetric submodular minimization

Input: A finite set V with $n \geq 2$ and a symmetric submodular function f
Output: A nonempty proper set X^* minimizing f

```
P ← {{v} : v ∈ V}; best ← +∞
while |P| ≥ 2 do
    (S, T, c) ← PendentPair(P, f)
    if c < best then
        best ← c; save a copy X* ← T
    P ← (P \ {S, T}) ∪ {S ∪ T}
return X*
```

Theorem 275 (Queyranne's algorithm)**.** *Algorithm 36 finds a nontrivial minimizer using $O(n^3)$ evaluations of f. With the implementation below, it uses $O(n^3)$ additional operations and $O(n)$ words, excluding space used by the function evaluator. If one evaluation costs $T_f(n)$, the total running time is $O(n^3(T_f(n)+1))$.*

Proof. Maintain the invariant that either a global optimum has been saved or some global optimum is a union of current blocks. Initially every set is such a union. Fix a surviving optimum in one phase. If it separates the last two blocks S, T, the pendent-pair theorem for $f_{\mathcal{P}}$ gives $f(T)$ no larger than its value. The block T is itself a nonempty proper subset of V, so it is also globally optimal and is recorded. Otherwise the optimum is still a union of blocks after merging S, T. This proves the invariant. With one block left, no nontrivial union of blocks remains, so an optimum must have been saved.

With m current blocks, the singleton evaluations cost m calls, and the greedy comparisons cost at most $(m-1)+(m-2)+\cdots+1$. Thus a phase uses $O(m^2)$ calls. There are phases of sizes $n, n-1, \ldots, 2$, so the total is $\sum_{m=2}^{n} O(m^2) = O(n^3)$.

For explicit set manipulation, store the blocks as disjoint lists and keep an n-entry membership vector for W, together with one selected flag per block. To evaluate $f(W \cup B)$, temporarily mark the vertices of the unselected block B, call the evaluator on the vector, and then unmark them. Once a block is selected, mark it permanently for this phase. In one greedy step, the candidate blocks are disjoint, so all temporary marking and unmarking costs $O(n)$ in total, not $O(n)$ per candidate. A phase therefore has $O(nm + m^2)$ additional work; summed over phases this is $O(n^3)$. Computing singleton values, merging block lists, clearing

the membership vector, and copying the best block cost at most $O(n)$ per phase. The lists, vectors, scores, and saved answer require $O(n)$ words. □

7 Complex Analysis

7.1 Cauchy's Theorem

The key fact of complex integration is that a holomorphic function has zero integral around a closed contour. The essential case is a triangle.

Theorem 276 (Cauchy–Goursat for Triangles)**.** *Let T be a triangle, and suppose that f is holomorphic on an open neighborhood of T. Then*

$$\oint_{\partial T} f(z)\,dz = 0.$$

Proof. Write

$$I(T) = \oint_{\partial T} f(z)\,dz.$$

Divide T into four similar triangles by joining the midpoints of its sides. The oriented boundary integrals of the four smaller triangles sum to $I(T)$, since every interior edge appears twice with opposite orientations. Hence one of the four triangles, call it T_1, satisfies

$$|I(T_1)| \geq \frac{1}{4}|I(T)|.$$

Repeating this construction gives nested triangles

$$T \supset T_1 \supset T_2 \supset \cdots$$

such that

$$|I(T_n)| \geq 4^{-n}|I(T)|, \qquad \operatorname{diam}(T_n) = 2^{-n}\operatorname{diam}(T).$$

The triangles shrink to a single point z_0.

Since f is differentiable at z_0,

$$f(z) = f(z_0) + f'(z_0)(z - z_0) + r(z),$$

where

$$r(z) = o(|z - z_0|).$$

Thus, for every $\varepsilon > 0$, for all sufficiently large n and every $z \in T_n$,

$$|r(z)| \leq \varepsilon|z - z_0| \leq \varepsilon \operatorname{diam}(T_n).$$

The constant and linear terms have zero integral around a closed contour:

$$\oint_{\partial T_n} 1\,dz = 0, \qquad \oint_{\partial T_n} (z - z_0)\,dz = 0.$$

Therefore

$$I(T_n) = \oint_{\partial T_n} r(z)\,dz.$$

Hence

$$\begin{aligned} |I(T_n)| &\leq \max_{z \in T_n} |r(z)| \operatorname{perimeter}(T_n) \\ &\leq \varepsilon \operatorname{diam}(T_n) \operatorname{perimeter}(T_n) \\ &= C\varepsilon\, 4^{-n}, \end{aligned}$$

where

$$C = \operatorname{diam}(T) \operatorname{perimeter}(T).$$

Combining this with

$$4^{-n}|I(T)| \le |I(T_n)|$$

gives

$$|I(T)| \le C\varepsilon.$$

Since $\varepsilon > 0$ is arbitrary, $I(T) = 0$. □

The proof has one main idea: repeatedly zoom in while retaining a constant fraction of the integral. At the limiting point, holomorphicity says that f is linear up to an error smaller than the scale of the triangle; the linear part integrates to zero and the error is too small to support a non-zero integral.

Corollary 277 (Cauchy's Theorem for Polygonal Curves)**.** *Let γ be a positively oriented simple closed polygonal curve, and suppose that f is holomorphic on an open neighborhood of γ and its interior. Then*

$$\oint_\gamma f(z)\,dz = 0.$$

Proof. Triangulate the polygon bounded by γ. By Theorem 276, the integral around every triangle is zero. Summing these integrals, every interior edge appears twice with opposite orientations and cancels, leaving exactly

$$\oint_\gamma f(z)\,dz = 0.$$

□

The polygonal statement immediately gives the usual form of Cauchy's theorem.

Theorem 278 (Cauchy's Theorem)**.** *Let $U \subseteq \mathbb{C}$ be simply connected, let $f : U \to \mathbb{C}$ be holomorphic, and let γ be a closed piecewise-C^1 curve in U. Then*

$$\oint_\gamma f(z)\,dz = 0.$$

Proof. First suppose that γ is a simple closed curve. Since U is simply connected, the interior of γ is contained in U.

Approximate γ by simple closed polygonal curves γ_m such that both the curves and their derivatives converge to γ on each C^1 piece. For sufficiently large m, γ_m and its interior lie inside U, so by Corollary 277,

$$\oint_{\gamma_m} f(z)\,dz = 0.$$

Writing the curves on the same parameter interval,

$$\begin{aligned}\left|\int_\gamma f(z)\,dz - \int_{\gamma_m} f(z)\,dz\right| &= \left|\int \left(f(\gamma(t))\gamma'(t) - f(\gamma_m(t))\gamma_m'(t)\right)\,dt\right| \\ &\le \int |f(\gamma) - f(\gamma_m)|\,|\gamma'|\,dt + \int |f(\gamma_m)|\,|\gamma' - \gamma_m'|\,dt.\end{aligned}$$

Since f is uniformly continuous and bounded on a compact neighborhood of the curves, both terms tend to zero. Hence

$$\oint_\gamma f(z)\,dz = 0.$$

A general closed piecewise-C^1 curve can be approximated in the same way by a closed polygonal curve. Splitting the polygon at its finitely many self-intersections decomposes it into finitely many simple closed polygonal curves, each of which has zero integral. Passing to the limit again proves the result. □

Corollary 279 (Cauchy's Theorem with Holes)**.** *Let Ω be a bounded region whose boundary consists of finitely many piecewise-C^1 curves, oriented so that Ω always lies to the left. If f is holomorphic on a neighborhood of $\overline{\Omega}$, then*

$$\oint_{\partial\Omega} f(z)\,dz = 0.$$

Equivalently, if γ is the counterclockwise outer boundary and $C_1, \ldots, C_m$ are the counterclockwise boundaries of the holes, then

$$\oint_{\gamma} f(z)\,dz = \sum_{j=1}^{m} \oint_{C_j} f(z)\,dz.$$

Proof. Join every inner boundary to the outer boundary by non-intersecting cuts. The resulting region is simply connected, so Cauchy's theorem applies. Each cut appears twice in the resulting boundary, once in each direction, and the two integrals cancel. What remains is the displayed boundary integral. □

7.2 Cauchy's Integral Formula

Cauchy's integral formula can be viewed retrospectively as the one-pole case of the residue theorem. We prove it first, however, since it follows directly from Cauchy's theorem and does not require Laurent series.

Theorem 280 (Cauchy's Integral Formula)**.** *Let γ be a positively oriented simple closed piecewise-C^1 curve, let z lie in its interior, and suppose that f is holomorphic on a neighborhood of γ and its interior. Then*

$$f(z) = \frac{1}{2\pi i} \oint_{\gamma} \frac{f(w)}{w - z}\,dw.$$

Proof. Choose a sufficiently small circle

$$C_\varepsilon = \{w : |w - z| = \varepsilon\}$$

inside γ. The function

$$w \longmapsto \frac{f(w)}{w - z}$$

is holomorphic on the region between γ and C_ε. By Corollary 279,

$$\oint_{\gamma} \frac{f(w)}{w - z}\,dw = \oint_{C_\varepsilon} \frac{f(w)}{w - z}\,dw.$$

Parameterize the small circle by

$$w = z + \varepsilon e^{it}, \qquad 0 \le t \le 2\pi.$$

Then

$$\begin{aligned}\oint_{C_\varepsilon} \frac{f(w)}{w - z}\,dw &= \int_0^{2\pi} \frac{f(z + \varepsilon e^{it})}{\varepsilon e^{it}} i\varepsilon e^{it}\,dt \\ &= i\int_0^{2\pi} f(z + \varepsilon e^{it})\,dt.\end{aligned}$$

As $\varepsilon \to 0$, continuity of f gives

$$i\int_0^{2\pi} f(z + \varepsilon e^{it})\,dt \longrightarrow 2\pi i f(z).$$

The left-hand side is independent of ε, so it must already equal $2\pi i f(z)$. □

Corollary 281 (Cauchy's Differentiation Formula)**.** *Under the hypotheses of Theorem 280, for every integer* $k \geq 0$*,*

$$f^{(k)}(z) = \frac{k!}{2\pi i} \oint_\gamma \frac{f(w)}{(w-z)^{k+1}}\, dw.$$

Proof. Differentiate the integral formula k times with respect to z. Since the contour stays a positive distance from z, differentiation under the integral is valid, and

$$\frac{d^k}{dz^k} \frac{1}{w-z} = \frac{k!}{(w-z)^{k+1}}.$$

□

7.3 Laurent Series

Cauchy's integral formula also explains why holomorphic functions are represented by power series, and why isolated singularities are represented by Laurent series.

Theorem 282 (Laurent Expansion)**.** *Suppose that* f *is holomorphic on the annulus*

$$r < |z-a| < R.$$

Then there are unique coefficients $(c_n)_{n\in\mathbb{Z}}$ *such that*

$$f(z) = \sum_{n=-\infty}^{\infty} c_n (z-a)^n$$

throughout the annulus, with convergence uniform on compact subannuli.

For every $r < \rho < R$*,*

$$\boxed{c_n = \frac{1}{2\pi i} \oint_{|w-a|=\rho} \frac{f(w)}{(w-a)^{n+1}}\, dw} \qquad (n \in \mathbb{Z}).$$

Proof. Fix z with

$$r < |z-a| < R,$$

and choose radii

$$r < \rho_- < |z-a| < \rho_+ < R.$$

Applying the same argument as Cauchy's integral formula to the annulus between the two circles gives

$$f(z) = \frac{1}{2\pi i} \oint_{|w-a|=\rho_+} \frac{f(w)}{w-z}\, dw - \frac{1}{2\pi i} \oint_{|w-a|=\rho_-} \frac{f(w)}{w-z}\, dw.$$

For the outer circle, $|z-a| < |w-a|$, so

$$\begin{aligned} \frac{1}{w-z} &= \frac{1}{(w-a)-(z-a)} \\ &= \frac{1}{w-a} \frac{1}{1-\frac{z-a}{w-a}} \\ &= \sum_{n=0}^{\infty} \frac{(z-a)^n}{(w-a)^{n+1}}. \end{aligned}$$

The geometric series converges uniformly on the circle, so we may integrate term-by-term:

$$\frac{1}{2\pi i} \oint_{|w-a|=\rho_+} \frac{f(w)}{w-z}\, dw = \sum_{n=0}^{\infty} \left(\frac{1}{2\pi i} \oint_{|w-a|=\rho_+} \frac{f(w)}{(w-a)^{n+1}}\, dw \right) (z-a)^n.$$

For the inner circle, $|w-a| < |z-a|$, so

$$\begin{aligned}\frac{1}{w-z} &= -\frac{1}{z-a}\frac{1}{1-\frac{w-a}{z-a}} \\ &= -\sum_{m=0}^{\infty}\frac{(w-a)^m}{(z-a)^{m+1}}.\end{aligned}$$

Again the convergence is uniform, hence

$$-\frac{1}{2\pi i}\oint_{|w-a|=\rho_-}\frac{f(w)}{w-z}\,dw = \sum_{m=0}^{\infty}\left(\frac{1}{2\pi i}\oint_{|w-a|=\rho_-} f(w)(w-a)^m\,dw\right)(z-a)^{-m-1}.$$

Together these give a Laurent expansion

$$f(z) = \sum_{n=-\infty}^{\infty} c_n(z-a)^n.$$

For any fixed integer n, the function

$$w \longmapsto \frac{f(w)}{(w-a)^{n+1}}$$

is holomorphic throughout the annulus. By Corollary 279, its integral around two circles centered at a inside the annulus is the same. Thus both the positive and negative coefficients can be written uniformly as

$$c_n = \frac{1}{2\pi i}\oint_{|w-a|=\rho}\frac{f(w)}{(w-a)^{n+1}}\,dw$$

for any $r < \rho < R$.

The geometric-series argument is uniform whenever z ranges over a compact subannulus, which gives locally uniform convergence.

Finally, the coefficient formula determines every c_n, so the expansion is unique. □

Corollary 283 (Taylor Expansion)**.** *If f is holomorphic on $|z-a| < R$, then*

$$f(z) = \sum_{n=0}^{\infty}\frac{f^{(n)}(a)}{n!}(z-a)^n.$$

Proof. Apply Theorem 282 with $r = 0$. For $n < 0$, the coefficient integrand

$$\frac{f(w)}{(w-a)^{n+1}}$$

has a non-negative power of $(w-a)$ and is holomorphic inside the circle, so Cauchy's theorem gives $c_n = 0$.

For $n \geq 0$, Corollary 281 gives

$$c_n = \frac{f^{(n)}(a)}{n!}.$$

□

7.4 Residue Theorem

Let a be an isolated singularity of f. By Theorem 282, in a punctured neighborhood of a,

$$f(z) = \sum_{n=-\infty}^{\infty} c_n (z-a)^n.$$

The coefficient

$$\operatorname{Res}(f,a) := c_{-1}$$

is called the *residue* of f at a.

The significance of this coefficient follows from the elementary identity

$$\oint_{|z-a|=\rho} (z-a)^n \, dz = \begin{cases} 2\pi i, & n=-1, \\ 0, & n \neq -1. \end{cases}$$

Indeed, uniform convergence of the Laurent series on the circle gives

$$\boxed{\oint_{|z-a|=\rho} f(z)\, dz = 2\pi i \operatorname{Res}(f,a).}$$

Theorem 284 (Residue Theorem)**.** *Let γ be a positively oriented simple closed piecewise-C^1 curve, and suppose that f is holomorphic on a neighborhood of γ and its interior except at finitely many isolated singularities*

$$a_1, \dots, a_m$$

inside γ. Then

$$\oint_\gamma f(z)\, dz = 2\pi i \sum_{j=1}^{m} \operatorname{Res}(f, a_j).$$

Proof. Choose disjoint small counterclockwise circles C_j around the singularities. On the region inside γ with the interiors of these circles removed, f is holomorphic. Therefore Corollary 279 gives

$$\oint_\gamma f(z)\, dz = \sum_{j=1}^{m} \oint_{C_j} f(z)\, dz.$$

By the Laurent expansion around a_j,

$$\oint_{C_j} f(z)\, dz = 2\pi i \operatorname{Res}(f, a_j).$$

Summing gives the result. □

Corollary 285 (Residue at a Pole)**.** *If a is a pole of order m of f, then*

$$\operatorname{Res}(f,a) = \frac{1}{(m-1)!} \frac{d^{m-1}}{dz^{m-1}} \left((z-a)^m f(z) \right) \Big|_{z=a}.$$

In particular, at a simple pole,

$$\operatorname{Res}(f,a) = \lim_{z \to a} (z-a) f(z).$$

Proof. Write

$$f(z) = \frac{g(z)}{(z-a)^m},$$

where g is holomorphic at a. The residue is the coefficient of $(z-a)^{-1}$, hence it is the coefficient of $(z-a)^{m-1}$ in the Taylor series of g. Therefore

$$\operatorname{Res}(f,a) = \frac{g^{(m-1)}(a)}{(m-1)!}.$$

□

7.5 Cotangent Partial Fractions

The residue theorem gives a partial fraction expansion of the cotangent.

Theorem 286 (Cotangent Partial Fraction Expansion)**.** *For every $z \notin \mathbb{Z}$,*

$$\pi \cot(\pi z) = \frac{1}{z} + 2z \sum_{n=1}^{\infty} \frac{1}{z^2 - n^2}.$$

Equivalently,

$$\pi \cot(\pi z) = \frac{1}{z} + \sum_{n=1}^{\infty} \left(\frac{1}{z-n} + \frac{1}{z+n} \right).$$

Proof. Fix $z \notin \mathbb{Z}$ and consider, as a function of w,

$$f(w) = \frac{\pi \cot(\pi w)}{w^2 - z^2}.$$

Let C_N be the positively oriented square with vertices

$$\left(N + \frac{1}{2}\right)(\pm 1 \pm i).$$

For sufficiently large N, the points $\pm z$ lie inside C_N.

The poles of f inside C_N are the integers

$$-N, -N+1, \ldots, N$$

and the two points $\pm z$.

At an integer n, the function $\pi \cot(\pi w)$ has residue 1, so

$$\operatorname{Res}(f, n) = \frac{1}{n^2 - z^2}.$$

At $w = z$,

$$\operatorname{Res}(f, z) = \frac{\pi \cot(\pi z)}{2z}.$$

Since cot is odd,

$$\operatorname{Res}(f, -z) = \frac{\pi \cot(-\pi z)}{-2z} = \frac{\pi \cot(\pi z)}{2z}.$$

Thus the residues at $\pm z$ sum to

$$\frac{\pi \cot(\pi z)}{z}.$$

It remains to show that the contour integral tends to zero. Writing $w = x + iy$,

$$|\cot(\pi w)|^2 = \frac{\cos^2(\pi x) + \sinh^2(\pi y)}{\sin^2(\pi x) + \sinh^2(\pi y)}.$$

On the vertical sides of C_N, $x = N + \frac{1}{2}$, so

$$|\cot(\pi w)| \leq 1.$$

On the horizontal sides, $|y| = N + \frac{1}{2}$, so $\cot(\pi w)$ is also uniformly bounded independently of N.

Furthermore,

$$|w^2 - z^2| = \Omega(N^2)$$

uniformly on C_N, while the length of C_N is $O(N)$. Hence

$$\oint_{C_N} f(w)\,dw = O(N^{-1}) \to 0.$$

By the residue theorem,

$$0 = \frac{\pi\cot(\pi z)}{z} + \sum_{n\in\mathbb{Z}} \frac{1}{n^2 - z^2}.$$

Therefore

$$\frac{\pi\cot(\pi z)}{z} = \sum_{n\in\mathbb{Z}} \frac{1}{z^2 - n^2}.$$

Separating the $n = 0$ term and pairing n with $-n$ gives

$$\frac{\pi\cot(\pi z)}{z} = \frac{1}{z^2} + 2\sum_{n=1}^{\infty} \frac{1}{z^2 - n^2}.$$

Multiplying by z proves

$$\pi\cot(\pi z) = \frac{1}{z} + 2z\sum_{n=1}^{\infty} \frac{1}{z^2 - n^2}.$$

Finally,

$$\frac{2z}{z^2 - n^2} = \frac{1}{z - n} + \frac{1}{z + n},$$

which gives the equivalent form. □

The pairing of n and $-n$ is important: the paired terms are $O(n^{-2})$, so the series converges on compact subsets of $\mathbb{C} \setminus \mathbb{Z}$.

7.6 Euler's Product for Sine

The sine product follows almost immediately by integrating the cotangent partial fraction expansion.

Theorem 287 (Euler's Product for Sine)**.** *For every $z \in \mathbb{C}$,*

$$\frac{\sin(\pi z)}{\pi z} = \prod_{n=1}^{\infty} \left(1 - \frac{z^2}{n^2}\right).$$

Equivalently,

$$\sin(\pi z) = \pi z \prod_{n=1}^{\infty} \left(1 - \frac{z^2}{n^2}\right).$$

Proof. From Theorem 286,

$$\pi\cot(\pi z) - \frac{1}{z} = 2z\sum_{n=1}^{\infty} \frac{1}{z^2 - n^2}.$$

The left-hand side is

$$\frac{d}{dz}\log\left(\frac{\sin(\pi z)}{\pi z}\right).$$

For every n,

$$\frac{d}{dz}\log\left(1 - \frac{z^2}{n^2}\right) = \frac{2z}{z^2 - n^2}.$$

Hence

$$\frac{d}{dz}\log\left(\frac{\sin(\pi z)}{\pi z}\right) = \sum_{n=1}^{\infty}\frac{d}{dz}\log\left(1-\frac{z^2}{n^2}\right).$$

The series on the right converges uniformly on compact subsets avoiding the integers, since its terms are $O(n^{-2})$. Thus we may integrate term-by-term.

Near $z = 0$, choose the logarithms so that they vanish at 0. Integrating from 0 to z gives

$$\log\left(\frac{\sin(\pi z)}{\pi z}\right) = \sum_{n=1}^{\infty}\log\left(1-\frac{z^2}{n^2}\right),$$

because

$$\lim_{z\to 0}\frac{\sin(\pi z)}{\pi z} = 1.$$

Exponentiating,

$$\frac{\sin(\pi z)}{\pi z} = \prod_{n=1}^{\infty}\left(1-\frac{z^2}{n^2}\right)$$

in a neighborhood of 0.

The infinite product converges uniformly on compact sets, since

$$\sum_{n=1}^{\infty}\frac{|z|^2}{n^2}$$

converges uniformly when z is restricted to a compact set. Thus the product defines an entire function. Since both sides are entire and agree on a neighborhood of 0, the identity theorem extends the equality to all $z \in \mathbb{C}$. □

8 Data Structures

8.1 RMQ

We want to find the index of the minimum element within an arbitrary subarray $A[l \dots r]$. We can achieve $O(1)$ query time with $O(n)$ preprocessing time and space.

1. Blocks We partition the array into blocks of size $B = \Theta(\log n)$. For each index $i \in [0, n-1]$, we maintain a bitmask $mask[i]$ of length up to B. This mask represents a strictly increasing subsequence of values ending at i, extending leftward up to the block boundary or at most B elements.

Specifically, the k-th bit (0-indexed) of $mask[i]$ is set to 1 if and only if $A[i-k]$ is the minimum element in the suffix $A[i-k \dots i]$. As we iterate i from 0 to $n-1$, we compute $mask[i]$ iteratively:

1. Shift the previous mask left by 1: $mask_{\text{curr}} \leftarrow mask_{i-1} \ll 1$.
2. While the current element $A[i]$ is strictly less than the element corresponding to the lowest set bit in $mask_{\text{curr}}$, unset that bit.
3. Set the 0-th bit to 1, representing the element $A[i]$ itself.

2. Sparse Table We construct a Sparse Table over the $\lfloor \frac{n}{B} \rfloor$ blocks. Instead of storing all elements, the 0-th level of the Sparse Table only stores the overall minimum of each block. The minimum of the j-th block (which ends at index $i = j \cdot B + B - 1$) is found using its mask: $mask[i]$ contains suffix minimums, so its highest set bit is the minimum of the entire block. Because there are only n/B blocks, the Sparse Table takes $O((n/B)\log(n/B)) = O(n)$ time and space.

3. **Query Resolution** To answer a query for the range $[l, r]$:

- **Case 1: The range spans at most B elements.** The range is small enough that the entire answer is encoded within $mask[r]$. We truncate the mask to only consider the last $r - l + 1$ elements by doing a bitwise AND with $(1 \ll (r - l + 1)) - 1$. The index of the highest set bit in this truncated mask corresponds to the minimum in $[l, r]$.
- **Case 2: The range spans multiple blocks.** We decompose the range into three parts:

  1. A partial block on the left: from l to the end of its block.
  2. A set of completely covered intermediate blocks.
  3. A partial block on the right: from the start of its block to r.

  The minimum of the left partial block is found using the highest set bit of the mask at the end of that block. The minimum of the right partial block is found using the highest set bit of $mask[r]$. The minimum of the completely covered intermediate blocks is queried in $O(1)$ time from the Sparse Table. We simply take the minimum of these three candidates.

Algorithm 37 RMQ

```
function Init(A[0...n-1])
    B ← Θ(log n)
    mask ← array of size n
    curr_mask ← 0
    for i ← 0 to n − 1 do
        curr_mask ← curr_mask ≪ 1
        while curr_mask ≠ 0 and A[i] < A[i − trailing_zeros(curr_mask)] do
            curr_mask ← curr_mask & (curr_mask − 1)                ▷ Pops the lowest set bit
        curr_mask ← curr_mask | 1
        mask[i] ← curr_mask
                                                                   ▷ Build Sparse Table over Blocks
    num_blocks ← ⌊n/B⌋
    ST ← 2D array of size log(num_blocks) × num_blocks
    for j ← 0 to num_blocks − 1 do
        end_idx ← j · B + B − 1
        ST[0][j] ← end_idx − ⌊log2(mask[end_idx])⌋
    for k ← 1 to ⌊log2(num_blocks)⌋ do
        for j ← 0 to num_blocks − (1 ≪ k) do
            ST[k][j] ← min(ST[k − 1][j], ST[k − 1][j + (1 ≪ (k − 1))])
function Query(l, r)
    if r − l + 1 ≤ B then
        trunc_mask ← mask[r] & ((1 ≪ (r − l + 1)) − 1)
        return r − ⌊log2(trunc_mask)⌋
                                                                   ▷ Left partial block
    left_end ← (l/B) · B + B − 1
    ans ← left_end − ⌊log2(mask[left_end])⌋
                                                                   ▷ Right partial block
    ans ← min(ans, r − ⌊log2(mask[r])⌋)
                                                                   ▷ Intermediate full blocks
    x ← l/B + 1
    y ← r/B − 1
    if x ≤ y then
        k ← ⌊log2(y − x + 1)⌋
        mid_ans ← min(ST[k][x], ST[k][y − (1 ≪ k) + 1])
        ans ← min(ans, mid_ans)
    return ans
```

8.2 DSU

8.2.1 Rank Facts

Use union by rank. Initially every vertex has rank 0. When roots of equal rank r are linked, the new root gets rank $r + 1$; otherwise the smaller-rank root is attached below the larger-rank root.

Hence ranks strictly increase along parent pointers.

Furthermore,

$$\#\{v : r(v) = r\} \leq \frac{n}{2^r},$$

because obtaining rank r requires having combined at least 2^r elements. In particular

$$\sum_v r(v) \le \sum_{r \ge 0} \frac{nr}{2^r} = O(n).$$

8.2.2 Ackermann Levels

Define

$$A_0(x) = x + 1, \qquad A_{k+1}(x) = A_k^{(x+1)}(1),$$

where $A_k^{(t)}$ denotes t-fold iteration.
Let

$$\alpha = \min\{k : A_k(1) > \log_2 n\}.$$

This differs from the usual definitions of $\alpha(n)$ by at most an additive constant.
For a nonroot vertex x of positive rank define

$$\text{level}(x) = \max\{k : A_k(r(x)) \le r(p(x))\}.$$

Since

$$r(p(x)) \le \log n < A_\alpha(1) \le A_\alpha(r(x)),$$

we always have

$$0 \le \text{level}(x) < \alpha.$$

Once $k = \text{level}(x)$ is fixed, define

$$\text{index}(x) = \max\{i : A_k^{(i)}(r(x)) \le r(p(x))\}.$$

Maximality of k implies

$$1 \le \text{index}(x) \le r(x).$$

Think of x as watching its parent's rank climb through the thresholds

$$A_k(r(x)), \quad A_k^{(2)}(r(x)), \quad A_k^{(3)}(r(x)), \ldots.$$

The index says how many thresholds have already been crossed; after all thresholds at level k are crossed, the level itself increases.

8.2.3 Potential

For a nonroot x of positive rank let

$$\phi(x) = (\alpha - \text{level}(x))r(x) - \text{index}(x).$$

For a root set

$$\phi(x) = \alpha r(x),$$

and put $\phi(x) = 0$ when $r(x) = 0$.
Because

$$\text{level}(x) \le \alpha - 1, \qquad \text{index}(x) \le r(x),$$

we have

$$0 \le \phi(x) \le \alpha r(x).$$

Therefore the total amount of potential that can ever be introduced is

$$O\left(\alpha \sum_x r(x)\right) = O(n\alpha).$$

More importantly, changing a parent to an ancestor can only increase the parent's rank. Therefore either the level or the index increases, and the potential never increases under path compression.

8.2.4 Why a Find Costs $O(\alpha)$ Amortized

Consider a find path

$$x_0, x_1, \ldots, x_t,$$

where x_t is the root.

Suppose x_i currently has level k and index j:

$$A_k^{(j)}(r(x_i)) \le r(x_{i+1}) < A_k^{(j+1)}(r(x_i)).$$

Suppose there is a later vertex x_q, $q > i$, having the same level k. Then

$$r(x_q) \ge r(x_{i+1}) \ge A_k^{(j)}(r(x_i)),$$

and hence

$$r(x_t) \ge A_k(r(x_q)) \ge A_k^{(j+1)}(r(x_i)).$$

After path compression x_i becomes a child of x_t. Its parent has therefore crossed the next threshold. Thus either its index increases or its level increases. In either case

$$\phi(x_i)$$

drops by at least 1.

Consequently, for each level k, only the *last* vertex on the find path having that level might fail to decrease its potential. There are only

$$\alpha$$

levels. There is also at most one rank-0 vertex on the path, since ranks strictly increase.

Therefore, if the path has length t,

$$t \le O(\alpha) + \text{potential decrease}.$$

Summing over all finds gives

$$O(m\alpha(n))$$

total time.

8.3 DSU with Potential

The DSU with Potential, maintains disjoint sets while tracking relative differences (potentials) between elements in the same set. The potentials must belong to a group G.

For any node x, the array $\texttt{p}[x]$ maintains the potential difference from x to its current parent in the DSU tree, which we denote as $\Delta(x, \text{parent}(x))$.

1. Path Compression When querying the root of a node x, the algorithm recursively compresses the path. Suppose the recursive call finds the root R and the potential from the parent to the root, $\Delta(\text{parent}(x), R)$.

To update x to point directly to R, we compose the potential from x to its parent with the potential from its parent to R:

$$\Delta(x, R) = \Delta(x, \text{parent}(x)) \circ \Delta(\text{parent}(x), R)$$

2. Querying Potential To find the potential $\Delta(a, b)$ from node a to node b, we first verify they belong to the same set. Let R be their common root. The `find` operation gives us $v_a = \Delta(a, R)$ and $v_b = \Delta(b, R)$.

The path from a to b can be constructed by traversing from a up to R, and then down from R to b. Thus, the combined potential is:

$$\Delta(a, b) = \Delta(a, R) \circ \Delta(R, b) = v_a \circ v_b^{-1}$$

3. Edge Insertion When merging the sets containing a and b by adding a directed edge from a to b with a given potential P, we require that $\Delta(a, b) = P$.

Let R_a and R_b be the respective roots of a and b. Suppose we link R_a as a child of R_b. We need to compute the required weight $W = \Delta(R_a, R_b)$ for this new edge.

We know the full path $a \to R_a \to R_b \to b$ must equal the target potential P:

$$\Delta(a, R_a) \circ \Delta(R_a, R_b) \circ \Delta(R_b, b) = P$$

Substituting our known variables $v_a = \Delta(a, R_a)$, $W = \Delta(R_a, R_b)$, and $v_b^{-1} = \Delta(R_b, b)$:

$$v_a \circ W \circ v_b^{-1} = P$$

Thus:

$$W = v_a^{-1} \circ P \circ v_b$$

If the union-by-size heuristic determined that R_b should become a child of R_a, the weight of the reversed edge is the inverse W^{-1}.

8.4 Persistent Queue

A fully persistent queue allows us to push and pop elements from any historical state of the queue. By modeling the sequence of pushes as a tree, we can implement it by using Algorithm 46.

1. State Each state in the persistent queue is represented as a node in a tree. A node u stores:

- v: The value inserted at this state.
- p: The parent node (the state of the queue before this element was pushed).
- depth: The depth of the node in the tree (the total number of pushes to reach this state).
- jmp: The jump pointer used for ancestor queries.
- fr: A pointer to the node representing the current front of the queue.

2. Push Operation When pushing a value onto a previous state p, we append a new child node u. The front pointer fr is inherited from p

We compute the new jump pointer $\text{jmp}[u]$ in $O(1)$ time using the combination rule from Algorithm 46. We check:

$$\text{depth}[p] - \text{depth}[\text{jmp}[p]] = \text{depth}[\text{jmp}[p]] - \text{depth}[\text{jmp}[\text{jmp}[p]]]$$

If they are equal, we merge them: $\text{jmp}[u] \leftarrow \text{jmp}[\text{jmp}[p]]$. Otherwise, we reset the jump length: $\text{jmp}[u] \leftarrow p$.

3. Pop Operation To pop an element from state p, we must advance the front pointer fr to the next element in the queue. Because the queue's elements form a path from fr down to the current tail p, the next element to become the front is the child of fr that lies on the path to p.

This is a Level Ancestor query: we must find the ancestor of p whose depth is exactly $\text{depth}[fr] + 1$. We can find this in $O(\log n)$ time and then create a new state that copies p, but updates its fr pointer to this new ancestor.

4. Front Operation Querying the front of the queue simply requires reading the value stored at the node pointed to by fr.

Algorithm 38 Persistent Queue

```
1:  function Push(v, p)
2:      if p = NULL then
3:          return QueueNode(v, parent: NULL, fr: nxt, jmp: nxt, depth: 0)
4:      fr ← nodes[p].fr
5:      p_j ← nodes[p].jmp
6:      p_jj ← nodes[p_j].jmp
7:      if depth[p] − depth[p_j] = depth[p_j] − depth[p_jj] then
8:          jmp ← p_jj
9:      else
10:         jmp ← p
11:     depth ← nodes[p].depth + 1
12:     return QueueNode(v, p, fr, jmp, depth)
13: function LevelAncestor(p, target_depth)
14:     while depth[p] > target_depth do
15:         if depth[jmp[p]] ≥ target_depth then
16:             p ← jmp[p]
17:         else
18:             p ← parent[p]
19:     return p
20: function Pop(p)
21:     new_node ← nodes[p].clone()
22:     target_depth ← nodes[nodes[p].fr].depth + 1
23:     new_node.fr ← LevelAncestor(p, target_depth)
24:     return new_node
```

8.5 Online Suffix Max

To answer suffix maximum queries $\max(A[i \dots n-1])$ in a setting where elements are appended, we can combine a monotonic stack with a DSU.

1. Monotonic Stack When a new element x is appended at index p, it invalidates any previous elements smaller than it. We maintain a stack of indices corresponding to a strictly decreasing subsequence of the array. As x is pushed, we pop all indices j from the stack where $A[j] \leq x$.

2. DSU Any index whose suffix maximum was previously $A[j]$ now shares a suffix maximum of x. For every popped index j, we union its set into p's set. Finally, we push p onto the stack.

3. Queries To find the maximum in the suffix starting at index i, we simply query the DSU for the root of i.

8.6 Mo's Algorithm

Suppose we have a static array of length n and

$$q$$

offline range queries

$$[L_i, R_i).$$

Assume we can maintain the answer while moving either endpoint by one position, with $O(1)$ work per move.

Mo's algorithm orders the queries so consecutive intervals differ only a small amount.

Choose a block size B. Sort queries by

$$\left(\left\lfloor \frac{L}{B} \right\rfloor, R\right),$$

except that the direction of R alternates between blocks:

$$\boxed{\begin{cases} R \text{ increasing}, & \lfloor L/B \rfloor \text{ even}, \\ R \text{ decreasing}, & \lfloor L/B \rfloor \text{ odd}. \end{cases}}$$

Algorithm 39 Mo's Algorithm

Input: Array of length n, range queries $[L_i, R_i)$

choose block size B
sort queries by

$$\Big(\lfloor L/B \rfloor,\ (-1)^{\lfloor L/B \rfloor} R\Big)$$

with the second coordinate interpreted as alternating direction
$L \leftarrow 0$, $R \leftarrow 0$
for query $[a, b)$ in this order **do**
    while $L > a$ **do**
        $L \leftarrow L - 1$; ADD(L)
    while $R < b$ **do**
        ADD(R); $R \leftarrow R + 1$
    while $L < a$ **do**
        REMOVE(L); $L \leftarrow L + 1$
    while $R > b$ **do**
        $R \leftarrow R - 1$; REMOVE(R)
    record current answer

Theorem 288 (Mo Movement Bound)**.** *With block size B, the total number of endpoint moves is*

$$\boxed{O\left(qB + \frac{n^2}{B} + n\right).}$$

Choosing

$$B \asymp \frac{n}{\sqrt{q}}$$

gives

$$\boxed{O(n\sqrt{q} + q)}$$

total work when each endpoint update costs $O(1)$.

Proof. First consider the left endpoint.

Inside one block,

$$L$$

always lies in an interval of width B. Thus between consecutive queries in that block it moves by at most B. Over q queries,

$$\text{total movement of } L = O(qB + n).$$

Now consider R. Within one L-block, queries are sorted monotonically by R. Thus R traverses the array at most once inside that block.

There are at most

$$\left\lceil \frac{n}{B} \right\rceil$$

blocks. Therefore

$$\text{total movement of } R = O\left(n\frac{n}{B} + n\right) = O\left(\frac{n^2}{B} + n\right).$$

The odd/even reversal means that when one block finishes near one end of the array, the next begins from that same end rather than forcing an unnecessary reset.

Combining the two bounds,

$$O\left(qB + \frac{n^2}{B} + n\right).$$

Balance the two main terms:

$$qB = \frac{n^2}{B},$$

so

$$B = \frac{n}{\sqrt{q}}.$$

Then

$$qB = \frac{n^2}{B} = n\sqrt{q}.$$

□

8.7 A Fast Succinct Hash Table

We first assume access to the random hash functions used below; the next subsection explains how to replace fully random functions by a small explicit random data structure.

We work on a word RAM with word size

$$w = \Theta(\log n),$$

and consider a fixed polynomial-length sequence of operations independent of the random choices of the data structure. We maintain a capacity parameter

$$N = \Theta(n)$$

throughout an epoch.

The goal is an exact dynamic dictionary using

$$\boxed{nw + O(n \log\log n)}$$

bits, with constant-time queries and, with high probability, constant time per update after deamortization.

The construction has three layers:

hash into polylogarithmic-size buckets
$\downarrow$
represent a key locally by an $O(\log\log n)$-bit fingerprint
$\downarrow$
tabulate all operations on the resulting tiny data.

The only updates that can occasionally be expensive are local fingerprint rebuilds. A final buffering step deamortizes these.

8.7.1 Small Buckets

Put

$$L = \lceil (\log N)^3 \rceil$$

and hash the keys uniformly into

$$B = \left\lceil \frac{N}{L} \right\rceil$$

buckets.

We maintain $n \le N$. For any fixed bucket, its load X is binomial with mean

$$\mu = \frac{n}{B} \le L.$$

Thus it exceeds $4L$ with superpolynomially small probability by a Chernoff bound.

At each time in a fixed update sequence, the current set of keys is fixed independently of the random top-level hash function, so the same argument applies. A union bound over $N^{O(1)}$ times still gives

$$N^{O(1)} e^{-\Omega(\log^3 N)} = N^{-\omega(1)}.$$

Henceforth condition on this event. Every bucket contains

$$O(\log^3 N)$$

keys throughout the epoch.

8.7.2 Storing the Full Keys Succinctly

The actual w-bit keys must be stored somewhere, but we do not want a word-sized pointer or other $\Theta(\log n)$-bit overhead for every key.

Within each bucket, divide the keys into blocks of

$$s = \lceil (\log N)^2 \rceil$$

keys. All blocks except possibly the last are full.

A bucket of size $b \le 4L$ therefore uses at most

$$\left\lceil \frac{b}{s} \right\rceil = O(\log N)$$

blocks. Give every bucket a directory containing the addresses of these blocks.

This representation has very little wasted space. Across all buckets, the number of allocated key slots is at most

$$\sum_{i=1}^{B} s \left\lceil \frac{|B_i|}{s} \right\rceil \le \sum_i |B_i| + Bs = n + O\left(\frac{N}{\log N}\right).$$

Since $w = \Theta(\log N)$, the full keys therefore occupy

$$\boxed{nw + O(N)}$$

bits.

Each bucket directory has $O(\log N)$ block addresses, each using $O(\log N)$ bits. Hence all directories together use

$$O\left(\frac{N}{\log^3 N} \log^2 N\right) = O\left(\frac{N}{\log N}\right)$$

bits.

A position within one bucket consists of

$$\text{block number} \in [O(\log N)]$$

and

$$\text{offset} \in [(\log N)^2],$$

so it requires only

$$\boxed{O(\log \log N)}$$

bits.

Insertion appends to the last block, allocating a new block if necessary. Deletion replaces the deleted key by the last key in its bucket and frees the last block if it becomes empty. Thus the key storage itself supports every local update in $O(1)$ time.

The only remaining task is to locate the local position of a queried key.

8.7.3 Local Fingerprints

For each bucket choose a random fingerprint function

$$f : U \to [M], \qquad M = L^3 = \Theta(\log^9 N).$$

A fingerprint therefore occupies

$$\log M = O(\log \log N)$$

bits.

We maintain the invariant that the fingerprints of the keys currently in a bucket are distinct.

For a bucket containing $b \leq 4L$ keys, a fresh random fingerprint function fails this condition with probability at most

$$\binom{b}{2}\frac{1}{M} \leq \frac{8L^2}{L^3} = \frac{8}{L}.$$

Thus a fresh function is collision-free with probability at least

$$1 - \frac{8}{L}.$$

Whenever a collision occurs, resample the local fingerprint function until all keys in that bucket have distinct fingerprints and rebuild its local metadata.

For a query key x, we compute its bucket and $f(x)$. If the local dictionary contains this fingerprint, it returns a candidate position. We then compare the full w-bit key stored at that position with x.

8.7.4 A Constant-Time Dictionary Inside a Bucket

Each bucket stores the map

$$f(x) \longmapsto \text{position}(x).$$

Both sides use

$$O(\log\log N)$$

bits.

Store this map in a B-tree of fanout

$$F = \lfloor \sqrt{\log N} \rfloor.$$

A node contains $O(F)$ fingerprints, local positions, and local child indices, and therefore has

$$O(F \log\log N) = O(\sqrt{\log N}\log\log N) = o(\log N)$$

bits.

Consequently there are only

$$2^{o(\log N)} = N^{o(1)}$$

possible node states. The input to any elementary node operation also has only $O(\log\log N)$ bits, so all possible node operations—searching within a node, inserting or deleting an entry, splitting, merging, and choosing a child—can be stored in one global

$$N^{o(1)}$$

lookup table.

A bucket contains at most $4L = O(\log^3 N)$ entries. Hence the B-tree depth is

$$O\left(\log_F(\log^3 N)\right) = O(1),$$

since

$$\log_F(\log^3 N) = \frac{3\log\log N + O(1)}{\frac{1}{2}\log\log N + O(1)} = 6 + o(1).$$

Thus lookup, insertion, deletion, splitting, and merging in the local dictionary all take

$$\boxed{O(1)}$$

time.

There is one dictionary entry per stored key, each using $O(\log\log N)$ bits. B-tree node overhead and bucket roots are smaller. Hence all local dictionaries use

$$\boxed{O(n\log\log N)}$$

bits.

8.7.5 Cost of Fingerprint Rebuilding

An ordinary update costs $O(1)$. The only expensive event is an insertion whose fingerprint collides with an existing fingerprint, forcing a rebuild of its $O(L)$-key bucket.

We first quantify how often this happens.

Suppose the current bucket contains $b \le 4L$ keys and its current random function is collision-free on those keys. For a new key whose fingerprint has not previously been exposed under this function, its value is uniform and independent of the b current values. Hence

$$\Pr[\text{collision}] = \frac{b}{M} \le \frac{4L}{L^3} = \frac{4}{L^2}.$$

The same bound follows for a fixed oblivious update sequence by deferred decisions even when keys are deleted and later reinserted: keys that coexisted with x are already known not to collide with it, while fingerprints first exposed while x was absent are uniform subject only to avoiding the $O(L)$ fingerprints then present. This changes the denominator from M to $M - O(L)$, and hence, for sufficiently large N,

$$\Pr[\text{collision} \mid \text{entire past}] \le \frac{5}{L^2}.$$

We will use

$$p := \frac{5}{L^2}.$$

If a collision occurs, a fresh fingerprint function on the whole bucket fails to be injective with probability at most

$$\frac{8}{L}.$$

Thus the number G of trials needed to obtain a good function satisfies

$$\Pr[G > r] \le \left(\frac{8}{L}\right)^r.$$

Taking

$$r = \lceil \log N \rceil,$$

$$\Pr[G > r] \le \left(\frac{8}{\log^3 N}\right)^{\log N} = \exp\left(-\Omega(\log N \log\log N)\right) = N^{-\Omega(\log\log N)}.$$

A union bound over any polynomial number of rebuilds preserves a $N^{-\omega(1)}$ failure probability.

Conditioned on $G \le \log N$, a rebuild costs

$$O(L \log N) = O(\log^4 N)$$

work.

8.7.6 Deamortizing Rebuilds

An individual rebuild still costs polylogarithmic time. We now turn this into constant time per external operation.

Set

$$q = \lfloor N^{1/4} \rfloor.$$

Maintain two deterministic $\sqrt{N}$-ary tries, each holding at most q pending updates.

The *growing buffer* receives new updates. The *shrinking buffer* contains the previous batch, which is being applied to the hash table. Queries first inspect the growing buffer, then the shrinking buffer, and then the main table, so pending updates are immediately visible.

Every external operation performs a fixed constant amount of work on the shrinking batch. When the growing buffer reaches q updates, we swap the two buffers.

It remains only to prove that a batch of q updates requires $O(q)$ total work with high probability.

Let

$$I_j = \mathbf{1}[\text{the } j\text{-th update causes a fingerprint collision}], \qquad C = \sum_{j=1}^{q} I_j.$$

Let $\mathcal{F}_{j-1}$ contain the entire history before update j. From the preceding calculation,

$$\Pr[I_j = 1 \mid \mathcal{F}_{j-1}] \le p = \frac{5}{L^2}.$$

Although the I_j's need not be independent, the usual Chernoff proof still applies. Indeed, for every $\lambda \ge 0$,

$$\begin{aligned}\mathbb{E}[e^{\lambda I_j} \mid \mathcal{F}_{j-1}] &= 1 + (e^\lambda - 1)\Pr[I_j = 1 \mid \mathcal{F}_{j-1}] \\ &\le 1 + p(e^\lambda - 1).\end{aligned}$$

Therefore, iterating conditional expectation,

$$\mathbb{E}e^{\lambda C} \le \left(1 + p(e^\lambda - 1)\right)^q.$$

This is exactly the MGF bound for a $\mathrm{Bin}(q, p)$ random variable. Hence all the standard binomial Chernoff upper-tail bounds remain valid.

Put

$$\mu = qp = \frac{5q}{L^2} = \frac{5q}{\log^6 N}.$$

Taking the standard $\delta = 1$ Chernoff bound,

$$\Pr[C \ge 2\mu] \le \left(\frac{e}{4}\right)^\mu \le e^{-\mu/3}.$$

Since $q = N^{1/4}$,

$$e^{-\mu/3} = \exp\left(-\Omega\left(\frac{N^{1/4}}{\log^6 N}\right)\right) = N^{-\omega(1)}.$$

Thus, with probability $1 - N^{-\omega(1)}$,

$$\boxed{C \le \frac{10q}{\log^6 N}.}$$

We already showed that, with probability $1 - N^{-\omega(1)}$, every fingerprint rebuild requires at most $\log N$ trials. Since one trial rebuilds an $O(L) = O(\log^3 N)$-key bucket, every collision costs at most

$$O(L \log N) = O(\log^4 N)$$

work. Therefore all rebuilds in the batch cost

$$O\left(\frac{q}{\log^6 N} \cdot \log^4 N\right) = O\left(\frac{q}{\log^2 N}\right) = O(q).$$

Together with the $O(q)$ ordinary update work, the entire batch costs $O(q)$ with probability $1 - N^{-\omega(1)}$.

Choosing the constant amount of draining work per external operation larger than this hidden constant ensures that the shrinking buffer is empty before the growing buffer fills.

A union bound over any polynomial number of batches still gives high probability.

8.7.7 The Update Buffers Are Succinct

A $\sqrt{N}$-ary trie reads

$$\frac{1}{2}\log N$$

bits of the key per level. Since $w = \Theta(\log N)$, its depth is $O(1)$.

A deliberately crude representation already suffices. There are $O(q)$ nodes and each node may contain a direct-access array of $\sqrt{N}$ child pointers, each of $O(\log N)$ bits. Thus one buffer uses at most

$$O(q\sqrt{N}\log N) = O(N^{3/4}\log N) = o(N)$$

bits.

The two buffers therefore do not affect the $nw + O(n \log\log n)$ space bound.

8.7.8 Dynamic Resizing

Finally, N must track the current number of keys.

Use the usual constant-factor hysteresis so that

$$N = \Theta(n).$$

When a resize is triggered, initialize a new table with capacity $N' = \Theta(n)$ and migrate a constant number of old keys per external operation.

Crucially, when a key is inserted into the new table, immediately delete its full representation from the old table. Thus a key is never stored in both tables.

If the old and new tables currently contain n_1, n_2 keys, then

$$n_1 + n_2 = n,$$

and their full-key payload occupies

$$n_1 w + n_2 w = nw$$

bits, apart from the $O(n)$ block slack already accounted for.

Both tables together contain only $O(n)$ keys, so their combined metadata uses

$$O(n \log\log n)$$

bits.

Queries inspect both tables during migration. Since resizing is triggered only after a constant-factor change in n, there are $\Theta(n)$ external operations available before another resize can be necessary, enough to finish the migration at constant additional work per operation.

8.7.9 Summary

We obtain, throughout the execution,

$$\boxed{nw + O(n \log\log n)}$$

bits of space.

Queries take worst-case $O(1)$ time. Updates take $O(1)$ external work, and with probability $1 - n^{-\omega(1)}$ every background batch finishes before its buffer is needed again.

8.8 Simulating Fully Random Hashing

A truly random function

$$h : U \to V$$

cannot be stored explicitly when U is large. For the hash-table applications above, however, we only need a function to look fully random on small fixed subsets of U.

The following construction gives exactly that.

8.8.1 Private Random Cells

Fix

$$S \subseteq U, \qquad |S| \leq n^{0.8}.$$

Choose a sufficiently large constant c. Independently for $j = 1, \ldots, c$, choose a universal hash function

$$g_j : U \to [n^{0.9}]$$

and an array

$$A_j[1, \ldots, n^{0.9}]$$

of independent uniform elements of the additive group V.

Define

$$\boxed{h(x) = \sum_{j=1}^{c} A_j[g_j(x)].}$$

Lemma 289 (Full Randomness on a Fixed Small Set)**.** *For every constant K, c can be chosen as a constant depending only on K such that, for every fixed $S \subseteq U$ with $|S| \leq n^{0.8}$,*

$$\Pr[h|_S \text{ is a fully random function}] \geq 1 - n^{-K}.$$

Proof. For $x \in S$, call the cell $A_j[g_j(x)]$ *private* if no other $y \in S$ accesses it.

For fixed x, j, universality and a union bound give

$$\begin{aligned}\Pr[A_j[g_j(x)] \text{ is not private}] &\leq \sum_{y \in S \setminus \{x\}} \Pr[g_j(x) = g_j(y)] \\ &\leq \frac{|S| - 1}{n^{0.9}} \\ &\leq n^{-0.1}.\end{aligned}$$

The functions $g_1, \ldots, g_c$ are independent, so

$$\Pr[x \text{ has no private cell}] \leq n^{-0.1c}.$$

Union-bounding over the at most $n^{0.8}$ elements of S,

$$\Pr[\exists x \in S \text{ with no private cell}] \leq n^{0.8-0.1c}.$$

Choosing

$$c \geq 10K + 8$$

makes this at most n^{-K}.

Condition now on the g_j's and on every $x \in S$ having a private cell. Choose one private cell

$$A_{j_x}[g_{j_x}(x)]$$

for each x.

These selected cells are all distinct. Moreover, the selected cell for x is not used in the hash value of any other key in S.

Condition on every array entry except these selected cells. Then, for every $x \in S$,

$$h(x) = A_{j_x}[g_{j_x}(x)] + c_x,$$

where c_x is now fixed.

The selected array entries are mutually independent and uniform in V. Adding fixed constants preserves uniformity, so

$$(h(x))_{x \in S}$$

is a vector of mutually independent uniform elements of V. Thus $h|_S$ is exactly a fully random function. □

The construction uses

$$\widetilde{O}(n^{0.9})$$

random bits when U, V have polynomial size, and evaluates h using a constant number of hash evaluations and table lookups.

8.8.2 Splitting a Large Set into Small Fixed Sets

The lemma only applies to fixed sets of size at most $n^{0.8}$. To use it for n stored keys, first split those keys into much smaller buckets.

Choose an independent function

$$h_1 : U \to [B], \qquad B = \lceil \sqrt{n} \rceil,$$

using the preceding construction.

We first prove that every bucket has

$$(1 + o(1))\sqrt{n}$$

keys.

For the analysis only, partition the n stored keys into

$$G = \lceil n^{0.2} \rceil$$

fixed groups

$$S_1, \ldots, S_G, \qquad |S_j| \le n^{0.8}.$$

Choose the constant c in Lemma 289 large enough that the failure probability for each group is at most n^{-10}. A union bound gives

$$\Pr[h_1 \text{ is fully random on every } S_j] \ge 1 - Gn^{-10} = 1 - n^{-\Omega(1)}.$$

Condition on this event.

For destination bucket i and group S_j, let

$$X_{i,j} = |\{x \in S_j : h_1(x) = i\}|.$$

Then

$$X_{i,j} \sim \mathrm{Bin}\left(|S_j|, \frac{1}{B}\right),$$

with mean

$$\mu_{i,j} = \frac{|S_j|}{B} \le n^{0.3}.$$

Set

$$a = n^{0.15} \log n.$$

The Bernstein inequality gives

$$\Pr[|X_{i,j} - \mu_{i,j}| \ge a] \le 2 \exp\left(-\frac{a^2}{2(\mu_{i,j} + a/3)}\right).$$

For sufficiently large n,

$$a \le n^{0.3},$$

so

$$2(\mu_{i,j} + a/3) \le 3n^{0.3}.$$

Hence

$$\Pr[|X_{i,j} - \mu_{i,j}| \geq a] \leq 2\exp\left(-\frac{n^{0.3}\log^2 n}{3n^{0.3}}\right)$$
$$= 2e^{-\log^2 n/3}.$$

There are at most

$$BG \leq 2n^{0.7}$$

pairs (i, j), so a union bound gives

$$\Pr[\exists i, j : |X_{i,j} - \mu_{i,j}| > a] \leq 4n^{0.7}e^{-\log^2 n/3} = n^{-\omega(1)}.$$

Therefore, simultaneously for every bucket i,

$$\begin{aligned}|B_i| &= \sum_j X_{i,j} \\ &= \sum_j \mu_{i,j} + O(Ga) \\ &= \frac{n}{B} + O(n^{0.35}\log n) \\ &= \sqrt{n} + o(\sqrt{n}).\end{aligned}$$

Thus

$$\boxed{\max_i |B_i| = (1 + o(1))\sqrt{n}}$$

with high probability.

8.8.3 Randomize After Splitting

Now choose a second, independent function h_2 from the private-cell construction.

Condition on h_1 and on the event that every top-level bucket has $(1 + o(1))\sqrt{n}$ keys. The sets

$$B_1, \ldots, B_B$$

are now fixed, and for sufficiently large n,

$$|B_i| < n^{0.8}.$$

By Lemma 289, for any desired constant K, we may choose the constant number of tables in h_2 so that

$$\Pr[h_2|_{B_i} \text{ is not fully random}] \leq n^{-K}$$

for each fixed i.

There are only $O(\sqrt{n})$ buckets. Hence

$$\Pr[\exists i : h_2|_{B_i} \text{ is not fully random}] \leq O(\sqrt{n})\, n^{-K}.$$

Taking K sufficiently large gives high probability that

$$\boxed{h_2 \text{ is fully random on every bucket simultaneously.}}$$

8.9 Soft Heaps

A soft heap saves comparisons by allowing a controlled number of keys to increase. This is not an approximation to the numerical value of the minimum: a corrupted key may increase arbitrarily. Instead, the guarantee controls how many items are affected. The implementation below uses ordinary heap-ordered ternary trees, except that high nodes combine two items into one shared-key pool.

Definition 290 (Soft heap)**.** Each item has an original key and a current key that never decreases and is never smaller than its original key. An item whose key has increased is *corrupted*. After N insertions, an ε-soft heap has at most εN corrupted items still present. Here N counts all insertions, not just the items currently present; when heaps are melded, their insertion counts add. Meld consumes two disjoint heaps with the same error parameter.

Find-min and extract-min use the current keys. An extraction returns a minimum *before* its subsequent repairs, together with the items newly corrupted by those repairs. No corruption will occur during insertion or melding. Break ties by insertion identifiers, so the keys can be treated as distinct.

Theorem 291 (Ternary soft heap)**.** *For $0 < \varepsilon \le 1/2$, the construction below supports insert and meld in amortized $O(1)$ time, find-min in worst-case $O(1)$ time, and extract-min in amortized $O(\log(1/\varepsilon))$ time. More precisely, N insertions, D extractions, M melds, and F find-min queries take*

$$O\big(N + M + F + D\log(1/\varepsilon)\big)$$

total time and $O(N)$ space. Reporting newly corrupted items is included.

8.9.1 A Forest of Heaps

A rank-r tree is a perfect ternary tree with 3^r original leaves. Each leaf records one insertion; items subsequently move only upward. A node stores either one representative item or nothing. Whenever a node is empty but its subtree is not, refill it from the child with minimum representative key, then recursively refill that child. An empty node after refilling therefore means that its entire subtree is empty.

Keep the nonempty trees in a doubly linked list of nondecreasing ranks, with at most two roots of each rank. Three rank-r roots are linked under a new rank-$(r+1)$ root, which is filled by one ordinary upward pull. This is carrying in base 3. A rank-r link costs $O(r)$, but at most $N/3^r$ rank-r trees are ever created: their insertion leaves are disjoint. Thus all linking work is $O(N\sum_{r\ge1} r/3^r) = O(N)$.

The obstacle is extraction. Pulling a separate representative through every level for every deletion can cost $\Theta(\log N)$ each. Above a threshold, we will make each upward representative account for two lower ones.

8.9.2 Combine Two Representatives, Keep the Larger Key

Every representative e carries a linked list $C(e)$ of corrupted items, all assigned current key $\mathrm{key}(e)$. The representative itself retains its original key. Thus a pool is $\{e\} \cup C(e)$, with e the largest original key in that pool. Lists are concatenated in constant time.

Set $h = \lceil \log_3(2/\varepsilon) \rceil$. At ranks at most h, refill in exactly the ordinary way. At a higher rank, pull two consecutive minimum representatives e_1, e_2 from the children, refilling after each pull. Their keys satisfy $\mathrm{key}(e_1) < \mathrm{key}(e_2)$. Keep e_2 and concatenate $C(e_2)$, $\{e_1\}$, and $C(e_1)$ into its corruption list. Only e_1 is newly corrupted; the other list members were already corrupted. If just one representative remains, keep it without combining.

This is the saving: two representatives arrive from below, but only one continues upward as a representative. The discarded ordering information is recorded explicitly as corruption.

Use a Boolean argument soft to distinguish deletion repairs from insertion or meld repairs. The latter always make single pulls, even at high ranks. In the following routines, $\mathcal{R}$ collects newly corrupted items during the current extraction; $\perp$ denotes an absent item.

Algorithm 40 Pulling representatives and refilling an empty node

```
1:  function Pull(v, soft)
2:      Choose a child u of v with minimum representative key
3:      if no child has a representative then
4:          return ⊥
5:      e ← u.rep; u.rep ← ⊥
6:      Fill(u, soft)
7:      return e                                   ▷ Move its entire pool with it.

8:  procedure Fill(v, soft)
9:      e ← Pull(v, soft)
10:     if e ≠ ⊥ and soft and rank(v) > h then
11:         f ← Pull(v, soft)
12:         if f ≠ ⊥ then
13:             C(f) ← C(f) ‖ [e] ‖ C(e)
14:             Append e to R; e ← f
15:     v.rep ← e

16: function Link(u1, u2, u3)
17:     Create an empty parent v of rank rank(u1) + 1
18:     Give v children u1, u2, u3
19:     Fill(v, false)
20:     return v
```

The symbol ‖ denotes linked-list concatenation. Only representatives' list headers are live; after splicing $C(e)$ into $C(f)$, the old header can be cleared. At a leaf, Pull returns $\perp$. Every call to Fill has an empty node as its input.

Lemma 292 (Ordering invariant)**.** *Representatives are heap ordered by their original keys. Every item in a representative's corruption list has original key at most its representative's key, and no item's current key decreases.*

Proof. A single pull removes the smallest child representative and replaces it by one whose key is no smaller. It therefore produces a representative no larger than any left in the children. For a double pull, the second key is no smaller than the first, and every remaining child key is at least the second. Attaching the first pool to the second raises its common key and leaves the second pool's key unchanged. Apply this argument inductively to the recursive repairs. Moving a pool upward without combining it changes no current key. □

8.9.3 Root Lists and Heap Operations

At each root-list position store a pointer to a minimum representative in the suffix beginning there. The first pointer finds the global minimum in constant time. If a root of rank r changes, recompute suffix minima backward from that position; at most $2(r+1)$ positions are affected. A pool member has the same current key as its representative, so find-min may return any member of the minimum pool.

To meld, merge the sorted root lists and carry whenever three equal ranks meet. Stop once one input list and the carry are empty: the other input's remaining suffix is unchanged and can be spliced in. Only the newly written prefix needs new suffix-minimum pointers. An absent list head or carry has rank $+\infty$ for the rank comparison below.

Algorithm 41 Melding and inserting

```
1:  function Meld(L_1, L_2)
2:      P ← empty list; c ← ⊥
3:      while (L_1 ≠ ∅ and L_2 ≠ ∅) or c ≠ ⊥ do
4:          r ← minimum rank among the input heads and c
5:          Remove all rank-r roots from the two heads and carry into a list G
6:          Clear the carry if it was removed
7:          if |G| ≥ 3 then
8:              Remove three roots u_1, u_2, u_3 from G
9:              c ← Link(u_1, u_2, u_3)
10:         Append the remaining roots of G to P
11:     Splice the remaining input suffix after P
12:     Recompute suffix minima backward on P only
13:     return the resulting root list

14: procedure Insert(L, e)
15:     C(e) ← ∅; create a rank-zero root v holding e
16:     L ← Meld(L, [v])
```

Algorithm 42 Extracting a current minimum

```
1:  function ExtractMin(L)
2:      if L = ∅ then
3:          return Empty
4:      v ← root named by the first suffix-minimum pointer
5:      e ← v.rep; κ ← key(e)
6:      if C(e) ≠ ∅ then
7:          Remove an item f from C(e); return (f, κ, ∅)
8:      v.rep ← ⊥; R ← ∅
9:      Fill(v, true)
10:     Remove v from L if its subtree is empty
11:     Repair suffix minima backward through the changed prefix
12:     return (e, κ, R)
```

A rank group has at most five roots: two from each heap and one carry. Consequently one link leaves at most two roots and one new carry. Find-min uses the same first three nonempty-heap steps as ExtractMin, returning a pool member and κ without modifying anything.

8.9.4 Why Only Few Items Are Corrupted

Let c_r bound a corruption list at a rank-r node. Below the threshold there are no corruptions. Above it, a double pull combines two lists and one newly corrupted representative, whereas a single pull only moves a list:

$$c_r = 0 \quad (r \le h), \qquad c_r \le 2c_{r-1} + 1 \quad (r > h).$$

Hence $c_{h+j} \le 2^j - 1$. There are at most $N/3^r$ nodes of rank r, even when all nodes over the whole history are counted. Pools partition the present items, so the number of corrupted items is at most

$$\sum_{j \ge 1} \frac{N}{3^{h+j}}(2^j - 1) \le \frac{N}{3^h} \sum_{j \ge 1} (2/3)^j = \frac{2N}{3^h} \le \varepsilon N.$$

This explains the ternary branching: pool sizes grow by at most a factor 2 per level, while the number of available nodes falls by a factor 3. Binary branching would not make this geometric sum converge.

8.9.5 Amortized Cost

Insertions and melds. A link producing rank $r+1$ makes only single pulls and costs $O(r+1)$. Root-list merging and suffix rebuilding cost the scanned prefix length. These costs are paid by a constant multiple of the nonnegative potential

$$\sum_{\text{heaps } H} \left(2 \sum_{v \text{ root of } H} (\text{rank}(v)+1) + R_H + 1 \right),$$

where R_H is the maximum root rank and is -1 for an empty heap. A link of three rank-r roots decreases the weighted root sum by $4r+2$, and can increase the maximum-rank term by at most one. This pays for its pull path and for list positions processed by a carry. Before carries, merging two nonempty heaps releases $\min\{R_{H_1}, R_{H_2}\}+1$ from the maximum-rank terms. This pays for merging the prefixes through the smaller maximum rank. Any further processing is a carry chain already charged to links. Creating a singleton increases the potential by a constant. Thus insert and meld have constant amortized cost. Extractions only remove roots, so cannot increase this potential.

Refilling below the threshold. A maximal single-pull path at ranks at most h costs $O(h+1)$. It is started either by an extraction at a low root or by moving one representative from rank h to rank $h+1$. Distinct such boundary crossings use distinct original items, because representatives only move upward. Every crossed item has either been extracted, remains a representative at a high node, or remains corrupted. There are at most D of the first kind, at most $\sum_{r>h} N/3^r = O(N/3^h)$ of the second, and $O(N/3^h)$ of the third by the preceding calculation. Thus all low-rank work costs $O((h+1)(D+N/3^h)) = O(N + D(h+1))$.

Refilling above the threshold. Every double fill newly corrupts one representative, and an item can lose representative status only once. There are therefore at most N double fills. A high node can have an unpaired final fill at most once, when the last representative leaves its children. Empty final repairs can likewise be charged to nodes. Since only $O(N)$ nodes are ever created, the local work at high nodes is $O(N)$. Recursive low-rank work was counted separately.

Repairing root minima. The first $h+1$ ranks cost $O(h+1)$ per extraction. To charge the additional cost at a high root, let U_r be the number of representatives ever placed at rank r, counting all nodes over the entire execution. We have $U_h \le N$. A usual high-rank placement consumes two representatives from rank $r-1$. There are only $O(N/3^r)$ exceptions: final unpaired fills, and exact fills caused by linking at rank r or higher. For the latter, each rank-s link causes at most one exact placement at rank r, and $\sum_{s \ge r} N/3^s = O(N/3^r)$. Consequently

$$U_r \le \tfrac{1}{2} U_{r-1} + O(N/3^r), \qquad U_{h+j} = O(N/2^j).$$

The recurrence follows by counting consumed representatives; its solution follows by summing the geometric error terms. Root removals contribute at most one additional change per historical node. Charging the extra $O(j)$ root-list repairs to these changes gives

$$O\left(N \sum_{j \ge 1} \frac{j}{2^j} \right) = O(N).$$

Together these estimates prove Theorem 291.

The construction does not need to maintain a nearly sorted order on every item. It keeps exact order on representatives, lets the pools share their largest key, and makes the number of representatives shrink faster than the cost of moving through the upper levels grows.

8.10 Sampling

Write UniformInt(N) for an exact uniform integer in $\{0, \dots, N-1\}$. Continuous-key algorithms below are stated in the real-arithmetic model.

8.10.1 Alias Tables

Let $w_1, \dots, w_n$ be nonnegative integer weights with $W = \sum_i w_i > 0$. We want to return i with probability w_i/W.

A prefix-sum structure locates a random point in the total mass by searching. An alias table instead rearranges the same mass into n equal columns, each containing at most two outcomes. Sampling then requires only a uniform column and one threshold comparison.

Scale the mass of item i to $b_i = nw_i$, and give every column capacity W. Total scaled mass and total capacity are both nW. If $b_s < W$, fill column s with all b_s units of item s and the remaining $W - b_s$ units from some item ℓ with $b_\ell \geq W$. The latter becomes the alias of s.

Algorithm 43 Building and sampling an integer alias table

```
function BuildAlias(w_1, ..., w_n)
    W ← Σ_i w_i; require W > 0
    b_i ← n w_i for every i
    S ← {i : b_i < W}, L ← {i : b_i ≥ W}
    while S ≠ ∅ and L ≠ ∅ do
        remove s from S and ℓ from L
        cut[s] ← b_s
        alias[s] ← ℓ
        b_ℓ ← b_ℓ − (W − b_s)
        if b_ℓ < W then
            insert ℓ into S
        else
            insert ℓ into L
    for each remaining i ∈ S ∪ L do
        cut[i] ← W
        alias[i] ← i
    return (W, cut, alias)

function AliasSample(W, cut, alias)
    n ← |cut|
    j ← 1 + UniformInt(n)
    r ← UniformInt(W)
    if r < cut[j] then
        return j
    else
        return alias[j]
```

Theorem 293 (Alias sampling)**.** *The alias table uses $O(n)$ preprocessing time and space and returns item i with probability w_i/W using $O(1)$ table operations per sample.*

Proof. If r columns remain, their unassigned masses sum to rW. Filling one column removes exactly W mass and one column, preserving the invariant. Hence a mass below W cannot remain unless some mass is at least W. If no small mass remains, every remaining mass must equal W, since their average is W.

Every item's original nw_i units are therefore placed into the n columns without changing their identity. Sampling a uniform column and then a uniform integer position in that column chooses each of the nW positions with probability $1/(nW)$. Exactly nw_i positions contain item i, so its probability is w_i/W.

Each iteration finishes one column, giving linear preprocessing. □

8.10.2 Exponential Races

Independent exponential clocks convert weighted sampling into taking minimums.

For each positive weight w_i, draw an independent $X_i \sim \text{Exp}(1)$ and put

$$E_i = \frac{X_i}{w_i}.$$

Then E_i is exponential with rate w_i, since $\Pr[E_i > t] = e^{-w_i t}$.

Theorem 294 (Exponential race)**.** *Let $W = \sum_i w_i$. Then*

$$\boxed{\Pr[E_i = \min_j E_j] = \frac{w_i}{W}.}$$

More generally, if $i_1, \dots, i_k$ are the indices of the k smallest clocks in increasing order, then for distinct indices

$$\boxed{\Pr[i_1, \dots, i_k] = \prod_{r=1}^{k} \frac{w_{i_r}}{W - \sum_{\ell<r} w_{i_\ell}}.}$$

Thus sorting the independent clocks gives successive proportional sampling without replacement.

Proof. The density of E_i at t is $w_i e^{-w_i t}$. Hence

$$\begin{aligned}\Pr[E_i = \min_j E_j] &= \int_0^\infty w_i e^{-w_i t} \prod_{j \neq i} e^{-w_j t}\, dt \\ &= \int_0^\infty w_i e^{-W t}\, dt = \frac{w_i}{W}.\end{aligned}$$

After the first clock at time t, condition each remaining clock on exceeding t. Exponentials are memoryless:

$$\Pr[E_j - t > s \mid E_j > t] = e^{-w_j s}.$$

Thus the residual clocks are again independent exponentials with their original rates. Repeating the first-clock argument proves the ordered formula. □

This representation is valuable algorithmically because only the smallest keys matter. In a stream, the k smallest clocks can be maintained in $O(k)$ space and $O(n \log(k+1))$ time with a max-heap.

It is also automatically mergeable.

Proposition 295 (Mergeability of bottom-k keys)**.** *Partition the items into disjoint batches. If each batch retains its k smallest keys, then the global k smallest keys are the k smallest among the retained keys.*

Proof. Any item discarded by its batch already has at least k smaller keys in that same batch, so it cannot belong to the global k smallest. □

8.10.3 Priority Sampling

Exponential races are designed to sample items. A different random-key construction is useful when the goal is to retain only k weighted items while later estimating the total weight of *any* requested subset.

Let items have nonnegative weights w_i. Independently draw U_i uniformly from $(0, 1)$, and assign priority

$$q_i = \frac{w_i}{U_i}.$$

Keep the k largest priorities. Let τ be the $(k+1)$-st largest priority. For a retained item define

$$\widehat{w}_i = \max\{w_i, \tau\},$$

and for an omitted item put $\widehat{w}_i = 0$.

If there are at most k positive-weight items, retain all of them and simply set $\widehat{w}_i = w_i$.

Theorem 296 (Priority sampling)**.** *For every item i,*

$$\boxed{\mathbb{E}[\widehat{w}_i] = w_i.}$$

Consequently, for every subset A,

$$\boxed{\widehat{W}(A) = \sum_{i \in A} \widehat{w}_i}$$

is an unbiased estimator of $W(A) = \sum_{i \in A} w_i$.

Proof. Fix i, and condition on all priorities except q_i. Let τ_i be the k-th largest priority among the other items. Then i is retained exactly when $q_i > \tau_i$. If it is retained, the global $(k+1)$-st priority is exactly τ_i.

If $\tau_i \le w_i$, then $q_i = w_i/U_i \ge w_i \ge \tau_i$ always, so i is retained and $\widehat{w}_i = w_i$.

Suppose instead that $\tau_i > w_i$. Then

$$q_i > \tau_i \iff U_i < \frac{w_i}{\tau_i},$$

which occurs with probability w_i/τ_i. When this occurs, $\widehat{w}_i = \tau_i$; otherwise it is zero. Therefore, conditioned on the other priorities,

$$\mathbb{E}[\widehat{w}_i] = \tau_i \frac{w_i}{\tau_i} = w_i.$$

Removing the conditioning proves the first claim, and linearity of expectation proves the subset-sum claim. □

The estimator has a useful interpretation. Heavy items with $w_i \ge \tau$ are represented by their exact weight. Lighter retained items all receive the common threshold estimate τ. Thus the sample spends its limited capacity on heavy items while randomly representing the tail.

Algorithm 44 Priority sampling

```
function PrioritySample((w_i)_{i=1}^n, k)
    H ← an empty min-heap of capacity k + 1
    for i = 1, ..., n with w_i > 0 do
        draw U_i uniformly from (0, 1)
        q_i ← w_i / U_i
        insert (q_i, i, w_i) into H
        if |H| > k + 1 then
            delete the minimum-priority item from H
    if |H| ≤ k then
        return every retained item with estimate w_i
    remove the minimum-priority item and let its priority be τ
    for each remaining (q_i, i, w_i) ∈ H do
        ŵ_i ← max{w_i, τ}
    return the k retained items, their estimates, and τ
```

The construction uses $O(k)$ space and $O(n \log(k+1))$ time. It is also mergeable: if the items are partitioned into disjoint batches, retaining the $k+1$ largest priorities from each batch is sufficient to recover the global $k+1$ largest priorities and therefore both the final sample and its threshold τ. The random priority of an item must be generated once and preserved; resampling priorities during a merge changes the distribution.

9 Trees

9.1 LCA

1. Pre-Order Reduction Let $\texttt{dfs}[0 \dots n-1]$ be the sequence of nodes visited in a standard pre-order DFS traversal. Let $\texttt{pos}[u]$ be the pre-order index of node u, such that $\texttt{dfs}[\texttt{pos}[u]] = u$. Let $p[u]$ be the parent of node u, and let $\text{depth}[u]$ be its distance from the root.

We construct our $O(n)$ RMQ from Algorithm 37 over an implicit array Z of size n, defined by the depth of the parent of the node at each pre-order step:

$$Z[i] = \text{depth}[p[\texttt{dfs}[i]]]$$

2. Queries To find $LCA(a, b)$, we first ensure that a is visited before b by swapping them if $\texttt{pos}[a] > \texttt{pos}[b]$. If $a = b$, the LCA is simply a. Otherwise, we query the RMQ for the index m that minimizes Z in the range $[\texttt{pos}[a] + 1, \texttt{pos}[b]]$. The LCA is exactly the parent of the node at that minimum index: $p[\texttt{dfs}[m]]$.

3. Correctness There are two cases:

- **Case 1 (a is an ancestor of b):** The pre-order interval $[\texttt{pos}[a] + 1, \texttt{pos}[b]]$ contains only proper descendants of a. The node in this interval with the shallowest parent is the direct child of a that lies on the path to b. The parent of this child is a.
- **Case 2 (a and b are in separate branches):** Let $L = LCA(a, b)$. After the DFS visits a and finishes its subtree, it backtracks to L and eventually descends into the branch containing b. Let v be the direct child of L that roots the subtree containing b. The node v is visited after a and before b, so $\texttt{pos}[v] \in [\texttt{pos}[a] + 1, \texttt{pos}[b]]$. Its parent is L. Because every other node visited in this interval is a descendant of L, their parents have a depth of at least $\text{depth}[L]$. Thus, the RMQ identifies v, and returns its parent $p[v] = L$.

Algorithm 45 LCA

```
1:  function InitLCA(n, p, dfs, pos, depth)
                                  ▷ Define a closure to compare the parent depths of two pre-order indices
2:      cmp ← Closure(i, j) returning depth[p[dfs[i]]] < depth[p[dfs[j]]]
                                  ▷ Initialize the RMQ data structure using this closure
3:      rmq ← RMQ_Init(n, cmp)
4:      return LCA_Struct{p, dfs, pos, rmq}
5:  function QueryLCA(a, b)
6:      if a = b then
7:          return a
                                  ▷ Order by entry time
8:      l ← min(pos[a], pos[b])
9:      r ← max(pos[a], pos[b])
10:     m ← rmq.query(l + 1, r)
11:     return p[dfs[m]]
```

9.2 $O(n)$ Binary Lifting

Binary Lifting usually requires $O(n \log n)$ time and space to precompute an ancestor table, but using a modified method we can do it in $O(n)$ preprocessing and space.

1. Preprocessing We define $\texttt{jmp}[v]$ as a pointer to an ancestor of v. The idea is to grow the jump distances in powers of two as we ascend.

During a pre-order traversal, consider a node v with parent p. We look at the jump from the parent $p \to \texttt{jmp}[p]$, and the subsequent jump $\texttt{jmp}[p] \to \texttt{jmp}[\texttt{jmp}[p]]$. If these two jumps cover the exact same change in depth, we combine them. The new jump from v covers the edge (v, p), the first jump of length L, and the second jump of length L. Thus, the new jump pointer points to $\texttt{jmp}[\texttt{jmp}[p]]$ and covers a distance of $1 + 2L$.

If the two preceding jumps do not match in length, we set $\texttt{jmp}[v] = p$, resetting the jump length to 1.

2. Querying The algorithm has two phases:

1. **Equate Depths:** If u and v are at different depths, we identify the deeper node and use our `depth_jmp` function to lift it exactly to the depth of the shallower node, this works basically the same as the LCA function. If the nodes are now identical, we have found the LCA.

2. **Simultaneous Ascent:** If the nodes are distinct but at the same depth, we ascend them simultaneously. We evaluate the jump pointers $\texttt{jmp}[u]$ and $\texttt{jmp}[v]$:

   - If $\texttt{jmp}[u] \neq \texttt{jmp}[v]$, the jump lands strictly below the LCA. We can safely take this maximal jump: $u \leftarrow \texttt{jmp}[u]$ and $v \leftarrow \texttt{jmp}[v]$.
   - If $\texttt{jmp}[u] = \texttt{jmp}[v]$, the jump has either hit the LCA exactly or overshot it into a higher common ancestor. Because we cannot distinguish between these two cases, we take a single step up to the parents: $u \leftarrow p[u]$ and $v \leftarrow p[v]$.

We repeat this until $u = v$, at which point u is the LCA.

3. Complexity Let $L(u) = \text{depth}[u] - \text{depth}[\texttt{jmp}[u]]$. By construction, $L(u)$ is always of the form $2^k - 1$ for some integer $k \geq 1$. We call k the rank of the jump. A jump of rank k implicitly spans a tree of $2^k - 1$ edges.

Lemma 297 (Monotonicity of Jump Sequences)**.** *For any node u, let $J(u) = [L_1, L_2, \ldots, L_m]$ be the sequence of jump lengths encountered by repeatedly following the `jmp` pointers from u to the root. The sequence $J(u)$ is monotonically non-decreasing.*

Proof. The sequence of jumps from a node depends entirely on its depth d. Let J_d denote the sequence of jump lengths from a node at depth d.

By induction, all jump lengths are drawn from the set of skew-binary weights: $\mathcal{W} = \{2^k - 1 \mid k \geq 1\}$.

We will prove by induction on depth d that:

$$J_d \text{ satisfies } L_1 \leq L_2 < L_3 < \cdots < L_m.$$

Base Case ($d = 1$):
Trivial.

Inductive Step:
Assume J_d satisfies the invariant. We consider J_{d+1}:

- **Case A:** J_d does *not* start with identical elements.
  By the invariant, we have $L_1 < L_2 < L_3 < \ldots$
  The algorithm prepends a jump of 1, making $J_{d+1} = [1, L_1, L_2, \ldots]$.
  Because all elements are ≥ 1, we have the invariant.

- **Case B:** J_d starts with two identical elements.
  By the invariant, we have: $J_d = [L, L, L_3, L_4, \ldots]$ with $L < L_3 < L_4 \ldots$
  The algorithm merges the first two jumps, replacing them with $2L{+}1$, making $J_{d+1} = [2L{+}1, L_3, L_4, \ldots]$.
  Because $L \in \mathcal{W}$, the next available weight in $\mathcal{W}$ is $2L{+}1$. Since $L_3 \in \mathcal{W}$ and $L_3 > L$, we have $L_3 \geq 2L{+}1$.
  Therefore, the new sequence satisfies the invariant again.

Thus J_d is always monotonically non-decreasing for any depth d. □

By the lemma, $J(u)$ can contain at most two jumps of any length $2^k - 1$. Thus the entire sequence $J(u)$ consists of at most $2\lfloor \log_2 n \rfloor$ jumps.

During a query, we attempt to follow $J(u)$ to cover a target distance. Every operation falls into one of two categories:

1. Successful Jumps:
When we take `jmp`$[u]$, we consume one of the elements of $J(u)$, so there are at most $O(\log n)$ of these operations.

2. Penalty Steps:
If taking the jump L would overshoot our target we cannot consume it. Instead, we take a penalty step to the parent p. Because $L > 1$ was built by merging a step of 1 and two jumps of size $(L-1)/2$, stepping to the parent "unpacks" L. The new sequence from the parent, $J(p)$, replaces L with two smaller jumps: $[(L-1)/2, (L-1)/2, \dots]$. We only unpack a jump of size L when it is greater than the remaining distance. Because the remaining distance strictly decreases, we will never need to unpack a jump of size L more than once across the query. Thus, the total number of penalty steps is bounded by the number of unique jump sizes, which is $\leq \log_2 n$.

Algorithm 46 Binary Lifting

```
1:  function BuildJumpTable(n, par, dfs, depth)
2:      jmp ← array of size n
3:      for each v in dfs do
4:          p ← par[v]
5:          if v = p then
6:              jmp[v] ← v
7:          else
8:              p_j ← jmp[p]
9:              p_jj ← jmp[p_j]
10:             if depth[p] − depth[p_j] = depth[p_j] − depth[p_jj] then
11:                 jmp[v] ← p_jj
12:             else
13:                 jmp[v] ← p
14:     return jmp
15: function QueryLCA(u, v, par, jmp, depth)
16:     if depth[u] > depth[v] then
17:         Swap u and v
18:     v ← DepthJump(v, depth[u], par, jmp, depth)
19:     while u ≠ v do
                                                  ▷ We would overshot the LCA.
20:         if jmp[u] = jmp[v] then
21:             u ← par[u]
22:             v ← par[v]
23:         else
24:             u ← jmp[u]
25:             v ← jmp[v]
26:     return u
```

9.3 Virtual Tree

Let T be a rooted tree. Suppose a query specifies a set

$$S = \{v_1, \ldots, v_k\}$$

of important vertices.

A *virtual tree* is the smallest compressed tree containing all vertices of S while preserving their ancestor relationships and distances in T.

Preprocess:

- DFS entry times $\mathrm{tin}(v)$;
- depths or root distances;
- an LCA data structure.

Construction Sort the query vertices by DFS order:

$$v_1, \ldots, v_k.$$

Add

$$\mathrm{LCA}(v_i, v_{i+1}) \qquad (1 \le i < k).$$

Sort and deduplicate the resulting set U by DFS order.

Now scan U with a stack. Before inserting v, pop until the stack top is an ancestor of v. Connect v to that stack top and push v.

Algorithm 47 Virtual Tree

Input: Query vertices S
Output: Virtual tree on S and necessary LCAs

```
sort S by tin
U ← S
for i = 1, ..., |S| − 1 do
    add LCA(S_i, S_{i+1}) to U
sort and deduplicate U by tin
stack ← empty
for v ∈ U in DFS order do
    while stack nonempty and top is not an ancestor of v do
        pop stack
    if stack nonempty then
        add virtual edge (top, v)
        assign its weight
            dist_T(top, v)
    push v
```

Lemma 298. *The set U contains every branching vertex required by the minimal subtree spanning S.*

Proof. Any branching vertex x of the spanning subtree is the LCA of two marked vertices lying in two different child subtrees of x.

Take the last marked vertex in DFS order from one such child subtree and the first marked vertex from the next nonempty child subtree. They are consecutive among the marked vertices lying across that boundary, and their LCA is exactly x.

Thus every necessary branching LCA occurs among LCAs of consecutive marked vertices. □

Lemma 299. *After sorting U, the parent of a vertex v in the virtual tree is the deepest earlier vertex of U that is an ancestor of v.*

Proof. DFS order makes the descendants of every vertex contiguous. The stack contains exactly the chain of currently open ancestors. Popping until the top contains v therefore leaves precisely its deepest ancestor in U. □

There are only

$$k - 1$$

added LCAs, so

$$|U| \le 2k - 1.$$

Therefore construction takes

$$O(k \log k)$$

time with $O(1)$ LCA queries, or

$$O(k \log k + k \log n)$$

with binary-lifting LCA.

9.4 XOR-Linked Traversal

XOR-Linked Traversal is a method to traverse a tree bottom-up iteratively with better constant factor.

We store:

- $d[u]$: The degree of node u.
- $p[u]$: The XOR sum of the indices of all neighbors of node u. Initially, $p[u] = \bigoplus_{v \in \text{neighbors}(u)} v$.

If a node u is a leaf, its degree is 1. Therefore, $p[u]$ is simply the index of its only neighbor, v. We can process the edge (u, v), and then delete the edge from the graph using $p[v] \leftarrow p[v] \oplus u$ and decrementing its degree.

If decrementing v's degree makes it a new leaf, we set $u \leftarrow v$ and continue.

Algorithm 48 XOR-Linked Traversal

Input: Number of nodes N, Initial XOR-sum array p, Initial degree array d
Output: Updated arrays p and d, processing nodes bottom-up

```
for i ← 0 to N − 1 do
    u ← i
    while d[u] = 1 do
        d[u] ← 0
        v ← p[u]
        Process(u, v)
        p[v] ← p[v] ⊕ u
        d[v] ← d[v] − 1
        u ← v
return p, d
```

9.5 DFS

The algorithm constructs a DFS order, alongside subtree sizes and depths, with a better constant factor.

Algorithm 49 DFS

Input: Number of nodes N, Parent array p, Degree array d
Output: Arrays $p, ss, pos, dfs, depth$

Initialize $ss \leftarrow [1, \ldots, 1]$ of size N
Initialize $dfs \leftarrow [0, \ldots, 0]$ of size N
Initialize $depth \leftarrow [0, \ldots, 0]$ of size N
$idx \leftarrow N - 1$

for all edge (u, v) processed via XOR-Linked Traversal **do**
    $ss[v] \leftarrow ss[v] + ss[u]$
    $dfs[idx] \leftarrow u$
    $idx \leftarrow idx - 1$

$pos \leftarrow ss$
for $i \leftarrow 1$ **to** $N - 1$ **do**
    $u \leftarrow dfs[i]$
    $v \leftarrow p[u]$
    $depth[u] \leftarrow depth[v] + 1$
    $temp \leftarrow pos[v]$
    $pos[v] \leftarrow pos[v] - pos[u]$
    $pos[u] \leftarrow temp$

for $i \leftarrow 0$ **to** $N - 1$ **do**
    $pos[i] \leftarrow pos[i] - 1$
    $dfs[pos[i]] \leftarrow i$
return $p, ss, pos, dfs, depth$

1. $ss[v] \leftarrow ss[v] + ss[u]$ calculates the subtree size for every node.
2. We iterate top-down using the order in dfs. We initialize a pos array with the subtree sizes ss.
    - Imagine node v owns a contiguous block of indices ending at $pos[v]$.
    - It gives its child u a chunk of this block equal to u's subtree size. u's block now ends where v's block ended.
    - v's remaining block end pointer is shifted back by u's size ($pos[v] - pos[u]$).
    - As this repeats for all children of v, the children partition the space reserved for v's descendants, and v's pointer ends up pointing to the beginning of its block.
3. $pos[i]$ points to the 1-based index where node i should appear. Subtracting 1 makes it a 0-indexed DFS array, and placing i at $dfs[pos[i]]$ computes dfs.

9.6 Diameter

With the bottom-up sequence from XOR-Linked traversal, we compute the diameter of the tree using DP.

Algorithm 50 Tree Diameter

Input: Number of nodes N, Parent array p, Degree array d
Output: Integer representing the tree diameter

Initialize $max_depth \leftarrow [0, \ldots, 0]$ of size N
Initialize $diameter \leftarrow [0, \ldots, 0]$ of size N

for all edge (u, v) processed via XOR-Linked traversal **do**
    ▷ *Path through v connecting u's deepest leaf and previously processed children's deepest leaf*
    $path_through_v \leftarrow max_depth[v] + max_depth[u] + 1$
    $diameter[v] \leftarrow \max(\{diameter[v], diameter[u], path_through_v\})$
    $max_depth[v] \leftarrow \max(max_depth[v], max_depth[u] + 1)$
return $\max_{i \in [0, N-1]} diameter[i]$

The algorithm maintains two variables for each vertex v:

- $max_depth[v]$: The length of the longest path from v down to a leaf in its subtree already processed.
- $diameter[v]$: The maximum diameter completely contained within the subtree rooted at v already processed.

Because the traversal processes children before their parents, when we evaluate the edge between child u and parent v, $max_depth[u]$ and $diameter[u]$ are already computed. We update v's state by checking if joining u's deepest path to v's currently known deepest path creates a new maximum diameter for v's subtree. Finally, we update v's maximum depth to include u.

9.7 Shallowest Decomposition Tree

The Shallowest Decomposition Tree possesses the same properties as a Centroid Decomposition but can be constructed in $O(n)$ time with optimal depth.

Definition 300 (Valid Tree Labeling)**.** A valid tree labeling is an assignment of integer labels $\ell : V \to \mathbb{N}$ such that for any two distinct vertices u, v sharing the same label ($\ell(u) = \ell(v)$), the simple path between them contains at least one vertex w with a strictly greater label ($\ell(w) > \ell(u)$).

The depth of the resulting decomposition tree is exactly the maximum label assigned.

1. The DP We compute the optimal labeling in $O(n)$ time using a bottom-up DP. For each node u, we maintain a bitmask $\mathcal{F}(u)$ representing the set of forbidden labels. The k-th bit of $\mathcal{F}(u)$ is 1 if assigning label k to u would violate the labeling condition with its descendants.

The optimal label $\ell(u)$ is the smallest non-forbidden integer, corresponding to the index of the lowest unset bit in $\mathcal{F}(u)$.

When a child v passes its constraints up to its parent u, the newly assigned label $\ell(v)$ acts as a "shield." Any forbidden labels strictly smaller than $\ell(v)$ are shielded and are no longer forbidden for u. However, $\ell(v)$ itself becomes forbidden for u, and any constraints strictly greater than $\ell(v)$ pass through unaffected.

It is easy to see that the propagation is exactly:

$$\mathcal{P}(v) = \mathcal{F}(v) + 1$$

When u aggregates the propagated masks $\mathcal{P}(v)$ from all its children, it must resolve collisions:

1. **Union:** If a label is forbidden by any child, it is forbidden for u. Let $\mathcal{O}_{\text{once}} = \bigvee \mathcal{P}(v)$.
2. **Collision:** If a label k is forbidden by two or more distinct children, u must act as a shield for them by taking a label strictly greater than k. Let $\mathcal{O}_{\text{twice}}$ be the mask of bits set by at least two children.

If $\mathcal{O}_{\text{twice}} > 0$, let $k = \lfloor \log_2(\mathcal{O}_{\text{twice}}) \rfloor$ be its highest set bit. To force $\ell(u) > k$, we saturate all bits up to k by applying a mask $M_k = (1 \ll (k+1)) - 1$. The final forbidden state for u is exactly:

$$\mathcal{F}(u) = \mathcal{O}_{\text{once}} \vee M_k$$

From this, we evaluate $\ell(u) = \text{trailing_zeros}(\mathcal{F}(u) + 1)$.

2. Decomposition Chains Let $\text{anc}(x)$ denote the lowest ancestor of x in the original rooted tree such that $\ell(\text{anc}(x)) > \ell(x)$. In the decomposition tree, u becomes the direct parent of all vertices x where $\text{anc}(x) = u$.

Two vertices x and y sharing the same $\text{anc}(x) = u$ must reside in the subtree of the same child v of u. Furthermore, they must have distinct labels (if they shared a label, a higher label would exist between them, violating the premise that u is their lowest higher-labeled ancestor).

Thus, the set of vertices in v's subtree that attach to u form a strict total order: $x_1 > x_2 > \cdots > x_k$. When u is removed during decomposition, x_1 becomes the highest-labeled root of the newly isolated component, x_2 becomes its child, and so on. This vertical parent-child line in the decomposition tree is called a Decomposition Chain.

The labels present in the chain from child v to parent u are exactly the labels exposed in v's subtree that are strictly less than $\ell(u)$. Because $\mathcal{P}(v)$ encodes all exposed labels, we can extract this chain mask:

$$\text{ChainMask}(v \to u) = \mathcal{P}(v) \wedge ((1 \ll \ell(u)) - 1)$$

2. Global Chain Consider the set of vertices $S \subseteq V$ that have no higher-labeled ancestor in the original rooted tree. These vertices represent the sequence of maximum labels in the decomposition process.

By the same proof as before, the labels in S are distinct, let them be $x_1 > x_2 > \cdots > x_k$.

In the decomposition tree, the vertex x_1 (which possesses the maximum label in the entire tree) naturally becomes the root, r_{decomp}. When x_1 is removed during decomposition, x_2 becomes the highest-labeled root of its newly isolated component, thus a child of x_1. Thus the vertices in S form a chain.

3. Vertex Retrieval Knowing the labels in a chain is insufficient; we must retrieve the vertices in $O(1)$ time.

We maintain a global array of stacks, allocating one stack per possible label. During the DFS, immediately after computing $\ell(u)$, we push u onto stacks$[\ell(u)]$.

Because multiple subtrees share this global stack, we must manage the traversal order.

- **Forward:** In the DFS, we iterate through the children of u in forward order $(v_1, v_2, \ldots, v_m)$. Their vertices are recursively evaluated and pushed onto the stacks in that order. The vertices belonging to the final child v_m reside at the top of the stacks.
- **Reversed:** To extract the chains to form edges, we iterate through the children of u in reverse $(v_m, \ldots, v_1)$. Because we process the last child v_m first, the vertices defined by its ChainMask are guaranteed to be exactly at the top of their respective stacks.

For a given ChainMask, we iterate through its set bits in decreasing order (extracting the highest bit $L = \lfloor \log_2(\text{mask}) \rfloor$ first). This ensures that higher-labeled vertices become ancestors of lower-labeled vertices in the decomposition tree. We pop the vertex w from stacks$[L]$, add the directed edge $u \to w$, and update $u \leftarrow w$ to continue forming the chain downwards.

Algorithm 51 Shallowest Decomposition Tree

```
function ExtractChain(mask, u)
    while mask ≠ 0 do
        L ← ⌊log2(mask)⌋
        mask ← mask ⊕ (1 ≪ L)
        if stacks[L] ≠ ∅ then
            v ← stacks[L].pop()
            Add edge u → v to decomposition tree
            u ← v
function DFS(u, p)
    once ← 0; twice ← 0
    for all v ∈ adj[u] do
        if v ≠ p then
            DFS(v, u)
            prop ← forbidden[v] + 1
            twice ← twice ∨ (once ∧ prop)
            once ← once ∨ prop
    if twice = 0 then
        M ← 0
    else
        k ← ⌊log2(twice)⌋
        M ← (1 ≪ (k + 1)) − 1
    forbidden[u] ← once ∨ M
    label_u ← trailing_zeros(forbidden[u] + 1)
    stacks[label_u].push(u)
    for all v ∈ reversed(adj[u]) do
        if v ≠ p then
            extract_mask ← (forbidden[v] + 1) ∧ ((1 ≪ label_u) − 1)
            ExtractChain(extract_mask, u)
function ShallowestDecomp(adj, root)
    Initialize forbidden array to 0 for all nodes
    Initialize stacks as an array of empty stacks
    DFS(root, null)
    max_label ← ⌊log2(forbidden[root] + 1)⌋
    decomp_root ← stacks[max_label].pop()
    final_mask ← (forbidden[root] + 1) ∧ ((1 ≪ max_label) − 1)
    ExtractChain(final_mask, decomp_root)
    return decomp_root
```

9.8 Static Top Tree

A static top tree is best viewed as a tree expressing a rooted tree DP as a balanced expression.

The input tree itself never changes. We choose heavy children once, break the tree into heavy paths, and build a binary expression whose leaves are the original vertices. A point update changes one leaf and therefore requires recomputing only the $O(\log n)$ expression nodes above it.

The useful feature is that the expression distinguishes two kinds of partial trees:

Path	and	Point.

A Path object represents a connected piece whose distinguished direction follows one heavy path.

A Point object represents one or more completed child pieces hanging from the same vertex.

Only four operations are needed:

$$\begin{aligned} \mathsf{Vertex}_u &: \mathsf{Point} \longrightarrow \mathsf{Path}, \\ \mathsf{AddEdge} &: \mathsf{Path} \longrightarrow \mathsf{Point}, \\ \mathsf{Rake} &: \mathsf{Point} \times \mathsf{Point} \longrightarrow \mathsf{Point}, \\ \mathsf{Compress} &: \mathsf{Path} \times \mathsf{Path} \longrightarrow \mathsf{Path}. \end{aligned}$$

We also require an identity point

$$\mathbf{0}_{\mathsf{P}}$$

for Rake.

9.8.1 The Four Operations

Suppose the original tree is rooted.

For a vertex u, all light-child subtrees are first converted to Point objects and combined. Inserting u turns the resulting point object into a path object:

$$\mathsf{Vertex}_u : \mathsf{Point} \to \mathsf{Path}.$$

A completed child path becomes something hanging from its parent after we cross the parent–child edge:

$$\mathsf{AddEdge} : \mathsf{Path} \to \mathsf{Point}.$$

Several light-child contributions hanging from the same parent are combined by

$$\mathsf{Rake}.$$

Finally, consecutive pieces of one heavy path are concatenated by

$$\mathsf{Compress}.$$

Thus the recursion around a vertex u has the form

$$\begin{array}{c} \text{light child paths} \\ \downarrow\ \mathsf{AddEdge} \\ \text{point contributions} \\ \downarrow\ \mathsf{Rake} \\ \text{all light children of } u \\ \downarrow\ \mathsf{Vertex}_u \\ \text{one heavy-path piece.} \end{array}$$

Consecutive heavy-path pieces are then joined with Compress.

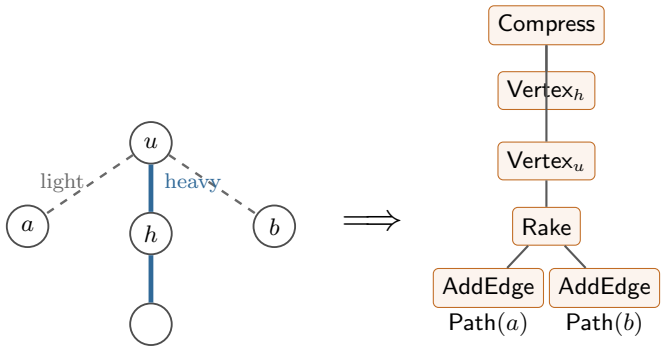


Figure 1: A vertex's light-child subtrees are converted to points and raked together; the vertex turns them back into a path piece. Heavy-path pieces are then compressed.

9.8.2 Heavy Paths

Let

$$\mathrm{sz}(u)$$

be the number of vertices in the rooted subtree of u.

For every nonleaf u, choose as its heavy child

$$h(u) \in \arg \max_{v \text{ child of } u} \mathrm{sz}(v).$$

All other child edges are light.

We use the standard fact:

Lemma 301 (Light-Edge Halving)**.** *If v is a light child of u, then*

$$\boxed{\mathrm{sz}(v) \le \frac{\mathrm{sz}(u) - 1}{2}.}$$

Consequently every root–leaf path contains at most

$$\lfloor \log_2 n \rfloor$$

light edges.

Proof. Since v is not the heavy child, there is another child $h(u)$ satisfying

$$\mathrm{sz}(h(u)) \ge \mathrm{sz}(v).$$

Hence

$$\mathrm{sz}(u) \ge 1 + \mathrm{sz}(v) + \mathrm{sz}(h(u)) \ge 1 + 2\,\mathrm{sz}(v).$$

Whenever a root–leaf path crosses a light edge, the subtree size therefore drops by at least a factor 2. Starting from at most n, this can happen at most $\lfloor \log_2 n \rfloor$ times. □

9.8.3 The Expression Tree

For every original vertex u, define

$$\lambda(u) = 1 + \sum_{\substack{v \text{ child of } u \\ v \neq h(u)}} \mathrm{sz}(v).$$

This is the number of original vertices represented by the local Vertex_u piece: u itself together with all of its light-child subtrees.

We construct the expression recursively.

For every light child v of u, recursively construct the heavy-path expression beginning at v, then wrap it in an $\mathsf{AddEdge}$ node.

Merge all such point objects with Rake, or use $\mathbf{0}_{\mathsf{P}}$ if there are none. Applying Vertex_u gives one path object of weight $\lambda(u)$.

Now follow the heavy chain

$$u,\ h(u),\ h^2(u), \ldots$$

and construct one such path object for every vertex on it. Merge them, in top-to-bottom order, with $\mathsf{Compress}$.

Thus if

$$u = u_0, u_1, \ldots, u_k$$

is a maximal heavy path,

$$\mathrm{BuildPath}(u) = \mathsf{Compress}\big(\mathsf{Vertex}_{u_0}(\cdots), \ldots, \mathsf{Vertex}_{u_k}(\cdots)\big).$$

The root expression is

$$\mathrm{BuildPath}(r).$$

Lemma 302 (Correctness of the Construction)**.** *For every heavy-path head u,*

$$\text{BuildPath}(u)$$

represents exactly the rooted subtree T_u.

In particular, the root of the expression tree represents the entire input tree.

Proof. We induct on $\text{sz}(u)$.

Let

$$u = u_0, u_1, \ldots, u_k$$

be the maximal heavy path beginning at u.

For every u_i, each light child v has strictly smaller subtree. By induction,

$$\text{BuildPath}(v)$$

represents exactly T_v. After AddEdge, it is a point contribution attached to u_i. Raking these contributions therefore represents exactly all light-child subtrees of u_i.

Applying Vertex_{u_i} adds u_i, so the resulting path object contains

$$u_i \quad + \quad \text{all light-child subtrees of } u_i.$$

The only missing descendants of u_i are those continuing through its heavy child u_{i+1}. Compressing the path pieces in order therefore adds precisely these heavy continuations.

Every vertex of T_u lies either on the heavy path or in the subtree of a unique light child of a heavy-path vertex. Hence the resulting object contains exactly T_u. □

9.8.4 Weight-Balanced Merging

A naive left-associated merge of a heavy path could have linear depth. We therefore balance every Compress and Rake expression according to the number of original vertices represented by its children.

Suppose we must merge ordered objects

$$C_1, \ldots, C_q$$

of positive weights

$$s_1, \ldots, s_q.$$

Let

$$S = \sum_i s_i.$$

Scan from left to right, maintaining

$$z = S - 2\sum_{j<i} s_j.$$

Put C_i on the left while

$$z > s_i;$$

once this fails, put C_i and all later objects on the right. Recursively merge the two sides.

Equivalently, C_i goes left precisely when the midpoint of its weighted interval lies to the left of $S/2$.

Lemma 303 (Weighted-Merge Depth)**.** *Consider an item of weight s inside a weighted merge of total weight S.*

Its depth in the resulting binary merge tree is

$$\boxed{O\left(1 + \log \frac{S}{s}\right).}$$

Proof. At a recursive call of total weight W, let C be the weight of the child containing our item.

If

$$C \le \frac{2W}{3},$$

then after one level the containing weight has shrunk by a factor at least $3/2$.

Suppose instead that

$$C > \frac{2W}{3}.$$

Assume first that the heavy child is the right side. Let $P = W - C$ be the weight of the left side, and let x be the first item placed on the right, with weight s_x.

The condition for x to be placed on the right is

$$W - 2P \le s_x.$$

Since $P = W - C$,

$$s_x \ge 2C - W.$$

Because $C > 2W/3$,

$$s_x > \frac{C}{2}.$$

When the right side is recursively split, x is its first item and has more than half its total weight. Hence x becomes a singleton child, and all remaining items lie in a child of weight

$$C - s_x \le C - (2C - W) = W - C < \frac{W}{3}.$$

Thus after at most one additional level, our item is either isolated or lies in a subproblem of weight less than $W/3$.

The case in which the left side has weight $> 2W/3$ is symmetric: the last item on the left has weight $> C/2$, becomes isolated at the next split, and all other items lie in a subproblem of weight $< W/3$.

Therefore, unless the item has already become a leaf, every two levels reduce the containing total weight by at least a constant factor.

Starting from S, this can happen only

$$O\left(\log \frac{S}{s}\right)$$

times before the containing weight is $O(s)$, at which point the item is isolated after $O(1)$ further levels. □

Theorem 304 (Static Top-Tree Height)**.** *The expression tree constructed above has*

$$\boxed{O(\log n)}$$

height and $O(n)$ nodes.

Proof. First consider the node count.

There is exactly one Vertex node for each original vertex. Every light tree edge contributes one AddEdge node.

A binary merge of q items contributes $q - 1$ internal nodes. Summed over all rake lists, this is $O(n)$, because every raked item corresponds to a distinct light child.

Likewise, summed over all heavy paths, the number of Compress nodes is

$$n - \#\{\text{heavy paths}\} = O(n).$$

Hence the expression contains $O(n)$ nodes.

Now fix an original vertex x, and follow the expression-parent chain from its Vertex node to the root.

Every time this chain passes from a child heavy path into its parent's local piece, it crosses a light represented-tree edge. By Lemma 301, this happens only $O(\log n)$ times.

Between two such light transitions, the chain passes through one weighted rake tree and one weighted compress tree. By Lemma 303, the number of merge nodes contributed is bounded by a logarithm of the ratio between the enclosing represented size and the size of the child piece.

These size ratios telescope as we move upward through nested represented pieces. Their total logarithm is at most

$$\log \frac{n}{1} = O(\log n).$$

The nonmerge constructors contribute only $O(1)$ nodes per light transition. Thus the whole expression-parent chain has length $O(\log n)$. □

Theorem 305 (Construction Time)**.** *The recursive construction above takes*

$$\boxed{O(n \log n)}$$

time and $O(n)$ space.

Once the expression structure has been built, evaluating all DP values bottom-up takes $O(n)$ time.

Proof. Computing subtree sizes and heavy children takes $O(n)$ time.

The expression itself has $O(n)$ nodes. The only superlinear work comes from the simple weighted-merge routine, which scans the current list at every recursive level.

By Lemma 303, every item occurs in $O(\log n)$ such scans. Thus all merge construction costs sum to $O(n \log n)$.

A bottom-up evaluation performs $O(1)$ work at each expression node, hence takes $O(n)$. □

9.8.5 The DP Interface

After combining the two vertex cases, a static-top-tree DP needs

$$\boxed{\begin{aligned} \mathsf{vertex} &: (\mathsf{Point}, u) \mapsto \mathsf{Path}, \\ \mathsf{compress} &: (\mathsf{Path}, \mathsf{Path}) \mapsto \mathsf{Path}, \\ \mathsf{rake} &: (\mathsf{Point}, \mathsf{Point}) \mapsto \mathsf{Point}, \\ \mathsf{addEdge} &: \mathsf{Path} \mapsto \mathsf{Point}, \end{aligned}}$$

together with $\mathbf{0}_{\mathsf{P}}$.

At a vertex node u, if there is no rake child we evaluate

$$\mathsf{vertex}(\mathbf{0}_{\mathsf{P}}, u).$$

Otherwise we evaluate

$$\mathsf{vertex}(\mathsf{point}(\mathrm{left}(u)), u).$$

9.8.6 Point Updates

Store the value of every expression node.

If the value of an original vertex u changes, start at its Vertex node and recompute every expression ancestor.

Theorem 306 (Static Top-Tree Update)**.** *A vertex update takes*

$$\boxed{O(\log n)}$$

time.

Proof. Only ancestors of the corresponding expression leaf can depend on the updated value.

By Theorem 304, there are $O(\log n)$ such ancestors, and every constructor is recomputed in $O(1)$ time. □

9.8.7 Rooted-Subtree Queries

The representation also gives subtree queries.

The Vertex node for u already contains

$$u + \text{every light-child subtree of } u.$$

The only descendants missing from T_u are therefore the vertices below u on its heavy path, together with their light subtrees.

Those are exactly the later pieces of the same Compress expression.

Let

$$x = u, \qquad R = \mathsf{path}[u].$$

While the expression parent $p(x)$ is a Compress node:

$$\begin{cases} R \leftarrow \mathsf{compress}\big(R, \mathsf{path}[r(p(x))]\big), & x = l(p(x)), \\ R \text{ unchanged}, & x = r(p(x)). \end{cases}$$

Then set

$$x \leftarrow p(x)$$

and continue.

Stop at the first non-Compress parent.

Theorem 307 (Static Rooted-Subtree Query)**.** *The preceding procedure returns the* Path *value of exactly the rooted subtree T_u, in*

$$\boxed{O(\log n)}$$

time.

Proof. The compress expression for a heavy path preserves the top-to-bottom order.

At any compress node, the left child represents an earlier, higher segment of the heavy path, and the right child represents a later, lower segment.

While climbing from the leaf corresponding to u:

If the current object is the left child, then the right sibling lies strictly below it on the same heavy path and therefore belongs to T_u. It must be appended.

If the current object is the right child, then the left sibling lies strictly above u and must be excluded.

Consequently the procedure accumulates exactly the suffix of the heavy path beginning at u. Each vertex piece on that suffix already contains all of its light-child subtrees.

Thus the accumulated represented vertices are exactly T_u.

When we reach a noncompress parent, we have reached the head of this heavy path. Crossing the next AddEdge would move to its represented parent, which lies outside T_u, so the query must stop there.

The number of expression ancestors visited is $O(\log n)$. □

9.8.8 Path Queries

The weight-balanced compress and rake trees also give

$$\boxed{O(\log n)}$$

path queries, rather than $O(\log^2 n)$.

Recall that for a vertex u,

$$\lambda(u) = 1 + \sum_{\substack{v \text{ child of } u \\ v \neq h(u)}} \mathrm{sz}(v).$$

Equivalently, if $h(u)$ exists,

$$\lambda(u) = \mathrm{sz}(u) - \mathrm{sz}(h(u)).$$

These $\lambda(u)$'s are exactly the weights used to balance the Compress tree of each heavy path.

We first record how to query a prefix of one heavy path.

Lemma 308 (Heavy-Path Prefix)**.** *Let H be a maximal heavy path with head h, and let $x \in H$. The aggregate on the path segment*

$$P(h, x)$$

can be obtained in

$$\boxed{O\left(1 + \log \frac{\mathrm{sz}(h)}{\lambda(x)}\right)}$$

time.

Proof. The leaves of the compress expression for H occur in top-to-bottom order along the heavy path.

Start at the leaf corresponding to x, with the aggregate of x itself, and climb toward the root of this compress tree.

Whenever the current node is a right child, its left sibling lies entirely before it on the heavy path, so prepend the aggregate of that left sibling. Whenever the current node is a left child, the right sibling lies below x and is ignored.

When the root is reached, the accumulated value is exactly the prefix

$$P(h, x).$$

The work is proportional to the depth of the leaf x. The total weight of the compress tree is

$$\sum_{u \in H} \lambda(u) = \mathrm{sz}(h),$$

because the differences telescope along the heavy path.

By Lemma 303, the leaf of weight $\lambda(x)$ has depth

$$O\left(1 + \log \frac{\mathrm{sz}(h)}{\lambda(x)}\right).$$

□

For noncommutative aggregates, store both orientations of every Path value. Then the same procedure returns the segment in either direction.

Why the costs telescope. Consider walking upward from a vertex x_0 toward the root by the usual heavy-light decomposition.

Let

$$h_i$$

be the head of the heavy path containing x_i, and after processing the segment

$$P(h_i, x_i)$$

set

$$x_{i+1} = \mathrm{parent}(h_i).$$

Thus the edge

$$x_{i+1} h_i$$

is light.

The cost of the i-th heavy-path segment is

$$O\left(1+\log\frac{\mathrm{sz}(h_i)}{\lambda(x_i)}\right).$$

Now observe the reason for having balanced the compress tree using the weights λ. For $i \geq 1$, the previous heavy-path head h_{i-1} is a light child of x_i. Therefore its entire subtree is included in the local weight of x_i:

$$\boxed{\lambda(x_i) \geq \mathrm{sz}(h_{i-1}).}$$

Consequently,

$$\begin{aligned}\sum_{i=0}^{k}\log\frac{\mathrm{sz}(h_i)}{\lambda(x_i)} &\leq \log\frac{\mathrm{sz}(h_0)}{\lambda(x_0)}+\sum_{i=1}^{k}\log\frac{\mathrm{sz}(h_i)}{\mathrm{sz}(h_{i-1})}\\ &= \log\frac{\mathrm{sz}(h_k)}{\lambda(x_0)}\\ &\leq \log n.\end{aligned}$$

There are also only $O(\log n)$ segments, because moving from h_i to x_{i+1} crosses a light edge, and every root–leaf path has only $O(\log n)$ light edges.

Hence all heavy-path prefixes on one upward walk cost

$$\boxed{O(\log n)}$$

in total.

Theorem 309 (Static Path Query)**.** *An associative aggregate on an arbitrary represented-tree path*

$$P(u,v)$$

can be queried in

$$\boxed{O(\log n)}$$

time.

Proof. Use the ordinary heavy-light path decomposition.

While u and v lie on different heavy paths, process the endpoint whose heavy-path head is deeper. If h is the head of the heavy path containing that endpoint x, query

$$P(h,x)$$

using Lemma 308, append or prepend it with the appropriate orientation, and replace

$$x \leftarrow \mathrm{parent}(h).$$

The segments processed from the u-side form one upward heavy-light walk, and those processed from the v-side form another. By the telescoping calculation above, each side costs

$$O(\log n)$$

in total.

Eventually u and v lie on the same heavy path. The remaining segment between them is one interval in a single weight-balanced compress tree.

An interval in a binary expression tree can be obtained by climbing from its two endpoint leaves toward their least common ancestor, accumulating the siblings lying inside the interval. Its cost is at most the sum of the two endpoint depths, hence

$$O(\log n)$$

by Lemma 303.

Combining the two upward portions and the final heavy-path interval gives

$$O(\log n)$$

total time. □

Theorem 310 (Static Top-Tree Queries)**.** *After preprocessing, and after any point updates, the static top tree supports*

$$\boxed{\begin{array}{l} \textit{rooted-subtree query} : O(\log n), \\ \textit{arbitrary path query} : O(\log n). \end{array}}$$

Path aggregates may be noncommutative by storing both orientations of every Path *value.*

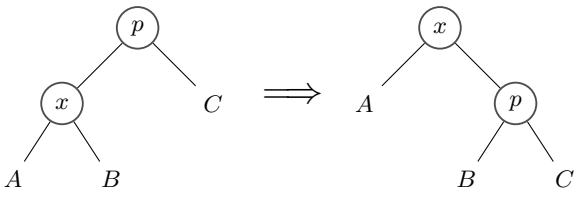
9.9 Splay Tree

A splay tree is a binary tree with no explicit balancing information. Whenever a node is accessed, rotations move it to the root.

Individual operations may take linear time, but the amortized cost of a splay is logarithmic.

9.9.1 Rotation

Suppose x is the left child of p.



The inorder order

$$A, x, B, p, C$$

is unchanged.

The other rotation is symmetric.

9.9.2 Splaying

To splay x, repeatedly inspect its parent p and, when it exists, its grandparent g.

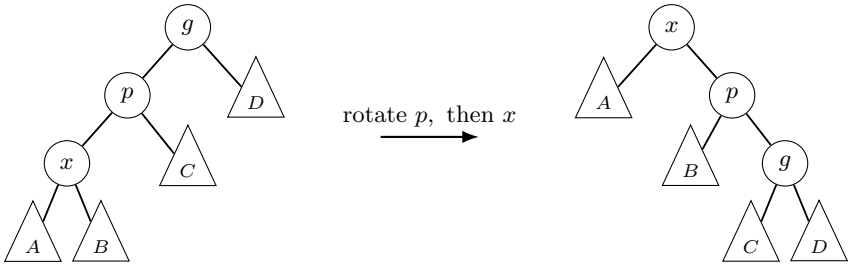
A final *zig* rotates x once.

In a *zig–zig*, x and p have the same orientation; rotate p, then x.

In a *zig–zag*, they have opposite orientations; rotate x twice.

Before rotating, all lazy tags on the auxiliary root–x path must be pushed from top to bottom.

Zig–Zig The only non-obvious part of splaying is the zig–zig step. In the left–left case it performs



The point of zig–zig is that splaying should improve more than just the accessed vertex.

If we blindly rotate x upward twice, x reaches the top, but both vertices above it are pushed down:

$$g, p, x \quad \longrightarrow \quad x, g, p.$$

In particular, p, which was immediately next to the accessed vertex, actually becomes deeper.

Zig–zig instead reverses the whole three-vertex piece:

$$g, p, x \quad \longrightarrow \quad x, p, g.$$

Thus x gains two levels while p stays close to it; only the more distant ancestor g is pushed far downward. Vertices that were close to x remain relatively close to the top as well. This is the improvement that makes the amortized bound possible.

9.9.3 Access Lemma

Give every node v an arbitrary positive analysis weight

$$w(v) > 0.$$

For a splay subtree T_v, define

$$s(v) = \sum_{u \in T_v} w(u), \qquad r(v) = \log_2 s(v),$$

and use the potential

$$\Phi = \sum_{v} r(v).$$

We need one elementary inequality.

Lemma 311. *If $a, b > 0$ and*

$$a + b \le c,$$

then

$$\boxed{\log_2 a + \log_2 b \le 2 \log_2 c - 2.}$$

Proof. By AM–GM,

$$ab \le \left(\frac{a+b}{2} \right)^2 \le \frac{c^2}{4}.$$

Take base-2 logarithms. □

Theorem 312 (Splay Access Lemma)**.** *Suppose x is splayed in a tree of total analysis weight W. Its amortized number of rotations is at most*

$$\boxed{1 + 3 \log_2 \frac{W}{w(x)}.}$$

Proof. We prove that every double rotation has amortized cost at most

$$3(r'(x) - r(x)),$$

while the final zig has amortized cost at most

$$1 + 3(r'(x) - r(x)).$$

For a zig, only x and its parent p change rank, and

$$r'(x) = r(p).$$

Thus

$$\begin{aligned} 1 + \Delta\Phi &= 1 + r'(x) + r'(p) - r(x) - r(p) \\ &= 1 + r'(p) - r(x) \\ &\le 1 + r'(x) - r(x). \end{aligned}$$

Now consider zig–zig:

$$x \subset p \subset g.$$

Again

$$r'(x) = r(g).$$

The old subtree of x and the new subtree of g are disjoint and both lie inside the new subtree of x. Hence

$$s(x) + s'(g) \le s'(x).$$

By Lemma 311,

$$r(x) + r'(g) \le 2r'(x) - 2.$$

Also,

$$r'(p) \le r'(x), \qquad r(p) \ge r(x).$$

Therefore

$$\begin{aligned} 2 + \Delta\Phi &= 2 + r'(p) + r'(g) - r(x) - r(p) \\ &\le 2 + r'(x) + r'(g) - 2r(x) \\ &\le 3(r'(x) - r(x)). \end{aligned}$$

For zig–zag, the new subtrees rooted at p and g are disjoint inside the new subtree rooted at x, so

$$r'(p) + r'(g) \le 2r'(x) - 2.$$

Thus

$$\begin{aligned} 2 + \Delta\Phi &= 2 + r'(p) + r'(g) - r(x) - r(p) \\ &\le 2r'(x) - r(x) - r(p) \\ &\le 2(r'(x) - r(x)) \\ &\le 3(r'(x) - r(x)). \end{aligned}$$

Summing over the whole splay telescopes:

$$\sum (r'(x) - r(x)) = r_{\text{final}}(x) - r_{\text{initial}}(x).$$

Finally,

$$r_{\text{final}}(x) \le \log_2 W, \qquad r_{\text{initial}}(x) \ge \log_2 w(x).$$

□

With unit weights,

$$W = n,$$

so a splay costs

$$O(\log n)$$

amortized.

9.9.4 Augmentation

Suppose values lie in a monoid

$$(\mathcal{M}, \circ, e).$$

Store

$$\mathrm{agg}(v) = \mathrm{agg}(L_v) \circ \mathrm{val}(v) \circ \mathrm{agg}(R_v).$$

A rotation changes only constantly many child pointers, so the aggregates are restored in $O(1)$ time.

For sequence reversal, maintain a lazy reversal bit. If the monoid is noncommutative, store both

$$\mathrm{fwd}(v) \qquad \text{and} \qquad \mathrm{rev}(v).$$

Reversal swaps the children and these two aggregates.

9.10 Link-Cut Tree

A link-cut tree is a data structure for maintaining a forest while supporting operations on arbitrary tree paths in amortized

$$\boxed{O(\log n)}$$

time.

The main difficulty is that a tree path can be long. Therefore, we decompose it into paths, store each path in a splay tree, and change this path decomposition whenever a new path is accessed.

It is useful to distinguish three objects:

1. the *represented tree*, which is the actual tree we care about;
2. the *auxiliary trees*, which are splay trees representing disjoint paths of the represented tree;
3. the *tree of auxiliary trees*, obtained by contracting each such path to one object and keeping the represented edges between different paths.

9.10.1 Preferred Paths

At every vertex, at most one edge to a child is designated *preferred*. These choices are not determined by subtree size and are not permanent; they change during accesses.

Since every vertex has at most one preferred child, the preferred edges form a collection of vertex-disjoint downward paths.

Each such preferred path is stored in one *auxiliary splay tree*.

All remaining represented-tree edges are called *virtual*. They join one preferred path to another. Thus if we contract every preferred path, the virtual edges form a tree whose nodes are the auxiliary trees.

For example, consider the following represented tree.

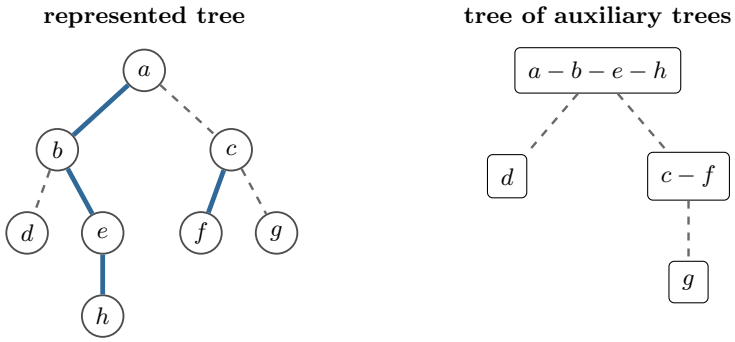


Figure 2: On the left, preferred edges partition the represented tree into the paths $a - b - e - h$, $c - f$, d, and g. Each path is stored in one auxiliary splay tree. Contracting those paths gives the tree of auxiliary trees on the right.

This second view is particularly useful for understanding Access: an access starts in one auxiliary tree and repeatedly moves through a virtual edge to the auxiliary tree above it.

9.10.2 How a Preferred Path Is Stored

Suppose a preferred path is

$$v_1, v_2, \ldots, v_k$$

from top to bottom in the represented tree.

Its auxiliary tree is a splay tree whose *inorder order* is

$$v_1, v_2, \ldots, v_k.$$

The shape of the splay tree has nothing to do with the shape of the represented tree.

For example, the represented path

$$a - b - e - h$$

could currently be stored as follows.

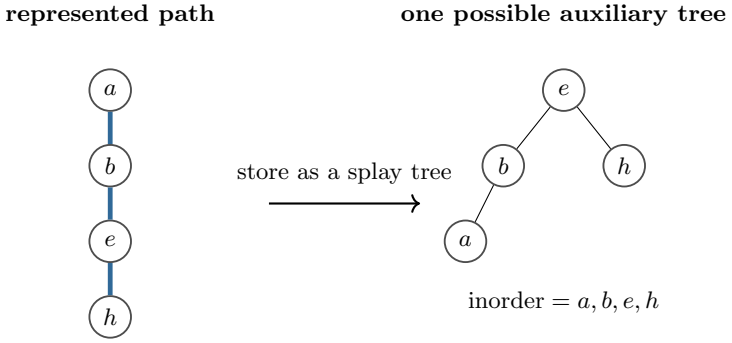


Figure 3: An auxiliary tree does not imitate the represented-tree parent pointers. Its only structural requirement is that inorder order agrees with top-to-bottom order along the preferred path.

Splaying may completely rearrange an auxiliary tree, but it preserves inorder order, so it continues to represent the same tree path.

It also means that ordinary binary-tree augmentation works immediately. If every splay node stores an aggregate of its subtree, then the root of an auxiliary tree stores the aggregate of the entire preferred path.

9.10.3 Connecting the Auxiliary Trees

The auxiliary trees must still remember how the preferred paths are joined inside the represented tree.

Suppose a preferred path begins at x, and the represented parent of x is p. Then the edge

$$px$$

is virtual and connects this auxiliary tree to the auxiliary tree containing p.

This connection is called the *path-parent* relation.

Conceptually, we could store two kinds of pointers:

- ordinary splay-tree parent pointers inside an auxiliary tree;
- a path-parent pointer from the whole auxiliary tree to the auxiliary tree above it.

A link-cut tree can encode both with a single field

$$\text{parent}(x).$$

If x is not the root of its auxiliary tree, this is its ordinary splay parent.

If x *is* the root of its auxiliary tree, then its parent pointer instead stores the path-parent connection of the whole auxiliary tree.

Thus x is an auxiliary root precisely when

$$\text{parent}(x) = \bot$$

or

$$x \notin \{\text{left}(\text{parent}(x)), \text{right}(\text{parent}(x))\}.$$

Notice that if x is an auxiliary root, $\text{parent}(x)$ need not be the represented parent of x itself. The pointer represents the virtual edge immediately above the top of the path stored by the entire auxiliary tree.

This is what allows us to climb from one auxiliary tree to the next.

9.10.4 Access

The fundamental operation is

$$\mathsf{Access}(v).$$

Its purpose is simple:

make the entire represented root–v path one preferred path.

Equivalently, after an access, the root–v path is stored in one auxiliary splay tree.

Before the operation, that path may pass through many auxiliary trees. In the tree-of-auxiliary-trees picture, $\mathsf{Access}(v)$ walks upward through these auxiliary trees and splices them together one at a time.

The key local operation is easiest to understand after splaying a vertex x.

Since inorder order is top-to-bottom represented-tree order,

- the left subtree of x consists of the part of its current preferred path *above* x;
- the right subtree of x consists of the part *below* x.

Suppose we have already exposed the portion from x downward toward v, stored in an auxiliary tree called *last*.

We then replace

$$\text{right}(x)$$

by *last*.

This does exactly what we want:

- the old preferred continuation below x stops being preferred and becomes a separate auxiliary tree;
- the path from x toward v becomes preferred;
- hence the exposed suffix toward v is merged with the preferred path containing x.

We then follow the path-parent pointer to the auxiliary tree immediately above and repeat.

Algorithm 52 Link-Cut Tree Access

```
1: function Access(v)
2:     last ← ⊥
3:     x ← v
4:     while x ≠ ⊥ do
5:         Splay(x)
6:         right(x) ← last
7:         Pull(x)
8:         last ← x
9:         x ← parent(x)
10:    Splay(v)
```

Here and below, the assignment

$$\text{right}(x) \leftarrow \mathit{last}$$

includes the corresponding parent-pointer bookkeeping.

Lemma 313 (Correctness of Access)**.** *After* Access(v)*, the represented root–v path is one preferred path, represented by the auxiliary splay tree rooted at v.*

Proof. Consider an iteration after x has been splayed.

The right subtree of x is exactly the portion of the current preferred path strictly below x. Replacing it by *last* therefore discards the old preferred continuation and makes the already exposed route from x toward v preferred.

Consequently, after the splice, the auxiliary tree rooted at x represents a preferred path ending at v.

Its path-parent pointer leads to the represented edge immediately above the top of this preferred path. The next loop iteration enters the auxiliary tree above this edge and splices that path onto the front.

Inductively, *last* always represents an increasingly long suffix of the represented root–v path. When there is no path-parent left, that suffix begins at the represented root and hence is the entire path.

Finally, splaying v makes v the root of the resulting auxiliary tree. □

9.10.5 Why Access Is Fast

One access can pass through many auxiliary trees, so it can perform many splays. Thus, a single access need not take $O(\log n)$ worst-case time.

There are two pieces to the analysis:

1. if an access encounters k auxiliary trees, all of its splaying costs only

   $$O(k + \log n)$$

   amortized time;

2. over a sequence of accesses, the number of auxiliary-tree boundaries crossed is only $O(\log n)$ amortized per access.

The first part comes from the splay-tree potential. The second is a heavy–light accounting argument.

Importantly, *heavy and light edges are used only for the analysis.* The link-cut tree does not choose its preferred edges according to heavy–light decomposition.

9.10.6 The Analysis Tree

For the splay analysis, expand the tree of auxiliary trees back to the vertex level.

Keep every splay left/right edge. Whenever an auxiliary tree has a path-parent pointer, also keep that connection.

Ignoring which edges are splay edges and which are path-parent edges gives one ordinary rooted tree on the same vertex set. Call this the *analysis tree*.

For a vertex x, let

$$\sigma(x) = \#\{\text{descendants of } x \text{ in the analysis tree}\},$$

including x, and set

$$\rho(x) = \log_2 \sigma(x).$$

Use the potential

$$\boxed{\Phi = \sum_x \rho(x).}$$

The useful feature of this viewpoint is that the splice performed by Access does not change the underlying analysis tree.

When the old right subtree of x stops being a splay child, its root becomes the root of a separate auxiliary tree whose path-parent is x. Thus it was a child of x in the analysis tree before the splice and remains a child afterward.

Likewise, *last* was connected to x through a path-parent relation before the splice and becomes its right splay child afterward.

Thus a splice merely changes how an analysis-tree edge is represented: as a splay edge or as a path-parent edge.

Only splay rotations change the analysis tree and therefore the potential.

Lemma 314 (Total Splay Cost During Access)**.** *Suppose an access performs k loop iterations. The total amortized rotation cost of its splays is*

$$\boxed{O(k + \log n).}$$

Proof. Consider one splay of x.

All auxiliary trees hanging virtually from vertices of the current preferred path may be regarded as fixed positive weights attached to those vertices. Then the weighted size of a splay subtree is exactly the corresponding analysis-tree subtree size σ.

Hence the weighted splay access lemma gives

$$\text{amortizedCost}(\mathsf{Splay}(x)) \le 1 + 3\big(\rho_{\text{after}}(x) - \rho_{\text{before}}(x)\big).$$

Let

$$x_1, x_2, \ldots, x_k$$

be the successive loop vertices, starting near v and moving toward the represented root.

After processing x_i, the exposed structure rooted at x_i lies below the next loop vertex x_{i+1} in the analysis tree. Therefore

$$\sigma_{\text{after}}(x_i) \le \sigma_{\text{before}}(x_{i+1}),$$

so

$$\rho_{\text{after}}(x_i) \le \rho_{\text{before}}(x_{i+1}).$$

The rank increases therefore telescope:

$$\begin{aligned}\sum_{i=1}^{k} \big(\rho_{\text{after}}(x_i) - \rho_{\text{before}}(x_i)\big) &\le \rho_{\text{after}}(x_k) - \rho_{\text{before}}(x_1) \\ &\le \log_2 n.\end{aligned}$$

Thus the total amortized splay cost is

$$O(k + \log n).$$

The final splay of v contributes another $O(\log n)$, which is absorbed into the same bound. □

9.10.7 How Many Auxiliary Trees Does Access Cross?

It remains to account for k, the number of preferred-path pieces encountered.

For this argument only, fix the represented tree and classify a child edge uv as *heavy* when

$$\mathrm{sz}(v) > \frac{1}{2}\,\mathrm{sz}(u),$$

and *light* otherwise.

Each vertex has at most one heavy child, and every represented root–leaf path contains at most

$$\log_2 n$$

light edges.

Now consider what an access does to preferred edges.

Every virtual edge on the newly exposed root–v path becomes preferred. Among these, only

$$O(\log n)$$

can be light.

What about heavy edges?

A heavy preferred edge ux can become nonpreferred only if some other child y of u becomes preferred. Since u has at most one heavy child, the new preferred edge uy must then be light.

Thus every event that destroys a heavy preferred edge can be charged to the creation of a light preferred edge.

Furthermore, after a heavy edge has become preferred once, making it preferred again requires it to have become nonpreferred in between.

Hence, over q accesses,

$$\#\{\text{heavy preferred creations}\} \le n + \#\{\text{light preferred creations}\}.$$

Since each access creates only $O(\log n)$ light preferred edges,

$$\#\{\text{light preferred creations}\} = O(q \log n).$$

Therefore the total number of preferred-path boundaries crossed or changed is

$$O(n + q \log n).$$

Combining this with Lemma 314, the total time for q accesses is

$$O(n + q \log n).$$

Thus access takes

$$\boxed{O(\log n)}$$

amortized time.

Theorem 315 (Link-Cut Access)**.**

$$\boxed{\mathsf{Access}(v) \text{ takes } O(\log n) \text{ amortized time.}}$$

9.10.8 Rerooting

Once we can expose a root–vertex path, rerooting is immediate.

After

$$\mathsf{Access}(v),$$

the represented root–v path is exactly one auxiliary tree, in top-to-bottom inorder order.

To make v the represented root, simply reverse this path.

We therefore define

$$\boxed{\mathsf{MakeRoot}(v): \quad \mathsf{Access}(v); \quad \mathsf{Reverse}(v).}$$

The reversal is stored lazily in the splay tree.

Reversing the order of the exposed path reverses every represented parent relation along it, so v becomes the represented root.

9.10.9 Dynamic-Forest Operations

Once Access and MakeRoot are available, the usual dynamic-tree operations become short.

Finding the root. Perform

$$\mathsf{Access}(v).$$

The entire represented root–v path is now one auxiliary tree. Its leftmost vertex is therefore the represented root.

Walk left while pushing lazy reversals, and splay the resulting vertex.

Link. Suppose u and v lie in different represented trees.

First perform

$$\mathsf{MakeRoot}(u).$$

Now u has no represented parent. Make its auxiliary tree a virtual child of v, thereby adding the represented edge

$$uv.$$

Cut. Suppose uv is a represented edge.

Perform

$$\mathsf{MakeRoot}(u), \qquad \mathsf{Access}(v).$$

The auxiliary tree rooted at v now represents exactly

$$P(u, v).$$

Since u and v are adjacent in the represented tree, this path contains exactly their one edge. Detaching that exposed connection removes uv from the represented forest.

9.10.10 Path Queries and Updates

The most useful consequence of access is that an arbitrary represented path can be turned into one auxiliary tree.

To expose

$$P(u, v),$$

perform

$$\boxed{\mathsf{MakeRoot}(u), \qquad \mathsf{Access}(v).}$$

After MakeRoot(u), the path $P(u, v)$ is a root–v path. The access therefore makes exactly this path one preferred path.

Hence the entire auxiliary tree rooted at v consists precisely of $P(u, v)$.

If every splay node stores an aggregate of its subtree, then

$$\boxed{\text{PathQuery}(u, v) = \text{agg}(v).}$$

Similarly, any update that can be represented by a compatible lazy tag can be applied to the root v, thereby updating the entire path at once.

Theorem 316 (Link-Cut Tree)**.** *A link-cut tree supports*

$$\text{makeRoot}, \quad \text{findRoot}, \quad \text{link}, \quad \text{cut},$$

together with path aggregates and compatible lazy path updates, in

$$\boxed{O(\log n)}$$

amortized time per operation and $O(n)$ space.

Proof. Every operation consists of a constant number of calls to Access, together with $O(1)$ additional splays, lazy reversals, and pointer changes.

By Theorem 315, each access costs $O(\log n)$ amortized time.

Splay-tree aggregates and lazy information are maintained by Pull and Push in $O(1)$ time per rotation, so augmenting the auxiliary trees does not change the asymptotic bound. □

9.11 Top Tree

The cleanest way to arrive at a top tree is to start from the one thing a link-cut tree does not maintain well.

A link-cut tree knows everything about the current preferred path.

It knows almost nothing about the collection of virtual children hanging from that path.

Suppose u has virtual child subtrees

$$T_1, T_2, \ldots, T_k.$$

An ordinary LCT may maintain their total aggregate explicitly, but if

$$k = \Theta(n),$$

there is no balanced structure representing those children individually.

This becomes a problem as soon as we want subtree queries, subtree lazy updates, or more complicated DP information.

The top-tree idea is:

keep the LCT preferred-path splay tree, and also put every collection of virtual children in a splay tree.

The preferred-path splay is the *compress tree*.

The virtual-child splay is the *rake tree*.

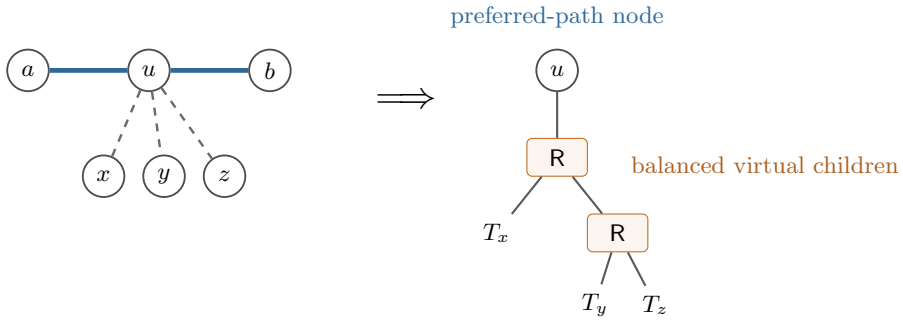


Figure 4: A top tree adds a balanced rake structure for the virtual children that an ordinary link-cut tree leaves implicit.

9.11.1 Real and Fake Nodes

A real represented-tree vertex stores its ordinary two splay children

$$\text{ch}[0], \qquad \text{ch}[1].$$

These are the preferred-path or compress children.

We also give it two rake slots

$$\text{ch}[2], \qquad \text{ch}[3].$$

A represented vertex may have arbitrarily many virtual children, so the rake slots contain a binary tree of *fake nodes*.

Fake nodes contain no represented-tree vertex value. Their sole purpose is to combine two virtual-child aggregates.

Whenever a virtual child is inserted or deleted, ordinary splay rotations within the rake tree keep this collection self-adjusting.

Since every fake node is an internal node of a binary tree whose leaves are represented virtual pieces, the total number of fake nodes is $O(n)$.

9.11.2 The Same Path/Point Interface Reappears

Recall the static-top-tree interface.

A preferred path is a

$$\mathsf{Path}$$

object.

A collection of virtual children attached at one vertex is a

$$\mathsf{Point}$$

object.

The four operations are again

$$\boxed{\begin{aligned} \mathsf{Vertex}_u &: \mathsf{Point} \to \mathsf{Path}, \\ \mathsf{AddEdge} &: \mathsf{Path} \to \mathsf{Point}, \\ \mathsf{Rake} &: \mathsf{Point}^2 \to \mathsf{Point}, \\ \mathsf{Compress} &: \mathsf{Path}^2 \to \mathsf{Path}. \end{aligned}}$$

The difference is only that the binary expression is now maintained dynamically by splay rotations.

9.11.3 Access with Virtual-Child Trees

Consider one iteration of ordinary

$$\mathsf{Access}(v)$$

at a vertex x.

After splaying x, let

$$old = \mathrm{right}(x).$$

This is the old preferred continuation below x.

In an ordinary LCT we simply execute

$$\mathrm{right}(x) \leftarrow last.$$

With virtual-child structures, we must also maintain where those represented subtrees live.

The old preferred continuation becomes virtual, so if $old \neq \bot$, convert it to a point cluster with

$$\mathsf{AddEdge}(old)$$

and insert that point into x's rake tree.

Conversely, *last* was previously a virtual child contribution of x. Remove its point representation from the rake tree and install it as the new right preferred child.

Thus one splice is

$$\boxed{\begin{array}{ll} \text{if } old \neq \bot: & \mathsf{InsertVirtual}(x, old), \\ \text{if } last \neq \bot: & \mathsf{EraseVirtual}(x, last), \\ & \mathrm{right}(x) \leftarrow last. \end{array}}$$

9.11.4 Aggregates

For a real node u, its auxiliary subtree consists of a piece of a preferred path together with arbitrary represented subtrees hanging off that path. Thus its represented material has a natural partition into two parts:

$$\boxed{\text{preferred-path spine} \quad \sqcup \quad \text{material hanging off the spine.}}$$

We maintain one aggregate for each part.

For a real node u, let

$$\boxed{\begin{array}{l} \mathsf{path}(u) = \text{aggregate of the represented vertices on its preferred-path spine,} \\ \mathsf{sub}(u) = \text{aggregate of everything hanging off that spine,} \\ \mathsf{all}(u) = \text{aggregate of the entire represented cluster.} \end{array}}$$

Compress children versus rake children. Recall that a real node u has two different kinds of auxiliary children.

The children

$$\mathrm{ch}_0(u), \mathrm{ch}_1(u)$$

are *compress children*. They continue the same preferred-path spine as u.

The children

$$\mathrm{ch}_2(u), \mathrm{ch}_3(u)$$

are *rake children*. They represent entire clusters attached virtually to u, and hence lie completely off the preferred path containing u.

The path aggregate. Only the compress children lie on the same preferred path as u. Therefore

$$\boxed{\mathsf{path}(u) = \mathsf{path}(\mathrm{ch}_0\, u) \oplus \mathrm{val}(u) \oplus \mathsf{path}(\mathrm{ch}_1\, u).}$$

Inorder order in the compress tree is represented-tree path order.

The rake children do not appear here because none of their represented vertices lies on this preferred path.

The off-path aggregate. Now consider everything in the cluster that is *not* on the spine.

There are two sources.

First, each compress child may itself have represented material hanging from its own part of the spine. We therefore take

$$\mathsf{sub}(\mathrm{ch}_0\, u) \qquad \text{and} \qquad \mathsf{sub}(\mathrm{ch}_1\, u).$$

Second, each rake child lies entirely off the current spine. Consequently we want the *whole* cluster of each rake child, not merely its own off-path part. Hence

$$\mathsf{all}(\mathrm{ch}_2\, u) \qquad \text{and} \qquad \mathsf{all}(\mathrm{ch}_3\, u).$$

Putting these together,

$$\boxed{\begin{aligned}\mathsf{sub}(u) = \mathsf{sub}(\mathrm{ch}_0\, u) &\oplus \mathsf{sub}(\mathrm{ch}_1\, u) \\ &\oplus \mathsf{all}(\mathrm{ch}_2\, u) \oplus \mathsf{all}(\mathrm{ch}_3\, u).\end{aligned}}$$

The whole cluster. Since the path and off-path pieces are disjoint and together form the entire cluster,

$$\boxed{\mathsf{all}(u) = \mathsf{path}(u) \oplus \mathsf{sub}(u).}$$

Missing children contribute the identity element.

Fake rake nodes. A fake rake node does not correspond to a represented-tree vertex and does not lie on a preferred path. Its only purpose is to balance a collection of virtual child clusters.

Thus it simply combines the represented material of its two children. If x is a fake rake node with children a, b, then

$$\boxed{\mathsf{all}(x) = \mathsf{all}(a) \oplus \mathsf{all}(b).}$$

Ordered versus unordered aggregates. Along a preferred path there is a canonical order, so path may represent a noncommutative aggregate. As in an ordinary link-cut tree, in that case we store both

$$\mathsf{path}^{\rightarrow} \qquad \text{and} \qquad \mathsf{path}^{\leftarrow}$$

and rerooting can reverse the path.

Rake children, on the other hand, have no meaningful order. Any aggregate that combines independent virtual subtrees must therefore be commutative.

9.11.5 Two Kinds of Lazy Tags

The same path/off-path distinction must also be remembered by lazy updates.

Suppose an auxiliary cluster represents

$$\boxed{\text{spine} \sqcup \text{material hanging from the spine.}}$$

An update might be intended for only the first part or only the second. We therefore maintain two tags:

$$\boxed{\mathsf{plazy} \qquad \text{and} \qquad \mathsf{slazy}.}$$

Here

$$\mathsf{plazy} = \text{pending update to the preferred-path spine,}$$

while

$$\mathsf{slazy} = \text{pending update to the material off that spine.}$$

Pushing a path tag. A path tag affects u and the parts of its compress children lying on the same spine. It does not affect any rake child.

Thus when a path tag is pushed through a real node, it is propagated as a path tag only to

$$\mathrm{ch}_0(u), \qquad \mathrm{ch}_1(u).$$

The rake children are untouched.

This is exactly what we would expect: a path update follows the preferred path and nowhere else.

Pushing an off-path tag. An off-path tag is slightly more subtle.

Consider first a compress child c. The path of c is part of the parent's path as well. Therefore it is *not* part of the parent's off-path region. Only the material hanging from that child should receive the update.

Hence an off-path tag pushed into a compress child remains

$$\boxed{\mathsf{slazy}.}$$

Now consider a rake child c.

The entire represented cluster of c hangs off the parent's path. Therefore *every represented vertex inside c* must receive the update.

But internally c again divides its material into

$$\mathsf{path}(c) \qquad \text{and} \qquad \mathsf{sub}(c).$$

Both pieces belong to the parent's off-path region.

Consequently, when an off-path update crosses a rake edge, it becomes an update to the whole child cluster, plazy and slazy.

9.11.6 Push Before Changing the Decomposition

There is one important consequence of this interpretation.

Lazy tags describe a set of *represented vertices*, not merely a set of auxiliary nodes. We must therefore never change which represented material is classified as compress or rake while an unpushed tag above it still depends on the old classification.

In particular, before removing a virtual child y, push all tags along the rake-root–y path.

After the removal, recompute aggregates on the way back up and suppress any fake rake node left with only one child.

9.11.7 Path Operations

Path operations are exactly as in an ordinary link-cut tree.

To expose

$$P(u, v),$$

perform

$$\boxed{\mathsf{MakeRoot}(u), \qquad \mathsf{Access}(v).}$$

Afterward every edge of $P(u, v)$ is preferred, so the path is precisely the preferred-path spine of the auxiliary cluster rooted at v.

Hence

$$\boxed{\mathrm{PathQuery}(u, v) = \mathsf{path}(v).}$$

Likewise, a lazy update to $P(u, v)$ is recorded as a path update:

$$\boxed{\mathsf{plazy}(v).}$$

9.11.8 Subtree Operations

Now fix a represented root r, and suppose we want the rooted subtree T_u.

Perform

$$\boxed{\mathsf{MakeRoot}(r), \qquad \mathsf{Access}(u).}$$

Afterward the preferred path ending at u is exactly

$$P(r, u).$$

Since u is the bottom endpoint of this path, it has no preferred child below it.

Therefore every proper descendant of u lies in a virtual child cluster attached at u.

Meanwhile, the compress material on the other side of u is precisely the ancestor path

$$r \rightsquigarrow u,$$

which must not be included in T_u.

Thus the rooted subtree has the decomposition

$$\boxed{T_u = \{u\} \sqcup \{\text{all virtual-child clusters attached at } u\}.}$$

If the two rake children of u are $\mathrm{ch}_2\, u$ and $\mathrm{ch}_3\, u$, then

$$\boxed{\mathrm{SubtreeQuery}(r, u) = \mathrm{val}(u) \oplus \mathsf{all}(\mathrm{ch}_2\, u) \oplus \mathsf{all}(\mathrm{ch}_3\, u).}$$

For a lazy update to the whole rooted subtree, apply the update to

$$\mathrm{val}(u)$$

and to the complete clusters represented by the rake children.

9.11.9 Link, Cut, and Reroot

The dynamic-forest operations retain the same interpretation as in the underlying link-cut tree.

To link two represented trees by an edge uv, reroot the first tree at u, expose v, and attach the complete represented tree rooted at u as a new virtual child cluster of v.

To cut an existing represented edge uv, perform

$$\mathsf{MakeRoot}(u), \qquad \mathsf{Access}(v).$$

The edge uv is now exposed in the compress structure and can be detached exactly as in the ordinary link-cut tree.

Rerooting remains

$\mathsf{Access}(u)$	followed by a lazy reversal of the exposed path.

9.11.10 Amortized Analysis

Regard every real and fake auxiliary node as belonging to one forest for analysis. For a node x, let

$$\sigma(x) = \text{represented mass contained below } x,$$

and define its rank by

$$\rho(x) = \log_2 \sigma(x).$$

Use the usual splay potential

$$\boxed{\Phi = \sum_x \rho(x).}$$

A rotation inside a compress tree and a rotation inside a rake tree are both ordinary splay tree rotations. The weighted splay access lemma therefore gives, for either kind of splay,

$$\text{amortizedCost}(\mathsf{Splay}(x)) \le 1 + 3\big(\rho_{\text{after}}(x) - \rho_{\text{before}}(x)\big).$$

Now consider the successive clusters reached during an expose,

$$x_1, x_2, \ldots, x_k.$$

After x_i has been processed, its represented cluster is contained in the larger cluster reached at the next step. Consequently

$$\sigma_{\text{after}}(x_i) \le \sigma_{\text{before}}(x_{i+1}),$$

and hence

$$\rho_{\text{after}}(x_i) \le \rho_{\text{before}}(x_{i+1}).$$

The rank increases telescope exactly as in the ordinary link-cut-tree analysis:

$$\sum_i \big(\rho_{\text{after}}(x_i) - \rho_{\text{before}}(x_i)\big) = O(\log n).$$

The important point is that moving a represented cluster between the compress and rake structures does not alter the represented material. The same vertices are still present; only the auxiliary decomposition used to store them has changed.

The additive +1 terms in the splay access bound are charged to the same preferred-path changes as in the ordinary link-cut tree. Inserting or removing a cluster from a rake tree happens precisely because a represented edge has changed between preferred and virtual status, so these are the same.

It follows that one expose costs

$$\boxed{O(\log n)}$$

amortized time.

Aggregate recomputation and lazy propagation take $O(1)$ work per visited real or fake auxiliary node and are therefore absorbed into the same bound.

Theorem 317 (Self-Adjusting Top Tree)**.** *The augmented link-cut structure above uses*

$$O(n)$$

space and supports

$$\mathsf{link}, \quad \mathsf{cut}, \quad \mathsf{reroot},$$

together with compatible path and rooted-subtree queries and lazy updates, in

$$\boxed{O(\log n)}$$

amortized time.

Proof. There are n real nodes. Every fake node is an internal node of a binary rake tree, so the total number of fake nodes is also $O(n)$.

By the preceding potential argument, every expose costs $O(\log n)$ amortized time.

Each supported operation performs only a constant number of exposes, together with a constant number of extra cluster insertions, deletions, reversals, or lazy-tag operations.

Therefore every operation takes $O(\log n)$ amortized time and the data structure uses $O(n)$ space. □

9.12 Lazy Anti-Monopoly Tree

The Lazy AM-Tree maintains an incremental MSF in $O(\log n)$ amortized time per insertion. It maintains a Transformed MST (T-MST), a rooted tree that minimizes depth while preserving the results of Path-Max queries.

TW-Transformations and Promotion A fundamental property of T-MSTs is the *TW-Transformation*: given two adjacent edges (x, y, w_1) and (y, z, w_2) with $w_1 \geq w_2$, replacing the edge (x, y, w_1) with (x, z, w_1) preserves all Path-Max queries in the tree.

The Promote(x) operation leverages this to move a node x up the tree while preserving Path-Max queries. Let $y = \mathit{parent}[x]$ and $z = \mathit{parent}[y]$, with edge weights $w_1 = \mathit{weight}(x \to y)$ and $w_2 = \mathit{weight}(y \to z)$.

- **Shortcut** ($w_1 > w_2$): x is directly promoted to become z's child with weight w_1. Node y becomes a sibling of x.
- **Rotate** ($w_1 < w_2$ or y is root): y is pushed down to become a child of x with weight w_1. If y is not the root, x is attached to z with weight w_2.

Stitch-Based Insertion Instead of finding the heaviest edge on the cycle, the Stitch-based algorithm moves the new edge (u, v, w) to its correct position using TW-transformations.

If $weight(u \to parent[u]) < w$, the edge (u, v, w) is TW-transformed to $(parent[u], v, w)$ and recursively moved upwards. Once both $weight[u \to parent[u]] > w$ and $weight[v \to parent[v]] > w$, we attach the smaller subtree to the larger one (e.g., u becomes a child of v with weight w). This displaces u's old parent edge (u, u', w'). Because $w' > w$, we apply another TW-transformation, moving the displaced edge to (u', v, w'), and recursively route it.

If the original edge closed a cycle, the relocation forces the two endpoints to eventually meet at their LCA $(u = v)$. At this point, we discard the edge.

Algorithm 53 Lazy AM-Tree

```
function Promote(x)
    y ← parent[x]
    z ← parent[y]
    w_1 ← weight(x → y); w_2 ← weight(y → z)
    if z ≠ null and w_1 > w_2 then
                                          ▷ Shortcut: x bypasses y to become its sibling
        size[y] ← size[y] − size[x]
        parent[x] ← z
        weight(x → parent) ← w_1
    else
                                          ▷ Rotate: y is pushed down as a child of x
        size[y] ← size[y] − size[x]
        size[x] ← size[x] + size[y]
        parent[x] ← z
        weight(x → parent) ← w_2
        parent[y] ← x
        weight(y → parent) ← w_1
function UpwardCalibrate(x)
    while parent[x] ≠ null do
        y ← parent[x]
        if size[x] ≤ (2/3) size[y] then
            x ← y
        else
            Promote(x)
function LinkByStitch(u, v, w)
    if u = v or u = null or v = null then
        return
    if parent[u] ≠ null and w > weight(u → parent) then
        LinkByStitch(parent[u], v, w)
    else if parent[v] ≠ null and w > weight(v → parent) then
        LinkByStitch(u, parent[v], w)
    else
        if size[u] > size[v] then
            Swap u ↔ v
        u_old_parent ← parent[u]
        w_old ← weight(u → parent)
        parent[u] ← v
        weight(u → parent) ← w
        size[v] ← size[v] + size[u]
                                          ▷ Route the displaced heavier edge
        LinkByStitch(u_old_parent, v, w_old)
function LazyInsert(u, v, w)
    UpwardCalibrate(u)
    UpwardCalibrate(v)
    LinkByStitch(u, v, w)
```

Correctness and Complexity

Theorem 318. *The Lazy AM-Tree maintains a valid T-MST and supports insertions in $O(\log n)$ amortized time.*

Proof. **Correctness:** Every operation in LinkByStitch is a valid TW-transformation. By continually replacing edges with equivalent higher-reaching edges, the sequence preserves Path-Max queries. If u and v were previously connected, the endpoints of the heaviest cycle-edge move to their LCA, where $u = v$ triggers the base case that drops the redundant edge. Therefore, the tree remains valid.
Complexity: We use the potential function $\Phi = \sum_{i=1}^{n} \log(size[i])$.

1. **Calibrate:** The Promote function is only called when x is a heavy child ($size[x] > \frac{2}{3} size[y]$). Under this condition, rotating x upwards decreases y's size by at least a factor of $1/3$, meaning Φ decreases by an amount strictly greater than the $O(1)$ actual cost of the rotation. Therefore, Promote has an amortized cost of zero. Because the loop only advances upward when the AM-rule holds, size increases by a factor of $3/2$ per advancement, limiting iterations to $O(\log n)$.

2. **Link:** The LinkByStitch recursion traverses upward, performing $O(d(u) + d(v))$ actual operations. Structural size changes only occur when the smaller subtree is attached to the larger one. Because the smaller subtree's size at most doubles, the potential Φ increases by at most 1 per topological attachment. Across the $O(d(u) + d(v))$ affected nodes, the total potential increase is bounded by $O(\log n)$.

Thus the total amortized cost per insertion is $O(\log n)$. □

10 Graph Algorithms

10.1 Incremental SCCs

Suppose directed edges

$$e_1, e_2, \ldots, e_m$$

are inserted in this order, and let G_t denote the graph containing the first t edges.
Strongly connected components only merge as t increases. Thus for every pair u, v, there is a threshold

$$\tau(u, v) = \min\{t : u, v \text{ lie in the same SCC of } G_t\},$$

with $\tau(u, v) = m + 1$ if they never become strongly connected.
In particular, if $e_t = (u_t, v_t)$, then e_t strictly changes the SCC partition exactly when

$$\boxed{\tau(u_t, v_t) = t.}$$

Indeed, if the endpoints were already strongly connected before e_t arrived, then adding e_t cannot change any reachability relation: whenever a path uses the new edge $u_t \to v_t$, we may replace it by the old path from u_t to v_t inside their SCC. Conversely, if the endpoints first become strongly connected at time t, the SCC partition has clearly changed.

Thus it is enough to compute these m threshold values simultaneously.

Divide and Conquer on Time The basic idea is binary search, with one important optimization.

Suppose we are considering a time interval $[l, r]$, and let $h = \lfloor (l + r)/2 \rfloor$. Compute the SCCs of the graph at time h.

For a query pair u, v,

- if u, v are already in the same SCC at time h, then $\tau(u, v) \le h$, so the query goes left;
- otherwise $\tau(u, v) > h$, so the query goes right.

If we did this independently at every recursive node, we would repeatedly process the same old edges. The crucial observation is that we can simplify the graph separately for the two recursive calls.

For the left half, an edge joining two different SCCs at time h is irrelevant. Such an edge cannot participate in any directed cycle at any earlier time: if it did, its endpoints would already belong to the same SCC at time h.

For the right half, every SCC present at time h will remain strongly connected forever. We may therefore contract each such SCC to a single vertex.

Thus every relevant edge is sent to only one child.

Algorithm 54 Offline Incremental SCCs

Input: Vertices V, chronological edges $E = \langle e_1, \ldots, e_m \rangle$
Output: The edges whose insertion strictly changes the SCC partition

$Q \leftarrow \langle (i, u_i, v_i) : e_i = (u_i, v_i) \rangle$
$\tau_i \leftarrow m + 1$ for every i
$A \leftarrow \langle (i, u_i, v_i) : e_i = (u_i, v_i) \rangle$
SOLVE$(0, m + 1, V, A, Q, \tau)$
$M \leftarrow \langle\rangle$
for $i = 1, \ldots, m$ **do**
    if $\tau_i = i$ **then**
        append e_i to M
return M
function SOLVE(l, r, V', A, Q, τ)
    if $l = r$ **then**
        for $(i, u, v) \in Q$ **do**
            $\tau_i \leftarrow l$
        return
    $h \leftarrow \lfloor (l + r)/2 \rfloor$
    ▷ *SCCs at the midpoint using edges already present by time h*
    $C \leftarrow \mathrm{SCC}(V', \langle (u, v) : (t, u, v) \in A,\ t \le h \rangle)$
    $A_L, A_R, Q_L, Q_R \leftarrow \langle\rangle$
    for $(t, u, v) \in A$ **do**
        if $t \le h$ **and** $C[u] = C[v]$ **then**
            append (t, u, v) to A_L
        else if $C[u] \neq C[v]$ **then**
            append $(t, C[u], C[v])$ to A_R
    ▷ *If $t > h$ and $C[u] = C[v]$, this future edge is already internal and can be discarded*
    for $(i, u, v) \in Q$ **do**
        if $C[u] = C[v]$ **then**
            append (i, u, v) to Q_L
        else
            append $(i, C[u], C[v])$ to Q_R
    $V_R \leftarrow \{C[v] : v \in V'\}$
    SOLVE$(l, h, V', A_L, Q_L, \tau)$
    SOLVE$(h + 1, r, V_R, A_R, Q_R, \tau)$

Why the Edge Reduction is Correct The query routing is immediate from monotonicity of SCCs. The only issue is showing that the reduced edge sets still contain all information needed by the two recursive calls.

Let C be the SCC partition at time h.

For the left call, consider an edge $e = (u, v)$ present by time h with $C[u] \neq C[v]$. This edge cannot affect the SCC partition at any time $t \le h$. Indeed, if it belonged to a directed cycle at such a time, then u and v

would be mutually reachable at time t, and hence also at time h, contradicting $C[u] \neq C[v]$. More generally, the edges between different C-components form the condensation DAG, and deleting edges of this DAG does not alter the SCCs themselves.

Hence the left recursion needs only the edges already present by time h whose endpoints lie in the same C-component.

For the right call, every C-component is already strongly connected at time h, so it remains strongly connected at every later time. Contracting it is therefore exact. Edges already internal to a C-component can be discarded. Every other edge is represented simply by the edge between the corresponding contracted components.

Notice also that if an edge arrives only after time h but its endpoints already satisfy $C[u] = C[v]$, then it can be discarded permanently: when it eventually arrives, its endpoints are already strongly connected, so it can never change the SCC partition.

These are precisely the rules used by Algorithm 54.

Theorem 319. *Algorithm 54 returns exactly the edges whose insertion strictly changes the strongly connected component partition, in*

$$\boxed{O(m \log m)}$$

time, apart from the initial $O(|V|)$ cost of reading isolated vertices.

Proof. We prove recursively that every query (i, u, v) passed to a call $\textsc{Solve}(l, r, \dots)$ satisfies

$$\tau(u, v) \in [l, r],$$

where the vertices are interpreted after undoing the contractions made on the path from the root.

At a midpoint h, the preceding discussion shows that the SCC partition computed from the reduced graph is exactly the SCC partition of G_h, modulo the contractions already made.

If $C[u] = C[v]$, then the two query vertices are strongly connected by time h, so their threshold lies in $[l, h]$. Otherwise their threshold is strictly larger than h, so it lies in $[h + 1, r]$. Thus every query is sent to the child containing its threshold.

The edge-reduction argument above proves that the reduced graph passed to each child has exactly the same SCC partitions, throughout that child's time interval, as the corresponding original graphs. Hence the invariant continues to hold recursively.

At a leaf $l = r$, the only possible threshold is l, so assigning $\tau_i = l$ is correct.

Finally, as observed at the beginning, $e_i = (u_i, v_i)$ changes the SCC partition exactly when $\tau(u_i, v_i) = i$. Therefore the returned list M is correct.

For the running time, implement each recursive call only on vertices appearing as endpoints of its active edges or queries. Computing SCCs and partitioning the lists then takes linear time in the size of that call.

Every active edge is passed to at most one child, and every query is passed to exactly one child. Hence the total size of all calls at any fixed recursion depth is $O(m)$. The recursion has $O(\log m)$ levels, giving total time

$$O(m \log m).$$

□

10.2 All-Pairs Connectivity

Let

$$G = (V, E)$$

be a directed graph with

$$m = |E|.$$

For vertices s, t, let

$$\lambda(s,t)$$

be the maximum number of pairwise edge-disjoint directed s–t paths.

There is a direct algebraic representation of all these values at once.

10.2.1 A Walk Generating Matrix

Index rows and columns by directed edges.

For every composable pair

$$e, f \in E, \qquad \text{head}(e) = \text{tail}(f),$$

introduce a variable

$$x_{ef}.$$

Define

$$X \in \mathbb{F}[x]^{E \times E}$$

by

$$X[e,f] = \begin{cases} x_{ef}, & \text{head}(e) = \text{tail}(f), \\ 0, & \text{otherwise.} \end{cases}$$

Work first over the formal power-series ring. Then

$$\boxed{\Gamma = (I - X)^{-1} = I + X + X^2 + \cdots.}$$

If

$$W = (e_1, \ldots, e_\ell)$$

is a directed walk, define

$$\xi(W) = \prod_{j=1}^{\ell-1} x_{e_j, e_{j+1}}.$$

Lemma 320 (Walk Enumeration)**.** *For edges e, f,*

$$\boxed{\Gamma[e,f] = \sum_{W : e \rightsquigarrow f} \xi(W),}$$

where the sum ranges over directed walks beginning with e and ending with f.

Proof. The entry

$$X^j[e,f]$$

is exactly the sum of the monomials of walks consisting of $j+1$ edges, starting at e and ending at f. Sum over $j \geq 0$. □

10.2.2 Determinants Enumerate Edge-Disjoint Walks

Let

$$S = \{e_1, \ldots, e_r\}, \qquad T = \{f_1, \ldots, f_r\}$$

be equally sized edge sets.

Expand

$$\det \Gamma[S,T].$$

Each term chooses

- a permutation $\sigma \in S_r$;
- for every i, a walk from e_i to $f_{\sigma(i)}$.

Thus it is a signed generating function for families of walks.

We now work in characteristic 2.

Lemma 321 (Walk Cancellation)**.** *Over characteristic* 2*,*

$$\det \Gamma[S,T]$$

is the generating function of pairwise edge-disjoint walk families connecting S to T. In particular,

$$\boxed{\det \Gamma[S,T] \not\equiv 0}$$

if and only if there exist r pairwise edge-disjoint directed paths joining the starting edges in S to the ending edges in T in some pairing.

Proof. Consider a family in which two walks share an edge. Fix an ordering of walks and positions, and choose canonically the first shared edge

$$h$$

and the first pair of walks containing it.

Write the two walks as

$$P = P^- h P^+, \qquad Q = Q^- h Q^+.$$

Swap the suffixes:

$$P' = P^- h Q^+, \qquad Q' = Q^- h P^+.$$

The endpoint permutation is composed with a transposition. More importantly, the multiset of transition variables

$$x_{ef}$$

is unchanged, so the two families have exactly the same monomial.

The operation is an involution. In characteristic 2, the two identical contributions cancel.

Thus only pairwise edge-disjoint families remain.

For an edge-disjoint family, its transition monomial together with its starting edges reconstructs every walk, so distinct surviving families have distinct monomials and cannot cancel.

Finally, any edge-disjoint walk can have directed cycles deleted without creating an edge intersection, giving edge-disjoint paths. □

10.2.3 Connectivity is Matrix Rank

Let

$$E^{\text{out}}(s) = \{e : \text{tail}(e) = s\}$$

and

$$E^{\text{in}}(t) = \{e : \text{head}(e) = t\}.$$

Theorem 322 (Connectivity via Rank)**.** *Over the field of rational functions,*

$$\boxed{\lambda(s,t) = \operatorname{rank} \Gamma[E^{\text{out}}(s), E^{\text{in}}(t)].}$$

Proof. A matrix has rank at least r exactly when it has a nonzero $r \times r$ minor.

By Lemma 321, such a minor is nonzero exactly when there are r pairwise edge-disjoint directed s–t paths.

The largest possible r is therefore

$$\lambda(s,t).$$

□

Thus all-pairs connectivity is encoded by one matrix:

$$\boxed{(I - X)^{-1}.}$$

10.2.4 Random Evaluation

We cannot manipulate the rational functions explicitly. Instead choose a sufficiently large finite field

$$\mathbb{F}_{2^q}, \qquad q = \Theta(\log n),$$

and substitute independent uniformly random field elements for all variables x_{ef}.

Let

$$A$$

be the resulting numerical matrix.

By Schwartz–Zippel, with high probability,

- $I - A$ is invertible;
- every minor relevant to any pair (s,t) is zero after evaluation exactly when it was identically zero.

Hence simultaneously for every s, t,

$$\boxed{\lambda(s,t) = \operatorname{rank}\left((I-A)^{-1}[E^{\mathrm{out}}(s), E^{\mathrm{in}}(t)]\right)}$$

with high probability.

Algorithm 55 All-Pairs Directed Edge Connectivity

```
Choose random transition weights A[e, f] ∈ F_{2^q} whenever head(e) = tail(f)
M ← (I − A)^{−1}
for s, t ∈ V do
    λ(s, t) ← rank M[E^out(s), E^in(t)]
return all λ(s, t)
```

Theorem 323. *Algorithm 55 computes every directed edge-connectivity with high probability in*

$$\boxed{\widetilde{O}(m^{\omega})}$$

time.

Proof. The matrix inversion costs

$$O(m^{\omega})$$

field operations.

The rank of an $a \times b$ matrix can be computed in

$$O(ab^{\omega-1})$$

operations after exchanging a, b if necessary. Summing these costs over all pairs (s,t) and using

$$\sum_{s} \deg^{+}(s) = \sum_{t} \deg^{-}(t) = m$$

is bounded by

$$O(m^{\omega}).$$

The field operations cost only polylogarithmic time. □

10.2.5 Bounded All-Pairs Connectivity

Suppose now that we only want

$$\lambda_k(s,t) = \min\{k, \lambda(s,t)\}$$

for every pair s, t.

The algorithm above uses an $m \times m$ inverse. For small k, this contains much more information than necessary. We can compress the transition matrix to rank $O(kn)$ while preserving all minors of size at most k.

There are two steps:

1. modify the graph so every queried source has outdegree k and every queried target has indegree k;
2. factor the edge-transition matrix through only k copies of each vertex.

Degree Reduction Construct a graph G' as follows. For every original vertex v, create two additional vertices v_{in} and v_{out}.

Replace every original edge

$$(u, v)$$

by

$$(u_{\text{out}}, v_{\text{in}}).$$

Also add k parallel copies of

$$v \to v_{\text{out}} \qquad \text{and} \qquad v_{\text{in}} \to v.$$

Thus every original vertex has exactly k outgoing edges and exactly k incoming edges in G'.

The new graph has

$$n' = 3n, \qquad m' = m + 2kn.$$

Lemma 324 (Degree Reduction)**.** *For original vertices s, t,*

$$\boxed{\lambda_{G'}(s,t) = \min\{k, \lambda_G(s,t)\}.}$$

Proof. Any directed s–t path in G' projects to a directed s–t path in G by retaining only the original vertices. Furthermore, edge-disjoint paths in G' project to edge-disjoint paths in G, because every transformed original edge $u_{\text{out}} \to v_{\text{in}}$ corresponds to a unique original edge $u \to v$.

Hence

$$\lambda_{G'}(s,t) \le \lambda_G(s,t).$$

Also s has only k outgoing edges in G', so

$$\lambda_{G'}(s,t) \le k.$$

For the reverse inequality, let

$$r = \min\{k, \lambda_G(s,t)\},$$

and choose r edge-disjoint s–t paths $P_1, \dots, P_r$ in G.

Lift P_i to G' by replacing every original edge $u \to v$ by

$$u \to u_{\text{out}} \to v_{\text{in}} \to v,$$

using the i-th parallel copy of each gadget edge $u \to u_{\text{out}}$ and $v_{\text{in}} \to v$.

The transformed original edges remain disjoint because the P_i were edge-disjoint, and the gadget edges are disjoint because the paths use different copy indices i. Thus the lifted paths are edge-disjoint in G', proving

$$\lambda_{G'}(s,t) \geq r.$$

□

A Low-Rank Transition Matrix We now apply the walk-generating construction to G'.

Instead of giving every composable pair of edges e, f an independent variable x_{ef}, factor this transition through one of k copies of their common vertex.

For every edge e, vertex v, and $j \in [k]$, introduce variables $y_{e,j}$ and $z_{j,e}$. Define

$$Y \in \mathbb{F}[y]^{E' \times (V' \times [k])}$$

by

$$Y[e,(v,j)] = \begin{cases} y_{e,j}, & \text{head}(e) = v, \\ 0, & \text{otherwise}, \end{cases}$$

and define

$$Z \in \mathbb{F}[z]^{(V' \times [k]) \times E'}$$

by

$$Z[(v,j),f] = \begin{cases} z_{j,f}, & \text{tail}(f) = v, \\ 0, & \text{otherwise.} \end{cases}$$

Then

$$(YZ)[e,f] = \begin{cases} \displaystyle\sum_{j=1}^{k} y_{e,j} z_{j,f}, & \text{head}(e) = \text{tail}(f), \\ 0, & \text{otherwise.} \end{cases}$$

Thus YZ is obtained from the old transition matrix X by the substitution

$$\boxed{x_{ef} \longmapsto \sum_{j=1}^{k} y_{e,j} z_{j,f}.}$$

The advantage is that

$$Y : \mathbb{F}^{kn'} \to \mathbb{F}^{m'}, \qquad Z : \mathbb{F}^{m'} \to \mathbb{F}^{kn'},$$

so

$$\text{rank}(YZ) \leq kn' = O(kn).$$

Lemma 325 (Low-Rank Path Detection)**.** *Let $S, T \subseteq E'$ have*

$$|S| = |T| = r \leq k.$$

Then

$$\boxed{\det(I - YZ)^{-1}[S,T] \not\equiv 0}$$

if and only if there are r pairwise edge-disjoint directed paths joining S to T.

Proof. Start with the exact generating function

$$\det(I - X)^{-1}[S,T].$$

By Lemma 321, this is nonzero exactly when there are r edge-disjoint paths from S to T.
If no such paths exist, this formal power series is identically zero. Substituting

$$x_{ef} = \sum_{j=1}^{k} y_{e,j} z_{j,f}$$

cannot turn the zero series into a nonzero one. Since this substitution sends X to YZ, we obtain

$$\det(I - YZ)^{-1}[S,T] \equiv 0.$$

Conversely, suppose

$$P_1, \ldots, P_r$$

are edge-disjoint paths from S to T. Since $r \leq k$, assign a different label i to each path P_i.
If

$$P_i = (e_{i,1}, e_{i,2}, \ldots, e_{i,\ell_i}),$$

then in every transition $e_{i,a} \to e_{i,a+1}$, choose the $j = i$ term from

$$\sum_{j=1}^{k} y_{e_{i,a},j} z_{j,e_{i,a+1}}.$$

This produces the monomial

$$\mu = \prod_{i=1}^{r} \prod_{a=1}^{\ell_i - 1} y_{e_{i,a},i} z_{i,e_{i,a+1}}.$$

Because the paths are edge-disjoint and use distinct labels, this monomial records the edges belonging to each individual path. The starting and ending edges are fixed by S and T, so the corresponding path family is recovered uniquely. Hence μ cannot be cancelled by another surviving family.
Therefore its coefficient is nonzero, and

$$\det(I - YZ)^{-1}[S,T] \not\equiv 0.$$

□

Reducing the Matrix Inversion Now use the inverse identity

$$\boxed{(I - YZ)^{-1} = I + Y(I - ZY)^{-1}Z}$$

from Theorem 24.

The left side is an $m' \times m'$ matrix, but the only matrix we actually need to invert on the right is

$$I - ZY,$$

whose dimension is

$$kn' = 3kn.$$

This is the source of the speedup.

Random Evaluation Choose a sufficiently large field $\mathbb{F}_{2^q}$, with $q = \Theta(\log n)$, and independently evaluate all y- and z-variables at uniformly random field elements.

Let the resulting numerical matrices be

$$B \in \mathbb{F}^{m' \times kn'}, \qquad C \in \mathbb{F}^{kn' \times m'}.$$

Let E^{out} be the union of the k edges leaving each *original* vertex, and let E^{in} be the union of the k edges entering each original vertex. Define

$$\widetilde{B} = B[E^{\text{out}}, \cdot], \qquad \widetilde{C} = C[\cdot, E^{\text{in}}],$$

and compute

$$\boxed{M = \widetilde{B}(I - CB)^{-1}\widetilde{C}.}$$

For original vertices s, t, the sets $E^{\text{out}}(s)$ and $E^{\text{in}}(t)$ are disjoint, so the identity term in

$$(I - BC)^{-1} = I + B(I - CB)^{-1}C$$

vanishes on the corresponding submatrix. Hence

$$(I - BC)^{-1}[E^{\text{out}}(s), E^{\text{in}}(t)] = M[E^{\text{out}}(s), E^{\text{in}}(t)].$$

By Lemma 325 and Schwartz–Zippel, with high probability every minor of size at most k needed below has the same zero/nonzero status after the random evaluation as it had formally. Therefore, simultaneously for every original s, t,

$$\boxed{\operatorname{rank} M[E^{\text{out}}(s), E^{\text{in}}(t)] = \min\{k, \lambda_G(s,t)\}.}$$

Algorithm 56 k-Bounded All-Pairs Directed Edge Connectivity

1: Construct the degree-reduced graph G'
2: Randomly evaluate the low-rank matrices Y, Z to B, C
3: $M \leftarrow \widetilde{B}(I - CB)^{-1}\widetilde{C}$
4: **for** $s, t \in V$ **do**
5: $\quad \lambda_k(s,t) \leftarrow \operatorname{rank} M[E^{\text{out}}(s), E^{\text{in}}(t)]$
6: **return** all $\lambda_k(s,t)$

Theorem 326 (Bounded All-Pairs Connectivity)**.** *For a simple directed graph on n vertices and any $1 \le k < n$, all values*

$$\min\{k, \lambda(s,t)\}$$

can be computed with high probability in

$$\boxed{\widetilde{O}((kn)^\omega)}$$

time.

Proof. Correctness follows from Lemmas 324 and 325 and the random-evaluation argument above.

It remains to bound the running time.

The matrix CB has dimension

$$kn' \times kn' = 3kn \times 3kn.$$

Its entries have the form

$$(CB)[(u,i),(v,j)] = \sum_{\substack{e \in E' \\ \text{tail}(e)=u \\ \text{head}(e)=v}} c_{i,e} b_{e,j}.$$

Because of the special structure of G', this matrix can be assembled in $O((kn)^2)$ field operations: between non-original gadget vertices there is at most one transformed original edge, while the only parallel classes have exactly k edges.

We can therefore compute

$$(I - CB)^{-1}$$

in

$$O((kn)^\omega)$$

field operations. Multiplying on the left by $\widetilde{B}$ and on the right by $\widetilde{C}$ has the same asymptotic cost.

Finally, each queried block

$$M[E^{\text{out}}(s), E^{\text{in}}(t)]$$

is only $k \times k$. Computing its rank costs $O(k^\omega)$ field operations, so all n^2 ranks cost

$$O(k^\omega n^2) \le O((kn)^\omega).$$

All field elements have $O(\log n)$ bits, giving the stated $\widetilde{O}((kn)^\omega)$ bit complexity. □

10.3 Flows

10.3.1 Highest Label Push-Relabel

We analyze the highest-level selection rule for the preflow-push algorithm.

Let $K = \sqrt{m}$. For any node v, define $d'(v) = |\{w : d(w) \le d(v)\}|$. Thus $d'(v)$ is the number of nodes with a distance label no greater than v's label. We define the potential function Φ as:

$$\Phi = \sum_{v \text{ active}} \frac{d'(v)}{K}$$

Initially, $\Phi \le n^2/K$, and $\Phi \ge 0$ throughout the execution. Let us examine how the operations affect Φ:

- **Relabeling:** A relabeling of a node v increases Φ by at most n/K, because $d'(v)$ may increase to at most n, while $d'(w)$ for all other nodes $w \neq v$ does not increase.
- **Saturating push:** This operation creates at most one new active node, increasing Φ by at most n/K.
- **Nonsaturating push:** A nonsaturating push from v to w deactivates v and activates w (if it is not already active). Since the push only happens on eligible edges where $d(w) < d(v)$, we know $d'(w) \leq d'(v)$, meaning Φ does not increase.

To bound the number of nonsaturating pushes, we divide the execution into phases. A phase consists of all pushes between two consecutive changes of the maximum active node label, $d^* = \max\{d(v) : v \text{ is active}\}$. We classify a phase as *cheap* if it contains at most K nonsaturating pushes, and *expensive* otherwise.

Claim 327. *The number of phases is at most $4n^2$, and the number of nonsaturating pushes in cheap phases is at most $4n^2K$.*

Proof. Initially, $d^* = 0$, and any increase in d^* is strictly due to a relabeling. The number of relabelings is bounded by $2n^2$. Thus, d^* increases at most $2n^2$ times and consequently decreases at most $2n^2$ times, limiting the total number of changes (and thus phases) to $4n^2$. The bound on cheap pushes follows immediately since each cheap phase has at most K pushes. □

Claim 328. *An expensive phase containing $Q > K$ nonsaturating pushes decreases Φ by at least Q.*

Proof. During an expensive phase, d^* remains constant, meaning all Q nonsaturating pushes originate from nodes at level d^*. The phase ends either when all nodes at level d^* are deactivated, or a node is relabeled to $d^* + 1$. Therefore, level d^* must contain $Q > K$ nodes (active or inactive) at all times during the phase. For any nonsaturating push along an edge (u, v), because there are at least K nodes at level $d(u)$, each push decreases Φ by at least one. □

The total increase of Φ is bounded by the sum of increases from relabelings and saturating pushes. There are at most $2n^2$ relabelings and $2nm$ saturating pushes, so the total increase is at most $(2n^2 + 2nm)(n/K)$. Because $\Phi \geq 0$, the total decrease is bounded by the total increase plus the initial Φ value (n^2/K).

This bounds the number of nonsaturating pushes in expensive phases by $(2n^3 + 2n^2m + n^2)/K \leq (3n^3 + 2n^2m)/K$. Adding the cheap pushes, the total number of nonsaturating pushes is at most $(3n^3 + 2n^2m)/K + 4n^2K$. By setting $K = \sqrt{m}$ and noting $n = O(m)$, this evaluates to $O(n^2\sqrt{m})$.

Theorem 329. *The running time of the preflow-push algorithm using the highest-level selection rule is $O(n^2\sqrt{m})$.*

Proof. Each push and relabel operation can be implemented in $O(1)$ time. The highest-level selection rule can be implemented using a bucket data structure where each bucket i holds all nodes w with $d(w) = i$. Maintaining this structure incurs a total running time overhead of $O(nm)$. Therefore, the algorithm's running time is dominated by the $O(n^2\sqrt{m})$ nonsaturating pushes, resulting in an overall time complexity of $O(n^2\sqrt{m})$. □

10.3.2 Flow Rounding

Flow rounding converts a fractional circulation into an integral one without worsening the overall cost. We do this by eliminating fractional cycles.

Availability and Cycle Cancelling To maintain edge capacity constraints while pushing flow around cycles, the algorithm relies on the concept of availability. The availability of a directed edge (u, v) is the amount of flow that can be pushed before the flow reaches the next integer bound.

Link-Cut Trees To efficiently detect cycles and push flow, the algorithm maintains a forest of the current fractional edges using a Link-Cut Tree. It supports querying the minimum availability along a path, calculating path costs, and pushing flow along paths in $O(\log n)$ time.

Algorithm 57 Rounding Fractional Circulations

Input: Graph $G = (V, E)$, Original fractional circulation f
Output: Integral circulation f' with equal or lesser cost

```
1:  function RoundFlow(G, f)
2:      f' ← f
3:      Initialize empty Link-Cut Tree forest G' = (V, E' = ∅)
4:      for each edge (u, v) ∈ E with fractional f'(u, v) do
5:          if FindRoot(u) ≠ FindRoot(v) then
6:              Link(u, v)                                  ▷ Edge does not form a cycle
7:          else
                                  ▷ A fractional cycle is formed by (u, v) and the v → u tree path
8:              P ← Path between v and u in G'
9:              Determine cycle direction (forward or reverse) that yields ≤ 0 cost
                                          ▷ Push flow until at least one edge becomes integral
10:             δ ← min(FindMin(u, v), available(v, u))
11:             PushFlow(u, v, δ)                           ▷ Update flow along cycle
                                        ▷ Remove all newly integral edges from the forest
12:             while (e_int ← FindIntegralEdge(u, v)) ≠ null do
13:                 Cut(e_int)
14:             if f'(u, v) is still fractional then
15:                 Link(u, v)
16:     return f'
```

Correctness

Theorem 330. *The algorithm correctly produces an integral circulation of no worse cost in $O(m \log n)$ time.*

Proof. By evaluating the cost of the path in the Link-Cut tree, we can always push flow around a newly formed cycle in a direction where the total cost is non-positive, ensuring the overall circulation cost never increases. At termination, every fractional edge has been evaluated. The remaining edges in the Link-Cut tree explicitly form a forest, meaning the final circulation f' must be integral. □

Lemma 331 (Split flow rounding)**.** *Let f be an integral circulation on a digraph D and d any positive integer. Then f can be written as the sum $g_1 + g_2 + \cdots + g_d$ of integral circulations such that for each index j and each edge e:*

$$\lfloor f(e)/d \rfloor \leq g_j(e) \leq \lceil f(e)/d \rceil$$

Proof Sketch. We proceed by induction on d. Round $\frac{f}{d}$ to some integral circulation g, then consider $f - g = g_2 + \cdots + g_d$ inductively, then the bounds we get for $g_2, \ldots, g_d$ imply also that they satisfy the needed bounds.

□

Corollary 332 (Split matrix rounding)**.** *Let $A \in \mathbb{Z}^{m \times n}$.*

1. *For every $\alpha \in \mathbb{R}$, there is an integral matrix B such that every entry, row sum, column sum, and the total sum of B is the corresponding quantity of αA, rounded either up or down.*

2. *For every positive integer d,*

$$A = B_1 + \cdots + B_d$$

for integral matrices $B_1, \ldots, B_d$, where for each B_k every entry, row sum, column sum, and total sum is the corresponding quantity of A/d, rounded either up or down.

Proof. Represent $A = (a_{ij})$ by a circulation on the digraph with vertices

$$s, \quad r_1, \ldots, r_m, \quad c_1, \ldots, c_n, \quad t$$

and edges

$$s \to r_i, \qquad r_i \to c_j, \qquad c_j \to t, \qquad t \to s.$$

Give these edges respectively the values

$$\sum_j a_{ij}, \qquad a_{ij}, \qquad \sum_i a_{ij}, \qquad \sum_{i,j} a_{ij}.$$

Flow conservation holds at every vertex, so this is an integral circulation. Thus rounding the circulation preserves simultaneously the entries, row sums, column sums, and total sum.

For the first claim, round the circulation αf to an integral circulation, with every edge rounded up or down, and read off the values on the edges $r_i \to c_j$.

For the second, apply the preceding circulation-decomposition lemma to write

$$f = g_1 + \cdots + g_d, \qquad \left\lfloor \frac{f(e)}{d} \right\rfloor \le g_k(e) \le \left\lceil \frac{f(e)}{d} \right\rceil.$$

Let $(B_k)_{ij} = g_k(r_i, c_j)$. Since each g_k is a circulation, its values on the other three types of edges are exactly the row sums, column sums, and total sum of B_k, so all the required rounding bounds follow. □

10.4 Cut-Matching Game and Sparse Cuts

For an undirected graph with capacities c_e, define its sparsity by $\psi(G) = \min_{\emptyset \neq S \neq V} c(\partial S)/\min\{|S|, |V \setminus S|\}$. Cut-matching certifies expansion by constructing a multicommodity flow that sends a small amount from every vertex to every other vertex. A sparse cut would have too little capacity to carry all the flow between its two sides, so such a routing with low congestion gives a lower bound on $\psi(G)$.

Initially, each vertex v holds one unit of its own commodity. In each round, we route a matching and let its endpoints exchange half of everything they hold. Both endpoints then hold the average of their previous mixtures. For ordinary matchings, after one round v's commodity is shared equally by two vertices. If each holder next meets a vertex with none of that commodity, it is shared equally by four vertices, then eight, and so on. This suggests that repeated averaging can spread every commodity throughout the graph in logarithmically many rounds. The difficulty is to keep bringing different mixtures together: exchanging identical mixtures makes no progress. We choose a bisection that forces useful averaging across it, even though the flow computation chooses the matching. The analysis below guarantees $O(\log^2 n)$ rounds with high probability. Assume $n = |V|$ is even.

Theorem 333 (Khandekar–Rao–Vazirani)**.** *Given $\psi > 0$ and $0 < \delta < 1$, a randomized algorithm using $T = O(\log n \log(n/\delta))$ max-flow computations either returns a cut of sparsity less than ψ, or, with failure probability at most δ, establishes $\psi(G) \ge \psi/(2T)$. In particular, polynomially small failure probability costs $O(\log^2 n)$ flows.*

A *fractional perfect matching* across a bisection assigns nonnegative weights to crossing pairs, with total weight one at each vertex. Write W_t for its symmetric adjacency matrix. Vertex i keeps half of each commodity it holds and sends a fraction $W_t(i,j)/2$ of each to vertex j. The corresponding averaging matrix

is $A_t = (I + W_t)/2$. Ordinary perfect matchings are a special case; fractional ones allow arbitrary input capacities.

To choose a bisection, give each commodity a fresh independent Gaussian label and score each vertex by the weighted sum of the labels in its mixture. We cut between the lower and upper halves of these scores. Every crossing matching then pairs low scores with high scores; the proof shows that this forces enough averaging of different mixtures. The scores can be computed by applying the previous averaging steps to the vector of labels, so we never need to store the full multicommodity flow explicitly.

Algorithm 58 Cut-Matching Test

Input: Capacitated graph G, target $\psi > 0$, number of rounds T

for $t = 1, \dots, T$ **do**
  Sample $h \sim N(0, I_n)$ and set $g \leftarrow h - (\sum_i h_i/n)\mathbf{1}$
  $u \leftarrow A_{t-1} \cdots A_1 g$ ▷ *Replay the previous averaging steps*
  Let S contain the $n/2$ smallest coordinates of u
  Give each original edge capacity c_e/ψ; add unit-capacity arcs $s \to v$ for $v \in S$ and $v \to z$ for $v \notin S$
  Compute a maximum s–z flow and a minimum cut
  if the flow value is less than $n/2$ **then**
    return the original vertices on the source side of the cut
  Cancel opposite flows, discard cycles, and store a decomposition into weighted paths
  Give each endpoint pair its total path weight to obtain W_t, and set $A_t \leftarrow (I + W_t)/2$
return the matchings and their path routings ▷ *They encode the multicommodity flow*

Proof. First check the flow step. If a minimum cut has original vertex set U on its source side, write $a = |S \cap U|$ and $b = |U \setminus S|$. Its capacity is $n/2 - a + b + c(\partial U)/\psi$: the first two terms count the terminal arcs crossing the cut. A value below $n/2$ therefore gives

$$\frac{c(\partial U)}{\psi} < a - b \le \min\{|U|, n - |U|\}.$$

Here $a - b \le a + b = |U|$, and $a - b \le n - a - b = n - |U|$ because $a \le n/2$. Thus U is nontrivial and has sparsity less than ψ. Otherwise every terminal arc is saturated. After cancelling opposite flows and removing cycles, path decomposition gives a fractional perfect matching across the bisection, routed with total load at most c_e/ψ on each original edge e.

Now interpret these matchings as exchanges of commodities. Let $P_t(i, v)$ be the amount of commodity v held at vertex i after t rounds. Initially $P_0 = I$, and the exchange rule gives

$$P_t(i, v) = \frac{1}{2} P_{t-1}(i, v) + \frac{1}{2} \sum_j W_t(i, j) P_{t-1}(j, v), \qquad P_t = A_t \cdots A_1.$$

The columns of P_t sum to one because each commodity has total amount one. Its rows also sum to one: each vertex sends half a unit and receives half a unit, so it always holds one unit in total.

Suppose all the single-commodity flows succeed. A routing path of matching weight a can implement its exchange by carrying $a/2$ of each endpoint's mixture to the other endpoint. Since both mixtures have total amount one, this puts total load a on the path, counting both directions. Thus one round places load at most c_e/ψ on edge e, and all T rounds together place load at most Tc_e/ψ. Tracing each portion of a commodity through the successive exchanges gives a route from its original vertex to its final holder. These routes form a multicommodity flow with congestion at most T/ψ; any loops in a route may be deleted without increasing the load.

Our target is $P_T(i, v) \ge 1/(2n)$ for every i, v. By discarding excess flow separately for each source and destination, this gives a uniform multicommodity flow of $1/(2n)$ units per ordered pair. For any nontrivial cut U, the commodities originating in U send at least $|U|(n - |U|)/(2n)$ units outside, and those originating

outside send at least the same amount into U. All this flow must cross the cut, so the congestion bound gives

$$\frac{|U|(n-|U|)}{n} \le \frac{T}{\psi} c(\partial U).$$

Since $|U|(n-|U|)/n \ge \frac{1}{2}\min\{|U|, n-|U|\}$, this directly certifies $\psi(G) \ge \psi/(2T)$.

It remains to show that the exchanges distribute every commodity sufficiently evenly, regardless of the flow choices. For this analysis, continue with arbitrary crossing matchings if a flow fails; the actual algorithm would already have returned a valid cut. The ideal final matrix is $J = \mathbf{1}\mathbf{1}^\top/n$: every vertex holds $1/n$ of every commodity. We measure the discrepancy from this ideal by $\Phi_t = \|P_t - J\|_F^2$, initially $\Phi_0 = n-1$. It suffices to make $\Phi_T \le 1/(4n^2)$, because then $|P_T(i,v) - 1/n| \le \sqrt{\Phi_T} \le 1/(2n)$ for every i, v.

Fix the previous rounds and write $X = P_{t-1} - J$, with rows x_i. Thus x_i records how vertex i's mixture differs from the uniform mixture, and $x_i - x_j$ records the difference between two mixtures. All probabilities and expectations in the one-round analysis below are conditional on these previous rounds, so X is fixed. Since $A_t J = J$, the new centered matrix is $A_t X$. For an ordinary matching, each matched pair of rows is replaced by its midpoint, reducing their combined squared norm by $\|x_i - x_j\|^2/2$. The fractional case has the same lower bound: row i becomes the convex combination $\sum_j W_t(i,j)(x_i + x_j)/2$, so convexity of squared norm gives

$$\begin{aligned}\Phi_t &\le \sum_{i,j} W_t(i,j) \left\| \frac{x_i + x_j}{2} \right\|^2 \\ &= \Phi_{t-1} - \frac{1}{2} \sum_{\{i,j\}} W_t(i,j) \|x_i - x_j\|^2.\end{aligned}$$

The equality follows by expanding the squared norm and using that every row and column of W_t sums to one; the last sum counts each unordered pair once. Thus averaging always decreases the potential, and the decrease is large when the matching pairs sufficiently different mixtures.

The Gaussian labels give a scalar measure of these differences. Let h be the fresh standard Gaussian vector before centering. The score of the mixture at i is $\sum_v P_{t-1}(i,v) h_v$. Subtracting the common mean label centers these scores without changing their order. Since $P_{t-1}J = J$, the algorithm therefore computes $u = P_{t-1}(I-J)h = Xh$, so $u_i = \langle x_i, h\rangle$ and $\sum_i u_i = 0$. Choose μ between the two middle coordinates of u. Every pair with positive matching weight has one endpoint on each side of μ. For $u_i \le \mu \le u_j$,

$$(u_i - u_j)^2 = \big((\mu - u_i) + (u_j - \mu)\big)^2 \ge (u_i - \mu)^2 + (u_j - \mu)^2.$$

Each vertex has total matching weight one, so summing charges its entire squared distance from μ to the matching edges. Consequently, writing $Z = \sum_i u_i^2$, we obtain

$$\sum_{\{i,j\}} W_t(i,j)(u_i - u_j)^2 \ge \sum_i (u_i - \mu)^2 = Z + n\mu^2 \ge Z.$$

This is why we cut at the median: every crossing matching must account for all the spread in the scores.

To turn separation of scores into a potential decrease, we need to relate it to differences between the actual mixtures. For a fixed pair, $u_i - u_j = \langle x_i - x_j, h\rangle$ is Gaussian with variance $\|x_i - x_j\|^2$. Its tail satisfies $\Pr((u_i - u_j)^2 > L\|x_i - x_j\|^2) \le 2e^{-L/2}$ (and the bound is immediate if $x_i = x_j$). Take $L = 2\log(12n^2)$. A union bound over fewer than $n^2/2$ pairs gives an event $\mathcal{E}$ with probability at least $11/12$ on which $(u_i - u_j)^2 \le L\|x_i - x_j\|^2$ for every pair. The bound must hold simultaneously for all pairs because the matching is chosen after seeing u. On $\mathcal{E}$, the averaging and median bounds therefore imply

$$\Phi_{t-1} - \Phi_t \ge \frac{1}{2L} \sum_{\{i,j\}} W_t(i,j)(u_i - u_j)^2 \ge \frac{Z}{2L}.$$

The total squared score equals the potential in expectation: $\mathbb{E}u_i^2 = \|x_i\|^2$, so $\mathbb{E}Z = \Phi_{t-1}$. It remains to check that restricting to $\mathcal{E}$ retains a constant fraction of this expectation. The Gaussian fourth moment gives

$\mathbb{E}u_i^4 = 3\|x_i\|^4$; hence Cauchy–Schwarz, applied to each coordinate, yields

$$\begin{aligned}\mathbb{E}[Z\mathbf{1}_{\mathcal{E}^c}] &= \sum_i \mathbb{E}[u_i^2 \mathbf{1}_{\mathcal{E}^c}] \\ &\le \sum_i \sqrt{\mathbb{E}u_i^4 \Pr(\mathcal{E}^c)} \le \frac{1}{2}\sum_i \|x_i\|^2 = \frac{\Phi_{t-1}}{2}.\end{aligned}$$

Thus at least half the expected total squared score lies on $\mathcal{E}$. Since the potential never increases, even on $\mathcal{E}^c$, we conclude that

$$\mathbb{E}[\Phi_{t-1} - \Phi_t] \ge \frac{\mathbb{E}[Z\mathbf{1}_{\mathcal{E}}]}{2L} \ge \frac{\Phi_{t-1}}{4L}.$$

Each round therefore removes an $\Omega(1/\log n)$ fraction of the remaining potential in conditional expectation.

Removing the conditioning and iterating gives $\mathbb{E}\Phi_T \le (n-1)(1-1/(4L))^T \le (n-1)e^{-T/(4L)}$. For $T = \lceil 4L\log(4n^2(n-1)/\delta)\rceil$, this is at most $\delta/(4n^2)$. Markov's inequality now gives $\Phi_T \le 1/(4n^2)$ with probability at least $1-\delta$. On this event every vertex holds at least $1/(2n)$ of every commodity, giving the multicommodity flow certificate above whenever all T single-commodity flows succeed. □

10.5 Arboricity

The k-disjoint forest partition algorithm determines whether the edges of a weighted multigraph G can be partitioned into k edge-disjoint forests. By binary searching over k, this algorithm can compute the exact arboricity of a graph. It operates in $O(n(n+m)^2\log(kn))$ time using a series of max-flow computations.

Nash-Williams Theorem The Nash-Williams Theorem (refer to Diestel for a simple proof of the stronger Packing-Covering Theorem and the implication) states that a multigraph can be partitioned into k edge-disjoint forests if and only if for every non-empty subset of vertices $U \subseteq V$, the sum of the weights of the edges induced by U satisfies:

$$c(E(U)) \le k(|U|-1)$$

Network Construction For a chosen "root" vertex $r \in V$, we construct a bipartite flow network to test all subsets U containing r. The network consists of a source s, a sink t, a layer of vertex-nodes representing V, and a layer of edge-nodes representing E.

1. **Source to Vertices:** A directed edge from s to every vertex $v \in V \setminus \{r\}$ with capacity k.
2. **Vertices to Edges:** For every edge $e = (u, v)$ in the original graph, we add directed edges from node u to node e, and from node v to node e, both with capacity ∞.
3. **Edges to Sink:** A directed edge from every edge-node e to t with capacity $c(e)$ (the weight of the edge).

Consider an arbitrary finite-capacity $s-t$ cut, partitioned into sets S and T, let $U = V \cap T$.

Because the directed edges from vertices to edge-nodes have infinite capacity, placing an edge-node in T while one of its endpoints remains in S would cut infinite capacity. Therefore, the only edge-nodes in T are in $E(U)$ and to minimize the cut capacity for a given U, we let exactly $E(U)$ be on the sink side.

The cut edges come from:

1. Edges from s to the vertices in $U \setminus \{r\}$.
2. Edges from $E \setminus E(U)$ to the sink t.

And the total capacity is:

$$\text{Cut Capacity} = k|U \setminus \{r\}| + c(E) - c(E(U))$$

For the maximum flow to satisfy the total edge weight $c(E)$, every cut must have a capacity of at least $c(E)$. Thus:

$$k|U \setminus \{r\}| + c(E) - c(E(U)) \geq c(E) \implies c(E(U)) \leq k|U \setminus \{r\}| = k(|U| - 1)$$

Algorithm 59 k-Disjoint Forest Partition

Input: Graph V, edges E with weights c_e, integer k
Output: True if E can be partitioned into $\leq k$ forests, False otherwise

```
function IsKForestPartitionable(V, E, k)
    n ← |V|; m ← |E|
    C_total ← Σ_{e∈E} c_e
    if C_total > k(n − 1) then
        return False
    for r ← 0 to n − 1 do
        Initialize flow network G_flow with source s, sink t
        for v ∈ V do
            if v ≠ r then
                Add edge s → v with capacity k
        for each edge e = (u, v) ∈ E do
            Add edge u → e with capacity ∞
            Add edge v → e with capacity ∞
            Add edge e → t with capacity c_e
        flow ← DinitzScaling(G_flow, s, t)
        if flow < C_total then
            return False
    return True
```

Complexity

Theorem 334. *The algorithm correctly verifies the k-forest partition property in $O(n(n+m)^2 \log(kn))$ time.*

Proof. We use Dinitz's algorithm with capacity scaling, which finds a maximum flow in $O(VE \log U)$ time. For our network, the number of vertices is $V = n + m + 2$ and the number of edges is $E = n + 3m$. The maximum capacity connected to the source is $U \leq kn$. Thus we get a runtime of $O((n+m)^2 \log(kn))$. Because we execute this flow algorithm n times (once for each root r), the overall time complexity is $O(n(n+m)^2 \log(kn))$. □

10.6 Directed Minimum Spanning Tree

Let

$$G = (V, E)$$

be a directed graph with edge weights $w : E \to \mathbb{R}$, and fix a root $r \in V$.

An *r-arborescence* is a set of $n - 1$ edges such that every $v \neq r$ has exactly one incoming edge and every vertex is reachable from r.

We want a minimum-weight r-arborescence.

10.6.1 The Two Reductions

We first isolate the two transformations on which the algorithm is based.

Lemma 335 (Incoming-Edge Reduction)**.** *Let X be a non-root vertex of the current contracted graph, and let*

$$\delta = \min_{e \in \delta^-(X)} w(e).$$

Subtract δ from the weight of every edge entering X.

Then the minimum arborescence weight decreases by exactly δ.

Proof. Every rooted arborescence contains exactly one edge entering X. Thus every feasible solution loses exactly δ in weight. □

After the reduction, at least one incoming edge to X has weight zero.

Choose one such zero-weight edge for every non-root vertex. If the chosen edges are acyclic, they form an optimal arborescence.

Otherwise they contain a directed cycle.

Lemma 336 (Zero-Cycle Contraction)**.** *Suppose*

$$C = X_1 X_2 \cdots X_k X_1$$

is a directed cycle of chosen zero-weight edges. Contract C to one supervertex.

The minimum residual arborescence weight is unchanged.

Proof. Consider any arborescence of the original graph. At least one edge enters C from outside.

Suppose an entering edge enters X_i. Delete all other edges entering vertices of C and instead include every zero-weight cycle edge except the cycle edge entering X_i. This preserves connectivity and indegree one at every vertex of C and does not increase the weight.

Hence some optimum contains exactly one external edge entering C.

Such solutions correspond exactly to arborescences after contracting C: if the contracted solution enters the supervertex through an edge whose original head is X_i, expand C and add every zero cycle edge except the one entering X_i. The weights are identical. □

10.6.2 Meldable Incoming-Edge Heaps

For each current contracted component X, maintain a heap

$$H_X$$

containing the original edges whose heads lie in X.

An edge

$$e = (u, v)$$

stored in H_X may later become internal because its tail is also contracted into X. We do not eagerly delete such edges. Instead, whenever an internal edge reaches the minimum of the heap, we discard it.

Thus we need the following heap operations:

operation	meaning
$\mathsf{Min}(H)$	minimum reduced-cost edge
$\mathsf{Pop}(H)$	delete the minimum
$\mathsf{AddAll}(H, -\delta)$	subtract δ from every key
$\mathsf{Meld}(H_1, H_2)$	merge two incoming-edge heaps.

A skew heap with one lazy additive tag per heap node supports these operations in amortized $O(\log m)$ time, with AddAll taking $O(1)$ time.

When the minimum incoming edge of X has reduced weight δ, we perform

$$\mathsf{AddAll}(H_X, -\delta).$$

When a cycle is contracted,

$$H_C = \mathsf{Meld}\left(H_{X_1}, \ldots, H_{X_k}\right).$$

Edges that have become internal are discarded lazily from the top.

10.6.3 Detecting Cycles with DSU

Maintain the already chosen zero edges as an undirected forest in a disjoint set union structure D_{cyc}.

The important invariant is that before a current component X chooses its incoming edge, X is the unique vertex of its chosen-edge component with no chosen incoming edge. Thus the chosen edges in that component form an arborescence directed away from X.

Suppose the chosen minimum edge is

$$e = (Y, X).$$

If X and Y lie in different components of D_{cyc}, adding e cannot create a cycle, so we simply union them.

If they already lie in the same component, the unique chosen-edge path from X to Y is directed

$$X \rightsquigarrow Y.$$

Adding

$$Y \to X$$

therefore creates exactly one directed cycle.

Hence an ordinary DSU detects the cycles.

10.6.4 Maintaining Contractions

A second DSU,

$$D_{\text{ctr}},$$

represents the currently contracted vertices.

For every original vertex v,

$$D_{\text{ctr}}.\mathsf{find}(v)$$

is its present supervertex.

When a chosen edge

$$e = (u, v)$$

is inspected, its current tail is therefore

$$D_{\text{ctr}}.\mathsf{find}(u).$$

When a cycle is found, union all its vertices in D_{ctr} and meld their heaps.

We make D_{ctr} rollback-capable. Before contracting a cycle, record the current rollback time. This lets us expand all contractions in reverse order once the optimum weight has been found.

10.6.5 Fast Directed MST Algorithm

For a current component X, let

$$e_X$$

denote the zero reduced-cost incoming edge currently chosen for X.

The algorithm processes a component as follows.

Algorithm 60 Tarjan's Directed MST Algorithm

Input: Directed graph $G = (V, E)$ and root r
Output: Minimum r-arborescence, or None

```
put every edge (u, v) into the meldable heap H_v
initialize D_cyc
initialize rollback DSU D_ctr
answer ← 0
C ← ()                                              ▷ contraction records
for each original vertex s ≠ r do
    X ← s
    loop
        while H_X ≠ ∅ and tail(Min(H_X)) is currently inside X do
            Pop(H_X)
        if H_X = ∅ then
            return None
        e_X ← Min(H_X)
        δ ← w_red(e_X)
        answer ← answer + δ
        AddAll(H_X, −δ)
        Y ← D_ctr.find(tail(e_X))
        if D_cyc.union(X, Y) succeeds then
            break
        record rollback time τ of D_ctr
        find the directed chosen-edge cycle containing X
        contract all its components into a new component Z
        meld all their incoming-edge heaps into H_Z
        append the contraction information and τ to C
        X ← Z
expand the contractions in reverse order
return answer and the resulting parent edges
```

10.6.6 Correctness

Theorem 337. *Algorithm 60 returns a minimum-weight rooted arborescence whenever one exists.*

Proof. Maintain the invariant

$$\boxed{\mathrm{OPT}(G_{\text{original}}) = \mathit{answer} + \mathrm{OPT}(G_{\text{current}}).}$$

Initially this is immediate.

Suppose we process a current non-root component X, whose cheapest external incoming edge has reduced weight δ. Subtracting δ from every edge entering X decreases every feasible arborescence by exactly δ by Lemma 335. Adding δ to *answer* therefore preserves the invariant.

The chosen edge now has reduced weight zero.

If no cycle is created, no contraction is required.

If a zero-weight directed cycle is created, Lemma 336 shows that contracting it leaves the residual optimum unchanged. Thus the invariant again survives.

Eventually all current non-root components have chosen incoming edges and no chosen cycle remains. Those edges form an arborescence of residual cost zero. Therefore

$$\mathrm{OPT}(G_{\text{current}}) = 0,$$

and

$$\mathit{answer} = \mathrm{OPT}(G_{\text{original}}).$$

It remains to recover the actual edges. Expand contractions in reverse order. Suppose a contracted cycle is entered by the already reconstructed edge f, whose original head lies in cycle component X_i. Restore every selected zero cycle edge except the one entering X_i, and use f instead. By the proof of Lemma 336, this preserves an arborescence and its weight.

Repeating through all contractions recovers an optimum arborescence of the original graph. □

10.6.7 Rollback Reconstruction

When contracting a cycle, save its DSU snapshot τ.

During reverse processing, the cycle is still one supervertex. Its currently selected incoming edge tells us which original member of the cycle receives the external edge.

Roll back to τ. The cycle members are now distinct again. If the external edge enters X_i, restore the old cycle edge for every other member and replace precisely the old edge entering X_i.

Thus every contraction requires only its rollback snapshot and the chosen edges already stored by the algorithm; the contracted graph never needs to be explicitly rebuilt.

10.6.8 Running Time

Theorem 338 (Tarjan)**.** *Using lazy skew heaps and disjoint-set union, Algorithm 60 runs in*

$$\boxed{O(m \log n)}$$

time and $O(m+n)$ space.

Proof. Each original edge is inserted into exactly one heap.

Once an edge becomes internal to a contracted component, it remains internal forever. Thus every edge is popped at most once.

A contraction permanently reduces the number of current components, so there are fewer than n contractions and only $O(n)$ successful contraction-DSU unions.

Every heap is formed only by melding heaps whose components are contracted. Hence there are $O(m+n)$ heap operations in total.

Lazy subtraction costs $O(1)$. Meld and pop in a skew heap cost $O(\log m) = O(\log n)$ amortized on the relevant sparse representation, and the DSU work is also at most this bound.

Therefore the total running time is

$$O(m \log n).$$

The reverse rollback pass performs only $O(n)$ unions/rollbacks and edge changes. □

After reconstruction, if $e_v = (u, v)$ is the selected incoming edge of $v \neq r$, one may return simply

$$\mathrm{parent}[v] = u, \qquad \mathrm{parent}[r] = r.$$

10.7 Matching and Coloring

10.7.1 Bipartite Regular Matching

Given a d-regular bipartite graph $G = (P, Q, E)$ with $|P| = |Q| = n$, a perfect matching can be found in $O(n \log n)$ expected time.

The algorithm iteratively augments a partial matching M by discovering augmenting paths via random walks. If M currently leaves k nodes unmatched in both P and Q, we implicitly construct a directed matching graph H:

1. Orient all edges of G from P to Q.
2. Contract each matched edge pair $(u, v) \in M$ into a single supernode.
3. Add a source vertex s connected by d parallel edges to each unmatched node in P, directed outward.
4. Add a sink vertex t connected by d parallel edges from each unmatched node in Q, directed inward.

Any $s \to t$ path in H corresponds directly to a valid augmenting path in G with respect to M. The algorithm simply performs a standard random walk starting from s, choosing an outgoing edge uniformly at random at each step, until it hits t.

Lemma 339 (Random Walk Hitting Time)**.** *Given a partial matching leaving k vertices unmatched, the expected number of steps before a random walk started at s reaches t is at most $2 + \frac{n}{k}$.*

Proof. Construct a modified graph H^* by identifying the source s and sink t into a single unified node s^*. In H^*, the out-degree of every vertex equals its in-degree, making it an Eulerian digraph.

The out-degree of the unified node s^* is kd, while the out-degree of every other node is $d-1$. The expected time for a random walk in H^* starting at s^* to return to s^* is exactly the inverse of its stationary probability π_{s^*}. Because H^* is balanced, this stationary probability is simply the ratio of the degree of s^* to the sum of all degrees in H^*. Evaluating this yields an expected return time of:

$$\frac{1}{\pi_{s^*}} = \frac{n(d-1) + 2kd + kd}{kd} \leq 2 + \frac{n}{k}$$

Running a walk from s^* until it returns to s^* in H^* is identical to running a walk from s until it hits t in the original matching graph H. □

To construct a perfect matching, the algorithm starts with an empty matching ($k = n$) and repeatedly runs untruncated random walks to augment the matching n times until $k = 0$. By the lemma, the total expected number of steps taken across all random walks is bounded by:

$$\sum_{k=1}^{n} \left(2 + \frac{n}{k}\right) = 2n + nH_n = O(n \log n)$$

The same runtime with high probability can easily be acheived by terminating the walks if they take too long and repeating. We then use Markov's inequality and a Chernoff bound.

10.7.2 Kasteleyn Signing and Planar Perfect Matchings

For a general graph, the determinant or Pfaffian expansion contains one term for every perfect matching, but the terms have different signs. For planar graphs, Kasteleyn's observation is that the edges can be signed so that all perfect matchings receive the same sign. Counting perfect matchings then reduces to one Pfaffian, or one determinant in the bipartite case.

We work with a fixed plane embedding. For an oriented edge on the boundary of a face, say that it is *clockwise* if its direction agrees with a clockwise traversal of that face. Boundary walks may repeat edges; occurrences are counted separately.

Definition 340 (Kasteleyn orientation)**.** An orientation of a connected plane graph is *Kasteleyn* if every bounded face has an odd number of clockwise-oriented boundary-edge occurrences.

The outer face is deliberately left unconstrained.

Lemma 341 (Existence of a Kasteleyn orientation)**.** *Every connected plane graph has a Kasteleyn orientation. Given the embedding, one can construct one in $O(|E|)$ time.*

Proof. Take a spanning tree of the dual graph, rooted at the outer face. Initially orient every primal edge whose dual edge is not in this tree arbitrarily.

Process the remaining dual-tree edges from the leaves toward the root. When a bounded face F is processed, exactly one still-unoriented primal edge on its boundary is the edge dual to its parent edge in the dual tree. Orient this edge in whichever direction makes the number of clockwise edges around F odd.

Once F is processed, its parity will never change. Every bounded face is eventually processed, while the outer face is the root and requires no condition. Each edge is handled only constantly many times. □

The face condition is useful because it implies the correct parity not only for faces, but for every even cycle that can occur as the symmetric difference of two perfect matchings.

Lemma 342 (Parity inside a cycle)**.** *Let C be a simple even cycle in a Kasteleyn-oriented plane graph, and let v_{in} be the number of vertices strictly inside C. If $c(C)$ is the number of edges oriented clockwise around C, then*

$$\boxed{c(C) \equiv v_{\text{in}} + 1 \pmod 2.}$$

Proof. Let H consist of C together with everything drawn inside C. Since the original graph is connected, H is connected. Let E_{in} be the number of edges of H not on C, and let F be the number of faces of H inside C.

Sum the number of clockwise-oriented edges over these F faces. Every interior edge occurs twice, once in each direction, so exactly one of its two boundary occurrences agrees with its orientation. Every edge of C occurs once and contributes exactly when it is clockwise around C. Hence

$$F \equiv E_{\text{in}} + c(C) \pmod 2,$$

because every bounded face has odd clockwise parity.

Euler's formula for H, with F bounded faces and one outer face, gives $V - E + F = 1$. Since $V = v_{\text{in}} + |C|$ and $E = E_{\text{in}} + |C|$,

$$F \equiv 1 + v_{\text{in}} + E_{\text{in}} \pmod 2.$$

Combining the two congruences gives $c(C) \equiv v_{\text{in}} + 1 \pmod 2$. □

Call an even cycle C *nice* if deleting its vertices leaves a graph with a perfect matching. For such a cycle, the number of vertices strictly inside C is even: after deleting C, no edge can join an inside vertex to an outside vertex without crossing C, so the remaining perfect matching pairs all inside vertices among themselves. Thus Lemma 342 immediately gives:

Corollary 343 (Nice cycles are oddly oriented)**.** *Every nice cycle in a Kasteleyn orientation has an odd number of edges oriented in either direction around the cycle.*

We now connect this parity to Pfaffian signs. For an oriented weighted graph, define the skew-symmetric matrix K by

$$K_{uv} = \begin{cases} w_{uv}, & u \to v, \\ -w_{uv}, & v \to u, \\ 0, & uv \notin E. \end{cases}$$

Thus $K_{vu} = -K_{uv}$. Parallel edges, if present, may simply be summed in the corresponding matrix entry.

Recall that the Pfaffian expansion of K has one term for each perfect matching:

$$\mathrm{Pf}(K) = \sum_{M} \mathrm{sgn}_K(M) \prod_{e \in M} w_e,$$

where $\mathrm{sgn}_K(M) \in \{\pm 1\}$ combines the usual Pfaffian pairing sign with the orientation signs of its edges.

Lemma 344 (Flipping an alternating cycle)**.** *Let two perfect matchings agree except on one even cycle C. Their Pfaffian terms have the same sign if C is oddly oriented.*

Proof. Relative signs are unchanged by simultaneously relabelling the vertices, so write

$$C = (1, 2, \ldots, 2r, 1)$$

in cyclic order and place all other vertices afterward.

Write $\varepsilon_{ij} = 1$ when the edge is oriented $i \to j$ and $\varepsilon_{ij} = -1$ otherwise. The two alternating matchings on C are

$$M_0 = \{12, 34, \ldots, (2r-1)(2r)\}$$

and

$$M_1 = \{23, 45, \ldots, (2r)1\}.$$

In this ordering the ordinary Pfaffian pairing sign of each displayed pairing is positive. Their orientation-sign ratio is therefore

$$\begin{aligned} \frac{\mathrm{sgn}_K(M_1)}{\mathrm{sgn}_K(M_0)} &= \frac{\varepsilon_{1,2r}\varepsilon_{23}\varepsilon_{45}\cdots\varepsilon_{2r-2,2r-1}}{\varepsilon_{12}\varepsilon_{34}\cdots\varepsilon_{2r-1,2r}} \\ &= -\prod_{i=1}^{2r} \varepsilon_{i,i+1}, \end{aligned}$$

where $2r + 1 = 1$.

If C is oddly oriented, then an odd number of its edges disagree with either traversal direction, so $\prod_i \varepsilon_{i,i+1} = -1$. The ratio is therefore 1. □

Theorem 345 (Kasteleyn–Pfaffian formula)**.** *Let G be a weighted planar graph with a Kasteleyn orientation, and let K be its signed skew adjacency matrix. Then*

$$\boxed{|\mathrm{Pf}(K)| = \sum_{M \ \mathit{perfect}} \prod_{e \in M} w_e.}$$

In particular, for an unweighted planar graph,

$$\boxed{\#\{\mathit{perfect\ matchings}\} = |\,\mathrm{Pf}(K)| = \sqrt{\det K}.}$$

Proof. Fix two perfect matchings M, M'. Their symmetric difference is a disjoint union of alternating even cycles

$$M \triangle M' = C_1 \cup \cdots \cup C_s.$$

Each C_i is nice: the edges of M outside C_i form a perfect matching after the vertices of C_i are removed.

Hence every C_i is oddly oriented by Corollary 343. Starting from M, flip the alternating cycles one at a time. By Lemma 344, none of these flips changes the Pfaffian sign. Thus every perfect matching occurs in $\mathrm{Pf}(K)$ with the same sign.

Taking the absolute value therefore removes the common sign and leaves exactly the weighted perfect-matching sum. Finally $\det K = \mathrm{Pf}(K)^2$. □

Thus, once a planar embedding is available, the signing is constructed in linear time and we can take the square root of the determinant in $O(n^3)$ arithmetic operations to recover the Pfaffian up to sign.

Bipartite graphs If $G = (L \cup R, E)$ is bipartite with $|L| = |R| = n$, order all vertices of L before those of R. Then the Kasteleyn matrix has block form

$$K = \begin{pmatrix} 0 & B \\ -B^{\mathsf{T}} & 0 \end{pmatrix},$$

where B is a signed weighted bipartite adjacency matrix. The standard Pfaffian block identity gives

$$\mathrm{Pf}(K) = (-1)^{n(n-1)/2} \det B.$$

Hence

$$\boxed{\sum_{M \text{ perfect}} \prod_{e \in M} w_e = |\det B|.}$$

This is the determinant form of Kasteleyn signing: planarity lets one choose signs on the bipartite adjacency matrix so that the determinant has no cancellation between perfect matchings.

Domino tilings of a rectangle. The Kasteleyn matrix of the rectangular grid can be diagonalized almost by inspection. This gives the classical closed form for the number $T_{m,n}$ of domino tilings of an $m \times n$ rectangle.

Regard the mn squares as the vertices of the grid graph $P_m \square P_n$, with its checkerboard bipartition. Give every horizontal edge weight 1 and every vertical edge weight i, and let B be the resulting black-to-white matrix. This is a Kasteleyn matrix up to harmless unit complex scalings. Indeed, if a square lies in row r, multiply its black row of B by i^{-r} and its white column by i^r. Horizontal entries remain 1, while the two vertical edges of every elementary face become 1 and -1. Thus every bounded square has the required odd sign, and the preceding Kasteleyn theorem gives

$$T_{m,n} = |\det B|.$$

It is convenient to recover B as one block of a matrix on all mn squares. Let A_r denote the adjacency matrix of the path P_r, and put

$$K = A_m \otimes I_n + i I_m \otimes A_n.$$

If the vertices are reordered with all black vertices first, then

$$K = \begin{pmatrix} 0 & B \\ B^{\mathsf{T}} & 0 \end{pmatrix}.$$

When mn is even, the two parts have size $mn/2$, so

$$|\det K| = |\det B|^2 = T_{m,n}^2. \tag{1}$$

The path matrix A_r has an especially simple eigenbasis. It's clear to see that for the infinite path the natural eigenfunctions are $e^{i\theta x}$, with eigenvalues of $2\cos\theta$. For a finite path pretend we have two ghost vertices where the eigenvector must vanish, $f(0) = f(r+1) = 0$. We can cancel out waves of opposite frequencies since they agree at 0: $e^{i\theta x} + e^{-i\theta x} = 2i\sin(\theta x)$. Thus we need $\sin((r+1)\theta) = 0$. For $1 \le j \le r$, set

$$v_j(a) = \sin\left(\frac{\pi j a}{r+1}\right), \qquad 1 \le a \le r.$$

Writing $\theta_j = \pi j/(r+1)$ and using

$$\sin((a-1)\theta_j) + \sin((a+1)\theta_j) = 2\cos\theta_j \sin(a\theta_j),$$

together with $v_j(0) = v_j(r+1) = 0$, gives

$$A_r v_j = 2\cos\left(\frac{\pi j}{r+1}\right) v_j.$$

The r eigenvalues are distinct, so these vectors form a basis.

Consequently $v_j \otimes v_k$ is an eigenvector of K with eigenvalue

$$2\cos\left(\frac{\pi j}{m+1}\right) + 2i\cos\left(\frac{\pi k}{n+1}\right).$$

The mn tensor products form a basis, and hence

$$\begin{aligned}|\det K| &= \prod_{j=1}^{m}\prod_{k=1}^{n}\left|2\cos\left(\frac{\pi j}{m+1}\right) + 2i\cos\left(\frac{\pi k}{n+1}\right)\right| \\ &= \prod_{j=1}^{m}\prod_{k=1}^{n}\left(4\cos^2\left(\frac{\pi j}{m+1}\right) + 4\cos^2\left(\frac{\pi k}{n+1}\right)\right)^{1/2}.\end{aligned}$$

Combining this with (1) yields the classical formula

$$\boxed{T_{m,n} = \prod_{j=1}^{m}\prod_{k=1}^{n}\left(4\cos^2\frac{\pi j}{m+1} + 4\cos^2\frac{\pi k}{n+1}\right)^{1/4}.}$$

If mn is odd, there is no perfect matching and $T_{m,n} = 0$; indeed the product above contains the zero factor corresponding to $j = (m+1)/2$ and $k = (n+1)/2$.

When both m and n are even, pairing j with $m+1-j$ and k with $n+1-k$ gives the form

$$\boxed{T_{m,n} = \prod_{j=1}^{m/2}\prod_{k=1}^{n/2}\left(4\cos^2\frac{\pi j}{m+1} + 4\cos^2\frac{\pi k}{n+1}\right).}$$

10.7.3 Bipartite Edge Coloring

Let

$$G = (L \sqcup R, E)$$

be a bipartite multigraph with maximum degree

$$\Delta.$$

König's line-coloring theorem states that

$$\chi'(G) = \Delta.$$

We give an algorithmic proof which constructs a proper Δ-edge-coloring in

$$O(m \log m)$$

time.

The algorithm has three ingredients:

1. reduce to a regular bipartite multigraph of only $O(m)$ edges;
2. split any even-regular multigraph into two equal-degree regular subgraphs using Euler tours;
3. use the splitting operation to find a perfect matching, and then recursively color the graph.

Regularization The first reduction is useful because a proper edge-coloring of a graph obtained by identifying vertices on the same side remains proper when those vertices are separated again.

Lemma 346 (Regularization)**.** *Let G be a nonempty bipartite multigraph with m edges and maximum degree Δ. In $O(m)$ time we can construct a Δ-regular bipartite multigraph G' with*

$$O(m)$$

edges such that any proper Δ-edge-coloring of G' restricts to a proper Δ-edge-coloring of G.

Proof. Consider one side of the bipartition. Greedily combine original vertices into groups, keeping the sum of degrees in each group at most Δ. When the next vertex would make the current group exceed Δ, start a new group.

Let the resulting group loads be

$$d_1, \dots, d_q \le \Delta.$$

Whenever group $i+1$ was started,

$$d_i + d_{i+1} > \Delta,$$

because the first vertex placed in group $i+1$ already failed to fit in group i. Hence, pairing consecutive groups,

$$q = O\left(\frac{m}{\Delta} + 1\right).$$

Do this independently on both sides.

Identify all vertices belonging to one group. Every resulting vertex still has degree at most Δ. A coloring of the identified graph is also valid before identification, because identifying vertices only introduces additional constraints.

Add isolated dummy vertices to the smaller side until both sides contain the same number N of vertices. Since the degree sum on each side is m, both sides have the same total deficit

$$N\Delta - m.$$

Repeatedly join a deficient left vertex to a deficient right vertex until all degrees equal Δ.

The final graph has

$$N\Delta = O(m + \Delta) = O(m)$$

edges, since $\Delta \le m$. Deleting the dummy edges and undoing the identifications gives a coloring of the original graph. □

Euler Splitting The fundamental operation is especially simple.

Lemma 347 (Euler Split)**.** *Let H be a $2d$-regular multigraph. Then its edges can be partitioned into two d-regular spanning subgraphs*

$$H_0, H_1$$

in linear time.

Proof. Every connected component of H is Eulerian. Take an Euler tour of each component and alternately color its edges red and blue.

At every vertex, every entrance in the Euler tour is paired with the following exit, and the two edges receive opposite colors. Thus exactly half of the $2d$ incident edges are red and half are blue. □

The same operation works efficiently when an edge e is stored with a multiplicity $m(e)$ rather than explicitly storing all copies.

Lemma 348 (Euler Split with Multiplicities)**.** *Suppose a $2d$-regular multigraph is represented by multiplicities*

$$m(e)$$

on a set F of distinct edges. It can be split into two d-regular spanning multigraphs in

$$O(|F|)$$

time.

Proof. For every edge e, place

$$\left\lfloor \frac{m(e)}{2} \right\rfloor$$

copies in each side.

One copy remains precisely when $m(e)$ is odd. At every vertex, the number of remaining copies is even, because the original degree is even and we removed an even number of incident copies. Hence the graph consisting of the remaining copies is Eulerian.

Take Euler tours of this residual graph and alternate its edges between the two sides. Each side therefore receives exactly half of every vertex's residual degree, and consequently has degree d. Only the distinct edges need to be inspected. □

Lemma 349 (Perfect Matching in a Regular Bipartite Multigraph)**.** *Let G be a k-regular bipartite multigraph with N vertices on each side and*

$$m = Nk$$

edges. A perfect matching can be found in

$$O(m \log m)$$

time.

Proof. Let

$$Q = 2^t$$

be the smallest power of two satisfying

$$Q \geq m.$$

Write

$$Q = \alpha k + \beta, \qquad 0 \leq \beta < k.$$

Choose an arbitrary bijection between the two vertex classes, and let M_0 be the corresponding perfect matching in the complete bipartite graph. The edges of M_0 need not belong to G.

Construct a multigraph H by

- replacing every edge of G by α parallel copies;
- adding β copies of every edge of M_0.

Call the latter copies *bad*. Every vertex now has degree

$$\alpha k + \beta = Q,$$

so H is Q-regular.

The initial number of bad edges is

$$N\beta < Nk = m \leq Q.$$

Since Q is a power of two, repeatedly apply Lemma 348. At each split, retain the half containing fewer bad edges. Thus after j splits, the retained graph is

$$Q/2^j$$

regular and contains fewer than

$$Q/2^j$$

bad edges.

After t splits it is 1-regular and contains fewer than one bad edge. Hence it contains no bad edge at all.

A 1-regular spanning bipartite graph is a perfect matching, and all of its edges are copies of edges of G. Discarding multiplicities therefore gives a perfect matching of G.

At every level there are only

$$O(m)$$

distinct edges to process, and

$$t = O(\log m).$$

□

It is tempting to take Q to be merely the smallest power of two with $Q \geq k$, pad G to a Q-regular graph, and then repeatedly halve. The problem is that the padding introduces bad edges at *every* vertex.

Indeed, writing $Q = \alpha k + \beta$, the construction adds β bad matching edges at each of the N vertices on one side, for a total of $N\beta$ bad edges. At each Euler split we can keep the half containing fewer bad edges, so after j splits we only know that the number of bad edges has fallen by a factor 2^j. When the degree has finally fallen from Q to 1, this gives only

$$\#\text{bad edges} \leq \frac{N\beta}{Q}.$$

If Q is only $O(k)$, this can still be $\Theta(N)$.

Coloring a Power-of-Two Regular Graph

Lemma 350. *If G is 2^t-regular, then it can be properly edge-colored with exactly 2^t colors in*

$$O(m \log 2^t)$$

time.

Proof. Repeatedly Euler-split every current regular graph into two equal-degree subgraphs. After t levels, every leaf is 1-regular, hence a matching. Give each leaf a different color.

At each level the subgraphs partition the original edge set, so the total work per level is $O(m)$. □

The General Recursion We can now color a k-regular graph using exactly k colors.

Suppose first that k is even, and put

$$d = k/2.$$

Euler-split

$$G = G_1 \sqcup G_2,$$

where both graphs are d-regular.

Recursively color G_1 with colors

$$1, \ldots, d.$$

Each color class is a perfect matching.

Let

$$q$$

be the smallest power of two satisfying

$$q \geq d.$$

Since $q < 2d = k$ unless $d = q$, we need exactly

$$q - d$$

additional perfect matchings to make G_2 q-regular.

Take any $q - d$ color classes from G_1 and temporarily move them into G_2. The resulting graph

$$H$$

is q-regular, so Lemma 350 colors it with q colors.

The part of G_1 that was not moved contains

$$d - (q - d) = 2d - q = k - q$$

of its old color classes. Give these $k - q$ matchings colors disjoint from those used on H.

Altogether we use exactly

$$q + (k - q) = k$$

colors.

If k is odd, first find a perfect matching using Lemma 349, give it one color, remove it, and apply the even-degree procedure to the remaining $(k - 1)$-regular graph.

Algorithm 61 Edge Coloring a Regular Bipartite Multigraph

```
function ColorRegular(G, k)
    if k = 0 then
        return
    if k = 1 then
        give all edges of G one new color
        return
    if k is odd then
        M ← PerfectMatching(G)
        give M one new color
        G ← G − M
        k ← k − 1
    d ← k/2
    (G1, G2) ← EulerSplit(G)
    ColorRegular(G1, d)
    q ← least power of two with q ≥ d
    move any q − d color classes of G1 into G2
    H ← the resulting q-regular graph
    color H with q fresh colors by repeated Euler splitting
    retain the remaining k − q old color classes of G1
```

Theorem 351 (König's Line-Coloring Theorem, Algorithmic Form)**.** *Every bipartite multigraph of maximum degree Δ has a proper Δ-edge-coloring. Such a coloring can be found in*

$$\boxed{O(m \log m)}$$

time.

Proof. Apply Lemma 346, and let $M = O(m)$ be the number of edges after regularization.

Correctness follows from the construction above.

For the running time, consider a recursive call on a k-regular graph with M' edges. Everything outside the recursive call costs

$$O(M' \log M) :$$

Euler splitting is linear, power-of-two coloring costs $O(M' \log k)$, and when k is odd the perfect matching costs $O(M' \log M)$.

The recursive call is made on a graph with half as many edges. Thus

$$T(M') \leq T(M'/2) + O(M' \log M),$$

and therefore

$$T(M) = O(M \log M) = O(m \log m).$$

Finally discard dummy edges and undo the identified vertices. □

10.7.4 Chromatic Number

Let

$$G = (V, E), \qquad |V| = n.$$

For

$$S \subseteq V,$$

let

$$I(S) = |\{T \subseteq S : T \text{ is independent}\}|.$$

The key inclusion–exclusion identity is extremely simple.

Theorem 352 (Coloring by Independent-Set Counts)**.** *For $k \geq 1$, define*

$$\boxed{C_k = \sum_{S \subseteq V} (-1)^{n-|S|} I(S)^k.}$$

Then

$$\boxed{G \text{ is } k\text{-colorable} \iff C_k > 0.}$$

Proof. The quantity

$$I(S)^k$$

counts ordered tuples

$$(A_1, \ldots, A_k)$$

of independent subsets of S.

Consider one fixed tuple of independent subsets of V. Its contribution to C_k is

$$\sum_{\substack{S \subseteq V \\ A_1 \cup \cdots \cup A_k \subseteq S}} (-1)^{n-|S|}.$$

This equals

$$\begin{cases} 1, & A_1 \cup \cdots \cup A_k = V, \\ 0, & \text{otherwise}, \end{cases}$$

by inclusion–exclusion.

Hence C_k counts ordered k-tuples of independent sets whose union is all of V.

Such a tuple exists exactly when G is k-colorable: given a coloring, take its color classes; conversely, from a covering tuple, assign each vertex to any independent set containing it. □

Define an array

$$e(S)$$

initially by

$$e(\{u,v\}) = 1 \qquad \text{for every edge } \{u,v\} \in E,$$

and 0 elsewhere.

Apply the subset zeta transform:

$$E(S) = \sum_{T \subseteq S} e(T).$$

Then

$$E(S)$$

is exactly the number of graph edges contained in S.

Thus

$$a(S) = \begin{cases} 1, & E(S) = 0, \\ 0, & E(S) > 0 \end{cases}$$

is the indicator that S is independent.

Apply the subset zeta transform a second time:

$$\boxed{I(S) = \sum_{T \subseteq S} a(T).}$$

This computes every independent-subset count in

$$O(n2^n)$$

arithmetic operations.

Algorithm 62 Exact Chromatic Number

```
initialize e(S) ← 0 for all S ⊆ V
for each edge {u, v} ∈ E do
    e({u, v}) ← 1
E ← ζ↓e
for S ⊆ V do
    a(S) ← 1[E(S) = 0]
I ← ζ↓a
P(S) ← 1 for all S
for k = 1, ..., n do
    for S ⊆ V do
        P(S) ← P(S)I(S)
    C_k ← Σ_{S⊆V} (−1)^{n−|S|} P(S)
    if C_k ≠ 0 then
        return k
```

Theorem 353. *The chromatic number of an n-vertex graph can be computed in*

$$\boxed{O(n2^n)}$$

arithmetic operations and

$$\boxed{O(2^n)}$$

space.

10.8 Node-Weighted Triangle

Let $G = (A \sqcup B \sqcup C, E)$ be tripartite, with a weight

$$w(v)$$

on every vertex. The *Node-Weighted Triangle* problem asks whether there is a triangle

$$a \in A, \qquad b \in B, \qquad c \in C$$

such that

$$\boxed{w(a) + w(b) + w(c) = 0.}$$

There is no loss in assuming the graph is tripartite. Given an arbitrary graph, make three copies of its vertex set and put an edge between two copies exactly when the corresponding original vertices are adjacent. Triangles are preserved up to a constant multiplicity.

The main point is that unequal multiplicities of the vertex weights do not make the problem harder than ordinary triangle detection.

10.8.1 One Fixed Weight

The basic case is immediate.

Lemma 354 (One Fixed Weight)**.** *Suppose every vertex of A has the same weight u. Then Node-Weighted Triangle can be solved in*

$$\boxed{O(\mathrm{MM}(|A|, |B|, |C|))}$$

time, up to linear input-processing time.

Proof. For every edge $bc \in E(B, C)$, keep it only if

$$u + w(b) + w(c) = 0.$$

Then a triangle

$$a, b, c$$

in the remaining graph satisfies

$$w(a) + w(b) + w(c) = u + w(b) + w(c) = 0.$$

Conversely every zero-weight triangle survives.

Thus the problem has become ordinary tripartite triangle detection, which is one rectangular matrix multiplication. □

So the whole problem is to partition the vertices into instances in which one side has a fixed weight, without paying too much for the partition.

10.8.2 The Square Case

Assume

$$|A|, |B|, |C| \le n.$$

For a weight u, write

$$f_A(u) = |\{a \in A : w(a) = u\}|,$$

and define f_B similarly.

We first process triangles for which the A-weight is at least as frequent as the B-weight.

Fix an A-weight u, and put

$$k = f_A(u).$$

Only consider B-weights v satisfying

$$f_B(v) \le k.$$

Greedily partition these weights into groups P such that

$$\sum_{v \in P} f_B(v) \le 2k,$$

and every group except possibly the last has total frequency at least k. This is possible because every individual weight has frequency at most k.

For a group P, define

$$A_u = \{a \in A : w(a) = u\},$$

$$B_P = \{b \in B : w(b) \in P\},$$

and

$$C_P = \{c \in C : -u - w(c) \in P\}.$$

Every zero-weight triangle whose A-vertex has weight u and whose B-weight lies in P is contained in

$$A_u \times B_P \times C_P.$$

Moreover,

$$|A_u| = k, \qquad |B_P| \le 2k.$$

Split C_P into blocks of at most $2k$ vertices and apply Lemma 354 to every block.

Theorem 355 (Node-Weighted Triangle)**.** *Node-Weighted Triangle on a tripartite graph with $O(n)$ vertices in each part can be solved in*

$$\boxed{O(n^\omega))}$$

time, the same time as ordinary triangle detection.

Proof. Fix an A-weight u of frequency

$$k = f_A(u).$$

Because every completed B-group contains at least k vertices, there are

$$O(n/k)$$

groups.

Also, every $c \in C$ belongs to at most one set C_P, because its required complementary B-weight

$$-u - w(c)$$

belongs to at most one group. Hence

$$\sum_P |C_P| \le n.$$

After splitting the C_P's into $O(k)$-sized blocks, the total number of blocks is therefore

$$O(n/k).$$

Each block has dimensions

$$O(k) \times O(k) \times O(k),$$

so it costs

$$O(k^\omega)$$

to process. Thus the total cost for the fixed weight u is

$$\boxed{O(n\,k^{\omega-1}).}$$

Now sum over all distinct weights u in A. Since

$$\sum_u f_A(u) = |A| \le n$$

and $\omega \ge 2$,

$$k^{\omega-1} \le n^{\omega-2}k.$$

Therefore

$$\begin{aligned}\sum_u n\, f_A(u)^{\omega-1} &\le n^1 n^{\omega-2} \sum_u f_A(u) \\ &= n^\omega.\end{aligned}$$

This detects every zero-weight triangle satisfying

$$f_A(w(a)) \ge f_B(w(b)).$$

Run the same construction with A and B interchanged. Every triangle satisfies one of

$$f_A(w(a)) \ge f_B(w(b)), \qquad f_B(w(b)) > f_A(w(a)),$$

so every zero-weight triangle is detected.

The second orientation has the same running time. □

The balancing principle is simple. A weight occurring k times gives a matrix dimension of order k, but there can be only $O(n/k)$ relevant blocks. Since matrix multiplication grows at least quadratically with the dimension, these effects balance.

10.8.3 Rectangular Version

Suppose the three parts have sizes

$$|A| \le n^a, \qquad |B| \le n^b, \qquad |C| \le n^c,$$

and let

$$\omega = \omega(a, b, c).$$

For an A-weight u, write

$$f_A(u) = |\{x \in A : w(x) = u\}|,$$

and similarly define $f_B(v)$.

Since A and B have different sizes, absolute frequencies should not be compared directly. Define their normalized frequencies by

$$\boxed{\rho_A(u) = f_A(u)^{1/a}, \qquad \rho_B(v) = f_B(v)^{1/b}.}$$

Thus

$$f_A(u) = \rho_A(u)^a, \qquad f_B(v) = \rho_B(v)^b.$$

The point is that a normalized frequency ρ naturally gives matrix dimensions

$$\rho^a, \qquad \rho^b, \qquad \rho^c,$$

whose multiplication cost is

$$O(\rho^\omega).$$

Theorem 356 (Rectangular Node-Weighted Triangle)**.** *Node-Weighted Triangle on a tripartite graph with part sizes*

$$n^a, \qquad n^b, \qquad n^c$$

can be solved in

$$\boxed{O(n^{\omega(a,b,c)})}$$

time.

Proof. We first find every zero-weight triangle satisfying

$$\rho_A(w(x)) \geq \rho_B(w(y)).$$

Fix an A-weight u, and set

$$k = f_A(u), \qquad \rho = k^{1/a}.$$

Thus

$$k = \rho^a.$$

Consider exactly those B-weights v for which

$$f_B(v) \leq \rho^b.$$

Greedily partition these weights into groups P such that

$$\sum_{v \in P} f_B(v) \leq 2\rho^b,$$

and every completed group has total frequency at least ρ^b.

For each group P, define

$$A_u = \{x \in A : w(x) = u\},$$

$$B_P = \{y \in B : w(y) \in P\},$$

and

$$C_P = \{z \in C : -u - w(z) \in P\}.$$

Then

$$|A_u| = \rho^a, \qquad |B_P| = O(\rho^b).$$

Split every C_P into blocks of size at most

$$\rho^c.$$

On each resulting instance, all vertices of A_u have the same weight, so Lemma 354 reduces it to ordinary triangle detection with dimensions

$$O(\rho^a) \times O(\rho^b) \times O(\rho^c).$$

By the definition of $\omega(a, b, c)$, each such instance costs

$$O(\rho^{\omega(a,b,c)}).$$

We now count the number of instances for this fixed u.

Since every completed B-group contains at least ρ^b vertices, there are

$$O\left(\frac{n^b}{\rho^b}\right)$$

groups.

Moreover, for fixed u, the sets C_P are disjoint: a vertex z determines the unique complementary weight

$$-u - w(z).$$

Hence

$$\sum_P |C_P| \le n^c.$$

After splitting into ρ^c-sized blocks, the total number of blocks is therefore

$$O\left(\frac{n^b}{\rho^b} + \frac{n^c}{\rho^c}\right).$$

Thus the total cost associated with u is

$$O\left(n^b \rho^{\omega(a,b,c)-b+\varepsilon} + n^c \rho^{\omega(a,b,c)-c+\varepsilon}\right).$$

Now sum this directly over all A-weights u.

Since

$$\rho^a = f_A(u)$$

and

$$\sum_u \rho^a = \sum_u f_A(u) = |A| \le n^a,$$

we can charge both terms to this sum.

First, the trivial input-size lower bound

$$\omega(a,b,c) \ge a + b$$

gives

$$\omega(a,b,c) - a - b \ge 0.$$

Since $\rho \le n$,

$$\rho^{\omega(a,b,c)-b} = \rho^a \rho^{\omega(a,b,c)-a-b} \le \rho^a n^{\omega(a,b,c)-a-b}.$$

Therefore

$$\begin{aligned}\sum_u n^b \rho^{\omega(a,b,c)-b} &\le n^b n^{\omega(a,b,c)-a-b} \sum_u \rho^a \\ &\le n^b n^{\omega(a,b,c)-a-b} n^a \\ &= n^{\omega(a,b,c)}.\end{aligned}$$

Likewise,

$$\omega(a,b,c) \ge a + c,$$

so

$$\begin{aligned}\sum_u n^c \rho^{\omega(a,b,c)-c} &\le n^c n^{\omega(a,b,c)-a-c} \sum_u \rho^a \\ &\le n^{\omega(a,b,c)}.\end{aligned}$$

Hence this orientation costs

$$O(n^{\omega(a,b,c)}).$$

It finds every zero-weight triangle x, y, z satisfying

$$\rho_A(w(x)) \ge \rho_B(w(y)),$$

because when $u = w(x)$, the weight $w(y)$ is among the allowed B-weights, and the group containing it also contains z in its corresponding set C_P.

Run the same construction with A and B interchanged. This finds all remaining triangles. Rectangular matrix multiplication has the elementary symmetry

$$\omega(b,a,c) = \omega(a,b,c),$$

so the second run has the same complexity. □

10.8.4 Minimum Node-Weighted Triangle

Suppose now that the vertex weights are integers satisfying

$$|w(v)| \le W,$$

and we want a triangle of minimum total weight rather than merely a zero-weight triangle.

Only a logarithmic number of NWT calls are needed.

Lemma 357 (Testing a Target)**.** *Suppose $G = (A \sqcup B \sqcup C, E)$ is tripartite. For any integer t, we can test whether G contains a triangle of total weight exactly t using one call to NWT.*

Proof. Replace the weights on A by

$$w'(a) = w(a) - t$$

and leave the weights on $B \cup C$ unchanged.

Then

$$w'(a) + w(b) + w(c) = 0$$

if and only if

$$w(a) + w(b) + w(c) = t.$$

□

Theorem 358 (Minimum Node-Weighted Triangle from NWT)**.** *Suppose NWT on a given family of n-vertex instances can be solved in time $T(n)$. Then Minimum Node-Weighted Triangle with integer weights*

$$|w(v)| \le W$$

can be solved in

$$\boxed{O(T(n)\log(W+2))}$$

time.

Proof. We recursively reduce the magnitude of the weights.

Replace every weight $w(v)$ by

$$\widehat{w}(v) = \left\lfloor \frac{w(v)}{2} \right\rfloor.$$

Recursively compute the minimum triangle weight

$$s$$

under the rounded weights.

For every integer x,

$$x = 2\left\lfloor \frac{x}{2} \right\rfloor + \varepsilon_x, \qquad \varepsilon_x \in \{0,1\}.$$

Therefore for every triangle T,

$$\boxed{w(T) = 2\widehat{w}(T) + \varepsilon_T, \qquad \varepsilon_T \in \{0,1,2,3\}.}$$

Let M be the minimum triangle weight under the original weights.

Since

$$\widehat{w}(T) \ge s$$

for every triangle,

$$w(T) \ge 2s,$$

and hence

$$M \geq 2s.$$

On the other hand, let T_0 be a triangle with

$$\widehat{w}(T_0) = s.$$

Then

$$w(T_0) \leq 2s + 3,$$

so

$$M \leq 2s + 3.$$

Thus

$$\boxed{M \in \{2s, 2s+1, 2s+2, 2s+3\}.}$$

Using Lemma 357, make at most four NWT calls to determine which of these four values is attained, and return the smallest one.

The rounded weights satisfy

$$|\widehat{w}(v)| \leq \left\lceil \frac{W}{2} \right\rceil.$$

Thus, if $T_{\min}(W)$ denotes the running time,

$$T_{\min}(W) \leq T_{\min}(\lceil W/2 \rceil) + 4T(n) + O(n).$$

After $O(\log(W+2))$ levels the weights are bounded by a constant, at which point the minimum can be found with constantly many target queries.

Hence

$$T_{\min}(W) = O(T(n) \log(W+2)).$$

□

10.9 Undirected Hamiltonicity

Let $G = (V, E)$ be a simple undirected graph with $n \geq 3$ vertices. Fix a root s. We can generate closed walks

$$W = (v_0 = s, v_1, \ldots, v_{n-1}, v_n = s)$$

by dynamic programming, allowing repeated vertices but forbidding an intermediate return to s. Such a walk is a Hamiltonian cycle exactly when $v_0, \ldots, v_{n-1}$ are distinct.

Two simple tests. The direct test assigns a variable y_v to each vertex and uses the monomial $\prod_{i=0}^{n-1} y_{v_i}$, counting the root only once. Distinctness is exactly multilinearity, but the degree is n, so the preceding detector costs 2^n evaluations.

Undirectedness offers another approach. Consider the first repeated vertex before the final return to s. The intervening closed segment has distinct internal vertices. If it has at least three edges, reversing it gives a different walk with the same undirected-edge multiset. In characteristic 2, weights that preserve this reversal make the two contributions cancel. A two-edge backtrack u, v, u is its own reversal, but traverses the same edge twice, so an edge variable detects it by a square.

Thus we could track vertices and reject repetitions directly, or track edges and cancel the longer repeated-vertex segments. Either full test uses n factors. We will mix them so that some edges need not be tracked at all. We will also arrange auxiliary weights so that our desired cycles survive the cancellation.

Use a different test in each part. Choose a partition $V = A \sqcup B$ with $s \in A$. Track visits to vertices of A by variables y_v, and traversals of edges inside B by variables z_e. A repeated vertex in A produces a square, so after the multilinearity test only repeated vertices in B need attention. Their closed segments of length at least three will cancel by reversal.

For a two-edge backtrack u, v, u with $u \in B$, a middle vertex $v \in B$ squares the tracked variable $z_{\{u,v\}}$. If $v \in A$, neither test produces a square, so we exclude this one pattern locally:

$$\boxed{u, v, u \qquad \text{with } u \in B,\ v \in A.}$$

Call a walk *legitimate* if it contains no such consecutive triple. This excludes no Hamiltonian cycle and requires remembering only the last two vertices. Thus vertices in A, edges inside B, and one local restriction handle all the obstructions to reversal cancellation.

Where the saving comes from. Put $E_B = E(G[B])$. The tracked monomial of a walk is

$$M_W = \prod_{\substack{0 \le i < n \\ v_i \in A}} y_{v_i} \prod_{\substack{1 \le i \le n \\ v_{i-1}, v_i \in B}} z_{\{v_{i-1}, v_i\}}.$$

Products count occurrences, not just distinct vertices or edges. There is no tracked factor for a vertex of B or an edge crossing between A and B.

On a Hamiltonian cycle, break the B vertices into maximal consecutive runs. Since the root belongs to A, every run has an A vertex at both ends. A run with r vertices contributes only $r - 1$ internal edges, so replacing its vertex tests by edge tests saves one factor. If there are t runs, then

$$\boxed{\deg M_W = |A| + |E(W) \cap E_B| = n - t.} \tag{31}$$

The crossing edges let us move between the two kinds of tests without paying an additional tracked factor. We now make the cancellation argument precise.

Distinguish desired cycles. The tracked monomials alone do not distinguish different Hamiltonian cycles. Introduce auxiliary edge variables x_{uv}: away from s, use the same variable in both directions, $x_{uv} = x_{vu}$; on edges incident to s, use distinct variables x_{sv} and x_{vs}. All variables are otherwise distinct. Set

$$X_W = \prod_{i=1}^{n} x_{v_{i-1} v_i}.$$

An internal closed segment never visits s, so reversing it preserves X_W. Reversing an entire Hamiltonian cycle instead exchanges the incoming and outgoing root edges and changes X_W. This small asymmetry prevents the desired cycles from canceling with their own reversals.

Work over a field $\mathbb{F}$ of characteristic 2. For a chosen degree d, define

$$F_d(x, y, z) = \sum_{\substack{W \text{ legitimate closed } n\text{-edge walk} \\ v_0 = v_n = s,\ v_i \neq s\ (0 < i < n) \\ \deg M_W = d}} X_W M_W.$$

We test multilinearity only in (y, z), not in x. Restricting to degree d is important: the detector for degree d requires degree at most d in its tracked variables. A counter in the dynamic program will perform this restriction without expanding the polynomial.

Lemma 359 (Hybrid cancellation)**.** *The monomials of F_d that are multilinear in (y, z) are precisely the monomials of the rooted, oriented Hamiltonian cycles with d tracked factors. Each has coefficient* 1.

Proof. First consider a legitimate walk whose tracked monomial is multilinear but which is not Hamiltonian. Choose the smallest $j < n$ such that $v_j = v_i$ for some $i < j$. All vertices before j are distinct, so i is unique. Also $i > 0$, because intermediate visits to s are forbidden. The repeated vertex lies in B, or else its y variable would be squared.

The segment $(v_i, \ldots, v_j = v_i)$ cannot have two edges: a middle vertex in A would violate legitimacy, while a middle vertex in B would square an internal-edge variable. There are no loops, so the segment has at least three edges.

Reverse this segment. Its internal vertices are distinct, so the walk changes. Its vertex and undirected-edge multisets do not change, and the segment avoids s, so the full monomial $X_W M_W$ is preserved. The first repetition is still at the same positions i, j, making this operation an involution.

Legitimacy is preserved as well. Triples inside the segment are reversed, and the forbidden pattern is invariant under reversal. Triples outside are unchanged. A changed triple crossing a boundary of the segment has its middle vertex at $v_i = v_j \in B$, whereas a forbidden triple has its middle vertex in A. Thus these nonsimple walks cancel in pairs in characteristic 2.

A Hamiltonian cycle, on the other hand, is legitimate and has a multilinear tracked monomial. Its auxiliary monomial records its undirected edge set and its outgoing edge at s. These determine its entire traversal: after leaving s, the next unused edge is forced at every vertex. No other walk has the same auxiliary monomial, so its coefficient is 1. All remaining walks have a squared tracked variable and cannot contribute a multilinear monomial. □

Evaluate walks. For a numerical assignment of the variables, F_d has a polynomial-time evaluation. Store the total weight of partial walks by their length, their last two vertices, and their tracked degree. These records suffice because legitimacy is a local condition.

For a step $v \to w$ with $w \neq s$, its tracked factor $g(v, w)$ and its tracked-degree increment $c(v, w)$ are

$$(g(v,w), c(v,w)) = \begin{cases} (y_w, 1), & w \in A, \\ (z_{\{v,w\}}, 1), & v, w \in B, \\ (1, 0), & v \in A,\ w \in B. \end{cases}$$

The full step weight is $x_{vw} g(v, w)$. The symbol $\perp \notin V$ marks the absent predecessor at the root.

Algorithm 63 Evaluating the hybrid walk polynomial

```
function WalkValue(G, s, A, B, d, x, y, z)
    Set all entries of D to zero; set D(⊥, s, 1) ← y_s
    for ℓ = 1, ..., n − 1 do
        Set all entries of D′ to zero
        for each (u, v, j) with 0 ≤ j ≤ d and D(u, v, j) ≠ 0 do
            for each w ∈ N(v) \ {s} do
                if not (u = w ∈ B and v ∈ A) then
                    j′ ← j + c(v, w)
                    if j′ ≤ d then
                        D′(v, w, j′) ← D′(v, w, j′) + D(u, v, j) x_vw g(v, w)
        D ← D′
    return Σ_{v∈N(s)} Σ_u D(u, v, d) x_vs
```

At layer ℓ, the entry $D(u, v, j)$ sums the weights of legitimate ℓ-vertex prefixes with the indicated last vertices and degree. Initialization counts s once; each transition appends exactly one vertex and edge. After $n - 1$ extensions, the last line closes the walk. The closing edge adds no tracked factor because $s \in A$ and has already been counted. It cannot create a forbidden triple ending at s, since the ends of a forbidden triple belong to B. This proves the invariant and the evaluation formula.

There are $O(dn^2)$ states per layer and at most n extensions per state. Thus one evaluation uses $O(dn^4)$ field operations and $O(dn^2)$ field elements of storage. In particular, it has polynomial cost.

The multilinear test. Regard the combined variables (y, z) as one vector q, indexed by the disjoint union $A \sqcup E_B$. By Theorem 535, the polynomial

$$H_d(x, \xi_1, \dots, \xi_d) = \sum_{J \subseteq [d]} F_d \left(x, \sum_{j \in J} \xi_j \right)$$

is nonzero exactly when F_d has a monomial multilinear in q. Here $\xi_1, \dots, \xi_d$ are vectors with those same coordinates. The cancellation lemma therefore makes this a test for Hamiltonian cycles with the chosen tracked degree.

The degree of H_d is at most $n + d \le 2n$: every walk contributes n auxiliary edge factors and d tracked factors. Choose a characteristic-2 field with $4n \le |\mathbb{F}| < 8n$ and assign all auxiliary variables and all coordinates of the ξ_j independently and uniformly. If $H_d \not\equiv 0$, Schwartz–Zippel gives $\Pr[H_d \neq 0] \ge 1/2$. If it is identically zero, every evaluation is zero. Computing this test takes 2^d calls to Algorithm 63, performed sequentially with the *same* auxiliary assignment and direction vectors.

How much degree can a random partition save? Put s in A and place every other vertex independently in A or B with equal probability. Fix a Hamiltonian cycle. Reading its partition labels from s back to s gives a word that begins and ends with A, with n boundaries between consecutive labels. There are 2^{n-1} equally likely words.

Each run of B has exactly two changing boundaries, one entering and one leaving. Conversely, choosing any $2t$ of the n boundaries uniquely determines a word with t runs: start with A and switch labels exactly at the chosen boundaries. Therefore

$$\pi_t := \Pr[\text{the fixed cycle has } t \text{ runs of } B] = \frac{\binom{n}{2t}}{2^{n-1}}, \qquad 0 \le t \le \lfloor n/2 \rfloor.$$

Such a partition lets us use degree $d = n - t$. Each trial then costs $O^*(2^{n-t})$ and succeeds with probability at least $\pi_t/2$, where O^* suppresses factors polynomial in n. Thus the relevant cost is

$$\frac{2^{n-t}}{\pi_t} = \frac{2^{2n-1}}{2^t \binom{n}{2t}}.$$

More runs make the sieve cheaper, but a prescribed unusually large number of runs makes the partition harder to obtain. We choose t to maximize the denominator, balancing these two effects.

Algorithm 64 Hamiltonicity by a hybrid multilinear sieve

Input: Simple undirected graph $G = (V, E)$
Output: No false positives; false-negative probability at most $1/3$

if $n < 3$ **then**
  return No
Fix $s \in V$; choose a characteristic-2 field $\mathbb{F}$ with $4n \le |\mathbb{F}| < 8n$
Choose $t \in \arg\max_{0 \le j \le \lfloor n/2 \rfloor} 2^j \binom{n}{2j}$
$d \leftarrow n - t$, $\pi \leftarrow 2^{-(n-1)} \binom{n}{2t}$, $T \leftarrow \lceil 2 \log(3)/\pi \rceil$
for $i = 1, \dots, T$ **do**
  Put s in A; assign every other vertex independently and uniformly to A or B
  Choose independent uniform values for the distinct auxiliary variables x
  Choose independent uniform vectors $\xi_1, \dots, \xi_d \in \mathbb{F}^{A \sqcup E_B}$
  $R \leftarrow 0$
  for each $J \subseteq [d]$ **do**
    $(y, z) \leftarrow \sum_{j \in J} \xi_j$
    $R \leftarrow R + \textsc{WalkValue}(G, s, A, B, d, x, y, z)$
  if $R \neq 0$ **then**
    return Yes
return No

Theorem 360 (Hybrid Hamiltonicity sieve)**.** *Algorithm 64 decides Hamiltonicity in* $O^*((4(\sqrt{2}-1))^n) = O^*(1.657^n)$ *time and polynomial space, with no false positives and false-negative probability at most* $1/3$.

Proof. If there is no Hamiltonian cycle, every tested polynomial is identically zero. Otherwise fix one cycle. Each independent trial succeeds with probability at least $\pi/2$, so the probability that all trials fail is at most $(1 - \pi/2)^T \le e^{-\pi T/2} \le 1/3$.

For the running time, the even part of the binomial theorem gives

$$\sum_{t=0}^{\lfloor n/2 \rfloor} 2^t \binom{n}{2t} = \frac{(1+\sqrt{2})^n + (1-\sqrt{2})^n}{2} = \Theta\big((1+\sqrt{2})^n\big).$$

The largest of these $O(n)$ summands is at least their average. Consequently the total cost is

$$O^*\left(\frac{2^{2n-1}}{\max_t 2^t \binom{n}{2t}} \right) = O^*\left(\left(\frac{4}{1+\sqrt{2}} \right)^n \right).$$

Since $4/(1+\sqrt{2}) = 4(\sqrt{2}-1)$, this is the claimed bound. The algorithm stores only one trial, its direction vectors, and two layers of the walk dynamic program at a time, so its space usage is polynomial. Field elements have $O(\log n)$ bits. □

10.10 An Algebraic Criterion for Bipartite Perfect Matching

Let

$$G = (L \sqcup R, E), \qquad L = R = [n],$$

with bipartite adjacency matrix M.

Hall's theorem says that G has no perfect matching exactly when some set

$$I \subseteq L$$

has fewer neighbors than vertices:

$$|N(I)| < |I|.$$

We would like to turn this directly into a homogeneous linear system.

The intended picture is simple. Each active left vertex should contribute some number of variables, while each neighboring right vertex should impose some number of linear constraints. Then a Hall-deficient set I should leave more variables than constraints and therefore give a nonzero solution.

The difficulty is the converse. Even if

$$|N(I)| \geq |I|,$$

the constraints arising from different right vertices could conceivably be highly dependent. We need a way to force every active left vertex and every neighboring right vertex to consume some genuinely independent resource.

We use high-multiplicity zeros for this purpose.

Each left vertex i receives its own point α_i, and its polynomial will be forced to have a very high-order zero there. These are the *private signatures* of the left vertices.

Each right vertex j receives a fresh point β_j, and the linear combination corresponding to j will be forced to have a high-order zero there.

The Wronskian gives the crucial fact: a low-dimensional space of low-degree polynomials has only a limited total supply of such high-multiplicity zeros.

10.10.1 The Wronskian as a Multiplicity Budget

Let W be an s-dimensional space of polynomials of degree at most d over a field of characteristic greater than d. For $\gamma \in \mathbb{F}$, define

$$N_W(\gamma) = \max_{0 \neq f \in W} \operatorname{mult}_\gamma(f).$$

Thus $N_W(\gamma)$ is the largest order of vanishing at γ that can be achieved by a nonzero polynomial in W.

Lemma 361 (Wronskian Multiplicity Budget)**.**

$$\boxed{\sum_{\gamma \in \mathbb{F}} \max\{N_W(\gamma) - s + 1, 0\} \leq sd.}$$

Proof. Choose a basis

$$f_1, \ldots, f_s$$

of W, and form the Wronskian

$$\mathcal{W}(X) = \det \begin{pmatrix} f_1 & \cdots & f_s \\ f_1' & \cdots & f_s' \\ \vdots & & \vdots \\ f_1^{(s-1)} & \cdots & f_s^{(s-1)} \end{pmatrix}.$$

We first note that

$$\mathcal{W} \neq 0.$$

Indeed, by row-reducing the coefficient vectors of the basis polynomials, we may choose a basis with distinct degrees

$$d_1 < d_2 < \cdots < d_s.$$

If the leading coefficient of f_j is c_j, then the highest-degree term of the Wronskian is

$$\left(\prod_{j=1}^{s} c_j \right) \det((d_j)_{i-1})_{i,j=1}^{s} \, X^{\sum_j d_j - \binom{s}{2}},$$

where

$$(d)_r = d(d-1)\cdots(d-r+1).$$

The determinant is the Vandermonde

$$\prod_{1\le i<j\le s}(d_j - d_i),$$

which is nonzero because the d_i's are distinct and all are smaller than the characteristic.

Now fix γ, and let

$$t = N_W(\gamma).$$

Choose $f \in W$ with

$$\mathrm{mult}_\gamma(f) = t,$$

and change the basis so that f is its first element. Changing the basis multiplies the Wronskian by a nonzero scalar, so its zeros and their multiplicities do not change.

For

$$0 \le j < s,$$

the derivative $f^{(j)}$ vanishes at γ to order at least

$$\max\{t-j, 0\}.$$

Hence every entry of the first Wronskian column is divisible by

$$(X-\gamma)^{\max\{t-s+1,0\}}.$$

Every term in the determinant uses exactly one entry from that column, so

$$\mathrm{mult}_\gamma(\mathcal{W}) \ge \max\{t-s+1, 0\}.$$

Therefore

$$\mathrm{mult}_\gamma(\mathcal{W}) \ge \max\{N_W(\gamma) - s + 1, 0\}.$$

Summing over all γ,

$$\sum_\gamma \max\{N_W(\gamma) - s + 1, 0\} \le \deg \mathcal{W}.$$

Finally,

$$\deg \mathcal{W} \le \sum_{j=1}^{s} \deg f_j \le sd.$$

□

10.10.2 Designing the Polynomial System

Choose a field of characteristic larger than all degrees appearing below, and choose

$$\alpha_1, \dots, \alpha_n, \beta_1, \dots, \beta_n$$

to be $2n$ distinct field elements.

Set

$$\ell = 2n^2, \qquad a = 2n^2 - 2n + 1, \qquad k = 2n^3.$$

The roles of these parameters are different:

- $a+1$ is the number of degrees of freedom attached to each left vertex;
- ℓ is the number of vanishing constraints imposed by each right vertex;

- k is made much larger than both, solely to ensure that the private left-vertex signatures are linearly independent.

The first two parameters are chosen so that two opposite inequalities hold:

$$(a+1)s > \ell(s-1) \qquad (1 \le s \le n),$$

while

$$\ell > a + 2n - 2.$$

Indeed,

$$(a+1)s - \ell(s-1) = 2n^2 - (2n-2)s > 0,$$

and

$$a + 2n - 2 = 2n^2 - 1 < \ell.$$

The first inequality will make every Hall deficiency produce a nonzero solution. The second will make Hall expansion incompatible with the Wronskian budget.

Variables For each left vertex i, introduce a polynomial

$$A_i(X), \qquad \deg A_i \le a,$$

and set

$$\boxed{P_i(X) = (X - \alpha_i)^k A_i(X).}$$

Thus a nonzero P_i has a private k-fold zero at α_i.

There are

$$a + 1$$

coefficients in A_i, so altogether we have

$$n(a+1)$$

scalar variables.

Constraints For each right vertex j, define

$$\boxed{Q_j(X) = \sum_{i=1}^{n} M_{ij} P_i(X).}$$

Require

$$\boxed{(X - \beta_j)^\ell \mid Q_j(X).}$$

Since the characteristic exceeds all relevant degrees, this is equivalent to

$$Q_j(\beta_j) = Q_j'(\beta_j) = \cdots = Q_j^{(\ell-1)}(\beta_j) = 0.$$

Thus each right vertex contributes ℓ homogeneous linear equations in the coefficients of the A_i's.

The entire construction is therefore one explicit homogeneous linear system with

$$n(a+1) = O(n^3)$$

variables and

$$n\ell = O(n^3)$$

equations.

Theorem 362 (Polynomial Hall Criterion)**.** *With the construction above,*

$$\boxed{G \text{ has no perfect matching} \iff \text{the polynomial system has a nonzero solution.}}$$

Equivalently,

$$G \text{ has a perfect matching} \iff P_1 = \cdots = P_n = 0$$

is the only solution.

The two directions mirror the two sides of Hall's theorem.

10.10.3 Hall Deficiency Leaves Too Much Freedom

Lemma 363 (Deficiency Gives a Nonzero Solution)**.** *If there is a set*

$$I \subseteq L$$

with

$$|N(I)| < |I|,$$

then the polynomial system has a nonzero solution.

Proof. Let

$$J = N(I), \qquad s = |I|.$$

Set

$$P_i = 0 \qquad (i \notin I).$$

If

$$j \notin J,$$

then $M_{ij} = 0$ for every $i \in I$, so

$$Q_j = 0.$$

Thus all constraints belonging to right vertices outside J are automatically satisfied.

The remaining variables are the coefficients of

$$A_i, \qquad i \in I.$$

There are

$$(a+1)s$$

such variables.

Each $j \in J$ contributes at most ℓ homogeneous linear equations, so there are at most

$$\ell|J| \le \ell(s-1)$$

constraints.

But

$$(a+1)s - \ell(s-1) = 2n^2 - (2n-2)s > 0.$$

Hence there are strictly more variables than homogeneous equations, so the restricted system has a nonzero solution. □

This direction is exactly Hall deficiency expressed as dimension counting:

$$\boxed{|I| > |N(I)| \quad \Longrightarrow \quad \text{more left-vertex freedom than right-vertex constraints.}}$$

Nothing about the Wronskian is needed here.

10.10.4 Private Zeros Force the Active Polynomials to be Independent

The converse needs more work. A nonzero solution may activate several left vertices, and we need to know that these contribute distinct directions in polynomial space.

That is exactly what the huge private zeros at the α_i's guarantee.

Lemma 364 (Private-Zero Independence)**.** *Choose distinct indices*

$$i_1, \dots, i_t, \qquad t \le n,$$

and nonzero polynomials

$$P_{i_r} = (X - \alpha_{i_r})^k A_{i_r}, \qquad \deg A_{i_r} \le a.$$

Then

$$P_{i_1}, \dots, P_{i_t}$$

are linearly independent.

Proof. Suppose instead that their span

$$W$$

has dimension

$$r < t.$$

Every polynomial in W has degree at most

$$k + a.$$

For each of the t private points α_{i_j}, the space W contains the corresponding nonzero polynomial P_{i_j}, which has a k-fold zero there. Hence

$$N_W(\alpha_{i_j}) \ge k.$$

Applying Lemma 361,

$$t(k - r + 1) \le r(k + a).$$

We show that this is impossible.

For fixed t, the difference

$$t(k - r + 1) - r(k + a)$$

decreases with r, so the hardest case is

$$r = t - 1.$$

It is therefore enough to prove

$$t(k - t + 2) > (t - 1)(k + a),$$

or equivalently

$$k + t > (t - 1)(t + a).$$

Since $t \le n$,

$$(t - 1)(t + a) \le (n - 1)(n + a).$$

Using

$$a = 2n^2 - 2n + 1$$

and

$$k = 2n^3,$$

we obtain

$$(n - 1)(n + a) = 2n^3 - 3n^2 + 2n - 1 < 2n^3 \le k + t.$$

This is a contradiction. □

The point of the large value of k is now clear. If t active left polynomials tried to live in a space of dimension $r < t$, that r-dimensional space would have to support t different k-fold private zeros. The Wronskian budget cannot afford them.

10.10.5 Hall Expansion Exhausts the Remaining Multiplicity Budget

We can now prove the converse.

Lemma 365 (Hall Expansion Forces the Zero Solution)**.** *Suppose*

$$|N(I)| \geq |I| \qquad \text{for every } I \subseteq L.$$

Then the polynomial system has only the zero solution.

Proof. Suppose for contradiction that there is a nonzero solution, and let

$$I = \{i : P_i \neq 0\}, \qquad s = |I|.$$

Set

$$W = \operatorname{span}\{P_i : i \in I\}.$$

By Lemma 364,

$$\boxed{\dim W = s.}$$

Every active left vertex contributes its private zero:

$$N_W(\alpha_i) \geq k \qquad (i \in I).$$

Now let

$$J = N(I).$$

Hall's condition gives

$$|J| \geq s.$$

For every $j \in J$,

$$Q_j = \sum_{i \in I} M_{ij} P_i.$$

Because $j \in N(I)$, at least one coefficient M_{ij} in this combination is nonzero. Since the P_i's are linearly independent,

$$Q_j \neq 0.$$

The defining constraints give

$$(X - \beta_j)^\ell \mid Q_j,$$

so

$$N_W(\beta_j) \geq \ell.$$

Thus the same s-dimensional space W contains

$$s \text{ distinct points } \alpha_i \text{ with multiplicity at least } k,$$

and at least

$$s \text{ distinct points } \beta_j \text{ with multiplicity at least } \ell.$$

All these points are distinct. Applying Lemma 361 and keeping only s of the β_j's gives

$$s(k - s + 1) + s(\ell - s + 1) \leq s(k + a),$$

because every polynomial in W has degree at most $k + a$.

Divide by s and cancel k:

$$\ell \leq a + 2s - 2.$$

Since

$$s \le n,$$

this implies

$$\ell \le a + 2n - 2.$$

But our parameters satisfy

$$a + 2n - 2 = 2n^2 - 1 < 2n^2 = \ell,$$

a contradiction. □

Combining Lemmas 363 and 365 proves Theorem 362.

10.10.6 The Resulting Algorithm

The criterion above is already an algorithm: all of the divisibility conditions are homogeneous linear equations in the coefficients of the polynomials A_i. We now write this linear system explicitly.

It is convenient to use Hasse derivatives. Define

$$D_t(X^r) = \binom{r}{t} X^{r-t}.$$

They satisfy the product rule

$$D_t(fg) = \sum_{t_1+t_2=t} D_{t_1}(f) D_{t_2}(g).$$

Since the characteristic is larger than all relevant degrees,

$$(X-\beta)^\ell \mid f(X)$$

if and only if

$$D_t f(\beta) = 0 \qquad (0 \le t < \ell).$$

Write

$$A_i(X) = \sum_{r=0}^{a} A_{i,r} X^r.$$

The variables of our linear system are therefore

$$A_{i,r}, \qquad i \in [n], \quad 0 \le r \le a.$$

Recall that

$$Q_j(X) = \sum_{i=1}^{n} M_{ij} A_i(X) (X-\alpha_i)^k.$$

The constraint

$$(X-\beta_j)^\ell \mid Q_j(X)$$

is equivalent to

$$D_t Q_j(\beta_j) = 0 \qquad (0 \le t < \ell).$$

Expanding by the Hasse product rule,

$$\begin{aligned} D_t Q_j(\beta_j) &= \sum_i M_{ij} D_t \left(A_i(X)(X-\alpha_i)^k \right)(\beta_j) \\ &= \sum_i M_{ij} \sum_{t_1+t_2=t} D_{t_1} A_i(\beta_j) D_{t_2}(X-\alpha_i)^k(\beta_j). \end{aligned}$$

Now

$$D_{t_1} A_i(\beta_j) = \sum_{r=0}^{a} A_{i,r} \binom{r}{t_1} \beta_j^{r-t_1},$$

while

$$D_{t_2}(X - \alpha_i)^k(\beta_j) = \binom{k}{t_2} (\beta_j - \alpha_i)^{k-t_2}.$$

Hence

$$D_t Q_j(\beta_j) = \sum_{i=1}^{n} \sum_{r=0}^{a} A_{i,r} M_{ij} \Gamma_{i,r,j,t},$$

where

$$\boxed{\Gamma_{i,r,j,t} = \sum_{t_1+t_2=t} \binom{r}{t_1} \binom{k}{t_2} \beta_j^{r-t_1} (\beta_j - \alpha_i)^{k-t_2}.}$$

Thus define the matrix

$$F \in \mathbb{F}^{n(a+1) \times n\ell}$$

whose rows are indexed by

$$(i, r), \qquad i \in [n], \quad 0 \le r \le a,$$

whose columns are indexed by

$$(j, t), \qquad j \in [n], \quad 0 \le t < \ell,$$

and whose entries are

$$\boxed{F_{(i,r),(j,t)} = M_{ij} \Gamma_{i,r,j,t}.}$$

If

$$\mathbf{A} = (A_{i,r})_{i,r}$$

is the row vector containing all coefficients of the A_i's, then all of the divisibility constraints are exactly

$$\boxed{\mathbf{A}F = 0.}$$

There are

$$n(a+1) = O(n^3)$$

rows and

$$n\ell = O(n^3)$$

columns. Therefore F has a nonzero left kernel exactly when

$$\operatorname{rank}(F) < n(a+1).$$

By Theorem 362, we conclude that

$$\boxed{G \text{ has a perfect matching} \iff \operatorname{rank}(F) = n(a+1).}$$

Thus the algorithm is simply:

1. choose distinct
$$\alpha_1, \ldots, \alpha_n, \beta_1, \ldots, \beta_n$$
over a field of sufficiently large characteristic;
2. construct the matrix F above from the adjacency matrix M;
3. compute $\operatorname{rank}(F)$;
4. return that G has a perfect matching exactly when F has full row rank.

Constructing the Matrix The constants $\Gamma_{i,r,j,t}$ can of course be computed directly. There is also a convenient way to compute all of them simultaneously.

For fixed i, j, observe that

$$\binom{a}{r}\Gamma_{i,r,j,t}$$

is exactly the coefficient of $Y^r Z^t$ in

$$\boxed{(1+Y(Z+\beta_j))^a\,(Z+(\beta_j-\alpha_i))^k\,.}$$

Indeed, the coefficient of Y^r in the first factor is

$$\binom{a}{r}(Z+\beta_j)^r,$$

and then the coefficient of Z^t in

$$(Z+\beta_j)^r(Z+(\beta_j-\alpha_i))^k$$

is

$$\sum_{t_1+t_2=t}\binom{r}{t_1}\binom{k}{t_2}\beta_j^{r-t_1}(\beta_j-\alpha_i)^{k-t_2}.$$

Only coefficients with

$$t<\ell$$

are ever needed. Hence for each i, j we may perform the computation modulo Z^ℓ:

$$(1+Y(Z+\beta_j))^a\,(Z+(\beta_j-\alpha_i))^k \bmod Z^\ell.$$

Reading off its coefficients gives every $\Gamma_{i,r,j,t}$ for this pair i, j.

Since

$$a,\ell=O(n^2),$$

the relevant bivariate polynomial has only $O(n^4)$ coefficients. Using repeated squaring and fast polynomial multiplication, all of these coefficients for one pair i, j can be computed in

$$\widetilde{O}(n^4)$$

field operations, and doing this for all n^2 pairs takes

$$\widetilde{O}(n^6)$$

operations.

The graph itself enters only through the final multiplication

$$F_{(i,r),(j,t)}=M_{ij}\Gamma_{i,r,j,t};$$

the $\Gamma_{i,r,j,t}$'s depend only on i, r, j, t and the fixed field points.

10.11 Shortest Paths

10.11.1 Eppstein's Algorithm

Eppstein's algorithm elegantly computes the k shortest paths from a source s to a destination t in a directed graph in $O((m+n)\log n + k\log k)$ time. Rather than searching for full paths from scratch, the algorithm characterizes every possible $s \to t$ path as the base shortest path plus a sequence of deviations, or sidetracks.

First, we compute the shortest path from every vertex u to t using a reverse Dijkstra's search. This yields the shortest path distance $d[u]$ for all vertices and implicitly defines a shortest path tree T rooted at t.

For any directed edge $e = (u, v)$ with weight w, its sidetrack cost represents the distance penalty incurred by taking e instead of following the shortest path tree T from u. It is defined as:

$$\delta(e) = w + d[v] - d[u] \geq 0$$

Any path P from s to t can be uniquely identified by the sequence of non-tree edges (sidetracks) it traverses: $e_1, e_2, \ldots, e_p$. The total length of P is exactly the base distance plus the sum of its sidetrack penalties:

$$\text{length}(P) = d[s] + \sum_{i=1}^{p} \delta(e_i)$$

State Space and Persistent Heaps To efficiently find the k lightest sidetrack sequences, we construct a min-heap $H[u]$ for every vertex u. This heap contains all possible sidetrack edges originating from u and all ancestors of u in T, ordered by $\delta(e)$.

Because $H[u]$ shares almost all its elements with the heap of its parent in T, we use a Persistent Skew Heap. This allows us to implicitly build the heaps for all vertices in $O(m \log n)$ time. Once the heaps are built, enumerating the k-shortest paths reduces to a Dijkstra-like search over the structure of these heaps, transitioning either by swapping to heavier sidetracks within the same heap or appending new sidetracks from a destination vertex.

Algorithm 65 Eppstein's k-Shortest Paths

Input: Graph $G = (V, E)$, source s, destination t, integer k
Output: Lengths of the k shortest $s \to t$ paths

```
function EPPSTEIN(G, s, t, k)
    d, T ← REVERSEDIJKSTRA(G, t)
    if d[s] = ∞ then
        return ∅
    Order V by d[u] ascending
    for u ∈ V do
        root ← heap root of u's parent in T (or null if none)
        for each outgoing edge e = (u, v) ∈ E \ T do
            δ ← weight(e) + d[v] − d[u]
            root ← PERSISTENTINSERT(root, δ, v)
        H[u] ← root
    Q ← empty min-priority queue
    Q.push(length = d[s], node = H[s])
    R ← empty list
    while Q is not empty and |R| < k do
        (L, node) ← Q.pop()
        Append L to R
        if node.left ≠ null then
            Q.push(L − node.δ + node.left.δ, node.left)
        if node.right ≠ null then
            Q.push(L − node.δ + node.right.δ, node.right)
        next_heap ← H[node.dest]
        if next_heap ≠ null then
            Q.push(L + next_heap.δ, next_heap)
    return R
```

Complexity Analysis

Theorem 366. *Eppstein's algorithm correctly enumerates the k shortest paths in $O((m+n)\log n + k\log k)$ time.*

Proof. The base distances and shortest path tree are found in $O((m+n)\log n)$ time. Constructing the persistent heaps requires inserting at most m non-tree edges. Each persistent insertion takes $O(\log n)$ time, leading to $O(m \log n)$ time and space for the heap construction.

During the path extraction phase, every valid transition in the state space explores paths in non-decreasing order of total weight because the heaps are strictly min-ordered ($\delta(\text{child}) \geq \delta(\text{parent})$) and $\delta(e) \geq 0$. Since each state expands to at most 3 subsequent states, the priority queue size never exceeds $3k$. Performing k extractions from this queue takes $O(k \log k)$ time. Summing these phases yields the final bounds. □

10.11.2 Bellman–Ford–Dijkstra

Dijkstra's algorithm efficiently computes shortest paths under nonnegative edges. Bellman–Ford handles negative edges but has a bad running time.

The following hybrid separates the two jobs: after every use of a negative edge, run Dijkstra to exhaustion on all nonnegative edges.

Let

$$E^+ = \{e : w(e) \geq 0\}, \qquad E^- = \{e : w(e) < 0\}.$$

For $i \geq 1$, define

$$d_i(v) = \min\{w(P) : P \text{ is an } s\text{-}v \text{ path containing fewer than } i \text{ negative edges}\}.$$

Thus

$$d_1$$

is simply shortest-path distance in the nonnegative subgraph.

Also define

$$d'_i(v) = \min\left\{d_i(v), \min_{(u,v)\in E^-} d_i(u) + w(u,v)\right\}.$$

The crucial recurrence is

$$\boxed{d_{i+1}(v) = \min\left\{d_i(v), \min_u \left(d'_i(u) + \text{dist}_{G^+}(u,v)\right)\right\},}$$

where it is enough to consider vertices u whose label actually decreased in the negative-edge relaxation.

Indeed, take a shortest path using at most i negative edges and cut it after its last negative edge. Everything after that edge lies in G^+.

Algorithm 66 Bellman–Ford–Dijkstra

Input: Directed weighted graph G, source s
Output: Shortest-path distances, assuming no reachable negative cycle

```
d[s] ← 0
d[v] ← ∞ for v ≠ s
insert s into priority queue Q
repeat
    A ← ∅
                                          ▷ Dijkstra through nonnegative edges
    while Q is nonempty do
        remove a minimum-label vertex u from Q
        add u to A
        for each (u, v) ∈ E⁺ do
            if d[u] + w(u, v) < d[v] then
                d[v] ← d[u] + w(u, v)
                decrease-key v in Q
                                          ▷ Allow one additional negative edge
    for u ∈ A do
        for each (u, v) ∈ E⁻ do
            if d[u] + w(u, v) < d[v] then
                d[v] ← d[u] + w(u, v)
                decrease-key v in Q
until Q is empty
return d
```

Theorem 367. *After the i-th Dijkstra phase,*

$$d[v] = d_i(v)$$

for every vertex v.

After the following negative-edge phase,

$$d[v] = d'_i(v).$$

Consequently, if the graph contains no reachable negative cycle, the algorithm terminates with the true shortest-path distances.

Proof. We induct on i.

For $i = 1$, the first phase is ordinary Dijkstra on G^+, so it computes d_1.

Suppose a negative phase has produced d'_i. The only newly improved vertices are inserted into Q. Running Dijkstra from these labels computes

$$\min_u \left(d'_i(u) + \mathrm{dist}_{G^+}(u, v) \right),$$

while retaining any old smaller value $d_i(v)$. By the last-negative-edge decomposition, this is exactly

$$d_{i+1}(v).$$

Now suppose the Dijkstra phase has produced d_i. Relaxing every negative outgoing edge from a vertex whose value became newly relevant computes

$$d'_i.$$

A negative edge from a vertex whose d_i value did not change was already relaxed when that value was first obtained, so rescanning it is unnecessary.

Thus the invariant holds in every round.

If no reachable negative cycle exists, every shortest path may be chosen simple and therefore contains at most $n-1$ negative edges. Hence eventually

$$d_i(v) = \operatorname{dist}(s, v)$$

for all v, after which no negative edge improves a label and Q becomes empty. □

Let

$$\eta(v)$$

be the minimum number of negative edges among shortest s–v paths. Then v can become newly active in at most

$$\eta(v) + 1$$

Dijkstra phases.

With a binary heap, the running time is therefore

$$\boxed{O\left(\sum_v (\deg(v) + \log n)(\eta(v) + 1)\right).}$$

If every shortest path uses at most h negative edges or we only consider such paths, this is

$$O((m + n \log n)(h + 1)).$$

Thus the algorithm interpolates naturally between Dijkstra ($h = 0$) and Bellman–Ford ($h = \Theta(n)$).

11 Algebraic and Spectral Graph Theory

11.1 The Matrix–Tree Theorem

Lemma 368 (Spanning Tree Minors)**.** *For*

$$F \subseteq E, \qquad |F| = n - 1,$$

$$\det B_r[F, :] = \begin{cases} \pm 1, & F \text{ is a spanning tree}, \\ 0, & \text{otherwise}. \end{cases}$$

Proof. If F contains a cycle, the corresponding incidence rows are dependent. If F is disconnected, its incidence matrix has rank less than $n-1$.

Thus only spanning trees can give a nonzero determinant. For a tree, repeatedly remove a leaf other than r. The corresponding column contains a single nonzero entry ± 1, so expansion along that column reduces the determinant to a smaller tree. Induction gives ± 1. □

A more useful form of Matrix–Tree is the following asymmetric statement.

Theorem 369 (Generalized Matrix–Tree Identity)**.** *Let*

$$W = \operatorname{diag}(w_e)$$

and let

$$A \in \mathbb{F}^{E \times (n-1)}$$

be arbitrary. Then

$$\boxed{\det(B_r^\top W A) = \sum_{T \text{ spanning tree}} \det(B_r[T, :]) \det(A[T, :]) \prod_{e \in T} w_e.}$$

Proof. Apply Cauchy–Binet:

$$\begin{aligned}\det(B_r^\top W A) &= \sum_{\substack{F\subseteq E\\ |F|=n-1}} \det(B_r[F,:]^\top W_F)\det(A[F,:]) \\ &= \sum_{F} \det(B_r[F,:])\det(A[F,:])\prod_{e\in F} w_e.\end{aligned}$$

By Lemma 368, every non-tree term vanishes. □

Taking

$$A = B_r$$

gives the usual weighted Matrix–Tree theorem:

$$\boxed{\det(B_r^\top W B_r) = \sum_{T \text{ spanning tree}} \prod_{e\in T} w_e.}$$

The arbitrary second matrix is often useful because it can impose additional combinatorial conditions while the first incidence matrix enforces that the chosen edge set is a tree.

11.1.1 Counting Hamiltonian Paths

Let $G = (V, E)$ be an unweighted graph,

$$|V| = n,$$

and fix vertices s, t.

Give an edge

$$e = \{u, v\}$$

weight

$$w_e = x_u x_v.$$

By Matrix–Tree,

$$P_G(x) := \det(B_r^\top W(x) B_r) = \sum_{T \text{ tree}} \prod_{v\in V} x_v^{\deg_T(v)}.$$

Theorem 370 (Hamiltonian Paths from Matrix–Tree)**.**

$$\boxed{\#\,\mathrm{HamPath}(G; s, t) = \frac{1}{2^n} \sum_{x\in\{-1,1\}^V} x_s x_t P_G(x).}$$

Consequently s–t Hamiltonian paths can be counted in

$$O^*(2^n)$$

time and polynomial space.

Proof. Consider one spanning tree T. Its contribution after multiplying by $x_s x_t$ is

$$x_s^{\deg_T(s)+1} x_t^{\deg_T(t)+1} \prod_{v\notin\{s,t\}} x_v^{\deg_T(v)}.$$

For an integer a,

$$\sum_{x\in\{-1,1\}} x^a = \begin{cases} 2, & a \text{ even}, \\ 0, & a \text{ odd}. \end{cases}$$

Thus T survives the outer sum exactly when

$$\deg_T(s), \deg_T(t)$$

are odd and every other degree is even.

All degrees are positive, and

$$\sum_v \deg_T(v) = 2n - 2.$$

The minimum possible sum under these parity conditions is already

$$1 + 1 + 2(n-2) = 2n - 2.$$

Therefore equality must hold everywhere:

$$\deg_T(s) = \deg_T(t) = 1$$

and

$$\deg_T(v) = 2 \qquad (v \neq s, t).$$

Such a tree is precisely an s–t Hamiltonian path.

Every surviving tree contributes 2^n. □

11.1.2 Counting k-Matchings in a Bipartite Graph

Let

$$G = (U \sqcup V, E), \qquad |U| = n_1 \leq n_2 = |V|.$$

Add a new vertex s adjacent to every vertex of $U \cup V$, obtaining H.

Assign weights

$$w(uv) = x_u \qquad (u \in U,\ v \in V),$$
$$w(su) = y_1 \qquad (u \in U),$$

and

$$w(sv) = y_2 \qquad (v \in V).$$

Let

$$P_H(X, y_1, y_2)$$

be the resulting spanning-tree polynomial.

Also define the elementary symmetric polynomial

$$e_k(X) = \sum_{\substack{S \subseteq U \\ |S| = k}} \prod_{u \in S} x_u.$$

Theorem 371 (k-Matching Count)**.**

$$\boxed{\#M_k(G) = \frac{1}{2^{n_1}} \sum_{x \in \{-1,1\}^U} [y_1^{n_1} y_2^{n_2 - k}] \left(e_k(X) P_H(X, y_1, y_2)\right).}$$

Hence all size-k matchings can be counted in

$$O^*(2^{n_1}) \leq O^*(2^{n/2})$$

time and polynomial space.

Proof. The coefficient

$$[y_1^{n_1} y_2^{n_2-k}]$$

restricts attention to spanning trees containing

- all n_1 edges from s to U;
- exactly $n_2 - k$ edges from s to V.

Such a tree therefore contains exactly k original edges of G.

Fix a term

$$x_S = \prod_{u \in S} x_u, \qquad |S| = k,$$

from $e_k(X)$. For a spanning tree T, the exponent of x_u in

$$x_S P_H$$

is

$$d_T^G(u) + \mathbf{1}[u \in S],$$

where $d_T^G(u)$ counts original G-edges incident to u.

Averaging over

$$x_u \in \{-1, 1\}$$

kills the term unless all these exponents are even.

Since there are exactly k original G-edges,

$$\sum_{u \in U} d_T^G(u) = k.$$

Also

$$|S| = k.$$

The parity condition therefore forces

$$d_T^G(u) = \begin{cases} 1, & u \in S, \\ 0, & u \notin S. \end{cases}$$

The k original edges hence have distinct endpoints in U. They must also have distinct endpoints in V: if

$$u_1 v, \ u_2 v$$

were both chosen, then because the tree already contains

$$s u_1, \ s u_2,$$

the four vertices would contain the cycle

$$s - u_1 - v - u_2 - s.$$

Thus the k original edges form a matching.

Conversely, every k-matching extends uniquely to one such spanning tree by adding

- every edge su for $u \in U$;
- every edge sv for unmatched $v \in V$.

Taking S to be the saturated vertices of U, its parity term contributes exactly 2^{n_1}. □

11.1.3 The BEST Theorem

An Euler tour gives an order to the outgoing edges at every vertex. Conversely, prescribing these local orders tells us how to walk: always take the next unused outgoing edge. The only obstruction is that the walk might close before using all edges.

The remedy is to reserve one outgoing edge at each nonroot vertex for its last exit. If these reserved edges form a tree directed toward the root, any region containing unused edges still has a route back to the root. This turns a tour into one arborescence and independent local permutations.

Let $G = (V, E)$ be a finite strongly connected directed multigraph with at least one edge, and suppose every vertex is balanced: $d^+(v) = d^-(v) = d_v$. Parallel edges are distinct; loops are allowed and contribute one to both degrees. Fix an edge e_0 leaving a root r. An Euler tour here is a sequence of all edges, each used once, forming a directed closed walk beginning with e_0. Let τ_r count spanning arborescences directed toward r. For a one-vertex graph, the empty arborescence is counted once.

Theorem 372 (BEST)**.** *The number of Euler tours beginning with the prescribed edge e_0 is*

$$\boxed{\tau_r \prod_{v \in V} (d_v - 1)!.}$$

Proof. We construct a bijection between such tours and an arborescence together with one ordering of $d_v - 1$ edges at every vertex.

A tour determines its last-exit tree. For each $v \neq r$, select the last outgoing edge e_v used at v. If e_v leads to $w \neq r$, the tour must leave w afterward, since it ends at r. Thus the last-exit time of w is strictly later than that of v. These times cannot strictly increase around a directed cycle. Every nonroot vertex has one selected outgoing edge, so following selected edges must terminate at r. They therefore form an arborescence toward r.

Record the outgoing-edge order at each vertex. At $v \neq r$, its last edge is now fixed as e_v, while the other $d_v - 1$ edges may be listed in any order. At r, the first edge is fixed as e_0, leaving its other $d_r - 1$ edges to order.

The last-exit tree prevents premature stopping. Conversely, choose an arborescence T toward r and such local orders. At each nonroot vertex place its tree edge last; at r place e_0 first. Start at r and repeatedly take the next edge in the current vertex's list, stopping when that list is exhausted. No edge is repeated, so the process stops after at most $|E|$ steps.

It cannot stop at $v \neq r$. At such an endpoint, the used trail has entered v once more than it has left. If all d_v outgoing edges had been used, at least $d_v + 1$ incoming edges would have been used, contradicting $d^-(v) = d_v$. Thus it stops at r and the used trail is closed.

Suppose some edges are still unused. The unused graph is balanced, because removing a closed trail subtracts equally many incoming and outgoing edges at every vertex. Choose a vertex v with an unused outgoing edge. It is not r, where the walk stopped. Its tree edge is still unused, because that edge was placed last. The head of this edge has an unused incoming edge, and hence, by balance, an unused outgoing edge. Repeating along T reaches r and forces an unused outgoing edge there, a contradiction.

Thus all edges are used. Their order at each vertex is exactly the prescribed list, and their last-exit tree is T. The two constructions are inverse. There are τ_r choices of T and $(d_v - 1)!$ independent choices of local order at each vertex, proving the formula. □

The bijection. Given the tree and local permutations, recovering the tour needs only one cursor per outgoing-edge list.

Algorithm 67 Recovering an Euler tour from last exits and local orders

Input: G, r, e_0 as above; arborescence T toward r; for each v, a permutation σ_v of its nonfixed outgoing edges

Output: The unique Euler tour with these last exits and local orders

for each $v \in V$ **do**
  if $v = r$ **then**
    $L_v \leftarrow [e_0] \,\|\, \sigma_v$
  else
    Let e_v be the outgoing edge of v in T
    $L_v \leftarrow \sigma_v \,\|\, [e_v]$
  $c_v \leftarrow 1$
$v \leftarrow r$; $\mathcal{E} \leftarrow$ an empty list
while $c_v \le d_v$ **do**
  $e \leftarrow L_v[c_v]$; $c_v \leftarrow c_v + 1$
  Append e to $\mathcal{E}$; $v \leftarrow \mathrm{head}(e)$
return $\mathcal{E}$

Here $\|$ denotes list concatenation. Each edge is stored once and read once, giving $O(|V| + |E|)$ time and space. Strong connectivity supplies an inward arborescence, for example by searching from r in the graph with all arcs reversed and then reversing the search tree's arcs.

Counting tours by a determinant. Let a_{uv} be the number of arcs $u \to v$, including a_{vv} loops, and put

$$L_{\mathrm{out}} = \mathrm{diag}(d_v) - (a_{uv})_{u,v \in V}.$$

By the Matrix-Tree theorem,

$$\boxed{\#\{\text{Euler tours beginning with } e_0\} = \det L_{\mathrm{out}}[\bar{r}, \bar{r}] \prod_{v \in V} (d_v - 1)!, \qquad \bar{r} = V \setminus \{r\}.}$$

Thus construct the Laplacian, delete the root row and column, evaluate one determinant, and multiply the factorials. For $n = |V|$ and $m = |E|$, This gives $O(m + n^{\omega})$ arithmetic operations.

If only the starting vertex r is fixed, there are d_r possible first edges, so multiply the theorem's answer by d_r. If tours are identified up to cyclic rotation of their edge sequences, no extra factor is needed: every cyclic tour has exactly one representative beginning with the particular edge e_0. Parallel edges remain distinct under both conventions.

This also shows that τ_r is independent of the root in a balanced strongly connected digraph. The number of cyclic tours does not depend on the edge chosen to mark its beginning, and the factorial product does not depend on the root. Applying BEST with an edge leaving each root therefore gives the same τ_r.

11.2 Cheeger Inequality

Let

$$G = (V, E, w)$$

be an undirected graph with nonnegative edge weights. Define

$$d_v = \sum_u w_{uv}, \qquad \mathrm{vol}(S) = \sum_{v \in S} d_v,$$

and

$$w(S, \bar{S}) = \sum_{\substack{u \in S \\ v \notin S}} w_{uv}.$$

The conductance of S is

$$\phi(S) = \frac{w(S, \bar{S})}{\min\{\mathrm{vol}(S), \mathrm{vol}(\bar{S})\}},$$

and

$$\phi(G) = \min_{\emptyset \neq S \neq V} \phi(S).$$

Let

$$D = \mathrm{diag}(d_v), \qquad L = D - A, \qquad \mathcal{L} = D^{-1/2} L D^{-1/2}$$

be the normalized Laplacian.

Equivalently, its second eigenvalue satisfies

$$\lambda_2 = \min_{\substack{f \neq 0 \\ \sum_v d_v f_v = 0}} \frac{\sum_{\{u,v\} \in E} w_{uv}(f_u - f_v)^2}{\sum_v d_v f_v^2}.$$

Theorem 373 (Weighted Cheeger Inequality)**.** *For every connected weighted graph,*

$$\boxed{\frac{\lambda_2}{2} \leq \phi(G) \leq \sqrt{2\lambda_2}.}$$

The Easy Direction

Lemma 374.

$$\lambda_2 \leq 2\phi(G).$$

Proof. Take any set S with

$$s = \mathrm{vol}(S) \leq \frac{1}{2}\mathrm{vol}(V).$$

Let

$$T = \mathrm{vol}(V)$$

and define

$$f_v = \begin{cases} 1 - s/T, & v \in S, \\ -s/T, & v \notin S. \end{cases}$$

Then

$$\sum_v d_v f_v = 0.$$

Only crossing edges contribute to the numerator, and the difference across such an edge is 1. Thus

$$f^T L f = w(S, \bar{S}).$$

Meanwhile,

$$\begin{aligned} f^T D f &= s(1 - s/T)^2 + (T - s)(s/T)^2 \\ &= \frac{s(T - s)}{T} \\ &\geq \frac{s}{2}. \end{aligned}$$

Therefore

$$\lambda_2 \leq \frac{w(S, \bar{S})}{s/2} = 2\phi(S).$$

Minimize over S. □

The Sweep Lemma

Lemma 375 (Sweep Cut)**.** *Let $g : V \to \mathbb{R}_{\geq 0}$ be nonzero and suppose*

$$\operatorname{vol}(\operatorname{supp} g) \leq \frac{1}{2}\operatorname{vol}(V).$$

Then some threshold set

$$S_t = \{v : g_v^2 \geq t\}$$

satisfies

$$\boxed{\phi(S_t) \leq \sqrt{2\,\frac{\sum_{\{u,v\}} w_{uv}(g_u - g_v)^2}{\sum_v d_v g_v^2}}.}$$

Proof. We have that

$$\int_0^\infty \operatorname{vol}(S_t)\,dt = \sum_v d_v g_v^2.$$

Similarly,

$$\int_0^\infty w(S_t, \bar{S}_t)\,dt = \sum_{\{u,v\}} w_{uv}|g_u^2 - g_v^2|.$$

Every S_t lies inside $\operatorname{supp} g$, so

$$\operatorname{vol}(S_t) \leq \frac{1}{2}\operatorname{vol}(V).$$

Thus

$$w(S_t, \bar{S}_t) \geq \phi(G)\operatorname{vol}(S_t).$$

Integrating,

$$\phi(G) \leq \frac{\sum_{\{u,v\}} w_{uv}|g_u^2 - g_v^2|}{\sum_v d_v g_v^2}.$$

Now

$$|g_u^2 - g_v^2| = |g_u - g_v|(g_u + g_v),$$

so Cauchy–Schwarz gives

$$\sum_{\{u,v\}} w_{uv}|g_u^2 - g_v^2| \leq \left(\sum_{\{u,v\}} w_{uv}(g_u - g_v)^2\right)^{1/2} \left(\sum_{\{u,v\}} w_{uv}(g_u + g_v)^2\right)^{1/2}.$$

Since

$$(g_u + g_v)^2 \leq 2g_u^2 + 2g_v^2,$$

the second factor is at most

$$\sqrt{2\sum_v d_v g_v^2}.$$

The result follows. □

The Hard Direction

Proof that $\phi(G) \le \sqrt{2\lambda_2}$. Let f be a λ_2 eigenvector of $D^{-1}L$, which is similar to $\mathcal{L}$:

$$Lf = \lambda_2 Df.$$

Define

$$f_+ = \max(f, 0), \qquad f_- = \max(-f, 0).$$

Their supports are disjoint, so at least one has volume at most half of the total volume. Call that function g.

We claim

$$\frac{g^T L g}{g^T D g} \le \lambda_2.$$

For example, if $g = f_+$, then

$$f^T L f_+ = \lambda_2 f^T D f_+ = \lambda_2 f_+^T D f_+.$$

For each edge,

$$(f_u - f_v)(f_{+,u} - f_{+,v}) \ge (f_{+,u} - f_{+,v})^2,$$

so

$$f^T L f_+ \ge f_+^T L f_+.$$

Hence

$$f_+^T L f_+ \le \lambda_2 f_+^T D f_+.$$

The same proof applies to f_-.

Apply Lemma 375 to g:

$$\phi(G) \le \sqrt{2\,\frac{g^T L g}{g^T D g}} \le \sqrt{2\lambda_2}.$$

□

11.2.1 An Explicit Constant-Degree Expander

We give an explicit expander construction on every number of vertices. The basic graph lives on $\mathbb{Z}_m^2$ and uses two translations and two shears.

Recall that for a d-regular graph we use normalized conductance

$$\Phi(G) = \min_{0<|A|\le|V|/2} \frac{|\partial A|}{d|A|}.$$

With the normalized Laplacian convention, Cheeger's inequality is

$$\frac{\Phi(G)^2}{2} \le \lambda_2 \le 2\Phi(G).$$

We will use both directions.

The finite graphs. For $m \ge 2$, let $V_m = \mathbb{Z}_m^2$, and define

$$U(x,y) = (x+1, y), \quad V(x,y) = (x, y+1),$$
$$S(x,y) = (x+y, y), \quad T(x,y) = (x, x+y).$$

All coordinates are taken modulo m.

Let G_m be the undirected multigraph obtained by adding, for every $z \in V_m$, the four edges

$$\{z, Uz\}, \qquad \{z, Vz\}, \qquad \{z, Sz\}, \qquad \{z, Tz\}.$$

Loops and parallel edges are retained, with a loop contributing two to the degree. Since each of the four maps is a permutation, every vertex has two incident edges from each map, so G_m is 8-regular. The translations already make it connected.

Loops cause no difficulty below: they contribute neither to edge boundaries nor to Dirichlet energy, and the proof of Cheeger's inequality is unchanged for multigraphs with loops.

The shear graph on the integer lattice. Consider first the infinite graph Γ on $\mathbb{Z}^2 \setminus \{0\}$ with edges $\{z, Sz\}$ and $\{z, Tz\}$. Thus its neighbors are $S^{\pm 1}z, T^{\pm 1}z$, and Γ is 4-regular.

Lemma 376 (Expansion of the lattice shear graph)**.** *Every finite $A \subseteq \mathbb{Z}^2 \setminus \{0\}$ satisfies*

$$\boxed{|\partial_\Gamma A| \geq |A|.}$$

Proof. Partition the nonzero lattice into

$$Q_1 = \{(x,y) : x > 0,\ y \geq 0\}, \quad Q_2 = \{(x,y) : x \leq 0,\ y > 0\},$$
$$Q_3 = \{(x,y) : x < 0,\ y \leq 0\}, \quad Q_4 = \{(x,y) : x \geq 0,\ y < 0\}.$$

On Q_1, both S and T preserve the region, and their images are disjoint. Indeed, if $(X,Y) = S(x,y)$, then $X > Y$, whereas if $(X,Y) = T(x,y)$, then $Y \geq X$.

Hence, for $A_1 = A \cap Q_1$, the $2|A_1|$ edges $z \to Sz$ and $z \to Tz$ have $2|A_1|$ distinct targets in Q_1. At most $|A_1|$ of these targets belong to A_1, so at least $|A_1|$ of the edges leave A.

The same argument works on Q_3 using S, T. On Q_2 and Q_4, use S^{-1}, T^{-1}. These maps preserve the corresponding region, and their images are disjoint: on Q_2, the two images are distinguished by $X + Y \leq 0$ and $X + Y > 0$, while on Q_4 they are distinguished by $X + Y \geq 0$ and $X + Y < 0$.

The chosen edges for different regions remain inside different regions, so they are distinct. Summing gives

$$|\partial_\Gamma A| \geq \sum_{i=1}^{4} |A \cap Q_i| = |A|.$$

□

Thus the normalized conductance of the 4-regular graph Γ is at least $1/4$. Although Γ is infinite, the proof of the lower bound in Cheeger's inequality applies verbatim to finitely supported functions, since every level set appearing in that proof is finite. Hence every finitely supported $a : \mathbb{Z}^2 \setminus \{0\} \to \mathbb{C}$ satisfies

$$\frac{\sum_z \big(|a(z) - a(Sz)|^2 + |a(z) - a(Tz)|^2\big)}{4\sum_z |a(z)|^2} \geq \frac{(1/4)^2}{2} = \frac{1}{32}.$$

Equivalently,

$$\boxed{\sum_z \big(|a(z) - a(Sz)|^2 + |a(z) - a(Tz)|^2\big) \geq \frac{1}{8}\sum_z |a(z)|^2.} \tag{32}$$

The same inequality holds for complex-valued a by applying the real-valued statement to its real and imaginary parts.

Finite Fourier transform. Put $\omega = e^{2\pi i/m}$. For $f : \mathbb{Z}_m^2 \to \mathbb{C}$, define

$$\widehat{f}(a,b) = \frac{1}{m^2} \sum_{x,y \in \mathbb{Z}_m} f(x,y)\omega^{-ax-by}.$$

Everything we need follows from

$$\frac{1}{m}\sum_{x=0}^{m-1} \omega^{kx} = \begin{cases} 1, & k = 0 \pmod m, \\ 0, & k \neq 0 \pmod m. \end{cases}$$

For $k \neq 0$, this is just the geometric-series identity $(1 - \omega^{km})/(1 - \omega^k) = 0$.

Lemma 377 (Finite Fourier identities)**.** *For every $f : \mathbb{Z}_m^2 \to \mathbb{C}$,*

$$\boxed{\sum_{\xi \in \mathbb{Z}_m^2} |\widehat{f}(\xi)|^2 = \frac{1}{m^2} \sum_{z \in \mathbb{Z}_m^2} |f(z)|^2.} \tag{33}$$

Moreover,

$$\widehat{f \circ U}(a,b) = \omega^a \widehat{f}(a,b), \qquad \widehat{f \circ V}(a,b) = \omega^b \widehat{f}(a,b),$$

and if M is an invertible linear map on $\mathbb{Z}_m^2$, then

$$\widehat{f \circ M}(\xi) = \widehat{f}(M^{-\mathsf{T}} \xi).$$

Proof. Expanding the left side of (33) gives

$$\sum_{\xi} |\widehat{f}(\xi)|^2 = \frac{1}{m^4} \sum_{z,w} f(z)\overline{f(w)} \sum_{\xi} \omega^{-\langle \xi, z-w \rangle}.$$

The inner sum is m^2 when $z = w$ and zero otherwise, by the geometric-series identity in each coordinate. This proves Parseval.

The remaining identities follow directly from the definition by changing variables in the finite sum. □

The matrices of S and T satisfy $S^{\mathsf{T}} = T$. Consequently Parseval, followed by a reindexing of the frequencies, gives

$$\begin{aligned} \frac{1}{m^2} \sum_z |f(z) - f(Sz)|^2 &= \sum_{\xi} |\widehat{f}(\xi) - \widehat{f}(T\xi)|^2, \\ \frac{1}{m^2} \sum_z |f(z) - f(Tz)|^2 &= \sum_{\xi} |\widehat{f}(\xi) - \widehat{f}(S\xi)|^2. \end{aligned} \tag{34}$$

For the translations,

$$\begin{aligned} &\frac{1}{m^2} \sum_z \left(|f(z) - f(Uz)|^2 + |f(z) - f(Vz)|^2 \right) \\ &\qquad = \sum_{(a,b) \in \mathbb{Z}_m^2} \rho(a,b) |\widehat{f}(a,b)|^2, \end{aligned} \tag{35}$$

where

$$\rho(a,b) = |1 - \omega^a|^2 + |1 - \omega^b|^2.$$

Cutting open the torus. The Fourier frequencies live in $\mathbb{Z}_m^2$, while (32) lives on the ordinary lattice $\mathbb{Z}^2$. We connect them by choosing one representative of every frequency.

Let

$$I_m = \{-\lfloor m/2 \rfloor, \dots, \lceil m/2 \rceil - 1\}, \qquad Q_m = I_m^2.$$

Identify $\widehat{f}$ with a function $h : Q_m \to \mathbb{C}$, and extend it by zero outside Q_m, obtaining $\widetilde{h} : \mathbb{Z}^2 \to \mathbb{C}$.

If f has mean zero, then $\widehat{f}(0,0) = m^{-2} \sum_z f(z) = 0$, so $\widetilde{h}(0,0) = 0$. Thus $\widetilde{h}$ is a finitely supported function on $\mathbb{Z}^2 \setminus \{0\}$, exactly as required in (32).

If an integer shear edge has both endpoints in Q_m, its contribution to the energy of $\widetilde{h}$ is exactly the corresponding shear term in (34). We only have to pay for shear edges crossing the boundary of Q_m.

Lemma 378 (Translations pay for wraparound)**.** *With the preceding notation,*

$$\begin{aligned} &\sum_{\xi \in \mathbb{Z}^2} \left(|\widetilde{h}(\xi) - \widetilde{h}(S\xi)|^2 + |\widetilde{h}(\xi) - \widetilde{h}(T\xi)|^2 \right) \\ &\qquad \le \frac{1}{m^2} \sum_{R \in \{U,V,S,T\}} \sum_{z \in V_m} |f(z) - f(Rz)|^2. \end{aligned}$$

Proof. Suppose $(a,b) \in Q_m$ is incident to a shear edge that leaves Q_m. Then one of $a+b$ or $a-b$ lies outside I_m, and hence

$$|a| + |b| \geq \frac{m}{2}.$$

Set $\alpha = \pi|a|/m$ and $\beta = \pi|b|/m$. Then $0 \leq \alpha, \beta \leq \pi/2$ and $\alpha + \beta \geq \pi/2$. Since $\beta \geq \frac{\pi}{2} - \alpha$ and $\sin^2$ is increasing on this interval,

$$\sin^2 \alpha + \sin^2 \beta \geq 1.$$

Using the elementary identity

$$|1 - e^{i\theta}|^2 = 2 - 2\cos\theta = 4\sin^2(\theta/2),$$

we have

$$\rho(a,b) = 4\sin^2 \alpha + 4\sin^2 \beta \geq 4. \tag{1}$$

A vertex of Q_m is incident to at most four crossing shear edges: one in each of the S, S^{-1}, T, T^{-1} directions. Since $\widetilde{h}$ vanishes outside Q_m, their total contribution charged to (a,b) is at most $4|h(a,b)|^2$, which by (1) is at most $\rho(a,b)|h(a,b)|^2$.

Thus the internal shear edges are paid for by (34), while all crossing shear edges are paid for by (35). Summing proves the claim. □

This is the reason the translation generators are useful in the argument: a modular shear may wrap around the boundary of the chosen square, but wraparound can occur only when at least one frequency coordinate is large, and translations have correspondingly large Fourier energy there.

Theorem 379 (Expansion of the square family)**.** *The graph G_m is an explicit 8-regular multigraph on m^2 vertices satisfying*

$$\boxed{|\partial A| \geq \frac{1}{16}|A| \qquad \text{whenever } |A| \leq \frac{m^2}{2}.}$$

Its adjacency lists can be constructed in $O(m^2)$ time.

Proof. Let $f : V_m \to \mathbb{C}$ have mean zero, and let $\widetilde{h}$ be the lifted Fourier transform above. Combining (32) with Lemma 378 and Parseval gives

$$\begin{aligned}\frac{1}{m^2} \sum_{R \in \{U,V,S,T\}} \sum_z |f(z) - f(Rz)|^2 &\geq \frac{1}{8} \sum_{\xi \in \mathbb{Z}^2} |\widetilde{h}(\xi)|^2 \\ &= \frac{1}{8m^2} \sum_z |f(z)|^2.\end{aligned}$$

Hence

$$\sum_{R \in \{U,V,S,T\}} \sum_z |f(z) - f(Rz)|^2 \geq \frac{1}{8} \sum_z |f(z)|^2. \tag{2}$$

The left side is the unnormalized Dirichlet energy of G_m. Since G_m is 8-regular, dividing (2) by $8\sum_z |f(z)|^2$ shows that its normalized spectral gap satisfies

$$\lambda_2(G_m) \geq \frac{1}{64}.$$

The upper bound in Cheeger's inequality, $\lambda_2 \leq 2\Phi(G_m)$, therefore gives

$$\Phi(G_m) \geq \frac{1}{128}.$$

Multiplying by the degree 8 yields

$$\frac{|\partial A|}{|A|} \geq 8\Phi(G_m) \geq \frac{1}{16}$$

for every A of size at most $m^2/2$.

Each vertex and its four generator images are computed using $O(1)$ modular additions, so the graph is constructed in $O(m^2)$ time. □

Exactly N vertices. The square restriction is easily removed. Contracting blocks of size one or two preserves the edge-expansion constant exactly; only the degree increases from 8 to 16.

Theorem 380 (Explicit expanders on every number of vertices)**.** *For every $N \geq 2$, one can construct in $O(N)$ time a 16-regular multigraph H_N on exactly N vertices such that*

$$\boxed{|\partial A| \geq \frac{1}{16}|A| \qquad \text{whenever } |A| \leq \frac{N}{2}.}$$

Proof. Let $m = \lceil \sqrt{N} \rceil$ and $M = m^2$. Then $N \leq M \leq 2N$. Indeed $N \geq (m-1)^2 + 1$, so

$$2N - m^2 \geq 2(m-1)^2 + 2 - m^2 = (m-2)^2 \geq 0.$$

Construct G_m, which has M vertices. Partition its vertices into N nonempty blocks, each of size one or two: take $M - N$ blocks of size two and $2N - M$ blocks of size one. Contract each block to a single vertex, retaining loops and parallel edges.

A block B has degree $8|B|$ after contraction, since degrees add and internal edges become loops. Thus contracted vertices have degree either 8 or 16. Add four loops to each degree-8 vertex. Loops contribute eight to its degree and do not change any cut, so the resulting multigraph H_N is 16-regular.

It remains to check expansion. Let $A \subseteq V(H_N)$ with $|A| \leq N/2$, and let $\widetilde{A} \subseteq V(G_m)$ be the union of the blocks corresponding to A. Contraction preserves exactly the edges crossing the cut, so

$$|\partial_{H_N} A| = |\partial_{G_m} \widetilde{A}|.$$

The expansion theorem for G_m, applied to the smaller of $\widetilde{A}$ and its complement, gives

$$|\partial_{G_m} \widetilde{A}| \geq \frac{1}{16} \min\{|\widetilde{A}|, M - |\widetilde{A}|\}.$$

Every block is nonempty, so $|\widetilde{A}| \geq |A|$. The complement contains $N - |A|$ nonempty blocks, and therefore $M - |\widetilde{A}| \geq N - |A| \geq |A|$. Hence

$$|\partial_{H_N} A| \geq \frac{1}{16}|A|.$$

Finally $M \leq 2N$, so constructing G_m, forming the blocks, contracting them, and adding loops all take $O(N)$ time. □

11.3 Expander Products

The preceding construction gives a constant-degree expander directly. Graph products let us improve expansion, change the number of vertices, and control the degree in separate steps. The basic operation is squaring: two independent steps of a D-regular walk replace its nontrivial eigenvalues by their squares, but increase the degree to D^2. Repeated squaring quickly makes the graph dense.

We will use a small expander on the D edge labels to reduce this cost. Derandomized squaring uses it to correlate the two edge choices, giving degree Dd instead of D^2. The zig-zag product makes the edge label part of the vertex and needs only two choices from $[d]$, giving degree d^2 on a larger vertex set. Both use the decomposition into averages over edge labels and deviations from those averages.

11.3.1 Lifting a Walk to Edge Labels

We allow regular undirected multigraphs and count each loop twice in the degree. Assign the D incident edge ends at each vertex labels in $[D]$, called *ports*. The rotation map $\mathrm{Rot}_G : V \times [D] \to V \times [D]$ sends (v, a) to (w, b) when these are the two ends of the same edge. It is an involution without fixed points; a loop exchanges two distinct ports at the same vertex. Recording the arrival port matters because the next step will use it to select another edge.

Write $M_G = A_G/D$ and $\mathcal{L}_G = I - M_G$. We measure contraction by $\eta(G) = \|M_G|_{\mathbf{1}^\perp}\|$. Thus $\|M_G x\| \le \eta(G)\|x\|$ for every mean-zero x, and $\eta(G^2) = \eta(G)^2$. The absolute value in this definition is necessary: an eigenvalue near -1 also becomes near 1 on squaring. To obtain absolute contraction from a normalized Laplacian gap, make the walk lazy. The matrix $(I + M_G)/2$ has nonnegative eigenvalues and contraction $1 - \lambda_2(\mathcal{L}_G)/2$. If D is even, adding $D/2$ loops at each vertex realizes it as a $2D$-regular graph.

The set $\{v\} \times [D]$ is the *cloud* above v. Lift a vector on vertices to a cloud-constant vector by the isometry $Q : \mathbb{R}^V \to \mathbb{R}^{V \times [D]}$, defined by $(Qx)_{v,a} = x_v/\sqrt{D}$. Its adjoint satisfies $(Q^\top y)_v = D^{-1/2} \sum_a y_{v,a}$, so $Q^\top Q = I$ and $P = QQ^\top$ replaces the entries in each cloud by their average. Every port vector has the orthogonal decomposition $y = Py + (I - P)y$: its cloud averages and its within-cloud fluctuations.

Let R be the permutation matrix of the rotation. Reversing an edge twice does nothing, so $R = R^\top$ and $R^2 = I$. Lifting a vertex vector, following each port, and averaging at the other end is exactly one step of the original walk:

$$Q^\top R Q = M_G.$$

The rotation itself preserves norm. The contraction of M_G occurs when the final averaging discards variation among ports of the same vertex. This observation will account for the gain in both products.

Now let H be a d-regular graph on $[D]$, and put $W = I_V \otimes M_H$. This takes an H-step independently within each cloud. It preserves cloud averages and contracts fluctuations by at most $\beta = \eta(H)$.

More explicitly, decompose a port vector as $y = Py + (I - P)y$. The first part is constant within each cloud, while the second has mean zero in every cloud. The walk on H fixes the first part and contracts the second by at most β. It also preserves each cloud's average, so $WP = PW = P$.

Thus, setting $E = W - P$, we have $W = P + E$ and $EP = PE = 0$. Moreover,

$$\|Ey\| = \|W(I - P)y\| \le \beta\|(I - P)y\| \le \beta\|y\|,$$

11.3.2 Derandomized Squaring

An ordinary two-step walk first follows a uniformly chosen port of G, then chooses a fresh uniform port at the intermediate vertex. On the space of ports, this fresh choice is the averaging operator P. Indeed, $M_G^2 = Q^\top RPRQ$. Replace that averaging by one step of H: after arriving at port b, choose one of its d neighbors in H and use that port for the second G-step. The resulting walk is $N = Q^\top RWRQ$. It uses one choice from $[D]$ and one from $[d]$, so its graph has the original vertex set and degree Dd.

Algorithm 68 Rotation in the Derandomized Square

Input: Vertex v and edge label $(a, i) \in [D] \times [d]$
Output: The opposite endpoint and its reverse edge label

1: $(w, b) \leftarrow \mathrm{Rot}_G(v, a)$
2: $(c, j) \leftarrow \mathrm{Rot}_H(b, i)$
3: $(u, e) \leftarrow \mathrm{Rot}_G(w, c)$
4: **return** $(u, (e, j))$

Reversing the path follows the same three rotations in reverse order. A fixed port of the product would force the middle H-rotation to have a fixed port, so the product respects our convention for loops.

The approximation error only sees the fluctuations discarded by P. For $y = RQx$, its squared norm is $\|x\|^2$, while its cloud-constant part has squared norm $\|Q^\top y\|^2 = \|M_G x\|^2$. The available error is therefore proportional to $\|x\|^2 - \|M_G x\|^2$. This is smaller than $\|x\|^2$ precisely when the first walk step preserves most of the vector's norm.

Theorem 381 (Derandomized Squaring)**.** *If $\eta(H) \le \beta \le 1$, then, for every real vector x,*

$$|x^\top (N - M_G^2)x| \le \beta\, x^\top (I - M_G^2)x.$$

Consequently, $(1-\beta)(I-M_G^2) \preceq I-N \preceq (1+\beta)(I-M_G^2)$*, and*

$$\boxed{\eta(G \operatorname{dsq} H) \le \eta(G)^2 + \beta\big(1-\eta(G)^2\big).}$$

Proof. Put $y = RQx$. Since $W-P$ vanishes on cloud constants,

$$\begin{aligned}|x^\top (N-M_G^2)x| &= |y^\top (W-P)y| \\ &\le \beta\|(I-P)y\|^2 = \beta\big(\|x\|^2 - \|M_G x\|^2\big).\end{aligned}$$

Rearranging gives the two Loewner inequalities. For $x \perp \mathbf{1}$, the same estimate and $x^\top M_G^2 x = \|M_G x\|^2$ give $|x^\top N x| \le \beta\|x\|^2 + (1-\beta)\|M_G x\|^2 \le [\beta + (1-\beta)\eta(G)^2]\|x\|^2$. Since N is symmetric, maximizing over unit x proves the contraction bound. □

In particular, if $\gamma = 1-\eta(G)$, then the new absolute gap is at least $(1-\beta)\gamma(2-\gamma)$. For a sufficiently small γ and a fixed $\beta < 1/2$, one step increases the gap by a constant factor. An additive error estimate $\|N - M_G^2\| \le \beta$ would lose this conclusion when $\gamma \ll \beta$. The relative estimate also says that every quadratic form of the normalized Laplacian of G^2 is preserved within a factor $1 \pm \beta$.

11.3.3 The Zig-Zag Product

Derandomized squaring still pays for one arbitrary choice among the D ports. Zig-zag avoids that choice by retaining the port as part of the vertex. Its vertex set is $V \times [D]$. From (v, a), take an H-step to select a nearby port, follow the corresponding edge of G, and take another H-step in the arrival cloud. The middle step is determined by the current port; only the two small steps require choices. Thus G zigzag H has degree d^2 and walk matrix WRW.

Algorithm 69 Rotation in the Zig-Zag Product

Input: Vertex (v, a) and edge label $(i, j) \in [d]^2$
Output: The opposite endpoint and its reverse edge label
1: $(a', i') \leftarrow \mathrm{Rot}_H(a, i)$
2: $(w, b') \leftarrow \mathrm{Rot}_G(v, a')$
3: $(b, j') \leftarrow \mathrm{Rot}_H(b', j)$
4: **return** $((w, b), (j', i'))$

The reverse label uses the two small steps in reverse order. As with derandomized squaring, this gives an involution without fixed ports; here a fixed port would force the middle G-rotation to be fixed.

The two small steps have different roles. The first contracts any variation already present within clouds. If a mean-zero vector is instead constant on clouds, the first step leaves it alone. But its cloud averages after the rotation are governed by G: for such a vector u, $\|PRu\| \le \alpha\|u\|$, where $\alpha = \eta(G)$. Since R preserves norm, at least a $1-\alpha^2$ fraction of its squared norm has become within-cloud variation. The second small step contracts this newly created variation.

Theorem 382 (Zig-Zag Contraction)**.** *If* $\eta(G) \le \alpha \le 1$ *and* $\eta(H) \le \beta \le 1$*, then*

$$\eta(G \operatorname{zigzag} H) \le f(\alpha, \beta) := \frac{(1-\beta^2)\alpha + \sqrt{(1-\beta^2)^2\alpha^2 + 4\beta^2}}{2} \le \alpha + \beta - \alpha\beta.$$

In particular, the product has absolute gap at least $(1-\alpha)(1-\beta)$*, and hence at least this normalized Laplacian gap.*

Proof. Work throughout on $\mathbf{1}^\perp$. The compression bound $\|PRP|_U\| \le \alpha$ means that $\|PRu\| \le \alpha\|u\|$ for every $u \in U$.

First replace W by $K = P + \beta(I - P)$, which fixes cloud constants and contracts every fluctuation by exactly β. If $\beta = 0$, then $W = P$ and the desired bound is simply $\|PRP\| \le \alpha$, so assume $\beta > 0$. For any x, put $y = Px + Ex/\beta$. Then $Ky = Wx$, while

$$\|y\|^2 = \|Px\|^2 + \frac{\|Ex\|^2}{\beta^2} \le \|Px\|^2 + \|(I - P)x\|^2 = \|x\|^2.$$

Thus $|x^\top WRWx| = |y^\top KRKy| \le \|KRK\|\,\|x\|^2$, proving $\|WRW\| \le \|KRK\|$.

It remains to bound the eigenvalues of KRK. Since $K^{-1}(KRK)K = RK^2$, every such eigenvalue λ has a nonzero vector z satisfying $RK^2z = \lambda z$. The advantage is that $K^2 = \beta^2 I + (1 - \beta^2)P$.

Put $u = Pz$. If $u = 0$, then the eigenvalue equation reduces to $\beta^2 Rz = \lambda z$. Since R preserves norms, this gives $|\lambda| = \beta^2 \le \beta \le f(\alpha, \beta)$.

Now suppose $u \neq 0$. Projecting $RK^2z = \lambda z$ onto U gives the first identity below. Multiplying by R before projecting, and using $R^2 = I$, gives the second:

$$\lambda u = \beta^2 PRz + (1 - \beta^2)PRu, \qquad \lambda PRz = u.$$

Multiply the first identity by λ and substitute the second to eliminate PRz. This gives $(\lambda^2 - \beta^2)u = \lambda(1 - \beta^2)PRu$.

Since $Pu = u$, we have $|u^\top Ru| = |u^\top PRu| \le \|u\|\,\|PRu\| \le \alpha\|u\|^2$. Taking the inner product of the preceding identity with u therefore yields

$$(\lambda^2 - \beta^2)\|u\|^2 = \lambda(1 - \beta^2)u^\top Ru \le |\lambda|(1 - \beta^2)\alpha\|u\|^2.$$

Canceling $\|u\|^2 > 0$ gives $|\lambda|^2 - (1 - \beta^2)\alpha|\lambda| - \beta^2 \le 0$. Hence $|\lambda|$ is at most the positive root $f(\alpha, \beta)$. Since KRK is symmetric, this proves $\|WRW\| \le \|KRK\| \le f(\alpha, \beta)$.

Finally, $a \mapsto f(a, \beta)$ is convex: its square-root term is the Euclidean norm of $((1 - \beta^2)a, 2\beta)$. Using $f(0, \beta) = \beta$ and $f(1, \beta) = 1$, we obtain

$$f(\alpha, \beta) \le (1 - \alpha)f(0, \beta) + \alpha f(1, \beta) = \alpha + \beta - \alpha\beta.$$

□

11.4 Alon–Boppana

We now return to simple graphs and unnormalized adjacency matrices. Write $\theta_1(G) \ge \theta_2(G) \ge \cdots$ for the adjacency eigenvalues. A d-regular graph has $\theta_1 = d$; good expansion asks that the other eigenvalues be much smaller. Alon–Boppana gives a limit on how far this can go: for fixed d and growing order, θ_2 cannot stay below $2\sqrt{d - 1}$.

The constant comes from a tree. After the first step away from a root, a d-regular tree has $d - 1$ forward choices at each vertex. On vectors constant on levels, normalization turns this branching into a half-infinite path with edge weight $\sqrt{d - 1}$. We first see this on the binary tree, then solve the finite path to obtain the right test vector. Finally, we place two such vectors in distant graph balls. The balls may contain cycles; the covering tree will let us transfer the vectors without losing their Rayleigh quotient.

11.4.1 The Infinite Binary Tree

We only need finitely supported vectors; the norm below is the supremum of their absolute Rayleigh quotients. Let L_j be level j of the rooted infinite binary tree. It has 2^j vertices. To put one unit of squared norm on a level, give each of its vertices value $2^{-j/2}$; thus $e_j = 2^{-j/2}\mathbf{1}_{L_j}$ form an orthonormal sequence. There are 2^{j+1} edges between levels j and $j + 1$, so $\langle e_{j+1}, Ae_j \rangle = \sqrt{2}$. Consequently, for $f = \sum_j c_j e_j$ of finite support,

$$\|f\|^2 = \sum_j c_j^2, \qquad f^\top Af = 2\sqrt{2}\sum_j c_j c_{j+1}.$$

These are exactly the norm and quadratic form of a half-infinite path with edge weight $\sqrt{2}$. In particular, the profile $c_0 = \cdots = c_r = 1$ has Rayleigh quotient $2\sqrt{2}\, r/(r+1)$, approaching $2\sqrt{2}$.

To bound arbitrary vectors, orient the edges from parent to child and use $2|ab| \le ta^2 + t^{-1}b^2$ for $t > 0$. Each non-root vertex occurs as a parent twice and as a child once, so its total coefficient is $2t + t^{-1}$. The best choice balances these contributions: $t = 1/\sqrt{2}$, giving $2\sqrt{2}$. The root receives a smaller coefficient. Summing over edges yields $|f^\top A f| \le 2\sqrt{2}\|f\|^2$ for every finitely supported f. Together with the level-constant vectors above, this proves that the adjacency operator has norm exactly $2\sqrt{2}$.

The same argument explains the general tree bound.

Lemma 383 (Tree Norm Bound)**.** *A finite tree of maximum degree at most $d \ge 2$ has adjacency norm at most $2\sqrt{d-1}$.*

Proof. Root the tree and put $q = d - 1$. Every non-root vertex has at most q children, so the preceding edge inequality gives coefficient at most $qt + t^{-1}$. Choose $t = 1/\sqrt{q}$, making this $2\sqrt{q}$. The root has at most d children and receives coefficient at most $d/\sqrt{q} \le 2\sqrt{q}$. Thus $|x^\top A_T x| \le 2\sqrt{q}\|x\|^2$ for all x. □

Lemma 384 (Finite Branching Tree)**.** *For integers $q \ge 1$ and $r \ge 0$, let $T_{q,r}$ be the rooted tree of depth r in which every non-leaf vertex has q children. Then*

$$\rho(T_{q,r}) = 2\sqrt{q}\cos\frac{\pi}{r+2}.$$

Proof. The vectors $e_j = q^{-j/2}\mathbf{1}_{L_j}$ identify the level-constant subspace with a path on $r+1$ vertices whose edge weights are $\sqrt{q}$. Maximizing its quadratic form subject to $\sum_j c_j^2 = 1$ gives the eigenvalue equation

$$\sqrt{q}\,(c_{j-1} + c_{j+1}) = \rho c_j \quad (0 \le j \le r), \qquad c_{-1} = c_{r+1} = 0.$$

If it was infinite we could select complex phases; we need to cancel two of these to get $c_{-1} = c_{r+1} = 0$, producing sines. For $\rho = 2\sqrt{q}\cos\vartheta$, the recurrence is the addition formula for sine and cosine. The left boundary selects $c_j = \sin((j+1)\vartheta)$ up to scale; the right boundary requires $\sin((r+2)\vartheta) = 0$. Taking the smallest positive angle, $\vartheta = \pi/(r+2)$, makes all c_j positive. Thus $f = \sum_j c_j e_j$ is a positive eigenvector of the whole tree with the claimed eigenvalue ρ.

The positive eigenvector f also certifies that ρ is the full adjacency norm. Indeed, let $A_T x = \lambda x$ with $x \ne 0$, and choose u maximizing $|x(u)|/f(u)$. Then

$$|\lambda|\,|x(u)| \le \sum_{v\sim u} |x(v)| \le \frac{|x(u)|}{f(u)} \sum_{v\sim u} f(v) = \rho|x(u)|.$$

Since $|x(u)| > 0$, we obtain $|\lambda| \le \rho$. As ρ itself is an eigenvalue, $\|A_T\| = \rho$. □

This loses only $O(\sqrt{q}/r^2)$ from $2\sqrt{q}$, since $1 - \cos t \le t^2/2$.

11.4.2 From a Tree to a Graph Ball

A large minimum degree supplies many possible continuations of a walk, but they need not reach distinct vertices. We therefore first keep the walks distinct, obtaining a tree, and only then identify walks with the same endpoint. To preserve the denominator of the Rayleigh quotient, combine their values by adding squares and taking a square root. Local injectivity of the covering map makes the numerator increase under this operation.

Lemma 385 (A Spectral Witness in Every Ball)**.** *For every vertex v of a graph of minimum degree at least $d \ge 2$ and every integer $r \ge 0$, there is a nonzero nonnegative vector f supported in $B_G(v, r)$ such that $f^\top A_G f \ge 2\sqrt{d-1}\cos(\pi/(r+2))\|f\|^2$.*

Proof. Form a rooted tree whose vertices are the nonbacktracking walks starting at v, including the empty walk. Here nonbacktracking means that a step never immediately reverses the preceding step; later visits to an old graph vertex are allowed. The parent of a nonempty walk deletes its last step. The root has at least d children, and every other vertex has at least $d-1$ children. Keep exactly $d-1$ children at each vertex and stop at depth r. This leaves $T = T_{d-1,r}$, with its positive eigenvector g from Lemma 384.

Map each tree vertex a to its walk's endpoint $\pi(a)$. Define $f(u) = (\sum_{a:\,\pi(a)=u} g(a)^2)^{1/2}$. Then $\|f\|^2 = \|g\|^2$ and $\operatorname{supp} f \subseteq B_G(v,r)$. To compare quadratic forms, fix a graph edge uv. Its preimages form a matching between the two fibers: at a tree vertex above u, there is at most one incident tree edge above the specified edge uv. For a non-root vertex, that edge is either its parent edge or its unique allowed extension toward v, but cannot be both. Cauchy–Schwarz therefore gives

$$\sum_{ab\in E(T):\,\pi(a)=u,\,\pi(b)=v} g(a)g(b) \le \left(\sum_{\pi(a)=u} g(a)^2\right)^{1/2} \left(\sum_{\pi(b)=v} g(b)^2\right)^{1/2} = f(u)f(v).$$

Summing over graph edges and multiplying by two proves $f^\top A_G f \ge g^\top A_T g = 2\sqrt{d-1}\cos(\pi/(r+2))\|f\|^2$. □

One such vector only lower-bounds θ_1. To reach θ_2, we need a large Rayleigh quotient after removing the top eigendirection. Two witnesses with disjoint, nonadjacent supports provide a two-dimensional space of large Rayleigh quotients; some vector in that space is orthogonal to a top eigenvector. Vertices far apart provide exactly these supports.

Theorem 386 (Alon–Boppana from Diameter)**.** *If a connected graph of minimum degree at least $d \ge 2$ has diameter $D \ge 2r+2$ for an integer $r \ge 0$, then*

$$\boxed{\theta_2(G) \ge 2\sqrt{d-1}\cos\frac{\pi}{r+2}.}$$

More generally, k vertices at pairwise distance at least $2r+2$ give $\theta_k(G) \ge 2\sqrt{d-1}\cos(\pi/(r+2))$. For fixed $d \ge 3$, this implies $\theta_2(G) \ge 2\sqrt{d-1} - O_d((\log n)^{-2})$ for d-regular graphs as $n \to \infty$.

Proof. Choose centers at distance at least $2r+2$. Their radius-r balls are disjoint and no edge joins two balls: such an edge would give a path of length at most $2r+1$ between their centers. Let f_i be the witnesses from Lemma 385. Both $f_i^\top f_j$ and $f_i^\top A_G f_j$ vanish for $i \ne j$. Thus the Rayleigh quotient of any nonzero $\sum_i a_i f_i$ is

$$\frac{\sum_i a_i^2 f_i^\top A_G f_i}{\sum_i a_i^2 \|f_i\|^2} \ge 2\sqrt{d-1}\cos\frac{\pi}{r+2}.$$

The min–max principle now gives the kth-eigenvalue bound, including $k=2$ for two diameter endpoints.

A connected graph of maximum degree d and diameter D has at most $1 + d\sum_{j=0}^{D-1}(d-1)^j$ vertices: count a breadth-first search from one vertex. For fixed $d \ge 3$, this gives $D \ge \log_{d-1} n - O_d(1)$. Take $r = \lfloor (D-2)/2 \rfloor$ and use $1 - \cos t \le t^2/2$ to obtain the asymptotic bound. If the regular graph is disconnected, d is an eigenvalue at least twice, so $\theta_2 = d$ and the bound also holds. □

The same argument gives many large eigenvalues. Greedily choose centers at pairwise distance at least $2r+2$. Each chosen center excludes at most $1 + d\sum_{j=0}^{2r}(d-1)^j$ vertices, so at least $n/(1 + d\sum_{j=0}^{2r}(d-1)^j)$ eigenvalues satisfy the displayed lower bound. For any fixed d and $\epsilon > 0$, choosing r large enough shows that a positive fraction of the eigenvalues are at least $2\sqrt{d-1} - \epsilon$.

A connected d-regular graph is called *Ramanujan* if every adjacency eigenvalue other than d and, in the bipartite case, $-d$, has absolute value at most $2\sqrt{d-1}$. Alon–Boppana explains the threshold: no smaller uniform bound can hold for an infinite family of fixed-degree graphs.

11.5 Lower Bound on Spectral Sparsification

Theorem 387 (Weighted Spectral Sparsifier Lower Bound)**.** *Let H be a nonnegatively weighted graph on $n \geq 2$ vertices with m positive-weight edges. If $(1-\epsilon)L_{K_n} \preceq L_H \preceq (1+\epsilon)L_{K_n}$ for $0 \leq \epsilon < 1$, then*

$$\boxed{m \geq \frac{n(n-1)(1-\epsilon^2)}{2\big(1+(n-1)\epsilon^2\big)}.}$$

In particular, for every $0 < \epsilon < 1$, one needs $m = \Omega\big(\min\{n^2, n/\epsilon^2\}\big)$.

Proof. Merging parallel edges and discarding loops only decreases m and does not change the Laplacian, so assume H is simple. Write $L = L_H$, let d_v be its weighted degrees and w_e its edge weights, and put $s = \operatorname{tr} L = \sum_v d_v = 2\sum_e w_e$. Since $\epsilon < 1$, we have $m > 0$. The diagonal entries of L are d_v and its two entries on each edge are $-w_e$, so Cauchy–Schwarz gives

$$\operatorname{tr}(L^2) = \sum_v d_v^2 + 2\sum_e w_e^2 \geq \frac{s^2}{n} + \frac{s^2}{2m}.$$

Thus fixing the trace and edge count already forces this much squared Frobenius norm.

On the other hand, all $n-1$ eigenvalues of $L - n\Pi$ on $\mathbf{1}^\perp$ lie in $[-n\epsilon, n\epsilon]$, and it vanishes on constants. Using $L\Pi = L$, we obtain

$$\begin{aligned} n^2(n-1)\epsilon^2 &\geq \|L - n\Pi\|_F^2 \\ &= \operatorname{tr}(L^2) - 2ns + n^2(n-1) \\ &\geq \left(\frac{1}{n} + \frac{1}{2m}\right) s^2 - 2ns + n^2(n-1) \\ &\geq n^2 \frac{n(n-1) - 2m}{2m+n}. \end{aligned}$$

For the last inequality, minimize the quadratic in s, or use $as^2 - 2ns \geq -n^2/a$ with $a = 1/n + 1/(2m)$. Rearranging proves the displayed bound, which gives the claimed order when $\epsilon \leq 1/2$. For $\epsilon > 1/2$, approximation with error less than one forces H to be connected, so $m \geq n - 1 = \Omega(n/\epsilon^2)$. □

12 Electrical Flows

Let

$$G = (V, E, c)$$

be a connected undirected graph with positive edge conductances

$$c_e > 0, \qquad r_e = \frac{1}{c_e}$$

the corresponding resistances.

Fix an arbitrary orientation of the edges. For an oriented edge $e = (u, v)$, write

$$b_e = \mathbf{1}_u - \mathbf{1}_v.$$

Let

$$B \in \mathbb{R}^{E \times V}$$

have rows $b_e^\top$, and let

$$C = \operatorname{diag}(c_e).$$

Then the weighted Laplacian is

$$\boxed{L = B^\top C B.}$$

For a flow $\theta \in \mathbb{R}^E$,

$$B^\top \theta$$

is its divergence vector, and its energy is

$$\boxed{\mathcal{E}(\theta) = \theta^\top C^{-1} \theta = \sum_{e \in E} r_e \theta_e^2.}$$

12.1 Electrical Flows

More generally than s–t flows, let

$$b \in \mathbb{R}^V, \qquad \mathbf{1}^\top b = 0.$$

A flow realizing demand b satisfies

$$B^\top \theta = b.$$

The familiar unit s–t case is

$$b = \mathbf{1}_s - \mathbf{1}_t.$$

The form of the solution to electrical flows can be seen by applying Lagrange multipliers to the optimization problem. Thereby we can see that there exists some $\phi \in \mathbb{R}^n$ such that $\theta = CB\phi$, then we know $L\phi = b$ so $\phi = L^+ b$ is natural.

Theorem 388 (Thomson's Principle)**.** *For every demand $b \perp \mathbf{1}$, the unique minimum-energy flow satisfying*

$$B^\top \theta = b$$

is

$$\boxed{\theta^\star = CBL^+ b.}$$

Its energy is

$$\boxed{\mathcal{E}(\theta^\star) = b^\top L^+ b.}$$

Proof. First,

$$B^\top \theta^\star = B^\top CBL^+ b = LL^+ b = b,$$

since $b \perp \mathbf{1} = \ker L$.

Now let θ be any other feasible flow and write

$$\theta = \theta^\star + z.$$

Then

$$B^\top z = 0.$$

The energy expands as

$$\mathcal{E}(\theta) = \mathcal{E}(\theta^\star) + 2(\theta^\star)^\top C^{-1} z + \mathcal{E}(z).$$

The cross term vanishes:

$$(\theta^\star)^\top C^{-1} z = (CBL^+ b)^\top C^{-1} z = (L^+ b)^\top B^\top z = 0.$$

Therefore

$$\mathcal{E}(\theta) = \mathcal{E}(\theta^\star) + \mathcal{E}(z) \geq \mathcal{E}(\theta^\star),$$

with equality only for $z = 0$.

Finally,

$$\begin{aligned}\mathcal{E}(\theta^\star) &= b^\top L^+ B^\top C C^{-1} C B L^+ b \\ &= b^\top L^+ L L^+ b \\ &= b^\top L^+ b.\end{aligned}$$

□

12.1.1 Potentials and Ohm's Law

A potential

$$\phi : V \to \mathbb{R}$$

induces the flow

$$\theta = CB\phi,$$

or edgewise

$$\boxed{\theta_{uv} = c_{uv}(\phi_u - \phi_v).}$$

Thus Theorem 388 says that the electrical flow for demand b is induced by the potential

$$\boxed{\phi = L^+ b.}$$

Potentials are unique up to adding a constant.

12.1.2 Effective Resistance

For vertices s, t, define

$$b_{st} = \mathbf{1}_s - \mathbf{1}_t.$$

Definition 389 (Effective Resistance)**.**

$$\boxed{R_{\text{eff}}(s,t) = b_{st}^\top L^+ b_{st}.}$$

By Thomson's principle,

$$\boxed{R_{\text{eff}}(s,t) = \min_{\theta:\, B^\top \theta = b_{st}} \mathcal{E}(\theta).}$$

If

$$\phi = L^+ b_{st}$$

is the unit-current potential, then

$$\boxed{R_{\text{eff}}(s,t) = \phi_s - \phi_t = \mathcal{E}(\theta^\star).}$$

Indeed,

$$\phi_s - \phi_t = b_{st}^\top L^+ b_{st}.$$

12.1.3 Maximum Principle

Lemma 390 (Maximum Principle)**.** *Let ϕ be harmonic on*

$$V \setminus S,$$

meaning

$$(L\phi)_v = 0 \qquad (v \notin S).$$

Then every maximum and minimum of ϕ is attained on S.

Proof. At a harmonic vertex v,

$$0 = \sum_{u \sim v} c_{uv}(\phi_v - \phi_u),$$

so

$$\phi_v = \frac{\sum_{u \sim v} c_{uv}\phi_u}{\sum_{u \sim v} c_{uv}}.$$

Thus ϕ_v is a weighted average of its neighbors.

If an interior vertex achieves the global maximum, all of its neighbors must have the same value. By connectedness this propagates until reaching the boundary. The minimum case is identical. □

Because of the maximum principle and nonnegativity of effective resistances we have, for the unit s–t electrical potential,

$$\boxed{\phi_s \geq \phi_v \geq \phi_t \qquad \text{for every } v.}$$

12.1.4 Dirichlet's Principle

Dirichlet's principle is most naturally stated as a boundary-value problem. Fix potentials on some set of boundary vertices; among all extensions to the rest of the graph, the minimum-energy extension is exactly the harmonic one.

Let $G = (V, E)$ be connected, with weighted Laplacian L, and define

$$\mathcal{E}(\phi) = \phi^\top L\phi = \sum_{uv \in E} w_{uv}(\phi_u - \phi_v)^2.$$

Theorem 391 (Dirichlet's Principle)**.** *Let $\emptyset \neq S \subseteq V$, and prescribe boundary values $b \in \mathbb{R}^S$. Among all potentials satisfying*

$$\phi|_S = b,$$

there is a unique minimizer h of $\mathcal{E}(\phi)$. It is characterized by

$$\boxed{(Lh)_v = 0 \qquad (v \in V \setminus S),}$$

that is, h is harmonic at every non-boundary vertex.

Proof. Let $U = V \setminus S$. The harmonicity equations are

$$L_{U,U}h_U = -L_{U,S}b.$$

The matrix $L_{U,U}$ is positive definite. Indeed, if $x \in \mathbb{R}^U$, extend it by 0 on S. Then

$$x^\top L_{U,U}x = \sum_{uv \in E} w_{uv}(x_u - x_v)^2.$$

If this is 0, the extended vector is constant on every edge. Since G is connected and it vanishes on the nonempty set S, it must vanish everywhere. Hence $x = 0$.

Thus there is a unique harmonic extension h.

Now let ϕ be any other extension of the same boundary and write

$$\phi = h + \psi, \qquad \psi|_S = 0.$$

Then

$$\phi^\top L\phi = h^\top Lh + 2\psi^\top Lh + \psi^\top L\psi.$$

Since h is harmonic on U, the vector Lh is supported on S. Since ψ vanishes on S,

$$\psi^\top Lh = 0.$$

Therefore

$$\boxed{\mathcal{E}(\phi) = \mathcal{E}(h) + \mathcal{E}(\psi) \geq \mathcal{E}(h).}$$

Equality forces $\mathcal{E}(\psi) = 0$, so ψ is constant; because it vanishes on S, it is identically zero. Thus h is the unique minimizer. □

Boundary Current and Power For the harmonic extension h, let

$$j = Lh.$$

The coordinate j_v is the net current injected at v. Harmonicity says that $j_v = 0$ away from S, so current enters and leaves the network only through the boundary.

Moreover,

$$\boxed{\mathcal{E}(h) = h^\top Lh = h^\top j = \sum_{v \in S} b_v j_v.}$$

Thus the energy is exactly the power supplied through the boundary: potential times injected current, summed over the boundary vertices.

Define the effective conductance

$$C_{\text{eff}}(s,t) = \frac{1}{R_{\text{eff}}(s,t)}.$$

Corollary 392 (Two-Terminal Dirichlet Principle)**.** *For distinct vertices s, t, let h be harmonic on $V \setminus \{s,t\}$ with*

$$h_s = 1, \qquad h_t = 0.$$

Then

$$\boxed{C_{\text{eff}}(s,t) = \min_{\substack{\phi_s = 1 \\ \phi_t = 0}} \phi^\top L\phi.}$$

Proof. By Theorem 391, h is the unique minimizer.

Let

$$I = (Lh)_s$$

be the total current leaving s. Since Lh vanishes away from s, t, and

$$\mathbf{1}^\top Lh = 0,$$

we have

$$(Lh)_t = -I.$$

Therefore

$$\begin{aligned} h^\top Lh &= h_s(Lh)_s + h_t(Lh)_t \\ &= 1 \cdot I + 0 \cdot (-I) \\ &= I. \end{aligned}$$

Equivalently, the energy is

$$(\text{voltage drop}) \cdot (\text{current}) = 1 \cdot I.$$

Now scale the potential by $1/I$. Currents scale by the same factor, so the new potential sends one unit of current from s to t. Its voltage drop is

$$\frac{1}{I}.$$

Now, by the definition of effective resistance,

$$R_{\text{eff}}(s,t) = \frac{1}{I}.$$

Hence

$$I = \frac{1}{R_{\text{eff}}(s,t)} = C_{\text{eff}}(s,t),$$

and therefore

$$h^\top L h = C_{\text{eff}}(s,t).$$

□

	current	voltage drop	energy
unit current	1	R_{eff}	R_{eff}
unit voltage	C_{eff}	1	C_{eff}.

12.1.5 Series and Parallel Laws

If two networks of resistances R_1, R_2 are connected in series, every unit flow must pass through both. Hence Thomson's principle immediately gives

$$\boxed{R_{\text{series}} = R_1 + R_2.}$$

If they are connected in parallel and a fraction α of the unit current uses the first branch, the minimum possible energy for this split is

$$\alpha^2 R_1 + (1-\alpha)^2 R_2.$$

Minimizing gives

$$\alpha R_1 = (1-\alpha) R_2$$

and hence

$$\boxed{R_{\text{parallel}} = \frac{R_1 R_2}{R_1 + R_2}.}$$

Equivalently,

$$\boxed{C_{\text{parallel}} = C_1 + C_2.}$$

More generally, parallel currents divide proportionally to conductance.

12.1.6 Rayleigh Monotonicity

Theorem 393 (Rayleigh Monotonicity)**.** *Increasing edge resistances can only increase effective resistance. Equivalently, increasing conductances can only decrease effective resistance.*

Proof. Suppose

$$r'_e \geq r_e$$

for every edge. For every unit s–t flow θ,

$$\sum_e r'_e \theta_e^2 \geq \sum_e r_e \theta_e^2.$$

Taking the minimum over all unit flows and applying Thomson's principle gives

$$R'_{\text{eff}}(s,t) \geq R_{\text{eff}}(s,t).$$

□

Thus:

- deleting edges increases resistance;
- adding edges decreases resistance;
- contracting or shorting vertices decreases resistance.

12.1.7 Effective Resistance is a Metric

Theorem 394. *On the vertices of a connected graph,*

$$R_{\text{eff}}$$

is a metric.

Proof. Nonnegativity, symmetry, and

$$R_{\text{eff}}(s,t) = 0 \iff s = t$$

are immediate from

$$R_{\text{eff}}(s,t) = b_{st}^\top L^+ b_{st}.$$

For the triangle inequality, let

$$p = L^+(\mathbf{1}_s - \mathbf{1}_t).$$

By the maximum principle,

$$p_u \geq p_t.$$

Now

$$R_{\text{eff}}(s,t) + R_{\text{eff}}(t,u) - R_{\text{eff}}(s,u) = 2(p_u - p_t) \geq 0.$$

Hence

$$R_{\text{eff}}(s,u) \leq R_{\text{eff}}(s,t) + R_{\text{eff}}(t,u).$$

□

12.2 Random Walks and Electrical Networks

Define the weighted random walk by

$$P(u,v) = \frac{c_{uv}}{c_u}, \qquad c_u = \sum_v c_{uv}.$$

Its stationary distribution is

$$\pi(u) = \frac{c_u}{\text{vol}(G)}, \qquad \text{vol}(G) = \sum_u c_u.$$

This walk is reversible:

$$\boxed{c_u P(u,v) = c_{uv} = c_v P(v,u).}$$

12.2.1 Voltages are Hitting Probabilities

Let

$$T_A = \inf\{j \geq 0 : X_j \in A\}.$$

Theorem 395. *Let*

$$q(u) = \Pr_u[T_s < T_t].$$

Then q is exactly the unit-voltage electrical potential:

$$q(s) = 1, \qquad q(t) = 0,$$

and

$$Lq = 0 \qquad \text{on } V \setminus \{s, t\}.$$

Proof. Conditioning on the first random-walk step gives, for

$$u \notin \{s, t\},$$

$$q(u) = \sum_v P(u, v)q(v).$$

Multiplying by c_u gives

$$\sum_v c_{uv}(q(u) - q(v)) = 0,$$

which is precisely

$$(Lq)_u = 0.$$

The boundary values are clearly 1 and 0. Uniqueness follows from the maximum principle. □

If

$$p = L^+(\mathbf{1}_s - \mathbf{1}_t)$$

is instead the unit-current potential, then

$$\boxed{q(u) = \frac{p_u - p_t}{p_s - p_t} = \frac{p_u - p_t}{R_{\text{eff}}(s, t)}.}$$

12.2.2 Currents are Expected Net Traversals

Run a random walk from s until its first visit to t. For $v \neq t$, let

$$G(v) = \mathbb{E}_s[\#\text{ visits to } v \text{ before } T_t],$$

and set

$$G(t) = 0.$$

Define

$$g(v) = \frac{G(v)}{c_v}.$$

Also let $N_{u\to v}$ be the number of times we walk on the edge $u \to v$ in that direction.

Theorem 396.

$$\boxed{Lg = \mathbf{1}_s - \mathbf{1}_t.}$$

Consequently,

$$g = L^+(\mathbf{1}_s - \mathbf{1}_t) + \alpha\mathbf{1}$$

for some constant α, and the electrical current on (u, v) is

$$\boxed{c_{uv}(g(u) - g(v)) = \mathbb{E}_s[N_{u\to v} - N_{v\to u}].}$$

Proof. For $v \neq t$, every visit to v is either the initial visit when $v = s$ or results from a transition into v:

$$G(v) = \mathbf{1}[v = s] + \sum_u G(u)P(u, v).$$

We have

$$G(u)P(u, v) = c_{uv}g(u).$$

Hence

$$c_v g(v) - \sum_u c_{uv} g(u) = \mathbf{1}[v = s],$$

so

$$(Lg)_v = \mathbf{1}[v = s] \qquad (v \neq t).$$

Because the entries of Lg sum to zero, its t-coordinate is -1.

Also

$$\mathbb{E}N_{u\to v} = G(u)P(u, v) = c_{uv}g(u),$$

so subtraction gives the final identity. □

12.2.3 Hitting a Vertex Through a Given Edge

Corollary 397. *Suppose $\{s, t\} \in E$. A walk starting at s first enters t through the edge $s \to t$ with probability*

$$\boxed{c_{st}R_{\text{eff}}(s, t).}$$

Proof. Before T_t, the edge $s \to t$ can be traversed at most once, and traversing it ends the walk. Thus its expected number of traversals is exactly the desired probability.

By Theorem 396,

$$\mathbb{E}N_{s\to t} = c_{st}(g(s) - g(t)) = c_{st}R_{\text{eff}}(s, t).$$

□

In particular,

$$c_{st}R_{\text{eff}}(s, t) \leq 1.$$

12.2.4 Hitting Times

Let

$$H_{u,t} = \mathbb{E}_u[T_t].$$

For fixed t, define

$$h_t(u) = H_{u,t}.$$

For $u \neq t$,

$$h_t(u) = 1 + \sum_v P(u, v)h_t(v).$$

Multiplying by c_u gives

$$(Lh_t)_u = c_u.$$

Let

$$\mathbf{c} = (c_v)_{v\in V}, \qquad \text{vol}(G) = \sum_v c_v,$$

and define

$$b_t = \mathbf{c} - \text{vol}(G)\mathbf{1}_t.$$

Since both Lh_t and b_t have coordinate sum zero,

$$\boxed{Lh_t = b_t.}$$

Theorem 398 (Hitting-Time Potential)**.** *Let*

$$p_t = L^+ b_t.$$

Then

$$\boxed{H_{u,t} = p_t(u) - p_t(t).}$$

Proof. Both h_t and p_t solve

$$Lx = b_t.$$

Their difference therefore lies in $\ker L = \operatorname{span}\{\mathbf{1}\}$. Since

$$h_t(t) = 0,$$

subtracting the value at t gives the formula. □

12.2.5 Commute Time

Define

$$C_{u,v} = H_{u,v} + H_{v,u}.$$

Theorem 399 (Commute-Time Identity)**.**

$$\boxed{C_{u,v} = \operatorname{vol}(G) R_{\text{eff}}(u,v).}$$

Proof. Using the preceding theorem,

$$p_v - p_u = L^+(b_v - b_u) = \operatorname{vol}(G) L^+(\mathbf{1}_u - \mathbf{1}_v).$$

Therefore

$$\begin{aligned} H_{u,v} + H_{v,u} &= (\mathbf{1}_u - \mathbf{1}_v)^\top (p_v - p_u) \\ &= \operatorname{vol}(G)(\mathbf{1}_u - \mathbf{1}_v)^\top L^+ (\mathbf{1}_u - \mathbf{1}_v). \end{aligned}$$

□

12.2.6 Tetali's Hitting-Time Formula

Theorem 400 (Tetali's Formula)**.** *For all s, t,*

$$\boxed{H_{s,t} = \frac{1}{2} \sum_{x \in V} c_x \left(R_{\text{eff}}(s,t) + R_{\text{eff}}(t,x) - R_{\text{eff}}(s,x) \right).}$$

Proof. Write

$$R(a,b) = L^+_{aa} + L^+_{bb} - 2L^+_{ab}.$$

Then

$$\begin{aligned} &\frac{1}{2} \sum_x c_x \big(R(s,t) + R(t,x) - R(s,x) \big) \\ &= \sum_x c_x (L^+_{sx} - L^+_{tx}) - \operatorname{vol}(G)(L^+_{st} - L^+_{tt}) \\ &= (\mathbf{1}_s - \mathbf{1}_t)^\top L^+ \left(\mathbf{c} - \operatorname{vol}(G) \mathbf{1}_t \right) \\ &= p_t(s) - p_t(t) \\ &= H_{s,t}. \end{aligned}$$

□

12.2.7 Cover-Time Bound

Let

$$\mathrm{cov}_u(G)$$

be the expected time for an unweighted random walk starting at u to visit every vertex.

Theorem 401. *For every connected unweighted graph with n vertices and m edges,*

$$\boxed{\mathrm{cov}_u(G) \le 2m(n-1).}$$

Proof. Fix any spanning tree T. Take an Euler tour of T, which traverses every tree edge once in each direction.

Starting from u, wait until the random walk visits the successive vertices of this tour. Once this process finishes, every vertex has been visited. Its expected duration is at most

$$\sum_{\{x,y\}\in T} (H_{x,y} + H_{y,x}) = \sum_{\{x,y\}\in T} C_{x,y}.$$

For a tree edge $\{x, y\}$, Rayleigh monotonicity gives

$$R_{\mathrm{eff}}(x, y) \le 1.$$

Since

$$\mathrm{vol}(G) = 2m,$$

the commute-time identity yields

$$C_{x,y} \le 2m.$$

There are $n - 1$ tree edges. □

12.3 Kron Reduction

Electrical networks remain electrical networks after eliminating internal vertices.

Partition the vertices into a boundary S and interior I, and write

$$L = \begin{pmatrix} L_{SS} & L_{SI} \\ L_{IS} & L_{II} \end{pmatrix}.$$

Theorem 402 (Kron Reduction)**.** *The network induced on the boundary after eliminating the interior has Laplacian*

$$\boxed{L_{\mathrm{Kron}} = L_{SS} - L_{SI} L_{II}^{-1} L_{IS}.}$$

It preserves all effective resistances between vertices of S.

Proof. Fix boundary potentials x_S. The energy is

$$\begin{pmatrix} x_S \\ x_I \end{pmatrix}^{\top} L \begin{pmatrix} x_S \\ x_I \end{pmatrix}.$$

The minimizing interior potential satisfies

$$L_{IS} x_S + L_{II} x_I = 0,$$

so

$$x_I = -L_{II}^{-1} L_{IS} x_S.$$

Substituting gives

$$x_S^{\top} \left(L_{SS} - L_{SI} L_{II}^{-1} L_{IS} \right) x_S.$$

Dirichlet's principle therefore shows that all boundary effective resistances are unchanged. □

There is also a random-walk interpretation.

Let

$$P = D^{-1}A$$

be the transition matrix. Partition it in the same way.

Theorem 403 (Trace Chain)**.** *Observe the random walk only at visits to S. Its transition matrix is*

$$\boxed{Q = P_{SS} + P_{SI}(I - P_{II})^{-1}P_{IS}.}$$

Moreover,

$$\boxed{L_{\text{Kron}} = D_S(I - Q).}$$

Proof. The term

$$P_{SI}P_{II}^{k}P_{IS}$$

is the probability of leaving S, taking exactly k internal steps, and returning to S. Summing over $k \geq 0$ gives the formula for Q.

Also,

$$L_{II} = D_I(I - P_{II}),$$
$$L_{SI} = -D_S P_{SI}, \qquad L_{IS} = -D_I P_{IS},$$

and

$$L_{SS} = D_S(I - P_{SS}).$$

Substitution into the Schur complement gives

$$L_{\text{Kron}} = D_S\left(I - P_{SS} - P_{SI}(I - P_{II})^{-1}P_{IS}\right) = D_S(I - Q).$$

□

Thus for distinct boundary vertices a, b, the reduced conductance is

$$\boxed{\widetilde{c}_{ab} = c_a Q(a, b).}$$

As a special case, eliminating the center of a star whose incident conductances are

$$c_1, \ldots, c_k$$

creates between leaves i, j the conductance

$$\boxed{\widetilde{c}_{ij} = \frac{c_i c_j}{\sum_\ell c_\ell}.}$$

12.4 Infinite Graphs, Recurrence, and Transience

Now let G be a connected locally finite infinite graph.

Choose finite connected sets

$$K_1 \subseteq K_2 \subseteq \cdots, \qquad \bigcup_n K_n = V,$$

containing a fixed vertex o.

Wire every vertex outside K_n into one sink ∂_n, and let

$$R_n = R_{\text{eff}}(o, \partial_n).$$

Because the n-th wired network is obtained from the $(n+1)$-st by additional contractions, Rayleigh monotonicity gives

$$R_n \leq R_{n+1}.$$

Define

$$\boxed{R_{\text{eff}}(o \leftrightarrow \infty) = \lim_{n\to\infty} R_n \in [0, \infty].}$$

12.4.1 Infinite Thomson Principle

A unit flow from o to infinity satisfies

$$\operatorname{div}\theta(o) = 1$$

and

$$\operatorname{div}\theta(v) = 0 \qquad (v \neq o).$$

Equivalently, one unit of net current crosses every finite cut separating o from infinity.

Theorem 404 (Infinite Thomson Principle)**.**

$$\boxed{R_{\text{eff}}(o \leftrightarrow \infty) = \inf_{\theta:\ \textit{unit flow to infinity}} \mathcal{E}(\theta).}$$

In particular,

$$R_{\text{eff}}(o \leftrightarrow \infty) < \infty$$

if and only if there exists a finite-energy unit flow to infinity.

Proof. Let θ be any unit flow to infinity. Restrict it to the edges inside K_n and crossing its boundary. After wiring the exterior, this is a feasible unit flow from o to ∂_n. Therefore finite Thomson gives

$$R_n \leq \mathcal{E}(\theta).$$

Taking $n \to \infty$ gives

$$R_\infty \leq \mathcal{E}(\theta).$$

Conversely, suppose

$$R_\infty < \infty.$$

Let θ_n be the electrical unit flow from o to ∂_n, extended by zero outside K_n. Then

$$\mathcal{E}(\theta_n) = R_n \leq R_\infty.$$

For every fixed edge e,

$$r_e \theta_n(e)^2 \leq R_\infty,$$

so the sequence $\theta_n(e)$ is bounded.

Since E is countable, enumerate its edges as

$$E = \{e_1, e_2, \dots\}.$$

For each fixed edge e, the sequence $(\theta_n(e))_n$ is bounded. By Bolzano–Weierstrass, first pass to a subsequence on which $\theta_n(e_1)$ converges; then to a further subsequence on which $\theta_n(e_2)$ also converges; and so on. Taking the diagonal subsequence of these nested subsequences gives indices n_j such that, simultaneously for every fixed edge e,

$$\theta_{n_j}(e) \to \theta(e).$$

Local finiteness lets us pass the limit through every divergence sum, so θ is a unit flow to infinity.

Finally Fatou's lemma gives

$$\mathcal{E}(\theta) \leq \liminf_j \mathcal{E}(\theta_{n_j}) = R_\infty.$$

Together with the opposite inequality, equality holds. □

12.4.2 Escape Probability

Let

$$\tau_o^+ = \inf\{j \geq 1 : X_j = o\}.$$

Theorem 405.

$$\boxed{C_{\text{eff}}(o \leftrightarrow \infty) = c_o \Pr_o[\tau_o^+ = \infty].}$$

Consequently,

$$\boxed{G \text{ is transient} \iff R_{\text{eff}}(o \leftrightarrow \infty) < \infty.}$$

Proof. In the finite wired network, impose voltage 1 at o and 0 at ∂_n. The voltage at v is

$$h(v) = \Pr_v[T_o < T_{\partial_n}].$$

Thus the total current leaving o is

$$\begin{aligned} C_{\text{eff}}(o, \partial_n) &= \sum_v c_{ov}(1 - h(v)) \\ &= c_o \sum_v P(o, v) \Pr_v[T_{\partial_n} < T_o] \\ &= c_o \Pr_o[T_{\partial_n} < \tau_o^+]. \end{aligned}$$

Letting $n \to \infty$,

$$C_{\text{eff}}(o \leftrightarrow \infty) = c_o \Pr_o[\tau_o^+ = \infty].$$

The right-hand side is positive exactly when the walk is transient. □

Corollary 406 (Green Function)**.** *Let*

$$G(o, o) = \sum_{j \geq 0} \Pr_o[X_j = o]$$

be the expected number of visits to o. Then in a transient network,

$$\boxed{G(o, o) = c_o R_{\text{eff}}(o \leftrightarrow \infty).}$$

Proof. After every visit to o, the walk escapes forever with probability

$$p = \Pr_o[\tau_o^+ = \infty].$$

Thus the number of visits is geometric with mean $1/p$. Apply Theorem 405. □

12.4.3 Nash–Williams Recurrence Criterion

Theorem 407 (Nash–Williams)**.** *Let*

$$\Pi_1, \Pi_2, \ldots$$

be pairwise edge-disjoint cutsets separating o from infinity. Then

$$\boxed{R_{\text{eff}}(o \leftrightarrow \infty) \geq \sum_i \frac{1}{\sum_{e \in \Pi_i} c_e}.}$$

Proof. Let θ be any unit flow to infinity. The net current crossing each Π_i is 1. Hence by weighted Cauchy–Schwarz,

$$1 = \left(\sum_{e \in \Pi_i} \theta_e \right)^2 \leq \left(\sum_{e \in \Pi_i} c_e \right) \left(\sum_{e \in \Pi_i} \frac{\theta_e^2}{c_e} \right).$$

Therefore

$$\sum_{e \in \Pi_i} \frac{\theta_e^2}{c_e} \geq \frac{1}{\sum_{e \in \Pi_i} c_e}.$$

The cutsets are edge-disjoint, so summing gives the claimed lower bound on $\mathcal{E}(\theta)$. Now minimize using infinite Thomson. □

For the unit grid $\mathbb{Z}^2$, take square shells. The r-th shell has

$$O(r)$$

edges, so

$$R_{\text{eff}}(0 \leftrightarrow \infty) \geq \sum_{r \geq 1} \Omega(1/r) = \infty.$$

Thus the two-dimensional random walk is recurrent.

12.4.4 Trees

For a finite binary tree of depth n with all leaves wired together, let R_n be the resistance from the root to the boundary.

Then

$$R_0 = 0$$

and the two branches each have resistance

$$1 + R_n.$$

They occur in parallel, so

$$\boxed{R_{n+1} = \frac{1 + R_n}{2}.}$$

Hence

$$R_n = 1 - 2^{-n}$$

and

$$\boxed{R_{\text{eff}}(o \leftrightarrow \infty) = 1.}$$

The minimizing flow sends

$$2^{-j}$$

through every edge at level j, and its energy is

$$\sum_{j \geq 1} 2^j (2^{-j})^2 = 1.$$

12.5 Random Spanning Trees

Fix a root vertex r. Let

$$B_r \in \mathbb{R}^{E \times (V \setminus \{r\})}$$

be the incidence matrix with the column of r deleted.

12.5.1 Weighted Random Spanning Trees

The weighted spanning-tree distribution is

$$\Pr[T] = \frac{\prod_{e\in T} c_e}{Z},$$

where

$$Z = \sum_{T} \prod_{e\in T} c_e = \det(B_r^\top C B_r).$$

Define the transfer-current matrix

$$\boxed{K = C^{1/2} B L^{+} B^\top C^{1/2}.}$$

Thus

$$K_{ef} = \sqrt{c_e c_f}\, b_e^\top L^{+} b_f.$$

For one edge,

$$K_{ee} = c_e R_{\text{eff}}(e).$$

Theorem 408 (Burton–Pemantle)**.** *For every set of edges*

$$F \subseteq E,$$

$$\boxed{\Pr[F \subseteq T] = \det K[F,F].}$$

Proof. Write

$$F = \{e_1, \dots, e_k\}.$$

Introduce variables

$$z_1, \dots, z_k$$

and multiply the conductance of e_i by

$$1 + z_i.$$

Let

$$U = \begin{pmatrix} \sqrt{c_{e_1}}\, b_{e_1,r} & \cdots & \sqrt{c_{e_k}}\, b_{e_k,r} \end{pmatrix}, \qquad Z_F = \operatorname{diag}(z_1, \dots, z_k),$$

where $b_{e,r}$ denotes the incidence vector with the root coordinate deleted.

The reduced Laplacian becomes

$$L_r(z) = L_r + U Z_F U^\top.$$

Matrix–Tree gives

$$\frac{\det L_r(z)}{\det L_r} = \mathbb{E}_T\left[\prod_{e_i\in T} (1+z_i)\right].$$

Therefore the coefficient of

$$z_1 z_2 \cdots z_k$$

is exactly

$$\Pr[F \subseteq T].$$

The matrix determinant lemma gives

$$\begin{aligned} \frac{\det L_r(z)}{\det L_r} &= \det\left(I + Z_F U^\top L_r^{-1} U\right) \\ &= \det(I + Z_F K[F,F]). \end{aligned}$$

The coefficient of

$$\prod_{i=1}^{k} z_i$$

in this determinant is the principal minor

$$\det K[F, F].$$

Equating coefficients proves the theorem. □

Corollary 409 (Edge Marginal)**.**

$$\boxed{\Pr[e \in T] = c_e R_{\text{eff}}(e).}$$

For an unweighted graph this is simply

$$\Pr[e \in T] = R_{\text{eff}}(e).$$

Corollary 410 (Foster's Identity)**.**

$$\boxed{\sum_{e \in E} c_e R_{\text{eff}}(e) = n - 1.}$$

Proof. The matrix

$$K = C^{1/2} B L^{+} B^{\top} C^{1/2}$$

is an orthogonal projection:

$$\begin{aligned} K^2 &= C^{1/2} B L^{+} (B^{\top} C B) L^{+} B^{\top} C^{1/2} \\ &= C^{1/2} B L^{+} L L^{+} B^{\top} C^{1/2} \\ &= K. \end{aligned}$$

Its rank is $n - 1$, so

$$\operatorname{tr} K = n - 1.$$

But

$$\operatorname{tr} K = \sum_{e} K_{ee} = \sum_{e} c_e R_{\text{eff}}(e).$$

□

Corollary 411 (Negative Correlation)**.** *For every set of edges $F \subseteq E$,*

$$\boxed{\Pr[F \subseteq T] \leq \prod_{e \in F} \Pr[e \in T].}$$

Proof. The principal submatrix $K[F, F]$ is positive semidefinite, so Burton–Pemantle and Hadamard's inequality give

$$\Pr[F \subseteq T] = \det K[F, F] \leq \prod_{e \in F} K_{ee} = \prod_{e \in F} \Pr[e \in T].$$

□

For two distinct edges, the deficit is $\Pr[e \in T] \Pr[f \in T] - \Pr[e, f \in T] = K_{ef}^2$.

12.6 Localization of Electrical Flows

Recall the symmetric transfer-current matrix

$$\Pi = C^{1/2} B L^{+} B^{\top} C^{1/2}.$$

It is an orthogonal projection, so

$$\|\Pi\|_{2\to 2} = 1.$$

This bound uses cancellation between positive and negative entries. We want to understand what remains if all signs are discarded.

Write

$$\overline{\Pi} = |\Pi|$$

for entrywise absolute value.

Why localization is interesting. For an unweighted graph, the e-th column of

$$\Pi = B L^{+} B^{\top}$$

is exactly the electrical flow obtained by sending one unit of current between the endpoints of e. Thus

$$\|\Pi \mathbf{1}_e\|_1 = \sum_{f \in E} |\Pi_{f,e}|$$

measures how much total edge usage this electrical flow has.

This is a natural measure of how far the flow spreads. Indeed, an electrical flow has no circulation: current always moves from higher to lower potential. It can therefore be decomposed into s–t paths carrying nonnegative amounts of flow. Since the total flow is 1, its ℓ_1-norm is the average number of edges used by a unit of current under such a path decomposition. A flow concentrated on short paths has small ℓ_1-norm, while a flow spread through long routes has large ℓ_1-norm.

There is no reason for every individual electrical flow to be local. For some edges, the unit current may spread through a large part of the graph. The localization theorem says that this cannot happen for too many source edges at once:

$$\frac{1}{m} \sum_{e \in E} \|\Pi \mathbf{1}_e\|_1 = O(\log n).$$

Equivalently, although exceptional electrical flows may be highly delocalized, the average electrical flow generated by an edge has only $O(\log n)$ total edge usage.

The matrix Π itself does not reveal this, since

$$\|\Pi\|_{2\to 2} = 1$$

relies on cancellations between positive and negative entries. To measure unsigned spreading we must instead understand

$$|\Pi|.$$

Theorem 412 (Localization of Electrical Flows)**.** *For every connected weighted graph on n vertices,*

$$\boxed{\|\overline{\Pi}\|_{2\to 2} \le 2 \log n.}$$

We prove the stronger statement

$$\boxed{w^{\top} \overline{\Pi} w \le 2 H(\mu_w) \|w\|_2^2,}$$

where

$$\mu_w(x) = \frac{\sum_{e \ni x} w_e^2}{2\|w\|_2^2}.$$

Since $H(\mu_w) \le \log n$, the theorem follows immediately.

12.6.1 Replacing L^+ by a Continuous-Time Walk

Fix $w \geq 0$. We want to bound

$$w^\top \overline{\Pi} w = \sum_{e,f} w_e w_f \sqrt{c_e c_f}\, \left| b_e^\top L^+ b_f \right|.$$

The difficult object here is L^+. A useful scalar identity is

$$\boxed{\frac{1}{\lambda} = \int_0^\infty e^{-t\lambda}\, dt \qquad (\lambda > 0).}$$

Thus instead of treating an inverse eigenvalue $1/\lambda$ directly, we can regard it as the total contribution over time of $e^{-t\lambda}$.

This suggests representing L^+ by a continuous-time random walk.

For the moment let μ be any strictly positive probability distribution on V, and write

$$M = \mathrm{diag}(\mu).$$

Consider the continuous-time walk whose generator is

$$-M^{-1}L.$$

Its transition matrix at time t is

$$P_t = e^{-tM^{-1}L}.$$

Equivalently, from x the walk jumps to y at rate

$$\frac{c_{xy}}{\mu(x)}.$$

The distribution μ is stationary, and the chain is reversible.

It is convenient to use the symmetric kernel

$$\boxed{H_t = P_t M^{-1}.}$$

By reversibility, H_t is symmetric.

There is also a probabilistic meaning to its columns. If

$$h_t^{(v)} = H_t \mathbf{1}_v,$$

then

$$\boxed{\Pr_v[X_t = x] = \mu(x) h_t^{(v)}(x).}$$

Thus $h_t^{(v)}$ is simply the distribution of the walk started at v, written as a density relative to its stationary distribution μ. In particular,

$$\mu^\top h_t^{(v)} = 1.$$

At $t = 0$,

$$h_0^{(v)}(x) = \frac{\mathbf{1}[x = v]}{\mu(v)},$$

while as $t \to \infty$,

$$h_t^{(v)}(x) \to 1.$$

The reason for introducing this walk is the following identity.

Lemma 413 (Integrated Walk Kernel)**.**

$$\boxed{BL^{+}B^{\top} = \int_{0}^{\infty} BH_{t}B^{\top}\,dt.}$$

Proof. Set

$$S = M^{-1/2}LM^{-1/2}.$$

Then

$$P_{t} = M^{-1/2}e^{-tS}M^{1/2}, \qquad H_{t} = M^{-1/2}e^{-tS}M^{-1/2}.$$

Diagonalize

$$S = \sum_{i=1}^{n} \lambda_{i}\psi_{i}\psi_{i}^{\top}, \qquad 0 = \lambda_{1} < \lambda_{2} \le \cdots \le \lambda_{n}.$$

The zero eigenvector is proportional to $M^{1/2}\mathbf{1}$, so

$$BM^{-1/2}\psi_{1} = 0.$$

Therefore

$$BH_{t}B^{\top} = \sum_{i=2}^{n} e^{-t\lambda_{i}} BM^{-1/2}\psi_{i}\psi_{i}^{\top}M^{-1/2}B^{\top}.$$

Integrating and using

$$\int_{0}^{\infty} e^{-t\lambda_{i}}\,dt = \frac{1}{\lambda_{i}}$$

gives

$$\int_{0}^{\infty} BH_{t}B^{\top}\,dt = BM^{-1/2}S^{+}M^{-1/2}B^{\top}.$$

Let

$$\Gamma = M^{-1/2}S^{+}M^{-1/2}.$$

For every edge incidence vector b_{e},

$$M^{-1/2}b_{e} \perp M^{1/2}\mathbf{1} = \ker S,$$

so

$$L\Gamma b_{e} = M^{1/2}SS^{+}M^{-1/2}b_{e} = b_{e}.$$

Also

$$LL^{+}b_{e} = b_{e}.$$

Hence

$$\Gamma b_{e} - L^{+}b_{e} \in \ker L = \operatorname{span}\{\mathbf{1}\}.$$

Applying B kills this difference, proving the identity. □

So our original quantity satisfies

$$w^{\top}\overline{\Pi}w \le \int_{0}^{\infty} w^{\top}C^{1/2}|BH_{t}B^{\top}|C^{1/2}w\,dt.$$

We have replaced the inverse L^{+} by something more concrete: transition probabilities of a continuous-time walk, integrated over time.

12.6.2 Splitting the Walk at its Midpoint

After replacing L^+ by the time integral of H_t, we are left with matrices of the form

$$BH_tB^\top.$$

The difficulty is that an entry of this matrix involves two edges at once. The semigroup property of the walk can give us more symmetry in the expression by duplicating H_t up to a factor of M. This will allow us to express everything in terms of edge differences of a single function.

Recall that

$$H_t = P_tM^{-1}, \qquad M = \operatorname{diag}(\mu).$$

Since $P_{s+t} = P_sP_t$,

$$H_{s+t} = P_sP_tM^{-1} = P_sM^{-1}MP_tM^{-1} = H_sMH_t.$$

In particular,

$$\boxed{H_{2t} = H_tMH_t.}$$

Now expand the diagonal matrix M as

$$M = \sum_{v\in V} \mu(v)\mathbf{1}_v\mathbf{1}_v^\top.$$

Substituting this into the preceding identity gives

$$\begin{aligned} H_{2t} &= H_t\left(\sum_v \mu(v)\mathbf{1}_v\mathbf{1}_v^\top\right)H_t \\ &= \sum_v \mu(v)(H_t\mathbf{1}_v)(\mathbf{1}_v^\top H_t). \end{aligned}$$

Since H_t is symmetric we have $\mathbf{1}_v^\top H_t = (H_t\mathbf{1}_v)^\top$. Writing

$$h_t^{(v)} = H_t\mathbf{1}_v,$$

we obtain

$$\boxed{H_{2t} = \sum_{v\in V} \mu(v)h_t^{(v)}(h_t^{(v)})^\top.}$$

Applying the incidence matrix on both sides gives

$$\begin{aligned} BH_{2t}B^\top &= B\left(\sum_v \mu(v)h_t^{(v)}(h_t^{(v)})^\top\right)B^\top \\ &= \sum_v \mu(v)(Bh_t^{(v)})(Bh_t^{(v)})^\top. \end{aligned}$$

Hence

$$\boxed{BH_{2t}B^\top = \sum_v \mu(v)(Bh_t^{(v)})(Bh_t^{(v)})^\top.}$$

This is exactly the form we wanted. The vector

$$Bh_t^{(v)}$$

has one coordinate for each oriented edge $e = (x, y)$, namely

$$(Bh_t^{(v)})_e = h_t^{(v)}(x) - h_t^{(v)}(y).$$

So it records the edge-by-edge differences of the probability density of a walk started at v.

The original matrix $BH_{2t}B^\top$ contained correlations between every pair of edges. The midpoint identity rewrites it as an average of rank-one matrices

$$(Bh)(Bh)^\top.$$

After taking entrywise absolute values,

$$|BH_{2t}B^\top| \le \sum_v \mu(v)|Bh_t^{(v)}||Bh_t^{(v)}|^\top.$$

Therefore, for $w \ge 0$,

$$\begin{aligned} &w^\top C^{1/2}|BH_{2t}B^\top|C^{1/2}w \\ &\quad\le \sum_v \mu(v)\left(w^\top C^{1/2}|Bh_t^{(v)}|\right)^2. \end{aligned}$$

Thus it is enough to control, for one starting vertex v,

$$\boxed{w^\top C^{1/2}|Bh_t^{(v)}| = \sum_{e=xy} w_e\sqrt{c_e}\,|h_t^{(v)}(x) - h_t^{(v)}(y)|.}$$

This is a much more concrete object: it is simply a weighted sum of how different the time-t walk probabilities are across adjacent vertices. The rest of the proof is devoted to bounding the integral over time of the square of this quantity.

12.6.3 Turning Edge Differences into a Time Derivative

After splitting the walk at its midpoint, it remains to control, for a positive vertex function h,

$$Q(h) := \left(\sum_{e=xy} w_e\sqrt{c_e}\,|h_x - h_y|\right)^2.$$

Eventually $h = h_t^{(v)}$ will evolve according to

$$\frac{d}{dt}h_t = -M^{-1}Lh_t.$$

Since we will integrate $Q(h_t)$ over all time, ideally, one factor in the bound should be a time derivative, so that its integral telescopes.

Start with weighted Cauchy–Schwarz. For any positive edge scales s_{xy},

$$\begin{aligned} Q(h) &= \left(\sum_{e=xy} w_e\sqrt{s_{xy}}\cdot\sqrt{c_e}\frac{|h_x - h_y|}{\sqrt{s_{xy}}}\right)^2 \\ &\le \left(\sum_{e=xy} w_e^2 s_{xy}\right)\left(\sum_{e=xy} c_e\frac{(h_x - h_y)^2}{s_{xy}}\right). \end{aligned}$$

We now choose s_{xy}.

Consider a real-valued functional

$$F_\varphi(h) = \sum_x \mu(x)\varphi(h_x).$$

The walk traces out a vector-valued path $t \mapsto h_t$; composing with F_φ gives an ordinary real-valued function

$$t \longmapsto F_\varphi(h_t).$$

Let

$$F_\varphi(h) = \sum_x \mu(x)\varphi(h_x), \qquad M = \operatorname{diag}(\mu).$$

Here

$$\varphi'(h)$$

denotes the vector obtained by applying φ' coordinatewise:

$$\varphi'(h) = \big(\varphi'(h_x)\big)_{x\in V}.$$

Along the continuous-time walk,

$$\frac{d}{dt}h_t = -M^{-1}Lh_t.$$

Applying the ordinary multivariable chain rule,

$$\begin{aligned}\frac{d}{dt}F_\varphi(h_t) &= \sum_x \mu(x)\varphi'(h_t(x))\frac{d}{dt}h_t(x)\\ &= \varphi'(h_t)^\top M\frac{d}{dt}h_t\\ &= -\varphi'(h_t)^\top Lh_t.\end{aligned}$$

Therefore

$$\boxed{-\frac{d}{dt}F_\varphi(h_t) = \varphi'(h_t)^\top Lh_t.}$$

Now use

$$L = B^\top CB.$$

Then

$$\begin{aligned}\varphi'(h)^\top Lh &= \varphi'(h)^\top B^\top CBh\\ &= (B\varphi'(h))^\top C(Bh).\end{aligned}$$

For an oriented edge $e = (x, y)$,

$$(Bh)_e = h_x - h_y$$

and

$$(B\varphi'(h))_e = \varphi'(h_x) - \varphi'(h_y).$$

Since C is diagonal with $C_{ee} = c_e$,

$$(B\varphi'(h))^\top C(Bh) = \sum_{e=(x,y)} c_e\big(\varphi'(h_x) - \varphi'(h_y)\big)(h_x - h_y).$$

Hence

$$\boxed{-\frac{d}{dt}F_\varphi(h_t) = \sum_{e=xy} c_e(h_x - h_y)\big(\varphi'(h_x) - \varphi'(h_y)\big).}$$

Notice that the orientation of e does not matter: reversing x and y changes the sign of both factors, so their product is unchanged.

Thus the second Cauchy–Schwarz factor becomes exactly $-\frac{d}{dt}F_\varphi(h_t)$ if we set

$$\boxed{s_{xy} = \frac{h_x - h_y}{\varphi'(h_x) - \varphi'(h_y)}.}$$

For convex φ, this quantity is positive.

There are still many possible choices of φ. To choose one, look at the first Cauchy–Schwarz factor

$$\sum_{e=xy} w_e^2 s(h_x, h_y), \qquad s(a,b) = \frac{a-b}{\varphi'(a) - \varphi'(b)}.$$

Ultimately we want to control this using the conserved mass

$$\sum_x \mu(x) h_x = 1.$$

Since conserved mass is linear in h, it is natural to require the edge scale s to scale linearly as well:

$$\boxed{s(ta, tb) = t\, s(a,b) \qquad (t > 0).}$$

In other words, multiplying all densities by t should multiply the corresponding edge scale by t.

Substituting the definition of s, for $a \neq b$,

$$\frac{t(a-b)}{\varphi'(ta) - \varphi'(tb)} = t \frac{a-b}{\varphi'(a) - \varphi'(b)}.$$

Hence

$$\varphi'(ta) - \varphi'(tb) = \varphi'(a) - \varphi'(b).$$

For each fixed t, this says that

$$\varphi'(tx) - \varphi'(x)$$

is independent of x. Differentiating in x,

$$t\varphi''(tx) = \varphi''(x).$$

Therefore

$$(tx)\varphi''(tx) = x\varphi''(x),$$

so $x\varphi''(x)$ is constant. Thus

$$\varphi''(x) = \frac{c}{x}.$$

Integrating twice,

$$\varphi(x) = c\, x \log x + \text{affine terms}.$$

Affine terms disappear from $\varphi'(a) - \varphi'(b)$, and an overall positive constant only rescales the two Cauchy–Schwarz factors inversely. We may therefore take

$$\boxed{\varphi(x) = x \log x.}$$

Then

$$\varphi'(x) = 1 + \log x,$$

and the corresponding edge scale is

$$\boxed{s(a,b) = \frac{a-b}{\log a - \log b} = \Lambda(a,b),}$$

the logarithmic mean.

The logarithmic mean has exactly the upper bound we need:

$$\boxed{\Lambda(a,b) \le \frac{a+b}{2}.}$$

Indeed,

$$\Lambda(a,b) = \int_0^1 a^t b^{1-t}\, dt,$$

while weighted AM–GM gives

$$a^t b^{1-t} \le ta + (1-t)b.$$

Integrating over $t \in [0,1]$ yields the claim.

We can now carry out Cauchy–Schwarz with

$$s_{xy} = \Lambda(h_x, h_y).$$

The second factor becomes

$$\begin{aligned}\sum_{e=xy} c_e \frac{(h_x - h_y)^2}{\Lambda(h_x,h_y)} &= \sum_{e=xy} c_e (h_x - h_y)(\log h_x - \log h_y) \\ &= h^\top L \log h.\end{aligned}$$

Define

$$\boxed{I(h) = h^\top L \log h.}$$

Then

$$Q(h) \le \left(\sum_{e=xy} w_e^2 \Lambda(h_x,h_y) \right) I(h).$$

Using

$$\Lambda(h_x,h_y) \le \frac{h_x + h_y}{2},$$

we obtain

$$\begin{aligned}Q(h) &\le \frac{1}{2} \left(\sum_{e=xy} w_e^2 (h_x + h_y) \right) I(h) \\ &= \boxed{\frac{1}{2} \left(\sum_x h_x \sum_{e \ni x} w_e^2 \right) I(h).}\end{aligned}$$

The calculation has now produced both objects that will finish the proof: the vertex weights

$$\sum_{e \ni x} w_e^2$$

and the quantity

$$I(h) = h^\top L \log h.$$

12.6.4 Choosing the Stationary Distribution

Recall that the representation of L^+ by the continuous-time walk was valid for any positive stationary distribution μ. We now choose that distribution to simplify the factor that just appeared.

Let

$$d_w(x) = \sum_{e \ni x} w_e^2.$$

Since every edge contributes once at each endpoint,

$$\sum_x d_w(x) = 2 \sum_e w_e^2 = 2\|w\|_2^2.$$

Therefore

$$\boxed{\mu_w(x) = \frac{d_w(x)}{2\|w\|_2^2}}$$

is a probability distribution.

Now let the continuous-time walk use this μ_w as its stationary distribution. Every density $h_t^{(v)}$ arising from the walk satisfies

$$\sum_x \mu_w(x) h_t^{(v)}(x) = 1.$$

Consequently,

$$\begin{aligned} \frac{1}{2} \sum_x h_x d_w(x) &= \frac{1}{2} \sum_x h_x \big(2\|w\|_2^2 \mu_w(x)\big) \\ &= \|w\|_2^2. \end{aligned}$$

Thus the preceding inequality simplifies to

$$\boxed{Q(h) \le \|w\|_2^2 I(h)}$$

for every probability density h relative to μ_w.

In particular,

$$\boxed{\left(w^\top C^{1/2} |B h_t^{(v)}|\right)^2 \le \|w\|_2^2 I(h_t^{(v)}).}$$

12.6.5 Integrating the Dissipation

It remains to understand

$$\int_0^\infty I(h_t)\, dt.$$

But $I(h)$ was constructed precisely to be a time derivative.

For a density h relative to $\mu = \mu_w$, define the KL-divergence-like value

$$\boxed{F(h) = \sum_x \mu(x) h_x \log h_x.}$$

Along the walk,

$$\frac{d}{dt} h_t = -M^{-1} L h_t, \qquad M = \operatorname{diag}(\mu).$$

Since

$$\nabla F(h) = M(\mathbf{1} + \log h),$$

the chain rule gives

$$\begin{aligned}\frac{d}{dt}F(h_t) &= \nabla F(h_t)^\top \frac{d}{dt}h_t \\ &= -(\mathbf{1} + \log h_t)^\top L h_t \\ &= -(\log h_t)^\top L h_t \\ &= -I(h_t),\end{aligned}$$

where we used

$$\mathbf{1}^\top L = 0.$$

Hence

$$\boxed{I(h_t) = -\frac{d}{dt}F(h_t).}$$

Integrating in time now telescopes:

$$\boxed{\int_0^\infty I(h_t)\,dt = F(h_0) - F(h_\infty).}$$

For the walk started at v,

$$h_t^{(v)} = H_t \mathbf{1}_v.$$

At time 0,

$$h_0^{(v)}(x) = \frac{\mathbf{1}[x = v]}{\mu(v)},$$

while as $t \to \infty$, the walk converges to stationarity, so

$$h_t^{(v)} \longrightarrow \mathbf{1}.$$

Therefore

$$F(h_\infty^{(v)}) = 0,$$

while, with the convention $0 \log 0 = 0$,

$$F(h_0^{(v)}) = \mu(v)\frac{1}{\mu(v)} \log \frac{1}{\mu(v)} = \log \frac{1}{\mu(v)}.$$

Thus

$$\boxed{\int_0^\infty I(h_t^{(v)})\,dt = \log \frac{1}{\mu(v)}.}$$

Strictly speaking, $h_0^{(v)}$ has zero coordinates, whereas the derivative calculation used $\log h_t$. For every $t > 0$, connectedness makes $h_t^{(v)}$ strictly positive. Applying the identity on $[\varepsilon, T]$ and then taking $\varepsilon \downarrow 0$ and $T \to \infty$ gives the displayed formula.

Thus the logarithm that arose locally from the logarithmic mean has now turned into

$$\log \frac{1}{\mu(v)}$$

after integration over the walk.

12.6.6 Completing the Localization Bound

We now return to

$$\Pi = C^{1/2} B L^{+} B^{\top} C^{1/2}.$$

Assume first that $w_e > 0$ for every edge, so that $\mu_w(x) > 0$ for every vertex.

The integrated-walk representation gives

$$\Pi = \int_0^\infty C^{1/2} B H_t B^{\top} C^{1/2}\, dt.$$

Taking entrywise absolute values,

$$|\Pi| \le \int_0^\infty C^{1/2} |B H_t B^{\top}| C^{1/2}\, dt.$$

Therefore, after writing $t = 2s$,

$$w^{\top} |\Pi| w \le 2 \int_0^\infty w^{\top} C^{1/2} |B H_{2s} B^{\top}| C^{1/2} w\, ds.$$

From the midpoint decomposition,

$$H_{2s} = \sum_v \mu(v) h_s^{(v)} (h_s^{(v)})^{\top},$$

and hence

$$|B H_{2s} B^{\top}| \le \sum_v \mu(v) |B h_s^{(v)}| |B h_s^{(v)}|^{\top}.$$

Thus

$$\begin{aligned} w^{\top} |\Pi| w &\le 2 \sum_v \mu(v) \int_0^\infty \left(w^{\top} C^{1/2} |B h_s^{(v)}| \right)^2 ds \\ &\le 2 \|w\|_2^2 \sum_v \mu(v) \int_0^\infty I(h_s^{(v)})\, ds \\ &= 2 \|w\|_2^2 \sum_v \mu(v) \log \frac{1}{\mu(v)}. \end{aligned}$$

Writing

$$H(\mu) = \sum_v \mu(v) \log \frac{1}{\mu(v)},$$

we have proved

$$\boxed{w^{\top} |\Pi| w \le 2 H(\mu_w) \|w\|_2^2.}$$

For an arbitrary signed vector w,

$$|w^{\top} |\Pi| w| \le |w|^{\top} |\Pi| |w|.$$

If some coordinates of w vanish, apply the preceding argument to

$$|w| + \varepsilon \mathbf{1}$$

and let $\varepsilon \downarrow 0$. With $0 \log 0 = 0$, the entropy is continuous in this limit. Hence for every w,

$$\boxed{|w^{\top} |\Pi| w| \le 2 H(\mu_w) \|w\|_2^2.}$$

Finally, every probability distribution on n points satisfies

$$H(\mu_w) \le \log n.$$

Since $|\Pi|$ is symmetric,

$$\boxed{\| |\Pi| \|_{2 \to 2} \le 2 \log n.}$$

13 Matrix Analysis

13.1 Sampling Matrices without Replacement

We can show that sampling without replacement is more concentrated than sampling with replacement, this works for both scalars and matrices.

Lemma 414 (Hoeffding's Coupling)**.** *Let $\mathcal{C}$ be a finite set of Hermitian matrices. Let $X = (X_1, \dots, X_m)$ be a sequence of m matrices sampled uniformly with replacement from $\mathcal{C}$. Let $Y = (Y_1, \dots, Y_m)$ be a sequence sampled uniformly without replacement from $\mathcal{C}$.*

There exists a randomized surrogate function $Z(Y) = (Z_1, \dots, Z_m)$ such that:

1. *The unconditional distribution of $Z(Y)$ is identical to X.*
2. *The conditional expectation satisfies $\mathbb{E}_{Z|Y}\left[\sum_{i=1}^m Z_i(Y)\right] = \sum_{i=1}^m Y_i$.*

Proof. We construct the sequence $Z_1, \dots, Z_m$ conditionally from the drawn set Y via the following sequential recipe.

For the k-th step, let D_k be the subset of matrices from Y that have already been chosen in the sequence $Z_1, \dots, Z_{k-1}$. We draw Z_k as follows:

- With probability $\frac{|D_k|}{|\mathcal{C}|}$, we sample Z_k uniformly from the already-drawn elements D_k.
- With probability $1 - \frac{|D_k|}{|\mathcal{C}|}$, we sample Z_k uniformly from the remaining un-drawn elements $Y \setminus D_k$.

It is straightforward to verify that the joint probability of drawing any sequence $x \in \mathcal{C}^m$ is $|\mathcal{C}|^{-m}$.

For the second property, observe that the conditional expectation $\mathbb{E}_{Z|Y}[\sum_{i=1}^m Z_i]$ is a linear combination of the variables $Y_1, \dots, Y_m$ and invariant under a permutation of the indices of Y.

It thus must take the form $K \sum_{i=1}^m Y_i$ for some scalar constant K. To determine K, we take the expectation over Y:

$$\mathbb{E}_Y\left[\mathbb{E}_{Z|Y}\left[\sum_{i=1}^m Z_i\right]\right] = m\mathbb{E}[Y_1] \quad \text{and} \quad \mathbb{E}_Y\left[K\sum_{i=1}^m Y_i\right] = Km\mathbb{E}[Y_1]$$

So $K = 1$. □

13.2 Schur and Kronecker Product

Lemma 415 (Kronecker Product Preserves Positivity)**.** *Let A and B be Hermitian matrices. If $A, B \succeq 0$, then $A \otimes B \succeq 0$. Furthermore, if $A, B \succ 0$, then $A \otimes B \succ 0$.*

Proof. Assume $A \succeq 0$ and $B \succeq 0$. Every PSD matrix has a unique PSD square root. We can factor the Kronecker product:

$$A \otimes B = (A^{1/2}A^{1/2}) \otimes (B^{1/2}B^{1/2}) = (A^{1/2} \otimes B^{1/2})(A^{1/2} \otimes B^{1/2})$$

Because the Kronecker product of two Hermitian matrices is Hermitian, $A^{1/2} \otimes B^{1/2}$ is Hermitian. Therefore, we have expressed $A \otimes B$ as the square of a Hermitian matrix, which implies $A \otimes B \succeq 0$.

Now assume $A \succ 0$ and $B \succ 0$. In this case, both A and B are invertible. The Kronecker product of invertible matrices is invertible, with its inverse given by $(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}$. □

The Schur product theorem has an especially transparent interpretation in terms of Gram matrices. A positive semidefinite matrix is a table of inner products; entrywise multiplication simply tensors the vectors producing those inner products.

Lemma 416 (Schur Product Theorem)**.** *Let $A, B \in M_n(\mathbb{C})$ be Hermitian. If $A, B \succeq 0$, then*

$$A \circ B \succeq 0.$$

If $A, B \succ 0$, then $A \circ B \succ 0$.

Proof. Because $A \succeq 0$ and $B \succeq 0$, there are vectors $u_1, \dots, u_n$ and $v_1, \dots, v_n$ such that

$$A_{ij} = \langle u_i, u_j \rangle, \qquad B_{ij} = \langle v_i, v_j \rangle.$$

Therefore

$$(A \circ B)_{ij} = \langle u_i, u_j \rangle \langle v_i, v_j \rangle = \langle u_i \otimes v_i, u_j \otimes v_j \rangle.$$

Thus $A \circ B$ is the Gram matrix of the vectors

$$u_1 \otimes v_1, \dots, u_n \otimes v_n,$$

and every Gram matrix is positive semidefinite.

Now suppose $A, B \succ 0$. Then the Gram vectors $u_1, \dots, u_n$ are linearly independent, and every v_i is nonzero. If

$$\sum_i c_i u_i \otimes v_i = 0,$$

choose, for each j, a linear functional φ_j on $\operatorname{span}\{u_i\}$ satisfying $\varphi_j(u_i) = \delta_{ij}$. Applying $\varphi_j \otimes I$ gives $c_j v_j = 0$, hence $c_j = 0$. So the vectors $u_i \otimes v_i$ are linearly independent, and their Gram matrix $A \circ B$ is positive definite. □

13.3 Differentiating Matrix Functions

For scalar functions, one writes

$$\frac{d}{dt} f(x(t)) = f'(x(t)) x'(t).$$

The analogous formula is generally false for matrices because X and X' need not commute. There are two simple replacements.

For algebraic expressions, differentiate by the ordinary product rule while preserving the order of the factors. For functions defined by spectral calculus, scalar derivatives are replaced by *divided differences*. Inside a trace, only the diagonal divided differences survive, and the usual scalar derivative rule reappears.

For a differentiable matrix-valued map F, write

$$DF(X)[H] = \left. \frac{d}{dt} F(X + tH) \right|_{t=0}.$$

Thus for a differentiable path $X(t)$,

$$\frac{d}{dt} F(X(t)) = DF(X(t))[X'(t)].$$

13.3.1 Product, Inverse, and Power Rules

The ordinary product rule remains valid because multiplication is bilinear; the only care is that the order of the factors cannot be changed.

Lemma 417 (Basic matrix differential rules)**.** *Let $A(t), B(t)$ be differentiable matrix-valued functions.*

Then

$$\boxed{\frac{d}{dt}(AB) = A'B + AB'.}$$

If $A(t)$ is invertible, then

$$\boxed{\frac{d}{dt}A^{-1} = -A^{-1}A'A^{-1}.}$$

For every integer $k \geq 1$,

$$\boxed{\frac{d}{dt}A^{k} = \sum_{r=0}^{k-1} A^{r}A'A^{k-1-r}.}$$

Proof. The product rule follows by inserting and subtracting $A(t+h)B(t)$:

$$\begin{aligned} A(t+h)B(t+h) - A(t)B(t) = {} & \big(A(t+h) - A(t)\big)B(t) \\ & + A(t+h)\big(B(t+h) - B(t)\big). \end{aligned}$$

Divide by h and let $h \to 0$.

For the inverse, differentiate

$$AA^{-1} = I.$$

The product rule gives

$$A'A^{-1} + A(A^{-1})' = 0.$$

Multiplying on the left by A^{-1} yields

$$(A^{-1})' = -A^{-1}A'A^{-1}.$$

Finally, repeatedly applying the product rule to $A^{k} = A \cdots A$ gives one term for each factor at which the derivative may land:

$$(A^{k})' = A'A^{k-1} + AA'A^{k-2} + \cdots + A^{k-1}A'.$$

□

When A commutes with A', all k terms are equal and this reduces to the formula

$$(A^{k})' = kA^{k-1}A'.$$

13.3.2 Divided Differences

Let $f : I \to \mathbb{R}$ be differentiable. Its first divided difference is

$$f^{[1]}(a,b) = \begin{cases} \dfrac{f(a) - f(b)}{a - b}, & a \neq b, \\ f'(a), & a = b. \end{cases}$$

By the fundamental theorem of calculus,

$$\boxed{f^{[1]}(a,b) = \int_{0}^{1} f'\big((1-s)b + sa\big)\, ds.}$$

Thus a divided difference is simply the average scalar derivative between the two eigenvalues.

For a Hermitian matrix

$$X = U \operatorname{diag}(\lambda_1, \ldots, \lambda_n) U^{*},$$

recall that spectral calculus defines

$$f(X) = U \operatorname{diag}\big(f(\lambda_1), \ldots, f(\lambda_n)\big) U^{*}.$$

Theorem 418 (Derivative of a spectral matrix function)**.** *Let X be Hermitian with spectrum in the interior of an interval I, and let $f \in C^1(I)$. For a Hermitian direction H, put*

$$\widetilde{H} = U^* H U.$$

Then

$$\boxed{U^* Df(X)[H]U = \left(f^{[1]}(\lambda_i, \lambda_j)\widetilde{H}_{ij}\right)_{i,j}.}$$

Equivalently,

$$Df(X)[H] = U\left(\left(f^{[1]}(\lambda_i, \lambda_j)\right)_{i,j} \circ \widetilde{H}\right) U^*.$$

Proof. First take $f(x) = x^k$. By Lemma 417,

$$D(X^k)[H] = \sum_{r=0}^{k-1} X^r H X^{k-1-r}.$$

In an eigenbasis of X, its ij-entry is

$$\widetilde{H}_{ij} \sum_{r=0}^{k-1} \lambda_i^r \lambda_j^{k-1-r}.$$

If $\lambda_i \neq \lambda_j$, the geometric-sum identity gives

$$\sum_{r=0}^{k-1} \lambda_i^r \lambda_j^{k-1-r} = \frac{\lambda_i^k - \lambda_j^k}{\lambda_i - \lambda_j}.$$

If $\lambda_i = \lambda_j$, the sum is $k\lambda_i^{k-1}$. Thus in either case the coefficient is $f^{[1]}(\lambda_i, \lambda_j)$. Linearity proves the formula for polynomials.

For a general C^1 function, let $\theta_1, \ldots, \theta_r$ be the distinct eigenvalues of X. Choose a Hermite interpolation polynomial p satisfying

$$p(\theta_a) = f(\theta_a), \qquad p'(\theta_a) = f'(\theta_a) \qquad (a = 1, \ldots, r).$$

Put $g = f - p$. Then

$$g(\theta_a) = g'(\theta_a) = 0,$$

so uniformly over the finitely many eigenvalues,

$$g(\theta_a + s) = o(|s|) \qquad (s \to 0).$$

If E is Hermitian and small, every eigenvalue of $X + E$ lies within $\|E\|$ of an eigenvalue of X. Indeed,

$$-\|E\| I \preceq E \preceq \|E\| I$$

gives

$$X - \|E\| I \preceq X + E \preceq X + \|E\| I,$$

and hence the same bound on the ordered eigenvalues. Therefore

$$\|g(X + E)\| = \max_{\mu \in \sigma(X+E)} |g(\mu)| = o(\|E\|).$$

Since $g(X) = 0$,

$$f(X + E) - f(X) = p(X + E) - p(X) + o(\|E\|).$$

Thus f and p have the same Fréchet derivative at X.

Finally, p agrees with f at every eigenvalue of X, and their derivatives agree there as well, so

$$p^{[1]}(\lambda_i, \lambda_j) = f^{[1]}(\lambda_i, \lambda_j).$$

The polynomial formula therefore gives the stated result. □

Thus we use the ordinary derivative on one eigenspace and divided difference between two eigenspaces. In particular, if X and H commute, they can be simultaneously diagonalized and

$$Df(X)[H] = f'(X)H.$$

13.3.3 Traces Recover the Scalar Derivative Rule

Although $Df(X)[H]$ is not generally $f'(X)H$, its trace behaves as though the matrices commuted.

Corollary 419 (Derivative of a trace spectral function)**.** *Let*

$$F(X) = \operatorname{Tr} f(X).$$

Then

$$\boxed{DF(X)[H] = \operatorname{Tr}\big(f'(X)H\big).}$$

With the trace inner product, therefore,

$$\boxed{\nabla F(X) = f'(X).}$$

Proof. Work in an eigenbasis of X. Trace keeps only the diagonal entries of $Df(X)[H]$, and on the diagonal

$$f^{[1]}(\lambda_i, \lambda_i) = f'(\lambda_i).$$

Hence

$$\begin{aligned} DF(X)[H] &= \sum_i f'(\lambda_i)\widetilde{H}_{ii} \\ &= \operatorname{Tr}\big(f'(X)H\big). \end{aligned}$$

□

There is also a second-order version.

Corollary 420 (Hessian of a trace spectral function)**.** *Let $f \in C^2(I)$, and let*

$$F(X) = \operatorname{Tr} f(X).$$

In an eigenbasis of $X = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$,

$$\boxed{D^2F(X)[H,H] = \sum_{i,j} (f')^{[1]}(\lambda_i, \lambda_j)|H_{ij}|^2.}$$

Proof. By Corollary 419,

$$\nabla F(X) = f'(X).$$

Therefore

$$D^2F(X)[H,H] = \operatorname{Tr}\big(H\, D(f')(X)[H]\big).$$

Apply Theorem 418 to f':

$$\big(D(f')(X)[H]\big)_{ij} = (f')^{[1]}(\lambda_i, \lambda_j)H_{ij}.$$

Since H is Hermitian, $H_{ji} = \overline{H_{ij}}$, and hence

$$\begin{aligned} \operatorname{Tr}\big(H\, D(f')(X)[H]\big) &= \sum_{i,j} H_{ji}(f')^{[1]}(\lambda_i, \lambda_j)H_{ij} \\ &= \sum_{i,j} (f')^{[1]}(\lambda_i, \lambda_j)|H_{ij}|^2. \end{aligned}$$

□

Examples. For $f(x) = x^{-1}$,

$$f^{[1]}(a, b) = -\frac{1}{ab},$$

including the limiting value $-1/a^2$ when $a = b$. Thus

$$D(X^{-1})[H] = -X^{-1}HX^{-1},$$

recovering the inverse rule.

For matrix entropy,

$$h(X) = \operatorname{Tr}(X \log X),$$

take $f(x) = x \log x$. Since

$$f'(x) = 1 + \log x,$$

Corollary 419 gives immediately

$$\boxed{\nabla h(X) = I + \log X.}$$

The Hessian is

$$\boxed{D^2h(X)[H, H] = \sum_{i,j} (\log)^{[1]}(\lambda_i, \lambda_j)|H_{ij}|^2.}$$

Likewise,

$$\log \det X = \operatorname{Tr} \log X,$$

so

$$\boxed{D(\log \det X)[H] = \operatorname{Tr}(X^{-1}H).}$$

Exponentiating gives Jacobi's formula

$$\boxed{D(\det X)[H] = \det(X) \operatorname{Tr}(X^{-1}H).}$$

Finally, when the *exponent* rather than the matrix varies, no noncommutativity issue arises. For $X \succ 0$,

$$X^s = U \operatorname{diag}(\lambda_i^s) U^*,$$

so differentiating the scalar eigenvalues gives

$$\boxed{\frac{d}{ds} X^s = X^s \log X = (\log X) X^s.}$$

In particular,

$$\frac{d}{ds} X^s \bigg|_{s=0} = \log X, \qquad \frac{d}{ds} X^{1-s} \bigg|_{s=0} = -X \log X.$$

13.3.4 The Matrix Exponential

There is also another useful formula for the derivative of the matrix exponential.

Proposition 421 (Derivative of the matrix exponential)**.** *If $X(t)$ is differentiable, then*

$$\boxed{\frac{d}{dt} e^{X(t)} = \int_0^1 e^{\alpha X(t)} X'(t) e^{(1-\alpha)X(t)} \, d\alpha.}$$

Equivalently,

$$De^X[H] = \int_0^1 e^{\alpha X} H e^{(1-\alpha)X} \, d\alpha.$$

Proof. We first prove the finite-difference identity

$$e^A - e^B = \int_0^1 e^{\alpha A}(A - B)e^{(1-\alpha)B}\, d\alpha.$$

Define

$$F(\alpha) = e^{\alpha A}e^{(1-\alpha)B}.$$

Since A commutes with $e^{\alpha A}$ and B commutes with $e^{(1-\alpha)B}$,

$$\begin{aligned} F'(\alpha) &= Ae^{\alpha A}e^{(1-\alpha)B} - e^{\alpha A}Be^{(1-\alpha)B} \\ &= e^{\alpha A}(A - B)e^{(1-\alpha)B}. \end{aligned}$$

Thus the fundamental theorem of calculus gives

$$\begin{aligned} e^A - e^B &= F(1) - F(0) \\ &= \int_0^1 e^{\alpha A}(A - B)e^{(1-\alpha)B}\, d\alpha. \end{aligned}$$

Now take

$$A = X(t + h), \qquad B = X(t).$$

Then

$$\frac{e^{X(t+h)} - e^{X(t)}}{h} = \int_0^1 e^{\alpha X(t+h)} \frac{X(t + h) - X(t)}{h} e^{(1-\alpha)X(t)}\, d\alpha.$$

As $h \to 0$,

$$X(t + h) \to X(t), \qquad \frac{X(t + h) - X(t)}{h} \to X'(t),$$

so continuity of the matrix exponential lets us pass the limit through the integral. Hence

$$\frac{d}{dt}e^{X(t)} = \int_0^1 e^{\alpha X(t)}X'(t)e^{(1-\alpha)X(t)}\, d\alpha.$$

Taking $X(t) = X + tH$ gives the Fréchet derivative formula. □

If X commutes with H, the integrand is independent of α and we recover the scalar formula.

13.4 Integral Representations and Fractional Powers

13.4.1 The Logarithm

The logarithm has a simpler resolvent representation than the fractional powers, so it is best proved directly.

Lemma 422 (Resolvent representation of the logarithm)**.** *For every $A \succ 0$,*

$$\boxed{\log A = \int_0^\infty \left(\frac{1}{1 + t}I - (A + tI)^{-1}\right) dt.}$$

The integral converges in operator norm.

Proof. Start with the scalar identity. For $x > 0$ and $R > 0$,

$$\begin{aligned} \int_0^R \left(\frac{1}{1 + t} - \frac{1}{x + t}\right) dt &= [\log(1 + t) - \log(x + t)]_0^R \\ &= \log(1 + R) - \log(x + R) + \log x \\ &= \log x + \log \frac{1 + R}{x + R}. \end{aligned}$$

As $R \to \infty$,

$$\frac{1+R}{x+R} \to 1,$$

so

$$\boxed{\log x = \int_0^\infty \left(\frac{1}{1+t} - \frac{1}{x+t} \right) dt.}$$

Now diagonalize

$$A = U \operatorname{diag}(\lambda_1, \ldots, \lambda_n) U^*.$$

Then

$$(A+tI)^{-1} = U \operatorname{diag}\left(\frac{1}{\lambda_1 + t}, \ldots, \frac{1}{\lambda_n + t} \right) U^*.$$

Applying the scalar identity to each eigenvalue gives

$$\begin{aligned} \int_0^\infty \left(\frac{1}{1+t} I - (A+tI)^{-1} \right) dt &= U \operatorname{diag}(\log \lambda_1, \ldots, \log \lambda_n) U^* \\ &= \log A. \end{aligned}$$

Since there are finitely many positive eigenvalues and the scalar integrand is $O(t^{-2})$ as $t \to \infty$, the integral converges in operator norm. □

Differentiating this representation gives another useful form of the Fréchet derivative. By the inverse rule,

$$D\big((X+tI)^{-1}\big)[H] = -(X+tI)^{-1} H (X+tI)^{-1}.$$

Therefore

$$\boxed{D \log(X)[H] = \int_0^\infty (X+tI)^{-1} H (X+tI)^{-1} \, dt.}$$

Differentiation under the integral is justified locally uniformly for $X \succ 0$, since the differentiated integrand is $O((1+t)^{-2})$.

13.4.2 Fractional Powers

Many matrix inequalities for fractional powers come from one scalar formula. For $0 < \alpha < 1$, let

$$c_\alpha = \frac{\sin(\pi\alpha)}{\pi}.$$

For $0 < \alpha < 1$, set

$$I_\alpha = \int_0^\infty \frac{x^{\alpha-1}}{1+x} \, dx.$$

Consider

$$f(z) = \frac{z^{\alpha-1}}{1+z},$$

using the branch

$$0 < \arg z < 2\pi,$$

so the branch cut is the positive real axis. Integrate around a keyhole contour enclosing the slit plane.

On the upper side of the positive real axis,

$$z^{\alpha-1} = x^{\alpha-1},$$

so the contribution tends to I_α.

On the lower side, $\arg z = 2\pi$, and the contour runs from ∞ back to 0. Hence

$$z^{\alpha-1} = e^{2\pi i(\alpha-1)}x^{\alpha-1} = e^{2\pi i\alpha}x^{\alpha-1},$$

and its contribution tends to

$$-e^{2\pi i\alpha}I_\alpha.$$

The two circular arcs vanish: on the outer circle their contribution is

$$O(R^{\alpha-1}) \to 0,$$

while on the inner circle it is

$$O(\varepsilon^\alpha) \to 0.$$

The only pole inside the contour is $z = -1$. Since $\arg(-1) = \pi$,

$$\operatorname{Res}_{z=-1} f(z) = (-1)^{\alpha-1} = e^{i\pi(\alpha-1)} = -e^{i\pi\alpha}.$$

Therefore the residue theorem gives

$$(1 - e^{2\pi i\alpha})I_\alpha = -2\pi i e^{i\pi\alpha}.$$

Using

$$1 - e^{2\pi i\alpha} = -2ie^{i\pi\alpha}\sin(\pi\alpha),$$

we obtain

$$\boxed{I_\alpha = \frac{\pi}{\sin(\pi\alpha)}.}$$

Consequently, after the substitution $t = xs$,

$$\int_0^\infty t^{\alpha-1}\frac{x}{t+x}\,dt = x^\alpha\int_0^\infty \frac{s^{\alpha-1}}{1+s}\,ds = x^\alpha\frac{\pi}{\sin(\pi\alpha)}.$$

Thus

$$x^\alpha = \frac{\sin(\pi\alpha)}{\pi}\int_0^\infty t^{\alpha-1}\frac{x}{t+x}\,dt.$$

Applying the scalar identity to every eigenvalue gives the matrix formula.

Lemma 423 (Formula for Fractional Powers)**.** *For $A \succ 0$ and $0 < \alpha < 1$,*

$$\boxed{A^\alpha = c_\alpha\int_0^\infty t^{\alpha-1}A(tI+A)^{-1}\,dt.}$$

Equivalently,

$$A^\alpha = c_\alpha\int_0^\infty t^\alpha\left(\frac{1}{t}I - (tI+A)^{-1}\right)dt.$$

Two other useful forms follow immediately. If $-1 < p < 0$, then

$$\boxed{A^p = \frac{\sin(-\pi p)}{\pi}\int_0^\infty t^p(tI+A)^{-1}\,dt.}$$

If $1 < p < 2$, applying the preceding formula to A^{p-1} and multiplying by A gives

$$\boxed{A^p = c_{p-1}\int_0^\infty t^{p-2}A^2(tI+A)^{-1}\,dt.}$$

Using

$$A^2(tI + A)^{-1} = A - tI + t^2(tI + A)^{-1}$$

recovers the more expanded resolvent formula when that form is convenient.

We also have the scalar identity

$$|x|^\alpha = c'_\alpha \int_0^\infty \frac{1 - \cos(tx)}{t^{\alpha+1}}\, dt, \qquad 0 < \alpha < 2,$$

for a positive constant c'_α.

13.4.3 Operator Convexity of Inversion

The formulas are useful because inversion has a very simple matrix-convexity property.

Lemma 424 (Operator Convexity of Inversion)**.** *For $A, B \succ 0$ and $0 \le \lambda \le 1$,*

$$(\lambda A + (1 - \lambda)B)^{-1} \preceq \lambda A^{-1} + (1 - \lambda)B^{-1}.$$

Proof. For every $A \succ 0$,

$$\begin{pmatrix} A^{-1} & I \\ I & A \end{pmatrix} = \begin{pmatrix} A^{-1/2} \\ A^{1/2} \end{pmatrix} \begin{pmatrix} A^{-1/2} & A^{1/2} \end{pmatrix} \succeq 0.$$

Average this matrix with its analogue for B:

$$\begin{pmatrix} \lambda A^{-1} + (1 - \lambda)B^{-1} & I \\ I & \lambda A + (1 - \lambda)B \end{pmatrix} \succeq 0.$$

Taking the Schur complement of the lower-right block gives exactly the claimed inequality. □

We also repeatedly use that inversion reverses the Loewner order:

$$0 \prec A \preceq B \qquad \Longrightarrow \qquad B^{-1} \preceq A^{-1}.$$

Indeed,

$$I \preceq A^{-1/2} B A^{-1/2}.$$

Inverting reverses the inequality,

$$A^{1/2} B^{-1} A^{1/2} \preceq I,$$

and conjugating by $A^{-1/2}$ gives $B^{-1} \preceq A^{-1}$.

13.4.4 Löwner–Heinz and Operator Concavity of Powers

The same formula now gives two fundamental facts about fractional powers essentially for free.

Theorem 425 (Löwner–Heinz)**.** *If*

$$0 \preceq A \preceq B$$

and $0 \le \alpha \le 1$, then

$$\boxed{A^\alpha \preceq B^\alpha.}$$

Thus $x \mapsto x^\alpha$ is operator monotone on the positive cone.

Proof. It is enough to consider $0 < \alpha < 1$ and positive definite A, B; the semidefinite case follows by replacing A, B with $A + \varepsilon I, B + \varepsilon I$ and taking $\varepsilon \downarrow 0$.

For every $t > 0$,

$$A \preceq B \quad \Longrightarrow \quad (tI + A)^{-1} \succeq (tI + B)^{-1}.$$

Therefore

$$A(tI + A)^{-1} = I - t(tI + A)^{-1} \preceq I - t(tI + B)^{-1} = B(tI + B)^{-1}.$$

Integrating this inequality against the positive weight $c_\alpha t^{\alpha-1}$ proves $A^\alpha \preceq B^\alpha$. □

Lemma 426 (Operator Concavity of Fractional Powers)**.** *For $0 \le \alpha \le 1$, the map*

$$A \longmapsto A^\alpha$$

is operator concave on the positive cone.

Proof. Again take $0 < \alpha < 1$. For fixed $t > 0$,

$$A(tI + A)^{-1} = I - t(tI + A)^{-1}.$$

By Lemma 424, the map

$$A \longmapsto (tI + A)^{-1}$$

is operator convex. Hence its negative, and therefore $A \mapsto A(tI + A)^{-1}$, is operator concave.

The representation Lemma 423 is a positive weighted integral of these maps, so

$$A \longmapsto A^\alpha$$

is operator concave. □

13.5 Golden–Thompson Inequality

The proof below has one basic mechanism: repeated Cauchy–Schwarz makes a word in A and A^* more alternating. The completely alternating word is therefore the extremal one. The Lie–Trotter formula then turns this "word-straightening" statement into Golden–Thompson.

Lemma 427 (Word Straightening)**.** *Let $A \in M_d(\mathbb{C})$, and let*

$$A_1, \dots, A_{2n} \in \{A, A^*\}.$$

Then

$$\boxed{|\mathrm{Tr}(A_1 \cdots A_{2n})| \le \mathrm{Tr}((A^*A)^n).}$$

Proof. If $A = A^*$ the claim is immediate. Otherwise regard A and A^* as two distinct symbols.

Among all length-$2n$ words in A, A^*, choose one

$$P = A_1 \cdots A_{2n}$$

maximizing $|\operatorname{Tr} P|$, and among all such maximizers choose one with the largest number of transitions between A and A^* in cyclic order.

If every adjacent pair is different, the word alternates, so it is a cyclic shift of either

$$(A^*A)^n \qquad \text{or} \qquad (AA^*)^n.$$

These have the same trace, and we are done.

Otherwise two adjacent symbols agree. By cyclically rotating the word, we may arrange that the equal pair occurs across the middle. Write

$$P = QR, \qquad Q = A_1 \cdots A_n, \qquad R = A_{n+1} \cdots A_{2n},$$

with

$$A_n = A_{n+1}.$$

Frobenius Cauchy–Schwarz gives

$$|\operatorname{Tr}(QR)|^2 \le \operatorname{Tr}(Q^*Q) \operatorname{Tr}(R^*R).$$

Both Q^*Q and R^*R are again length-$2n$ words in A, A^*. Since P was a maximizer, neither trace magnitude can exceed $|\operatorname{Tr} P|$. Hence equality must hold throughout, and both Q^*Q and R^*R are themselves maximizers.

It remains to compare transitions. Let t_Q, t_R be the numbers of transitions internal to Q, R. The chosen middle pair contributes no transition, so P has at most

$$t_Q + t_R + 1$$

cyclic transitions.

In Q^*Q, the transitions internal to Q occur once in each half, and the two junctions are automatically transitions because a letter meets its adjoint. Thus Q^*Q has exactly

$$2t_Q + 2$$

transitions, and similarly R^*R has $2t_R + 2$. Their average is

$$t_Q + t_R + 2,$$

strictly larger than the number of transitions of P. Therefore one of Q^*Q, R^*R is a maximizing word with more transitions than P, a contradiction.

So every maximizing word alternates, proving the claim. □

Lemma 428 (Disentangling)**.** *Let U, V be Hermitian and let $k \geq 1$. Then*

$$\boxed{\begin{aligned}\left|\operatorname{Tr}((UV)^{2^k})\right| &\leq \operatorname{Tr}((U^2V^2)^{2^{k-1}}) \\ &\leq \operatorname{Tr}((U^4V^4)^{2^{k-2}}) \\ &\leq \cdots \leq \operatorname{Tr}(U^{2^k}V^{2^k}).\end{aligned}}$$

Proof. Apply Lemma 427 to the matrix $A = UV$ and to the word consisting of 2^k copies of A. Since $A^* = VU$,

$$\left|\operatorname{Tr}((UV)^{2^k})\right| \leq \operatorname{Tr}((VU^2V)^{2^{k-1}}).$$

Cyclicity of trace turns the right-hand side into

$$\operatorname{Tr}((U^2V^2)^{2^{k-1}}).$$

Now repeat with U^2, V^2, then U^4, V^4, and so on. □

Theorem 429 (Golden–Thompson)**.** *For Hermitian A, B,*

$$\boxed{\operatorname{Tr}(e^{A+B}) \leq \operatorname{Tr}(e^A e^B).}$$

Proof. The Lie–Trotter product formula says

$$e^{A+B} = \lim_{N\to\infty} \left(e^{A/N} e^{B/N}\right)^N.$$

For completeness, this follows from

$$e^{A/N} e^{B/N} = I + \frac{A+B}{N} + O(N^{-2})$$

in operator norm. For sufficiently large N the matrix on the left lies near I, so its matrix logarithm is defined there and

$$\log(e^{A/N} e^{B/N}) = \frac{A+B}{N} + O(N^{-2}).$$

Thus

$$\left(e^{A/N}e^{B/N}\right)^N = \exp\left(A + B + O(N^{-1})\right) \longrightarrow e^{A+B}.$$

Take $N = 2^k$ and set

$$U = e^{A/2^k}, \qquad V = e^{B/2^k}.$$

These are positive definite, hence Hermitian. By Lemma 428,

$$\left|\mathrm{Tr}\left((e^{A/2^k}e^{B/2^k})^{2^k}\right)\right| \leq \mathrm{Tr}(e^A e^B).$$

Letting $k \to \infty$, the matrix inside the trace converges to e^{A+B}. Since $e^{A+B} \succ 0$, its trace is positive, and the limit gives

$$\mathrm{Tr}(e^{A+B}) \leq \mathrm{Tr}(e^A e^B).$$

□

13.6 Trace Rearrangement and Schatten Hölder Inequalities

The matrix analogue of the scalar rearrangement inequality is von Neumann's trace inequality. Once this is established, the two-factor Schatten Hölder inequality is literally scalar Hölder applied to the singular values. Exterior powers then give the corresponding product inequality and hence the general multi-factor trace form.

Lemma 430 (Trace Rearrangement for Hermitian Matrices)**.** *Let X, Y be Hermitian with eigenvalues*

$$\lambda_1(X) \geq \cdots \geq \lambda_n(X), \qquad \lambda_1(Y) \geq \cdots \geq \lambda_n(Y).$$

Then

$$\boxed{\mathrm{Tr}(XY) \leq \sum_{i=1}^{n} \lambda_i(X)\lambda_i(Y).}$$

Proof. Write

$$X = \sum_i \lambda_i(X) u_i u_i^*, \qquad Y = \sum_j \lambda_j(Y) v_j v_j^*.$$

Then

$$\mathrm{Tr}(XY) = \sum_{i,j} \lambda_i(X)\lambda_j(Y)|\langle u_i, v_j\rangle|^2.$$

The matrix

$$M_{ij} = |\langle u_i, v_j\rangle|^2$$

is doubly stochastic, because both (u_i) and (v_j) are orthonormal bases.

By the Birkhoff–von Neumann theorem, M is a convex combination of permutation matrices. Therefore $\mathrm{Tr}(XY)$ is a convex combination of quantities

$$\sum_i \lambda_i(X)\lambda_{\pi(i)}(Y).$$

The scalar rearrangement inequality says that each is at most the aligned sum

$$\sum_i \lambda_i(X)\lambda_i(Y).$$

□

Theorem 431 (Von Neumann's Trace Inequality)**.** *For arbitrary $A, B \in M_n(\mathbb{C})$,*

$$\boxed{|\operatorname{Tr}(A^*B)| \le \sum_{i=1}^{n} \sigma_i(A)\sigma_i(B),}$$

where the singular values are sorted in decreasing order.

Proof. For a matrix A, form its Hermitian dilation

$$\mathcal{H}(A) = \begin{pmatrix} 0 & A \\ A^* & 0 \end{pmatrix}.$$

If

$$A = U\Sigma V^*$$

is a singular-value decomposition, then the eigenvalues of $\mathcal{H}(A)$ are

$$\sigma_1(A), \dots, \sigma_n(A), -\sigma_n(A), \dots, -\sigma_1(A).$$

Let

$$z = \operatorname{Tr}(A^*B).$$

If $z = 0$ there is nothing to prove. Otherwise multiply B by a complex phase so that z becomes positive real; this does not change the singular values of B. Then

$$\operatorname{Tr}(\mathcal{H}(A)\mathcal{H}(B)) = 2\operatorname{Re}\operatorname{Tr}(A^*B) = 2|z|.$$

Apply Lemma 430 to the two Hermitian dilations. Pairing their ordered eigenvalues gives

$$2|z| \le 2\sum_{i=1}^{n} \sigma_i(A)\sigma_i(B).$$

Divide by 2. □

Corollary 432 (Schatten Hölder, Trace Form)**.** *If*

$$\frac{1}{p} + \frac{1}{q} = 1, \qquad 1 \le p, q \le \infty,$$

then

$$\boxed{|\operatorname{Tr}(A^*B)| \le \|A\|_{S_p}\|B\|_{S_q}.}$$

Proof. By von Neumann's inequality and scalar Hölder,

$$\begin{aligned} |\operatorname{Tr}(A^*B)| &\le \sum_i \sigma_i(A)\sigma_i(B) \\ &\le \left(\sum_i \sigma_i(A)^p\right)^{1/p} \left(\sum_i \sigma_i(B)^q\right)^{1/q} \\ &= \|A\|_{S_p}\|B\|_{S_q}. \end{aligned}$$

The endpoint cases have the usual interpretation. □

The same exterior-power idea gives the stronger product form. Recall that $\bigwedge^k A$ acts on $\bigwedge^k \mathbb{C}^n$, satisfies

$$\bigwedge^k (AB) = (\bigwedge^k A)(\bigwedge^k B),$$

and has operator norm

$$\left\| \bigwedge^k A \right\| = \prod_{i=1}^k \sigma_i(A).$$

The last identity follows immediately from an SVD: the singular values of $\bigwedge^k A$ are all k-fold products of singular values of A, and the largest such product uses the k largest singular values.

Lemma 433 (Singular Values of a Product)**.** *For every k,*

$$\boxed{\prod_{i=1}^k \sigma_i(AB) \le \prod_{i=1}^k \sigma_i(A)\sigma_i(B).}$$

Equivalently,

$$\sigma(AB) \prec_{w,\log} \sigma(A)\sigma(B),$$

where the product on the right is coordinatewise.

Proof. By multiplicativity of exterior powers and submultiplicativity of the operator norm,

$$\begin{aligned}
\prod_{i=1}^k \sigma_i(AB) &= \left\| \bigwedge^k (AB) \right\| \\
&= \left\| (\bigwedge^k A)(\bigwedge^k B) \right\| \\
&\le \left\| \bigwedge^k A \right\| \left\| \bigwedge^k B \right\| \\
&= \prod_{i=1}^k \sigma_i(A)\sigma_i(B).
\end{aligned}$$

□

Theorem 434 (Schatten Hölder, Product Form)**.** *Let*

$$1 \le p, q, r \le \infty, \qquad \frac{1}{r} = \frac{1}{p} + \frac{1}{q}.$$

Then

$$\boxed{\|AB\|_{S_r} \le \|A\|_{S_p} \|B\|_{S_q}.}$$

Consequently, if $p_1, \dots, p_m \in [1, \infty]$ satisfy

$$\sum_{j=1}^m \frac{1}{p_j} = 1,$$

then

$$\boxed{|\operatorname{Tr}(A_1 \cdots A_m)| \le \prod_{j=1}^m \|A_j\|_{S_{p_j}}.}$$

Proof. Assume first that A, B are invertible; the general case follows by regularization and continuity.

By Lemma 433, the logarithms of the singular values of AB are weakly majorized by

$$\log \sigma_i(A) + \log \sigma_i(B).$$

Applying the increasing convex function

$$x \longmapsto e^{rx}$$

gives

$$\sum_i \sigma_i(AB)^r \le \sum_i \big(\sigma_i(A)\sigma_i(B)\big)^r.$$

Scalar Hölder with conjugate exponents p/r and q/r now gives

$$\sum_i \big(\sigma_i(A)\sigma_i(B)\big)^r \le \left(\sum_i \sigma_i(A)^p\right)^{r/p} \left(\sum_i \sigma_i(B)^q\right)^{r/q}.$$

Taking the r-th root proves

$$\|AB\|_{S_r} \le \|A\|_{S_p} \|B\|_{S_q}.$$

For the multi-factor form, define

$$\frac{1}{r_j} = \sum_{i=1}^{j} \frac{1}{p_i}.$$

Since the total reciprocal sum is 1, each intermediate $r_j \ge 1$. Repeatedly applying the product inequality gives

$$\|A_1 \cdots A_m\|_{S_1} \le \prod_{j=1}^{m} \|A_j\|_{S_{p_j}}.$$

Finally,

$$|\operatorname{Tr} X| \le \|X\|_{S_1}$$

by Theorem 431 applied to I and X. □

13.7 Generalized Klein Inequality

Lemma 435 (Generalized Klein Inequality)**.** *Let $f_i, g_i : I \to \mathbb{R}$ be functions on an interval I such that $\sum_i f_i(x)g_i(y) \ge 0$ for all $x, y \in I$. If A and H are Hermitian matrices with eigenvalues in I, then $\sum_i \operatorname{Tr}[f_i(A)g_i(H)] \ge 0$.*

Proof. Consider the eigenvalue decompositions $A = \sum_j \lambda_j u_j u_j^*$ and $H = \sum_k \mu_k v_k v_k^*$. We evaluate the sum of traces:

$$\sum_i \operatorname{Tr}[f_i(A)g_i(H)] = \sum_i \operatorname{Tr}\left[\left(\sum_j f_i(\lambda_j) u_j u_j^*\right)\left(\sum_k g_i(\mu_k) v_k v_k^*\right)\right]$$

Using the linearity and cyclic property of the trace, this simplifies to:

$$\sum_{j,k}\left[\sum_i f_i(\lambda_j)g_i(\mu_k)\right] \operatorname{Tr}(u_j u_j^* v_k v_k^*) = \sum_{j,k}\left[\sum_i f_i(\lambda_j)g_i(\mu_k)\right] |\langle u_j, v_k\rangle|^2$$

By assumption, the bracketed sum is non-negative for all eigenvalue pairs (λ_j, μ_k), and the squared inner product $|\langle u_j, v_k\rangle|^2 \ge 0$. Thus, the entire sum is non-negative. □

13.8 Wielandt Minimax Formula

Theorem 436 (Wielandt Minimax Formula)**.** *Let A be an $n \times n$ Hermitian matrix with eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$. For any sequence of indices $1 \leq i_1 < i_2 < \cdots < i_k \leq n$,*

$$\sum_{j=1}^{k} \lambda_{i_j}(A) = \max_{\substack{V_1 \subset \cdots \subset V_k \\ \dim(V_j) = i_j}} \min_{W \in X(V_1, \ldots, V_k)} \operatorname{tr}_W(A)$$

where $X(V_1, \ldots, V_k)$ denotes the set of k-dimensional subspaces W that admit a basis $v_1, \ldots, v_k$ with $v_j \in V_j$.

Proof. **The $\leq$ case:** Let $V_j = \operatorname{span}(e_1, \ldots, e_{i_j})$, where e_i are the eigenvectors of A. For any $W \in X(V_1, \ldots, V_k)$, it remains to show that $\sum_{j=1}^{k} \lambda_{i_j}(A) \leq \operatorname{tr}_W(A)$.

To show this, we construct an orthonormal set of vectors $v_1, \ldots, v_k$ such that $v_j \in V_j \cap W$. Since $\dim(V_1 \cap W) \geq 1$, we pick any unit vector $v_1 \in V_1 \cap W$. Next, since $\dim(V_2 \cap W) \geq 2$, we pick any unit vector $v_2 \in V_2 \cap W$ that is perpendicular to v_1, and so on. Then we have

$$\operatorname{tr}_W(A) \geq \sum_{j=1}^{k} \langle v_j, A v_j \rangle \geq \sum_{j=1}^{k} \lambda_{i_j}(A),$$

where the last inequality follows from the Rayleigh quotient minimum property on the spaces V_j.

The $\geq$ case: For any sequence of subspaces $V_1 \subset \cdots \subset V_k$ with $\dim(V_j) = i_j$, we must find some $W \in X(V_1, \ldots, V_k)$ such that $\sum_{j=1}^{k} \lambda_{i_j}(A) \geq \operatorname{tr}_W(A)$.

We prove this by induction on n. The $n = 1$ case is trivially the Courant-Fischer theorem. Assume now $n \geq 2$.

If $i_1 \geq 2$, then we can apply induction. Let $E = \operatorname{span}(e_{i_1}, \ldots, e_n)$. We construct a partial flag within E from the intersection of E with $V_1, \ldots, V_k$. We begin by picking an $(i_k - (i_1 - 1))$-dimensional subspace $W'_k \subset E \cap V_k$, which exists by counting dimensions (it has codimension $i_1 - 1$ within V_k). Then we go down by one space, picking an $(i_{k-1} - (i_1 - 1))$-dimensional subspace $W'_{k-1} \subset W'_k \cap V_{k-1}$. This still exists. We repeat this to form a flag $W'_1 \subset \cdots \subset W'_k$. Now, since $\dim(E) \leq n - 1$, we can apply the induction hypothesis: there exists some $W \in X(W'_1, \ldots, W'_k)$ such that

$$\sum_{j=1}^{k} \lambda_{i_j - (i_1 - 1)}(A|_E) \geq \operatorname{tr}_W(A).$$

Note that $\lambda_{i_j - (i_1 - 1)}(A|_E)$ is the $(i_j - (i_1 - 1))$-th eigenvalue of A orthogonally projected down to E. By the Cauchy interlacing theorem, $\lambda_{i_j - (i_1 - 1)}(A|_E) \leq \lambda_{i_j}(A)$. Since $X(W'_1, \ldots, W'_k) \subset X(V_1, \ldots, V_k)$, we are done.

If $i_1 = 1$, we perform a similar construction. Let $E = \operatorname{span}(e_2, \ldots, e_n)$. If $V_k \subset E$, then we can induct immediately. Otherwise, we construct a partial flag sequence $W'_2 \subset \cdots \subset W'_k$ inside E. By induction, there exists some $W' \in X(W'_2, \ldots, W'_k) \subset X(V_2, \ldots, V_k)$ such that

$$\sum_{j=2}^{k} \lambda_{i_j - 1}(A|_E) \geq \operatorname{tr}_{W'}(A).$$

Thus, by Cauchy interlacing, $\sum_{j=2}^{k} \lambda_{i_j}(A) \geq \operatorname{tr}_{W'}(A)$. It remains to find some vector v such that $W' \oplus \operatorname{span}(v) \in X(V_1, \ldots, V_k)$. If $V_1 \not\subset W'$, then any $v \in V_1 \setminus W'$ would work. Otherwise, if $V_2 \not\subset W'$, then any $v \in V_2 \setminus W'$ would work, and so on. If none of these work, it implies $V_k \subset E$, which is a contradiction. This completes the induction. □

Corollary 437 (Weyl's Inequality)**.** *Let A and B be $n \times n$ Hermitian matrices with eigenvalues sorted in descending order: $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$. For all $1 \leq i, j \leq n$ such that $i + j - 1 \leq n$, we have:*

$$\lambda_{i+j-1}(A + B) \leq \lambda_i(A) + \lambda_j(B)$$

Theorem 438 (Lidskii's Inequality)**.** *Let A and B be $n \times n$ Hermitian matrices with their eigenvalues sorted in descending order, $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$. For any sequence of indices $1 \leq i_1 < \cdots < i_k \leq n$, it holds that*

$$\sum_{j=1}^{k} \lambda_{i_j}(A+B) \leq \sum_{j=1}^{k} \lambda_{i_j}(A) + \sum_{j=1}^{k} \lambda_j(B)$$
$$\sum_{j=1}^{k} \lambda_{i_j}(A+B) \geq \sum_{j=1}^{k} \lambda_{i_j}(A) + \sum_{j=1}^{k} \lambda_{n-j+1}(B)$$

Corollary 439 (Extremal Partial Trace)**.** *Let A be a Hermitian matrix. We have:*

$$\sum_{j=1}^{k} \lambda_j(A) = \sup_{\dim(V)=k} \operatorname{tr}_V(A)$$
$$\sum_{j=1}^{k} \lambda_{n-j+1}(A) = \inf_{\dim(V)=k} \operatorname{tr}_V(A)$$

Corollary 440 (Schur-Horn Inequality)**.** *The sum of the largest k eigenvalues is a convex function, and the sum of the smallest k eigenvalues is a concave function. Furthermore, for any subset of indices $i_1 < \cdots < i_k$,*

$$\sum_{j=1}^{k} \lambda_{n-j+1}(A) \leq \sum_{j=1}^{k} A_{i_j,i_j} \leq \sum_{j=1}^{k} \lambda_j(A).$$

Corollary 441 (Hadamard's Determinant Inequality)**.** *For every positive definite matrix A,*

$$\boxed{\det A \leq \prod_{i=1}^{n} A_{ii}.}$$

Proof. Write

$$A = V^* V$$

for an invertible matrix V, and let $v_1, \ldots, v_n$ be the columns of V. Then

$$A_{ij} = \langle v_i, v_j \rangle, \qquad A_{ii} = \|v_i\|_2^2.$$

Moreover,

$$\det A = |\det V|^2.$$

Geometrically, $|\det V|$ is the volume of the parallelepiped spanned by $v_1, \ldots, v_n$. Gram–Schmidt writes this volume as the product of the successive perpendicular heights:

$$|\det V| = \prod_{i=1}^{n} \operatorname{dist}\big(v_i, \operatorname{span}(v_1, \ldots, v_{i-1})\big) \leq \prod_{i=1}^{n} \|v_i\|_2.$$

Squaring gives

$$\det A \leq \prod_i \|v_i\|_2^2 = \prod_i A_{ii}.$$

□

13.9 The Operator Jensen Inequality

Ordinary Jensen says that applying a convex function after averaging is better than averaging after applying the function. The operator version allows the "weights" themselves to be matrices.

The clean way to understand the proof is in two steps. First embed the matrix convex combination as the compression of a larger block matrix. Then average that block matrix with a sign-flipped copy to erase its off-diagonal blocks.

Definition 442 (Matrix Convex Combination)**.** Let A_1, A_2 be Hermitian and suppose

$$K_1^* K_1 + K_2^* K_2 = I.$$

Then

$$K_1^* A_1 K_1 + K_2^* A_2 K_2$$

is called a matrix convex combination of A_1, A_2.

Theorem 443 (Operator Jensen)**.** *Let f be operator convex on an interval I, let A_1, A_2 be Hermitian with spectra in I, and suppose*

$$K_1^* K_1 + K_2^* K_2 = I.$$

Then

$$\boxed{f(K_1^* A_1 K_1 + K_2^* A_2 K_2) \preceq K_1^* f(A_1) K_1 + K_2^* f(A_2) K_2.}$$

For operator-concave f, the inequality reverses.

Proof. Put

$$\mathcal{A} = \begin{pmatrix} A_1 & 0 \\ 0 & A_2 \end{pmatrix}, \qquad V = \begin{pmatrix} K_1 \\ K_2 \end{pmatrix}.$$

The condition on K_1, K_2 says precisely that

$$V^* V = I,$$

so V is an isometry. Extend its columns to a unitary matrix

$$Q = (V\ W).$$

Then

$$M := Q^* \mathcal{A} Q = \begin{pmatrix} C & * \\ * & D \end{pmatrix},$$

where

$$C = V^* \mathcal{A} V = K_1^* A_1 K_1 + K_2^* A_2 K_2.$$

Now let

$$U = \begin{pmatrix} I & 0 \\ 0 & -I \end{pmatrix}.$$

Averaging M with $U^* M U$ deletes the off-diagonal blocks:

$$\frac{1}{2}(M + U^* M U) = \begin{pmatrix} C & 0 \\ 0 & D \end{pmatrix}.$$

Operator convexity therefore gives

$$\begin{aligned} \begin{pmatrix} f(C) & 0 \\ 0 & f(D) \end{pmatrix} &= f\left(\frac{1}{2}(M + U^* M U)\right) \\ &\preceq \frac{1}{2}\big(f(M) + U^* f(M) U\big). \end{aligned}$$

Because functional calculus commutes with unitary conjugation,

$$f(M) = Q^* f(\mathcal{A}) Q.$$

Taking the upper-left block gives

$$f(C) \preceq V^* f(\mathcal{A}) V = K_1^* f(A_1) K_1 + K_2^* f(A_2) K_2.$$

The concave version follows by applying the convex statement to $-f$. □

13.10 Lieb's Concavity Theorem

In one variable,

$$x^q y^{1-q}$$

is the weighted geometric mean of x and y. The operator perspective gives the canonical noncommutative version of this expression, and Operator Jensen gives its joint concavity.

Definition 444 (Operator Perspective)**.** Let $f : (0, \infty) \to \mathbb{R}$ and let $A, B \succ 0$. Define

$$\boxed{\mathcal{P}_f(A, B) = B^{1/2} f(B^{-1/2} A B^{-1/2}) B^{1/2}.}$$

If A and B commute, this reduces to

$$\mathcal{P}_f(A, B) = B f(AB^{-1}).$$

In particular, for $f(t) = t^q$,

$$\mathcal{P}_q(A, B) = B^{1/2} (B^{-1/2} A B^{-1/2})^q B^{1/2},$$

the weighted matrix geometric mean.

Lemma 445 (Perspective Preserves Operator Concavity)**.** *If f is operator concave, then*

$$(A, B) \longmapsto \mathcal{P}_f(A, B)$$

is jointly operator concave on positive-definite pairs.

Proof. Take two positive-definite pairs

$$(A_1, B_1), \qquad (A_2, B_2)$$

and a weight $\lambda \in [0, 1]$. Put

$$A = \lambda A_1 + (1 - \lambda) A_2, \qquad B = \lambda B_1 + (1 - \lambda) B_2,$$

and define

$$K_1 = \sqrt{\lambda}\, B_1^{1/2} B^{-1/2}, \qquad K_2 = \sqrt{1 - \lambda}\, B_2^{1/2} B^{-1/2}.$$

Then

$$K_1^* K_1 + K_2^* K_2 = B^{-1/2} \big(\lambda B_1 + (1 - \lambda) B_2\big) B^{-1/2} = I.$$

Set

$$X_i = B_i^{-1/2} A_i B_i^{-1/2}.$$

A direct calculation gives

$$K_1^* X_1 K_1 + K_2^* X_2 K_2 = B^{-1/2} A B^{-1/2}.$$

The concave form of Operator Jensen therefore gives

$$f(B^{-1/2} A B^{-1/2}) \succeq K_1^* f(X_1) K_1 + K_2^* f(X_2) K_2.$$

Conjugating by $B^{1/2}$ yields

$$\mathcal{P}_f(A, B) \succeq \lambda \mathcal{P}_f(A_1, B_1) + (1 - \lambda) \mathcal{P}_f(A_2, B_2).$$

□

Corollary 446 (Joint Concavity of Tensor Powers)**.** *For $0 \le q \le 1$,*

$$\boxed{(A, B) \longmapsto A^q \otimes B^{1-q}}$$

is jointly operator concave.

Proof. Take

$$f(t) = t^q,$$

which is operator concave by Lemma 426. Apply Lemma 445 to

$$X = A \otimes I, \qquad Y = I \otimes B.$$

Since X and Y commute,

$$\begin{aligned} \mathcal{P}_q(X, Y) &= (I \otimes B^{1/2})(A \otimes B^{-1})^q (I \otimes B^{1/2}) \\ &= A^q \otimes B^{1-q}. \end{aligned}$$

Both X and Y depend linearly on A, B, so the claimed joint concavity follows. □

The same argument gives all exponents whose total is at most one.

Lemma 447 (Joint Concavity Below Total Exponent One)**.** *If*

$$q, r \ge 0, \qquad q + r \le 1,$$

then

$$(A, B) \longmapsto A^q \otimes B^r$$

is jointly operator concave.

Proof. The cases $q + r = 0$ and $q + r = 1$ are immediate from the preceding corollary. Suppose

$$s = q + r \in (0, 1).$$

Define

$$G(A, B) = A^{q/s} \otimes B^{r/s}.$$

The exponents in G sum to one, so G is jointly operator concave. Also

$$A^q \otimes B^r = G(A, B)^s.$$

Now $X \mapsto X^s$ is both operator monotone by Theorem 425 and operator concave by Lemma 426. Hence

$$\begin{aligned} G(\lambda z_1 + (1-\lambda) z_2)^s &\succeq \big(\lambda G(z_1) + (1-\lambda) G(z_2)\big)^s \\ &\succeq \lambda G(z_1)^s + (1-\lambda) G(z_2)^s. \end{aligned}$$

This is exactly the desired joint concavity. □

The last step in Lieb's theorem is easiest to understand by viewing a matrix as a vector.
For $K \in \mathbb{C}^{m \times n}$, define

$$\boxed{\operatorname{vec}(K) = \sum_{i=1}^{m} \sum_{j=1}^{n} K_{ij}\, e_i \otimes e_j.}$$

Thus the row index becomes the first tensor coordinate and the column index the second. Consequently, left multiplication acts on the first factor and right multiplication on the second:

$$\boxed{\operatorname{vec}(XKY) = (X \otimes Y^T) \operatorname{vec}(K).}$$

This can easily be checked on $K = e_i e_j^*$ which implies it for all matrices. It is clear that $\operatorname{vec}(A)^* \operatorname{vec}(B) = \operatorname{Tr}(A^* B)$, thus we can now deal in terms of inner products rather than traces.

Lemma 448 (Lieb's Concavity Theorem)**.** *Let $K \in \mathbb{C}^{m\times n}$ be fixed. If*

$$q, r \geq 0, \qquad q + r \leq 1,$$

then

$$\boxed{F(A, B) = \operatorname{Tr}(K^* A^q K B^r)}$$

is jointly concave for $A \succ 0$ and $B \succ 0$.

Proof. Vectorization turns the trace into a fixed quadratic form. Using the preceding formulas, we obtain

$$F(A, B) = \operatorname{vec}(K)^* \big((B^T)^r \otimes A^q\big) \operatorname{vec}(K).$$

The transpose map is linear and preserves positive definiteness. By Lemma 447,

$$(A, B) \longmapsto (B^T)^r \otimes A^q$$

is jointly operator concave. Applying the fixed positive linear functional

$$M \longmapsto \operatorname{vec}(K)^* M \operatorname{vec}(K)$$

preserves concavity. □

13.10.1 Joint Convexity of Matrix Relative Entropy

The first important consequence is obtained by differentiating Lieb's concavity at exponent zero.

Lemma 449 (Joint Convexity of Matrix Relative Entropy)**.** *For $A, B \succ 0$, define*

$$S(A\|B) = \operatorname{Tr}\big(A \log A - A \log B - A + B\big).$$

Then $S(A\|B)$ is jointly convex in (A, B).

Proof. For $t \in (0, 1)$ define

$$J_t(A, B) = \frac{1}{t}\left(\operatorname{Tr}(B^t A^{1-t}) - \operatorname{Tr} A\right).$$

By Lemma 448, the first trace is jointly concave, while the second term is linear. Hence J_t is jointly concave. As $t \downarrow 0$,

$$\frac{d}{dt} B^t\Big|_{t=0} = \log B, \qquad \frac{d}{dt} A^{1-t}\Big|_{t=0} = -A \log A.$$

Therefore

$$\begin{aligned}\lim_{t\downarrow 0} J_t(A, B) &= \operatorname{Tr}\big((\log B)A - A \log A\big) \\ &= \operatorname{Tr}\big(A \log B - A \log A\big).\end{aligned}$$

A finite pointwise limit of concave functions is concave. Hence

$$(A, B) \longmapsto \operatorname{Tr}(A \log B - A \log A)$$

is jointly concave. Negating it and adding the linear term

$$\operatorname{Tr}(B - A)$$

gives the claimed joint convexity. □

13.10.2 A Variational Formula for the Trace

The next identity is simply nonnegativity of relative entropy, rearranged as a variational principle.

Lemma 450 (Variational Formula for Trace)**.** *For $M \succ 0$,*

$$\boxed{\operatorname{Tr} M = \max_{T \succ 0} \operatorname{Tr}\big(T \log M - T \log T + T\big).}$$

Equality is attained at $T = M$.

Proof. The scalar inequality

$$a \log a - a \log h - a + h \geq 0 \qquad (a, h > 0)$$

is the tangent-line inequality for the convex function

$$x \longmapsto x \log x - x.$$

By the generalized Klein inequality,

$$S(T \| M) = \operatorname{Tr}\big(T \log T - T \log M - T + M\big) \geq 0.$$

Rearranging gives

$$\operatorname{Tr} M \geq \operatorname{Tr}\big(T \log M - T \log T + T\big).$$

Equality holds for $T = M$. □

13.10.3 Lieb's Trace-Exponential Theorem

Now the trace exponential becomes a partial maximization of a jointly concave function. This is the form of Lieb's theorem used throughout matrix concentration.

Theorem 451 (Lieb)**.** *For fixed Hermitian H, the map*

$$\boxed{A \longmapsto \operatorname{Tr} \exp(H + \log A)}$$

is concave on the positive-definite cone.

Proof. Apply Lemma 450 to

$$M = e^{H + \log A}.$$

Then

$$\begin{aligned} \operatorname{Tr} e^{H + \log A} &= \max_{T \succ 0} \operatorname{Tr}\big(T(H + \log A) - T \log T + T\big) \\ &= \max_{T \succ 0} \left\{ \operatorname{Tr}(TH) - S(T \| A) + \operatorname{Tr} A \right\}. \end{aligned}$$

By Lemma 449, $S(T \| A)$ is jointly convex. Therefore the quantity inside braces is jointly concave in (T, A).

Partial maximization preserves concavity. Indeed, take A_1, A_2 and nearly optimal T_1, T_2. At

$$A = \lambda A_1 + (1 - \lambda) A_2$$

the point

$$T = \lambda T_1 + (1 - \lambda) T_2$$

is admissible. Joint concavity then gives

$$F(A, T) \geq \lambda F(A_1, T_1) + (1 - \lambda) F(A_2, T_2).$$

Taking the approximations to the maxima arbitrarily close proves concavity. □

Corollary 452 (Lieb's Trace Inequality)**.** *For fixed Hermitian H and a random Hermitian matrix X for which the expectations exist,*

$$\boxed{\mathbb{E}\operatorname{Tr} e^{H+X} \le \operatorname{Tr}\exp\left(H + \log \mathbb{E} e^{X}\right).}$$

Proof. Apply Jensen's inequality to the concave map from Theorem 451 with the random positive-definite matrix

$$A = e^{X}.$$

Then

$$\mathbb{E}\operatorname{Tr} e^{H+\log(e^{X})} \le \operatorname{Tr} e^{H+\log \mathbb{E} e^{X}}.$$

Since X is Hermitian,

$$\log(e^{X}) = X.$$

□

13.11 Lieb–Thirring Inequalities

For powers of two, the word-straightening argument from Golden–Thompson gives the inequality almost for free. For arbitrary real exponents, the main ingredient is the Cordes inequality, itself a one-line consequence of Löwner–Heinz. Exterior powers then upgrade an operator-norm inequality to all eigenvalues at once.

Theorem 453 (Lieb–Thirring Inequality)**.** *For $A, B \succ 0$ and every real $q \ge 1$,*

$$\boxed{\operatorname{Tr}\big((BAB)^{q}\big) \le \operatorname{Tr}(B^{q}A^{q}B^{q}).}$$

The powers-of-two case. Suppose

$$q = 2^{m}.$$

Cyclicity gives

$$\operatorname{Tr}((BAB)^{q}) = \operatorname{Tr}((B^{2}A)^{q}).$$

Applying Lemma 428 to

$$U = B^{2}, \qquad V = A$$

yields

$$\operatorname{Tr}((B^{2}A)^{q}) \le \operatorname{Tr}(B^{2q}A^{q}) = \operatorname{Tr}(B^{q}A^{q}B^{q}).$$

Lemma 454 (Cordes Inequality)**.** *If $C, D \succ 0$ and $0 \le s \le 1$, then*

$$\boxed{\|C^{s}D^{s}\| \le \|CD\|^{s}.}$$

Here $\|\cdot\|$ is the operator norm.

Proof. Let

$$c = \|CD\|.$$

Then

$$DC^{2}D = (CD)^{*}(CD) \preceq c^{2}I.$$

Conjugating by D^{-1} gives

$$C^{2} \preceq c^{2}D^{-2}.$$

Apply Löwner–Heinz, Theorem 425, with exponent s:

$$C^{2s} \preceq c^{2s}D^{-2s}.$$

Conjugating by D^s,

$$D^s C^{2s} D^s \preceq c^{2s} I.$$

Finally,

$$\|C^s D^s\|^2 = \|D^s C^{2s} D^s\|,$$

so

$$\|C^s D^s\| \le c^s = \|CD\|^s.$$

□

Proof of Theorem 453 for general q. Set

$$s = \frac{1}{q}.$$

Apply Lemma 454 to

$$C = B^q, \qquad D = A^{q/2}.$$

Since

$$C^s = B, \qquad D^s = A^{1/2},$$

we obtain

$$\|BA^{1/2}\| \le \|B^q A^{q/2}\|^{1/q}.$$

Raise to the power $2q$. Since

$$\|BA^{1/2}\|^2 = \|BAB\|$$

and

$$\|B^q A^{q/2}\|^2 = \|B^q A^q B^q\|,$$

we get

$$\boxed{\|(BAB)^q\| \le \|B^q A^q B^q\|.}$$

This controls only the largest eigenvalue. Exterior powers apply the same inequality simultaneously to every product of the largest eigenvalues.

Apply the operator-norm inequality above to the positive matrices

$$\bigwedge^k A, \qquad \bigwedge^k B.$$

Exterior powers commute with products and functional calculus, so

$$\left\| \bigwedge^k (BAB)^q \right\| \le \left\| \bigwedge^k (B^q A^q B^q) \right\|.$$

Since the largest eigenvalue of $\bigwedge^k X$ is the product of the largest k eigenvalues of X, we obtain, for every k,

$$\prod_{i=1}^{k} \lambda_i((BAB)^q) \le \prod_{i=1}^{k} \lambda_i(B^q A^q B^q).$$

At $k = n$ there is equality, because

$$\begin{aligned} \det((BAB)^q) &= \det(B)^{2q} \det(A)^q \\ &= \det(B^q A^q B^q). \end{aligned}$$

Hence, after taking logarithms,

$$\big(\log \lambda_i((BAB)^q)\big)_{i=1}^{n}$$

is majorized by

$$\left(\log \lambda_i(B^q A^q B^q)\right)_{i=1}^n.$$

Applying Karamata to the convex function

$$x \longmapsto e^x$$

gives

$$\sum_i \lambda_i((BAB)^q) \le \sum_i \lambda_i(B^q A^q B^q),$$

which is precisely

$$\operatorname{Tr}((BAB)^q) \le \operatorname{Tr}(B^q A^q B^q).$$

□

The following "extended" inequality looks more complicated, but its proof is much easier. Once A is diagonal, the entire theorem reduces to the scalar weighted AM–GM inequality.

Theorem 455 (Extended Lieb–Thirring Inequality)**.** *For $A \succ 0$, Hermitian B, and $\alpha \in [0,1]$,*

$$\boxed{\operatorname{Tr}(BA^\alpha BA^{1-\alpha}) \le \operatorname{Tr}(B^2 A).}$$

Proof. Work in an eigenbasis of A:

$$A = \operatorname{diag}(\lambda_1, \dots, \lambda_n), \qquad \lambda_i > 0.$$

Since B is Hermitian,

$$B_{ji} = \overline{B_{ij}}.$$

Therefore

$$\operatorname{Tr}(BA^\alpha BA^{1-\alpha}) = \sum_{i,j} |B_{ij}|^2 \lambda_i^{1-\alpha} \lambda_j^\alpha.$$

Symmetrizing the pair (i,j),

$$\operatorname{Tr}(BA^\alpha BA^{1-\alpha}) = \frac{1}{2} \sum_{i,j} |B_{ij}|^2 \left(\lambda_i^{1-\alpha}\lambda_j^\alpha + \lambda_j^{1-\alpha}\lambda_i^\alpha\right).$$

Weighted AM–GM gives

$$\lambda_i^{1-\alpha}\lambda_j^\alpha \le (1-\alpha)\lambda_i + \alpha\lambda_j,$$

while swapping i, j gives

$$\lambda_j^{1-\alpha}\lambda_i^\alpha \le (1-\alpha)\lambda_j + \alpha\lambda_i.$$

Adding,

$$\lambda_i^{1-\alpha}\lambda_j^\alpha + \lambda_j^{1-\alpha}\lambda_i^\alpha \le \lambda_i + \lambda_j.$$

Hence

$$\begin{aligned}
\operatorname{Tr}(BA^\alpha BA^{1-\alpha}) &\le \frac{1}{2} \sum_{i,j} |B_{ij}|^2 (\lambda_i + \lambda_j) \\
&= \sum_{i,j} |B_{ij}|^2 \lambda_i \\
&= \operatorname{Tr}(B^2 A).
\end{aligned}$$

□

13.12 Matrix Concentration

The scalar Chernoff/Bernstein method has two steps:

$$\Pr[S \geq t] \leq e^{-\theta t}\mathbb{E}e^{\theta S},$$

and, for independent scalars,

$$\mathbb{E}e^{\theta \sum_i X_i} = \prod_i \mathbb{E}e^{\theta X_i}.$$

For matrices, the first step survives after replacing the scalar exponential by

$$\operatorname{Tr} e^{(\cdot)}.$$

The second step fails because noncommuting exponentials do not factor. Lieb's theorem is exactly the replacement: it lets us move out one random matrix at a time while putting its matrix MGF inside a logarithm.

After this one substitution, Chernoff and Bernstein are similar to their scalar proofs.

Let all matrices below be $d \times d$ self-adjoint matrices.

Lemma 456 (Matrix Laplace Transform)**.** *For independent self-adjoint random matrices $X_1, \ldots, X_n$,*

$$\boxed{\Pr\left[\lambda_{\max}\left(\sum_i X_i\right) \geq t\right] \leq \inf_{\theta>0} e^{-\theta t} \operatorname{Tr}\exp\left(\sum_i \log \mathbb{E}e^{\theta X_i}\right).}$$

Proof. Let

$$Y = \sum_i X_i.$$

Since

$$e^{\theta \lambda_{\max}(Y)} \leq \operatorname{Tr} e^{\theta Y},$$

Markov's inequality gives

$$\Pr[\lambda_{\max}(Y) \geq t] \leq e^{-\theta t}\mathbb{E}\operatorname{Tr} e^{\theta Y}.$$

It remains to control the trace MGF.

Condition on

$$X_1, \ldots, X_{n-1}$$

and apply Corollary 452 with

$$H = \theta \sum_{i<n} X_i, \qquad X = \theta X_n.$$

Then

$$\mathbb{E}_{X_n} \operatorname{Tr} e^{\theta \sum_{i=1}^n X_i} \leq \operatorname{Tr}\exp\left(\theta \sum_{i<n} X_i + \log \mathbb{E}e^{\theta X_n}\right).$$

Now condition on

$$X_1, \ldots, X_{n-2}$$

and apply the same inequality to X_{n-1}, treating the deterministic matrix

$$\log \mathbb{E}e^{\theta X_n}$$

as part of H. Continuing backwards gives

$$\mathbb{E}\operatorname{Tr} e^{\theta \sum_i X_i} \leq \operatorname{Tr}\exp\left(\sum_i \log \mathbb{E}e^{\theta X_i}\right).$$

Substitute this into the Markov bound and optimize over $\theta > 0$. □

Theorem 457 (Matrix Chernoff)**.** *Let X_i be independent and*

$$0 \preceq X_i \preceq RI.$$

Set

$$\mu_{\max} = \lambda_{\max}\left(\sum_i \mathbb{E}X_i\right), \qquad \mu_{\min} = \lambda_{\min}\left(\sum_i \mathbb{E}X_i\right).$$

Then

$$\boxed{\Pr[\lambda_{\max}(\sum_i X_i) \geq (1+\delta)\mu_{\max}] \leq d\left(\frac{e^{\delta}}{(1+\delta)^{1+\delta}}\right)^{\mu_{\max}/R}.}$$

For $0 \leq \delta < 1$,

$$\boxed{\Pr[\lambda_{\min}(\sum_i X_i) \leq (1-\delta)\mu_{\min}] \leq d\left(\frac{e^{-\delta}}{(1-\delta)^{1-\delta}}\right)^{\mu_{\min}/R}.}$$

Proof. For $0 \preceq X \preceq RI$, convexity of $e^{\theta x}$ on $[0, R]$ gives by functional calculus

$$e^{\theta X} \preceq I + \frac{e^{\theta R} - 1}{R} X.$$

Hence

$$\log \mathbb{E}e^{\theta X} \preceq \frac{e^{\theta R} - 1}{R} \mathbb{E}X.$$

Insert this into Lemma 456 and optimize exactly as in the scalar Chernoff proof. The lower tail uses $\theta < 0$. □

Theorem 458 (Matrix Bernstein)**.** *Let X_i be independent with*

$$\mathbb{E}X_i = 0, \qquad \lambda_{\max}(X_i) \leq R.$$

Set

$$\sigma^2 = \left\|\sum_i \mathbb{E}X_i^2\right\|.$$

Then

$$\boxed{\Pr\left[\lambda_{\max}\left(\sum_i X_i\right) \geq t\right] \leq d\exp\left(-\frac{t^2}{2(\sigma^2 + Rt/3)}\right).}$$

If

$$\|X_i\| \leq R,$$

then

$$\boxed{\Pr\left[\left\|\sum_i X_i\right\| \geq t\right] \leq 2d\exp\left(-\frac{t^2}{2(\sigma^2 + Rt/3)}\right).}$$

Proof. The scalar inequality used for Freedman holds under functional calculus:

$$e^{\theta X} \preceq I + \theta X + g(\theta)X^2.$$

Taking expectations,

$$\mathbb{E}e^{\theta X} \preceq I + g(\theta)\mathbb{E}X^2 \preceq \exp(g(\theta)\mathbb{E}X^2).$$

Hence

$$\log \mathbb{E}e^{\theta X} \preceq g(\theta)\mathbb{E}X^2.$$

Apply Lemma 456:

$$\Pr[\cdots] \le d \exp(-\theta t + g(\theta)\sigma^2),$$

and optimize as before.

Apply the same bound to $-\sum_i X_i$ for the lower tail. □

Theorem 459 (Matrix Freedman)**.** *Let X_k be a self-adjoint matrix martingale difference sequence:*

$$\mathbb{E}[X_k \mid \mathcal{F}_{k-1}] = 0, \qquad \lambda_{\max}(X_k) \le R.$$

Define

$$Y_k = \sum_{i=1}^{k} X_i$$

and

$$W_k = \sum_{i=1}^{k} \mathbb{E}[X_i^2 \mid \mathcal{F}_{i-1}].$$

Then

$$\boxed{\Pr\left[\exists k : \lambda_{\max}(Y_k) \ge t \text{ and } \lambda_{\max}(W_k) \le \sigma^2\right] \le d \exp\left(-\frac{t^2}{2(\sigma^2 + Rt/3)}\right).}$$

Proof. Fix $\theta \ge 0$ and write

$$g(\theta) = \frac{e^{\theta R} - 1 - \theta R}{R^2}.$$

The scalar inequality

$$e^{\theta x} \le 1 + \theta x + g(\theta) x^2 \qquad (x \le R)$$

gives, by functional calculus,

$$e^{\theta X_k} \preceq I + \theta X_k + g(\theta) X_k^2.$$

Taking conditional expectation and using $\mathbb{E}[X_k \mid \mathcal{F}_{k-1}] = 0$,

$$\mathbb{E}[e^{\theta X_k} \mid \mathcal{F}_{k-1}] \preceq I + g(\theta) V_k \preceq e^{g(\theta) V_k},$$

where

$$V_k = \mathbb{E}[X_k^2 \mid \mathcal{F}_{k-1}].$$

Since the matrix logarithm is operator monotone,

$$\boxed{\log \mathbb{E}[e^{\theta X_k} \mid \mathcal{F}_{k-1}] \preceq g(\theta) V_k.}$$

Now define the scalar process

$$Z_k = \operatorname{tr} \exp\left(\theta Y_k - g(\theta) W_k\right).$$

We claim that Z_k is a positive supermartingale.

Indeed, conditioned on $\mathcal{F}_{k-1}$, the matrix V_k is fixed, and

$$W_k = W_{k-1} + V_k.$$

Lieb's trace inequality gives

$$\begin{aligned}
\mathbb{E}[Z_k \mid \mathcal{F}_{k-1}] &= \mathbb{E} \operatorname{tr} \exp\left(\theta Y_{k-1} - g(\theta) W_k + \theta X_k\right) \\
&\le \operatorname{tr} \exp\left(\theta Y_{k-1} - g(\theta) W_k + \log \mathbb{E}[e^{\theta X_k} \mid \mathcal{F}_{k-1}]\right) \\
&\le \operatorname{tr} \exp\left(\theta Y_{k-1} - g(\theta) W_k + g(\theta) V_k\right) \\
&= Z_{k-1}.
\end{aligned}$$

Also

$$Z_0 = \operatorname{tr} I = d.$$

Suppose that for some k,

$$\lambda_{\max}(Y_k) \geq t, \qquad \lambda_{\max}(W_k) \leq \sigma^2.$$

Then

$$\begin{aligned}\lambda_{\max}\left(\theta Y_k - g(\theta) W_k\right) &\geq \theta \lambda_{\max}(Y_k) - g(\theta)\lambda_{\max}(W_k) \\ &\geq \theta t - g(\theta)\sigma^2.\end{aligned}$$

Hence

$$Z_k \geq \exp\left(\theta t - g(\theta)\sigma^2\right).$$

Ville's inequality for the positive supermartingale Z_k therefore gives

$$\Pr\left[\exists k : \lambda_{\max}(Y_k) \geq t,\ \lambda_{\max}(W_k) \leq \sigma^2\right] \leq d \exp\left(-\theta t + g(\theta)\sigma^2\right).$$

Finally, for

$$0 \leq \theta < \frac{3}{R},$$

the scalar Bernstein estimate

$$g(\theta) \leq \frac{\theta^2}{2(1 - \theta R/3)}$$

holds. Choose

$$\theta = \frac{t}{\sigma^2 + Rt/3}.$$

Then

$$1 - \frac{\theta R}{3} = \frac{\sigma^2}{\sigma^2 + Rt/3},$$

and so

$$-\theta t + g(\theta)\sigma^2 \leq -\frac{t^2}{2(\sigma^2 + Rt/3)}.$$

This proves the result. □

Lemma 460 (Matrix Azuma MGF)**.** *Let X be a random self-adjoint matrix with*

$$\mathbb{E} X = 0, \qquad X^2 \preceq A^2,$$

where A is deterministic. Then for every self-adjoint H,

$$\boxed{\mathbb{E} \operatorname{tr} e^{H + \theta X} \leq \operatorname{tr} e^{H + 2\theta^2 A^2}.}$$

The same statement holds conditionally when H, A are measurable.

Proof. The reason for symmetrization is that our hypothesis controls X^2, rather than X itself. Introducing an independent random sign turns the exponential into an even function of X, which can be controlled by X^2.

We first use the standard trace symmetrization inequality

$$\boxed{\mathbb{E} \operatorname{tr} e^{H + \theta X} \leq \mathbb{E}_{X,\varepsilon} \operatorname{tr} e^{H + 2\theta\varepsilon X},}$$

where ε is an independent Rademacher sign.

For completeness, let X' be an independent copy of X. Since $\mathbb{E} X' = 0$, convexity of the trace exponential gives

$$\mathbb{E}_X \operatorname{tr} e^{H + \theta X} \leq \mathbb{E}_{X,X'} \operatorname{tr} e^{H + \theta(X - X')}.$$

The matrix $X - X'$ is symmetric in distribution, so we may multiply it by an independent Rademacher sign ε. Split

$$H + \theta\varepsilon(X - X') = \left(\frac{H}{2} + \theta\varepsilon X\right) + \left(\frac{H}{2} - \theta\varepsilon X'\right).$$

Golden–Thompson and Cauchy–Schwarz for the trace give

$$\begin{aligned}
&\operatorname{tr} e^{H+\theta\varepsilon(X-X')} \\
&\quad\le \operatorname{tr}\left(e^{H/2+\theta\varepsilon X} e^{H/2-\theta\varepsilon X'}\right) \\
&\quad\le \sqrt{\operatorname{tr} e^{H+2\theta\varepsilon X} \operatorname{tr} e^{H-2\theta\varepsilon X'}}.
\end{aligned}$$

Taking expectation and applying scalar Cauchy–Schwarz, the two factors have the same distribution, so

$$\mathbb{E} \operatorname{tr} e^{H+\theta X} \le \mathbb{E}_{X,\varepsilon} \operatorname{tr} e^{H+2\theta\varepsilon X}.$$

Now condition on X. Lieb's inequality gives

$$\mathbb{E}_\varepsilon \operatorname{tr} e^{H+2\theta\varepsilon X} \le \operatorname{tr} \exp\left(H + \log \mathbb{E}_\varepsilon e^{2\theta\varepsilon X}\right).$$

But

$$\mathbb{E}_\varepsilon e^{2\theta\varepsilon X} = \cosh(2\theta X).$$

The scalar inequality

$$\cosh z \le e^{z^2/2}$$

gives by functional calculus

$$\cosh(2\theta X) \preceq e^{2\theta^2 X^2}.$$

Taking logarithms,

$$\log\cosh(2\theta X) \preceq 2\theta^2 X^2 \preceq 2\theta^2 A^2.$$

Therefore

$$\mathbb{E}_\varepsilon \operatorname{tr} e^{H+2\theta\varepsilon X} \le \operatorname{tr} e^{H+2\theta^2 A^2}.$$

□

Theorem 461 (Matrix Azuma–Hoeffding)**.** *Let X_k be self-adjoint martingale differences and suppose*

$$X_k^2 \preceq A_k^2$$

for deterministic self-adjoint matrices A_k. Set

$$\sigma^2 = \left\| \sum_k A_k^2 \right\|.$$

Then

$$\boxed{\Pr\left[\lambda_{\max}\left(\sum_k X_k\right) \ge t\right] \le d \exp\left(-\frac{t^2}{8\sigma^2}\right).}$$

Consequently,

$$\boxed{\Pr\left[\left\|\sum_k X_k\right\| \ge t\right] \le 2d \exp\left(-\frac{t^2}{8\sigma^2}\right).}$$

Proof. The matrix Laplace bound gives

$$\Pr\left[\lambda_{\max}\left(\sum_k X_k\right) \geq t\right] \leq e^{-\theta t}\mathbb{E}\,\mathrm{tr}\, e^{\theta \sum_k X_k}.$$

Apply Lemma 460 conditionally to the last martingale difference, then the next-to-last, and so on. This gives

$$\mathbb{E}\,\mathrm{tr}\, e^{\theta \sum_k X_k} \leq \mathrm{tr}\exp\left(2\theta^2 \sum_k A_k^2\right).$$

Since

$$\sum_k A_k^2 \preceq \sigma^2 I,$$

we obtain

$$\mathbb{E}\,\mathrm{tr}\, e^{\theta \sum_k X_k} \leq d e^{2\theta^2\sigma^2}.$$

Hence

$$\Pr\left[\lambda_{\max}\left(\sum_k X_k\right) \geq t\right] \leq d e^{-\theta t + 2\theta^2\sigma^2}.$$

The exponent is minimized at

$$\theta = \frac{t}{4\sigma^2},$$

giving

$$d e^{-t^2/(8\sigma^2)}.$$

Apply the same result to $-X_k$ and take a union bound to obtain the spectral-norm estimate. □

Theorem 462 (Matrix McDiarmid)**.** *Let $Z_1, \ldots, Z_n$ be independent and let*

$$H(Z_1, \ldots, Z_n)$$

be self-adjoint. Suppose that whenever z, z' differ only in coordinate i,

$$\big(H(z) - H(z')\big)^2 \preceq A_i^2$$

for a deterministic self-adjoint matrix A_i. Set

$$\sigma^2 = \left\|\sum_i A_i^2\right\|.$$

Then

$$\boxed{\Pr\left[\lambda_{\max}\big(H(Z) - \mathbb{E}H(Z)\big) \geq t\right] \leq d e^{-t^2/(8\sigma^2)}.}$$

Proof. Take the Doob martingale

$$M_i = \mathbb{E}[H(Z) \mid Z_1, \ldots, Z_i], \qquad X_i = M_i - M_{i-1}.$$

Then

$$\mathbb{E}[X_i \mid \mathcal{F}_{i-1}] = 0$$

and

$$\sum_i X_i = H(Z) - \mathbb{E}H(Z).$$

We only need to show

$$X_i^2 \preceq A_i^2.$$

Condition on $Z_1, \dots, Z_i$. Let Z_i' be an independent copy of Z_i, and use the same random future

$$Z_{i+1}, \dots, Z_n$$

on both sides. Then

$$X_i = \mathbb{E}\left[H(Z_1, \dots, Z_i, \dots, Z_n) - H(Z_1, \dots, Z_i', \dots, Z_n) \mid Z_1, \dots, Z_i\right].$$

The map

$$X \longmapsto X^2$$

is operator convex, so Jensen gives

$$\begin{aligned} X_i^2 &\preceq \mathbb{E}\left[\left(H(Z_1, \dots, Z_i, \dots, Z_n) - H(Z_1, \dots, Z_i', \dots, Z_n)\right)^2 \mid Z_1, \dots, Z_i\right] \\ &\preceq A_i^2. \end{aligned}$$

Thus the Doob differences satisfy the hypotheses of Theorem 461, which proves the claim. □

14 Convex Optimization

A gradient identifies a promising direction, but does not say how far to move. For a differentiable convex function f, the error of its linear prediction is the *Bregman divergence*

$$D_f(y, x) = f(y) - f(x) - \langle \nabla f(x), y - x \rangle.$$

We linearize at the second argument and evaluate at the first. Convexity says exactly that $D_f(y, x) \geq 0$; symmetry and the triangle inequality need not hold. An algorithm chooses a tractable bound on this error and minimizes the resulting local objective. Its proof must then explain why these local decisions produce global progress.

Unless specified otherwise, optimization is over $\mathbb{R}^n$; a minimizer is assumed to exist whenever one is used. Constraint sets are convex. Logarithms are natural, $\|v\|_M^2 = v^\top M v$ for $M \succeq 0$, and vector powers and products are coordinatewise when indicated. We count arithmetic operations, treating scalar logarithms, exponentials, and elementary random draws as unit-cost operations. Dependence polynomial in $1/\varepsilon$ is called *low accuracy*; logarithmic dependence is called *high accuracy*.

Lemma 463 (Integrated curvature)**.** *If f is C^2 along a segment and $d = y - x$, then*

$$D_f(y, x) = \int_0^1 (1 - t) d^\top \nabla^2 f(x + td) d \, dt.$$

Proof. For $\varphi(t) = f(x+td)$, integrate φ'' twice: $\varphi(1) - \varphi(0) - \varphi'(0) = \int_0^1 \int_0^s \varphi''(t) \, dt \, ds = \int_0^1 (1-t)\varphi''(t) \, dt$. □

Thus a Bregman error records curvature along the whole displacement. A Hessian at the starting point is only a local approximation.

14.1 Linear Systems and Preconditioned Richardson Iteration

For $A \succ 0$, solving $Ax = b$ is equivalent to minimizing $f(x) = \frac{1}{2} x^\top A x - b^\top x$. With $g = Ax - b$,

$$f(x + \Delta) - f(x) = \langle g, \Delta \rangle + \frac{1}{2} \|\Delta\|_A^2, \qquad f(x) - f(x^*) = \frac{1}{2} \|x - x^*\|_A^2.$$

Suppose an easier positive definite matrix B satisfies $\mu B \preceq A \preceq LB$, where $0 < \mu \le L$. Replacing the movement cost by its upper bound $L\|\Delta\|_B^2/2$ gives the step

$$\Delta = -\frac{1}{L} B^{-1}(Ax - b).$$

This is Richardson iteration when $B = I$. A preconditioner is useful when solving in B is cheap and the comparison ratio L/μ is small. Taking $B = A$ would give a perfect comparison but leave the original solve inside every step.

Algorithm 70 Preconditioned Richardson iteration

Input: $\mu B \preceq A \preceq LB$, $A, B \succ 0$, right-hand side b, accuracy $0 < \varepsilon < 1$
Output: $\|x - A^{-1}b\|_A \le \varepsilon \|A^{-1}b\|_A$

```
function Richardson(A, B, μ, L, b, ε)
    x ← 0, T ← ⌈(L/μ) log(1/ε)⌉
    for t = 1, ..., T do
        Solve Bz = Ax − b
        x ← x − z/L
    return x
```

Theorem 464 (Preconditioned contraction)**.** *One Richardson step contracts the solution error in the A-norm, and the residual in the B^{-1}-norm, by a factor at most $1 - \mu/L$. Consequently Algorithm 70 has its stated guarantee and also $\|Ax - b\|_{B^{-1}} \le \varepsilon \|b\|_{B^{-1}}$.*

Proof. Put $C = B^{-1/2} A B^{-1/2}$ and $e = B^{1/2}(x - x^*)$. Then $\mu I \preceq C \preceq LI$ and $e^+ = (I - C/L)e$. In an orthonormal eigenbasis of C, the component with eigenvalue λ is multiplied by $1 - \lambda/L \in [0, 1 - \mu/L]$. The same factor therefore contracts $\|e\|_C = \|x - x^*\|_A$ and $\|Ce\|_2 = \|Ax - b\|_{B^{-1}}$. Iterating and using $(1 - \mu/L)^T \le e^{-\mu T/L}$ proves both bounds. □

The change of coordinates explains the geometry; the algorithm never forms $B^{1/2}$. With a constant-factor preconditioner, high accuracy costs only $O(\log(1/\varepsilon))$ applications of A and solves in B. For a positive semidefinite system with b in its range, the same argument applies after restricting to that range.

14.2 Smooth Gradient Descent

For a nonlinear convex function, replace spectral bounds by bounds on its Bregman error. Write $q(x) = \|x\|_2^2/2$. The function is *L-smooth* if $D_f(y, x) \le L D_q(y, x)$ and *μ-strongly convex* if $D_f(y, x) \ge \mu D_q(y, x)$. Smoothness makes $x^+ = x - \nabla f(x)/L$ safe: minimizing its quadratic upper bound gives $f(x^+) \le f(x) - \|\nabla f(x)\|_2^2/(2L)$.

Algorithm 71 Smooth gradient descent

Input: Convex L-smooth f, initial point x_0, iteration count $T \ge 1$

```
for t = 0, ..., T − 1 do
    x_{t+1} ← x_t − ∇f(x_t)/L
return x_T
```

Theorem 465 (Smooth gradient descent)**.** *For every minimizer x^*, $f(x_T) - f(x^*) \le L\|x_0 - x^*\|_2^2/(2T)$. If f is also μ-strongly convex with $\mu > 0$, then $f(x_{t+1}) - f(x^*) \le (1 - \mu/L)(f(x_t) - f(x^*))$.*

Proof. Put $g = \nabla f(x)$. Add convexity at x to the descent bound:

$$f(x^+) - f(x^*) \le \langle g, x - x^* \rangle - \frac{\|g\|_2^2}{2L} = \frac{L}{2}\big(\|x - x^*\|_2^2 - \|x^+ - x^*\|_2^2\big).$$

The equality follows by expanding $x^+ = x - g/L$. Summing telescopes the distances. Since objective values decrease, each of the T gaps on the left is at least the final gap, proving the first claim.

For the second, evaluate the strong-convexity lower bound from x at x^*:

$$f(x^*) \geq f(x) + \langle g, x^* - x\rangle + \frac{\mu}{2}\|x^* - x\|_2^2 \geq f(x) - \frac{\|g\|_2^2}{2\mu}.$$

Thus the descent of $\|g\|_2^2/(2L)$ is at least μ/L times the current gap, which proves the contraction. □

The two arguments use different sources of progress. Without a lower curvature bound, distance pays for the *sum* of gaps. With one, a single step removes a fixed fraction of the remaining gap.

Lemma 466 (Equivalent Euclidean curvature conditions)**.** *For differentiable convex $f : \mathbb{R}^n \to \mathbb{R}$, smoothness is equivalent to convexity of $Lq - f$ and to $\|\nabla f(y) - \nabla f(x)\|_2 \leq L\|y - x\|_2$. Strong convexity is equivalent to convexity of $f - \mu q$ and to $\langle \nabla f(y) - \nabla f(x), y - x\rangle \geq \mu\|y - x\|_2^2$. For C^2 functions these are respectively $\nabla^2 f \preceq LI$ and $\nabla^2 f \succeq \mu I$. Moreover, an L-smooth convex function satisfies*

$$\langle \nabla f(x) - \nabla f(y), x - y\rangle \geq \frac{1}{L}\|\nabla f(x) - \nabla f(y)\|_2^2.$$

Proof. Linearity of Bregman divergence in its defining function gives the shifted convexity conditions. Gradient monotonicity of $f - \mu q$ gives the strong-convexity gradient condition; the Hessian conditions follow from convexity of the same shifted functions. An L-Lipschitz gradient gives the smoothness bound by integrating $\langle \nabla f(x + td) - \nabla f(x), d\rangle \leq Lt\|d\|_2^2$.

Conversely, let $g = \nabla f(x) - \nabla f(y)$ and $\phi(z) = f(z) - \langle \nabla f(y), z\rangle$. This convex function is minimized at y, and its gradient at x is g. Its descent bound gives $\phi(y) \leq \phi(x - g/L) \leq \phi(x) - \|g\|_2^2/(2L)$, hence $D_f(x, y) \geq \|g\|_2^2/(2L)$. Swap x, y and add, using $D_f(x, y) + D_f(y, x) = \langle g, x - y\rangle$. This proves the displayed inequality; Cauchy–Schwarz then gives gradient Lipschitzness, with the case $g = 0$ immediate. □

14.3 Bregman Proximal Steps and Relative Curvature

A fixed quadratic penalty may overestimate the cost of moving in some regions and underestimate it in others. A convex reference function h provides a position-dependent penalty $D_h(y, x)$. The choice must balance geometry and computation: its curvature should fit the problem, and its proximal step must be tractable.

For a convex set K, define

$$\mathrm{Prox}_x^h(v) \in \arg\min_{z \in K}\{\langle v, z\rangle + D_h(z, x)\}.$$

We assume the required minimizers exist and h is differentiable at the base points and proximal outputs. Boundary comparators are allowed when h extends continuously there. With no constraints, the optimality equation is $\nabla h(x^+) = \nabla h(x) - v$. For quadratic h, this is a preconditioned additive update; for entropy, it becomes a multiplicative update.

Lemma 467 (Three-point identity and proximal inequality)**.** *For differentiable convex h,*

$$\langle \nabla h(y) - \nabla h(x), u - y\rangle = D_h(u, x) - D_h(u, y) - D_h(y, x).$$

Consequently, if $x^+ = \mathrm{Prox}_x^h(\eta g)$, then every $u \in K$ satisfies

$$\eta\langle g, x^+ - u\rangle \leq D_h(u, x) - D_h(u, x^+) - D_h(x^+, x).$$

Proof. Expand the three divergences: their function values cancel, leaving $\langle \nabla h(y) - \nabla h(x), u - y\rangle$. Optimality of x^+ in the feasible direction $u - x^+$ gives $\langle \eta g + \nabla h(x^+) - \nabla h(x), u - x^+\rangle \geq 0$. Rearrange and apply the identity with $y = x^+$. □

The identity compares linearizing once at x with relinearizing at y. The proximal inequality turns that comparison into a potential decrease. Keep its negative movement term until some local error it may absorb is known.

14.3.1 Relative Descent

Relative smoothness and strong convexity mean $\mu D_h(y,x) \le D_f(y,x) \le L D_h(y,x)$, where $0 \le \mu \le L$. Equivalently, $f - \mu h$ and $Lh - f$ are convex. For C^2 functions, the integrated-curvature formula makes this equivalent to $\mu \nabla^2 h \preceq \nabla^2 f \preceq L \nabla^2 h$. The reference Hessian may vary with position while the comparison factors remain uniform.

Algorithm 72 Relative gradient descent

Input: Convex f that is L-smooth relative to h on K, point x_0, $T \ge 1$

1: **for** $t = 0, \dots, T-1$ **do**
2: $\quad x_{t+1} \leftarrow \mathrm{Prox}^h_{x_t}(\nabla f(x_t)/L)$
3: **return** x_T

Theorem 468 (Relative descent)**.** *If f is L-smooth and μ-strongly convex relative to h, then one step satisfies*

$$f(x^+) - f(x^*) + L D_h(x^*, x^+) \le (L - \mu) D_h(x^*, x).$$

Thus $D_h(x^*, x^+) \le (1 - \mu/L) D_h(x^*, x)$. For $\mu = 0$, the last iterate after T steps satisfies $f(x_T) - f(x^*) \le L D_h(x^*, x_0)/T$.

Proof. Put $g = \nabla f(x)$. Split the linear term in the smoothness bound and use the proximal inequality:

$$\begin{aligned} f(x^+) &\le f(x) + \langle g, x^* - x \rangle + \langle g, x^+ - x^* \rangle + L D_h(x^+, x) \\ &\le f(x) + \langle g, x^* - x \rangle + L D_h(x^*, x) - L D_h(x^*, x^+) \\ &\le f(x^*) + (L - \mu) D_h(x^*, x) - L D_h(x^*, x^+). \end{aligned}$$

The last line is relative strong convexity. Discarding the nonnegative gap gives contraction. Also the minimized upper bound has value at most its value at x, so $f(x^+) \le f(x)$. For $\mu = 0$, summing the displayed inequality and using this monotonicity proves the last-iterate bound. □

This contracts the comparator divergence; it does not assert that the objective gap contracts by the same factor on every step. The stronger one-step inequality keeps track of both quantities.

14.3.2 Where Norms Enter

For a norm $\|\cdot\|$, define $\|g\|_* = \max_{\|u\| \le 1} \langle g, u \rangle$. Then $|\langle g, u \rangle| \le \|g\|_* \|u\|$. The norm measures displacements and its dual measures linear losses. For ℓ_p, the dual is ℓ_q with $1/p + 1/q = 1$, including ℓ_1/ℓ_∞; this is Hölder's inequality with equality in the direction proportional to $\mathrm{sgn}(g)|g|^{q-1}$ for $1 < p < \infty$.

A norm and a reference function are separate choices. Norm steepest descent penalizes $\|y - x\|^2/2$, whereas a Bregman step penalizes $D_h(y,x)$. Even for $h(x) = \|x\|^2/2$, these need not agree: a general squared norm does not expand as an inner-product quadratic. For example, with $h(x) = \|x\|_4^2/2$, we have $D_h(e_2, e_1) = 1$ but $\|e_2 - e_1\|_4^2/2 = 1/\sqrt{2}$. What mirror descent needs is the comparison $D_h(y,x) \ge (\sigma/2)\|y - x\|^2$, called σ-strong convexity of h in that norm. It allows the telescoping Bregman potential to pay for movement measured by the norm.

14.4 Mirror Descent, Entropy, and Multiplicative Weights

Suppose we choose $x_t \in K$ before seeing a loss vector g_t. There may be no single objective to decrease; instead compare $\sum_t \langle g_t, x_t \rangle$ with the loss of any fixed $u \in K$. The proximal inequality controls the updated point x_{t+1}, which has already seen g_t. The only additional cost is acting one step earlier.

Algorithm 73 Mirror descent

Input: Reference function h, initial $x_0 \in K$, step size $\eta > 0$, $T \geq 1$

for $t = 0, \dots, T-1$ **do**
  Play x_t and receive g_t
  $x_{t+1} \leftarrow \operatorname{Prox}_{x_t}^h(\eta g_t)$

Theorem 469 (Mirror descent)**.** *If $D_h(y,x) \geq (\sigma/2)\|y-x\|^2$ for $\sigma > 0$, then*

$$\sum_{t<T} \langle g_t, x_t - u\rangle \leq \frac{D_h(u,x_0)}{\eta} + \frac{\eta}{2\sigma}\sum_{t<T}\|g_t\|_*^2 \qquad (u \in K).$$

Proof. Split $x_t - u = (x_{t+1}-u) + (x_t - x_{t+1})$. The proximal inequality pays for the first part. Duality and $ab \leq \eta a^2/(2\sigma) + \sigma b^2/(2\eta)$ give

$$\langle g_t, x_t - x_{t+1}\rangle \leq \frac{\eta}{2\sigma}\|g_t\|_*^2 + \frac{1}{\eta}D_h(x_{t+1},x_t).$$

This cancels the negative movement divergence, leaving $\langle g_t, x_t - u\rangle \leq [D_h(u,x_t) - D_h(u,x_{t+1})]/\eta + \eta\|g_t\|_*^2/(2\sigma)$. Sum and discard the final nonnegative divergence. □

Corollary 470 (Subgradient descent)**.** *Let f be convex, let $g_t \in \partial f(x_t)$ satisfy $\|g_t\|_2 \leq G$, and set $x_{t+1} = x_t - \eta g_t$. For $\bar{x} = T^{-1}\sum_{t<T} x_t$ and $R = \|x_0 - x^*\|_2$, $f(\bar{x}) - f(x^*) \leq R^2/(2\eta T) + \eta G^2/2$. With $R, G > 0$, the choice $\eta = R/(G\sqrt{T})$ gives $RG/\sqrt{T}$.*

Proof. Take $h(x) = \|x\|_2^2/2$ in mirror descent, use $f(x_t) - f(x^*) \leq \langle g_t, x_t - x^*\rangle$, and average by convexity. □

Thus Euclidean subgradient descent is the same algorithm with a quadratic reference; return the average of the points played by Algorithm 73. Averaging matters because nonsmooth steps need not decrease the objective. For a G-Lipschitz convex function on all of $\mathbb{R}^n$, every subgradient has norm at most G: apply the subgradient inequality and Lipschitzness in its own unit direction.

14.4.1 Entropy on the Simplex

For coordinatewise bounded losses, the useful dual norm is ℓ_∞. We therefore want a reference strongly convex in ℓ_1, with small initial divergence to every comparator. On $\Delta_n = \{p \geq 0 : \mathbf{1}^\top p = 1\}$, negative entropy $h(p) = \sum_i p_i \log p_i$ supplies both. For positive p,

$$D_h(q,p) = \sum_i q_i \log(q_i/p_i), \qquad D_h(q, \mathbf{1}/n) = \log n + \sum_i q_i \log q_i \leq \log n.$$

The first identity follows by expanding the divergence and canceling $\sum_i (q_i - p_i) = 0$; terms $0 \log 0$ are interpreted as zero. Weighted Cauchy–Schwarz gives $z^\top \nabla^2 h(p) z = \sum_i z_i^2/p_i \geq \|z\|_1^2$. Integration along the segment therefore yields $D_h(q,p) \geq \|q-p\|_1^2/2$, including boundary comparators by continuity.

The proximal problem is equally simple. For $\beta > 0$, differentiation of $\langle v, q\rangle + \beta D_h(q,p)$ with a multiplier for total mass one gives $v_i + \beta \log(q_i/p_i) + \lambda = 0$, with a scalar term absorbed in λ. Thus $q_i = p_i e^{-v_i/\beta} / \sum_j p_j e^{-v_j/\beta}$. This positive vector satisfies the constraint and first-order condition, so convexity proves it minimizes the objective.

Algorithm 74 Entropy proximal step and multiplicative weights

function EntropyStep(p, v, β) ▷ $p_i > 0$, $\beta > 0$
  $s_i \leftarrow \log p_i - v_i/\beta$ for every i
  $s \leftarrow s - (\max_i s_i)\mathbf{1}$
  return $\exp(s)/(\mathbf{1}^\top \exp(s))$

procedure MultiplicativeWeights(n, η, T)
  $p \leftarrow \mathbf{1}/n$
  for $t = 0, \dots, T-1$ **do**
    Play p and receive g_t
    $p \leftarrow$ EntropyStep($p, \eta g_t, 1$)

Subtracting the largest log-weight leaves the normalized vector unchanged. Positive initial weights stay positive in exact arithmetic. The resulting regret is at most $\log n/\eta + (\eta/2)\sum_{t<T} \|g_t\|_\infty^2$. The logarithmic dependence on dimension comes from the initial entropy divergence.

14.4.2 Soft Maximum and the Same Entropy Weights

Entropy also gives a smooth approximation to a maximum. For $\lambda > 0$, let $s_\lambda(z) = \lambda^{-1} \log \sum_i e^{\lambda z_i}$. Then

$$\max_i z_i \le s_\lambda(z) \le \max_i z_i + \frac{\log n}{\lambda}, \qquad s_\lambda(z) = \max_{p \in \Delta_n} \left\{ \langle p, z \rangle - \frac{1}{\lambda} h(p) \right\}.$$

The inequalities compare the exponential sum with its largest summand. For the identity, put $p_i^* = e^{\lambda z_i}/\sum_j e^{\lambda z_j}$. Expansion gives $\langle p, z\rangle - h(p)/\lambda = s_\lambda(z) - D_h(p, p^*)/\lambda$, whose maximum is attained at $p = p^*$.

Differentiation gives $\nabla s_\lambda(z) = p^*$ and $\nabla^2 s_\lambda(z) = \lambda(\operatorname{diag}(p^*) - p^*(p^*)^\top)$. Consequently, for any direction u, $u^\top \nabla^2 s_\lambda(z) u = \lambda \operatorname{Var}_{i\sim p^*}(u_i) \le \lambda \|u\|_\infty^2$. Applying the construction to $(z, -z)$ approximates $\|z\|_\infty$ within $\log(2n)/\lambda$ and has the same directional curvature bound. Increasing λ reduces approximation error but increases curvature; the entropy weights describe both effects.

14.5 Matrix Multiplicative Weights

A density matrix $X \succeq 0$, $\operatorname{Tr} X = 1$, distributes mass over directions: if $X = \sum_i p_i u_i u_i^\top$, its loss against symmetric G is $\operatorname{Tr}(GX) = \sum_i p_i u_i^\top G u_i$. The feasible set is denoted $\mathcal{D}_n$. The trace inner product has dual trace and operator norms. Indeed, diagonalizing a symmetric H gives $|\operatorname{Tr}(GH)| \le \|G\|_{\mathrm{op}} \sum_i |\lambda_i(H)|$; equality in the dual characterization is attained by $G = \operatorname{sgn}(H)$.

Matrix negative entropy $h(X) = \operatorname{Tr}(X \log X)$ has gradient $I + \log X$ on positive definite matrices. Thus, on $\mathcal{D}_n$, $D_h(Y, X) = \operatorname{Tr}(Y(\log Y - \log X))$. The entropy proximal step is

$$X^+ = \frac{\exp(\log X - \eta G)}{\operatorname{Tr} \exp(\log X - \eta G)}. \tag{36}$$

To verify it, the gradient of the proximal objective at this point is $\eta G + \log X^+ - \log X = -(\log Z) I$, where Z is the denominator. Its inner product with every feasible displacement is zero. Convexity of matrix entropy therefore proves optimality, also for boundary comparators by continuity. In general $\exp(\log X - \eta G) \ne X e^{-\eta G}$: the factors need not commute, and their product need not be symmetric.

Starting from I/n, successive logarithmic updates give $X_t = e^{-\eta S_t}/\operatorname{Tr} e^{-\eta S_t}$, where $S_t = \sum_{s<t} G_s$; scalar multiples of I disappear in normalization.

Algorithm 75 Matrix multiplicative weights

Input: Dimension n, step size $\eta > 0$, number of rounds T

$S \leftarrow 0$
for $t = 0, \dots, T-1$ **do**
  Compute $-\eta S = U \operatorname{diag}(\lambda) U^\top$
  $p_i \leftarrow e^{\lambda_i - \max_j \lambda_j}$; normalize $p \leftarrow p/(\sum_i p_i)$
  $X_t \leftarrow U \operatorname{diag}(p) U^\top$
  Play X_t, receive symmetric G_t, and set $S \leftarrow S + G_t$

The trace-exponential potential permits a direct analysis. Golden–Thompson replaces the scalar identity for multiplying exponential weights; no commutativity assumption is needed.

Theorem 471 (Matrix multiplicative weights)**.** *If $\eta \|G_t\|_{\mathrm{op}} \le 1$ for every t, then every $U \in \mathcal{D}_n$ satisfies*

$$\sum_{t<T} \operatorname{Tr}(G_t(X_t - U)) \le \frac{\log n}{\eta} + \eta \sum_{t<T} \operatorname{Tr}(X_t G_t^2).$$

If $\|G_t\|_{\mathrm{op}} \le G$, the right side is at most $\log n/\eta + \eta T G^2$. For $n \ge 2$, $G > 0$, and $T \ge \log n$, choosing $\eta = \sqrt{\log n/(TG^2)}$ gives regret at most $2G\sqrt{T \log n}$.

Proof. Put $W_t = e^{-\eta S_t}$ and $Z_t = \operatorname{Tr} W_t$, so $Z_0 = n$. For $|a| \le 1$, the exponential series gives $e^{-a} \le 1 - a + a^2 \sum_{k \ge 2} 1/k! \le 1 - a + a^2$. Apply this in an eigenbasis of G_t, then use Golden–Thompson:

$$\begin{aligned} Z_{t+1} &\le \operatorname{Tr}(W_t e^{-\eta G_t}) \\ &\le Z_t\big(1 - \eta \operatorname{Tr}(X_t G_t) + \eta^2 \operatorname{Tr}(X_t G_t^2)\big). \end{aligned}$$

Taking the trace against $W_t \succeq 0$ preserves the matrix inequality. Using $\log(1+a) \le a$ and summing yields

$$\log Z_T \le \log n - \eta \sum_{t<T} \operatorname{Tr}(X_t G_t) + \eta^2 \sum_{t<T} \operatorname{Tr}(X_t G_t^2).$$

On the other hand, the trace contains an exponential for a minimum eigenvector of S_T, so $\log Z_T \ge -\eta \lambda_{\min}(S_T)$. Every density matrix has $\operatorname{Tr}(U S_T) \ge \lambda_{\min}(S_T)$: take the trace of $S_T \succeq \lambda_{\min}(S_T) I$ against U. Combining the bounds and dividing by η proves the first claim. Finally, $G_t^2 \preceq G^2 I$ implies $\operatorname{Tr}(X_t G_t^2) \le G^2$. The stated step size satisfies $\eta G \le 1$ and balances the two regret terms. □

14.6 Sparsification and Sensitivity

Richardson reduces a linear solve to a sequence of easier solves, but a useful preconditioner must still be constructed. For $A^\top A$, one route is to retain a small weighted set of rows of A while preserving $\|Ax\|_2^2$ for every x. The same idea applies more broadly to sums of nonnegative losses: sample according to how important a term can become, including at queries chosen only after the sample is drawn.

For $F(x) = \sum_i f_i(x)$ with $f_i \ge 0$, uniform sampling can fail when one term dominates at a particular query. A suitable sampling probability must account for the term's largest possible contribution, not just its value at the current iterate.

Definition 472 (Sensitivity)**.** The sensitivity of term i is

$$\tau_i = \sup_{x : F(x) > 0} \frac{f_i(x)}{F(x)}.$$

The sampling rule follows from the error we want to control. If term i is kept with probability p_i, unbiasedness requires multiplying it by $1/p_i$. Its largest possible relative contribution then becomes τ_i/p_i. Choosing p_i proportional to τ_i, capped at one, makes these worst-case contributions uniformly small. The sum of the sensitivities controls how many terms this rule keeps.

14.6.1 Leverage Scores

For $F(x) = \|Ax\|_2^2$, write $a_i^\top$ for row i and $M = A^\top A$. Assume first that A has full column rank. After the change of variables $y = M^{1/2}x$, the sensitivity becomes

$$\begin{aligned}\tau_i &= \sup_{x \neq 0} \frac{\langle a_i, x\rangle^2}{x^\top M x} \\ &= \sup_{y \neq 0} \frac{\langle M^{-1/2}a_i, y\rangle^2}{\|y\|_2^2} = \|M^{-1/2}a_i\|_2^2 = a_i^\top M^{-1} a_i.\end{aligned}$$

Cauchy–Schwarz proves the upper bound in the middle supremum; choosing $y = M^{-1/2}a_i$ attains it when the row is nonzero. These are the *leverage scores*. Whitening changes the denominator to Euclidean length, so the largest possible relative contribution of a row is just its squared length in the whitened coordinates.

Their total mass is exactly the dimension. A scalar quadratic form can be written as a trace, so

$$\begin{aligned}\sum_i \tau_i &= \sum_i \operatorname{Tr}(M^{-1}a_i a_i^\top) \\ &= \operatorname{Tr}\left(M^{-1}\sum_i a_i a_i^\top\right) = \operatorname{Tr}(M^{-1}M) = \operatorname{Tr}(I) = n.\end{aligned}$$

For rank-deficient A, work on its row space and replace M^{-1} by $M^\dagger$; the sum is then $\operatorname{rank}(A)$. Zero rows have zero sensitivity and can be discarded. Although there may be many rows, their worst-case importances collectively occupy only n units of mass.

14.6.2 The Sampling Algorithm

Exact scores are unnecessary. Upper estimates $s_i \geq \tau_i$ suffice, and constant-factor estimates preserve the sample bound. Let $0 < \varepsilon < 1$ and $0 < \delta < 1$. The squared-error target suggests rescaled contributions of order ε^2 times the whole objective. Matrix concentration requires an additional logarithmic factor for simultaneous control in all directions. This gives the probabilities below.

Algorithm 76 Leverage-score sampling

Input: Matrix A, upper scores $s_i \geq \tau_i$, accuracy ε, failure probability δ

```
Ã ← an empty matrix
for each nonzero row a_i^⊤ do
    p_i ← min{1, 16 s_i log(4n/δ)/ε²}
    Independently sample ξ_i ~ Bernoulli(p_i)
    if ξ_i = 1 then
        Append a_i^⊤/√p_i to Ã
return Ã
```

For a fixed x, the estimate $\widetilde{F}(x) = \sum_i (\xi_i/p_i)\langle a_i, x\rangle^2$ is unbiased. Whenever $p_i < 1$, sensitivity gives

$$\frac{\langle a_i, x\rangle^2}{p_i} \leq \frac{\varepsilon^2}{16\log(4n/\delta)} F(x).$$

Rows with $p_i = 1$ create no error. Thus every random contribution is small relative to the whole objective, and scalar Bernstein gives concentration for any fixed query. The remaining issue is uniformity over x.

14.6.3 Why Matrix Concentration Gives Uniform Preservation

Theorem 473 (Quadratic row sparsification)**.** *Let $s_i \geq \tau_i$ and $S = \sum_i s_i$. Then Algorithm 76 returns, with probability at least $1-\delta$, a matrix with $O(S\varepsilon^{-2}\log(2n/\delta))$ rows satisfying*

$$(1-\varepsilon)A^\top A \preceq \widetilde{A}^\top \widetilde{A} \preceq (1+\varepsilon)A^\top A.$$

In particular, constant-factor upper estimates give $O(n\varepsilon^{-2}\log(2n/\delta))$ rows.

Proof. Work on the row space of A; the zero-rank case is immediate. Put $M = A^\top A$ on that space and $v_i = M^{-1/2}a_i$. Then $\sum_i v_i v_i^\top = I$ and $\|v_i\|_2^2 = \tau_i$. After whitening, the sampled error is

$$Z = \sum_i Z_i, \qquad Z_i = (\xi_i/p_i - 1)v_i v_i^\top.$$

Put $q = 16\log(4n/\delta)/\varepsilon^2$, so $p_i = \min\{1, qs_i\}$. Rows with $p_i = 1$ contribute $Z_i = 0$. For the remaining rows, $\tau_i/p_i \leq 1/q$, and the rank-one matrix $v_i v_i^\top$ has norm $\|v_i\|_2^2 = \tau_i$. Consequently

$$\|Z_i\|_{\mathrm{op}} = |\xi_i/p_i - 1|\,\tau_i \leq \tau_i/p_i \leq 1/q.$$

The variance calculation also uses rank one: $(v_i v_i^\top)^2 = (v_i^\top v_i)v_i v_i^\top = \tau_i v_i v_i^\top$. Since $\mathbb{E}\xi_i = p_i$ and $\xi_i^2 = \xi_i$,

$$\mathbb{E}(\xi_i/p_i - 1)^2 = \frac{p_i}{p_i^2} - 2\frac{p_i}{p_i} + 1 = \frac{1}{p_i} - 1.$$

Therefore

$$\begin{aligned}\sum_i \mathbb{E}Z_i^2 &= \sum_{i:p_i<1} (p_i^{-1} - 1)\tau_i v_i v_i^\top \\ &\preceq \frac{1}{q}\sum_{i:p_i<1} v_i v_i^\top \preceq \frac{1}{q} I.\end{aligned}$$

Matrix Bernstein, with both the summand bound and variance bound at most $1/q$, now gives

$$\begin{aligned}\Pr[\|Z\|_{\mathrm{op}} \geq \varepsilon] &\leq 2n\exp\left(-\frac{\varepsilon^2}{2(q^{-1} + \varepsilon/(3q))}\right) \\ &\leq 2n\exp(-3q\varepsilon^2/8) \leq \delta/2,\end{aligned}$$

where $0 < \varepsilon < 1$ and the last inequality follows from the chosen q. Finally, $I + Z = M^{-1/2}\widetilde{A}^\top \widetilde{A} M^{-1/2}$. The inequality $\|Z\|_{\mathrm{op}} \leq \varepsilon$ is equivalent to $-\varepsilon I \preceq Z \preceq \varepsilon I$. Adding I and taking a congruence by $M^{1/2}$ gives

$$(1-\varepsilon)A^\top A \preceq \widetilde{A}^\top \widetilde{A} \preceq (1+\varepsilon)A^\top A.$$

This single matrix inequality is exactly the assertion that every quadratic query is preserved. On the kernel of A, both forms vanish identically.

The expected row count is at most qS. For the sum N of the independent row indicators, the scalar Chernoff bound gives $\Pr[N > 2qS] \leq e^{-qS/3} \leq \delta/2$, since $S \geq \mathrm{rank}(A) \geq 1$. A union bound proves both claims simultaneously. □

For regression, apply the theorem to the augmented matrix $[A\ -b]$. Preserving its quadratic form on every vector also preserves it on each $(x, 1)$, so the same sampled residual rows approximate $\|Ax - b\|_2^2$ uniformly over x. This uses at most one additional dimension.

14.6.4 Why Higher Powers Can Require More Rows

The small total leverage score is specific to quadratic geometry. Higher powers can make many rows individually indispensable, even if arbitrary nonnegative rescaling of the retained rows is allowed.

Lemma 474 (An obstruction for $p > 2$)**.** *For fixed $p > 2$ and sufficiently large n, there is a matrix with $m = \Omega_p(n^{p/2}/\log^{p/2}(2n))$ rows such that every row must be retained by any weighted row sample preserving $\|Ax\|_p^p$ within a factor $1 \pm 1/2$ for all x.*

Proof. Choose m normalized sign vectors $a_i \in \{-1/\sqrt{n}, 1/\sqrt{n}\}^n$. Hoeffding gives $\Pr[|\langle a_i, a_j\rangle| > \rho] \le 2e^{-n\rho^2/2}$. Take $\rho = C_p\sqrt{\log(2n)/n}$, with C_p sufficiently large, and $m = \lfloor c_p(n/\log(2n))^{p/2}\rfloor$, with c_p sufficiently small. A union bound has probability less than one of any bad pair, and the constants can also ensure $m\rho^p \le 1/8$. Fix a realization satisfying both properties.

At $x = a_i$, row i contributes 1 and all other rows together contribute at most $1/8$. In particular, each sensitivity is at least $8/9$. To rule out compensation by reweighting, query $x = e_1$: every row contributes $n^{-p/2}$, so the upper approximation forces the sum of the retained weights to be at most $3m/2$. If row i is absent, the sampled value at a_i is then at most $(3m/2)\rho^p \le 3/16$, whereas the original value is at least 1. This violates the lower approximation. □

The obstruction comes from concentration: a row's self-correlation is one, while its correlations with the other rows are small and become even smaller after taking the pth power.

14.7 Estimating Leverage Scores and Bootstrapping Linear Solvers

Leverage-score sampling seems circular: a sparsifier gives a preconditioner, but its sampling probabilities contain $(A^\top A)^{-1}$. There are really two issues. We must avoid a separate solve for every row, and we must obtain the first useful solver without already having a sparsifier. Random projection resolves the first issue; a chain of regularized systems resolves the second.

14.7.1 Sketch First, Then Solve

Assume A has full column rank and put $M = A^\top A$. The inverse quadratic form is also an ordinary squared norm:

$$\begin{aligned}\|AM^{-1}a_i\|_2^2 &= a_i^\top M^{-1}A^\top AM^{-1}a_i \\ &= a_i^\top M^{-1}MM^{-1}a_i = a_i^\top M^{-1}a_i = \tau_i.\end{aligned}$$

Since $a_i = A^\top e_i$, the vector $AM^{-1}a_i$ is column i of $\Pi = AM^{-1}A^\top$. This matrix is an orthogonal projection: it is symmetric, and $\Pi^2 = AM^{-1}MM^{-1}A^\top = \Pi$. In particular, $\tau_i = \Pi_{ii} = \|\Pi e_i\|_2^2 \le 1$. Computing this whole matrix would defeat the purpose. Instead, let $G \in \mathbb{R}^{k\times m}$ have independent entries $\pm 1/\sqrt{k}$. The sign version of Johnson–Lindenstrauss gives, for each fixed vector, failure probability at most $2e^{-k/24}$ at squared-norm distortion $1/2$. Thus $k = \lceil 64\log(4m/\delta)\rceil$ and a union bound give simultaneously

$$\|GAM^{-1}a_i\|_2^2 = (1 \pm 1/2)\tau_i$$

with probability at least $1 - \delta$.

The order of operations matters. We do not solve for all m columns and then sketch them. First form $Q = A^\top G^\top$, which has only k columns, and solve $MY = Q$. Then

$$Y = M^{-1}A^\top G^\top, \qquad Y^\top a_i = GAM^{-1}a_i.$$

Thus the k solved columns give the desired sketch for every row by a matrix–vector product.

Algorithm 77 All leverage scores from logarithmically many solves

Input: Full-column-rank C with m_C rows; factorable P with $P/3 \preceq C^\top C \preceq 2P$; $0 < \delta < 1$
Output: Upper leverage estimates $\tau_i \le s_i \le 7\tau_i$, except with probability δ

function EstimateLeverage(C, P, δ)
  $k \leftarrow \lceil 64 \log(4m_C/\delta) \rceil$, $M_C v := C^\top (Cv)$
  Draw $G \in \mathbb{R}^{k \times m_C}$ with independent entries $\pm 1/\sqrt{k}$
  Form $Q \leftarrow C^\top G^\top$
  for $j = 1, \ldots, k$ **do**
    $y_j \leftarrow$ Richardson($M_C, P, 1/3, 2, Q_{:,j}, 1/(100\sqrt{m_C})$)
  $Y \leftarrow [y_1 \ \cdots \ y_k]$
  return $s_i \leftarrow 4\|Y^\top c_i\|_2^2$ for every row $c_i^\top$ of C

Forming Q and computing all output scores takes $O(k \operatorname{nnz}(C))$ arithmetic operations. The Gram matrix is applied through C rather than formed. To justify the approximate solves, put $M_C = C^\top C$ and let $e_j = y_j - y_j^*$ be the error relative to the exact column solution. Cauchy–Schwarz in the M_C geometry gives

$$\left\|(c_i^\top e_j)_{j=1}^k\right\|_2 \le \sqrt{\tau_i} \left(\sum_{j=1}^k \|e_j\|_{M_C}^2 \right)^{1/2}.$$

Here is a way to meet that aggregate tolerance. Let $g_j \in \mathbb{R}^{m_C}$ be the transpose of row j of the sign sketch. Then $\|g_j\|_2 = \sqrt{m_C/k}$, the exact solution is $y_j^* = M_C^{-1} C^\top g_j$, and

$$\|y_j^*\|_{M_C}^2 = g_j^\top C M_C^{-1} C^\top g_j \le \|g_j\|_2^2 = \frac{m_C}{k},$$

because $C M_C^{-1} C^\top$ is an orthogonal projection. A relative energy-norm error of $1/(100\sqrt{m_C})$ per solve gives

$$\sum_{j=1}^k \|e_j\|_{M_C}^2 \le k \left(\frac{1}{100\sqrt{m_C}} \right)^2 \frac{m_C}{k} = \frac{1}{10000}.$$

The error in each sketched row norm is therefore at most $\sqrt{\tau_i}/100$, simultaneously for all rows. The factor 4 in the returned score absorbs this solve error. On the JL event the resulting estimate lies between τ_i and $7\tau_i$, since its unscaled square root lies between $(1/\sqrt{2} - 1/100)\sqrt{\tau_i}$ and $(\sqrt{3/2} + 1/100)\sqrt{\tau_i}$. The tolerance therefore costs $O(\log(m_C + 2))$ Richardson iterations with the supplied constant-factor preconditioner.

14.7.2 Why Regularization Removes the Circularity

Let $M_\lambda = A^\top A + \lambda I$. If λ is at least the largest eigenvalue of M, then $\lambda I \preceq M_\lambda \preceq 2\lambda I$, so a scalar matrix is already a constant-factor preconditioner. The next scale is not much harder:

$$M_\lambda \preceq M_{2\lambda} \preceq 2M_\lambda, \qquad \tfrac{1}{2} M_\lambda^{-1} \preceq M_{2\lambda}^{-1} \preceq M_\lambda^{-1}.$$

This identifies a continuation scheme that does not assume the desired solver in advance. The old preconditioner is not required to be highly accurate for the next matrix; a constant-factor comparison is enough for Richardson iteration. Those solves estimate the next leverage scores, which produce a new constant-factor preconditioner before we move to a harder scale.

One detail is important: ordinary leverage estimation must be applied to

$$C_\lambda = \begin{pmatrix} A \\ \sqrt{\lambda} I \end{pmatrix}, \qquad C_\lambda^\top C_\lambda = M_\lambda.$$

For an original row, its score is $a_i^\top M_\lambda^{-1} a_i$. Sampling the original rows with these scores and retaining λI exactly gives a constant-factor approximation $P = \widetilde{A}^\top \widetilde{A} + \lambda I$ to M_λ. This is the sampling proof applied to C_λ, with the identity rows kept deterministically: those rows contribute no error, so the same variance bound holds. The upper scores are for C_λ, not for A alone. The identity rows need not be sampled, and their products are trivial to compute. They must, however, be included in the norm identity used by the sketch. Indeed,

$$\begin{aligned}\|C_\lambda M_\lambda^{-1} a_i\|_2^2 &= \|A M_\lambda^{-1} a_i\|_2^2 + \lambda \|M_\lambda^{-1} a_i\|_2^2 \\ &= a_i^\top M_\lambda^{-1} (A^\top A + \lambda I) M_\lambda^{-1} a_i \\ &= a_i^\top M_\lambda^{-1} a_i.\end{aligned}$$

Sketching only the A part would omit the second nonnegative term and could severely underestimate the intended score. Keeping the regularizer exact in the sampled matrix and including it in the sketch are separate requirements.

Algorithm 78 Regularization bootstrap for a Gram-matrix solver

Input: $M = A^\top A \succ 0$, bounds $0 < \ell I \preceq M \preceq UI$, right-hand side b, $0 < \varepsilon, \delta < 1$

$\lambda \leftarrow U$, $P \leftarrow UI$, $N \leftarrow 1 + \lceil \log_2(U/\ell) \rceil$
while $\lambda > \ell$ **do**
  $\lambda \leftarrow \max\{\ell, \lambda/2\}$
  $s \leftarrow \textsc{EstimateLeverage}(C_\lambda, P, \delta/(2N))$
  Form $\widetilde{A}$ by applying the rule of Algorithm 76 to the original rows of C_λ
    Use scores s_i, accuracy $1/2$, and failure probability $\delta/(2N)$
  $P \leftarrow \widetilde{A}^\top \widetilde{A} + \lambda I$; factor P ▷ *Keep the regularizer exactly.*
$x \leftarrow \textsc{Richardson}(M, P, 1/3, 2, b, \varepsilon)$
return x

At the first target scale, $U/2 \le \lambda \le U$, so $P/2 \preceq M_\lambda \preceq 2P$. Thereafter a $(1 \pm 1/2)$ sparsifier at one scale gives $\frac{1}{2} M_{\text{next}} \preceq P \preceq 3M_{\text{next}}$ at the next, because $M_{\text{next}} \preceq M_{\text{current}} \preceq 2M_{\text{next}}$. Thus every call satisfies $P/3 \preceq M_\lambda \preceq 2P$. The preconditioned condition number is at most six, so score estimation to the displayed tolerance takes only logarithmically many iterations. Every solver call has therefore been justified by an earlier stage, starting from UI. Fresh sketches and sampling let us apply the probability bounds conditionally on all preceding stages.

14.7.3 Cost and the Final Unregularized Solve

There are $O(1 + \log(U/\ell))$ scales. At the last scale, $M \preceq M_\ell \preceq 2M$, so the final preconditioner already works for M itself: $M/2 \preceq P \preceq 3M$, exactly the comparison used in the final Richardson call. If $U = \ell$, the loop is empty and $M = UI = P$. We therefore solve the original system, using regularization only to construct its preconditioner. A union bound over at most N stages gives overall success probability at least $1 - \delta$. At each scale, matrix–vector products and row queries cost $\widetilde{O}(\text{nnz}(A) + n)$, while forming and factoring the $\widetilde{O}(n)$-row preconditioner costs $\widetilde{O}(n^\omega)$ using fast matrix multiplication. The resulting arithmetic bound is

$$\widetilde{O}\left((\text{nnz}(A) + n^\omega) \log \frac{2\kappa}{\varepsilon} \right).$$

Here the accuracy is relative error in the solution's M-norm.

14.8 Lewis Weights and ℓ_p Geometry

Leverage scores work because a quadratic objective supplies its own Euclidean geometry. For ℓ_p, we must choose such a geometry. Simply using the ambient ℓ_2 norm introduces a distortion depending on the number

of rows m, although the column space has only dimension n. Lewis weights choose a row scaling that replaces this dependence on m by a dependence on n.

Assume $A \in \mathbb{R}^{m\times n}$ has full column rank and no zero rows. Zero rows can be removed, and rank deficiency can be handled by working in a basis of the row space. Write $W = \mathrm{diag}(w)$ for positive weights w.

Definition 475 (ℓ_p Lewis weights)**.** For $p \geq 1$, the Lewis weights satisfy

$$w_i = \tau_i\Big(W^{1/2-1/p}A\Big), \qquad \text{equivalently} \qquad w_i^{2/p} = a_i^\top (A^\top W^{1-2/p}A)^{-1}a_i.$$

To check the equivalence, put $C = W^{1/2-1/p}A$ and $M_w = C^\top C = A^\top W^{1-2/p}A$. Its ith leverage score is

$$\tau_i(C) = (w_i^{1/2-1/p}a_i)^\top M_w^{-1}(w_i^{1/2-1/p}a_i) = w_i^{1-2/p}a_i^\top M_w^{-1}a_i.$$

The equation $w_i = \tau_i(C)$ therefore becomes the displayed quadratic form equation after division by $w_i^{1-2/p} > 0$. Also, since these are the leverage scores of a rank-n matrix, $\sum_i w_i = \sum_i \tau_i(C) = n$. At $p = 2$, the scaling disappears and Lewis weights are ordinary leverage scores. For general p, they are not defined to equal the exact ℓ_p sensitivities: they define a Euclidean geometry whose leverage scores are consistent with the weights used to construct it.

14.8.1 Why This Scaling Matches the p-Norm

Splitting a row into two copies of $2^{-1/p}a_i$ leaves every ℓ_p query unchanged. Its importance should split into two weights $w_i/2$. Suppose the associated Gram matrix uses row contribution $w_i^s a_i a_i^\top$. After splitting, the two contributions add to

$$2\left(\frac{w_i}{2}\right)^s (2^{-1/p}a_i)(2^{-1/p}a_i)^\top = 2^{1-s-2/p}w_i^s a_i a_i^\top.$$

To leave the Gram matrix unchanged, the exponent must vanish: $1 - s - 2/p = 0$. Thus $s = 1 - 2/p$, and the corresponding row scaling is $w_i^{s/2} = w_i^{1/2-1/p}$. The square root enters because a Gram matrix squares the row scaling.

14.8.2 The Lewis Ellipsoid

Before constructing the weights, we show their geometric consequence. The weighted Euclidean norm approximates $\|Ax\|_p$ with dimension-dependent, not row-count-dependent, distortion.

Theorem 476 (Lewis-weight ellipsoid)**.** *Let w be the ℓ_p Lewis weights of A. For $p \geq 2$,*

$$\|Ax\|_p \leq \|W^{1/2-1/p}Ax\|_2 \leq n^{1/2-1/p}\|Ax\|_p.$$

For $1 \leq p \leq 2$,

$$\|W^{1/2-1/p}Ax\|_2 \leq \|Ax\|_p \leq n^{1/p-1/2}\|W^{1/2-1/p}Ax\|_2.$$

Proof. Put $z = Ax$ and $E = \|W^{1/2-1/p}z\|_2$. If $E = 0$, positive row weights force $z = 0$, and all quantities vanish. Otherwise the leverage inequality for row i of the reweighted matrix says $w_i^{1-2/p}z_i^2 \leq w_i E^2$. Dividing gives $|z_i| \leq w_i^{1/p}E$. The normalized values $t_i = |z_i|/(w_i^{1/p}E)$ therefore belong to $[0, 1]$ and satisfy

$$\sum_i w_i t_i^2 = E^{-2}\sum_i w_i^{1-2/p}z_i^2 = 1,$$

$$\sum_i w_i = n, \qquad \sum_i w_i t_i^p = E^{-p}\sum_i |z_i|^p = \frac{\|z\|_p^p}{E^p}.$$

We have reduced the norm comparison to a comparison of two moments.

For $p \geq 2$, $t_i^p \leq t_i^2$ because $t_i \leq 1$, so $\sum_i w_i t_i^p \leq 1$. For the other inequality, the numbers w_i/n are probabilities. Jensen for the convex function $s \mapsto s^{p/2}$ gives

$$\frac{1}{n}\sum_i w_i t_i^p = \sum_i \frac{w_i}{n}(t_i^2)^{p/2} \geq \left(\sum_i \frac{w_i}{n} t_i^2\right)^{p/2} = n^{-p/2}.$$

Thus $n^{1-p/2} \leq \|z\|_p^p / E^p \leq 1$. Taking pth roots gives $n^{1/p-1/2}E \leq \|z\|_p \leq E$, exactly the first pair of bounds.

For $1 \leq p \leq 2$, $t_i^p \geq t_i^2$ and $s^{p/2}$ is concave. The same two calculations now give $1 \leq \|z\|_p^p / E^p \leq n^{1-p/2}$. Taking pth roots yields $E \leq \|z\|_p \leq n^{1/p-1/2}E$. At $p = 2$, both are equalities. □

Corollary 477 (John's theorem, symmetric case)**.** *Every origin-symmetric convex body $K \subseteq \mathbb{R}^n$ contains an origin-centered ellipsoid E such that $E \subseteq K \subseteq \sqrt{n}\, E$.*

Proof. First suppose $K = \{x : \|Ax\|_\infty \leq 1\}$ is a bounded polytope, where A has m rows and full column rank. For each $p \geq 2$, let W_p contain the ℓ_p Lewis weights and put $Q_p = A^\top W_p^{1-2/p} A$. The preceding theorem gives

$$\|Ax\|_p \leq \sqrt{x^\top Q_p x} \leq n^{1/2-1/p}\|Ax\|_p.$$

In particular, $m^{-1}A^\top A \preceq Q_p \preceq nA^\top A$. Thus, along a sequence $p \to \infty$, the matrices Q_p converge to a positive definite matrix Q. Since $\|Ax\|_p \to \|Ax\|_\infty$, taking limits gives $\|Ax\|_\infty \leq \sqrt{x^\top Q x} \leq \sqrt{n}\,\|Ax\|_\infty$. Hence $E = \{x : x^\top Q x \leq 1\}$ has the required inclusions.

For general K, approximate it by symmetric polytopes $K \subseteq P_j \subseteq (1+1/j)K$, obtained using finitely many supporting slabs, and take their ellipsoids $E_j \subseteq P_j \subseteq \sqrt{n}\, E_j$. Choose $r, R > 0$ with $rB_2^n \subseteq K \subseteq RB_2^n$. Then $(r/\sqrt{n})B_2^n \subseteq E_j \subseteq 2RB_2^n$, so their defining matrices again have a positive definite convergent subsequence. Passing to its limit gives $E \subseteq K \subseteq \sqrt{n}\, E$. □

14.8.3 Computing the Weights by a Fixed Point

Define

$$\Phi_i(w) = \left(a_i^\top (A^\top W^{1-2/p} A)^{-1} a_i\right)^{p/2}.$$

The desired weights are the fixed point of Φ. To understand iteration, measure error multiplicatively: $u \approx_\alpha w$ means $\alpha^{-1}u_i \leq w_i \leq \alpha u_i$ for every i. Put $\beta = 1 - 2/p$ and $q_0 = \alpha^{|\beta|}$. Whether β is positive or negative, the coordinatewise comparison implies $q_0^{-1}u_i^\beta \leq w_i^\beta \leq q_0 u_i^\beta$. For any direction z, multiply by $(a_i^\top z)^2$ and sum. This gives

$$q_0^{-1} A^\top U^\beta A \preceq A^\top W^\beta A \preceq q_0 A^\top U^\beta A.$$

Inversion reverses the order, but the two-sided factor remains q_0. Evaluating the inverse matrices on a_i preserves this scalar comparison. Finally, raising it to the positive power $p/2$ changes the factor to $q_0^{p/2} = \alpha^{|p-2|/2}$. Consequently

$$\Phi(u) \approx_{\alpha^{|p-2|/2}} \Phi(w), \qquad \|\log \Phi(u) - \log \Phi(w)\|_\infty \leq q \|\log u - \log w\|_\infty, \quad q = \frac{|p-2|}{2}.$$

Thus for $1 \leq p < 4$, the map $s \mapsto \log \Phi(e^s)$ is a contraction on $\mathbb{R}^m$. Banach's theorem gives a unique fixed point and geometric convergence from every positive starting vector. It also provides a stopping test that does not use the unknown weights. Let $\ell = \log w$, let ℓ^* be the fixed point in logarithmic coordinates, and put $T(\ell) = \log \Phi(e^\ell)$. By the triangle inequality and contraction,

$$\|\ell - \ell^*\|_\infty \leq \|\ell - T(\ell)\|_\infty + \|T(\ell) - T(\ell^*)\|_\infty \leq \|\ell - T(\ell)\|_\infty + q\|\ell - \ell^*\|_\infty.$$

Rearranging bounds the unknown error by the computable residual: $\|\ell - \ell^*\|_\infty \leq \|\ell - T(\ell)\|_\infty/(1-q)$. An error at most $\log(1+\varepsilon)$ means each original weight lies between $(1+\varepsilon)^{-1}w_i^*$ and $(1+\varepsilon)w_i^*$.

Algorithm 79 Lewis weights by multiplicative fixed-point iteration

Input: Full-column-rank A with nonzero rows, $1 \le p < 4$, relative accuracy $\varepsilon > 0$
Output: Weights within a factor $1+\varepsilon$ of the exact Lewis weights in every coordinate

$w \leftarrow \mathbf{1}$, $q \leftarrow |p-2|/2$
loop
  Form $M \leftarrow A^\top \operatorname{diag}(w^{1-2/p})A$
  $v_i \leftarrow (a_i^\top M^{-1} a_i)^{p/2}$ for every i
  if $\|\log v - \log w\|_\infty \le (1-q)\log(1+\varepsilon)$ **then**
    return w
  $w \leftarrow v$ ▷ *Update all coordinates simultaneously*

The returned weights differ from the true weights by at most a factor $1+\varepsilon$ coordinatewise. The stopping rule also bounds the iteration count without knowing the smallest true weight. If $R_0 = \|T(0)\|_\infty$ is the first residual, contraction gives $\|\ell_t - T(\ell_t)\|_\infty \le q^t R_0$. For $0 < q < 1$, it suffices that $q^t R_0 \le (1-q)\log(1+\varepsilon)$; when $p = 2$, one update gives the exact weights.

Approximate quadratic forms require an explicit error budget. If an approximate logarithmic update $\widehat{T}$ satisfies $\|\widehat{T}(\ell) - T(\ell)\|_\infty \le \zeta$, then $\|\ell_{t+1} - \ell^*\|_\infty \le q\|\ell_t - \ell^*\|_\infty + \zeta$. The resulting error floor is $\zeta/(1-q)$, and a valid stopping test is $\|\ell - \widehat{T}(\ell)\|_\infty + \zeta \le (1-q)\log(1+\varepsilon)$. Thus leverage estimation can accelerate the queries, but constant-factor estimates alone do not give arbitrarily precise Lewis weights. For $p \ge 4$, this particular map need not be contractive; a convex characterization will establish existence and uniqueness without that requirement.

14.8.4 A Convex Characterization for $p > 2$

For $p > 2$, we can derive a convex objective directly from the equation we want to solve. Put $v_i = w_i^{1-2/p}$ and $\alpha = 2/(p-2)$. The Lewis equation becomes $a_i^\top (A^\top V A)^{-1} a_i = v_i^\alpha$. The inverse quadratic form suggests a log-determinant objective. If $M(v) = A^\top V A$, then $\partial M/\partial v_i = a_i a_i^\top$, and

$$\frac{\partial}{\partial v_i} \log\det M(v) = \operatorname{Tr}(M(v)^{-1} a_i a_i^\top) = a_i^\top M(v)^{-1} a_i.$$

The other side, v_i^α, is the derivative of $v_i^{1+\alpha}/(1+\alpha)$. Subtracting these two derivatives produces the desired stationarity equation. This derives the objective

$$\Psi(v) = -\log\det(A^\top V A) + \frac{1}{1+\alpha}\sum_i v_i^{1+\alpha}, \qquad V = \operatorname{diag}(v).$$

The first term is convex and rewards increasing the Gram matrix; the second is strictly convex and penalizes large weights. Their balance determines the same self-consistent geometry, without relying on convergence of the fixed-point map.

A minimizer exists after allowing $v \ge 0$ and setting $\Psi = +\infty$ when the Gram matrix is singular. Indeed, $\lambda_{\max}(A^\top V A) \le \|A\|_{\mathrm{op}}^2 \|v\|_\infty$, so the negative log-determinant can decrease only logarithmically as $\|v\|_\infty \to \infty$, whereas the power penalty grows as $\|v\|_\infty^{1+\alpha}$. Hence every finite sublevel set is bounded. Within such a set, approaching a singular Gram matrix makes $-\log\det(A^\top V A)$ tend to $+\infty$. Lower semicontinuity and compactness therefore give a minimizer. A minimizing coordinate cannot be zero while the Gram matrix is nonsingular, because its one-sided derivative would be $-a_i^\top (A^\top V A)^{-1} a_i < 0$. Thus the minimizer is positive and unique, and its first-order condition is

$$a_i^\top (A^\top V A)^{-1} a_i = v_i^\alpha.$$

Substituting $v_i = w_i^{1-2/p}$ gives $v_i^\alpha = w_i^{2/p}$, exactly the Lewis-weight equation. Together with the fixed-point argument, this proves existence and uniqueness for every $p \ge 1$.

The two constructions answer different questions. The contraction explains why repeated reweighting succeeds in its range; the convex formulation identifies the same weights globally, without requiring that iteration to be contractive.

14.8.5 A Sensitivity Bound from the Ellipsoid

The ellipsoid comparison already gives a useful sensitivity bound. For $p \geq 1$, write $\tau_i^{(p)} = \sup_{Ax \neq 0} |a_i^\top x|^p / \|Ax\|_p^p$. The proof of Theorem 476 gave $|a_i^\top x| \leq w_i^{1/p} \|W^{1/2-1/p} Ax\|_2$. Combining this with that theorem yields

$$\tau_i^{(p)} \leq n^{\max\{p/2-1,0\}} w_i, \qquad \sum_i \tau_i^{(p)} \leq n^{\max\{1,p/2\}}.$$

For $p \leq 2$, the weighted Euclidean norm is at most $\|Ax\|_p$; for $p \geq 2$, its pth power is at most $n^{p/2-1}\|Ax\|_p^p$. These are exactly the two cases in the bound. For $p \leq 2$, the total sensitivity is at most n; for $p > 2$, the $n^{p/2}$ bound has the same dimension exponent as Lemma 474. This explains the sampling scale, but does not by itself prove uniform preservation of a nonquadratic objective: the matrix concentration argument above only handles quadratic forms. The consequences used here are the proved norm comparison and sensitivity bound. We next use weighted quadratics to approximate the *local* error of a power loss, where a constant-quality solve can be refined to high accuracy.

14.9 High-Accuracy ℓ_p Regression

Weighted least squares can also solve nonquadratic problems by repeatedly approximating their local error. For fixed $p > 1$, consider $f(x) = \|Ax - b\|_p^p$. Near an optimum, its remaining improvement can be much smaller than its total value, so a fixed relative approximation to the original objective may hide all remaining progress. The Bregman divergence isolates the relevant quantity: once the linear term is known, it is exactly the error in predicting an improvement.

For the power loss, this error has two regimes. A displacement much smaller than a nonzero residual sees approximately constant curvature and hence a quadratic cost. A displacement comparable to or larger than that residual sees the full pth-power growth. A useful local bound must retain both, rather than extrapolating the Hessian at the current point to every step.

We first identify the local error for all $p > 1$, then give an explicit weighted-least-squares algorithm for $p \geq 2$. Its inner solves need only constant relative accuracy. High accuracy comes from comparing each accepted decrease with the improvement still available, rather than from a uniform condition number for the original objective.

14.9.1 Two Scales in the Power Function

Let $\phi(t) = |t|^p$, with $\phi'(0) = 0$. For $t \neq 0$, $\phi''(t) = p(p-1)|t|^{p-2}$. For a nonzero displacement s,

$$D_\phi(t+s,t) = p(p-1)s^2 \int_0^1 (1-u)|t+us|^{p-2}\,du.$$

For $1 < p < 2$, the singularity at zero is integrable, so this identity remains valid as an improper integral. The case $s = 0$ is immediate.

The two scales agree at $|s| = |t|$. Their ratio is $|t|^{p-2}s^2/|s|^p = (|t|/|s|)^{p-2}$. Thus for $p > 2$ the appropriate cost is the larger of the two expressions, equivalently their sum up to a constant factor; for $p < 2$ it is the smaller. The proof below turns this small-step/large-step distinction into a uniform bound, including steps that cross a zero residual.

Lemma 478 (Power Bregman divergence)**.** *For each fixed $p > 1$,*

$$D_\phi(t+s,t) \asymp_p \begin{cases} |t|^{p-2}s^2 + |s|^p, & p \geq 2, \\ \min\{|t|^{p-2}s^2, |s|^p\}, & 1 < p \leq 2. \end{cases}$$

For $p \geq 2$, valid lower and upper constants are $c_p = 2^{3-2p}$ and $C_p = p(p-1)2^{p-3}$, respectively. At $p = 2$ the quadratic coefficient is 1, including at $t = 0$. For $1 < p < 2$ and $t = 0$, the minimum is interpreted as $|s|^p$.

Proof. Zero displacements are immediate. First take $p \geq 2$. The integral formula and $|t + us|^{p-2} \leq 2^{p-2}(|t|^{p-2} + |s|^{p-2})$ give the stated upper bound after integrating $1 - u$.

For the lower bound, first isolate the power cost. Among intervals of a fixed length, the integral of $|r|^{p-2}$ is minimized by centering the interval at zero: moving either endpoint away from this position replaces smaller values of the integrand by larger ones. Thus, for $u > v$,

$$\phi'(u) - \phi'(v) = p(p-1) \int_v^u |r|^{p-2}\, dr \geq p2^{2-p}(u-v)^{p-1}.$$

Integrating this derivative increment along the displacement, with its sign reversed when $s < 0$, yields

$$D_\phi(t+s, t) = s \int_0^1 \big(\phi'(t+vs) - \phi'(t)\big)\, dv \geq 2^{2-p}|s|^p.$$

If $|s| > |t|/2$, then $|t|^{p-2}s^2 + |s|^p \leq 2^{p-1}|s|^p$, giving the lower constant 2^{3-2p}. If $|s| \leq |t|/2$, the segment instead satisfies $|t + us| \geq |t|/2$. The integral formula gives $D_\phi(t+s, t) \geq p(p-1)2^{1-p}|t|^{p-2}s^2$, whereas $|t|^{p-2}s^2 + |s|^p \leq 2|t|^{p-2}s^2$. The ratio is at least $p(p-1)2^{-p} \geq 2^{3-2p}$, completing this case.

Now take $1 < p < 2$. If $0 < |s| \leq |t|/2$, then $|t|/2 \leq |t + us| \leq 3|t|/2$, so the integral is $\Theta_p(|t|^{p-2}s^2)$, the smaller of the two scales. Otherwise put $a = t/s \in [-2, 2]$. Homogeneity gives

$$D_\phi(t+s, t) = |s|^p\big(|a+1|^p - |a|^p - p|a|^{p-2}a\big).$$

The bracket is $D_\phi(a+1, a)$, with $|a|^{p-2}a = 0$ at $a = 0$. It is continuous and strictly positive on $[-2, 2]$ by strict convexity, so compactness bounds it above and below by positive constants depending only on p. For $t \neq 0$, the inequality $|t| < 2|s|$ gives $|t|^{p-2}s^2 \geq 2^{p-2}|s|^p$. The claimed minimum therefore lies between $2^{p-2}|s|^p$ and $|s|^p$. At $t = 0$, the divergence is exactly $|s|^p$. □

Writing $r = Ax - b$ and $s = A\Delta$, the gradient identity $\nabla f(x) = A^\top(\phi'(r_i))_i$ shows exactly how the scalar bounds transfer:

$$\begin{aligned} D_f(x+\Delta, x) &= \sum_i \big(\phi(r_i + s_i) - \phi(r_i)\big) - \big\langle A^\top(\phi'(r_i))_i, \Delta\big\rangle \\ &= \sum_i \big(\phi(r_i + s_i) - \phi(r_i) - \phi'(r_i)s_i\big) = \sum_i D_\phi(r_i + s_i, r_i). \end{aligned}$$

For $p \geq 2$, define

$$Q_x(\Delta) = \sum_i |r_i|^{p-2}(A\Delta)_i^2, \qquad P(\Delta) = \|A\Delta\|_p^p, \qquad R_x = Q_x + P.$$

The explicit constants in Lemma 478 give

$$\langle \nabla f(x), \Delta\rangle + c_p R_x(\Delta) \leq f(x+\Delta) - f(x) \leq \langle \nabla f(x), \Delta\rangle + C_p R_x(\Delta).$$

The weighted quadratic preserves the Hessian-scale cost of small changes. The p-th-power term prevents a large change from exploiting a misleadingly small quadratic weight, particularly at a zero residual.

For $1 < p \leq 2$, the corresponding local error is

$$D_f(x+\Delta, x) \asymp_p \sum_i \min\{|(A\Delta)_i|^p,\ |r_i|^{p-2}(A\Delta)_i^2\}.$$

The quadratic regime is near a zero *displacement* at a nonzero residual. At a zero residual, the cost is $|s|^p$ at every scale. The displayed minimum is useful for comparison but is not itself convex: when $1 < p < 2$, its derivative drops where the two pieces meet. A convex replacement is

$$\gamma_p(a,s) = \begin{cases} \frac{p}{2}a^{p-2}s^2, & |s| \le a, \quad a > 0, \\ |s|^p - (1-p/2)a^p, & |s| > a, \end{cases} \qquad \gamma_p(0,s) = |s|^p.$$

The values and first derivatives match at $s = \pm a$, and the derivative is nondecreasing on each piece, so this function is convex and C^1. For $a > 0$, it lies between $p/2$ and 1 times $\min\{a^{p-2}s^2, |s|^p\}$. On the outer piece this follows from $a^p \le |s|^p$; on the inner piece it is immediate. Thus $\sum_i \gamma_p(|r_i|, (A\Delta)_i)$ is a concrete convex penalty comparable to the Bregman error. For fixed a and $s \neq 0$, $p\gamma_p(a,s) \le s\partial_s\gamma_p(a,s) \le 2\gamma_p(a,s)$. Integrating this logarithmic derivative gives $\eta^2\gamma_p(a,s) \le \gamma_p(a,\eta s) \le \eta^p\gamma_p(a,s)$ for $0 < \eta \le 1$. These scaling bounds also permit refinement below $p = 2$, provided a solver for this convex local penalty is available. The least-squares reduction developed below concerns $p \ge 2$.

14.9.2 Why a Local Improvement Reduces the Global Gap

Suppose $\underline{R}_x(\Delta) \le D_f(x+\Delta, x) \le \overline{R}_x(\Delta)$. The lower bound is an optimistic estimate of movement cost: evaluating its linearized objective at $x^* - x$ shows that its best improvement is at least the true objective gap. The upper bound has the opposite role: it certifies what a computed step really achieves.

An optimistic direction need not be safe at full length. The remedy is to shrink it. The linear benefit shrinks proportionally to the step length, whereas a superlinear penalty shrinks faster. Once the upper penalty of the shortened step is no larger than the same fraction of the original lower penalty, any approximation to the available local improvement becomes a proportional decrease of the global gap.

Lemma 479 (Refinement from local bounds)**.** *Let f be differentiable and convex, and let nonnegative penalties satisfy $\underline{R}_x(\Delta) \le D_f(x+\Delta, x) \le \overline{R}_x(\Delta)$. Suppose $s \in (0,1]$ is a safe step length, meaning that*

$$\overline{R}_x(s\Delta) \le s\underline{R}_x(\Delta) \qquad \text{for every } \Delta.$$

Assume the local objective below attains a finite minimum. If $C \ge 1$ and $\widehat{\Delta}$ satisfies

$$\langle \nabla f(x), \widehat{\Delta}\rangle + \underline{R}_x(\widehat{\Delta}) \le \frac{1}{C}\min_{\Delta}\{\langle \nabla f(x), \Delta\rangle + \underline{R}_x(\Delta)\},$$

then

$$f(x + s\widehat{\Delta}) - f(x^*) \le (1 - s/C)\big(f(x) - f(x^*)\big).$$

Proof. Evaluation at $\Delta = x^* - x$ bounds the local minimum by $\langle \nabla f(x), x^* - x\rangle + D_f(x^*, x) = f(x^*) - f(x)$. The safe-step condition then gives

$$\begin{aligned} f(x + s\widehat{\Delta}) - f(x) &\le s\langle \nabla f(x), \widehat{\Delta}\rangle + \overline{R}_x(s\widehat{\Delta}) \\ &\le s\big(\langle \nabla f(x), \widehat{\Delta}\rangle + \underline{R}_x(\widehat{\Delta})\big) \\ &\le -\frac{s}{C}\big(f(x) - f(x^*)\big). \end{aligned}$$

Adding the current gap proves the claim. □

The step length is explicit whenever $\underline{R}_x = c_p R_x$, $\overline{R}_x = C_p R_x$, $0 < c_p \le C_p$, and $R_x(\eta\Delta) \le \eta^{1+\delta}R_x(\Delta)$ for $0 < \eta \le 1$: take $s = (c_p/C_p)^{1/\delta}$. Indeed, $C_p s^{1+\delta}R_x(\Delta) = sc_pR_x(\Delta)$. For the quadratic-plus-power penalty with $p \ge 2$, use $\delta = 1$. For the smoothed power penalty with $1 < p \le 2$, use $\delta = p - 1$. Thus a constant-factor local improvement gives a constant-factor gap reduction for every fixed $p > 1$.

Algorithm 80 Iterative refinement from local bounds

Input: Initial point x, bound $E_0 \geq f(x) - f(x^*)$, target gap $e > 0$; a common safe step length s and a C-approximate local-improvement routine as in Lemma 479

if $E_0 \leq e$ **then**
  return x
$T \leftarrow \lceil (C/s)\log(E_0/e) \rceil$
for $t = 1, \ldots, T$ **do**
  $\widehat{\Delta} \leftarrow$ the routine applied to $\langle \nabla f(x), \Delta \rangle + \underline{R}_x(\Delta)$
  $x \leftarrow x + s\widehat{\Delta}$
return x

The bound follows from $(1 - s/C)^T \leq e^{-sT/C}$ and uses a known initial gap bound. We next implement the local improvement using weighted least squares.

14.9.3 The Penalty of an Optimal Direction

An optimal local step cannot hide an arbitrarily large penalty behind its negative linear term. Shrinking the step reduces a superlinear penalty faster than the benefit; optimality limits how much cancellation is possible.

Lemma 480 (Penalty of an optimal local step)**.** *Suppose $D \geq 0$ and $D(\eta\Delta) \leq \eta^{1+\delta} D(\Delta)$ for every $0 < \eta \leq 1$, where $\delta > 0$. If Δ^* minimizes $\langle g, \Delta \rangle + D(\Delta)$ with value $-\Gamma < 0$, then*

$$D(\Delta^*) \leq \frac{\Gamma}{\delta}, \qquad -\langle g, \Delta^* \rangle \leq \left(1 + \frac{1}{\delta}\right)\Gamma.$$

Proof. Put $K = D(\Delta^*)$, so $\langle g, \Delta^* \rangle = -\Gamma - K$. For $0 < \eta < 1$, optimality and the scaling assumption give $-\Gamma \leq -\eta(\Gamma + K) + \eta^{1+\delta}K$. Rearranging yields $K \leq \Gamma(1-\eta)/(\eta - \eta^{1+\delta})$. As $\eta \uparrow 1$, the ratio tends to $1/\delta$, proving the first claim; the second follows from $-\langle g, \Delta^* \rangle = \Gamma + K$. No differentiability of D is needed. □

14.9.4 Replacing the Power Penalty by a Quadratic

Fix $p \geq 2$, put $g = \nabla f(x)$, and define the available improvement in the lower-bound objective by

$$\Gamma = -\min_{\Delta} \{\langle g, \Delta \rangle + c_p R_x(\Delta)\}.$$

If $g = 0$, the current point is optimal. Otherwise $\Gamma > 0$, and evaluation at $x^* - x$ gives $\Gamma \geq f(x) - f(x^*)$. Since $R_x(\eta\Delta) \leq \eta^2 R_x(\Delta)$ for $\eta \leq 1$, Lemma 480 gives $c_p R_x(\Delta^*) \leq \Gamma$ and $-\langle g, \Delta^* \rangle \in [\Gamma, 2\Gamma]$.

Suppose for the moment that we ask for a linear improvement $\gamma \leq \Gamma$. Rescale Δ^* to a benchmark Δ^*_γ satisfying $\langle g, \Delta^*_\gamma \rangle = -\gamma$. The factor is at most $\gamma/\Gamma \leq 1$, so its penalty is at most $(\gamma/\Gamma)^2 R_x(\Delta^*) \leq \gamma/c_p$. Thus $Q_x(\Delta^*_\gamma)$ and $P(\Delta^*_\gamma)$ are each at most γ/c_p. The second bound becomes a quadratic one by norm comparison:

$$\begin{aligned} \|A\Delta^*_\gamma\|_2^2 &\leq m^{1-2/p} \|A\Delta^*_\gamma\|_p^2 \\ &= m^{1-2/p} P(\Delta^*_\gamma)^{2/p} \leq m^{1-2/p} (\gamma/c_p)^{2/p}. \end{aligned}$$

Therefore the quadratic problem

$$\min_{\langle g, \Delta \rangle = -\gamma} \left\{ \frac{Q_x(\Delta)}{\gamma} + \frac{\|A\Delta\|_2^2}{m^{1-2/p}\gamma^{2/p}} \right\}$$

has optimum at most $K_p^{(0)} = c_p^{-1} + c_p^{-2/p}$. Both denominators are now explained: each normalizes a quadratic cost by the scale achieved by the unknown benchmark. The equality constraint fixes the linear benefit, so the least-squares solve only needs to find a direction with comparably small normalized movement costs.

Let H_γ be the matrix of this quadratic. Explicitly,

$$H_\gamma = A^\top \operatorname{diag}\left(\frac{|r_i|^{p-2}}{\gamma} + \frac{1}{m^{1-2/p}\gamma^{2/p}}\right) A.$$

Work on $(\ker A)^\perp$ if A is rank deficient. All diagonal weights are positive and $g \in \operatorname{im} A^\top = (\ker A)^\perp$, so H_γ is positive definite on this space. Write $H = H_\gamma$, $u_* = H^{-1}g$, and $q = \langle g, u_*\rangle = \|u_*\|_H^2 > 0$. Every feasible direction satisfies $\gamma^2 = \langle Hu_*, \Delta\rangle^2 \le q\,\Delta^\top H\Delta$. Equality holds for $\Delta_* = -\gamma u_*/q$, so the exact quadratic optimum is γ^2/q.

Crucially, this linear system needs only constant relative accuracy. Suppose a solver returns u with $\|u - u_*\|_H \le \frac{1}{4}\|u_*\|_H$. Cauchy–Schwarz gives

$$|\langle g, u - u_*\rangle| = |\langle Hu_*, u - u_*\rangle| \le q/4, \qquad \|u\|_H \le \frac{5}{4}\sqrt{q}.$$

Hence $\langle g, u\rangle \ge 3q/4 > 0$. Renormalizing the computed direction enforces the linear constraint exactly:

$$\Delta = -\frac{\gamma}{\langle g, u\rangle}u, \qquad \Delta^\top H\Delta = \frac{\gamma^2\|u\|_H^2}{\langle g, u\rangle^2} \le \frac{25}{9}\frac{\gamma^2}{q} \le 3K_p^{(0)}.$$

Thus set $K_p = 3(c_p^{-1} + c_p^{-2/p})$ from now on. Each nonnegative term of the normalized quadratic is at most K_p, so the approximate solve gives

$$Q_x(\Delta) \le K_p\gamma, \qquad P(\Delta) \le \|A\Delta\|_2^p \le K_p^{p/2}m^{(p-2)/2}\gamma.$$

The loss comes from the second inequality: a Euclidean bound allows a few coordinates to be much too large for the p-th-power cost.

Shorten the step by $\theta = c_p' m^{-(p-2)/(2p-2)}$, where $c_p' > 0$ is a sufficiently small constant. The linear gain scales as θ, the quadratic cost as θ^2, and the power cost as θ^p. Thus

$$\begin{aligned} f(x + \theta\Delta) - f(x) &\le -\theta\gamma + C_p\big(K_p\theta^2 + K_p^{p/2}\theta^p m^{(p-2)/2}\big)\gamma \\ &\le -\frac{\theta\gamma}{2}. \end{aligned}$$

To verify the last inequality, divide the positive terms by the linear gain $\theta\gamma$. We need

$$C_pK_p\theta + C_pK_p^{p/2}\theta^{p-1}m^{(p-2)/2} \le \frac{1}{2}.$$

For $\theta = c_p' m^{-(p-2)/(2p-2)}$, the powers of m in the second term cancel: $\theta^{p-1}m^{(p-2)/2} = (c_p')^{p-1}$, while $\theta \le c_p'$. Choose $c_p' \le 1$ so that $C_pK_pc_p' \le 1/4$ and $C_pK_p^{p/2}(c_p')^{p-1} \le 1/4$. Both requirements depend only on p. For definiteness, take $c_p' = \min\{1, (4C_pK_p)^{-1}, (4C_pK_p^{p/2})^{-1/(p-1)}\}$. This derives both the dimension exponent and a computable shortening factor. The guarantee holds for every $0 < \gamma \le \Gamma$; that one-sided range will make a failed trial a valid gap certificate.

All constants are now explicit in terms of p; their sizes affect only the dependence hidden in O_p. The inner accuracy stays fixed as the outer target error decreases. Renormalization preserves the linear gain, and the constant enlargement of K_p absorbs the solve error.

14.9.5 Choosing the Scale Without Knowing the Optimum

The analysis used $\gamma \le \Gamma$, but Γ is unknown. We do not need to compute it. Try a scale γ and test the actual objective decrease. If it is at least $\theta\gamma/2$, keep the new point. If not, the guarantee just proved implies $\gamma > \Gamma \ge f(x) - f(x^*)$. Thus failure certifies that the remaining gap is below γ, and we halve the scale without changing x.

Each trial therefore has a certified outcome: an actual decrease, or a smaller upper bound on the remaining gap.

Maintain $f(x) - f(x^*) \le 2\gamma$. Acceptance preserves this invariant because the gap decreases. Failure also preserves it after replacing γ by $\gamma/2$. At the beginning of a scale, the gap is at most 2γ. If j steps are accepted without changing the scale, the gap is at most $2\gamma - j\theta\gamma/2$. It cannot become negative. Thus $j \le 4/\theta$: either the optimum is reached or a trial must eventually fail and certify a smaller scale.

For a relative guarantee, initialize with an ordinary least-squares minimizer x_0. If $f_0 = f(x_0) > 0$, norm comparison gives

$$f_0 \le \|Ax_0 - b\|_2^p \le \|Ax^* - b\|_2^p \le m^{p/2-1} f(x^*).$$

Thus the known target gap $e = \varepsilon f_0 / m^{p/2-1}$ is at most $\varepsilon f(x^*)$.

Algorithm 81 High-accuracy ℓ_p regression using weighted least squares

Input: A, b, fixed $p \ge 2$, $\varepsilon \in (0,1)$; $\theta = c_p' m^{-(p-2)/(2p-2)}$, with the explicit constants above
Output: $f(x) \le (1+\varepsilon)\min_z f(z)$, where $f(z) = \|Az - b\|_p^p$

$x \leftarrow \arg\min_z \|Az - b\|_2^2$; $f_0 \leftarrow \|Ax - b\|_p^p$
if $f_0 = 0$ or $p = 2$ **then**
  return x
$e \leftarrow \varepsilon f_0 / m^{p/2-1}$; $\gamma \leftarrow f_0$
while $\gamma > e/2$ **do**
  $r \leftarrow Ax - b$; $g \leftarrow pA^\top(|r|^{p-2} \odot r)$
  if $g = 0$ **then**
    return x
  $d_i \leftarrow |r_i|^{p-2}/\gamma + 1/(m^{1-2/p}\gamma^{2/p})$ for every i
  $H \leftarrow A^\top \operatorname{diag}(d) A$
  Solve $Hu = g$ to relative H-norm error at most $1/4$ ▷ *on* $(\ker A)^\perp$
  $\Delta \leftarrow -\gamma u / \langle g, u \rangle$
  if $f(x + \theta\Delta) \le f(x) - \theta\gamma/2$ **then**
    $x \leftarrow x + \theta\Delta$
  else
    $\gamma \leftarrow \gamma/2$
return x

Theorem 481 (High-accuracy ℓ_p regression by least squares)**.** *For fixed $p \ge 2$, Algorithm 81 returns $f(x) \le (1+\varepsilon)\min_z f(z)$ using one initial least-squares minimization and*

$$O_p\left(m^{\frac{p-2}{2p-2}} \log \frac{2m}{\varepsilon}\right)$$

weighted least-squares solves at constant relative energy-norm accuracy. This is $\widetilde{O}_p(m^{(p-2)/(2p-2)} \log(2/\varepsilon))$; the explicit bound already includes the improvement-scale search.

Proof. Initially the gap is at most $f_0 = \gamma$. The preceding acceptance and failure arguments preserve the bound $f(x) - f(x^*) \le 2\gamma$. At termination it is at most $e \le \varepsilon f(x^*)$. There are $O(\log(2f_0/e)) = O_p(\log(2m/\varepsilon))$ scales, with at most $4/\theta$ accepted trials and one failed trial per scale. □

This method obtains high accuracy by repeatedly solving a *local* problem only approximately. The remaining dimension loss is entirely in the replacement of the power penalty by one global Euclidean bound. The next method addresses concentration directly, using ℓ_∞ regression as the model problem.

14.10 Width Reduction and Iteratively Reweighted Least Squares

The preceding reduction loses accuracy when a least-squares direction concentrates too much of its mass on a few coordinates. Shortening the entire direction protects against those coordinates, but also suppresses progress on every other coordinate. Width reduction instead makes the offending coordinates more expensive and solves again.

There are two possible outcomes. Reweighting may distribute the solution more evenly, giving a good primal solution. Alternatively, the weights may identify a collection of coordinates that every feasible solution must use heavily. Those weights then certify that the target was too ambitious. The algorithm is designed so that either outcome counts as progress.[1]

14.10.1 The Decision Problem and Its Quadratic Relaxation

Consider the feasible affine problem $\mathrm{OPT} = \min_{Bx=d} \|x\|_\infty$, where $x \in \mathbb{R}^m$. Ordinary regression can be written this way in residual space: choose B with $\ker B = \operatorname{im} A$ and set $d = -Bb$. Then $Bx = d$ means exactly that $x = Az - b$ for some z.

Given $M > 0$ and $0 < \varepsilon < 1$, we seek either a feasible x with $\|x\|_\infty \le (1+\varepsilon)M$, or a certificate that $\mathrm{OPT} \ge (1-\varepsilon)M$. The overlap between these conclusions is intentional: this is an approximate decision problem.

For positive weights r, let

$$\mathcal{E}_r(d) = \min_{Bx=d} \sum_i r_i x_i^2.$$

When $\sum_i r_i = 1$, this minimum is a lower bound on OPT^2: any feasible vector of width M has weighted energy at most M^2. Thus a least-squares solve serves two purposes. Its minimizer is a candidate primal solution, and its minimum value is a certificate against narrow solutions.

The quadratic problem is an explicit linear solve. With $R = \operatorname{diag}(r)$, its Lagrangian is $x^\top Rx + 2\phi^\top(d - Bx)$. Stationarity in x and feasibility give

$$2Rx - 2B^\top\phi = 0, \qquad x = R^{-1}B^\top\phi, \qquad BR^{-1}B^\top\phi = d.$$

Thus we first solve the last system and then recover x. Remove redundant equations so that B has full row rank; the system matrix is then positive definite because $z^\top BR^{-1}B^\top z = \|B^\top z\|_{R^{-1}}^2 > 0$ for $z \ne 0$. Equivalently, keep redundant equations and solve on the matrix's range.

For a regression input, there is no need to construct B. Minimize $\|R^{1/2}(Az-b)\|_2^2$ directly by solving $A^\top RAz = A^\top Rb$, and return the residual $x = Az - b$. This implements exactly the same energy minimization while retaining the sparsity of A; an explicitly formed matrix annihilating A could be dense. The analysis in this subsection uses exact weighted minimizers. Its stated solve count does not by itself justify replacing them with constant-accuracy solutions; the preceding refinement algorithm includes that separate error analysis.

14.10.2 The Algorithm

A coordinate with $|x_i| \le M$ needs no correction. A coordinate with $|x_i| > M$ is reweighted by its squared violation, x_i^2/M^2. This particular multiplier is not just a heuristic: the energy calculation below shows that it makes each unit of new weight produce at least M^2 units of new energy.

We average only solutions of width at most ρM. A very wide solution may be unsuitable for averaging, but it forces a large weight increase, so it is still useful. Set $\rho = \max\{2, m^{1/3}\}$ for the bound proved below.

[1] For iteratively reweighted least squares in these regression problems, see Alina Ene and Adrian Vladu, *Improved Convergence for ℓ_∞ and ℓ_1 Regression via Iteratively Reweighted Least Squares*, arXiv:1902.06391. The bounds below are proved for the displayed elementary procedure.

Algorithm 82 Thresholded IRLS for an ℓ_∞ decision problem

Input: Feasible system $Bx = d$, target $M > 0$, $0 < \varepsilon < 1$, threshold $\rho \geq 2$
Output: A feasible vector of width at most $(1+\varepsilon)M$, or a certificate that OPT $\geq (1-\varepsilon)M$

function ThresholdedIRLS($B, d, M, \varepsilon, \rho$)
  $r \leftarrow \mathbf{1}/m$, $s \leftarrow 0$, $q \leftarrow 0$
  while $\|r\|_1 < 1/\varepsilon$ **do**
    Solve $BR^{-1}B^\top \phi = d$, where $R = \mathrm{diag}(r)$
    $x \leftarrow R^{-1}B^\top \phi$
    if $\|x\|_\infty \leq (1+\varepsilon)M$ **then**
      return (primal, x)
    if $\|x\|_\infty \leq \rho M$ **then**
      $s \leftarrow s + x$, $q \leftarrow q + 1$
      if $\|s/q\|_\infty \leq (1+\varepsilon)M$ **then**
        return (primal, s/q) ▷ *Average of feasible solutions*
    $r_i \leftarrow r_i \max\{1, x_i^2/M^2\}$ for every i
  $R_{\mathrm{tot}} \leftarrow \|r\|_1$; $\bar{r} \leftarrow r/R_{\mathrm{tot}}$; $\bar{\phi} \leftarrow \phi/R_{\mathrm{tot}}$
  return (dual, $(\bar{r}, \bar{\phi})$)

The proof separates the two ways in which the algorithm can fail to return a primal solution: persistent moderate violations multiply one weight many times, while a single very large violation increases total weight sharply. First we show why large total weight is a valid stopping condition.

14.10.3 Why Squared Violations Are the Right Multipliers

To compare the minimum before and after reweighting, use its dual rather than guess how its minimizer moves. Writing β_i for column i of B, complete the square coordinatewise in the Lagrangian:

$$\begin{aligned}\sum_i r_i x_i^2 + 2\phi^\top(d - Bx) &= 2d^\top\phi + \sum_i \left(r_i x_i^2 - 2(\beta_i^\top\phi)x_i\right) \\ &= 2d^\top\phi - \sum_i \frac{(\beta_i^\top\phi)^2}{r_i} + \sum_i r_i\left(x_i - \frac{\beta_i^\top\phi}{r_i}\right)^2.\end{aligned}$$

Minimizing over x removes the squares. The primal solution and multiplier just constructed satisfy $\beta_i^\top\phi = r_i x_i$ and attain equality, so

$$\mathcal{E}_r(d) = \max_\phi \left\{2d^\top\phi - \sum_i \frac{(\beta_i^\top\phi)^2}{r_i}\right\}.$$

The old dual solution remains a valid comparison after reweighting. This is more useful than comparing primal solutions: the new minimizer may change substantially, but the old dual solution still gives a lower bound on the new minimum.

Lemma 482 (Energy increase under weight growth)**.** *If $r' \geq r > 0$ coordinatewise and x minimizes the energy for r, then*

$$\mathcal{E}_{r'}(d) \geq \mathcal{E}_r(d) + \sum_i r_i x_i^2\left(1 - \frac{r_i}{r_i'}\right).$$

Proof. Evaluate the new dual at an old maximizer ϕ:

$$\begin{aligned}\mathcal{E}_{r'}(d) &\geq 2d^\top\phi - \sum_i \frac{(\beta_i^\top\phi)^2}{r'_i} \\ &= \mathcal{E}_r(d) + \sum_i (\beta_i^\top\phi)^2 \left(\frac{1}{r_i} - \frac{1}{r'_i}\right) \\ &= \mathcal{E}_r(d) + \sum_i r_i x_i^2 \left(1 - \frac{r_i}{r'_i}\right).\end{aligned}$$

□

Suppose $r'_i = \alpha_i r_i$ with $\alpha_i > 1$. The energy lower bound per unit of new weight is

$$\frac{r_i x_i^2 (1 - 1/\alpha_i)}{r_i(\alpha_i - 1)} = \frac{x_i^2}{\alpha_i}.$$

We want this ratio to be at least M^2, so the largest allowable multiplier is $\alpha_i = x_i^2/M^2$. It exceeds one exactly at a violated coordinate. This derives the update: increase each violated weight as much as this energy guarantee permits, and leave the others unchanged. Consequently every update satisfies

$$\mathcal{E}_{r'}(d) - \mathcal{E}_r(d) \geq M^2\big(\|r'\|_1 - \|r\|_1\big).$$

Write $r^{(t)}$ for the weights after t updates. Summing the energy inequality makes the intermediate energies and weight sums cancel:

$$\begin{aligned}\mathcal{E}_{r^{(t)}}(d) &\geq \mathcal{E}_{r^{(0)}}(d) + M^2 \sum_{j<t} \big(\|r^{(j+1)}\|_1 - \|r^{(j)}\|_1\big) \\ &= \mathcal{E}_{r^{(0)}}(d) + M^2(\|r^{(t)}\|_1 - 1) \\ &\geq M^2(\|r^{(t)}\|_1 - 1).\end{aligned}$$

Now write $r = r^{(t)}$ and normalize to $\bar{r} = r/\|r\|_1$. For any feasible z, $\sum_i \bar{r}_i z_i^2 \leq \|z\|_\infty^2$. In particular, evaluating at a width-minimizing feasible vector shows $\mathcal{E}_{\bar{r}}(d) \leq \mathrm{OPT}^2$. Scaling all weights scales the minimum by the same factor, so

$$\mathrm{OPT}^2 \geq \mathcal{E}_{\bar{r}}(d) = \frac{\mathcal{E}_r(d)}{\|r\|_1} \geq M^2 \left(1 - \frac{1}{\|r\|_1}\right).$$

At $\|r\|_1 \geq 1/\varepsilon$, this proves $\mathrm{OPT} \geq M\sqrt{1-\varepsilon} \geq (1-\varepsilon)M$. The certificate can also be checked without another optimization problem. Let ϕ be the multiplier from the last solve, before the final update to r, and let $R_{\mathrm{tot}} = \|r\|_1$. The same old-dual calculation gives $2d^\top\phi - \sum_i (\beta_i^\top\phi)^2/r_i \geq M^2(R_{\mathrm{tot}} - 1)$. For $\bar{\phi} = \phi/R_{\mathrm{tot}}$ and $\bar{r} = r/R_{\mathrm{tot}}$, therefore,

$$2d^\top\bar{\phi} - \sum_i \frac{(\beta_i^\top\bar{\phi})^2}{\bar{r}_i} \geq (1-\varepsilon)M^2.$$

The left side is an explicitly evaluable dual lower bound on $\mathcal{E}_{\bar{r}}(d)$, hence on OPT^2. This is the pair returned by the algorithm when no primal solution is found.

The certificate also has a direct regression implementation. For the last weighted residual $x = Az - b$, retain $\xi = R_{\mathrm{old}}x$. The normal equations give $A^\top\xi = 0$, and any such vector certifies

$$\min_z \sum_i r_i(Az - b)_i^2 \geq -2b^\top\xi - \sum_i \frac{\xi_i^2}{r_i}.$$

Indeed, complete the square in each residual coordinate and use $\xi^\top Az = 0$. This is the same dual bound as above, with $\xi = B^\top\phi$ and $d = -Bb$. After the final update, return $(\bar{r}, \bar{\xi}) = (r/R_{\mathrm{tot}}, \xi/R_{\mathrm{tot}})$; its displayed dual value is at least $(1-\varepsilon)M^2$. Thus neither the primal implementation nor its certificate requires constructing B. When the output sought is a regression coefficient vector, retain each z alongside its residual and average the z's on exactly the iterations used for the residual average.

14.10.4 Counting Moderate and Very-Wide Iterations

Let I contain the moderate iterations whose updates have been completed, and put $q = |I| > 0$. If their average has not passed the test, then some coordinate i satisfies $|\sum_{t\in I} x_i^{(t)}| > (1+\varepsilon)Mq$. Define $z_t = \max\{1, |x_i^{(t)}|/M\}$. The triangle inequality gives

$$\sum_{t\in I}(z_t - 1) = \sum_{t\in I} z_t - q \geq \frac{1}{M}\sum_{t\in I}|x_i^{(t)}| - q \geq \frac{1}{M}\left|\sum_{t\in I} x_i^{(t)}\right| - q > \varepsilon q.$$

Since each of these iterations was moderate, $1 \leq z_t \leq \rho$. The coordinate i is chosen only after looking at the average; it need not have been the worst coordinate at every iteration. What matters is its cumulative excess. The bound $z_t \leq \rho$ is precisely why we excluded very-wide iterates: it converts that cumulative excess into a lower bound on cumulative log-weight growth. Indeed, write $z = (1-\theta)\cdot 1 + \theta\rho$, with $\theta = (z-1)/(\rho-1)$. Concavity of log says that its graph lies above the chord between 1 and ρ, hence

$$\log z \geq \frac{z-1}{\rho-1}\log\rho.$$

Applying this separately at every moderate iteration gives

$$\sum_{t\in I}\log z_t \geq \frac{\log\rho}{\rho-1}\sum_{t\in I}(z_t-1) > \frac{\varepsilon q\log\rho}{\rho-1}.$$

Each completed update multiplies r_i by z_t^2, and updates outside I can only increase it further. Thus, if no stopping condition has occurred,

$$\frac{1}{\varepsilon} > r_i \geq \frac{1}{m}\prod_{t\in I} z_t^2 > \frac{1}{m}\exp\left(\frac{2\varepsilon q\log\rho}{\rho-1}\right).$$

Taking logarithms and rearranging gives

$$q < \frac{\rho-1}{2\varepsilon\log\rho}\log\frac{m}{\varepsilon}.$$

Thus there are $O(1+\rho\log(m/\varepsilon)/(\varepsilon\log\rho))$ moderate iterations, including the possible final iteration that returns before updating weights. The argument chooses the witness coordinate after the iterations, but the lower bound on its weight uses the updates that have already occurred.

For a very-wide iterate, some $|x_i| > \rho M$, and its weight multiplier satisfies $\alpha_i = x_i^2/M^2 > \rho^2$. Since weights never decrease, $r_i \geq 1/m$, so this coordinate alone increases the total weight by

$$r_i' - r_i = r_i(\alpha_i - 1) > \frac{\rho^2-1}{m}.$$

Before the last update, total weight is less than $1/\varepsilon$. There can therefore be fewer than $m/(\varepsilon(\rho^2-1))$ very-wide updates before that last one. Allowing one final update to overshoot the threshold gives $O(1+m/(\varepsilon\rho^2))$ in total.

Ignoring logarithms, the two costs are ρ/ε and $m/(\varepsilon\rho^2)$. Allowing wider iterates makes the averaging argument weaker, but makes excluded iterates more expensive. Balancing these costs at $\rho = \max\{2, m^{1/3}\}$ gives

$$\widetilde{O}\left(\frac{m^{1/3}}{\varepsilon}\right)$$

weighted least-squares solves for the decision problem. This balance, rather than any property of a single least-squares solve, is the source of the cube-root exponent.

Theorem 483 (Thresholded IRLS)**.** *For $\rho \geq 2$, Algorithm 82 solves the approximate decision problem using*

$$O\left(1 + \frac{\rho \log(m/\varepsilon)}{\varepsilon \log \rho} + \frac{m}{\varepsilon \rho^2}\right)$$

weighted least-squares solves. With $\rho = \max\{2, m^{1/3}\}$, this is $\widetilde{O}(m^{1/3}/\varepsilon)$.

Proof. The energy argument proves the returned dual certificate; feasibility and the explicit width test prove every primal answer. The moderate and very-wide iteration counts bound all solves, including a final returning or threshold-crossing iteration. □

14.10.5 From Approximate Decision to Optimization

The overlapping answers still permit a contracting interval search. Let x_2 be a minimum-ℓ_2 feasible vector. If $x_2 = 0$, it is optimal. Otherwise

$$0 < \ell := \|x_2\|_2/\sqrt{m} \leq \mathrm{OPT} \leq U := \|x_2\|_\infty \leq \sqrt{m}\,\ell.$$

The lower bound follows by comparing x_2 with an ℓ_∞ optimum; the upper bound has the feasible witness x_2.

Algorithm 83 IRLS optimization from approximate decisions

Input: Feasible $Bx = d$ with m variables and $0 < \varepsilon < 1$
Output: Feasible x with $\|x\|_\infty \leq (1+\varepsilon)\,\mathrm{OPT}$

Solve $BB^\top \phi = d$ and set $x \leftarrow B^\top \phi$
if $x = 0$ **then**
  return x
$\ell \leftarrow \|x\|_2/\sqrt{m}$, $U \leftarrow \|x\|_\infty$, $\rho \leftarrow \max\{2, m^{1/3}\}$
while $U > (1+\varepsilon)\ell$ **do**
  $M \leftarrow (\ell + U)/2$
  $(\mathrm{type}, w) \leftarrow \textsc{ThresholdedIRLS}(B, d, M, \varepsilon/10, \rho)$
  if type = primal **then**
    if $\|w\|_\infty < U$ **then**
      $x \leftarrow w$, $U \leftarrow \|w\|_\infty$
  else
    $\ell \leftarrow \max\{\ell, (1 - \varepsilon/10)M\}$
return x

Both branches preserve $\ell \leq \mathrm{OPT} \leq U$ and a feasible vector of width U. While the stopping test fails, $\varepsilon\ell < U - \ell$, so $(\varepsilon/10)M < 3(U-\ell)/20$. A primal answer puts the new upper bound at most $M + (\varepsilon/10)M$, and a dual answer puts the new lower bound at least $M - (\varepsilon/10)M$. Either branch shrinks the interval length by a factor at most $13/20$. Since initially $U/\ell \leq \sqrt{m}$ and the lower bound never decreases, there are $O(\log(2m/\varepsilon))$ calls. At termination, the retained feasible vector has width at most $(1+\varepsilon)\,\mathrm{OPT}$. For ordinary regression, each solve can again be performed directly with A in residual space, as described above.

The two regression methods use weighted least squares differently. Refinement shortens a direction until its local error is affordable. Width reduction changes the weights until the large coordinates either become manageable or certify that the requested width is impossible. In both cases, the useful output of an unsuccessful trial is a quantitative certificate that determines what to do next.

14.11 Interior-Point Methods

For a constrained problem, even a short Euclidean step can cross a nearby boundary. A barrier supplies a scale tied to feasibility. For example, the Hessian norm of $-\sum_i \log x_i$ is $\|\Delta\|_x^2 = \sum_i (\Delta_i/x_i)^2$: it measures relative changes in the positive coordinates, not their absolute changes.

Interior-point methods optimize the objective plus a barrier and gradually reduce the barrier's coefficient. Unlike a Bregman prox, a Newton step uses only the Hessian at the current point, not the reference function along the whole displacement. Its advantage is a linear-system implementation; its analysis must ensure that the Hessian does not change too much during the step.

There are two separate requirements. The barrier problem must approximate the original problem, and its minimizer must move slowly enough that the previous iterate remains a useful starting point. For linear programming, complementarity makes both requirements explicit.

14.11.1 The Central Path and Its Optimality Certificate

Consider $\min\{c^\top x : Ax = b,\ x \geq 0\}$. Assume a finite attained optimum and strictly feasible primal and dual points; remove redundant equality constraints. We first analyze path following from a centered strictly feasible point, then give a Newton procedure for centering any supplied strictly feasible point. Finding a strictly feasible point remains a separate initialization problem.

For $\mu > 0$, minimize $c^\top x - \mu \sum_i \log x_i$ over $Ax = b$, $x > 0$. This minimizer exists and is unique. To see existence, choose a strictly feasible dual slack $s_0 = c - A^\top y_0 > 0$ and write the objective on the affine space as $b^\top y_0 + \sum_i (s_{0,i} x_i - \mu \log x_i)$. Each summand is bounded below and tends to infinity as its coordinate approaches zero or infinity. Nonempty sublevel sets are therefore compact and bounded away from the boundary. Strict convexity gives uniqueness.

The Lagrangian, with equality multiplier y, is $c^\top x - \mu \sum_i \log x_i + y^\top (b - Ax)$. Its stationarity equation is $c - \mu X^{-1}\mathbf{1} - A^\top y = 0$, where $X = \operatorname{diag}(x)$. Define the slack $s = c - A^\top y$. Then $s_i = \mu / x_i > 0$ and $x_i s_i = \mu$ for every coordinate. Together with primal feasibility these give the central-path equations

$$Ax = b, \qquad A^\top y + s = c, \qquad Xs = \mu \mathbf{1}, \qquad x, s > 0.$$

For any primal–dual feasible pair, substitute $b = Ax$ and $c - A^\top y = s$ to obtain

$$c^\top x - b^\top y = c^\top x - x^\top A^\top y = x^\top s = \sum_i x_i s_i.$$

Weak duality makes this a computable upper bound on the primal objective gap. On the path, every summand is μ, so the bound is exactly $n\mu$. Decreasing μ therefore has a concrete meaning: it decreases all complementarity products, and hence the total duality gap, at the same rate.

An exact path point is unnecessary. Say that (x, y, s) is α-centered at μ if it is strictly feasible and $\|Xs - \mu\mathbf{1}\|_2 \leq \alpha\mu$. Then each product $x_i s_i$ lies between $(1-\alpha)\mu$ and $(1+\alpha)\mu$, and the gap is $n\mu + O(\alpha\sqrt{n}\,\mu)$. Keeping a small fixed α is enough.

14.11.2 The Path-Following Algorithm

For a new target $\widehat{\mu}$, write $g = Xs - \widehat{\mu}\mathbf{1}$. The correction must preserve feasibility and cancel the first-order complementarity error:

$$A\Delta x = 0, \qquad \Delta s = -A^\top \Delta y, \qquad S\Delta x + X\Delta s = -g.$$

Solve the complementarity equation for Δx and substitute dual feasibility:

$$\Delta x = -S^{-1} g - S^{-1} X \Delta s = -S^{-1} g + S^{-1} X A^\top \Delta y.$$

Primal feasibility then becomes

$$0 = A\Delta x = -AS^{-1} g + AS^{-1} X A^\top \Delta y, \qquad AS^{-1} X A^\top \Delta y = AS^{-1} g.$$

This is a weighted linear system because $S^{-1}X$ is positive diagonal. After solving for Δy, recover Δs and Δx from the preceding equations. Notice that g uses the *new* target: the correction follows the moved path rather than recentering at the old one.

Algorithm 84 Primal–dual path following for a linear program

Input: Strictly feasible (x, y, s) that is $1/10$-centered at $\mu > 0$; target $\mu_f > 0$

$\gamma \leftarrow 1/20$
while $\mu > \mu_f$ **do**
  $\widehat{\mu} \leftarrow \max\{\mu_f, (1 - \gamma/\sqrt{n})\mu\}$
  $X \leftarrow \operatorname{diag}(x)$, $S \leftarrow \operatorname{diag}(s)$, $g \leftarrow Xs - \widehat{\mu}\mathbf{1}$
  Solve $AS^{-1}XA^\top \Delta y = AS^{-1}g$
  $\Delta s \leftarrow -A^\top \Delta y$, $\Delta x \leftarrow -S^{-1}(g + X\Delta s)$
  $(x, y, s) \leftarrow (x + \Delta x, y + \Delta y, s + \Delta s)$
  $\mu \leftarrow \widehat{\mu}$
return (x, y, s) ▷ *Primal–dual gap* $O(n\mu_f)$

Why can one Newton step suffice after moving the target? The residual is a product, so canceling its linear part leaves only

$$(X + \Delta X)(s + \Delta s) - \widehat{\mu}\mathbf{1} = \Delta X \Delta s.$$

We must show that both relative corrections are small. Feasibility provides the missing control: primal and dual corrections are orthogonal, so they cannot be large and cancel each other in the Newton equation.

Lemma 484 (Quadratic recentering)**.** *For $0 \le \alpha < 1/2$, a Newton correction at a fixed target μ preserves strict feasibility and takes an α-centered point to one with centrality at most $\alpha^2(1+\alpha)/(2(1-\alpha)^2)$.*

Proof. Put $w_i = x_i s_i$, $u = X^{-1}\Delta x$, and $v = S^{-1}\Delta s$. The Newton equation says $w \odot (u + v) = -g$, while feasibility gives

$$\sum_i w_i u_i v_i = (\Delta x)^\top \Delta s = -(A\Delta x)^\top \Delta y = 0.$$

The vanishing cross term prevents the two relative corrections from being large but canceling. Explicitly,

$$\begin{aligned}\sum_i w_i(u_i^2 + v_i^2) &= \sum_i w_i(u_i + v_i)^2 - 2\sum_i w_i u_i v_i \\ &= \sum_i w_i \left(-\frac{g_i}{w_i}\right)^2 = \sum_i \frac{g_i^2}{w_i}.\end{aligned}$$

Centrality gives $w_i \ge (1-\alpha)\mu$ and $\|g\|_2 \le \alpha\mu$. Use this lower bound on the weights on both sides of the identity:

$$\begin{aligned}(1-\alpha)\mu(\|u\|_2^2 + \|v\|_2^2) &\le \sum_i w_i(u_i^2 + v_i^2) \\ &= \sum_i \frac{g_i^2}{w_i} \le \frac{\|g\|_2^2}{(1-\alpha)\mu}.\end{aligned}$$

Dividing by $(1-\alpha)\mu$ therefore gives $\|u\|_2^2 + \|v\|_2^2 \le \alpha^2/(1-\alpha)^2$. For $\alpha < 1/2$, both relative corrections have every coordinate below one in absolute value, so positivity is preserved. The new error is $g_i^+ = w_i u_i v_i$. Using $\|u\|_2\|v\|_2 \le (\|u\|_2^2 + \|v\|_2^2)/2$ gives

$$\frac{\|g^+\|_2}{\mu} \le (1+\alpha)\|u\|_2\|v\|_2 \le \frac{\alpha^2(1+\alpha)}{2(1-\alpha)^2}.$$

□

Now choose $(1 - \gamma/\sqrt{n})\mu \le \widehat{\mu} \le \mu$, as in the algorithm. Before changing the iterate,

$$\|Xs - \widehat{\mu}\mathbf{1}\|_2 \le \|Xs - \mu\mathbf{1}\|_2 + (\mu - \widehat{\mu})\sqrt{n} \le (\alpha + \gamma)\mu.$$

The factor $\sqrt{n}$ appears because all n complementarity targets move at once. Dividing the estimate by the new parameter gives centrality at most $(\alpha+\gamma)/(1-\gamma/\sqrt{n})$ before recentering. For the algorithm's $\alpha = 1/10$

and $\gamma = 1/20$, this is below $1/6$. The preceding bound takes centrality $1/6$ to at most $7/300 < 1/10$, restoring the invariant after one Newton step.

Thus each iteration both decreases the parameter and returns to the same centrality neighborhood. For $\mu_0 > \mu_f$ the iteration count is $O(1 + \sqrt{n}\log(\mu_0/\mu_f))$, with one weighted solve per iteration. The returned gap is at most $(n + \sqrt{n}/10)\mu_f$; choosing $\mu_f = \varepsilon/(2n)$ therefore suffices for additive error ε. No iterations are needed when the initial parameter is already below the target.

14.11.3 Self-Concordance: When Is the Newton Model Reliable?

The LP proof used the explicit product structure. For a general barrier, self-concordance supplies the same protection: a short step in the current Hessian norm cannot change that Hessian too much.

Let Φ be a C^3 convex barrier with positive definite Hessian on the interior of its convex domain; in particular, Φ tends to $+\infty$ when an interior sequence approaches its boundary. Equality constraints are handled by restricting to the feasible affine space. Define $\|v\|_x^2 = v^\top \nabla^2\Phi(x)v$ and $\|g\|_{x,*}^2 = g^\top \nabla^2\Phi(x)^{-1}g$.

Definition 485 (Self-concordance)**.** We use the mixed-direction formulation: Φ is self-concordant if, for every x, u, v,

$$|\nabla^3\Phi(x)[u, v, v]| \le 2\|u\|_x\|v\|_x^2.$$

Setting $u = v$ gives the familiar diagonal inequality $|\nabla^3\Phi(x)[v, v, v]| \le 2\|v\|_x^3$. The mixed formulation also controls curvature in a direction different from the one in which we move: the relative rate of change of $\|v\|_x^2$ along u is at most $2\|u\|_x$. Because the Hessian supplies the units, the condition is unchanged by an invertible affine change of coordinates.

For the logarithmic barrier $\Phi(x) = -\sum_{i=1}^m \log(b_i - a_i^\top x)$, put $\alpha_i = a_i^\top u/(b_i - a_i^\top x)$ and $\beta_i = a_i^\top v/(b_i - a_i^\top x)$. Then $\|u\|_x^2 = \sum_i \alpha_i^2$ and $\nabla^3\Phi(x)[u, v, v] = 2\sum_i \alpha_i\beta_i^2$. The bound $|\sum_i \alpha_i\beta_i^2| \le \|\alpha\|_\infty\|\beta\|_2^2 \le \|\alpha\|_2\|\beta\|_2^2$ verifies self-concordance directly. Also $|\langle\nabla\Phi(x), u\rangle| = |\sum_i \alpha_i| \le \sqrt{m}\|u\|_x$, proving the barrier-parameter bound $\|\nabla\Phi(x)\|_{x,*}^2 \le m$ used below.

Lemma 486 (Hessian stability in a Dikin ball)**.** *If $\rho = \|h\|_x < 1$, then $x + h$ remains in the domain and*

$$(1 - \rho)^2\nabla^2\Phi(x) \preceq \nabla^2\Phi(x + h) \preceq (1 - \rho)^{-2}\nabla^2\Phi(x).$$

Proof. The zero direction is trivial. Along the segment, put $a(t) = \|h\|_{x+th}$. Since $(a^2)' = \nabla^3\Phi(x+th)[h, h, h]$, self-concordance gives $2a(t)|a'(t)| \le 2a(t)^3$, hence $|a'(t)| \le a(t)^2$. Taking a reciprocal removes the quadratic dependence:

$$\left|\frac{d}{dt}\frac{1}{a(t)}\right| = \frac{|a'(t)|}{a(t)^2} \le 1.$$

Starting from $a(0) = \rho > 0$, integration yields $1/\rho - t \le 1/a(t) \le 1/\rho + t$. In particular, while $t\rho < 1$,

$$\frac{\rho}{1 + t\rho} \le a(t) \le \frac{\rho}{1 - t\rho}.$$

This first controls how fast the direction becomes long in its *own* changing local norm. The lower bound will also be useful when comparing a near-centered point with the exact path. For a fixed nonzero v, put $q(t) = v^\top \nabla^2\Phi(x + th)v$. The mixed inequality gives

$$\left|\frac{d}{dt}\log q(t)\right| \le 2a(t) \le \frac{2\rho}{1 - t\rho}.$$

Integrating the logarithmic derivative gives

$$\left|\log\frac{q(t)}{q(0)}\right| \le \int_0^t \frac{2\rho}{1 - s\rho}\,ds = -2\log(1 - t\rho).$$

Exponentiation yields $(1 - t\rho)^2 q(0) \le q(t) \le (1 - t\rho)^{-2}q(0)$.

The calculation initially holds only while the segment remains in the domain. Suppose its first boundary point occurred at $t_* \le 1$. For $0 \le t < t_*$, the second derivative of $\varphi(t) = \Phi(x + th)$ is $a(t)^2 \le \rho^2/(1-\rho)^2$. Integrating twice would give $\varphi(t) \le \varphi(0) + t\varphi'(0) + t^2\rho^2/(2(1-\rho)^2)$, bounded as $t \uparrow t_*$. This contradicts the barrier property. Thus the whole segment is feasible. Take $t = 1$ and use the scalar quadratic-form bounds for every v to obtain the claimed Loewner inequalities. □

14.11.4 Damped Newton Steps and a Centered Starting Point

Hessian stability also gives bounds on the error of the linear model. Define $\omega(r) = r - \log(1+r)$ for $r \ge 0$ and $\omega_*(r) = -r - \log(1-r)$ for $0 \le r < 1$. For a feasible displacement h with $\rho = \|h\|_x$, the reciprocal-curvature calculation gives $\|h\|_{x+sh} \ge \rho/(1+s\rho)$ all along its segment. When $\rho < 1$, it also gives the upper bound $\rho/(1-s\rho)$. Integrating the second derivative twice therefore yields

$$\begin{aligned} D_\Phi(x+h,x) &= \int_0^1 (1-s)\|h\|_{x+sh}^2 \, ds \\ &\ge \int_0^1 \frac{(1-s)\rho^2}{(1+s\rho)^2} \, ds = \omega(\rho), \\ D_\Phi(x+h,x) &\le \int_0^1 \frac{(1-s)\rho^2}{(1-s\rho)^2} \, ds = \omega_*(\rho) \qquad (\rho < 1). \end{aligned}$$

The lower bound holds for every feasible displacement; the upper bound requires local length less than one, which also guarantees feasibility.

Use $F_t(x) = tc^\top x + \Phi(x)$, where $t = 1/\mu$, and assume F_t attains its minimum at an interior point for every parameter used. This holds, for example, on a bounded feasible domain and for the LP above. The positive definite Hessian makes the minimizer unique. The linear term does not affect the Hessian or Bregman error, so the same bounds apply to F_t.

Put $H = \nabla^2\Phi(x)$, $g_t(x) = tc + \nabla\Phi(x)$, and $d = -H^{-1}g_t(x)$. The *Newton decrement* $\lambda_t(x) = \|g_t(x)\|_{x,*} = \|d\|_x$ measures both the gradient error and the local length of the proposed correction. Write $\lambda = \lambda_t(x)$. At a general starting point λ may be large, so the full step need not be feasible. Shorten it to $h = \eta d$, with $0 \le \eta\lambda < 1$. Since $\langle g_t(x), d\rangle = -\lambda^2$, the upper model gives

$$F_t(x + \eta d) - F_t(x) \le -\eta\lambda^2 + \omega_*(\eta\lambda).$$

For $\lambda > 0$, differentiating the right side shows that it is minimized at $\eta = 1/(1+\lambda)$. This choice makes $\|\eta d\|_x = \lambda/(1+\lambda) < 1$ and yields the guaranteed decrease

$$F_t\left(x + \frac{d}{1+\lambda}\right) \le F_t(x) - \lambda + \log(1+\lambda) = F_t(x) - \omega(\lambda).$$

Thus the damping factor is determined by the largest decrease certified by the local model. It needs no line search or knowledge of the optimum.

Algorithm 85 Damped Newton centering: Center(t, x, τ)

Input: $t > 0$, a strictly feasible point x, target decrement $\tau \in (0,1)$
Output: A strictly feasible x with $\lambda_t(x) \le \tau$

```
while true do
    H ← ∇²Φ(x), g ← tc + ∇Φ(x)
    Solve Hd = −g; λ ← √(−⟨g, d⟩)
    if λ ≤ τ then
        return x
    x ← x + d/(1 + λ)
```

Because ω is increasing and positive on $(0,\infty)$, every update before termination decreases F_t by at least $\omega(\tau)$. Starting at x_0, the number of linear solves is at most $1 + (F_t(x_0) - \min F_t)/\omega(\tau)$. The unknown optimum appears only in this analysis; the stopping test uses the computed decrement.

For the LP, this procedure also supplies the dual point needed by the earlier primal–dual algorithm. Write the Newton equations before eliminating the equality constraints as

$$X^{-2}d + tc - X^{-1}\mathbf{1} = A^\top z, \qquad Ad = 0.$$

Explicitly, solve $AX^2A^\top z = AX^2(tc - X^{-1}\mathbf{1})$ and set $d = -X^2(tc - X^{-1}\mathbf{1} - A^\top z)$. Set $\mu = 1/t$, $y = z/t$, and $s = c - A^\top y$. Then $Xs = \mu(\mathbf{1} - X^{-1}d)$ and $\|Xs - \mu\mathbf{1}\|_2 = \mu\|X^{-1}d\|_2 = \mu\lambda_t(x)$. When $\lambda_t(x) < 1$, every coordinate of s is positive. Thus stopping at $\tau = 1/16$ and recovering y, s from the final Newton solve gives a strictly feasible pair within the earlier $1/10$ centrality threshold.

The decrement also certifies the barrier objective gap. When $\lambda < 1$, the lower model and duality give $F_t(x+h) \geq F_t(x) - \lambda\|h\|_x + \omega(\|h\|_x)$. The minimum of $\omega(r) - \lambda r$ over $r \geq 0$ is $-\omega_*(\lambda)$, attained at $r = \lambda/(1-\lambda)$. Combining this with the damped-step decrease proves $\omega(\lambda) \leq F_t(x) - \min F_t \leq \omega_*(\lambda)$. In particular, the gap is $\Theta(\lambda^2)$ near the minimizer.

14.11.5 Quadratic Recentering with Full Newton Steps

Once the decrement is small, a full Newton step is feasible and its first-order error cancels. The resulting quadratic contraction is what makes one correction per path move enough.

Put $H = \nabla^2\Phi(x)$ and $\lambda = \lambda_t(x) < 1$. The identity $Hd = -g_t(x)$ cancels the linearized gradient at the endpoint:

$$\begin{aligned} g_t(x+d) &= g_t(x) + \int_0^1 \nabla^2\Phi(x+sd)d\,ds \\ &= g_t(x) + Hd + \int_0^1 \big(\nabla^2\Phi(x+sd) - H\big)d\,ds \\ &= \int_0^1 \big(\nabla^2\Phi(x+sd) - H\big)d\,ds. \end{aligned}$$

Only the error in treating the Hessian as constant remains. To measure that error in the old geometry, let $E_s = H^{-1/2}\nabla^2\Phi(x+sd)H^{-1/2} - I$. Hessian stability places its eigenvalues between $(1-s\lambda)^2 - 1$ and $(1-s\lambda)^{-2} - 1$. The latter has the larger absolute value, so $\|E_s\|_{\mathrm{op}} \leq (1-s\lambda)^{-2} - 1$. Also $\|H^{1/2}d\|_2 = \lambda$. Hence

$$\begin{aligned} \|g_t(x+d)\|_{x,*} &= \left\| \int_0^1 E_s H^{1/2} d\,ds \right\|_2 \\ &\leq \lambda \int_0^1 \left((1-s\lambda)^{-2} - 1\right) ds \\ &= \lambda\left(\frac{1}{\lambda}\left(\frac{1}{1-\lambda} - 1\right) - 1\right) = \frac{\lambda^2}{1-\lambda}. \end{aligned}$$

The case $\lambda = 0$ is already stationary. For $\lambda > 0$, the calculation above bounds the new gradient in the *old* dual norm. Finally, $\nabla^2\Phi(x+d) \succeq (1-\lambda)^2 H$ implies $\nabla^2\Phi(x+d)^{-1} \preceq (1-\lambda)^{-2}H^{-1}$. Taking square roots in the corresponding quadratic forms changes any dual norm by at most $1/(1-\lambda)$. Therefore

$$\lambda_t(x+d) \leq \frac{1}{1-\lambda}\|g_t(x+d)\|_{x,*} \leq \frac{\lambda^2}{(1-\lambda)^2}.$$

This is the general quadratic-recentering estimate. A sufficiently small constant decrement is restored by one Newton step after a small path move.

14.11.6 How Far to Move the Path

Let $\nu \geq 1$ bound $\|\nabla\Phi(x)\|_{x,*}^2$ everywhere. At fixed x, changing t changes only the linear part of the gradient, so

$$\lambda_{t'}(x) \leq \lambda_t(x) + (t' - t)\|c\|_{x,*}.$$

This gives a directly computable step size. Rather than always using a worst-case multiplicative change, increase t until this upper bound reaches a fixed small threshold β. From a supplied strictly feasible point, first call Algorithm 85 with $\tau = \beta/2$ at the initial parameter. This establishes the following algorithm's input invariant; its path-following bound counts the subsequent iterations.

Algorithm 86 Greedy path following with a self-concordant barrier

Input: $t > 0$, x with $\lambda_t(x) \leq \beta/2$, target $t_f > t$, $\beta = 1/8$

if $c = 0$ **then**
  return x
while $t < t_f$ **do**
  $H \leftarrow \nabla^2\Phi(x)$, $g \leftarrow tc + \nabla\Phi(x)$
  $\lambda \leftarrow \sqrt{g^\top H^{-1} g}$, $a \leftarrow \sqrt{c^\top H^{-1} c}$
  $t \leftarrow \min\{t_f, t + (\beta - \lambda)/a\}$
  $x \leftarrow x - H^{-1}(tc + \nabla\Phi(x))$ ▷ *Newton correction at the new t*
return x

The parameter update makes the pre-correction decrement at most $\beta = 1/8$. Quadratic recentering then bounds the new decrement by $\beta^2/(1-\beta)^2 = 1/49 < \beta/2$, restoring the input invariant. If equality constraints have been eliminated, the test $c = 0$ concerns the objective restricted to the feasible affine space.

Because $tc = -\nabla\Phi(x) + g_t(x)$, $t\|c\|_{x,*} \leq \sqrt{\nu} + \lambda_t(x)$. After recentering, $\lambda_t(x) \leq \beta/2$, so every nonfinal path move increases t by a relative amount at least $(\beta/2)/(\sqrt{\nu} + \beta/2)$. For initial parameter t_0, the iteration count is therefore $O(1 + \sqrt{\nu}\log(t_f/t_0))$. When $\|c\|_{x,*}$ is smaller than its worst-case bound, the same rule takes a larger step; no new convergence argument is needed.

Converting the path parameter to objective accuracy. First suppose x_t is the exact minimizer of $tc^\top x + \Phi(x)$. The parameter bound implies $\langle\nabla\Phi(x), u - x\rangle \leq \nu$ for every feasible u. Here is the scalar calculation. For interior u, put $\varphi(s) = \Phi(x + s(u - x))$. Duality and the parameter bound give

$$|\varphi'(s)| \leq \|\nabla\Phi(x + s(u-x))\|_{x+s(u-x),*}\|u - x\|_{x+s(u-x)} \leq \sqrt{\nu\varphi''(s)}.$$

Thus $\varphi''(s) \geq \varphi'(s)^2/\nu$. If $\varphi'(0) > \nu$, then φ' stays positive, and

$$\frac{d}{ds}\frac{1}{\varphi'(s)} = -\frac{\varphi''(s)}{\varphi'(s)^2} \leq -\frac{1}{\nu}.$$

Integration would give $1/\varphi'(s) \leq 1/\varphi'(0) - s/\nu$, forcing a singularity before $s = 1$. The segment to the interior point u permits no such singularity. Therefore $\varphi'(0) \leq \nu$; boundary u follow by limits. Since $tc = -\nabla\Phi(x_t)$ at a path point,

$$t(c^\top x_t - c^\top u) = \langle\nabla\Phi(x_t), u - x_t\rangle \leq \nu.$$

Taking an optimum u gives $c^\top x_t - \mathrm{OPT} \leq \nu/t$.

The algorithm keeps only a constant decrement, so we also need to compare its point x with x_t. Put $d = x_t - x$, $r = \|d\|_x$, and $\lambda = \lambda_t(x) < 1$. Along this segment, the reciprocal-curvature calculation gives $\|d\|_{x+sd} \geq r/(1 + sr)$ for $0 \leq s \leq 1$. Using $g_t(x_t) = 0$ and integrating the gradient along the segment,

$$\begin{aligned}\langle g_t(x), x - x_t\rangle &= \int_0^1 d^\top\nabla^2\Phi(x + sd)d\,ds \\ &\geq \int_0^1 \frac{r^2}{(1+sr)^2}\,ds = \frac{r^2}{1+r}.\end{aligned}$$

Duality bounds the same inner product above by λr. If $r > 0$, division and rearrangement give $r/(1+r) \le \lambda$, hence $r \le \lambda/(1-\lambda)$; the case $r = 0$ is immediate. Consequently

$$c^\top x - \mathrm{OPT} \le \frac{\nu}{t} + \|c\|_{x,*}\|x - x_t\|_x \le \frac{1}{t}\left(\nu + \frac{(\sqrt{\nu}+\lambda)\lambda}{1-\lambda}\right).$$

A fixed small decrement and $\nu \ge 1$ make this $O(\nu/t)$. Thus increasing t by a factor $1/\varepsilon$ reduces the guaranteed error bound by that factor, using $O(\sqrt{\nu}\log(1/\varepsilon))$ steps after initialization. The ordinary logarithmic barrier for m inequalities has parameter at most m; other barriers can give a parameter closer to the dimension.

14.12 Saddle Points and Mirror-Prox

For a differentiable convex–concave function Φ on $K = X \times Y$, we seek a pair with small saddle gap

$$\mathrm{gap}(x, y) = \sup_{v \in Y} \Phi(x, v) - \inf_{u \in X} \Phi(u, y).$$

This measures both players' possible improvements. Define the saddle operator $F(x, y) = (\nabla_x \Phi(x, y), -\nabla_y \Phi(x, y))$. Convexity in x and concavity in y give

$$\Phi(x, v) - \Phi(u, y) \le \langle F(x, y), (x, y) - (u, v)\rangle. \tag{37}$$

Indeed, bound $\Phi(x, y) - \Phi(u, y)$ by the first gradient and $\Phi(x, v) - \Phi(x, y)$ by the second, then add. Applying the inequality in both directions also proves monotonicity: $\langle F(z) - F(w), z - w\rangle \ge 0$.

14.12.1 Why Prediction Helps

Monotonicity alone does not make a forward step stable. For the bilinear game $\Phi(x, y) = xy$, the operator is $F(z) = Jz$ with $J = \left(\begin{smallmatrix} 0 & 1 \\ -1 & 0 \end{smallmatrix}\right)$. Since $J^\top = -J$ and $J^\top J = I$, ordinary descent–ascent gives $\|z - \eta Jz\|_2^2 = (1+\eta^2)\|z\|_2^2$. Each player reacts to the other's old position, and the iterates move away from the equilibrium.

Predict a new point first, then evaluate the operator there while correcting from the original base. In this example the two steps give $z^+ = z - \eta J(z - \eta Jz) = ((1-\eta^2)I - \eta J)z$. Its squared norm is $(1 - \eta^2 + \eta^4)\|z\|_2^2$, which contracts for $0 < \eta < 1$. The second evaluation creates a negative quadratic correction. Mirror-Prox makes this prediction and correction in Bregman geometry.

Algorithm 87 Mirror-Prox for a smooth convex–concave saddle problem

Input: Saddle operator F, reference h that is 1-strongly convex in $\|\cdot\|$, and $\|F(z) - F(w)\|_* \le L\|z - w\|$ with $L > 0$

Input: Initial $z_0 \in K$, $0 < \eta \le 1/L$, iteration count $T \ge 1$

$s \leftarrow 0$
for $t = 0, \dots, T-1$ **do**
  $z_t' \leftarrow \mathrm{Prox}_{z_t}^h(\eta F(z_t))$
  $z_{t+1} \leftarrow \mathrm{Prox}_{z_t}^h(\eta F(z_t'))$
  $s \leftarrow s + z_t'$
return $\bar{z} \leftarrow s/T$

The two steps share a base point; averaging uses the predictions. These choices make the following cancellation possible.[2]

Lemma 487 (Mirror-Prox comparison)**.** *One iteration satisfies $\eta\langle F(z_t'), z_t' - u\rangle \le D_h(u, z_t) - D_h(u, z_{t+1})$ for every $u \in K$.*

[2]Arkadi Nemirovski, *Prox-Method with Rate of Convergence $O(1/t)$ for Variational Inequalities with Lipschitz Continuous Monotone Operators and Smooth Convex–Concave Saddle Point Problems*, SIAM Journal on Optimization 15 (2004), 229–251.

Proof. Write $z = z_t$, $z' = z'_t$, and $z^+ = z_{t+1}$. Apply the proximal inequality to the prediction with comparator z^+ and to the correction with comparator u:

$$\eta\langle F(z), z' - z^+\rangle \le D_h(z^+, z) - D_h(z^+, z') - D_h(z', z),$$
$$\eta\langle F(z'), z^+ - u\rangle \le D_h(u, z) - D_h(u, z^+) - D_h(z^+, z).$$

The copies of $D_h(z^+, z)$ cancel. To obtain the desired left side, add $\eta\langle F(z') - F(z), z' - z^+\rangle$. This leaves

$$\eta\langle F(z'), z' - u\rangle \le D_h(u, z) - D_h(u, z^+) - D_h(z', z) - D_h(z^+, z') + \eta\langle F(z') - F(z), z' - z^+\rangle.$$

Lipschitzness, $\eta L \le 1$, and strong convexity give

$$\eta\langle F(z') - F(z), z' - z^+\rangle \le \|z' - z\|\,\|z' - z^+\| \le \tfrac{1}{2}\|z' - z\|^2 + \tfrac{1}{2}\|z' - z^+\|^2 \le D_h(z', z) + D_h(z^+, z').$$

The local terms cancel, proving the claim. □

Theorem 488 (Saddle-gap guarantee)**.** *If $\Omega = \sup_{u\in K} D_h(u, z_0) < \infty$, then the returned average satisfies $\mathrm{gap}(\bar{x}, \bar{y}) \le \Omega/(\eta T)$. In particular, $\eta = 1/L$ gives $L\Omega/T$.*

Proof. Sum the comparison inequality and discard the final divergence:

$$\frac{1}{T}\sum_{t<T}\langle F(z'_t), z'_t - u\rangle \le \frac{D_h(u, z_0)}{\eta T}. \tag{38}$$

For a fixed comparator (u, v), convexity–concavity and (37) give

$$\Phi(\bar{x}, v) - \Phi(u, \bar{y}) \le \frac{1}{T}\sum_{t<T}\big(\Phi(x'_t, v) - \Phi(u, y'_t)\big) \le \frac{D_h((u, v), z_0)}{\eta T}.$$

Take the supremum over (u, v) only after summing. The independent choices of u and v produce exactly the saddle gap. □

Mirror descent pays for the size of each loss vector; Mirror-Prox pays for the *change* in the operator between current and predicted points. This permits a constant step size. More generally, the proof only needs the interaction term to fit within the negative local divergences. A global norm-Lipschitz bound is one way to obtain this; the next section constructs curvature directly from the matrix interaction and implements the resulting steps in linear work.

14.13 Area Convexity and Fast Box–Simplex Games

Consider the saddle problem

$$\min_{x\in[-1,1]^n}\max_{y\in\Delta_d}\Phi(x, y), \qquad \Phi(x, y) = x^\top A y - b^\top y + c^\top x.$$

We seek a pair with small *saddle gap* $\mathrm{gap}(x, y) = \max_{v\in\Delta_d}\Phi(x, v) - \min_{u\in[-1,1]^n}\Phi(u, y)$. Its operator is $g(x, y) = (Ay + c,\, b - A^\top x)$: a descent step for x and an ascent step for y are both represented by subtracting g.

Its linear part $J = \left(\begin{smallmatrix} 0 & A \\ -A^\top & 0 \end{smallmatrix}\right)$ is skew-symmetric: this is the forward-step obstruction from Section 14.12. Ordinary Mirror-Prox resolves that obstruction, but its generic theorem does not choose the geometry or make the proximal steps cheap. Those are the remaining tasks here. We will derive a regularizer directly from the matrix interaction, then implement its steps with a constant number of matrix-vector products and entropy updates per outer iteration.

The output has a directly computable certificate. In the original units,

$$\operatorname{gap}(x,y) = \max_j (A^\top x - b)_j + c^\top x + \|Ay + c\|_1 + b^\top y.$$

The simplex maximization selects the largest coordinate, and the box minimization of $(Ay + c)^\top u$ equals $-\|Ay + c\|_1$. This formula also permits early termination when a running average has already reached the requested accuracy.

A uniform norm-based geometry faces a dimension cost on the cube: a reference function that is strongly convex in the box norm must have a large worst-case divergence.

Lemma 489 (Strong convexity on a cube costs dimension)**.** *If h is 1-strongly convex in ℓ_∞ on $[-1,1]^n$ and differentiable at zero, then* $\sup_{u\in[-1,1]^n} D_h(u,0) \ge n/2$.

Proof. Let $X_k = (\sigma_1, \dots, \sigma_k, 0, \dots, 0)$, where the signs are independent and uniform. Conditional on X_{k-1}, the two possible values of X_k are distance 2 apart in ℓ_∞ and have midpoint X_{k-1}. Strong convexity therefore gives $\mathbb{E}[h(X_k) \mid X_{k-1}] \ge h(X_{k-1}) + 1/2$. Summing yields $\mathbb{E}h(X_n) - h(0) \ge n/2$. Since $\mathbb{E}X_n = 0$, the linear term in the Bregman divergence averages to zero, so $\mathbb{E}D_h(X_n, 0) \ge n/2$. At least one vertex has divergence at least this average. □

Thus a bound that asks for uniform strong convexity in ℓ_∞ pays for all box coordinates. We instead choose curvature to control the interaction through A. The regularizer below will have initial divergence $O(1 + \log d)$ while satisfying exactly the inequality used by prediction and correction.

14.13.1 The Error Created by Prediction and Correction

Start with the exact-prox method below. Unlike ordinary Mirror-Prox, it uses a half-sized correction. This leaves an additional negative divergence in the proof, allowing a weaker geometric condition. Its role is to identify that condition; exact joint prox access is not yet an efficient implementation.

Algorithm 88 Relaxed mirror-prox with exact proximal steps

Input: Initial point z_0, regularizer r, step size $\eta > 0$, iteration count T

for $t = 0, \dots, T-1$ **do**
  $z'_t \leftarrow \operatorname{Prox}^r_{z_t}(\eta g(z_t))$ ▷ *Predict.*
  $z_{t+1} \leftarrow \operatorname{Prox}^r_{z_t}((\eta/2) g(z'_t))$ ▷ *Correct from the same base.*
return $T^{-1} \sum_{t=0}^{T-1} z'_t$

The prediction and correction use different operator values but the same base point. Write their three points as z, z', z^+. Applying the proximal inequality to the prediction with comparator z^+, and twice to the correction with comparator u, gives

$$\eta\langle g(z), z' - z^+\rangle \le D_r(z^+, z) - D_r(z^+, z') - D_r(z', z),$$
$$\eta\langle g(z'), z^+ - u\rangle \le 2D_r(u, z) - 2D_r(u, z^+) - 2D_r(z^+, z).$$

The factor $1/2$ in the correction leaves an extra negative copy of $D_r(z^+, z)$. Adding the two lines nearly gives the loss we want; the only missing term is the change in the operator between z and z':

$$\begin{aligned}\eta\langle g(z'), z' - u\rangle \le{}& 2D_r(u, z) - 2D_r(u, z^+) \\ &- D_r(z', z) - D_r(z^+, z') - D_r(z^+, z) \\ &+ \eta\langle g(z') - g(z), z' - z^+\rangle.\end{aligned}$$

This calculation tells us exactly which curvature condition to seek.

Definition 490 (Relaxed relative Lipschitzness)**.** The operator g is $1/\eta$-relaxed relatively Lipschitz with respect to r if every triple satisfies

$$\eta\langle g(z') - g(z), z' - z^+\rangle \le D_r(z', z) + D_r(z^+, z') + D_r(z^+, z).$$

Lemma 491 (Relaxed mirror-prox)**.** *Under this condition, the exact-prox method satisfies*

$$\eta\langle g(z'_t), z'_t - u\rangle \le 2D_r(u, z_t) - 2D_r(u, z_{t+1})$$

for every comparator u.

Proof. The condition cancels the three local divergences in the preceding calculation. Only the difference of global potentials remains. □

The condition is about an interaction between *two* displacements, not about curvature along one displacement. This is why it can control a skew operator even though $\langle Jv, v\rangle = 0$.

14.13.2 Choosing Curvature to Match the Matrix

Let $L = \|A\|_{1\to 1} = \max_j \sum_i |A_{ij}|$. For now normalize to $L \le 1$ and put $B = |A|$. All vector powers, quotients, and logarithms below are coordinatewise. For a positive $y \in \Delta_d$, weighted Cauchy–Schwarz gives

$$\begin{aligned} |u^\top A v| &\le \sum_{ij} B_{ij}|u_i v_j| \\ &\le \left(\sum_{ij} B_{ij} y_j u_i^2\right)^{1/2} \left(\sum_{ij} B_{ij}\frac{v_j^2}{y_j}\right)^{1/2} \le \sqrt{Q_x(u)Q_y(v)}, \end{aligned}$$

where $Q_x(u) = \sum_i (By)_i u_i^2$ and $Q_y(v) = \sum_j v_j^2/y_j$. Thus the coupling asks for box curvature proportional to By and simplex curvature proportional to $1/y$. With $h(y) = \sum_j y_j \log y_j$, these are supplied by

$$r_a(x, y) = \langle By, x^2\rangle + ah(y), \qquad a \ge 2.$$

The first term adjusts box curvature to the current simplex weights; the entropy term supplies the reciprocal curvature in the simplex variable. This avoids paying for every coordinate at its worst scale simultaneously. We will use r_5 for prediction and r_{10} for correction. The first coefficient leaves enough curvature to control the game; the second will absorb the error of implementing the prediction with finite work. The full algorithm will return gap ε in $O((\mathrm{nnz}(A) + n + d)(1 + L\log(2d)/\varepsilon))$ operations; the remaining proof must justify both this iteration count and the constant number of matrix products per iteration.

14.13.3 Area Convexity and a Local Test

For an affine skew operator, area convexity converts the preceding pointwise curvature calculation into the three-point inequality we need.

Definition 492 (Area convexity)**.** Let $g(z) = Jz + q_0$ with $J^\top = -J$. A convex function r is η-area convex if, for every z, z', z^+ and $c_\triangle = (z + z' + z^+)/3$,

$$\eta\langle g(z') - g(z), z' - z^+\rangle \le r(z) + r(z') + r(z^+) - 3r(c_\triangle).$$

The left side is the skew form on two sides of the triangle. Reversing its orientation changes the sign. The right side is the Jensen gap of r on the same three vertices. It therefore measures curvature on the triangle, rather than on just one of its edges. Expanding around z gives

$$r(z) + r(z') + r(z^+) - 3r(c_\triangle) = D_r(z', z) + D_r(z^+, z) - 3D_r(c_\triangle, z).$$

Since the last divergence is nonnegative, area convexity implies

$$\eta\langle g(z') - g(z), z' - z^+\rangle \le D_r(z', z) + D_r(z^+, z). \tag{39}$$

In particular, it implies relaxed relative Lipschitzness. The stronger two-divergence form will be convenient for the finite-step implementation.

To verify area convexity, it suffices to compare Hessians with J. Restrict the function to a fixed two-dimensional triangle. The Hessian condition gives a lower bound on its Laplacian, and a cubic vanishing on the three sides converts this into a Jensen-gap bound.

Lemma 493 (A function on a triangle)**.** *Let $\mathcal{T}$ have vertices $(-1/\sqrt{3}, 0)$, $(1/\sqrt{3}, 0)$, and $(0, 1)$. Suppose ψ is continuous on $\mathcal{T}$, C^2 in its interior, $\psi \le 0$ on the boundary, and $\Delta\psi \ge 2\alpha$ for $\alpha \ge 0$. Then $\psi(0, 1/3) \le -2\alpha/27$. In particular, this holds for convex ψ satisfying $\det \nabla^2\psi \ge \alpha^2$.*

Proof. The three sides have equations $t = 0$ and $t - 1 \pm \sqrt{3}s = 0$. Their product therefore gives a function vanishing on the whole boundary. Choose its scale and sign as

$$h(s, t) = -\frac{1}{2}t(t - 1 - \sqrt{3}s)(t - 1 + \sqrt{3}s) = -\frac{1}{2}t^3 + t^2 - \frac{1}{2}t + \frac{3}{2}ts^2.$$

The two second derivatives are $h_{ss} = 3t$ and $h_{tt} = -3t + 2$, so $\Delta h = 2$. At the centroid, $h(0, 1/3) = -2/27$. Thus $v = \psi - \alpha h$ has $\Delta v \ge 0$ and nonpositive boundary values. For completeness, consider $v_\epsilon = v + \epsilon(s^2 + t^2)$. Its Laplacian is strictly positive, whereas at an interior maximum the Hessian would be negative semidefinite and its trace nonpositive. Hence its maximum lies on the boundary and is at most $\epsilon \max_{\mathcal{T}}(s^2 + t^2)$. Letting $\epsilon \downarrow 0$ gives $v \le 0$, or $\psi \le \alpha h$. Evaluating at the centroid proves the asserted bound. Finally, a positive semidefinite Hessian satisfies $\Delta\psi = \operatorname{Tr}(\nabla^2\psi) \ge 2\sqrt{\det \nabla^2\psi}$, proving the last assertion. □

Lemma 494 (Block-Hessian criterion for area convexity)**.** *Let $g(z) = Jz + q_0$ with $J^\top = -J$, and let r be convex and C^2. If throughout its convex domain*

$$\begin{pmatrix} \nabla^2 r(z) & -J \\ J & \nabla^2 r(z) \end{pmatrix} \succeq 0,$$

then r is $1/(3\sqrt{3})$-area convex under the definition above.

Proof. Put $H = \nabla^2 r(z)$. The quadratic form of the block matrix on (a, b) is $a^\top Ha + b^\top Hb - a^\top Jb + b^\top Ja = a^\top Ha + b^\top Hb + 2\langle Ja, b\rangle$, where skew-symmetry gives $a^\top Jb = -b^\top Ja$. Testing also on $(a, -b)$ reverses the cross term. Nonnegativity of both forms therefore says

$$a^\top Ha + b^\top Hb \ge 2|\langle Ja, b\rangle|.$$

Fix three points. If their skew term is nonpositive, convexity proves the claim. Otherwise translate the midpoint of the first two points to zero, writing the vertices as $u, -u, w$. Their skew term is $\langle J(-2u), -u - w\rangle = 2\langle Ju, w\rangle > 0$. They are therefore noncollinear, so subtract an affine function agreeing with r at all three vertices. This changes neither the Hessian nor the Jensen gap and makes the three vertex values zero.

Pull back to $\mathcal{T}$ by $\psi(s, t) = r(\sqrt{3}su + tw)$. Its vertices map to $-u, u, w$, and its centroid maps to $w/3$. Since the vertex values vanish, convexity gives $\psi \le 0$ throughout the triangle. The chain rule identifies its Laplacian, at $z = \sqrt{3}su + tw$, as

$$\Delta\psi(s, t) = (\sqrt{3}u)^\top \nabla^2 r(z)(\sqrt{3}u) + w^\top \nabla^2 r(z) w \ge 2\sqrt{3}\langle Ju, w\rangle.$$

Set $\alpha = \sqrt{3}\langle Ju, w\rangle > 0$. The triangle lemma gives $r(w/3) \le -2\alpha/27$. Since all three vertex values are zero, their Jensen gap is $-3r(w/3)$, and

$$-3r(w/3) \ge \frac{2\alpha}{9} = \frac{2\langle Ju, w\rangle}{3\sqrt{3}} = \frac{\langle J(-2u), -u - w\rangle}{3\sqrt{3}}.$$

Undoing the affine normalization preserves this gap and proves the claim. Boundary points follow by continuity when derivatives are defined only in the relative interior. □

Verifying the chosen coefficients. For $D(y) = \operatorname{diag}(By)$,

$$\nabla^2 r_a(x,y) = \begin{pmatrix} 2D(y) & 2\operatorname{diag}(x)B \\ 2B^\top \operatorname{diag}(x) & a\operatorname{diag}(1/y) \end{pmatrix}.$$

Because $|x_i| \le 1$, the same Cauchy–Schwarz calculation bounds the Hessian cross term by $|\sum_{ij} B_{ij}x_iu_iv_j| \le \sqrt{Q_x(u)Q_y(v)}$. For $a = 2$, its full quadratic form is therefore at least $2(\sqrt{Q_x(u)} - \sqrt{Q_y(v)})^2$. Hence r_2 is jointly convex, as is r_a for every $a \ge 2$.

For the block criterion at $a = 5$, take two directions (u, v) and (s, t). Abbreviate $U = Q_x(u)$, $V = Q_y(v)$, $S = Q_x(s)$, and $T = Q_y(t)$. The two diagonal Hessian forms supply $2(U + S) + 5(V + T)$. Their internal cross terms have total magnitude at most $4\sqrt{UV} + 4\sqrt{ST}$. Also

$$\langle J(u,v), (s,t)\rangle = s^\top Av - u^\top At,$$

so the game cross term in the block form has magnitude at most $2\sqrt{SV} + 2\sqrt{UT}$. The whole block form is therefore at least

$$2(U + S) + 5(V + T) - 4\sqrt{UV} - 4\sqrt{ST} - 2\sqrt{SV} - 2\sqrt{UT}.$$

Put $X = \sqrt{U + S}$ and $Y = \sqrt{V + T}$. Two applications of Cauchy–Schwarz give $\sqrt{UV} + \sqrt{ST} \le XY$ and $\sqrt{SV} + \sqrt{UT} \le XY$. The lower bound is therefore at least

$$2X^2 + 5Y^2 - 6XY = 2(X - 3Y/2)^2 + Y^2/2 \ge 0.$$

Thus the block matrix is positive semidefinite, and r_5 is $1/(3\sqrt{3})$-area convex. More generally the same calculation works with any entropy coefficient at least $9/2$; choosing 5 keeps the later constants simple.

The constants separate two demands: coefficient 2 suffices for joint convexity, and the extra curvature up to 5 absorbs the skew interaction.

14.13.4 Deriving the Inner Updates

Fix a base point $z = (x, y)$ and linear form $v = (v_x, v_y)$, and consider

$$F_a(\widehat{x}, \widehat{y}) = \langle v, (\widehat{x}, \widehat{y})\rangle + D_{r_a}((\widehat{x}, \widehat{y}), z).$$

Up to a constant independent of $(\widehat{x}, \widehat{y})$, this is

$$\langle By', (x')^2\rangle + \langle \ell, x'\rangle + \langle v_y - B^\top x^2, y'\rangle + aD_h(y', y), \qquad \ell = v_x - 2x \odot (By),$$

where $(x', y') = (\widehat{x}, \widehat{y})$. For fixed $y' = q$, the box coordinates are independent quadratics. Their exact minimizer is

$$X(q)_i = \operatorname{clip}_{[-1,1]}\left(-\frac{\ell_i}{2(Bq)_i}\right).$$

If $(Bq)_i = 0$, minimize the remaining linear term by taking $X(q)_i = -\operatorname{sgn}(\ell_i)$, with $\operatorname{sgn}(0) = 0$. Define the reduced simplex objective $\psi(q) = F_a(X(q), q)$. Differentiating after this exact partial minimization gives

$$p(q) := \nabla\psi(q) = v_y + B^\top(X(q)^2 - x^2) + a\log(q/y). \tag{40}$$

For positive q, a nonzero row of B gives $(Bq)_i > 0$, so the box minimizer is unique, even if clipped at an endpoint. In the envelope calculation, the derivative with respect to q is the partial derivative of F_a with the optimizing box vector held fixed. More explicitly, for a direction d tangent to the simplex, compare $F_a(X(q + td), q + td)$ with $F_a(X(q), q + td)$ from above, and compare it with $F_a(X(q + td), q)$ from below. After division by t and passage to the limit, continuity of the unique clipped minimizer gives the same partial derivative on both sides. No derivative of $X(q)$ is required. Zero rows contribute a minimized linear term independent of q, and cause no difficulty. This justifies the displayed formula even where a box coordinate starts or stops being clipped.

The point of eliminating x is not just to reduce dimension. It exposes a well-conditioned problem in a geometry whose proximal step we already know.

Lemma 495 (Relative conditioning survives partial minimization)**.** *Suppose $F(x, y)$ is jointly convex on a product of convex sets, the minimum over x is attained for each y, and $\psi(y) = \min_x F(x, y)$ is differentiable. If $F(x, y) - Lh(y)$ is affine in y for every fixed x, then ψ is L-smooth relative to h. If $F(x, y) - \mu h(y)$ is jointly convex, then ψ is μ-strongly convex relative to h.*

Proof. For the upper bound, $Lh(y) - \psi(y) = \sup_x\{Lh(y) - F(x, y)\}$ is a supremum of affine functions of y, hence convex.

For the lower bound, write $G(x, y) = F(x, y) - \mu h(y)$ and take minimizers x_0, x_1 at y_0, y_1. For $0 \le \theta \le 1$, joint convexity gives

$$\begin{aligned}(\psi - \mu h)((1-\theta)y_0 + \theta y_1) &\le G((1-\theta)x_0 + \theta x_1, (1-\theta)y_0 + \theta y_1)\\ &\le (1-\theta)G(x_0, y_0) + \theta G(x_1, y_1)\\ &= (1-\theta)(\psi - \mu h)(y_0) + \theta(\psi - \mu h)(y_1).\end{aligned}$$

Thus $\psi - \mu h$ is convex as well. The shifted-convexity characterizations give the two relative curvature bounds. □

Here $F_a - ah$ is affine in the simplex variable, and $F_a - (a-2)h$ is jointly convex because its nonlinear part is r_2. Thus ψ is a-smooth and $(a-2)$-strongly convex relative to entropy. For $a = 5$, the relative condition number is $5/3$ and relative gradient descent contracts the Bregman error by at most $2/5$ per step.

This would permit a logarithmic-length inner solve, but the desired running time needs constant inner work. We should therefore ask for less than an accurate minimizer: only an inequality that can replace the prox inequality in the outer proof. At $\widehat{z} = (X(\widehat{y}), \widehat{y})$, box optimality and the three-point identity give

$$\begin{aligned}\langle v, \widehat{z} - u\rangle \le D_{r_a}(u, z) - D_{r_a}(u, \widehat{z}) - D_{r_a}(\widehat{z}, z)\\ + \langle p(\widehat{y}), \widehat{y} - u_y\rangle.\end{aligned} \tag{41}$$

Indeed, the box component of $\langle \nabla F_a(\widehat{z}), \widehat{z} - u\rangle$ is nonpositive; its simplex component is the last term above. Expanding $\nabla F_a = v + \nabla r_a(\widehat{z}) - \nabla r_a(z)$ gives the remaining three divergences. The entire error from an unfinished joint prox is now one linearized error for ψ.

14.13.5 The Complete Finite-Step Implementation

One entropy step $y^+ = \mathrm{EntropyStep}(q, p(q), a)$, followed by $X(y^+)$, will suffice for the prediction. For the correction, take one additional entropy step $q^+ = \mathrm{EntropyStep}(q, p(y^+), a)$ from the *same* q. Keep y^+ as the returned simplex iterate; carry q^+ as an auxiliary state. The next proof shows why this arrangement turns the unfinished inner optimization into a difference of potentials.

The first entropy update simplifies algebraically: $\log q - p(q)/a = \log y - [v_y + B^\top(X(q)^2 - x^2)]/a$. Thus it can be implemented from base y without explicitly forming $\log(q/y)$. The second update still uses base q.

Algorithm 89 Explicit box response and box–simplex step

function BoxResponse(B, ℓ, q)
  $w \leftarrow Bq$
  for $i = 1, \dots, n$ **do**
    if $w_i = 0$ **then**
      $u_i \leftarrow -\operatorname{sgn}(\ell_i)$
    else
      $u_i \leftarrow \max\{-1, \min\{1, -\ell_i/(2w_i)\}\}$
  return u

function BoxStep($B, z = (x, y), v = (v_x, v_y), q, a,$ refresh)
  $\ell \leftarrow v_x - 2x \odot (By)$
  $u \leftarrow$ BoxResponse(B, ℓ, q)
  $d_0 \leftarrow v_y + B^\top(u^2 - x^2)$
  $y^+ \leftarrow$ EntropyStep(y, d_0, a) ▷ *Equivalent to a step from q using $p(q)$.*
  $x^+ \leftarrow$ BoxResponse(B, ℓ, y^+)
  $q^+ \leftarrow q$
  if refresh **then**
    $d_1 \leftarrow B^\top((x^+)^2 - u^2)$ ▷ *Equals $p(y^+)$ up to a constant vector*
    $q^+ \leftarrow$ EntropyStep(q, d_1, a) ▷ *Use q, not y^+.*
  return $((x^+, y^+), q^+)$

Here EntropyStep is the explicit normalization routine in Algorithm 74. All simplex bases start positive and remain positive in exact arithmetic. The refresh needs no logarithm ratio. Since $a \log(y^+/y) = -d_0 - a \log Z\, \mathbf{1}$ for a normalization constant Z, we have $p(y^+) = B^\top((x^+)^2 - u^2) - a \log Z\, \mathbf{1}$. Adding a constant vector to an entropy-step loss does not change its output, which justifies the simplified d_1 in the pseudocode. Both outer calls below use the same current point z. The prediction uses the current simplex iterate y as its inner reference; the correction uses the auxiliary q retained from the previous round.

Algorithm 90 Full linear-work box–simplex solver

Input: $A \in \mathbb{R}^{n \times d}$, $b \in \mathbb{R}^d$, $c \in \mathbb{R}^n$, additive gap target $\varepsilon > 0$
Output: A feasible pair $(\bar{x}, \bar{y})$ with $\operatorname{gap}(\bar{x}, \bar{y}) \le \varepsilon$

$L \leftarrow \max_j \sum_i |A_{ij}|$
if $L = 0$ **then**
  return $(-\operatorname{sgn}(c), e_j)$, where $j \in \arg\min_i b_i$
$(A, b, c) \leftarrow (A/L, b/L, c/L)$; $B \leftarrow |A|$; $\eta \leftarrow 1/(3\sqrt{3})$
$T \leftarrow \lceil (2 + 40 \log d) L/(\eta\varepsilon) \rceil$
$(x, y) \leftarrow (0, \mathbf{1}/d)$; $q \leftarrow \mathbf{1}/d$; $(s_x, s_y) \leftarrow (0, 0)$
for $t = 0, \dots, T - 1$ **do**
  $z \leftarrow (x, y)$; $v \leftarrow \eta(Ay + c,\ b - A^\top x)$
  $(z' = (x', y'), \widehat{q}) \leftarrow$ BoxStep$(B, z, v, y, 5,$ false$)$
  $(s_x, s_y) \leftarrow (s_x + x', s_y + y')$
  $w \leftarrow (\eta/2)(Ay' + c,\ b - A^\top x')$
  $(z^+, q^+) \leftarrow$ BoxStep$(B, z, w, q, 10,$ true$)$
  $(x, y) \leftarrow z^+$; $q \leftarrow q^+$
return $(s_x/T, s_y/T)$

The coefficients 5 and 10 are not accuracy tolerances. The prediction produces an extra entropy term, which doubling the correction's entropy coefficient can absorb. The separate state q makes the remaining correction error telescope. This finite-step implementation follows the asymmetric gradient and extragradient

constructions of Jambulapati and Tian.[3]

14.13.6 Why the Inner Errors Have the Required Form

Only convexity and relative smoothness of the reduced function are needed for the two guarantees. For any q, q', u, the three-point identity gives

$$\begin{aligned}\langle p(q') - p(q), q' - u\rangle &= D_\psi(u, q') + D_\psi(q', q) - D_\psi(u, q)\\ &\le a\big(D_h(u, q') + D_h(q', q)\big).\end{aligned} \tag{42}$$

It bounds the error from using the gradient before, rather than after, an entropy step. The two terms are exactly the local divergences that entropy optimality supplies with negative signs.

Prediction: absorb one extra entropy term. Start at $q = y$, take $y' = \mathrm{EntropyStep}(y, p(y), a)$, and let $z' = (X(y'), y')$. The entropy proximal inequality is

$$\langle p(y), y' - u_y\rangle \le a\big(D_h(u_y, y) - D_h(u_y, y') - D_h(y', y)\big).$$

Add (42) with $(q, q', u) = (y, y', u_y)$. The negative local terms cancel, leaving $\langle p(y'), y' - u_y\rangle \le aD_h(u_y, y)$. Substitution in (41) yields

$$\langle v, z' - u\rangle \le D_{r_{2a}}(u, z) - D_{r_a}(u, z') - D_{r_a}(z', z). \tag{43}$$

The extra error is precisely one additional entropy divergence at the base, not an unspecified additive tolerance.

Correction: pass the remaining error to an auxiliary state. For an arbitrary positive reference q, take $y^+ = \mathrm{EntropyStep}(q, p(q), a)$ and $q^+ = \mathrm{EntropyStep}(q, p(y^+), a)$. The two entropy proximal inequalities are

$$\begin{aligned}\langle p(q), y^+ - q^+\rangle &\le a\big(D_h(q^+, q) - D_h(q^+, y^+) - D_h(y^+, q)\big),\\ \langle p(y^+), q^+ - u_y\rangle &\le a\big(D_h(u_y, q) - D_h(u_y, q^+) - D_h(q^+, q)\big).\end{aligned}$$

They share the same base q, which is why $D_h(q^+, q)$ cancels. To obtain the desired left side, use

$$\begin{aligned}\langle p(y^+), y^+ - u_y\rangle = \langle p(q), y^+ - q^+\rangle &+ \langle p(y^+), q^+ - u_y\rangle\\ &+ \langle p(y^+) - p(q), y^+ - q^+\rangle.\end{aligned}$$

Substituting the proximal bounds yields

$$\begin{aligned}\langle p(y^+), y^+ - u_y\rangle \le\ & a\big(D_h(u_y, q) - D_h(u_y, q^+)\big)\\ &- aD_h(q^+, y^+) - aD_h(y^+, q)\\ &+ \langle p(y^+) - p(q), y^+ - q^+\rangle.\end{aligned}$$

Both steps start at q, so the copies of $D_h(q^+, q)$ have canceled. Equation (42) absorbs the last line into the two remaining negative local divergences. With $z^+ = (X(y^+), y^+)$, (41) now gives

$$\begin{aligned}\langle v, z^+ - u\rangle \le\ & D_{r_a}(u, z) - D_{r_a}(u, z^+) - D_{r_a}(z^+, z)\\ &+ a\big(D_h(u_y, q) - D_h(u_y, q^+)\big).\end{aligned} \tag{44}$$

This is the reason to retain q^+ between iterations. The inner error need not be small in isolation; it is a difference of nonnegative potentials and therefore has bounded cumulative cost.

[3] Arun Jambulapati and Kevin Tian, *Revisiting Area Convexity: Faster Box-Simplex Games and Spectrahedral Generalizations*, arXiv:2303.15627.

14.13.7 The Outer Telescope and Running Time

The two contracts determine the global potential:

$$\mathcal{P}_u(z,q) = 2D_{r_{10}}(u,z) + 20D_h(u_y,q).$$

Use (43) for the prediction with $a=5$, $v=\eta g(z)$, and comparator z^+. Use twice (44) for the correction with $a=10$, $v=(\eta/2)g(z')$, and comparator u. Adding gives

$$\begin{aligned}\eta\langle g(z'), z'-u\rangle \le{}& \mathcal{P}_u(z,q) - \mathcal{P}_u(z^+,q^+)\\ &- D_{r_5}(z',z) - D_{r_5}(z^+,z') - D_{r_{10}}(z^+,z)\\ &+ \eta\langle g(z')-g(z), z'-z^+\rangle.\end{aligned}$$

Area convexity bounds the interaction by $D_{r_5}(z',z)+D_{r_5}(z^+,z)$. After substitution, the remaining local terms are exactly

$$-D_{r_5}(z^+,z') - \big(D_{r_{10}}(z^+,z) - D_{r_5}(z^+,z)\big) = -D_{r_5}(z^+,z') - 5D_h(y^+,y) \le 0.$$

Thus one outer iteration decreases $\mathcal{P}_u$ by at least the linearized comparison loss. Summing and using nonnegativity of the final potential gives

$$\eta\sum_{t=0}^{T-1}\langle g(z_t'), z_t'-u\rangle \le \mathcal{P}_u(z_0,q_0) - \mathcal{P}_u(z_T,q_T) \le \mathcal{P}_u(z_0,q_0).$$

To bound the initial value, put $y_0=q_0=\mathbf{1}/d$. At $z_0=(0,y_0)$, the quadratic part has both value and gradient zero. Hence

$$\begin{aligned}D_{r_{10}}(u,z_0) &= \langle Bu_y, u_x^2\rangle + 10D_h(u_y,y_0),\\ \mathcal{P}_u(z_0,q_0) &= 2\langle Bu_y,u_x^2\rangle + 40D_h(u_y,\mathbf{1}/d).\end{aligned}$$

Since $|u_{x,i}|\le 1$, $u_y\in\Delta_d$, and each column sum of B is at most one, $\langle Bu_y,u_x^2\rangle \le \sum_j u_{y,j}\sum_i B_{ij} \le 1$. Also $D_h(u_y,\mathbf{1}/d)\le \log d$. Therefore $\mathcal{P}_u(z_0,q_0)\le 2+40\log d$.

For fixed u, the function $z\mapsto\langle g(z), z-u\rangle$ is affine: its quadratic term is $z^\top Jz=0$. Also $\langle g(x,y),(x,y)-(u_x,u_y)\rangle = \Phi(x,u_y)-\Phi(u_x,y)$. Averaging the *prediction* points and maximizing over u thus gives

$$\operatorname{gap}(\bar{x},\bar{y}) \le \frac{2+40\log d}{\eta T}$$

for the normalized game. Multiplication by L gives the accuracy asserted by the algorithm.

Theorem 496 (Fast box–simplex games)**.** *Algorithm 90 returns a pair of additive saddle gap at most ε in arithmetic time*

$$O\left((\operatorname{nnz}(A)+n+d)\left(1+\frac{\|A\|_{1\to1}\log(2d)}{\varepsilon}\right)\right).$$

When input and output work is absorbed by $\operatorname{nnz}(A)$ *and $\varepsilon\le\|A\|_{1\to1}$, this is the usual $O(\operatorname{nnz}(A)L\log(2d)/\varepsilon)$ bound.*

Each outer iteration uses only a constant number of products by A, B, and their transposes, plus clipping and entropy updates. The geometric choice has solved both difficulties: it controls the prediction error, and it makes the unfinished inner optimization produce an error that can be carried in a telescoping potential. Constant inner work follows from that second property, not from the inner condition number alone.

14.14 Online Discrepancy by the Gaussian Triplet Walk

Online vector balancing asks for irrevocable sign choices rather than an approximate minimum. It provides a different use of an iterative invariant: maintain prescribed marginal distributions, then obtain a concentration bound from those marginals.

Let $v_1, \dots, v_T \in \mathbb{R}^d$ be a fixed sequence with $\|v_t\|_2 \le 1$, revealed one vector at a time. After seeing v_t, an online algorithm must output $\varepsilon_t \in \{-1, 1\}$ before seeing v_{t+1}. We want every partial sum $S_t = \sum_{i \le t} \varepsilon_i v_i$ to be small in ℓ_∞. The sequence is fixed independently of the algorithm's randomness; the argument below is not a guarantee against an adversary adapting to past signs.

Instead of first choosing a signing rule and then trying to prove that its partial sums concentrate, prescribe the distributional guarantee we would like to maintain. Represent the discrepancy by three auxiliary states with

$$S_t = G_{t,1} + G_{t,2} + G_{t,3}, \qquad G_{t,j} \sim N(0, I_d) \quad \text{for each } j.$$

A sum of three variables with Gaussian marginals has constant subgaussian tails even when they are dependent; Hölder's inequality will prove this below. We therefore do not need to preserve their joint distribution. Their dependence is free to change so that the sum records the signs we have already chosen. This separates two requirements: each auxiliary transition must preserve a Gaussian marginal, while their coupling must make the total transition an actual sign.[4]

Theorem 497 (Online discrepancy in linear time)**.** *For every fixed sequence of $T \ge 1$ vectors in the Euclidean unit ball and every $0 < \delta < 1$, the Gaussian triplet algorithm runs in $O(dT)$ arithmetic time and uses $O(d)$ working space. With probability at least $1 - \delta$, it satisfies*

$$\max_{1 \le t \le T} \left\| \sum_{i=1}^{t} \varepsilon_i v_i \right\|_\infty \le \sqrt{18 \log \frac{2T^2}{\delta}}.$$

For only the final sum, the same algorithm satisfies

$$\left\| \sum_{t=1}^{T} \varepsilon_t v_t \right\|_\infty \le \sqrt{18 \log \frac{2 \min\{d, T\}}{\delta}}$$

with probability at least $1 - \delta$. For sparse coordinate-list inputs, the arithmetic time is $O(d + T + \sum_t \operatorname{nnz}(v_t))$.

14.14.1 The Full Triplet Algorithm

A single state is allowed a zero move as well as $+v$ and $-v$. This freedom makes an invariant transition easy to design. Let γ be the standard Gaussian density. To preserve it, require equal probability mass in the two directions between x and $x + v$: $\gamma(x) p_v^+(x) = \gamma(x + v) p_v^-(x + v)$. Choose that common mass to be $\frac{1}{3} \min\{\gamma(x), \gamma(x + v)\}$. Dividing by the departure density gives

$$p_v^{\pm}(x) = \frac{1}{3} \exp\left(\min\left\{ 0, \mp\langle x, v \rangle - \frac{\|v\|_2^2}{2} \right\} \right), \qquad p_v^0(x) = 1 - p_v^+(x) - p_v^-(x).$$

The remaining probability is assigned to staying still. The density ratio produces the exponent. Stationarity itself would allow other prefactors; $1/3$ ensures the balance conditions needed by the coupling. The coupling below preserves each state's two sign probabilities but makes the three moves sum to $+1$ or -1, never zero.

We also need three Gaussian states whose sum starts at zero. Draw independent $G, H \sim N(0, I_d)$ and try the symmetric form $G_1 = G$, $G_2 = -G/2 + aH$, $G_3 = -G/2 - aH$. Their sum is zero for every a; requiring unit marginal covariance gives $1/4 + a^2 = 1$, hence $a = \sqrt{3}/2$. Both choices are now explicit in the algorithm.

[4] Ishaq Aden-Ali, *Optimal Online Discrepancy Minimization in Linear Time*, arXiv:2607.04388.

Algorithm 91 Gaussian triplet walk for online discrepancy

Input: A fixed sequence, revealed online, of vectors $v_t \in \mathbb{R}^d$ with $\|v_t\|_2 \le 1$

Draw independent $G, H \sim N(0, I_d)$
$G_1 \leftarrow G$, $G_2 \leftarrow -G/2 + \sqrt{3}H/2$, $G_3 \leftarrow -G/2 - \sqrt{3}H/2$
$S \leftarrow 0$ ▷ *Initially* $G_1 + G_2 + G_3 = 0$
for each arriving vector v **do**
  $r \leftarrow \|v\|_2^2/2$
  for $j = 1, 2, 3$ **do**
    $z_j \leftarrow \langle G_j, v\rangle$
    $a_j \leftarrow \frac{1}{3}e^{\min\{0, -z_j - r\}}$, $b_j \leftarrow \frac{1}{3}e^{\min\{0, z_j - r\}}$
  $(\delta_1, \delta_2, \delta_3) \leftarrow$ CoupleThree(a, b) ▷ *Algorithm 92*
  $\varepsilon \leftarrow \delta_1 + \delta_2 + \delta_3$
  Output ε before requesting the next vector
  $G_j \leftarrow G_j + \delta_j v$ for $j = 1, 2, 3$; $S \leftarrow S + \varepsilon v$

The initialization is jointly Gaussian, but subsequent triplets need not be. The proof will keep only the three marginal laws and the exact sum identity. The transition and coupling lemmas below justify these two invariants separately.

14.14.2 Why the Fractional Walk Preserves a Gaussian

Lemma 498 (Gaussian fixed point)**.** *For fixed v, if $G \sim N(0, I_d)$ and, conditional on $G = x$, the move $\delta \in \{-1, 0, 1\}$ has probabilities $p_v^-(x), p_v^0(x), p_v^+(x)$, then $G + \delta v \sim N(0, I_d)$.*

Proof. The exponent is the change in the Gaussian quadratic:

$$\frac{\gamma(x+v)}{\gamma(x)} = \exp\left(-\frac{\|x+v\|_2^2 - \|x\|_2^2}{2}\right) = \exp\left(-\langle x, v\rangle - \frac{\|v\|_2^2}{2}\right).$$

Thus $p_v^+(x) = \frac{1}{3}\min\{1, \gamma(x+v)/\gamma(x)\}$. Multiplication by the departure density gives

$$\gamma(x)p_v^+(x) = \tfrac{1}{3}\min\{\gamma(x), \gamma(x+v)\} = \gamma(x+v)p_v^-(x+v).$$

This is the detailed-balance condition used to design the probabilities. To check that it really preserves the whole distribution, add the mass arriving from $x - v$, from $x + v$, and from the zero move. The resulting density is

$$\begin{aligned}&\gamma(x-v)p_v^+(x-v) + \gamma(x+v)p_v^-(x+v) + \gamma(x)p_v^0(x)\\ &\quad = \gamma(x)\big(p_v^-(x) + p_v^+(x) + p_v^0(x)\big) = \gamma(x).\end{aligned}$$

□

Lemma 499 (Balance)**.** *If $\|v\|_2 \le 1$, then $p_v^+(x), p_v^-(x) \le 1/3$ and $p_v^+(x) + p_v^-(x) \ge 1/3$ for every x.*

Proof. The upper bounds are immediate. Put $z = \langle x, v\rangle$ and $r = \|v\|_2^2/2 \le 1/2$. If $z \ge r$ or $z \le -r$, one of the two probabilities is $1/3$. Otherwise neither exponential is truncated, and

$$p_v^+(x) + p_v^-(x) = \tfrac{2}{3}e^{-r}\cosh z \ge \tfrac{2}{3}e^{-1/2} > \tfrac{1}{3}.$$

□

The factor $1/3$ makes each individual sign probability small enough for coupling, while the norm bound ensures enough total movement. Both inequalities matter: preserving a Gaussian alone would not guarantee that three transitions can be combined into a genuine sign.

14.14.3 Why the Coupling Has Two Branches

Suppose move j must have positive probability a_j and negative probability b_j, where $0 \le a_j, b_j \le 1/3$ and $a_j + b_j \ge 1/3$. For three entries in $\{-1, 0, 1\}$ to sum to ± 1, the number of nonzero entries must be odd: either one or three. This parity condition determines the structure of the coupling.

The prescribed expected number of nonzero entries is $s = \sum_j (a_j + b_j) \in [1, 2]$. If we use three nonzero entries with probability τ and one otherwise, then $s = 3\tau + (1 - \tau)$, so necessarily $\tau = (s - 1)/2$. In the three-entry branch we must avoid all-positive and all-negative signs. Thus there must be one or two positive entries.

It remains to distribute the positive and negative probabilities between the two branches. Let x_j be the *unconditional* positive probability allocated to coordinate j in the three-entry branch. Because that branch occurs with probability τ and always moves coordinate j, its negative allocation is $\tau - x_j$. Neither allocation may exceed the prescribed marginal, and both must be nonnegative. Equivalently,

$$\max\{0, \tau - b_j\} \le x_j \le \min\{\tau, a_j\}.$$

To permit one or two positive signs there, the total positive allocation must lie between τ and 2τ. We first choose x_j in these intervals with $\tau \le \sum_j x_j \le 2\tau$. When $\tau > 0$, the conditional positive probabilities are $q_j = x_j/\tau$; when $\tau = 0$, this branch is absent.

The last step is to realize those q_j without ever selecting zero or three positives. Place intervals of lengths q_1, q_2, q_3 consecutively on $[0, Q)$, where $1 \le Q \le 2$. A uniform $U \in [0, 1)$ hits exactly one interval, and $U + 1$ hits at most one more. Each interval has length at most one, so it cannot be hit twice. Its probability of being hit is exactly its length. This gives the desired one-or-two-positive coupling with only one uniform random number.

Algorithm 92 Coupling three prescribed fractional moves into one sign

Input: $a, b \in [0, 1/3]^3$ with $a_j + b_j \ge 1/3$ for every j
Output: The prescribed sign marginals and $\delta_1 + \delta_2 + \delta_3 \in \{-1, 1\}$

function CoupleThree(a, b)
  $\tau \leftarrow (\sum_{j=1}^{3}(a_j + b_j) - 1)/2$
  $\ell_j \leftarrow \max\{0, \tau - b_j\}$, $u_j \leftarrow \min\{\tau, a_j\}$, $x_j \leftarrow \ell_j$ for $j = 1, 2, 3$
  $D \leftarrow \max\{0, \tau - \sum_j x_j\}$
  for $j = 1, 2, 3$ **do**
    $t \leftarrow \min\{D, u_j - x_j\}$; $x_j \leftarrow x_j + t$; $D \leftarrow D - t$
  Draw $B_0 \sim$ Bernoulli(τ)
  if $B_0 = 1$ **then** ▷ *Three nonzero entries; this branch is absent if* $\tau = 0$
    $q_j \leftarrow x_j/\tau$; $c_0 \leftarrow 0$, $c_j \leftarrow \sum_{i=1}^{j} q_i$
    Draw U uniformly from $[0, 1)$
    for $j = 1, 2, 3$ **do**
      $Z_j \leftarrow \mathbf{1}[U \in [c_{j-1}, c_j)] + \mathbf{1}[U + 1 \in [c_{j-1}, c_j)]$
      $\delta_j \leftarrow 2Z_j - 1$
  else ▷ *Exactly one nonzero entry*
    Sample δ from $\{e_j, -e_j : j = 1, 2, 3\}$ with probabilities
      $\Pr[\delta = e_j] = (a_j - x_j)/(1 - \tau)$,
      $\Pr[\delta = -e_j] = (b_j - \tau + x_j)/(1 - \tau)$
  return $(\delta_1, \delta_2, \delta_3)$

Lemma 500 (Three-way coupling)**.** *Under the stated conditions on a_j, b_j, Algorithm 92 runs in constant time, has marginals $\Pr[\delta_j = 1] = a_j$ and $\Pr[\delta_j = -1] = b_j$, and always satisfies $\sum_j \delta_j \in \{-1, 1\}$.*

Proof. We first check that the probability allocation in the algorithm can be completed. Write $s_j = a_j + b_j \in [1/3, 2/3]$. Since $\tau = (s_1 + s_2 + s_3 - 1)/2$, we have $0 \le \tau \le 1/2$. For any coordinate j, the other two values

sum to at most 4/3, so

$$2\tau = \sum_i s_i - 1 \le s_j + \frac{1}{3} \le 2s_j.$$

Thus $\tau \le a_j + b_j$. This says exactly that the two signs in the three-entry branch can fit within coordinate j's total prescribed movement probability: $\tau - b_j \le a_j$, making $[\ell_j, u_j] = [\max\{0, \tau - b_j\}, \min\{\tau, a_j\}]$ nonempty.

We also need the coordinate intervals to allow a total allocation between τ and 2τ. Put $A_+ = \sum_j a_j$ and $A_- = \sum_j b_j$. Each is at most one, while $A_+ + A_- = 1 + 2\tau$, so both are at least 2τ. Consequently

$$\sum_j \min\{\tau, a_j\} \ge \tau, \qquad \sum_j \min\{\tau, b_j\} \ge \tau.$$

For either inequality, if one summand reaches τ it is immediate; otherwise no truncation occurs and the sum is at least 2τ. Therefore the upper endpoints sum to at least τ, and the lower endpoints satisfy

$$\sum_j \ell_j = \sum_j \big(\tau - \min\{\tau, b_j\}\big) = 3\tau - \sum_j \min\{\tau, b_j\} \le 2\tau.$$

Starting at the lower endpoints, increase the coordinates only until their sum reaches τ, if necessary. The upper-endpoint bound guarantees that this is possible. The final sum is $\max\{\tau, \sum_j \ell_j\} \in [\tau, 2\tau]$, and the remaining demand D in the algorithm is zero.

In the three-entry branch, $0 \le q_j \le 1$ and $1 \le Q = \sum_j q_j \le 2$. The intervals $[c_{j-1}, c_j)$ partition $[0, Q)$, so U lies in exactly one, and $U + 1$ lies in either zero or one. No interval contains both, since its length is at most 1. Thus each Z_j is an indicator and their sum is one or two. For its marginal, let $I_j = [c_{j-1}, c_j) \subseteq [0, 2)$. The two indicators have expectations $|I_j \cap [0, 1)|$ and $|I_j \cap [1, 2)|$, where $|\cdot|$ is interval length. They cannot both equal one, so

$$\Pr[Z_j = 1] = \mathbb{E}Z_j = |I_j \cap [0, 1)| + |I_j \cap [1, 2)| = |I_j| = q_j.$$

Since $\delta_j = 2Z_j - 1$ and $\sum_j Z_j \in \{1, 2\}$, the sign sum is $2\sum_j Z_j - 3 \in \{-1, 1\}$. Multiplying the conditional positive probability $q_j = x_j/\tau$ by the branch probability τ gives unconditional probability x_j; the unconditional negative probability is $\tau(1 - q_j) = \tau - x_j$.

In the one-entry branch, the interval constraints ensure that the remaining positive and negative masses, $a_j - x_j$ and $b_j - \tau + x_j$, are nonnegative. Their total unconditional mass is

$$\sum_j \big((a_j - x_j) + (b_j - \tau + x_j)\big) = \sum_j (a_j + b_j) - 3\tau = (1 + 2\tau) - 3\tau = 1 - \tau.$$

Dividing by $1 - \tau > 0$ therefore defines a probability distribution. Its outcomes are $\pm e_j$, so its sign sum is also ± 1. Combining both branches, the positive marginal at coordinate j is $x_j + (a_j - x_j) = a_j$ and the negative marginal is $(\tau - x_j) + (b_j - \tau + x_j) = b_j$. The zero marginal is the remaining $1 - a_j - b_j$. When $\tau = 0$, the first branch is absent and the same calculation applies with $x_j = 0$, so no division by zero is needed. □

Three states are useful because their total movement can vary between one and three while retaining an odd sign sum. With only two states, a sum of ± 1 would require exactly one nonzero move every time, which need not match the prescribed movement probabilities. The coupling uses dependence to meet this parity constraint without disturbing any individual transition.

14.14.4 The Invariant and Its Concentration Consequence

Lemma 501 (Triplet invariant)**.** *For every t, the triplet algorithm satisfies $S_t = G_{t,1} + G_{t,2} + G_{t,3}$ and each $G_{t,j}$ has marginal law $N(0, I_d)$.*

Proof. The sum identity holds initially. At round t, the sum of the auxiliary states changes by $\sum_j \delta_{t,j} v_t = (\sum_j \delta_{t,j}) v_t = \varepsilon_t v_t$, the same change as S_t.

For the marginal law, let $K_v\varphi(x)$ denote the conditional expectation of a bounded test function φ after one fractional move from x with vector v. Conditional on the *whole* current triplet, the coupling preserves the prescribed transition of coordinate j:

$$\mathbb{E}[\varphi(G_{t,j}) \mid G_{t-1,1}, G_{t-1,2}, G_{t-1,3}] = K_{v_t}\varphi(G_{t-1,j}).$$

Taking expectations and using the inductive Gaussian marginal,

$$\mathbb{E}\varphi(G_{t,j}) = \mathbb{E}K_{v_t}\varphi(G_{t-1,j}) = \mathbb{E}_{G\sim N(0,I_d)}K_{v_t}\varphi(G) = \mathbb{E}_{G\sim N(0,I_d)}\varphi(G).$$

The last equality is the Gaussian fixed-point lemma for the fixed vector v_t. Thus $G_{t,j}$ is marginally standard Gaussian, without any independence assumption between the three states. □

The Gaussian assertion is unconditional, not conditional on all previously output signs. That is enough. Fix t, let $E_t = \operatorname{span}(v_1, \dots, v_t)$, and let P_t be its orthogonal projection. The subspace is deterministic because the input sequence is fixed, and $S_t \in E_t$. Independence would give variance 3 for the sum in a unit direction; three identical Gaussian states would instead give variance 9. Hölder recovers the latter bound without assuming anything about their dependence:

$$\begin{aligned}\mathbb{E}e^{\langle S_t,\theta\rangle} &= \mathbb{E}\prod_{j=1}^{3} e^{\langle G_{t,j},P_t\theta\rangle} \\ &\le \prod_{j=1}^{3}\left(\mathbb{E}e^{3\langle G_{t,j},P_t\theta\rangle}\right)^{1/3} = \exp\left(\frac{9}{2}\|P_t\theta\|_2^2\right).\end{aligned}$$

Thus the sum has a constant subgaussian parameter even though its summands are dependent. Keeping the projection is important: it will replace a union bound over all d coordinates by the dimension actually used by the input vectors.

14.14.5 Why the Ambient Dimension Disappears

Proof of Theorem 497. For coordinate k, let $a_k = \|P_te_k\|_2^2$. Apply the moment bound with $\theta = \lambda e_k$: exponential Markov gives $\Pr[S_{t,k} > u] \le \exp(-\lambda u + 9\lambda^2 a_k/2)$. For $a_k > 0$, minimize at $\lambda = u/(9a_k)$ and apply the same bound to the negative tail. This yields

$$\Pr[|S_{t,k}| > u] \le 2\exp\left(-\frac{u^2}{18a_k}\right) \quad \text{when } a_k > 0.$$

If $a_k = 0$, that coordinate vanishes identically. Although there may be many nonzero a_k, they satisfy $0 \le a_k \le 1$ and $\sum_k a_k = \operatorname{Tr}(P_t) = \dim E_t \le \min\{d, t\}$. The quantities a_k are the diagonal entries of the projection, just as leverage scores are diagonal entries of a projection in the sparsification argument. They measure how strongly each coordinate sees the reached subspace. Counting every coordinate equally would lose this information; the tail calculation should instead charge coordinate k proportionally to a_k.

The following elementary inequality performs the weighted union bound: for $x \ge 1$ and $0 < a \le 1$, $e^{-x/a} \le ae^{-x}$. To verify it, set $a = e^{-y}$ with $y \ge 0$. The logarithm of the ratio of its left side to its right side is

$$-\frac{x}{a} - \log a + x = -x(e^y - 1) + y \le 0,$$

since $e^y - 1 \ge y$ and $x \ge 1$. With $x = u^2/18$, the coordinate tail estimates therefore give

$$\begin{aligned}\Pr[\|S_t\|_\infty > u] &\le 2\sum_{k:a_k>0}\exp\left(-\frac{u^2}{18a_k}\right) \\ &\le 2e^{-u^2/18}\sum_k a_k = 2\dim(E_t)e^{-u^2/18} \le 2te^{-u^2/18}.\end{aligned}$$

One final union bound is over time rather than coordinates:

$$\Pr\left[\max_{t\le T}\|S_t\|_\infty > u\right] \le 2e^{-u^2/18}\sum_{t=1}^{T} t = T(T+1)e^{-u^2/18} \le 2T^2e^{-u^2/18}.$$

Setting the last expression equal to δ gives $u = \sqrt{18\log(2T^2/\delta)}$. For $T \ge 2$, this choice satisfies the condition $u^2/18 \ge 1$; for $T = 1$, the claim is immediate from $\|v_1\|_\infty \le 1$.

For the final sum, the fixed-time bound has prefactor $2\dim E_T \le 2\min\{d, T\}$ instead of $2T^2$; there is no union over prefixes. Substitution gives the second threshold. If $\min\{d, T\} \ge 2$, this threshold satisfies $u^2/18 \ge 1$ as required. If $\min\{d, T\} = 1$ and E_T is nonzero, choose a unit vector v spanning it and write $S_T = Zv$. The moment bound gives $\mathbb{E}e^{\lambda Z} \le e^{9\lambda^2/2}$, and $\|S_T\|_\infty = |Z|\|v\|_\infty \le |Z|$. The scalar bound $\Pr[|Z| > u] \le 2e^{-u^2/18}$ proves the same threshold without the restriction $u^2/18 \ge 1$. If $E_T = \{0\}$, the sum is zero. Each round uses three inner products, a constant-size coupling, and three vector updates, proving the $O(dT)$ operation bound when scalar random draws and exponential evaluations have unit cost. The working space is $O(d)$. If vectors are supplied as sparse coordinate lists, the same algorithm takes $O(d + T + \sum_t \operatorname{nnz}(v_t))$ operations: initialize the three states once, then access only the arriving vector's nonzero coordinates for the inner products and updates. The proof is unchanged, and neither T nor δ is needed by the signing rule. □

A guarantee without a fixed horizon. For a fixed infinite input sequence, the same signing rule satisfies, with probability at least $1 - \delta$, the simultaneous bounds

$$\|S_t\|_\infty \le \sqrt{18\log\frac{4t^3}{\delta}} \qquad \text{for every } t \ge 1.$$

Indeed, apply the fixed-time estimate with $u_t = \sqrt{18\log(4t^3/\delta)}$. Its condition $u_t^2/18 \ge 1$ holds, and the failure probability at time t is at most $\delta/(2t^2)$. A union bound and $\sum_{t\ge1} t^{-2} \le 1 + \int_1^\infty x^{-2}\,dx = 2$ prove the claim. No stopping horizon is supplied to the algorithm, so the same event also controls a stopping time chosen from the observed signs. The input sequence itself must still be independent of the randomness.

15 Polynomial Algorithms

Coefficient spaces and precision. Coefficients lie in a field K, except where a commutative ring is specified. The space $K[x]_{<n}$ is represented by arrays of length n, including trailing zeros. Precision modulo x^n means that coefficients $0, \dots, n-1$ are known. Write $F_{<n} = F \bmod x^n$, $F_{\ge n} = F - F_{<n}$, and

$$\operatorname{rev}_d F = x^d F(x^{-1}) \qquad (\deg F \le d).$$

Reversal always uses the stated degree bound, with zero padding when necessary. We set $\deg 0 = -\infty$ and interpret out-of-range coefficients as zero in each specified coefficient space.

Complexities count coefficient operations unless a bit bound is stated. Division requires the indicated denominator to be a unit. Logarithms, exponentials, and factorial-based transforms require their integer divisors to be invertible; characteristic zero or characteristic larger than the working precision suffices. Results over small-characteristic fields or general rings state their assumptions separately.

15.1 Formal Power Series Operations

Let $D = d/dx$ be the formal derivative. If an error is $E = O(x^m)$, then $E^2 = O(x^{2m})$. Newton updates cancel the linear error, thereby doubling precision. A constant number of products at each doubling costs $\sum_{m=1,2,4,\dots<N} O(m\log(m+2)) = O(N\log(N+2))$. Truncate the last update to the requested precision.

15.1.1 Inverse

Let $A(0) \neq 0$, and suppose $AF \equiv 1 \pmod{x^m}$. Writing $E = 1 - AF$, the update $F_{\text{new}} = F(1+E)$ gives $AF_{\text{new}} = (1-E)(1+E) = 1 - E^2$. Therefore

$$\boxed{F_{\text{new}} = F(2 - AF) \bmod x^{2m}.}$$

Starting from $F = A(0)^{-1}$, these updates compute the inverse in $O(N \log(N+2))$ operations. If $AF = AG = 1$, multiplication by either inverse gives uniqueness.

Only the upper half changes. If $AF = 1 + x^m H \pmod{x^{2m}}$, then $F_{\text{new}} = F - x^m(FH \bmod x^m)$. To avoid a separate negation at each step, maintain the negative inverse $R = -F$. With $H = (AR+1)/x^m \bmod x^m$, its update is $R_{\text{new}} = R + x^m(RH \bmod x^m)$.

Algorithm 93 Formal power-series inverse

Input: $A \bmod x^N$, $A(0) \neq 0$, $N \geq 1$
Output: $A^{-1} \bmod x^N$

```
F ← A(0)^{-1}, m ← 1
while m < N do
    r ← min{2m, N}
    F ← F(2 − A_{<r} F) mod x^r
    m ← r
return F
```

15.1.2 Logarithm

For a unit A, its normalized logarithm is $\log(A/A(0))$, with constant coefficient zero. Its derivative is A'/A, so

$$\boxed{\log \frac{A}{A(0)} = \int_0^x \frac{A'(t)}{A(t)}\, dt.}$$

To obtain precision x^N, compute the integrand modulo x^{N-1}. One inverse and one product give $O(N \log(N+2))$ operations. No scalar logarithm of $A(0)$ is needed. The notation $\log A$ below is used when $A(0) = 1$.

15.1.3 Square Root

Assume 2 is invertible and choose $f_0 \neq 0$ with $f_0^2 = A(0)$. There is a unique square root with constant coefficient f_0. Maintain $F^2 \equiv A \pmod{x^m}$ and $G \equiv F^{-1} \pmod{x^m}$. Since $F^2 - A = O(x^m)$, the known precision of G already suffices for the correction

$$\boxed{F_{\text{new}} = F - \tfrac{1}{2}G(F^2 - A) \bmod x^{2m}, \qquad G_{\text{new}} = G(2 - F_{\text{new}}G) \bmod x^{2m}.}$$

The linear term in $F_{\text{new}}^2 - A$ cancels the residual; the squared correction and the inverse-approximation error start in degree $2m$. The second update lifts the inverse. Starting from $(F, G) = (f_0, f_0^{-1})$ gives total cost $O(N \log(N+2))$. Both corrections are confined to the upper half.

For a nonzero series $A = x^v U$ with $U(0) \neq 0$, a square root exists exactly when $v = 2r$ is even and $U(0)$ has a square root; it is $x^r\sqrt{U}$. Precision must be shifted as well: to determine this root modulo x^N for $N > r$, one needs $U \bmod x^{N-r}$, that is, $A \bmod x^{N+r}$. Knowing only $A \bmod x^N$, with $2r < N$, determines the chosen root only modulo x^{N-r}. Its remaining coefficients do not affect the square modulo x^N. A positive-valuation root has no power-series inverse. For a fixed unit root coefficient, uniqueness follows from $(F-G)(F+G) = 0$, since $F + G$ has unit constant coefficient.

15.1.4 Inverse Square Root

Assume two is invertible. For a unit A, maintain $AF^2 \equiv 1 \pmod{x^m}$, starting from a chosen $F(0)$ with $A(0)F(0)^2 = 1$. The update is

$$\boxed{F_{\text{new}} = \tfrac{1}{2}F(3 - AF^2) \bmod x^{2m}.}$$

If $AF^2 = 1 - E$, then $AF_{\text{new}}^2 = (1 - E)(1 + E/2)^2 = 1 + O(E^2)$. Thus the same doubling analysis gives $O(N \log(N + 2))$ operations, without maintaining a separate inverse.

15.1.5 Exponential

Assume $A(0) = 0$. The equation $\log F = A$ gives the Newton update $F_{\text{new}} = F(1 + A - \log F)$. To evaluate the correction without first extending the inverse of the old F, use $\theta = xD$, whose inverse on zero-constant series divides the coefficient of x^j by j. Unlike D, θ does not lower order.

Suppose $F \equiv \exp A \pmod{x^m}$, $G \equiv F^{-1} \pmod{x^m}$, and $\deg F, \deg G < m$. The residual $V = (\theta A_{<m})F - \theta F$ has order at least m. Since $\deg \theta F < m$, it is obtained through degree $2m - 1$ by keeping only the upper half of $(\theta A_{<m})F$. Now $\theta \log F = \theta A_{<m} - V/F$, and the known inverse already gives $V/F \equiv VG \pmod{x^{2m}}$. Consequently

$$\log F \equiv A_{<m} - \theta^{-1}(VG) \pmod{x^{2m}}.$$

Set $C = A_{<2m} - A_{<m} + \theta^{-1}(VG)$. Then $C = O(x^m)$ and $\exp C \equiv 1 + C \pmod{x^{2m}}$, giving

$$\boxed{F_{\text{new}} = F(1 + C) \bmod x^{2m}, \qquad G_{\text{new}} = G(2 - F_{\text{new}}G) \bmod x^{2m}.}$$

The old inverse remains valid for the new F through its first m coefficients, so one inverse update suffices. A constant number of products per level gives $O(N \log(N + 2))$ operations.

Algorithm 94 Formal power-series exponential

Input: $A \bmod x^N$, $A(0) = 0$, $N \geq 1$
Output: $\exp A \bmod x^N$

```
F ← 1, G ← 1, m ← 1
while m < N do
    r ← min{2m, N}
    V ← ((θA_{<m})F)_{≥m} mod x^r
    C ← A_{<r} − A_{<m} + θ^{-1}(VG mod x^r)
    F ← F(1 + C) mod x^r
    G ← G(2 − FG) mod x^r
    m ← r
return F
```

15.1.6 Powers

For a unit F, a scalar α, and a prescribed constant c, define $H = c\exp(\alpha \log(F/F(0)))$. The constant is supplied separately, since $F(0)^\alpha$ need not be defined in the coefficient field. It costs $O(N \log(N+2))$ operations through precision x^N. For an integer exponent k, take $c = F(0)^k$; obtaining that scalar adds $O(\log(|k| + 2))$ operations. For a nonnegative integer power of $F = x^v U$, first remove the valuation: $F^k = x^{vk}U^k$, and return zero modulo x^N if $vk \geq N$. The zeroth power is one, including for zero input.

A direct recurrence is preferable for low-degree input. Writing $F = \sum f_j x^j$, $H = \sum h_j x^j$, the identity $F\theta H = \alpha H \theta F$ gives

$$\boxed{nf_0h_n = \sum_{j=1}^{n} \big((\alpha + 1)j - n\big) f_j h_{n-j}, \qquad h_0 = c.}$$

Indeed, isolate the $j = 0$ term on the left and combine the remaining convolution terms. If $\deg F = d$, the sum stops at $\min\{d, n\}$, giving $O(N(d+1))$ operations.

15.2 Polynomial Arithmetic

15.2.1 Division

Let $A = QB + R$, where $B \neq 0$ and $\deg R < \deg B = m$. For $n = \deg A \geq m$, put $d = n - m$. Reversing with these specified bounds gives

$$\operatorname{rev}_n A = \operatorname{rev}_d Q \, \operatorname{rev}_m B + x^{d+1} \operatorname{rev}_{m-1} R.$$

If $m = 0$, the remainder is zero and the last term is omitted. The reversed divisor has nonzero constant coefficient, so

$$\boxed{Q = \operatorname{rev}_d \left((\operatorname{rev}_n A)(\operatorname{rev}_m B)^{-1} \bmod x^{d+1} \right), \qquad R = A - QB.}$$

This reduces division to series inversion and multiplication, costing $O((n+1)\log(n+2))$ operations. Over a commutative ring the same argument works when the leading coefficient of B is a unit; in particular, it works for every monic divisor.

Algorithm 95 Polynomial division

```
function DivMod(A, B)
    Require B ≠ 0
    if A = 0 or deg A < deg B then
        return (0, A)
    n ← deg A, m ← deg B, d ← n − m
    U ← rev_n A mod x^{d+1}
    V ← rev_m B mod x^{d+1}
    Q ← rev_d(U V^{-1} mod x^{d+1})
    return (Q, A − QB)
```

15.2.2 Taylor Shift

For $P(x) = \sum_{i=0}^{d} p_i x^i$, coefficient extraction gives

$$[x^j]P(x+c) = \frac{1}{j!} \sum_{i=j}^{d} (p_i i!) \frac{c^{i-j}}{(i-j)!}.$$

Reverse the factorial-scaled coefficients and convolve with $(c^i/i!)$. With $A(x) = \sum_{i=0}^{d} p_i i! x^{d-i}$ and $B(x) = \sum_{i=0}^{d} c^i x^i / i!$, the answer is $\sum_{j=0}^{d} [x^{d-j}](AB) x^j / j!$. Thus a Taylor shift costs $O((d+1)\log(d+2))$ operations when $1, \ldots, d$ are invertible. Generate powers of c, factorials, and inverse factorials successively in linear time.

Algorithm 96 Taylor shift

```
function TaylorShift(P, c, d)
    Require deg P ≤ d, with the integers 1, ..., d invertible
    A ← Σ_{i=0}^{d} p_i i! x^{d−i}, B ← Σ_{i=0}^{d} c^i x^i / i!
    C ← AB mod x^{d+1}
    return Σ_{j=0}^{d} C_{d−j} x^j / j!
```

A division-free shift. Over an arbitrary commutative ring, use a balanced coefficient split $P = P_0 + x^h P_1$. Then $P(x+c) = P_0(x+c) + (x+c)^h P_1(x+c)$. Pad the input length to a power of two and precompute $(x+c)^{2^j}$ by repeated squaring. Recursing on the two coefficient halves gives $O((d+1)\log^2(d+2))$ operations, with no factorial inverses. At each depth the total coefficient length is $O(d+1)$. This shift applies in small characteristic and over rings with nilpotents.

15.2.3 Evaluation Shift

Suppose $0, 1, \ldots, d$ are distinct in the coefficient field and we know $f(0), \ldots, f(d)$, where $\deg f \le d$. We want $f(s), \ldots, f(s+d)$ without first interpolating the coefficients.

Lemma 502 (Polynomial evaluation shift)**.** *These values can be computed in $O((d+1)\log(d+2))$ operations. The target points may overlap the original points.*

Proof. Let $P(x) = \prod_{j=0}^{d}(x-j)$. Lagrange interpolation gives

$$f(x) = P(x)\sum_{i=0}^{d}\frac{a_i}{x-i}, \qquad a_i = \frac{(-1)^{d-i}f(i)}{i!(d-i)!}.$$

Define $b_j = (s-d+j)^{-1}$ for $0 \le j \le 2d$, except that $b_j = 0$ when its denominator is zero. One convolution computes $v_k = \sum_{i=0}^{d} a_i b_{d+k-i}$ for $0 \le k \le d$. For a target outside $\{0, \ldots, d\}$, its value is $P(s+k)v_k$. An overlapping target is copied directly from the input; the artificial zero entries of b affect only such targets. For a zero slot j, the overlapping targets are given by $i = d+k-j \in [0, d]$. There are at most two such slots, since the characteristic is zero or exceeds d; all copies take linear time.

It remains to compute all $P(s+k)$ without dividing by a zero. They are products of consecutive windows of length $d+1$ in $s-d, s-d+1, \ldots, s+d$. Maintain the product of the nonzero entries and the number of zero entries in the current window. Delete the outgoing nonzero factor by division and multiply the incoming nonzero factor; update the zero count separately. The window product is zero exactly when this count is positive. The window updates take $O(d+1)$ operations, including in positive characteristic. □

For known values at $a, a+h, \ldots, a+dh$, with $h \neq 0$, apply the lemma to $g(x) = f(a+hx)$; translating the points by δ corresponds to the shift δ/h.

15.2.4 Half-GCD and the Euclidean Quotient Sequence

A Euclidean step with quotient q sends (A, B) to $(B, A - qB)$. Write its forward and inverse matrices as

$$E(q) = \begin{pmatrix} 0 & 1 \\ 1 & -q \end{pmatrix}, \qquad W(q) = E(q)^{-1} = \begin{pmatrix} q & 1 \\ 1 & 0 \end{pmatrix}.$$

The matrices have determinant -1, so every product is unimodular and preserves the ideal generated by the two polynomials. Half-GCD computes the initial Euclidean steps up to the first pair (C, D) satisfying $\deg C \ge m > \deg D$, where $m = \lceil \deg A/2 \rceil$. Its speed comes from determining these steps from high coefficients only.

Lemma 503 (Degrees of Euclidean matrices)**.** *For nonconstant $q_1, \ldots, q_j$, with $j \ge 1$, write $W = W(q_1)\cdots W(q_j) = \left(\begin{smallmatrix} P & Q \\ S & T \end{smallmatrix}\right)$. Then $\deg P = \sum_i \deg q_i$, both $\deg Q, \deg S < \deg P$, and $\deg T \le \min\{\deg Q, \deg S\}$. All entries of W and W^{-1} have degree at most $\deg P$. If $(a, b)^{\mathsf{T}} = W(c, d)^{\mathsf{T}}$, with $\deg c > \deg d$, then $\deg P = \deg a - \deg c$.*

Proof. For one factor the assertions are immediate. On appending $W(q)$, the entries become $Pq + Q, P, Sq + T, S$. The strict degree inequalities prevent leading-term cancellation and prove the claims by induction. The inverse has the same entries, permuted and signed, because $\det W = \pm 1$. Finally, $a = Pc + Qd$ has leading term from Pc, proving the last identity. □

Lemma 504 (Lifting a high-coefficient Euclidean prefix)**.** *Let $t \geq 0$, and write $A = x^t a + a_0$, $B = x^t b + b_0$, where $a \neq 0$, $\deg a > \deg b$, and $\deg a_0, \deg b_0 < t$. Put $n = \deg a$. Suppose a Euclidean prefix on (a, b) has forward matrix R and output (c, d), with $\deg c \geq \lceil n/2 \rceil > \deg d$. Then the same quotient list is a Euclidean prefix on (A, B). Its output $(C, D) = R(A, B)$ satisfies*

$$\boxed{\deg C = t + \deg c, \qquad \deg D \leq \max\{t + \deg d,\ t + n - \deg c - 1\} < \deg C.}$$

The empty prefix is allowed.

Proof. For the empty prefix the assertions follow directly from the truncation. Otherwise $R^{-1} = W(q_1) \cdots W(q_j)$, and the preceding lemma bounds every entry of R by $n - \deg c$. Consequently

$$(C, D)^{\mathsf{T}} = x^t (c, d)^{\mathsf{T}} + R(a_0, b_0)^{\mathsf{T}},$$

where both error coordinates have degree at most $t + n - \deg c - 1$. Since $2 \deg c \geq n$, these errors have smaller degree than $x^t c$, proving the degree bounds.

To identify the quotients, start from $\deg C > \deg D$ and reconstruct the input by applying $W(q_j), \ldots, W(q_1)$ in that order. At every step $(U, V) \mapsto (q_i U + V, U)$, where $\deg V < \deg U$. Thus division of the reconstructed first polynomial by the second has quotient exactly q_i and remainder V. The reconstructed input is (A, B), so the entire list consists of its actual initial Euclidean quotients. □

Algorithm 97 Half-GCD

```
1:  function HalfGCD(A, B)
2:      Require A ≠ 0 and deg A > deg B; allow B = 0
3:      n ← deg A, m ← ⌈n/2⌉
4:      if deg B < m then
5:          return I
6:      R ← HalfGCD(A div x^m, B div x^m)
7:      (C, D)^T ← R(A, B)^T
8:      if deg D < m then
9:          return R
10:     (q, H) ← DivMod(C, D)
11:     R ← E(q)R
12:     if deg H < m then
13:         return R
14:     t ← 2m − deg D
15:     S ← HalfGCD(D div x^t, H div x^t)
16:     return SR
```

Theorem 505 (Half-GCD)**.** *For $\deg A > \deg B$, $A \neq 0$, the algorithm returns the matrix of a Euclidean prefix with output (C, D) satisfying $\deg C \geq \lceil \deg A/2 \rceil > \deg D$. It costs $O((n+1)\log^2(n+2))$ operations for $n = \deg A$.*

Proof. Induct on n. The immediate return is correct. Otherwise put $a = n - m = \lfloor n/2 \rfloor$ and $h = \lceil a/2 \rceil$. The first recursive call has degree a; the lifting lemma makes its quotients valid for the full input and gives $\deg C \geq m$, $\deg D \leq m + h - 1$. If $\deg D < m$, we are done.

Otherwise one exact division gives the pair (D, H), with $c = \deg D \geq m$. Returning when $\deg H < m$ is correct. In the remaining case, $t = 2m - c > 0$, and the next truncated first polynomial has degree $c - t = 2(c - m)$. Its half threshold is $c - m$. The lifting lemma therefore gives a final first degree at least $t + (c - m) = m$, and a second degree at most $m - 1$. The lifted quotients continue the full Euclidean sequence.

The second recursive degree is at most $2h - 2 \leq a$; the first is a. All nonrecursive work consists of constantly many polynomial products and divisions of size $O(n + 1)$. Thus $T(n) \leq 2T(\lfloor n/2 \rfloor) + O((n+1)\log(n+2))$, giving the stated bound. □

Extended GCD and quotient recovery. After half-GCD, perform one additional Euclidean step unless the second polynomial is zero. Both remaining degrees are then below the old half threshold, so recurse and multiply the transformation matrices. An initial Euclidean step handles equal input degrees; swapping handles reversed degrees. For zero second input return the first polynomial and its trivial Bézout coefficients; handle $(0, 0)$ separately. Normalize the final nonzero gcd to be monic, scaling its Bézout row by the same scalar. The total cost remains $O((n+1)\log^2(n+2))$.

To recover the quotient list, concatenate the first recursive list, the explicit quotient, and the second recursive list at each call. The lifting lemma shows that this is the full-input quotient sequence. The quotients have total degree at most n and total stored length $O(n+1)$. The full remainders, whose total length may be quadratic, are not constructed.

Algorithm 98 Extended polynomial GCD

```
1:  function FullGCD(A, B)
2:      if B = 0 then
3:          if A = 0 then
4:              return (0, I)
5:          c ← lc(A)
6:          return (A/c, diag(c^{-1}, 1))
7:      if A = 0 or deg A < deg B then
8:          (g, R) ← FullGCD(B, A)
9:          return (g, R ( 0 1 ; 1 0 ))
10:     if deg A = deg B then
11:         (q, H) ← DivMod(A, B)
12:         (g, R) ← FullGCD(B, H)
13:         return (g, RE(q))
14:     R ← HalfGCD(A, B)
15:     (C, D)^T ← R(A, B)^T
16:     if D = 0 then
17:         c ← lc(C)
18:         return (C/c, diag(c^{-1}, 1)R)
19:     (q, H) ← DivMod(C, D)
20:     (g, S) ← FullGCD(D, H)
21:     return (g, SE(q)R)
```

The returned matrix satisfies $R(A, B)^{\mathsf{T}} = (g, 0)^{\mathsf{T}}$; its first row contains the Bézout coefficients. Swaps and any initial constant quotient are recorded separately when the exact quotient list is requested.

15.2.5 Modular Inversion

For a nonconstant modulus M, first reduce A modulo M. Extended GCD produces $uM + vA = g$. An inverse exists exactly when g is a nonzero constant, in which case

$$\boxed{A^{-1} \bmod M = (v/g) \bmod M.}$$

Necessity follows because any common divisor would divide one; sufficiency is the displayed Bézout identity. The cost is $O(n \log^2(n+2))$, with $n = \deg M$, after the initial reduction.

15.2.6 Resultants and Discriminants

For nonzero polynomials $A = a\prod_{i=1}^{n}(x - \alpha_i)$, $B = b\prod_{j=1}^{m}(x - \beta_j)$, define

$$\operatorname{Res}(A, B) = a^m \prod_i B(\alpha_i) = a^m b^n \prod_{i,j}(\alpha_i - \beta_j).$$

Roots are counted with multiplicity. The expression is symmetric in each root family, so it lies in the coefficient field; it vanishes exactly when the polynomials have a common root in a splitting field. The empty product gives $\operatorname{Res}(A, b) = b^n$ for constant $b \neq 0$.

Lemma 506 (Euclidean resultant identity)**.** *If $R = A \bmod B \neq 0$, with $\deg A = n$, $\deg B = m$, $\deg R = r$, and $b = \operatorname{lc}(B)$, then*

$$\boxed{\operatorname{Res}(A, B) = (-1)^{nm} b^{n-r} \operatorname{Res}(B, R).}$$

Proof. Swapping the root families contributes $(-1)^{nm}$. At every root β_j of B, $A(\beta_j) = R(\beta_j)$. The root-product definitions of $\operatorname{Res}(B, A)$ and $\operatorname{Res}(B, R)$ consequently differ only by b^{n-r}. □

Theorem 507 (Fast resultants)**.** *A resultant of polynomials of degrees at most n is computable in $O((n + 1)\log^2(n + 2))$ field operations.*

Proof. For constants use the definition; swap inputs if necessary and extract one division if their degrees are equal. In the remaining case recover the Euclidean quotient list by half-GCD, keeping the unnormalized remainders implicit. The list determines their degrees and leading coefficients: whenever $r_{i-1} = q_{i+1} r_i + r_{i+1}$, $\operatorname{lc}(r_i) = \operatorname{lc}(r_{i-1}) / \operatorname{lc}(q_{i+1})$.

If the gcd has positive degree, return zero. Otherwise apply the lemma successively, ending with the constant-remainder formula. Only the remainder degrees and leading coefficients are needed. The sum of the exponents $\deg r_{i-2} - \deg r_i$ is $O(n)$, so the scalar powers cost $O(n)$ additional multiplications by binary powering. Half-GCD dominates the cost. □

For a monic polynomial $F = \prod_{i=1}^{n} (x - \alpha_i)$, with $n \geq 1$, its discriminant is

$$\boxed{\operatorname{disc}(F) = \prod_{i<j} (\alpha_i - \alpha_j)^2 = (-1)^{n(n-1)/2} \operatorname{Res}(F, F').}$$

For distinct roots, multiply $F'(\alpha_i) = \prod_{j \neq i} (\alpha_i - \alpha_j)$; repeated roots make both sides zero. For a general degree-n polynomial with leading coefficient a, first normalize and use $\operatorname{disc}(F) = a^{2n-2} \operatorname{disc}(F/a)$. This computes the discriminant within the same bound. Normalization also covers positive characteristic, where F' may have smaller degree. If $F' = 0$ and $\deg F > 0$, return zero.

15.2.7 Root Finding over a Prime Field

Let $f \in \mathbb{F}_p[x]$ be nonzero of degree n. First compute $g = \gcd(f, x^p - x)$, using modular powering. Since $x^p - x = \prod_{a \in \mathbb{F}_p} (x - a)$ has distinct roots, g is exactly the product of the distinct linear factors of f. Only g, whose degree $r \leq \min\{n, p\}$, needs splitting. For $p = 2$, test zero and one.

Assume p is odd. For a factor h and a uniform random $b \in \mathbb{F}_p$, compute $u = (x + b)^{(p-1)/2} \bmod h$ and split by $\gcd(h, u - 1)$. This separates roots with quadratic character $+1$ from the others. For distinct roots α, β, put $c = \beta - \alpha \neq 0$. As $v = \alpha + b$ ranges over $\mathbb{F}_p^\times$, $1 + c/v$ ranges bijectively over $\mathbb{F}_p \setminus \{1\}$. Its $(p - 1)/2$ nonsquare values correspond to nonzero $v, v + c$ with opposite quadratic characters. Thus each pair is separated with probability at least $(p - 1)/(2p) \geq 1/3$.

Theorem 508 (Randomized root finding)**.** *All distinct roots of a nonzero degree-n polynomial over $\mathbb{F}_p$ can be found by a Las Vegas algorithm in expected*

$$\boxed{O((n + 1)\log^2(n + 2)\log(p + 1))}$$

field operations. Every reported root is correct.

Proof. Organize splitting into rounds, giving every current nonlinear factor one fresh trial, including factors whose preceding trial failed. A fixed pair remains together after j rounds with probability at most $(2/3)^j$, even conditioned on the earlier history. If L is the number of rounds, a union bound gives $\Pr(L > j) \le \min\{1, r^2(2/3)^j\}$, hence $\mathbb{E}L = O(\log(r+2))$.

A split at degree $d \le p$ costs $O(d\log(d+2)\log(p+1) + d\log^2(d+2)) = O(d\log(d+2)\log(p+1))$, including its gcd. The factors at a round have total degree at most r. This proves the splitting bound. Extracting g initially costs $O((n+1)\log(n+2)(\log(p+1)+\log(n+2)))$, which is covered by the displayed total. Each split uses exact gcds and divisions, so the algorithm is Las Vegas. □

Algorithm 99 Distinct roots over a prime field

Input: Nonzero $f \in \mathbb{F}_p[x]$
Output: All its roots in $\mathbb{F}_p$

if $p = 2$ **then**
  return the elements of $\{0,1\}$ at which f vanishes
if $\deg f = 0$ **then**
  return the empty set
$g \leftarrow \gcd(f, (x^p \bmod f) - x)$
$\mathcal{P} \leftarrow \{g\}$, $R \leftarrow \emptyset$
while $\mathcal{P} \neq \emptyset$ **do**
  $\mathcal{Q} \leftarrow \emptyset$
  for $h \in \mathcal{P}$ **do**
    if $\deg h = 1$ **then**
      Add $-h_0/h_1$ to R
    else if $\deg h > 1$ **then**
      Choose uniform $b \in \mathbb{F}_p$
      $u \leftarrow (x+b)^{(p-1)/2} \bmod h$
      $v \leftarrow \gcd(h, u-1)$
      if $0 < \deg v < \deg h$ **then**
        Add $v, h/v$ to $\mathcal{Q}$
      else
        Add h to $\mathcal{Q}$
  $\mathcal{P} \leftarrow \mathcal{Q}$
return R

15.2.8 Minimum Linear Recurrence

An order-d recurrence for a finite prefix $s_0, \dots, s_{N-1}$ is a monic polynomial $P(Z) = \sum_{j=0}^{d} p_j Z^j$ satisfying $\sum_{j=0}^{d} p_j s_{i+j} = 0$ for $0 \le i < N-d$. Its connection polynomial is $C(x) = \operatorname{rev}_d P$, with $C(0) = 1$. The order is d, even if $\deg C < d$; zero roots of P describe initial transients.

Reverse the input prefix with its full length: $S^R(x) = \sum_{j=0}^{N-1} s_j x^{N-1-j}$. Direct coefficient comparison shows

$$\boxed{P \text{ is a recurrence of order } d \iff \deg(PS^R \bmod x^N) < d.}$$

Indeed, the coefficient at x^{N-1-i} is exactly the recurrence sum at index i.

Run Euclid on $r_{-1} = x^N, r_0 = S^R$, and retain the coefficient t_i of S^R:

$$r_i = r_{i-2} - q_i r_{i-1}, \qquad t_i = t_{i-2} - q_i t_{i-1}, \qquad t_{-1} = 0, \quad t_0 = 1.$$

Then $t_i S^R \equiv r_i \pmod{x^N}$ and $\deg t_i + \deg r_{i-1} = N$. The latter follows inductively from $\deg q_i = \deg r_{i-2} - \deg r_{i-1}$: the degree of $q_i t_{i-1}$ strictly dominates that of t_{i-2}.

Theorem 509 (Minimum-order recurrence)**.** *If the input prefix is not zero, let i be the first Euclidean index with $\deg r_i < \deg t_i$, including a final zero remainder. Then $P = t_i/\operatorname{lc}(t_i)$ is a minimum-order recurrence. Such a recurrence is computable in $O(N \log^2(N+2))$ operations. Its coefficients need not be unique.*

Proof. The congruence proves validity. Suppose a smaller-degree nonzero U has remainder $V = US^R \bmod x^N$ with $\deg V < \deg U$. Cross-multiplication with the preceding Euclidean pair gives $Ur_{i-1} - t_{i-1}V \equiv 0 \pmod{x^N}$. Both products have degree less than N, since $\deg U < \deg t_i = N - \deg r_{i-1}$ and $\deg t_{i-1} \le \deg r_{i-1}$. Therefore they are equal. But $\deg(Ur_{i-1}) > \deg(t_{i-1}V)$; if $V = 0$, the left product is still nonzero. Either case is a contradiction.

Recover the quotient list by half-GCD. Its degree drops determine every remainder degree without constructing the remainders: for a nonfinal r_j, $\deg r_j = \deg r_{j-1} - \deg q_{j+1}$, and the final remainder is zero. This identifies the crossing index in linear time. A balanced product of the first i matrices $E(q_j)$, in reverse multiplication order, gives t_i as its lower-right entry. Their total degree is at most N; each tree level costs $O(N \log(N+2))$, and there are $O(\log(N+2))$ levels. □

Algorithm 100 Minimum recurrence of a finite prefix

Input: $s_0, \ldots, s_{N-1}$
Output: A monic minimum-order forward recurrence P and connection polynomial C

if the prefix is zero, including $N = 0$ **then**
  return $(1, 1)$, of order zero
$S^R \leftarrow \sum_{j=0}^{N-1} s_j x^{N-1-j}$
Obtain the exact quotient list $q_1, \ldots, q_\ell$ for (x^N, S^R) by half-GCD
$d_{-1} \leftarrow N$, $d_0 \leftarrow \deg S^R$
for $j = 1, \ldots, \ell - 1$ **do**
  $d_j \leftarrow d_{j-1} - \deg q_{j+1}$
$d_\ell \leftarrow -\infty$
Find the first $i \in \{1, \ldots, \ell\}$ with $d_i < N - d_{i-1}$
$R \leftarrow E(q_i) \cdots E(q_1)$, using a balanced product tree
$P \leftarrow R_{22}/\operatorname{lc}(R_{22})$, $d \leftarrow \deg P$
return $(P, \operatorname{rev}_d P)$, retaining order d

For example, $(1, 0, 1)$ has minimum order two, but every $P(Z) = Z^2 - tZ - 1$ fits the prefix. The prefix alone does not determine which recurrence its continuation satisfies.

Corollary 510 (Recovery with an order bound)**.** *If an infinite sequence satisfies a recurrence of order at most d from its initial state, its first $2d$ terms determine its unique monic minimum recurrence, recovered by the preceding algorithm.*

Proof. The true generating function is A/B, where $B(0) = 1$, $\deg B \le d$, and $\deg A < d$. A recovered recurrence of order $e \le d$ gives U/C, where $C(0) = 1$, $\deg C \le e$, and $\deg U < e$. The two series agree modulo x^{2d}, so $AC - UB$ is divisible by x^{2d}. Its degree is less than $d + e \le 2d$, hence it is zero. The recovered recurrence therefore holds for the whole sequence and has minimum order.

For a nonzero rational series in reduced form A/B, with $B(0) = 1$, its minimum forward order is $e = \max\{\deg B, 1 + \deg A\}$, and its monic minimum recurrence is $\operatorname{rev}_e B$. Any other representation multiplies both A, B by a polynomial and cannot decrease this order. This also proves uniqueness, including zero-root transients. □

15.3 Rational Functions and Transposition

15.3.1 Bostan–Mori

For a polynomial H, define the coefficient-decimation maps $\mathcal{E}H = \sum_j h_{2j} x^j$ and $\mathcal{O}H = \sum_j h_{2j+1} x^j$. They select coefficients; no division by two is part of their definition.

Let $V(0) \neq 0$. The product $V(x)V(-x)$ is even, so write it as $W(x^2)$. Multiplying numerator and denominator by $V(-x)$ gives

$$\boxed{[x^N]\frac{U}{V} = [x^{\lfloor N/2 \rfloor}]\frac{\mathcal{P}_{N \bmod 2}(U(x)V(-x))}{\mathcal{E}(V(x)V(-x))}, \quad \mathcal{P}_0 = \mathcal{E}, \quad \mathcal{P}_1 = \mathcal{O}.}$$

Only the numerator determines the coefficient parity, since the denominator is a series in x^2. This identity also holds in characteristic two: there $V(x)V(-x) = V(x)^2$ is still even.

Theorem 511 (A distant rational coefficient)**.** *For* $\deg U < d$, $\deg V \leq d$, $V(0) \neq 0$, *the value* $[x^N]U/V$ *is computable in* $O(d \log(d+2) \log(N+2))$ *operations.*

Proof. Each step preserves the numerator degree bound $< d$ and denominator bound $\leq d$, while halving N. After $O(\log(N+2))$ steps the answer is the ratio of constant coefficients. Each step uses two products of size $O(d)$. □

Algorithm 101 Bostan–Mori coefficient extraction

Input: $\deg U < d$, $\deg V \leq d$, $V(0) \neq 0$, $N \geq 0$
Output: $[x^N]U/V$

```
while N > 0 do
    A ← U(x)V(−x), B ← V(x)V(−x)
    U ← P_{N mod 2} A, V ← EB
    N ← ⌊N/2⌋
return U(0)/V(0)
```

For an improper rational function, first divide the numerator by the denominator. Add the requested coefficient of the polynomial part to the coefficient extracted from the proper fraction.

Truncation to the target index. Before a step, the numerator and denominator may be reduced modulo x^{N+1}, where N is the current target index. These prefixes determine the quotient through that index. Once $N < d$, the operand lengths therefore decrease geometrically along with N, rather than remaining at the original bound d. The remaining arithmetic costs $O((N+1)\log(N+2))$. Power projection uses the same truncation in its bivariate recurrence.

When $V(0) = 1$ and $V(x) = \prod_i (1 - \alpha_i x)$, its next value is $\prod_i (1 - \alpha_i^2 x)$. Thus coefficient decimation squares the exponential bases of the underlying recurrence while keeping the number of bases bounded. No roots need to be computed.

Antipodal transform values. To form W with $W(x^2) = V(x)V(-x)$, a transform of length $2L$, with $L > \deg V$, supplies paired values $V(\omega^j), V(-\omega^j)$. Their products are the values of W at the L-th roots ω^{2j}. In bit-reversed order these antipodal entries are adjacent, so multiplying each adjacent pair produces a length-L transform directly. The degree bound $L > \deg W$ prevents cyclic aliasing.

15.3.2 The Transposition Principle

Fix coefficient bases and use the bilinear dot product $\langle u, v \rangle = \sum_i u_i v_i$; no complex conjugation is used. For a linear map $L : K^n \to K^m$, its transpose is characterized by $\langle Lu, v \rangle = \langle u, L^{\mathsf{T}} v \rangle$.

Theorem 512 (Transposition of a linear algorithm)**.** *A straight-line linear algorithm of size* T*, using additions and multiplications by fixed scalars, gives an algorithm for its transpose of size* $O(T + n + m)$*.*

Proof. Give every intermediate scalar a separate name, and attach to it an accumulated coefficient, initially zero. Seed the output coefficients by v; their weighted sum is $\langle Lu, v\rangle$. Now substitute each defining instruction backward into this linear form. For $z = a + b$, replace the term $\bar{z}z$ by $\bar{z}a + \bar{z}b$, adding $\bar{z}$ to the two input accumulators. For $z = ca$, add $c\bar{z}$ to the accumulator of a. The processed variable z then disappears from the linear form. Copies and coordinate selections are treated in the same way.

Each substitution preserves the value of the linear form. At the end it is a weighted sum of the original inputs, equal to $\langle Lu, v\rangle$ for every u. Its coefficient vector is therefore $L^{\mathsf{T}}v$. Each reversed instruction takes constant work, in addition to seeding and reading the endpoint arrays. □

Auxiliary data remain fixed during transposition. For example, in Bostan–Mori with fixed V, N, precompute the denominator sequence and transpose only the numerator operations.

15.3.3 Transposed Multiplication and Reversal Pairings

For fixed P, let $M_P : K[x]_{<a} \to K[x]_{<b}$ send Q to $PQ \bmod x^b$. Its dot-product transpose is

$$\boxed{M_P^{\mathsf{T}}(Z) = \sum_{j=0}^{a-1} x^j \sum_{\ell \geq 0} p_\ell z_{j+\ell}.}$$

Here $z_i = 0$ outside $0 \leq i < b$. The formula follows by expanding $\sum_{i<b} z_i \sum_j p_{i-j} q_j$ and collecting the coefficient of q_j. We write this middle product as $(Z \circ P)_{<a}$, with the input length b understood. It is one ordinary product after reversing the array Z, with the required coefficient interval extracted afterward.

The reversal pairing is $\langle U, Z\rangle_a^{\mathrm{rev}} = [x^{a-1}]UZ$, which is the dot product after reversing one length-a array. With this pairing, for $b \geq a$, the same multiplication map has adjoint

$$\boxed{M_P^*(Z) = \big(PZ \operatorname{div} x^{b-a}\big) \bmod x^a.}$$

Indeed, for every input monomial x^j, the two pairings both read the coefficient $[x^{b-1-j}]PZ$. Division by x^{b-a} here discards the lower coefficients.

Under the reversal pairings of lengths $2d$ and d, the parity selectors have adjoints

$$\boxed{\mathcal{E}^*Z = xZ(x^2), \qquad \mathcal{O}^*Z = Z(x^2).}$$

For example, $[x^{d-1}](\mathcal{E}U)Z = [x^{2d-1}]U\,xZ(x^2)$. Under dot-product pairings the formulas are instead $\mathcal{E}^{\mathsf{T}}Z = Z(x^2)$ and $\mathcal{O}^{\mathsf{T}}Z = xZ(x^2)$. The change in parity comes from the reversal pairing.

Transposed transforms and bit reversal. Let $\mathcal{F}_- = (\omega^{-ij})_{i,j}$, $\mathcal{F}_+ = (\omega^{ij})_{i,j}$, and let P permute indices by bit reversal. Natural-order transforms are symmetric. An output-bit-reversed forward transform is $\mathcal{B} = P\mathcal{F}_-$, with inverse $\mathcal{B}^{-1} = n^{-1}\mathcal{F}_+P$. Writing J for the permutation $i \mapsto -i \pmod n$, we have

$$\mathcal{B}^{\mathsf{T}} = nJ\mathcal{B}^{-1}, \qquad (\mathcal{B}^{-1})^{\mathsf{T}} = n^{-1}\mathcal{B}J.$$

To transpose a padded convolution, reverse its stages: zero-pad the output weights, apply the transpose of the inverse transform, multiply by the fixed transformed factor, apply the transpose of the forward transform, then select the original input coordinates.

15.3.4 Transposed Bostan–Mori and Modular Powers

Fix V with $\deg V \leq d$, $V(0) \neq 0$. We seek the length-d reversal-pairing state A satisfying

$$[x^{d-1}]U(x)A(x) = [x^N]\frac{U(x)}{V(x)} \qquad (\deg U < d).$$

At $N = 0$, it is $A = x^{d-1}/V(0)$. For a recursive step, first recurse with $V_1 = \mathcal{E}(V(x)V(-x))$ and index $\lfloor N/2 \rfloor$. Lift the returned state by the appropriate parity adjoint, multiply by $V(-x)$, and extract degrees $d, \dots, 2d-1$. The dimensions remain d even if $\deg V_1 < d$.

Algorithm 102 Transposed rational coefficient extraction

```
function TransposedProj(N, V, d)
    if N = 0 then
        return x^{d-1}/V(0)
    V_1 ← E(V(x)V(−x))
    A ← TransposedProj(⌊N/2⌋, V_1, d)
    if N is even then
        Z ← xA(x^2)
    else
        Z ← A(x^2)
    return (ZV(−x) div x^d) mod x^d
```

The adjoint identities prove correctness. The cost is $O(d \log(d+2) \log(N+2))$.

Theorem 513 (Recovering a modular power)**.** *Let $P = \operatorname{rev}_d V$, so $\deg P = d$, and let A be the state just computed. Then*

$$\boxed{x^N \bmod P = \operatorname{rev}_{d-1}(AV \bmod x^d).}$$

Proof. Write $x^N = QP + R$, with $\deg R < d$. Substitute x^{-1}, multiply by U/V, and take constant terms. The quotient term is $x^{-d}Q(x^{-1})U(x)$, which has only negative powers. Therefore

$$[x^N]\frac{U}{V} = [x^0]\frac{R(x^{-1})U(x)}{V(x)} = [x^{d-1}]U(x)\frac{\operatorname{rev}_{d-1} R(x)}{V(x)}.$$

By nondegeneracy of the coefficient pairing, $A \equiv \operatorname{rev}_{d-1} R/V \pmod{x^d}$, proving the formula. □

15.4 Composition and Compositional Inversion

15.4.1 Power Projection

For $P, W \in K[x]_{<n}$, power projection asks for $v_j = \sum_{i<n} w_i [x^i] P(x)^j$, $0 \le j < n$. Its generating function is a coefficient of a bivariate rational function:

$$\boxed{\sum_{j \ge 0} v_j y^j = [x^{n-1}]\frac{\operatorname{rev}_{n-1} W(x)}{1 - yP(x)}.}$$

This identity is formal in y, even when $P(0) \neq 0$. First reduce to $P(0) = 0$, so the final denominator is one. If $c = P(0)$, project onto $P - c$, obtaining $\widetilde{v}_j$. Then $v_j = \sum_{i=0}^{j} \binom{j}{i} c^{j-i} \widetilde{v}_i$, a single convolution after factorial scaling.

Assume now $P(0) = 0$. Pad P, W to length m, the least power of two at least n, and reverse W with bound $m-1$. Padding before reversal preserves the coefficient functional. Initialize $A = \operatorname{rev}_{m-1} W$, $B = 1 - yP$, $h = m$, $k = 1$.

At a stage maintain

$$\deg_x A, \deg_x B < h, \qquad \deg_y A < k, \quad \deg_y B \le k, \qquad hk = m, \qquad B(0, y) = 1.$$

The desired generating function is $[x^{h-1}]A/B$. Since $h - 1$ is odd for $h > 1$, one Bostan–Mori step is

$$\boxed{A_{\text{new}} = \mathcal{O}_x(AB(-x, y) \bmod x^h), \qquad B_{\text{new}} = \mathcal{E}_x(BB(-x, y) \bmod x^h).}$$

Replace (h, k) by $(h/2, 2k)$. Higher x-coefficients cannot affect the target. Truncating them halves the x-dimension, while the y-degree bounds double. The number of stored coefficients remains $O(m)$, and $B(0, y) = 1$ is preserved. At $h = 1$, the answer is $A(0, y)$.

Theorem 514 (Fast power projection)**.** *The first n projections are computable in $O(n \log^2(n+2))$ coefficient operations. For $P(0) = 0$, the algorithm uses only additions and multiplications and works over any commutative coefficient ring. The constant-term shift additionally uses factorial inverses.*

Proof. The rational coefficient identity and each parity step preserve the required generating function. If $P(0) = 0$, then $P^j = 0 \bmod x^m$ for $j \geq m$, so the terminal generating function is a polynomial of degree less than m.

At a stage, ordinary bivariate products fit in a rectangle of size $O(hk) = O(m)$, including constant-factor padding in both dimensions. Kronecker substitution converts them to univariate products of length $O(m)$. There are $\log_2 m$ stages. The initial constant-term shift, when needed, costs $O(n \log(n+2))$. □

Algorithm 103 Power projection with explicit precision bounds

```
1:  function PowerProj(W, P, n)
2:      if n = 0 then
3:          return the empty vector
4:      c ← P(0)
5:      if c ≠ 0 then
6:          u ← PowerProj(W, P − c, n)
7:          v_j ← j![y^j]((Σ_{i<n} u_i y^i/i!)(Σ_{i<n} c^i y^i/i!))
8:          return (v_j)_{j<n}
9:      m ← 2^⌈log_2 n⌉; pad P, W to length m
10:     A ← rev_{m−1} W, B ← 1 − yP, h ← m, k ← 1
11:     while h > 1 do
12:         C ← B(−x, y)
13:         A ← O_x(AC mod x^h)
14:         B ← E_x(BC mod x^h)
15:         h ← h/2, k ← 2k
16:     return the first n coefficients of A(0, y)
```

15.4.2 Composition by Transposition

For fixed P, let M_P have columns $\operatorname{coeff}(P^j \bmod x^n)$, $0 \leq j < n$. Composition is $M_P \operatorname{coeff}(F)$, while power projection is $M_P^{\mathsf{T}} \operatorname{coeff}(W)$. Only the numerator states in power projection depend on W, so transposition gives composition within the same time bound.

For a bivariate array Z, define

$$\mathrm{MP}_{h,k}(Z, C) = \sum_{i<h,\, j<k} x^i y^j \sum_{a,b \geq 0} c_{a,b} z_{i+a, j+b},$$

with out-of-range entries of Z set to zero. This is the transpose of multiplication by fixed C, followed by restriction to the appropriate output rectangle. Reversing both axes reduces it to one bivariate convolution.

At a forward stage of dimensions (h, k), the numerator map is $A \mapsto \mathcal{O}_x(AC \bmod x^h)$, with $C = B(-x, y)$. Its dot-product transpose is

$$\boxed{Z \longmapsto \mathrm{MP}_{h,k}(xZ(x^2, y), C).}$$

After upsampling and the middle product, retain x-length h and y-length k.

Theorem 515 (Fast truncated composition)**.** *For $F, P \in K[x]_{<n}$, the coefficients of $F(P(x)) \bmod x^n$ are computable in $O(n \log^2(n+2))$ operations. If $P(0) = 0$, only those prefixes of formal input series are needed. If $P(0) \neq 0$, the outer input must be a finite polynomial or be specified by its expansion about $P(0)$.*

Proof. Precompute the denominator states from power projection. Start the reverse pass with the coefficient weights of F, zero-padded to the terminal length m. Apply the displayed adjoint at every level. Finally reverse the length-m numerator state and retain its first n coefficients, transposing the initial padded reversal. The pairing identity at each stage proves that the result is $M_P \operatorname{coeff}(F)$.

All reversed products have $O(m)$ coefficients, as in the forward algorithm. For $P(0) = c \neq 0$, first replace $F(t)$ by $F(t+c)$ and P by $P - c$. The Taylor shift has smaller cost. □

Algorithm 104 Truncated composition

```
1:  function COMPOSE(F, P, n)
2:      if n = 0 then
3:          return zero
4:      c ← P(0)
5:      if c ≠ 0 then
6:          F(t) ← F(t + c), P ← P − c
7:      m ← 2^⌈log₂ n⌉; pad both inputs to length m
8:      B ← 1 − yP, h ← m, k ← 1, S ← ∅
9:      while h > 1 do
10:         Append (h, k, B(−x, y)) to S
11:         B ← E_x(B(x, y)B(−x, y) mod x^h)
12:         h ← h/2, k ← 2k
13:     Z(y) ← F(y)
14:     for (h, k, C) in S, in reverse order do
15:         Z ← MP_{h,k}(xZ(x², y), C)
16:     return rev_{m−1}(Z(x, 0)) mod x^n
```

Storing all denominator levels uses $O(n \log(n+2))$ space. For a nonzero constant substitution, high coefficients of the outer polynomial can affect the output constant term. Shift the entire outer polynomial before truncating it.

Frequency-domain implementation. The transform implementation includes the same coefficient truncation. Put $H = \mathcal{O}_x(A(x,y)B(-x,y))$ *before* truncation. When two is invertible,

$$H(\omega^2, y) = \frac{A(\omega,y)B(-\omega,y) - A(-\omega,y)B(\omega,y)}{2\omega}, \qquad A_{\text{new}} = H \bmod x^{h/2}.$$

Since $\deg_x A, \deg_x B < h$, an x-transform of length $2h$ represents the ordinary product without cyclic aliasing, and the paired values represent H with coefficient length h. The final crop retains only its first $h/2$ coefficients.

In the transposed computation, first embed the incoming length-$h/2$ coefficient state into length h by appending zeros; this is the transpose of that crop. Transpose the inverse transform, then transpose the displayed two-input combination. An incoming frequency weight Z contributes $ZB(-\omega, y)/(2\omega)$ and $-ZB(\omega, y)/(2\omega)$ to the antipodal entries. Finally transpose the forward transform and crop to the original coefficient rectangle. Bit-reversed transforms make the antipodal entries adjacent. The transform normalizations and the analogous y-dimension crops remain part of the map.

The pointwise calculation yields H; the coefficient truncation then gives A_{new}.

15.4.3 Lagrange Inversion

Lemma 516 (Lagrange inversion)**.** *Let $\Phi, H \in K[[x]]$, and let $f \in xK[[x]]$ be the unique solution of $f = x\Phi(f)$. In characteristic zero, for $n \geq 1$,*

$$\boxed{[x^n]H(f(x)) = \frac{1}{n}[x^{n-1}]H'(x)\Phi(x)^n.}$$

The constant coefficient is $H(0)$.

Proof. The equation determines each coefficient of f from earlier ones, proving existence and uniqueness. By linearity it suffices to prove $[x^n]f^k = (k/n)[x^{n-k}]\Phi^n$ for monomials $H = x^k$. Induct on n, simultaneously for all k. The cases $k = 0$, $k > n$, and $k = n$ follow from constant coefficients and $\operatorname{ord} f \geq 1$. For $1 \leq k < n$, use the induction hypothesis at index $n - k$ with outer series Φ^k:

$$\begin{aligned}
[x^n]f^k &= [x^{n-k}]\Phi(f)^k \\
&= \frac{k}{n-k}[x^{n-k-1}]\Phi'\Phi^{n-1} \\
&= \frac{k}{n(n-k)}[x^{n-k-1}](\Phi^n)' = \frac{k}{n}[x^{n-k}]\Phi^n.
\end{aligned}$$

Only finitely many coefficients of each outer series contribute, so the linearity reduction applies to formal series as well as polynomials. □

Corollary 517 (Lagrange inversion for a compositional inverse)**.** *If $F(0) = 0$, $F'(0) \neq 0$, and $G = F^{\langle -1\rangle}$, then for $n \geq 1$,*

$$[x^n]H(G(x)) = \frac{1}{n}[x^{n-1}]H'(x)\left(\frac{x}{F(x)}\right)^n.$$

Proof. The series $\Phi = x/F$ is a unit, and $G = x\Phi(G)$ is equivalent to $F(G) = x$. Apply the lemma. □

15.4.4 Compositional Inversion by One Power Projection

Suppose $F = x + O(x^2)$, $G = F^{\langle -1\rangle}$, and write $H = x/G$, so $H(0) = 1$. Applying Lagrange inversion with the roles of F, G exchanged gives

$$[x^m]F(x)^k = \frac{k}{m}[x^{m-k}]H(x)^m \qquad (1 \leq k \leq m).$$

Thus one power projection at coefficient x^m determines the first m coefficients of H^m:

$$\boxed{u_j = \frac{m}{m-j}[x^m]F(x)^{m-j}, \qquad 0 \leq j < m.}$$

Recover $G = x(\sum_{j<m} u_j x^j)^{-1/m} \bmod x^{m+1}$ by one normalized power. All scalar divisors here lie in $1, \ldots, m$.

For target precision x^N, take $m = N - 1$. The input $F \bmod x^N$ then suffices. If $F'(0) = c \neq 1$, normalize $\widetilde{F} = F/c$. Then $F^{\langle -1\rangle}(x) = \widetilde{F}^{\langle -1\rangle}(x/c)$.

Theorem 518 (Fast compositional inverse)**.** *Given $F \bmod x^N$, with $F(0) = 0$, $F'(0) \neq 0$, its compositional inverse modulo x^N is computable in $O(N \log^2(N + 2))$ operations, provided $1, \ldots, N - 1$ are invertible.*

Proof. The displayed Lagrange identity determines H^m to the required precision, and its constant coefficient is one. The normalized power therefore recovers the uniquely chosen H^{-1}. Power projection dominates the cost; normalization and the final coefficient scaling are linear-time operations. □

Algorithm 105 Compositional inverse

Input: $F \bmod x^N$, $F(0) = 0$, $F'(0) \neq 0$
Output: $F^{\langle -1 \rangle} \bmod x^N$

1: **if** $N \leq 1$ **then**
2: ⌊ **return** zero
3: $m \leftarrow N - 1$, $c \leftarrow F'(0)$, $\widetilde{F} \leftarrow F/c$
4: $v \leftarrow \textsc{PowerProj}(x^m, \widetilde{F}, N)$
5: $U \leftarrow \sum_{j=0}^{m-1} \frac{m}{m-j} v_{m-j} x^j$
6: $\widetilde{G} \leftarrow x \exp(-\frac{1}{m} \log U) \bmod x^N$
7: **return** $\widetilde{G}(x/c)$

15.5 Evaluation and Interpolation

15.5.1 Multipoint Evaluation by Transposition

Given $F \in K[x]_{<n}$ and points $\alpha_1, \ldots, \alpha_r$, we want all $F(\alpha_i)$. The points need not be distinct. The transpose of this Vandermonde map takes weights c_i to the first n coefficients of

$$\sum_{i=1}^{r} \frac{c_i}{1 - \alpha_i x} = \sum_{j \geq 0} \left(\sum_i c_i \alpha_i^j \right) x^j.$$

Evaluation requires no division by point differences.

Build a balanced tree on the points. For a node u containing s_u points, store $D_u = \prod_{i \in u} (1 - \alpha_i x)$, retaining degree bound s_u even when some $\alpha_i = 0$. The weighted rational sum has numerator N_u of degree less than s_u. At leaves $N_i = c_i$, and at an internal node

$$N_u = N_L D_R + N_R D_L, \qquad D_u = D_L D_R.$$

At the root, output $N_u / D_u \bmod x^n$. This computes the transposed evaluations in $O((n + r) \log^2(n + r + 2))$ operations: each tree level has total polynomial length $O(r)$, and the final inverse and product have length $O(n + r)$.

Transposing these linear numerator operations gives ordinary evaluation. Initialize $T_{\text{root}} = (F \circ D_{\text{root}}^{-1})_{<r}$, where the reciprocal is known modulo x^n. At a node, pass

$$\boxed{T_L = (T_u \circ D_R)_{<s_L}, \qquad T_R = (T_u \circ D_L)_{<s_R}.}$$

The coefficient $[x^0]T_i$ at leaf i is $F(\alpha_i)$. Each state has length equal to its node's point count.

Theorem 519 (Multipoint evaluation)**.** *A degree-$< n$ polynomial can be evaluated at r arbitrary points in $O((n + r) \log^2(n + r + 2))$ coefficient operations. The transposed evaluation map has the same complexity. Both constructions work over a commutative ring, since all series denominators have constant coefficient one.*

Proof. The upward rational-sum identity proves the transposed algorithm. At each internal node, $\langle N_L D_R + N_R D_L, T_u \rangle = \langle N_L, T_u \circ D_R \rangle + \langle N_R, T_u \circ D_L \rangle$, with the stated child dimensions. The root transpose and leaf extraction therefore prove evaluation. The degree sums at each level give the same cost for both directions. □

Algorithm 106 Multipoint evaluation

Input: $F \in K[x]_{<n}$, points $\alpha_1, \ldots, \alpha_r$
Output: $(F(\alpha_i))_{i=1}^{r}$

Build the balanced tree with $D_i = 1 - \alpha_i x$ and $D_u = D_L D_R$
$I \leftarrow D_{\text{root}}^{-1} \bmod x^n$
DESCEND(root, $(F \circ I)_{<r}$)

function DESCEND(u, T)
    if u is leaf i **then**
        Output $T(0)$ at position i
        return
    DESCEND($L, (T \circ D_R)_{<s_L}$)
    DESCEND($R, (T \circ D_L)_{<s_R}$)

An empty point list returns no values, and the zero polynomial returns zeros without constructing a tree. Storing the tree takes $O(r \log(r+2))$ coefficients. The same tree can be reused for several polynomials or weight vectors.

15.5.2 Interpolation

For distinct points $\alpha_1, \ldots, \alpha_n$, put $P(x) = \prod_i (x - \alpha_i)$. The unique degree-$< n$ interpolant for values y_i is

$$\boxed{F(x) = \sum_{i=1}^{n} \frac{y_i}{P'(\alpha_i)} \frac{P(x)}{x - \alpha_i}.}$$

Every summand vanishes at the other points and has value y_i at α_i; uniqueness follows because a nonzero polynomial of degree less than n cannot have n distinct roots.

Construct the ordinary monic product tree P_u, evaluate P' at all points, and set $c_i = y_i / P'(\alpha_i)$. At leaves put $N_i = c_i$; upward, use $N_u = N_L P_R + N_R P_L$. Inductively $N_u = \sum_{i \in u} c_i P_u / (x - \alpha_i)$, so the root is F. Both passes cost $O(n \log^2(n+2))$. The reversed tree required by the evaluation algorithm is obtained as $D_u = \operatorname{rev}_{s_u} P_u$, preserving the degree bounds.

Algorithm 107 Polynomial interpolation

Input: Distinct points $\alpha_1, \ldots, \alpha_n$, values $y_1, \ldots, y_n$
Output: Their degree-$< n$ interpolant

Build $P_i = x - \alpha_i$, $P_u = P_L P_R$
Evaluate P'_{root} at all α_i, obtaining v_i
Set $N_i \leftarrow y_i / v_i$ at each leaf
for internal nodes u, bottom-up **do**
    $N_u \leftarrow N_L P_R + N_R P_L$
return N_{root}

Over a commutative ring the same construction works when all differences $\alpha_i - \alpha_j$ are units; then every $P'(\alpha_i)$ is a unit.

15.5.3 Chirp-Z: Geometric Evaluation

Let $F(x) = \sum_{i=0}^{d-1} f_i x^i$, let $z \neq 0$, and request $F(1), F(z), \ldots, F(z^{k-1})$. The identity $ij = \binom{i+j}{2} - \binom{i}{2} - \binom{j}{2}$ gives

$$F(z^j) = z^{-\binom{j}{2}} \sum_{i=0}^{d-1} \bigl(f_i z^{-\binom{i}{2}}\bigr) z^{\binom{i+j}{2}}.$$

Set $A_i = f_i z^{-\binom{i}{2}}$ and $B_t = z^{\binom{t}{2}}$ for $0 \le t \le d+k-2$. The required sum is $[x^j](B \circ A)$. One middle product and two diagonal scalings evaluate all points in $O((d+k)\log(d+k+2))$ operations. Distinctness of the powers of z is unnecessary.

The factors $z^{\binom{i}{2}}$ are generated in linear time. Start with $u = v = 1$; after using u at index i, replace (u, v) by (uv, vz). Their inverse factors use z^{-1}.

Algorithm 108 Geometric evaluation

```
function ChirpZ(F, z, k)
    Require z ≠ 0; return the empty vector if k = 0
    Let d be the supplied coefficient length of F
    if d = 0 then
        return k zeros
    A_i ← f_i z^{-binom(i,2)} for 0 ≤ i < d
    B_t ← z^{binom(t,2)} for 0 ≤ t ≤ d + k − 2
    Y ← (B ∘ A)_{<k}
    return (z^{-binom(j,2)} Y_j)_{j=0}^{k-1}
```

Evaluation at $a, az, \ldots, az^{k-1}$ first scales f_i by a^i. The zero polynomial is handled directly.

15.5.4 Inverse Chirp-Z

Assume $z \neq 0$ and $1, z, \ldots, z^{k-1}$ are distinct, equivalently $z^j \neq 1$ for $1 \le j < k$. Given $y_i = F(z^i)$, with $\deg F < k$, let $D(x) = \prod_{j=0}^{k-1}(x - z^j)$. Factoring its Lagrange denominators gives

$$D'(z^i) = z^{i(k-1)} \prod_{m=1}^{i} (1 - z^{-m}) \prod_{m=1}^{k-1-i} (1 - z^m).$$

Thus all weights $c_i = y_i / D'(z^i)$ are obtained from two prefix product arrays and successive powers of $z^{-(k-1)}$, in linear time.

Let $Q(x) = \prod_{j=0}^{k-1}(1 - z^j x)$. Reversing the interpolant in the Lagrange formula yields

$$\frac{\operatorname{rev}_{k-1} F(x)}{Q(x)} = \sum_{i=0}^{k-1} \frac{c_i}{1 - z^i x}.$$

The first k coefficients on the right are $s_m = \sum_i c_i z^{im}$, one Chirp-Z evaluation of the polynomial with coefficients c_i. Hence $\operatorname{rev}_{k-1} F = (\sum_{m<k} s_m x^m) Q \bmod x^k$.

The needed coefficients of Q are also linear-time computable. From $(1-x)Q(zx) = (1 - z^k x)Q(x)$, coefficient comparison gives

$$\boxed{q_0 = 1, \qquad q_j = -z^{j-1} \frac{1 - z^{k-j+1}}{1 - z^j} q_{j-1} \quad (1 \le j < k).}$$

Every denominator is nonzero; $z^k = 1$ is allowed, since $1 - z^k$ occurs only in a numerator.

Theorem 520 (Geometric interpolation)**.** *A degree-$< k$ polynomial is reconstructed from its values at $1, z, \dots, z^{k-1}$ in $O(k \log(k+2))$ operations under the preceding distinctness assumptions.*

Proof. Compute the Lagrange weights, perform one Chirp-Z transform, construct $Q \bmod x^k$ by its recurrence, multiply modulo x^k, and reverse with bound $k-1$. The established identities prove correctness. □

Algorithm 109 Geometric interpolation

Input: $k \geq 1$, $y_i = F(z^i)$, $0 \leq i < k$, $\deg F < k$, distinct nonzero nodes
Output: F

1: $P_0^+ = P_0^- \leftarrow 1$
2: **for** $j = 1, \dots, k-1$ **do**
3: $\quad P_j^+ \leftarrow P_{j-1}^+/(1-z^j)$, $P_j^- \leftarrow P_{j-1}^-/(1-z^{-j})$
4: $c_i \leftarrow y_i z^{-i(k-1)} P_i^- P_{k-1-i}^+$ for $0 \leq i < k$
5: $s \leftarrow \textsc{ChirpZ}(\sum_{i<k} c_i x^i, z, k)$
6: Compute $Q \bmod x^k$ by the coefficient recurrence above
7: **return** $\mathrm{rev}_{k-1}((\sum_{i<k} s_i x^i) Q \bmod x^k)$

15.5.5 Hermite Interpolation

Hermite interpolation specifies several Taylor coefficients at each point. Hasse derivatives make this independent of the characteristic: define $D^{(j)} f(a) = [z^j] f(a+z)$. When $j!$ is invertible, this is $f^{(j)}(a)/j!$, but the definition never requires that division.

Lemma 521 (Taylor data as a remainder)**.** *Over any commutative ring R,*

$$f(x) \equiv \sum_{j=0}^{s-1} D^{(j)} f(a)(x-a)^j \pmod{(x-a)^s}.$$

Proof. Substitute $z = x - a$ in $f(a+z) = \sum_j D^{(j)} f(a) z^j$, then omit the terms divisible by $(x-a)^s$. □

Lemma 522 (Coprimality of the local moduli)**.** *If $a - b$ is a unit, the ideals $((x-a)^r)$ and $((x-b)^s)$ are comaximal for all $r, s \geq 1$.*

Proof. The ideals $I = (x-a)$, $J = (x-b)$ contain elements u, v with $u+v = 1$, because $(x-a)-(x-b) = b-a$ is a unit. Every term of $(u+v)^{r+s-1} = 1$ lies in I^r or J^s. □

Theorem 523 (Hermite interpolation)**.** *Let $a_1, \dots, a_k \in R$ have unit pairwise differences, and let $s_i \geq 1$, $n = \sum_i s_i$. For arbitrary prescribed $y_{i,j} \in R$, $0 \leq j < s_i$, there is a unique polynomial of degree less than n satisfying $D^{(j)} f(a_i) = y_{i,j}$.*

Proof. Set $P_i = (x-a_i)^{s_i}$ and $T_i = \sum_{j<s_i} y_{i,j}(x-a_i)^j$. The Taylor conditions are $f \equiv T_i \pmod{P_i}$. The preceding lemma and the Chinese remainder theorem give a unique class modulo the monic degree-n polynomial $P = \prod_i P_i$, which has a unique representative of degree less than n. □

Product-tree construction. The interpolation formula is

$$f = \sum_i C_i \frac{P}{P_i}, \qquad C_i = T_i \left(\frac{P}{P_i}\right)^{-1} \bmod P_i.$$

Each summand has degree less than n. To compute every local inverse, build the product tree P_u, and propagate $q_u = (P/P_u) \bmod P_u$ downward. At the root $q_u = 1$, and

$$\boxed{q_L = q_u P_R \bmod P_L, \qquad q_R = q_u P_L \bmod P_R.}$$

The parent residue suffices because both child moduli divide P_u. At leaf i, shift q_i to $a_i + z$, invert modulo z^{s_i}, and multiply by $\sum_{j<s_i} y_{i,j} z^j$. Shift back to obtain C_i. Its inverse exists because $q_i(a_i) = \prod_{j \neq i} (a_i - a_j)^{s_j}$ is a unit.

Finally set $F_i = C_i$ at leaves and propagate upward by

$$\boxed{F_u = F_L P_R + F_R P_L.}$$

Induction gives $F_u = \sum_{i \in u} C_i P_u / P_i$, so the root is the required CRT sum.

Theorem 524 (Fast Hermite evaluation and interpolation)**.** *With total multiplicity n, Hermite interpolation takes $O(n \log^2(n+2))$ ring operations and inversions of the stated units. The first s_i Taylor coefficients of a given degree-$< n$ polynomial at each a_i can be computed within the same bound. No factorial denominators are required.*

Proof. Build each P_i by powering and the product tree by balanced multiplication. In each upward or downward pass, node degrees sum to n at every depth, so all products and monic reductions cost $O(n \log^2(n+2))$. At the leaves, use division-free Taylor shifts, followed by ordinary unit-series inversion. Their total cost is at most $\sum_i O(s_i \log^2(s_i+2)) = O(n \log^2(n+2))$.

For evaluation, propagate $f \bmod P_u$ downward through the monic product tree, then shift each leaf remainder by a_i. These are exactly its Taylor coefficients by Lemma 521. The same degree-sum argument applies. □

Algorithm 110 Hermite interpolation over a commutative ring

Input: Points a_i with unit differences, multiplicities s_i, Taylor data $y_{i,j}$
Output: The interpolant of degree less than $n = \sum_i s_i$

Build a balanced product tree with leaves $P_i = (x - a_i)^{s_i}$
$q_{\text{root}} \leftarrow 1$
for internal nodes u, top-down **do**
    $q_L \leftarrow q_u P_R \bmod P_L$, $q_R \leftarrow q_u P_L \bmod P_R$
for leaves i **do**
    $u_i(z) \leftarrow q_i(a_i + z)$, by a division-free Taylor shift
    $v_i(z) \leftarrow (\sum_{j<s_i} y_{i,j} z^j)/u_i(z) \bmod z^{s_i}$
    $F_i(x) \leftarrow v_i(x - a_i)$, by a division-free Taylor shift
for internal nodes u, bottom-up **do**
    $F_u \leftarrow F_L P_R + F_R P_L$
return F_{root}

15.6 Multivariate Evaluation by Integer Lifting

Over a finite field or a residue ring, let $f(x_1, \ldots, x_m)$ have individual degree less than d, so its dense representation has d^m coefficients. Given arbitrary points $a_1, \ldots, a_N$, we want all $f(a_i)$. Reduce the modulus until the complete residue grid is small, evaluate on that grid, then reconstruct the original answers by Chinese remaindering. Using prime powers and nilpotent Taylor expansion removes a factor m from the grid side length. For extension fields, the same Taylor expansion supplies data for Hermite interpolation.

The degree bound $d = 1$ is the constant-polynomial case; empty query lists are also immediate. In the recursive reductions assume $d \geq 2$, $m \geq 1$, and $N \geq 1$. All iterated logarithms are clipped below at one. Soft-O bounds below hide logarithms of the parameters and their bit lengths, but not factors exponential in m.

15.6.1 Product Grids and Nilpotent Taylor Expansion

Lemma 525 (Product-grid evaluation)**.** *Let $A_i \subseteq R$ be nonempty finite sets in a commutative ring, with $q_i = |A_i|$ and $Q = \prod_i q_i$. An m-variate polynomial of individual degree less than d can be evaluated on $A_1 \times \cdots \times A_m$ in*

$$\boxed{O\left(m(d^m + Q)\log^2(d + \max_i q_i + 2)\right)}$$

ring operations. The points in a coordinate set need not have unit differences.

Proof. Order the variables so that $q_1 \le \cdots \le q_m$. After evaluating the first i variables, the coefficient/evaluation array has size $D_i = q_1 \cdots q_i d^{m-i}$, with $D_0 = d^m$ and $D_m = Q$. The next stage evaluates $q_1 \cdots q_{i-1} d^{m-i}$ univariate polynomials of degree less than d at q_i points. By Theorem 519, it costs $O((D_{i-1} + D_i)\log^2(d + q_i + 2))$.

The ratios $D_i/D_{i-1} = q_i/d$ are nondecreasing, so the array sizes first decrease and then increase. None exceeds $\max\{d^m, Q\}$. Summing the stage costs proves the bound. The computation uses only evaluation, so point differences need not be units. □

For a multi-index $\alpha \in \mathbb{N}^m$, define the multivariate Hasse derivative by

$$f(x+z) = \sum_{\alpha \ge 0} (\partial^\alpha f)(x) z^\alpha.$$

If $f = \sum_\beta c_\beta x^\beta$, then $\partial^\alpha f = \sum_{\beta \ge \alpha} \binom{\beta}{\alpha} c_\beta x^{\beta-\alpha}$. The integer binomial coefficients make the formula valid in every characteristic, without factorial inverses.

Lemma 526 (Nilpotent Taylor expansion)**.** *Let I be an ideal of a commutative ring R, with $I^s = 0$. If $a, b \in R^m$ differ coordinatewise by elements of I, then*

$$\boxed{f(a) = \sum_{|\alpha| < s} (\partial^\alpha f)(b)(a-b)^\alpha.}$$

Proof. Substitute $z = a - b$ in the Hasse–Taylor identity. For $|\alpha| \ge s$, the factor $(a-b)^\alpha$ lies in $I^{|\alpha|} \subseteq I^s = 0$. □

We will always have $s \le m$. The number of needed derivatives is

$$\binom{m+s-1}{s-1} \le \binom{2m}{m} < 4^m.$$

They can be constructed from the coefficient formula in $O(4^m m d^m)$ ring operations. Pascal's recurrence supplies the needed binomial coefficients without division. For each query, powers $(a-b)^\alpha$ can be generated by extending one smaller multi-index at a time, rather than exponentiating each monomial independently.

15.6.2 Recursive Prime-Power Reduction

Reduction modulo primes. For coefficients and points modulo r, lift their representatives to $\{0, \ldots, r-1\}$. The integer value of the lifted polynomial is less than $d^m r^{dm}$. It is therefore determined by its residues modulo primes whose product exceeds this bound. The small-prime-product theorem permits primes of size $O(md(1 + \log r))$. Repeating the reduction decreases the modulus, but a terminal full grid inherits a side length proportional to md, up to logarithmic factors. Its extra m^m factor is expensive when there are many variables.

Using prime powers increases the reconstruction modulus without enlarging the base residue grid.

Lemma 527 (One prime-power reduction)**.** *Let $f \in (\mathbb{Z}/r^s\mathbb{Z})[x_1, \ldots, x_m]$, with $r \geq 2$, $1 \leq s \leq m$, and individual degree less than $d \geq 2$. Evaluation at N points reduces to fewer than 4^m families of evaluations over rings $\mathbb{Z}/p^m\mathbb{Z}$, where*

$$p \leq Cd(1 + \log r)$$

for an absolute constant C. Each family contains one problem per chosen prime, on the same number N of points.

Proof. Let $\bar{a}_i \in \{0, \ldots, r-1\}^m$ be the coordinatewise residue of a_i modulo r. The ideal generated by r in $\mathbb{Z}/r^s\mathbb{Z}$ has s-th power zero, so

$$f(a_i) = \sum_{|\alpha|<s} (\partial^\alpha f)(\bar{a}_i)(a_i - \bar{a}_i)^\alpha.$$

For each needed derivative, choose its integer lift F_α with coefficients in $[0, r^s)$. Then

$$0 \leq F_\alpha(\bar{a}_i) < d^m r^{s+m(d-1)} \leq (dr^d)^m.$$

Choose distinct primes $p \leq Cd(1 + \log r)$ with $\prod p > dr^d$, using the small-prime-product bound. Their m-th powers satisfy

$$\prod p^m > (dr^d)^m > F_\alpha(\bar{a}_i).$$

Thus evaluating F_α at $\bar{a}_i$ modulo every p^m and applying CRT recovers the integer value exactly. Reduce it modulo r^s, then use the Taylor formula. There are fewer than 4^m derivative indices. □

The contrast is between $\prod p > (dr^d)^m$ and $\prod p^m > (dr^d)^m$. The second inequality needs only $\prod p > dr^d$, so the base primes no longer carry the factor m. The recursive points are reduced modulo p^m, not merely modulo p; the next Taylor expansion handles their higher digits.

Base case and recursion. At depth zero, reduce every query point modulo the current base r. Evaluate all needed Hasse derivatives on the full grid $\{0, \ldots, r-1\}^m$, over the ring $\mathbb{Z}/r^s\mathbb{Z}$, and reconstruct the query values by nilpotent Taylor expansion. Grid evaluation works even when r is composite.

Algorithm 111 Recursive multivariate evaluation over a prime-power ring

```
function EvaluatePrimePower(f, (a_i)_{i=1}^N, r, s, t)
    ā_i ← a_i mod r, with coordinates in {0, ..., r − 1}
    for multi-indices α with |α| < s do
        Compute ∂^α f by its Hasse coefficient formula
    if t = 0 then
        for |α| < s do
            Evaluate ∂^α f on {0, ..., r − 1}^m over Z/r^s Z
            Read its values at ā_i, obtaining v_{α,i}
    else
        Choose distinct primes p ≤ Cd(1 + log r) with ∏ p > dr^d
        for |α| < s do
            Lift ∂^α f to F_α ∈ Z[x_1, ..., x_m] with coefficients in [0, r^s)
            for each chosen prime p do
                v_{p,i} ← EvaluatePrimePower(F_α mod p^m, (ā_i mod p^m)_i, p, m, t − 1)
            Recover F_α(ā_i) as an integer by CRT modulo ∏ p^m
            v_{α,i} ← F_α(ā_i) mod r^s
    return (∑_{|α|<s} v_{α,i}(a_i − ā_i)^α mod r^s)_{i=1}^N
```

Modulus size and branching. Each reduction replaces the base modulus by at most $Cd(1+\log r)$. Count both the derivative indices and the auxiliary primes when bounding the number of children, retaining their dependence on m, d.

Lemma 528 (Modulus shrinkage)**.** *After a fixed number $t \geq 1$ of reductions, every base modulus is*

$$O_t\big(d(1+\log d+\log^{\circ t} r)\big).$$

The product of the branching bounds contributed by the auxiliary primes alone is $\widetilde{O}_t(d^t(1+\log r))$.

Proof. Set $R_0 = r$ and $R_{j+1} = \lceil Cd(1+\log R_j)\rceil$. Induction gives $R_j = O_j(d(1+\log d+\log^{\circ j} r))$. There are at most R_{j+1} prime choices at level j. The first factor is $O(d(1+\log r))$; each later factor is d times a polynomial in $\log(d+2)$ and $\log(\log r+2)$, for fixed t. Multiplying gives the claim. □

Theorem 529 (Evaluation over prime-power rings)**.** *For fixed $t \geq 1$, the preceding algorithm evaluates $f \in (\mathbb{Z}/r^s\mathbb{Z})[x_1,\dots,x_m]$, with $1 \leq s \leq m$, at N points in*

$$\boxed{\widetilde{O}_t\left(2^{O((t+1)m)}m^3d^t(1+\log r)^2\left([d(1+\log d+\log^{\circ t} r)]^m+N\right)\right)}$$

bit operations.

Proof. With the bounds R_j from the lemma, the number of leaf instances is at most $4^{mt}\prod_{j=1}^{t} R_j$. A leaf evaluates fewer than 4^m derivatives on a grid of at most R_t^m points. Product-grid evaluation, derivative construction, and the final Taylor sums therefore have total cost bounded by

$$\widetilde{O}_t\left(4^{m(t+1)}m^3(1+\log r+\log d)\left(\prod_{j=1}^{t} R_j\right)(d^m+R_t^m+N)\right)$$

bit operations: a residue at level j has $O(m\log R_j)$ bits, and fast integer arithmetic supplies the polynomial products and reductions.

The same expression covers the internal nodes. Their derivative and query arrays have $O(4^m(d^m+mN))$ entries per node. Reducing them into all child rings costs quasi-linear time in their input and output bit lengths. Balanced CRT combines the child residues in quasi-linear time in the aggregate modulus bit length. The reconstruction cost is therefore charged to the aggregate bit length of the child data.

There are only t internal depths. Substitute the bounds on R_t and $\prod_j R_j$ from Lemma 528, absorbing powers of logarithms of d, m, and of bit lengths in soft-O. The remaining two factors $1+\log r$ give the stated bound. □

15.6.3 Extension Fields

Suppose $\mathbb{F}_q = \mathbb{F}_p[z]/(E(z))$, with $e = \deg E$. Represent each field element by a degree-$< e$ polynomial whose integer coefficients lie in $\{0,\dots,p-1\}$. Lift the input polynomial to $F(x,z) \in \mathbb{Z}[x_1,\dots,x_m,z]$ and each query to $\widetilde{a}_i(z) \in \mathbb{Z}[z]^m$. The integer polynomial

$$G_i(z) = F(\widetilde{a}_i(z), z)$$

satisfies $f(a_i) = G_i \bmod (p,E)$, and

$$\deg G_i < edm, \qquad \|G_i\|_1 < d^m(ep)^{m(d-1)+1} \leq \big(d(ep)^d\big)^m.$$

The norm bound follows by submultiplicativity of the coefficient ℓ_1-norm. All lifted coefficients are nonnegative.

Recover each G_i from its first m Taylor coefficients at ed points by Hermite interpolation.

Lemma 530 (Extension-field reduction)**.** *There is an integer r' with*

$$d(ep)^d \le r' < 2ed^2(ep)^d$$

such that the extension-field evaluation problem reduces to fewer than $e4^m$ evaluations over $\mathbb{Z}/(r')^m\mathbb{Z}$, each with edN query points. The additional work is

$$\widetilde{O}\left(e4^m m d^m + e^2 d\, 4^m m^2 N + Nedm\right)$$

operations on auxiliary residues, together with the corresponding coefficient lifts and reductions.

Proof. Choose a prime P with $ed \le P < 2ed$, and let r' be its smallest power at least $d(ep)^d$. This gives the stated interval for r'. Work in $S = \mathbb{Z}/(r')^m\mathbb{Z}$. The ed points $j = 0, \dots, ed-1$ have unit pairwise differences in S.

Write $F(x,z) = \sum_{u=0}^{e-1} f_u(x)z^u$, and set $b_{i,j} = \widetilde{a}_i(j) \in S^m$. Let $A_{i,j}(t) = \widetilde{a}_i(j+t) - b_{i,j} \bmod t^m$. Every coordinate of this vector is divisible by t. Applying nilpotent Taylor expansion in $S[t]/(t^m)$ gives

$$\boxed{G_i(j+t) = \sum_{u=0}^{e-1}(j+t)^u \sum_{|\alpha|<m} (\partial^\alpha f_u)(b_{i,j})A_{i,j}(t)^\alpha \pmod{t^m}.}$$

Thus fewer than $e4^m$ derivative polynomials, each evaluated at the edN points $b_{i,j}$, supply all the required Taylor data.

For each i, Hermite interpolation at the ed points, each of multiplicity m, recovers $G_i \bmod (r')^m$, since $\deg G_i < edm$. Also $\|G_i\|_1 < (r')^m$, so canonical residue representatives recover its integer coefficients exactly. Finally reduce modulo p and E.

Constructing the derivatives costs $O(e4^m m d^m)$. Obtain the coordinate jets $\widetilde{a}_i(j+t) \bmod t^m$ by Hermite evaluation, and generate the products $A_{i,j}^\alpha$ by extending multi-indices. Their assembly costs $\widetilde{O}(e^2 d\, 4^m m^2 N)$ operations. Hermite reconstruction costs $O(edm\log^2(edm+2))$ per output. All local polynomial operations use monic division and the unit point differences, not factorial inverses. □

Algorithm 112 Reduce extension-field queries to prime-power rings

Input: f and $a_1, \dots, a_N$ over $\mathbb{F}_p[z]/(E)$, recursion depth ℓ
Output: $f(a_1), \dots, f(a_N)$

```
Form the canonical integer lifts F(x,z) = Σ_{u<e} f_u(x)z^u and ã_i(z)
Choose P, r' as in Lemma 530; set S = Z/(r')^m Z
Compute b_{i,j} = ã_i(j) and A_{i,j}(t) = ã_i(j+t) − b_{i,j} mod t^m for 0 ≤ j < ed
for u = 0, ..., e−1 and |α| < m do
    Evaluate ∂^α f_u at all b_{i,j} using EvaluatePrimePower(∂^α f_u, (b_{i,j}), r', m, ℓ)
for i = 1, ..., N do
    Assemble G_i(j+t) mod t^m for each j by the nilpotent Taylor formula
    Reconstruct G_i mod (r')^m by Hermite interpolation
    Lift its coefficients to [0, (r')^m) and reduce modulo p, E
```

Theorem 531 (Elementary multivariate evaluation)**.** *Let $d \ge 2$, and suppose $q \le \exp^{\circ h}(d)$ for a fixed $h \ge 1$. Given $\mathbb{F}_q = \mathbb{F}_p[z]/(E(z))$, with $e = \deg E$, a dense m-variate polynomial of individual degree less than d can be evaluated deterministically at N points in*

$$\boxed{\widetilde{O}_h\left(2^{O(hm)} e m^3 d^{h+3}(1+\log q)^2\left([d\log(d+2)]^m + edN\right)\right)}$$

bit operations. In particular, the bound has the form $(d^m+N)^{1+o(1)}\,\mathrm{poly}_h(m,d,\log q)$ as $d \to \infty$.

Proof. Apply the extension-field reduction, then use $t = h+1$ recursive prime-power levels in each of its fewer than $e4^m$ instances. Its auxiliary modulus satisfies $\log r' = O(d(1+\log q)+\log d)$, so $\log^{\circ(h+1)} r' = O(1+\log d)$. The terminal grid size is therefore at most $[C_h d \log(d+2)]^m$.

In Theorem 529, the factor $d^t(1+\log r')^2$ is $\widetilde{O}_h(d^{h+3}(1+\log q)^2)$. Multiply by the number $e4^m$ of reduced instances, retaining their edN query count. Their local Taylor computations, Hermite interpolations, and residue bit lengths are covered by the same bound. This gives the displayed expression.

Finally, $2^{O(hm)} \log(d+2)^m = d^{o(m)}$ for fixed h, and $e \le \log_2 q$. These observations give the last formulation. The field representation is supplied; constructing its defining irreducible polynomial is not included. □

15.7 Multilinear Polynomials

15.7.1 Multilinear-Term Detection

Let $\mathbb{F}$ be a field of characteristic 2, and let $f \in \mathbb{F}[x_1, \dots, x_n]$ have total degree at most k.

For $r = 1, \dots, k$, introduce a vector of new variables

$$z_r = (z_{r,1}, \dots, z_{r,n}).$$

For $J \subseteq [k]$, write $z_J = \sum_{r \in J} z_r$.

Define

$$\boxed{\phi_k f(z_1, \dots, z_k) = \sum_{J \subseteq [k]} f(z_J).}$$

Thus $\phi_k f$ is a polynomial in the kn variables $z_{r,j}$.

This operator is a mixed forward difference.

Forward-Difference Interpretation For a direction $h \in \mathbb{F}^n$, define

$$\Delta_h f(x) = f(x+h) - f(x).$$

Iterating gives

$$(\Delta_{z_1} \cdots \Delta_{z_k} f)(0) = \sum_{J \subseteq [k]} (-1)^{k-|J|} f(z_J).$$

Since $\operatorname{char} \mathbb{F} = 2$, the signs disappear, so

$$\boxed{\phi_k f(z_1, \dots, z_k) = (\Delta_{z_1} \cdots \Delta_{z_k} f)(0).}$$

Thus ϕ_k is exactly the k-fold mixed finite difference of f.

To determine its action on monomials, introduce scalar variables $t_1, \dots, t_k$ and define

$$P_z(t_1, \dots, t_k) = f\left(\sum_{r=1}^{k} t_r z_r\right).$$

For $J \subseteq [k]$, let $1_J \in \{0,1\}^k$ be its indicator vector. Then

$$P_z(1_J) = f\left(\sum_{r \in J} z_r\right) = f(z_J),$$

and consequently

$$\phi_k f(z_1, \dots, z_k) = \sum_{J \subseteq [k]} P_z(1_J).$$

Lemma 532 (Mixed Difference as Coefficient Extraction)**.** *If* $\deg f \le k$*, then*

$$\boxed{\phi_k f(z_1, \dots, z_k) = [t_1 \cdots t_k]\, f\left(\sum_{r=1}^{k} t_r z_r\right).}$$

Proof. Since $\deg f \le k$, the polynomial $P_z(t)$ has total degree at most k.

Consider a monomial

$$c_\alpha(z) t_1^{\alpha_1} \cdots t_k^{\alpha_k}$$

of P_z. When we sum over $t = 1_J$, this monomial contributes $c_\alpha(z)$ exactly for those J containing every r with $\alpha_r > 0$. Hence its total contribution is

$$2^{k - |\operatorname{supp}(\alpha)|} c_\alpha(z).$$

In characteristic 2, this is zero unless

$$|\operatorname{supp}(\alpha)| = k.$$

A surviving monomial must involve all $t_1, \dots, t_k$. Since P_z has degree at most k, its k variables must each have exponent one: their total degree is already k. The only surviving monomial is

$$t_1 t_2 \cdots t_k.$$

Taking its coefficient proves the identity. □

Equivalently, the k-fold finite difference extracts the part of $f(t_1 z_1 + \cdots + t_k z_k)$ in which every direction z_r is used exactly once.

Lemma 533. *Let*

$$M = x_{i_1} \cdots x_{i_d}$$

be a monomial of degree $d \le k$*.*

1. *If* $d < k$*, then* $\phi_k M = 0$*.*
2. *If* $d = k$*, then*

$$\boxed{\phi_k M = \operatorname{perm} Z_M = \det Z_M,}$$

where

$$Z_M = \begin{pmatrix} z_{1,i_1} & z_{1,i_2} & \cdots & z_{1,i_k} \\ z_{2,i_1} & z_{2,i_2} & \cdots & z_{2,i_k} \\ \vdots & \vdots & & \vdots \\ z_{k,i_1} & z_{k,i_2} & \cdots & z_{k,i_k} \end{pmatrix}.$$

In particular, $\phi_k M = 0$ *if* M *is nonmultilinear, while* $\phi_k M \not\equiv 0$ *if* M *is multilinear.*

Proof. By Lemma 532,

$$\phi_k M = [t_1 \cdots t_k] \prod_{\ell=1}^{d} \left(\sum_{r=1}^{k} t_r z_{r,i_\ell}\right).$$

If $d < k$, this product has degree d, so it cannot contain $t_1 \cdots t_k$. Hence $\phi_k M = 0$.

Now suppose $d = k$. To obtain $t_1 \cdots t_k$, the k factors must choose the k variables $t_1, \dots, t_k$ exactly once each. These choices are indexed by permutations $\sigma \in S_k$, so

$$\phi_k M = \sum_{\sigma \in S_k} \prod_{\ell=1}^{k} z_{\sigma(\ell), i_\ell} = \operatorname{perm} Z_M.$$

In characteristic 2, $1 = -1$, so the signs in the determinant disappear:

$$\operatorname{perm} Z_M = \det Z_M.$$

If M is nonmultilinear, then $i_a = i_b$ for some $a \neq b$. The corresponding columns of Z_M are identical, so $\det Z_M = 0$.

If M is multilinear, then $i_1, \ldots, i_k$ are distinct. The determinant is nonzero: for example, the identity-permutation term

$$z_{1,i_1} z_{2,i_2} \cdots z_{k,i_k}$$

does not occur from any other permutation. □

Finite differences annihilate degrees below k. In characteristic 2, the surviving permanent is a determinant, so repeated original variables give repeated columns and are annihilated as well.

Theorem 534 (Multilinear Detection Operator)**.** *Let $f \in \mathbb{F}[x_1, \ldots, x_n]$ have degree at most k, where $\operatorname{char} \mathbb{F} = 2$. Then*

$$\boxed{\phi_k f \not\equiv 0 \iff f \text{ contains a degree-}k \text{ multilinear monomial.}}$$

Proof. Write

$$f = \sum_{I \in \binom{[n]}{k}} c_I x_I + g,$$

where g contains no degree-k multilinear monomial.

By Lemma 533, every term of degree less than k and every nonmultilinear term of degree k is annihilated. Therefore

$$\phi_k f = \sum_{I \in \binom{[n]}{k}} c_I \det Z_I.$$

Different I's cannot cancel. Indeed, every monomial in $\det Z_I$ contains exactly one variable from each row and has second indices precisely the elements of I. Thus if $I \neq I'$, the polynomials $\det Z_I$ and $\det Z_{I'}$ have disjoint monomial supports.

Hence $\phi_k f \equiv 0$ exactly when every $c_I = 0$. □

Black-Box Detection Evaluating $\phi_k f$ requires only evaluations of f. Evaluate f at the 2^k vertices

$$z_J = \sum_{r \in J} z_r \qquad (J \subseteq [k])$$

of the k-dimensional parallelepiped generated by $z_1, \ldots, z_k$.

Theorem 535 (Black-Box Multilinear-Term Detection)**.** *Let*

$$f(x, y) \in \mathbb{F}[x_1, \ldots, x_n, y_1, \ldots, y_m],$$

where $\operatorname{char} \mathbb{F} = 2$. Suppose f has degree at most k in the x-variables and degree at most d in the y-variables. There is a randomized one-sided-error algorithm deciding whether f contains a monomial that is multilinear of degree k in x.

One test uses exactly 2^k black-box evaluations of f. If every random coordinate is chosen independently and uniformly from a finite $S \subseteq \mathbb{F}$, then whenever such a monomial exists, the test succeeds with probability at least

$$\boxed{1 - \frac{d+k}{|S|}.}$$

Proof. Define the operator acting only on x by

$$\phi_k^x f(z_1, \ldots, z_k, y) = \sum_{J \subseteq [k]} f(z_J, y).$$

Write the degree-k multilinear part in x as

$$f(x, y) = \sum_{I \in \binom{[n]}{k}} \lambda_I(y) x_I + g(x, y),$$

where g contains no degree-k multilinear monomial in x. Then

$$\phi_k^x f(z, y) = \sum_I \lambda_I(y) \det Z_I.$$

Since the determinant supports for distinct I's are disjoint,

$$\phi_k^x f \not\equiv 0$$

if and only if at least one coefficient polynomial $\lambda_I(y)$ is nonzero.
Each determinant has degree k in the z-variables, while each λ_I has degree at most d in y. Hence

$$\deg(\phi_k^x f) \leq d + k.$$

Schwartz–Zippel therefore gives

$$\Pr[\phi_k^x f(z_1, \ldots, z_k, y) = 0] \leq \frac{d+k}{|S|}$$

whenever the polynomial is nonzero.
For fixed $z_1, \ldots, z_k, y$, its value is

$$\phi_k^x f(z_1, \ldots, z_k, y) = \sum_{J \subseteq [k]} f\left(\sum_{r \in J} z_r, y\right),$$

using one black-box query for each of the 2^k subsets $J \subseteq [k]$.
If no desired monomial exists, then $\phi_k^x f \equiv 0$, so the algorithm never produces a false positive. □

Algorithm 113 Black-Box Multilinear-Term Detection

Input: Black-box access to $f(x, y)$, target degree k, finite set S
Output: Whether f contains a degree-k multilinear monomial in x

```
Choose every coordinate of z_1, ..., z_k and y independently and uniformly from S
R ← 0
for every J ⊆ [k] do
    x^(J) ← Σ_{r∈J} z_r
    R ← R + f(x^(J), y)
if R ≠ 0 then
    return Yes
else
    return No
```

Independent repetitions decrease the false-negative probability geometrically. A larger finite extension of characteristic 2 may be used when the evaluation procedure accepts extension-field inputs, as an arithmetic circuit does. This is an additional requirement for a black box restricted to base-field inputs.

15.8 Algebraic Series, D-Finiteness, and Recurrence Algorithms

Rational series correspond to constant-coefficient recurrences, and D-finite series to polynomial-coefficient recurrences. Algebraic equations and diagonal representations provide ways to derive these recurrences. Over finite fields, rational diagonals also permit coefficient extraction by processing the digits of the index.

Throughout, complexity counts field operations. The coefficient field has characteristic zero except in the explicitly finite-field part. In evaluation bounds, differential orders, recurrence orders, and matrix dimensions are fixed unless stated otherwise. Evaluation algorithms take the defining equation and the coefficient data selecting its solution as input.

Write $D = d/dx$, $\theta = xD$, and $z_{\underline{i}} = z(z-1)\cdots(z-i+1)$, with $z_{\underline{0}} = 1$.

15.8.1 Differential Equations and Coefficient Recurrences

A series $F \in K[[x]]$ is *D-finite* if

$$\sum_{i=0}^{r} p_i(x) F^{(i)}(x) = 0$$

for polynomials p_i, not all zero.

Equivalently, its derivatives span a finite-dimensional space over $K(x)$. A finite-dimensional span gives an equation by dependence. Conversely, solve an equation with $p_r \neq 0$ for $F^{(r)}$ and differentiate repeatedly; all later derivatives remain in the span of $F, \ldots, F^{(r-1)}$.

A sequence $(a_n)_{n\geq 0}$ is *P-recursive* if

$$\sum_{j=0}^{s} q_j(n) a_{n+j} = 0$$

for all sufficiently large n, where the q_j's are polynomials, not all zero. Reindexing permits backward shifts instead. We interpret $a_n = 0$ for $n < 0$.

The coefficient dictionary. The Euler operator satisfies $\theta x^n = n x^n$, so

$$\boxed{[x^n]Q(\theta)F = Q(n)a_n, \qquad [x^n]x^j Q(\theta)F = Q(n-j)a_{n-j}.} \tag{45}$$

In the opposite operator order, $[x^n]Q(\theta)x^j F = Q(n)a_{n-j}$.

Also

$$\boxed{x^i D^i = \theta_{\underline{i}},}$$

because both operators send x^n to $n_{\underline{i}} x^n$. Operator products mean composition, with the rightmost operation performed first.

Theorem 536 (D-finite if and only if P-recursive)**.** *For $F(x) = \sum_{n\geq 0} a_n x^n$, the series F is D-finite if and only if (a_n) is P-recursive.*

Proof. Suppose $\mathcal{L}F = 0$, where $\mathcal{L} = \sum_{i=0}^{r} p_i(x) D^i \neq 0$. Multiplying by x^r,

$$x^r \mathcal{L} = \sum_i p_i(x) x^{r-i} \theta_{\underline{i}} = \sum_j x^j Q_j(\theta).$$

The Q_j's are not all zero: after grouping by powers of x, the falling factorials of distinct orders are linearly independent. Taking coefficients gives

$$\sum_j Q_j(n-j) a_{n-j} = 0.$$

Conversely, write a P-recurrence as $\sum_{j=0}^{s} q_j(n) a_{n-j} = 0$ for $n \geq n_0$. Multiply every q_j by $n_{\underline{n_0}}$, making the recurrence valid for every $n \geq 0$. The coefficient dictionary gives

$$\boxed{\left(\sum_{j=0}^{s} q_j(\theta) x^j \right) F = 0.}$$

The operator is nonzero: on x^m, its output is $\sum_j q_j(m+j) x^{m+j}$. The powers are distinct, and a nonzero polynomial cannot vanish at all sufficiently large integers in characteristic zero. □

In the converse, x^j shifts the sequence and $q_j(\theta)$ weights the resulting coefficient index.

Sequential recovery and exceptional indices.

Lemma 537 (Coefficient recovery)**.** *Let $\mathcal{L} = \sum_{i=0}^{r} p_i(x) D^i \neq 0$, and suppose $\mathcal{L}H = R$, where $H = \sum_{n\geq 0} h_n x^n$. For fixed order and coefficient degrees, $h_0, \ldots, h_N$ can be computed in $O(N+1)$ operations from the required coefficients of R and at most r prescribed coefficients of H.*

The prescribed indices are the nonnegative integer roots of an explicit nonzero polynomial of degree at most r. All supplied data are assumed consistent with a solution.

Proof. Expand

$$x^r \mathcal{L} = \sum_{j=\nu}^{J} x^j Q_j(\theta), \qquad Q_\nu \neq 0,$$

where ν is the smallest occurring exponent. Taking the coefficient of $x^{n+\nu}$,

$$\boxed{Q_\nu(n) h_n = [x^{n+\nu-r}] R - \sum_{j=\nu+1}^{J} Q_j(n+\nu-j) h_{n+\nu-j}.} \tag{46}$$

Negative-index coefficients are zero. Every coefficient of H on the right has smaller index than n.

When $Q_\nu(n) \neq 0$, solve for h_n. Otherwise use its prescribed value. Since $\deg Q_\nu \leq r$, there are at most r exceptional indices. The recurrence width is fixed, so every ordinary step costs $O(1)$.

If $d = \max_i \deg p_i$, then $J \leq r + d$, so coefficients of R through degree $N + d$ suffice. □

The exceptional coefficients need not form an initial consecutive block. For example, $(\theta - m)F = 0$ has solutions $F = cx^m$: the free coefficient occurs at index m, although the equation has order one.

The rational special case. A series is rational exactly when its coefficients satisfy an eventual constant-coefficient recurrence. Indeed, if $F = P/Q$, with $Q(0) = 1$, then $QF = P$ gives such a recurrence beyond $\deg P$. Conversely, a constant-coefficient recurrence makes QF polynomial for the corresponding denominator Q.

We use the forward-shift convention for characteristic polynomials: $P(Z) = Z^r + \sum_{j=0}^{r-1} p_j Z^j$ annihilates a sequence when $a_{n+r} + \sum_{j<r} p_j a_{n+j} = 0$. The corresponding generating-function connection polynomial is $Q(x) = x^r P(1/x)$. When a recurrence routine returns Q, reverse it with the prescribed bound r before treating its roots as exponential bases. Keep that bound even if $\deg Q < r$.

Constant-coefficient recurrences admit logarithmic-in-the-index evaluation by Bostan–Mori or matrix powering. For polynomial-coefficient recurrences, the block-product algorithm below has square-root dependence on the index.

15.8.2 Algebraic Series and Closure Properties

Theorem 538 (Algebraic series are D-finite)**.** *If $F \in K[[x]]$ is algebraic of degree $d = [K(x, F) : K(x)]$, then it satisfies a homogeneous differential equation of order at most d.*

Proof. Let $P(x, y) \in K(x)[y]$ be its minimal polynomial. Characteristic zero makes P separable, so $P_y(x, F) \neq 0$. Differentiating,

$$F' = -\frac{P_x(x, F)}{P_y(x, F)} \in K(x, F).$$

The product and quotient rules show that differentiation preserves $K(x, F)$. Thus $F, F', \ldots, F^{(d)}$ lie in a d-dimensional $K(x)$-space. A dependence, followed by clearing denominators, gives the equation. □

To construct the equation, work in $K(x)[y]/(P)$ with basis $1, y, \ldots, y^{d-1}$. Extended Euclid computes $P_y^{-1} \bmod P$. Set $H = -P_x P_y^{-1} \bmod P$; differentiation of a representative $R(x, y)$ is then

$$\boxed{R \longmapsto R_x + R_y H \pmod{P}.}$$

Starting from $y \bmod P$, compute the vectors for $F, F', \ldots, F^{(d)}$ and find a dependence by elimination over $K(x)$.

The minimal polynomial describes possible branches; initial coefficient data specify which branch is being computed.

Theorem 539 (Closure of D-finite series)**.** *D-finite series are closed under linear combinations, products, differentiation, integration, and Hadamard products*

$$(F \odot G)(x) = \sum_{n \geq 0} ([x^n]F)([x^n]G)x^n.$$

If A is algebraic with $A(0) = 0$, then $F(A(x))$ is D-finite whenever F is D-finite.

Proof. Let V_F, V_G be the finite-dimensional derivative spans over $K(x)$. Linear combinations lie in $V_F + V_G$; derivatives of a product lie, by Leibniz' rule, in the span of products of basis elements. Differentiation preserves a finite derivative span, and adjoining one antiderivative increases its dimension by at most one.

For substitution, the case $A = 0$ is immediate. Otherwise A has positive valuation. If a nonzero polynomial $p(t)$ has lowest nonzero term ct^j, then $p(A)$ has lowest term cA^j and is not zero. Thus substituting A in the differential equation for F does not destroy its leading coefficient. That equation reduces the functions $F^{(j)}(A)$ to a finite spanning family over $K(x, A)$. The chain rule keeps all derivatives in that span, and $K(x, A)$ is a finite extension of $K(x)$.

For the Hadamard product, write $F = \sum f_n x^n$, $G = \sum g_n x^n$. If either sequence is eventually zero, the conclusion is immediate. Otherwise choose P-recurrences of positive orders r, s. Beyond their exceptional indices, every forward shift of f_n is a $K(n)$-linear combination of $f_n, \ldots, f_{n+r-1}$, and similarly for g_n. Thus every shifted product $f_{n+\ell} g_{n+\ell}$ is a combination of the same rs products $f_{n+i} g_{n+j}$. A dependence among $rs + 1$ coefficient vectors gives a P-recurrence. □

Each closure construction represents derivatives or shifts in a finite spanning family and finds a linear dependence. The resulting equation need not have minimum order. Equation construction is separate from coefficient evaluation.

15.8.3 Root Power Sums and Products of Rational Sequences

For a monic polynomial $f(X) = \prod_{i=1}^{m}(X - \alpha_i)$, define $s_0(f) = m$, $s_k(f) = \sum_i \alpha_i^k$, and $f^*(z) = z^m f(1/z)$. Then

$$\boxed{\begin{aligned}\sum_{k\geq 0} s_k(f) z^k &= m - z\frac{(f^*)'(z)}{f^*(z)},\\ f^*(z) &= \exp\left(-\sum_{k\geq 1}\frac{s_k(f)}{k} z^k\right).\end{aligned}} \tag{47}$$

Both formulas follow from $\log f^* = \sum_i \log(1 - \alpha_i z)$.

Thus polynomial coefficients and root power sums can be converted in $O(D\log(D+2))$ operations through index D. Zero roots are accounted for by the specified degree and s_0.

Theorem 540 (Composed sums and products)**.** *Given monic polynomials of positive degrees m, n,*

$$f(X) = \prod_{i=1}^{m}(X - \alpha_i), \qquad g(X) = \prod_{j=1}^{n}(X - \beta_j),$$

one can compute

$$f \oplus g = \prod_{i,j}(X - \alpha_i - \beta_j), \qquad f \otimes g = \prod_{i,j}(X - \alpha_i\beta_j)$$

in

$$\boxed{O(mn\log(mn+2))}$$

field operations, without finding individual roots.

Proof. Compute both power-sum sequences through $D = mn$. For products,

$$s_k(f \otimes g) = s_k(f)s_k(g).$$

For sums, the binomial theorem gives

$$s_k(f \oplus g) = \sum_{\ell=0}^{k}\binom{k}{\ell} s_\ell(f)s_{k-\ell}(g).$$

This is one convolution after factorial scaling. Recover the degree-D outputs using (47). □

Algorithm 114 Composed root sums or products

Input: Monic f, g, operation $\circ \in \{+, \times\}$
Output: $\prod_{i,j}(X - (\alpha_i \circ \beta_j))$

```
D ← (deg f)(deg g)
Compute s_0, ..., s_D and t_0, ..., t_D by logarithmic differentiation
if ∘ = × then
    u_k ← s_k t_k
else
    Convolve (s_k/k!) and (t_k/k!)
    u_k ← k! times the k-th convolution coefficient
H(z) ← exp(−Σ_{k=1}^{D} u_k z^k/k) mod z^{D+1}
return X^D H(1/X)
```

Hadamard and binomial products of recurrence sequences. Suppose (a_k), (b_k) satisfy constant-coefficient recurrences from their initial states, with characteristic polynomials f, g of positive degrees m, n. Then

$$c_k = a_k b_k, \qquad d_k = \sum_{j=0}^{k} \binom{k}{j} a_j b_{k-j}$$

are annihilated respectively by $f \otimes g$ and $f \oplus g$.

Use companion-matrix states $a_k = u^{\mathsf{T}} A^k v$, $b_k = s^{\mathsf{T}} B^k t$. The two sequences are matrix coefficients of

$$(A \otimes B)^k, \qquad (A \otimes I + I \otimes B)^k.$$

Triangularizing A, B over a splitting field shows that the two characteristic polynomials are $f \otimes g$ and $f \oplus g$. Cayley–Hamilton proves the claims.

The annihilating recurrences and their first mn terms can therefore be constructed in $O(mn \log(mn+2))$ operations. The recurrences need not be minimal.

The Hadamard product of an algebraic series with a rational series is algebraic in characteristic zero. After partial fractions, a rational coefficient sequence is a finite sum of $P(n)\lambda^n$, apart from finitely many initial terms. Hadamard multiplication by such a sequence sends F to $P(\theta)F(\lambda x)$, which is algebraic. Finite changes add only a polynomial. Partial fractions may be taken in a finite splitting field; algebraicity over that finite extension implies algebraicity over $K(x)$ by transitivity of algebraic extensions.

15.8.4 Sparse Interpolation through Rational Recurrences

Sparse interpolation recovers a polynomial from a bound on its number of nonzero terms. Geometric evaluations turn the unknown exponents into the bases of exponential sequences, which can be recovered from a recurrence. The construction works over any field containing the required element ω.

Suppose

$$f(x) = \sum_{i=1}^{t} c_i x^{e_i}, \qquad t \le s, \quad 0 \le e_i \le D,$$

with distinct exponents and nonzero coefficients. Choose $\omega \in K^{\times}$ of multiplicative order greater than D. Then

$$b_j = f(\omega^j) = \sum_{i=1}^{t} c_i \gamma_i^j, \qquad \gamma_i = \omega^{e_i},$$

is a sum of exponentials with distinct nonzero bases.

Lemma 541 (Prony recurrence)**.** *The minimal recurrence polynomial of (b_j) is*

$$P(Z) = \prod_{i=1}^{t} (Z - \gamma_i).$$

It is determined by $b_0, \dots, b_{2s-1}$.

Proof. The displayed polynomial annihilates each exponential sequence. Conversely, an annihilator Q gives

$$\sum_i c_i Q(\gamma_i) \gamma_i^j = 0.$$

For $j = 0, \dots, t-1$, Vandermonde nonsingularity forces $Q(\gamma_i) = 0$ for every i. Thus no smaller-degree annihilator exists.

The leading $t \times t$ Hankel matrix factors as $V \operatorname{diag}(c_i) V^{\mathsf{T}}$, so it is nonsingular. The coefficient sequence has order $t \le s$, so Corollary 510 shows that $2s$ terms suffice to recover the infinite sequence's minimum recurrence. □

Once the roots γ_i are known, let

$$Q(z) = z^t P(1/z), \qquad R(z) = Q(z) \sum_{j=0}^{t-1} b_j z^j \bmod z^t.$$

Then

$$\sum_{j \geq 0} b_j z^j = \sum_i \frac{c_i}{1 - \gamma_i z} = \frac{R(z)}{Q(z)},$$

so

$$\boxed{c_i = -\gamma_i \frac{R(\gamma_i^{-1})}{Q'(\gamma_i^{-1})}.}$$

Multipoint evaluation recovers all amplitudes in $O(t \log^2(t+2))$ operations.

Theorem 542 (Sparse interpolation)**.** *The polynomial f is determined by $2s$ geometric evaluations. Beyond those evaluations, reconstruction costs $O(s \log^2(s+2))$ field operations, plus finding the roots of the recovered polynomial and decoding e_i from $\gamma_i = \omega^{e_i}$.*

Proof. Recover the recurrence, find its distinct roots, recover amplitudes by the displayed formula, and decode the exponents. □

Over a finite field, exponent decoding is a bounded discrete-logarithm problem. A table $1, \omega, \ldots, \omega^D$ gives direct lookup after $O(D)$ preprocessing, in addition to the stated bound.

Algorithm 115 Sparse interpolation

Input: Evaluation access to f, bounds s, D, $\operatorname{ord}(\omega) > D$
Output: The nonzero monomials of f

```
Evaluate b_j = f(ω^j) for 0 ≤ j < 2s
Recover the monic minimal recurrence polynomial P in the forward-shift convention
if P = 1 then
    return the zero polynomial
Find its roots γ_1, ..., γ_t
Q(z) ← z^t P(1/z)
R(z) ← Q(z) Σ_{j=0}^{t-1} b_j z^j mod z^t
Evaluate R, Q' at all γ_i^{-1}
for i = 1, ..., t do
    c_i ← −γ_i R(γ_i^{-1}) / Q'(γ_i^{-1})
    Decode e_i ∈ [0, D] with ω^{e_i} = γ_i
return Σ_i c_i x^{e_i}
```

For individual multivariate degree bounds $D_1, \ldots, D_r$, use mixed-radix substitution

$$x_i \longmapsto x^{\prod_{j<i}(D_j+1)}.$$

This preserves sparsity and encodes exponent vectors injectively. The univariate degree bound becomes $\prod_i (D_i + 1) - 1$.

If evaluations of $x_\ell \partial f / \partial x_\ell$ are additionally available, their exponential bases are the same γ_i, while their amplitudes are $e_{i,\ell} c_i$. Recovering these amplitudes and dividing by c_i gives the exponents in the field. When the characteristic exceeds the individual degree bounds, this replaces discrete logarithms by direct decoding. This variant requires access to those derivative evaluations.

15.8.5 Distant Polynomial and Geometric Recurrences

Write a polynomial-coefficient recurrence as a matrix transition. Suppose $r \geq 1$ and $\sum_{j=0}^{r} q_j(n) a_{n+j} = 0$, with $q_r \neq 0$, on the indices being processed. Use the state $u_n = (a_n, \ldots, a_{n+r-1})^{\mathsf{T}}$. Let $A(x)$ have $q_r(x)$ on its superdiagonal, last row $(-q_0(x), \ldots, -q_{r-1}(x))$, and all other entries zero. At every nonsingular index,

$$\boxed{u_{n+1} = \frac{A(n)}{q_r(n)} u_n.}$$

The first $r-1$ rows shift the known coefficients; the last row solves for the new one. This description also covers $r = 1$. A recurrence polynomial in γ^n gives the same construction with that quantity as the matrix parameter.

Group consecutive transitions into polynomial blocks. First accumulate the scalar denominators in one extra coordinate:

$$u_{j+1} = \frac{A(\xi_j)}{q(\xi_j)} u_j \quad \Longrightarrow \quad z_{j+1} = T(\xi_j) z_j, \qquad T(x) = \operatorname{diag}(A(x), q(x)), \qquad z_0 = (u_0, 1)^{\mathsf{T}}.$$

Here $Q_j = \prod_{i<j} q(\xi_i)$ and $z_j = (Q_j u_j, Q_j)^{\mathsf{T}}$. Multiplication by $\operatorname{diag}(A(\xi_j), q(\xi_j))$ updates both the numerator and the denominator. Divide by the last coordinate at the end.

Consider either $\xi_j = j$, corresponding to polynomial-coefficient recurrences, or $\xi_j = \gamma^j$, corresponding to q-holonomic recurrences, with $\gamma \neq 0$. Let $\sigma(x) = x + 1$ or $\sigma(x) = \gamma x$, respectively. Define the block polynomial

$$B_b(x) = T(\sigma^{b-1}(x)) \cdots T(x).$$

In both cases,

$$\boxed{B_{2b}(x) = B_b(\sigma^b(x)) B_b(x).}$$

For additive shifts, computing $B_b(x + b)$ is a Taylor shift. For multiplicative shifts, computing $B_b(\gamma^b x)$ only scales coefficients. In either case, a block of length b is constructed in $O(b \log(b+2))$ operations by doubling.

Evaluating the block starting points costs $O(b \log^2(b+2))$ by ordinary multipoint evaluation, or $O(b \log(b+2))$ for a geometric progression.

Theorem 543 (Distant recurrence terms)**.** *For fixed matrix dimension and polynomial degrees, a target state at index $N \geq 1$ can be computed in*

$$\boxed{\begin{array}{ll} O(\sqrt{N} \log^2(N+2)), & \xi_j = j, \\ O(\sqrt{N} \log(N+2)), & \xi_j = \gamma^j, \quad \gamma \neq 0. \end{array}}$$

All scalar denominators used by the target computation must be nonzero. The initial state is supplied separately.

Proof. Choose a power of two $b \in [\sqrt{N}, 2\sqrt{N})$. Construct B_b, then evaluate it at the $L = \lfloor N/b \rfloor$ block starting points $\xi_0, \xi_b, \ldots, \xi_{(L-1)b}$. Apply these numerical matrices and process the remaining fewer than b transitions directly.

The construction and numerical applications cost $O(\sqrt{N} \log(N+2))$. Use ordinary or geometric multipoint evaluation for the stated bounds. □

Algorithm 116 A distant recurrence term

Input: $u_0, A(x), q(x), N$, and either $\xi_j = j$ or $\xi_j = \gamma^j$
Output: u_N, assuming nonzero required denominators

if $N = 0$ **then**
  return u_0
$T(x) \leftarrow \operatorname{diag}(A(x), q(x))$
$z \leftarrow (u_0, 1)^{\mathsf{T}}$
Choose a power of two $b \in [\sqrt{N}, 2\sqrt{N})$
$B(x) \leftarrow T(x)$, $a \leftarrow 1$
while $a < b$ **do**
  $B(x) \leftarrow B(\sigma^a(x))B(x)$
  $a \leftarrow 2a$
$L \leftarrow \lfloor N/b \rfloor$
Evaluate B at ξ_{jb}, $0 \le j < L$
for $j = 0, \ldots, L-1$ **do**
  $z \leftarrow B(\xi_{jb})z$
for $j = Lb, \ldots, N-1$ **do**
  $z \leftarrow T(\xi_j)z$
return the numerator coordinates divided by the last coordinate

The geometric evaluation uses

$$\lambda^{ij} = \lambda^{\binom{i+j}{2}}\lambda^{-\binom{i}{2}}\lambda^{-\binom{j}{2}},$$

which turns the evaluation into a convolution. Repeated geometric evaluation points cause no difficulty; only the interpolation algorithm in Section 15.5.4 needs them to be distinct.

Examples are the factorial recurrence $u_{j+1} = (j+1)u_j$ and the q-Pochhammer product $(a;\gamma)_N = \prod_{j=0}^{N-1}(1 - a\gamma^j)$.

For a polynomial-index recurrence, a fixed nonzero denominator polynomial has only finitely many singular integer indices. If the coefficients needed at those indices are supplied, process them directly and apply the block algorithm to the intervening nonsingular intervals, shifting the polynomial parameter at the start of each interval. With fixed recurrence complexity, there are only constantly many such intervals, so the asymptotic bound is unchanged. The coefficients at singular indices must still be supplied.

Products along a geometric progression. For scalar series, logarithms reduce a product of geometrically scaled arguments to coefficientwise multiplication. Let $f(0) = 1$, and write $\log f(x) = \sum_{r \ge 1} \ell_r x^r$. Then

$$\boxed{\log \prod_{j=0}^{k-1} f(\gamma^j x) = \sum_{r \ge 1} \ell_r S_k(\gamma^r) x^r, \qquad S_k(a) = \sum_{j=0}^{k-1} a^j.} \tag{48}$$

Indeed, logarithms turn the product into a sum, and scaling the argument multiplies its r-th coefficient by γ^{jr}.

For $k \ge 1$, evaluate the scalar weight by

$$S_k(a) = \begin{cases} (a^k - 1)/(a - 1), & a \ne 1, \\ k, & a = 1. \end{cases}$$

The second case covers roots of unity γ, including $\gamma = 1$.

Corollary 544 (A truncated geometric product)**.** *Given $f \bmod x^n$, with $n \ge 1$, $f(0) = 1$, and an integer $k \ge 0$, the product $\prod_{j=0}^{k-1} f(\gamma^j x) \bmod x^n$ is computed in*

$$\boxed{O(n \log(n+2) + \log(k+1))}$$

field operations. No distinctness condition on the powers of γ is needed.

Proof. For $k = 0$, return one. Otherwise compute $\log f$, the scalar γ^k, and the successive powers of γ and γ^k. These give all $S_k(\gamma^r)$, $1 \le r < n$, in linear time. Scale the logarithm coefficients using (48), then exponentiate. □

If $f(0) = c \neq 0$, apply the normalized construction to f/c and multiply the result by c^k. This scalar power adds $O(\log(k+1))$ work.

15.8.6 Batched Recurrences with a Varying Parameter

For several queries, share transition products even when both the stopping index and the parameter value vary. Suppose

$$v_m(x) = \frac{A_m(x)}{q_m(x)} v_{m-1}(x), \qquad v_0(x) = \frac{b_0(x)}{q_0(x)},$$

where $q_0, \dots, q_L$ are nonzero polynomials, and query (m_i, x_i) requests $v_{m_i}(x_i)$, with $0 \le m_i \le L$. Use a product tree for the transitions and a remainder tree for the query values. Matrices act on column states from the left, so later transitions appear on the left.

Set

$$Q_m = q_0 \prod_{j=1}^{m} q_j, \qquad z_m = (Q_m v_m, Q_m)^{\mathsf{T}}.$$

Then $z_m = T_m z_{m-1}$, where $T_m = \operatorname{diag}(A_m, q_m)$ and $z_0 = (b_0, q_0)^{\mathsf{T}}$. The cumulative denominator $Q_{m_i}(x_i)$ must be nonzero for every query.

Build a balanced interval tree on $[1, L]$. At $I = [\ell, r]$, store

$$P_I = T_r \cdots T_\ell, \qquad R_I(x) = \prod_{i:\, m_i \in I} (x - x_i).$$

For children $I_L = [\ell, h]$, $I_R = [h+1, r]$, $P_I = P_{I_R} P_{I_L}$ and $R_I = R_{I_L} R_{I_R}$.

During descent, maintain

$$u_I \equiv z_{\ell-1} \pmod{R_I}, \qquad \deg u_I < \deg R_I.$$

The two child states are

$$\boxed{u_{I_L} = u_I \bmod R_{I_L}, \qquad u_{I_R} = P_{I_L} u_I \bmod R_{I_R}.}$$

At a leaf $\{m\}$, evaluate $T_m u_I$ at its query points. Repeated points are allowed; no interpolation is used.

Theorem 545 (Batched recurrence evaluation)**.** *Suppose the matrix dimension is fixed. Let s be the number of queries, d_0 bound the degrees of b_0, q_0, and d_m bound the degrees of A_m, q_m. If*

$$L + s + d_0 + \sum_{m=1}^{L} d_m = O(N),$$

all valid queries can be answered in

$$\boxed{O(N \log^2(N+2))}$$

operations and $O(N \log(N+2))$ space.

Proof. The root invariant holds after reducing z_0 modulo its query polynomial. Each child modulus divides the parent modulus, and multiplication by P_{I_L} advances the right child's starting state through the left interval. Thus the leaf evaluations are correct.

At any fixed tree depth, intervals are disjoint. The aggregate degrees of their transition products are $O(N)$, as are the aggregate degrees of their query polynomials. Building one level, or carrying out its remainder descent, therefore costs $O(N \log(N+2))$. There are $O(\log(N+2))$ depths. The within-leaf product trees and evaluations satisfy the same total bound. □

Algorithm 117 Batched rational recurrence evaluation

Input: Transitions, initial state, queries (m_i, x_i)
Output: All query values with nonzero cumulative denominators

Handle $m_i = 0$ queries by evaluating b_0, q_0; remove them
Stop if no queries remain
Form T_m, z_0 and build an interval tree on $[1, L]$
for each leaf $I = \{m\}$ **do**
    $P_I \leftarrow T_m$
    $R_I \leftarrow \prod_{i:\, m_i = m} (x - x_i)$, using a product tree
for each internal node I, bottom-up **do**
    $P_I \leftarrow P_{I_R} P_{I_L}$
    $R_I \leftarrow R_{I_L} R_{I_R}$
DESCEND($[1, L], z_0 \bmod R_{[1,L]}$)

function DESCEND(I, u)
    if $R_I = 1$ **then**
        return
    if I is a leaf **then**
        Evaluate $P_I u \bmod R_I$ at its query points
        Divide numerator coordinates by the last coordinate
        return
    if $R_{I_L} \neq 1$ **then**
        DESCEND($I_L, u \bmod R_{I_L}$)
    if $R_{I_R} \neq 1$ **then**
        DESCEND($I_R, P_{I_L} u \bmod R_{I_R}$)

A query-free interval may be skipped during descent, but its transition product may still be needed to reach later queries.

Binomial-prefix queries. Put $c_m(x) = \binom{x}{m}$ and $s_m(x) = \sum_{j=0}^{m} \binom{x}{j}$. Then

$$\boxed{\begin{pmatrix} c_m \\ s_m \end{pmatrix} = \frac{1}{m} \begin{pmatrix} x - m + 1 & 0 \\ x - m + 1 & m \end{pmatrix} \begin{pmatrix} c_{m-1} \\ s_{m-1} \end{pmatrix}, \qquad \begin{pmatrix} c_0 \\ s_0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.}$$

The matrices have degree one and the cumulative denominator is $m!$. Therefore N queries $\sum_{j=0}^{m_i} \binom{n_i}{j}$, with $0 \le n_i, m_i \le N$, are answered in $O(N \log^2(N+2))$ operations.

15.8.7 Exponential and Logarithmic Substitution

The substitutions $e^x - 1$ and $\log(1 + x)$ are compositional inverses. Transposed evaluation and interpolation compute their full coefficient transforms without expanding the individual powers.

For a polynomial $F(t) = \sum_{j=0}^{m-1} f_j t^j$,

$$\boxed{[x^k] F(e^x) = \frac{1}{k!} \sum_{j=0}^{m-1} f_j j^k.} \tag{49}$$

Here $0^0 = 1$. The sums on the right are the transpose of the map that evaluates a degree-$< n$ polynomial at $0, 1, \ldots, m-1$. Transposed multipoint evaluation followed by factorial scaling therefore computes $F(e^x) \bmod x^n$ in $O((m+n)\log^2(m+n+2))$ operations.

The other two substitutions require only square transforms of size n. Let $V_{ij} = i^j$, for $0 \le i, j < n$, let $D = \operatorname{diag}(1/0!, \ldots, 1/(n-1)!)$, and let S_c be the coefficient map $F(t) \mapsto F(t+c)$ on degree-$< n$ polynomials. With coefficient vectors written as columns,

$$F(e^x) \bmod x^n \quad \text{has map} \quad DV^{\mathsf{T}}.$$

To substitute $e^x - 1$, first shift the input polynomial by -1. The resulting map is $DV^{\mathsf{T}}S_{-1}$.

Theorem 546 (Fast exponential and logarithmic substitutions)**.** *For $F \in K[t]_{<n}$, each of*

$$F(e^x), \qquad F(e^x - 1), \qquad F(\log(1+x)) \pmod{x^n}$$

can be computed in $O(n \log^2(n+2))$ field operations. The transposes of these coefficient maps have the same complexity. More explicitly,

$$\boxed{\mathcal{C}_{e^x-1} = DV^{\mathsf{T}}S_{-1}, \qquad \mathcal{C}_{\log(1+x)} = S_1 V^{-\mathsf{T}} D^{-1}.} \tag{50}$$

Proof. The first formula follows from (49). The integers $0, \ldots, n-1$ are distinct, so V is invertible. Since $e^x - 1$ and $\log(1+x)$ have valuation one and are compositional inverses, their substitution maps on $K[[x]]/(x^n)$ are inverse linear maps. Inverting $DV^{\mathsf{T}}S_{-1}$ gives $S_1 V^{-\mathsf{T}} D^{-1}$.

Apply a Taylor shift, transposed evaluation or transposed interpolation, and diagonal scaling in the indicated order. These operations give the claimed bound. Reversing their order and transposing each factor gives the transposed maps within the same bound. □

Algorithm 118 Exponential and logarithmic substitution

Input: $F \in K[t]_{<n}$, precision $n \ge 1$, substitution $\psi \in \{e^x, e^x - 1, \log(1+x)\}$
Output: $F(\psi(x)) \bmod x^n$

```
Precompute factorials and inverse factorials through n − 1
if ψ = log(1 + x) then
    u_j ← j![t^j]F for 0 ≤ j < n
    Apply transposed interpolation at 0, ..., n − 1: v ← V^{−T} u
    return Σ_{j=0}^{n−1} v_j (x + 1)^j, by Taylor shift
else
    if ψ = e^x − 1 then
        F(t) ← F(t − 1)
    Apply transposed evaluation at 0, ..., n − 1: v ← V^T coeff(F)
    return Σ_{j=0}^{n−1} v_j x^j / j!
```

For a formal input series, $F(e^x - 1)$ and $F(\log(1+x))$ require only $F \bmod t^n$. In contrast, $F(e^x)$ is not a formal substitution at zero: every input coefficient can affect its constant term. The first transformation therefore requires a polynomial input, or a separately specified expansion of the function at $t = 1$.

Stirling transforms and power projection. The coefficient identities

$$\frac{(e^x - 1)^j}{j!} = \sum_{k \ge j} \left\{ {k \atop j} \right\} \frac{x^k}{k!}, \qquad \frac{\log(1+x)^j}{j!} = \sum_{k \ge j} s(k, j) \frac{x^k}{k!}$$

show that these are factorial-scaled Stirling transforms; $s(k, j)$ denotes the signed Stirling number of the first kind. Their inverse relation follows from compositional inversion.

For a linear functional $\ell(H) = \sum_{k<n} w_k [x^k] H$, transposing substitution returns $\ell(\psi^0), \ldots, \ell(\psi^{n-1})$. Thus the same factorization also computes all these power projections in $O(n \log^2(n+2))$ operations.

15.8.8 Linear-Time Translation and Single-Coefficient Queries

For a fixed D-finite series, one coefficient after exponential substitution can be computed in linear time. The main step is a linear-time Taylor shift of its truncation.

Applying the original differential equation to the truncation leaves an error in a bounded-width interval of degrees. An additional first-order operator annihilates this error, giving a homogeneous equation of bounded order and coefficient degrees. Translate that equation to recover the shifted polynomial. The error must be annihilated before translation, since translation can move it into low degrees.

An equation for the truncation. Suppose $\mathcal{L}A = 0$, where $\mathcal{L} = \sum_{i=0}^{r} p_i(z)D_z^i$, $p_r \neq 0$, and $\deg p_i \leq d$. Let $T = A \bmod z^{k+1}$.

Lemma 547 (A bounded-order equation for a truncation)**.** *Put $\ell = \max\{0, k+1-r\}$. Either $\mathcal{L}T = 0$, or*

$$\mathcal{L}T = z^\ell R(z), \qquad \deg R \leq r + d - 1,$$

and

$$\boxed{\big(zRD_z - \ell R - zR'\big)\mathcal{L}T = 0.}$$

The displayed operator is nonzero, has order $r+1$, and coefficient degrees at most $r + 2d$.

Proof. Since $T - A = O(z^{k+1})$, the error has no term below degree $k+1-r$, and applying $\mathcal{L}$ to a degree-k polynomial gives degree at most $k + d$. Factoring out z^ℓ proves the degree bound.

For $E = z^\ell R$, $zRE' = (\ell R + zR')E$. This gives the annihilating operator. Its leading differential coefficient is $zRp_r \neq 0$, and the product rule gives the coefficient-degree bound. □

The annihilator comes from $E'/E = \ell/z + R'/R$ after clearing denominators. The exponent ℓ appears only as a scalar, while R has bounded degree.

Translation of the equation. Let $\mathcal{D}T = 0$ be the bounded-size equation from the lemma. For $H(t) = T(t+\beta)$, replace z by $t+\beta$ in the polynomial coefficients of $\mathcal{D}$. This produces an equation for H of the same bounded order and degree.

Its recurrence computes successive $h_n = [t^n]H$. At the at most $r+1$ exceptional indices, use

$$\boxed{h_n = \sum_{j=n}^{k} a_j \binom{j}{n} \beta^{j-n}.} \tag{51}$$

After precomputing factorials and powers of β, each such sum costs $O(k+1)$.

Theorem 548 (Translation of a truncated D-finite series)**.** *For fixed differential order and coefficient degrees, given $\mathcal{L}A = 0$, $a_0, \ldots, a_k$, and β, every coefficient of*

$$H(t) = \sum_{j=0}^{k} a_j (t+\beta)^j$$

can be computed in $O(k+1)$ operations and $O(k+1)$ space.

Proof. Compute $E = \mathcal{L}T$ in $O(k+1)$ operations. Construct the bounded-order equation, translate it, and apply Lemma 537. Each ordinary step costs $O(1)$, and a constant number of exceptional coefficients are computed by (51). □

The construction translates the polynomial T; it does not require the infinite substitution $A(t+\beta)$ to be defined.

Algorithm 119 Translate a truncated D-finite series

Input: $\mathcal{L}A = 0$, $a_0, \ldots, a_k$, shift β
Output: Coefficients of $H(t) = \sum_{j=0}^{k} a_j (t+\beta)^j$

```
T ← Σ_{j=0}^{k} a_j z^j, E ← LT
if E = 0 then
    D ← L
else
    ℓ ← max{0, k + 1 − r}, R ← E/z^ℓ
    D ← (zRD_z − ℓR − zR′)L
Translate the coefficients of D by β
Derive its coefficient recurrence by (46)
Precompute factorials, inverse factorials, and powers of β
for n = 0, ..., k do
    if the coefficient of h_n in the recurrence is nonzero then
        Solve the recurrence for h_n
    else
        Compute h_n using (51)
return h_0, ..., h_k
```

One coefficient after exponential substitution. Let $A(z) = \sum a_j z^j$. The composition $A(e^x - 1)$ is formally defined because $e^x - 1$ has zero constant term. For a function $F(t)$, supply $A(z) = F(1+z)$, its Taylor series about 1.

Since $(e^x - 1)^j$ has valuation j, only $a_0, \ldots, a_k$ affect the target coefficient. Put $T = A \bmod z^{k+1}$ and $H(t) = T(t-1) = \sum h_j t^j$. Then, for $k \geq 1$,

$$\boxed{[x^k]A(e^x - 1) = [x^k]H(e^x) = \frac{1}{k!}\sum_{j=1}^{k} h_j j^k.} \tag{52}$$

Lemma 549 (A table of equal-exponent powers)**.** *For $k \geq 1$, all j^k, $1 \leq j \leq k$, can be computed in $O(k)$ field operations and $O(k)$ elementary integer operations.*

Proof. Use a linear sieve. Compute p^k by binary exponentiation only for primes $p \leq k$; for a composite $j = pa$, use $j^k = p^k a^k$.

Composites cost $O(k)$. Primes cost $O(\pi(k)\log(k+1)) = O(k)$. The elementary estimate $\pi(k) = O(k/\log(k+1))$ follows from the primorial upper bound: primes above $\sqrt{k}$ each contribute at least $\frac{1}{2}\log k$ to $\sum_{p \leq k} \log p = O(k)$. □

Algorithm 120 Equal-exponent power table

```
function PowerTable(k)
    Initialize least-prime-factor array lp to zero
    P ← ∅, v_1 ← 1
    for j = 2, ..., k do
        if lp[j] = 0 then
            lp[j] ← j; append j to P
            v_j ← j^k by binary exponentiation
        for primes p ∈ P, while p ≤ lp[j] and pj ≤ k do
            lp[pj] ← p, v_pj ← v_p v_j
    return v_1, ..., v_k
```

Each composite is generated exactly once, using its least prime factor.

Theorem 550 (One coefficient after exponential substitution)**.** *Given a fixed-order, fixed-degree equation for A, together with the necessary coefficient data specifying its solution,*

$$\boxed{[x^k]A(e^x-1)}$$

can be computed in $O(k+1)$ operations and $O(k+1)$ space. The cost of obtaining the prescribed input coefficients is additional.

Proof. Recover $a_0, \dots, a_k$, translate the truncation by -1, generate $1^k, \dots, k^k$, and apply (52). For $k = 0$, return a_0. □

Algorithm 121 One coefficient after exponential substitution

Input: $\mathcal{L}A = 0$, solution data, target $k \geq 0$
Output: $[x^k]A(e^x-1)$

```
Recover a_0, ..., a_k using Lemma 537
if k = 0 then
    return a_0
Translate the truncation by −1 using Algorithm 119, obtaining h_0, ..., h_k
(v_1, ..., v_k) ← PowerTable(k)
return (1/k!) Σ_{j=1}^{k} h_j v_j
```

Moments and classical sequences. For a polynomial $F(t) = \sum f_j t^j$,

$$\sum_j f_j j^k = k![x^k]F(e^x).$$

For an infinite sum, the identity requires convergence sufficient for termwise differentiation at $t = 1$.

For binomial moments,

$$\sum_{j=0}^{N} \binom{N}{j} j^k = k![x^k](1+e^x)^N.$$

Use $A(z) = (2+z)^N$, which satisfies

$$(2+z)A' = NA, \qquad a_0 = 2^N, \qquad a_{j+1} = \frac{N-j}{2(j+1)}a_j.$$

One moment costs $O(k + \log(N + 2))$, including the initial power 2^N.

For Bell numbers, $B_k = k![x^k]e^{e^x-1}$, so $A(z) = e^z$. Here

$$H(t) = \sum_{j=0}^{k} \frac{(t-1)^j}{j!}, \qquad H' - H = -\frac{(t-1)^k}{k!}.$$

Thus

$$h_0 = \sum_{j=0}^{k} \frac{(-1)^j}{j!}, \qquad (j+1)h_{j+1} = h_j - \frac{(-1)^{k-j}}{j!(k-j)!}.$$

This gives a direct linear-time implementation, followed by $B_k = \sum_{j=1}^{k} h_j j^k$ for $k \geq 1$.

For Bernoulli numbers,

$$\mathcal{B}_k = k![x^k]\frac{x}{e^x - 1}, \qquad A(z) = \frac{\log(1+z)}{z} = \sum_{j\geq 0} \frac{(-1)^j}{j+1} z^j.$$

The displayed series defines the value at the removable singularity, and

$$z(1+z)A'' + (2+3z)A' + A = 0.$$

For alternating permutations, work in a field containing i, and use

$$\sec x + \tan x = \frac{e^{ix} + i}{1 + ie^{ix}}.$$

With $A(z) = (1 + z + i)/(1 + i + iz)$, if $c_k = [x^k]A(e^x - 1)$, the desired number is $k!i^k c_k$.

Each computation uses a translated polynomial chosen for the target index; it does not compute the full prefix of the composed series.

15.8.9 Diagonals, Constant Terms, and Residues

For $R(s,t) = \sum_{i,j\geq 0} r_{ij} s^i t^j$, define

$$\operatorname{Diag} R(x) = \sum_{n\geq 0} r_{nn} x^n.$$

Substituting $t = x/s$ records the difference of the indices:

$$\boxed{\operatorname{Diag} R(x) = [s^0] R\left(s, \frac{x}{s}\right).}$$

Here one first expands R at $(0,0)$, then substitutes. For each fixed power x^j, the resulting Laurent series has no powers of s below $-j$, so its constant term is defined.

If R is analytic for $|s| < a$, $|t| < b$, choose $0 < \rho < a$. For $|x| < \rho b$, uniform absolute convergence on $|s| = \rho$ permits termwise integration:

$$\boxed{\operatorname{Diag} R(x) = \frac{1}{2\pi i} \oint_{|s|=\rho} R\left(s, \frac{x}{s}\right) \frac{ds}{s}.} \tag{53}$$

For rational R, the integral is the sum of all enclosed residues. For example, $R = 1/(1-t)$ has diagonal one, while its integrand is $1/(s-x)$; the enclosed pole is at $s = x$.

Central Delannoy numbers. Paths with steps $(1,0),(0,1),(1,1)$ have generating function

$$\sum_{m,n\geq 0} D(m,n)s^m t^n = \frac{1}{1-s-t-st}.$$

For $D_n = D(n,n)$,

$$D(x) = \frac{1}{2\pi i}\oint \frac{ds}{-s^2+(1-x)s-x}.$$

The denominator roots are

$$\alpha(x) = \frac{1-x-\sqrt{1-6x+x^2}}{2}, \qquad \beta(x) = \frac{1-x+\sqrt{1-6x+x^2}}{2},$$

where the square root has constant term one. Only $\alpha(x) \to 0$, so only it lies inside a small fixed contour. The derivative of the denominator there is $\sqrt{1-6x+x^2}$. Hence

$$\boxed{D(x) = \frac{1}{\sqrt{1-6x+x^2}}.}$$

Differentiation and coefficient extraction give

$$(1-6x+x^2)D' = (3-x)D$$

and

$$\boxed{(n+2)D_{n+2} = 3(2n+3)D_{n+1} - (n+1)D_n, \qquad D_0 = 1, \quad D_1 = 3.}$$

Theorem 551 (Bivariate rational diagonals are algebraic)**.** *If $R(s,t) \in \mathbb{C}(s,t)$ is regular at the origin, then* Diag $R(x)$ *is algebraic over $\mathbb{C}(x)$.*

Proof. The contour integrand $H(s,x) = R(s,x/s)/s$ is rational in s over $\mathbb{C}(x)$. In a finite splitting field of its denominator, write

$$H(s,x) = B(s,x) + \sum_{\alpha}\sum_{j=1}^{m_\alpha} \frac{c_{\alpha,j}(x)}{(s-\alpha(x))^j}.$$

The pole locations and coefficients are algebraic over $\mathbb{C}(x)$.

Near a sufficiently small nonzero parameter value avoiding pole collisions and degeneracies, choose analytic branches. The fixed contour encloses a fixed collection of poles. Its integral is the sum of their residues $c_{\alpha,1}(x)$, hence is algebraic.

This integral agrees with the analytic diagonal. After clearing denominators, its polynomial relation holds on an open set, and therefore throughout a neighborhood of zero by analyticity. It is consequently an identity of power series. □

15.8.10 Coefficients in Finite Fields by Reading Index Digits

Work over $\mathbb{F}_q$. In characteristic p, every series has zero p-th ordinary derivative, so D-finiteness imposes no restriction. Algebraicity instead permits coefficient extraction by section operators that preserve a fixed rational denominator.

Section operators. For $0 \leq r < q$, define

$$\Lambda_r\left(\sum_{n\geq 0} a_n x^n\right) = \sum_{n\geq 0} a_{qn+r}x^n.$$

If $N = r_0 + qr_1 + \cdots + q^\ell r_\ell$, then

$$a_N = \big(\Lambda_{r_\ell} \cdots \Lambda_{r_0} F\big)(0).$$

Process the least significant digit first.

For several variables,

$$\Lambda_{\mathbf{r}} \left(\sum_{\boldsymbol{\alpha} \geq 0} c_{\boldsymbol{\alpha}} X^{\boldsymbol{\alpha}} \right) = \sum_{\boldsymbol{\alpha} \geq 0} c_{q\boldsymbol{\alpha}+\mathbf{r}} X^{\boldsymbol{\alpha}}.$$

A diagonal query uses the same digit in every coordinate.

Lemma 552 (Sections preserve a denominator)**.** *For polynomials P, B over $\mathbb{F}_q$, with $B(0) \neq 0$,*

$$\boxed{\Lambda_{\mathbf{r}} \left(\frac{P}{B} \right) = \frac{\Lambda_{\mathbf{r}}(PB^{q-1})}{B}.}$$

Proof. The identity

$$\Lambda_{\mathbf{r}}(C(X)H(X^q)) = \Lambda_{\mathbf{r}}(C(X))H(X)$$

follows by comparing exponent residues. Frobenius gives $B(X)^q = B(X^q)$, so $P/B = PB^{q-1}/B(X^q)$. Apply the identity. □

Choose individual degree bounds $D_i \geq \max\{\deg_{X_i} P, \deg_{X_i} B\}$. The product PB^{q-1} has degree at most qD_i in X_i, and sectioning returns this to at most D_i. Thus all numerators lie in the same finite-dimensional space.

Theorem 553 (Rational coefficient queries over finite fields)**.** *For fixed number d of variables, put*

$$S = \prod_{i=1}^{d} (D_i + 1), \qquad V = \prod_{i=1}^{d} (qD_i + 1).$$

After computing $C = B^{q-1}$, a coefficient $[X_1^{N_1} \cdots X_d^{N_d}]P/B$ is obtained in

$$\boxed{O\Big(S^2(1 + \log_q(1 + \max_i N_i))\Big)}$$

operations. The preprocessing costs $O(V \log(V+2) \log(q+1))$ operations and $O(V)$ space.

Proof. For one digit vector, update the S numerator coefficients by

$$P'_{\boldsymbol{\alpha}} = \sum_{\boldsymbol{\beta}} P_{\boldsymbol{\beta}} C_{q\boldsymbol{\alpha}+\mathbf{r}-\boldsymbol{\beta}},$$

interpreting missing coefficients as zero. This costs $O(S^2)$. After all digits are removed, return $P'(0)/B(0)$.

Binary powering constructs B^{q-1}. All intermediate products fit in a coefficient box of size $O(V)$; Kronecker substitution gives the preprocessing bound. □

Algorithm 122 Finite-field rational coefficient extraction

Input: P, B, $B(0) \neq 0$, target vector $\mathbf{N}$
Output: $[X^{\mathbf{N}}]P/B$

```
C ← B^{q-1}
while some coordinate of N is positive do
    r ← N mod q, coordinatewise
    P ← Λ_r(PC)
    N ← ⌊N/q⌋, coordinatewise
return P(0)/B(0)
```

The coefficient formula in the proof implements the update without forming the full product PC. For a fixed field and rational function, the query cost is logarithmic in the indices. Preprocessing has the parameter dependence stated in the theorem.

Over $\mathbb{F}_q$, the numerator's S coefficients have at most q^S possible values. Each digit acts by a fixed linear map on this coefficient vector. Storing the vector avoids enumerating all reachable states.

Algebraic series as rational diagonals.

Theorem 554 (Rational diagonal of a simple branch)**.** *Over any field, suppose $F \in xK[[x]]$ satisfies*

$$P(x,F) = 0, \qquad P(0,0) = 0, \qquad P_y(0,0) \neq 0.$$

Then

$$\boxed{F(x) = \operatorname{Diag} \frac{v^2 P_y(uv,v)}{P(uv,v)}.}$$

After cancelling a factor v, the rational function is regular at the origin.

Proof. Factor over $K[[x]]$:

$$P(x,y) = (y - F(x))U(x,y), \qquad U(0,0) = P_y(0,0) \neq 0.$$

Then

$$\frac{v^2 P_y(uv,v)}{P(uv,v)} = \frac{v^2}{v - F(uv)} + v^2 \frac{U_y(uv,v)}{U(uv,v)}.$$

Every monomial of the second term has larger v-degree than u-degree, so its diagonal is zero.

Expand the first term as

$$\sum_{j \geq 0} v^{1-j} F(uv)^j.$$

This is formal because $F(uv)/v$ is divisible by u. Only $j = 1$ has equal degrees in u, v; it contributes $F(uv)$.

Finally, $P(uv,v)$ is divisible by v, and the quotient has constant term $P_y(0,0)$. □

Combining this with section operators gives logarithmic coefficient queries for simple algebraic branches over $\mathbb{F}_q$. For example, apply it to $F = C - 1$ for the Catalan equation $C = 1 + xC^2$.

Preparing a singular branch. Suppose $P(x,F) = 0$, $P_y(x,F) \neq 0$, and let $\nu = \operatorname{ord}_x P_y(x,F)$. Choose $t > \nu$, and write

$$F = h + x^t(g_0 + G), \qquad \deg h < t, \qquad G(0) = 0.$$

Then

$$Q(x,Y) = x^{-(t+\nu)} P\big(x, h + x^t(g_0 + Y)\big)$$

is polynomial, $Q(0,0) = 0$, and $Q_Y(0,0) \neq 0$.

Indeed, the coefficient of Y before division has valuation $t + \nu$; coefficients of Y-degree at least two have valuation at least $2t > t + \nu$; and the constant term has valuation at least $t + \nu + 1$, since $h + x^t g_0$ agrees with F modulo x^{t+1}.

For a finite-field algebraic series, a polynomial of minimum positive y-degree in $\mathbb{F}_q[x,y]$ annihilating it is separable in y. Otherwise, in characteristic p, write

$$P(x,y) = \sum_{r=0}^{p-1} x^r R_r(x,y)^p.$$

After substituting F, the summands occupy distinct exponent classes modulo p, so every $R_r(x,F)$ vanishes. A nonzero R_r has smaller y-degree, contradicting minimality. It follows that $P_y \neq 0$; its smaller y-degree also implies $P_y(x,F) \neq 0$. Thus the finite valuation ν used above exists.

This prepares any fixed algebraic branch from a sufficiently long initial coefficient prefix. The coefficients through index t are already known; for $N > t$, return $[x^{N-t}]G$ by the rational-diagonal algorithm.

Limitations. D-finite series are not closed under arbitrary composition or reciprocals. The single-coefficient method avoids requiring such closure by working with a target-dependent polynomial.

15.9 Fast Formal-Series Equations and Transformations

Work over a characteristic-zero field. Newton iteration solves formal equations by doubling precision; causal divide-and-conquer computes coefficients in the order imposed by their dependencies.

15.9.1 Newton Iteration for Scalar Differential Equations

Consider

$$y' = \Phi(x, y), \qquad y(0) = a,$$

where Φ is a polynomial of fixed degree in y, with power-series coefficients in x.

The coefficient of x^n determines $(n+1)y_{n+1}$ from earlier coefficients, so the formal solution is unique. Newton iteration groups the convolutions in this recurrence into precision-doubling steps.

Suppose $Y \equiv y \pmod{x^m}$, and put

$$E = \Phi(x, Y) - Y', \qquad A = \Phi_y(x, Y).$$

Choose h to solve

$$h' - Ah = E, \qquad h(0) = 0.$$

With $J = \int_0^x A$, multiplying by e^{-J} changes the equation to $(e^{-J}h)' = e^{-J}E$. The initial condition therefore gives

$$\boxed{h = e^{J} \int_0^x e^{-J} E.}$$

Since $E = O(x^{m-1})$, we have $h = O(x^m)$, and

$$\begin{aligned}(Y+h)' - \Phi(x, Y+h) &= Y' - \Phi(x, Y) + h' - Ah + O(h^2) \\ &= O(x^{2m}).\end{aligned}$$

A residual $O(x^r)$, together with the correct constant term, implies agreement with the solution modulo x^{r+1}. Otherwise the first nonzero term of the difference would differentiate to a lower-degree term than any term produced by $\Phi(x, Y) - \Phi(x, y)$. Thus the correction at least doubles precision. For the implementation, truncate the correction at $M = \min\{2m, N\}$. Only A and E modulo x^{M-1} are needed, since integration raises precision by one. The stored approximation has residual zero modulo x^{M-1}.

Theorem 555 (Fast formal ODE solution)**.** *The solution can be computed modulo x^N in*

$$\boxed{O(N \log(N+2))}$$

field operations and $O(N)$ space, for fixed degree of Φ in y, given its required coefficient prefixes.

Proof. Start from $Y = a \bmod x$. At precision M, evaluating Φ, Φ_y, then forming the integrating-factor correction, costs $O(M \log(M+2))$. Summing over geometrically increasing precisions proves the bound. □

Algorithm 123 Newton solution of a formal ODE

Input: $y' = \Phi(x, y)$, $y(0) = a$, precision $N \geq 1$
Output: $y \bmod x^N$

$Y \leftarrow a$, $m \leftarrow 1$
while $m < N$ **do**
  $M \leftarrow \min\{2m, N\}$
  $E \leftarrow \Phi(x, Y) - Y' \pmod{x^{M-1}}$
  $A \leftarrow \Phi_y(x, Y) \pmod{x^{M-1}}$
  $J \leftarrow \int_0^x A \pmod{x^M}$
  $U \leftarrow \exp(J) \pmod{x^M}$
  $h \leftarrow U \int_0^x (U^{-1}E) \pmod{x^M}$
  $Y \leftarrow Y + h \pmod{x^M}$
  $m \leftarrow M$
return Y

For example, $y' = 1 + y^2$, $y(0) = 0$, gives the tangent series. The same algorithm handles $y' = g(x) + y^2$ for a supplied series g.

The same algorithm applies to expressions Φ for which $\Phi(x, Y)$ and $\Phi_y(x, Y)$ can be evaluated within the stated bound, provided the required inverses and initial constants are defined. No D-finiteness assumption is needed.

15.9.2 CDQ Convolution and Causal Power-Series Equations

When input coefficients depend on earlier outputs, convolution must respect their order of determination. Use the *strictly causal* convolution

$$c_0 = 0, \qquad c_n = [x^{n-1}]A(x)B(x) = \sum_{i+j=n-1} a_i b_j \quad (n \geq 1).$$

Before choosing a_n, b_n, every term needed for c_n involves only earlier coefficients. A prescribed local rule may therefore read c_n and the completed prefixes, then choose a_n, b_n.

CDQ divide-and-conquer solves the left half, propagates its contributions to the right half, then solves the right half. The shift in $c_n = [x^{n-1}]AB$ makes this possible for arbitrary a_0, b_0.

Propagation between intervals. Consider a recursive interval $[l, r)$, with midpoint $m = \lfloor (l + r)/2 \rfloor$. After solving $[l, m)$, all coefficients with index below m are known. We must add to targets in $[m, r)$ exactly those pairs whose larger input index lies in $[l, m)$.

Use local block polynomials $A_{[l,m)} = \sum_{i=l}^{m-1} a_i x^{i-l}$, and similarly for B. Put $p = \min\{m, r - l\}$, $q = \min\{l, r - l\}$, and form

$$\boxed{U = A_{[l,m)} B_{[0,p)} + B_{[l,m)} A_{[0,q)}.} \tag{54}$$

Add $[x^{n-l-1}]U$ to c_n for $m \leq n < r$.

The first product handles pairs whose A-index is in the new block and whose B-index is below m. The second handles pairs whose B-index is in the new block and whose A-index is below l. Thus pairs with both indices in the new block are included only in the first product. The bounds p, q restrict the products to known coefficients and length $O(r - l)$. Prefix indices at least $r - l$ cannot contribute to targets below r.

Theorem 556 (Causal convolution by CDQ)**.** *For a rule that fixes a_n, b_n after receiving c_n, the first N stages of the preceding causal convolution can be generated in*

$$\boxed{O(N \log^2(N + 2))}$$

field operations, plus the work of the local rules, and $O(N)$ space. A fixed number of such convolutions can share the same recursion and local update within the same asymptotic bound.

Proof. Fix a contribution $a_i b_j$ to target $n = i + j + 1 < N$. Let $v = \max\{i, j\}$, so $v < n$. There is a unique lowest recursion node whose two children separate v from n. At this node $v \in [l, m)$, $n \in [m, r)$, and both input coefficients are known after the left recursion. Formula (54) includes this pair exactly once. No other node includes it, because no other split places its larger input index in the left child and its target in the right child. Every contribution is therefore accumulated before its target leaf is processed.

A node of length h performs two products of length $O(h)$, costing $O(h \log(h+2))$. At one recursion depth the interval lengths sum to N, and there are $O(\log(N+2))$ depths. Temporary products may be released before recursing right, giving $O(N)$ peak space. Repeating the products for constantly many convolutions changes only the constant factor. □

Algorithm 124 CDQ generation from a strictly causal convolution

Input: Length N, a local procedure that fixes a_n, b_n after receiving c_n
Output: $c_n = [x^{n-1}]AB$ for $1 \le n < N$, with $c_0 = 0$

```
Initialize a, b, c as zero arrays of length N
if N > 0 then
    SOLVE(0, N)

function SOLVE(l, r)
    if r − l = 1 then
        Run the local procedure at l, passing c_l and the completed prefixes; store a_l, b_l
        return
    m ← ⌊(l + r)/2⌋
    SOLVE(l, m)
    p ← min{m, r − l}, q ← min{l, r − l}
    U ← A_[l,m) B_[0,p) + B_[l,m) A_[0,q)
    for n = m, ..., r − 1 do
        c_n ← c_n + [x^{n−l−1}]U
    Release the temporary product U
    SOLVE(m, r)
```

Empty blocks represent zero polynomials. The local procedure never changes coefficients with smaller indices; it may maintain other state or schedule data for later indices.

Ordinary online convolution is the same primitive after a shift. Put $A_+ = (A - a_0)/x$ and $B_+ = (B - b_0)/x$. For $n \ge 2$,

$$[x^n]AB = a_0 b_n + a_n b_0 + [x^{n-2}]A_+ B_+.$$

At stage $n-1$, the causal engine for A_+, B_+ supplies the last term before receiving a_n, b_n. Add the two current-index terms after these inputs arrive; handle indices zero and one directly. Thus the routine also produces the usual convolution online within the same bound.

Exponential, inverse, logarithm, and powers. The differential identities reduce each operation to a fixed number of causal convolutions. After producing h_n, the local rule may choose f_{n+1}; thus f_n is known when h_n is computed. The input coefficients may also be supplied in advance.

Write $f = f_0 + xB$, so $B_n = f_{n+1}$, and put $\theta = xD$. The following formulas apply for $n \ge 1$.

For $h = \exp f$, assume $f_0 = 0$, set $h_0 = 1$, and use

$$h' = hf', \qquad nh_n = [x^{n-1}]hf'.$$

Feed $a_n = h_n$, $b_n = (n+1)f_{n+1}$ into one CDQ engine; its output is $c_n = nh_n$.

For $h = f^{-1}$, assume $f_0 \neq 0$, set $h_0 = f_0^{-1}$, and use

$$f_0 h_n = -[x^{n-1}]hB.$$

Feed $a_n = h_n$, $b_n = f_{n+1}$, and return $h_n = -c_n/f_0$ at each noninitial leaf.

For $\ell = \log(f/f_0)$, set $u = \theta\ell$, so $u_0 = 0$. The identity $fu = \theta f$ gives

$$f_0 u_n = n f_n - [x^{n-1}]Bu, \qquad \ell_n = u_n/n.$$

Use $a_n = u_n$, $b_n = f_{n+1}$, compute $u_n = (nf_n - c_n)/f_0$, and then $\ell_n = u_n/n$. Supply the initial coefficient $\ell_0 = 0$ separately.

Finally let $h = h_0(f/f_0)^\alpha$, with $f_0 \neq 0$ and chosen h_0. From $f\theta h = \alpha h \theta f$,

$$\boxed{n f_0 h_n = \alpha[x^{n-1}]hf' - [x^{n-1}]B\theta h.}$$

Use two engines, sharing the recursion. At leaf n, first compute $h_n = (\alpha c_n - d_n)/(nf_0)$. After the rule chooses f_{n+1}, feed the pairs

$$(h_n, (n+1)f_{n+1}), \qquad (nh_n, f_{n+1})$$

into the two engines. Their outputs are exactly c_n, d_n.

Each construction costs $O(N \log^2(N+2))$, in addition to the local rules. When all inputs are known in advance, use the faster offline algorithms.

Rooted unlabelled trees. Let $T(x) = \sum_{n\geq 1} t_n x^n$ count rooted unlabelled trees. A root carries a multiset of rooted trees, so

$$T(x) = x \exp\left(\sum_{j\geq 1} \frac{T(x^j)}{j}\right).$$

Logarithmic differentiation gives

$$xT' = T(1+G), \qquad G(x) = \sum_{n\geq 1} g_n x^n, \qquad g_n = \sum_{d|n} d\, t_d.$$

Thus $t_1 = 1$, and

$$(n-1)t_n = \sum_{j=1}^{n-1} t_j g_{n-j} \qquad (n \geq 2).$$

Use the CDQ engine with $a_i = t_{i+1}$, $b_i = g_{i+1}$. Its output at index $i \geq 1$ is $c_i = \sum_{j=1}^{i} t_j g_{i+1-j} = i\, t_{i+1}$. At leaf zero, set $t_1 = 1$; at every later leaf, set $t_{i+1} = c_i/i$. After finding t_n, add nt_n to $g_n, g_{2n}, g_{3n}, \ldots$ through the requested limit. At that point all divisors of n have been processed, so g_n is complete and can be fixed as the next input coefficient.

The divisor updates cost $\sum_{n\leq N} O(N/n) = O(N \log N)$. Consequently the first N rooted-tree counts take $O(N \log^2(N+2))$ operations and $O(N)$ space.

15.10 Fractional-Linear and Cubic Substitution

Fractional-linear substitution factors into Taylor shifts, scaling, and reversal. Combined with insertion or extraction of even coefficients, it gives a change of basis for symmetric Laurent polynomials and a near-linear algorithm for cubic substitution.

Throughout this subsection the field has characteristic zero. Degree bounds used for reversal and homogenization are retained even when the actual leading coefficients vanish.

15.10.1 Homogeneous Fractional-Linear Substitution

For a matrix $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ and a polynomial of degree at most n, define

$$\mathcal{H}_M^{(n)} F(x) = (cx+d)^n F\left(\frac{ax+b}{cx+d}\right).$$

This is a polynomial of degree at most n. The superscript records the chosen degree bound, not necessarily the actual degree of F.

Theorem 557 (Fast fractional-linear substitution)**.** *If $ad - bc \neq 0$, then $\mathcal{H}_M^{(n)} F$ can be computed in $O((n+1)\log(n+2))$ field operations. No nonvanishing assumption on the individual entries b, c, d is needed.*

Proof. For $c = 0$, we have $d \neq 0$, and the answer is $d^n F((a/d)x + b/d)$, an affine substitution. For $c \neq 0$, write

$$\frac{ax+b}{cx+d} = \frac{a}{c} + \frac{\kappa}{cx+d}, \qquad \kappa = \frac{bc-ad}{c}.$$

Compute $G(t) = F(t + a/c) = \sum_{j=0}^{n} g_j t^j$, then form

$$W(X) = \sum_{j=0}^{n} g_j \kappa^j X^{n-j}$$

by scaling and reversal. The answer is $W(cx+d)$. Two Taylor shifts and linear-time coefficient operations suffice. Division by d is not needed when $c \neq 0$. □

Algorithm 125 Homogeneous fractional-linear substitution

Input: $\deg F \leq n$, scalars a, b, c, d, $ad - bc \neq 0$
Output: $(cx+d)^n F((ax+b)/(cx+d))$

```
if c = 0 then
    G(t) ← F(t + b/d) by Taylor shift
    return d^n G((a/d)x)
G(t) ← F(t + a/c) by Taylor shift
κ ← (bc − ad)/c
W(X) ← Σ_{j=0}^{n} [t^j]G κ^j X^{n−j}
Compute U(X) = W(X + d) by Taylor shift
return U(cx)
```

If $d \neq 0$, the ordinary series $F((ax+b)/(cx+d)) \bmod x^N$ is obtained by dividing this output by $(cx+d)^n$. The total cost is $O((n+N+1)\log(n+N+2))$. If $d = 0$, the rational function may instead have a pole at zero; the homogeneous polynomial remains well-defined.

The outer input is a finite polynomial. An infinite series requires its expansion about the substituted constant term, rather than a truncation at zero.

Quadratic polynomial substitution. Constant substitution is ordinary evaluation, and affine substitution is a Taylor shift and a scaling. For $g(x) = ax^2 + bx + c$, $a \neq 0$, write

$$g(x) = a\left(x + \frac{b}{2a}\right)^2 + c - \frac{b^2}{4a}.$$

First make the affine change in the argument of F, then insert zeros to substitute x^2, and finally Taylor-shift x. The cost of computing $F(g(x))$ is near-linear in the output degree.

15.10.2 A Reversible Symmetric Laurent Substitution

Define

$$\mathcal{J}_n F(z) = z^n F(z + z^{-1}), \qquad \deg F \le n.$$

The output has degree at most $2n$ and satisfies $P(z) = z^{2n} P(1/z)$: it is reciprocal with respect to $2n$, including when its leading or trailing coefficients vanish.

Theorem 558 (Fast Joukowski transform and its inverse)**.** *The map $\mathcal{J}_n$ is a linear isomorphism from degree-$\le n$ polynomials to polynomials reciprocal with respect to $2n$. Both directions take $O((n+1)\log(n+2))$ operations.*

Proof. The forward map follows from the identity

$$z + z^{-1} = \frac{2(u+1)}{u-1}, \qquad u = \left(\frac{z+1}{z-1}\right)^2.$$

Compute

$$U(u) = (u-1)^n F\left(\frac{2(u+1)}{u-1}\right), \qquad V(v) = U(v^2).$$

Then

$$\boxed{\mathcal{J}_n F(z) = 4^{-n}(z-1)^{2n} V\left(\frac{z+1}{z-1}\right).} \tag{55}$$

Indeed, $((z+1)/(z-1))^2 - 1 = 4z/(z-1)^2$. Both fractional-linear substitutions are covered by the preceding theorem.

For the inverse, let P be reciprocal with respect to $2n$, and form

$$V(v) = (v-1)^{2n} P\left(\frac{v+1}{v-1}\right).$$

Reciprocity gives $V(-v) = V(v)$, so V has only even powers. Write $V(v) = U(v^2)$ by extracting its even coefficients, and set

$$\boxed{F(x) = 4^{-n}(x-2)^n U\left(\frac{x+2}{x-2}\right).} \tag{56}$$

To verify inversion, for $P = \mathcal{J}_n F$ the intermediate value is

$$V(v) = (v^2-1)^n F\left(\frac{2(v^2+1)}{v^2-1}\right).$$

Thus the recovered U is exactly the forward U, and (56) recovers F. The map $\mathcal{J}_n$ is injective, since the top nonzero power of F gives a distinct top power of z. Its domain and the reciprocal subspace both have dimension $n+1$, so it is onto. The formulas use two fractional-linear substitutions and one insertion or extraction of even coefficients in each direction. □

Extraction of even coefficients is valid because the transformed polynomial is even; no general substitution of $x^{1/2}$ is used.

Algorithm 126 Joukowski transform and inverse

function JOUKOWSKI(F, n)
  $U \leftarrow \mathcal{H}^{(n)}_{\left(\begin{smallmatrix}2 & 2\\ 1 & -1\end{smallmatrix}\right)} F$
  $V(v) \leftarrow U(v^2)$
  return $4^{-n}\mathcal{H}^{(2n)}_{\left(\begin{smallmatrix}1 & 1\\ 1 & -1\end{smallmatrix}\right)} V$
function INVERSEJOUKOWSKI(P, n)
  $V \leftarrow \mathcal{H}^{(2n)}_{\left(\begin{smallmatrix}1 & 1\\ 1 & -1\end{smallmatrix}\right)} P$
  $U(u) \leftarrow \sum_{j=0}^{n} [v^{2j}]V \, u^j$
  return $4^{-n}\mathcal{H}^{(n)}_{\left(\begin{smallmatrix}1 & 2\\ 1 & -2\end{smallmatrix}\right)} U$

Each $\mathcal{H}$ in this algorithm is computed by Algorithm 125 with the displayed degree bound, including any zero padding.

15.10.3 Chebyshev and Cubic Substitution

Let $C_0(x) = 2$, $C_1(x) = x$, and $C_{d+1}(x) = xC_d(x) - C_{d-1}(x)$. Induction gives

$$\boxed{C_d(z + z^{-1}) = z^d + z^{-d}.}$$

These are the rescaled Chebyshev polynomials $C_d(x) = 2T_d(x/2)$; in particular, $C_3(x) = x^3 - 3x$.

Theorem 559 (Fast composition with a Chebyshev polynomial)**.** *For* $\deg F \le n$ *and* $d \ge 1$*, the polynomial* $F(C_d(x))$ *can be computed in*

$$\boxed{O((dn+1)\log(dn+2))}$$

operations, without first expanding C_d*.*

Proof. Compute $P = \mathcal{J}_n F$, insert zeros to form $P(z^d)$, and apply the inverse transform with degree bound dn. Indeed,

$$P(z^d) = z^{dn} F(z^d + z^{-d}) = \mathcal{J}_{dn}(F \circ C_d)(z).$$

The output size bounds the cost of all three operations. □

Theorem 560 (Fast composition with an arbitrary cubic)**.** *If* $\deg F \le n$ *and* g *has degree at most three, then*

$$\boxed{F(g(x))}$$

can be computed in $O((n+1)\log(n+2))$ *operations over* K*. No scalar square-root computation is required.*

Proof. Degrees at most two were handled above. Write $g(x) = ax^3 + bx^2 + cx + d$, with $a \neq 0$, and set

$$h = \frac{b}{3a}, \qquad p = \frac{c}{a} - 3h^2, \qquad q = d - a(h^3 + ph).$$

Then $g(x) = a((x+h)^3 + p(x+h)) + q$. If $p = 0$, affine substitutions and $x \mapsto x^3$ suffice.

For $p \neq 0$, work in the two-dimensional K-algebra

$$E = K[\xi]/(\xi^2 + p/3).$$

The element ξ is a unit, with inverse $-3\xi/p$. Define $\widetilde{F}(t) = F(a\xi^3 t + q)$. Then

$$F(g(x)) = \widetilde{F}\left(C_3\left(\frac{x+h}{\xi}\right)\right),$$

because $\xi^3 C_3(u/\xi) = u^3 - 3\xi^2 u = u^3 + pu$. Compute the affine change defining $\widetilde{F}$, compose with C_3 by Theorem 559, and make the final affine substitution $t = (x+h)/\xi$.

Every element of E is represented by two coefficients in K. Addition and multiplication use a constant number of base-field operations; the only additional inverse is that of ξ. All other divisions in the Taylor shifts and homogeneous substitutions are by nonzero scalars from K. Hence E need not be a field, and the overhead is constant. The result equals $F(g(x)) \in K[x]$, so its ξ-components vanish and its constant components are the required coefficients. □

Algorithm 127 Composition with a cubic polynomial

Input: $\deg F \le n$, $g(x) = ax^3 + bx^2 + cx + d$, $a \neq 0$
Output: $F(g(x))$

$h \leftarrow b/(3a)$, $p \leftarrow c/a - 3h^2$, $q \leftarrow d - a(h^3 + ph)$
if $p = 0$ **then**
  $G(t) \leftarrow F(at + q)$
  return $G((x+h)^3)$, by insertion of zeros and a Taylor shift
Use coefficient pairs for $E = K[\xi]/(\xi^2 + p/3)$
$G(t) \leftarrow F(a\xi^3 t + q)$
$P \leftarrow \textsc{Joukowski}(G, n)$
$Q(z) \leftarrow P(z^3)$
$H \leftarrow \textsc{InverseJoukowski}(Q, 3n)$
$R(x) \leftarrow H((x+h)/\xi)$
return the constant component of every coefficient of R

For a fixed substitution, these maps are linear in the outer coefficients. Their transposes give power projection and coefficient pairing within the same bounds.

15.11 Dirichlet Series

Ordinary power series use addition of indices; Dirichlet series use multiplication. Write $e_n = n^{-s}$ as a formal symbol with $e_a e_b = e_{ab}$. Then

$$f = \sum_{n \ge 1} f_n e_n, \qquad (f * g)_n = \sum_{ab=n} f_a g_b, \qquad \delta = e_1.$$

We work over a characteristic-zero field until stated otherwise. No analytic convergence is involved. The coefficient f_1, not f_0, plays the role of the constant term, and f is a unit exactly when $f_1 \neq 0$.

Unique factorization identifies e_n with $\prod_p x_p^{v_p(n)}$. This gives a power-series algebra in prime-indexed variables, allowing infinitely many monomials even of one fixed total degree. The operations are defined coefficientwise: each monomial of a product receives only finitely many contributions.

Full coefficient arrays are computed by triangular divisor recurrences. Compressed prefix tables support convolution but not arbitrary coefficientwise operations, so they require different logarithm and exponential algorithms.

15.11.1 The Dirichlet Derivative and Triangular Recurrences

For $a_1 = 0$, define

$$\exp_*(a) = \sum_{j\ge 0} \frac{a^{*j}}{j!}, \qquad \log_*(\delta + a) = \sum_{j\ge 1} \frac{(-1)^{j+1}}{j} a^{*j}.$$

These sums are coefficientwise finite: a product of j indices at least two is at least 2^j. More precisely, the coefficient at n can involve at most $\Omega(n)$ nonconstant factors, where $\Omega(n) = \sum_p v_p(n)$.

For every unit f, define its *normalized logarithm* by

$$\operatorname{Log}_* f = \log_*(f/f_1).$$

It has coefficient zero at 1, and does not require a scalar logarithm of f_1. In particular, $\exp_*(\operatorname{Log}_* f) = f/f_1$.

The Euler derivation. The prime-variable interpretation suggests the Euler derivation

$$\partial f = \sum_{n\ge 1} \Omega(n) f_n e_n.$$

Since $\Omega(ab) = \Omega(a) + \Omega(b)$,

$$\boxed{\partial(f * g) = (\partial f) * g + f * (\partial g).} \tag{57}$$

Every $\Omega(n)$ with $n > 1$ is positive. Thus, once the coefficient at 1 is prescribed, integration amounts to dividing the n-th coefficient by $\Omega(n)$.

The usual differentiation identities now hold formally:

$$f * \partial(\operatorname{Log}_* f) = \partial f, \qquad \partial(\exp_* a) = (\exp_* a) * \partial a.$$

Apply (57) termwise to the coefficientwise finite defining sums.

Logarithm and exponential. For $\ell = \operatorname{Log}_* f$, coefficient comparison gives

$$\boxed{f_1 \Omega(n) \ell_n = \Omega(n) f_n - \sum_{\substack{d\mid n \\ 1<d<n}} \Omega(d) \ell_d f_{n/d}, \qquad \ell_1 = 0.} \tag{58}$$

For $h = \exp_* a$, where $a_1 = 0$, it gives

$$\boxed{\Omega(n) h_n = \Omega(n) a_n + \sum_{\substack{d\mid n \\ 1<d<n}} h_d \Omega(n/d) a_{n/d}, \qquad h_1 = 1.} \tag{59}$$

Every unknown on either right-hand side has smaller index.

Evaluate the recurrences directly with divisor-sieve accumulators. For the logarithm, initialize $R_n = \Omega(n) f_n$ for the logarithm. After computing $\ell_d = R_d/(f_1 \Omega(d))$, propagate $-\Omega(d)\ell_d f_b$ to R_{db} for $b \ge 2$. For the exponential, initialize $R_n = \Omega(n) a_n$, compute $h_d = R_d/\Omega(d)$, and propagate $h_d \Omega(b) a_b$ to R_{db}.

Powers, inverses, and unit roots. For a unit f, a scalar $\alpha \in K$, and a prescribed $c \in K$, define

$$h = c \exp_*\big(\alpha \operatorname{Log}_* f\big).$$

The scalar factor is supplied separately; f_1^α need not be defined in the coefficient field. Its differential equation is

$$\boxed{f * \partial h = \alpha h * \partial f, \qquad h_1 = c.}$$

Comparing coefficients and reindexing one sum gives

$$\boxed{f_1\Omega(n)h_n = \alpha c\Omega(n)f_n + \sum_{\substack{d\mid n\\ 1<d<n}} \big(\alpha\Omega(n/d) - \Omega(d)\big)h_d f_{n/d}.} \tag{60}$$

The condition $h_1 = c$ and this recurrence uniquely determine h.

For an inverse, take $\alpha = -1$, $c = f_1^{-1}$. For an integer power f^{*k}, take $\alpha = k$, $c = f_1^k$. For a k-th root of a unit, choose a scalar c with $c^k = f_1$ and take $\alpha = 1/k$; the resulting series is the unique root with that coefficient at 1. The scalar root must be supplied, and k must be invertible in the coefficient field.

For $\alpha = -1$, the summand weight becomes $-\Omega(n)$. Cancelling $\Omega(n)$ recovers the simpler ordinary inverse recurrence. More generally, solving $f * g = h$ directly uses

$$g_1 = h_1/f_1, \qquad g_n = \frac{h_n - \sum_{\substack{d\mid n\\ d>1}} f_d g_{n/d}}{f_1}.$$

Theorem 561 (Full Dirichlet-series operations)**.** *Through index N, convolution, division by a unit, normalized logarithm, exponential, and the normalized powers and unit roots just described can be computed in*

$$\boxed{O(N\log(N+2))}$$

field operations and $O(N)$ space. The cost of obtaining any supplied scalar power or scalar root is separate.

Proof. A sieve computes $\Omega(1),\dots,\Omega(N)$. Each triangular recurrence is processed in increasing index order by propagating contributions along pairs $db \le N$. There are $\sum_{d\le N}\lfloor N/d\rfloor = O(N\log N)$ such pairs. Convolution and division have the same enumeration bound. Each recurrence uniquely determines the coefficients of its solution. □

Algorithm 128 A Dirichlet power with prescribed coefficient at one

Input: $f_1,\dots,f_N$, $f_1 \neq 0$, scalars α, c
Output: $h = c\exp_*(\alpha\operatorname{Log}_* f)$ through N

```
Sieve Ω(1), ..., Ω(N)
h_1 ← c
R_n ← αcΩ(n) f_n for 2 ≤ n ≤ N
for d = 2, ..., N do
    h_d ← R_d/(f_1 Ω(d))
    for b = 2, ..., ⌊N/d⌋ do
        R_db ← R_db + (αΩ(b) − Ω(d)) h_d f_b
return h
```

For an integer exponent k, computing f_1^k adds $O(\log(|k|+2))$ scalar operations. If $f_1 = 0$, the unit recurrences do not apply. Nonnegative integer powers still exist (the zeroth power is δ): if the first nonzero index is $m \ge 2$, then f^{*k} is supported on indices at least m^k. It therefore vanishes through N when $m^k > N$; otherwise binary powering costs $O(N\log(N+2)\log(k+1))$. Nonunit roots may also exist, but are not covered by the unit-root recurrence.

The displayed unit algorithms also work in positive characteristic when $1,\dots,\lfloor\log_2 N\rfloor$ are invertible, together with any separately required scalar denominators. The divisions by $\Omega(n)$ and the defining logarithm and exponential sums then remain valid. Plain convolution and division by a unit do not need this characteristic restriction.

Newton iteration. The usual identities remain valid:

$$g \longmapsto g * (2\delta - f * g), \qquad g \longmapsto g * (\delta + a - \log_* g),$$

where the second iteration maintains $g_1 = 1$. An error supported on indices at least m has square supported on indices at least m^2. Thus squaring the error squares its index threshold. The triangular recurrences give the same asymptotic bound for full arrays.

15.11.2 Compressed Prefix Tables

Fix $N \geq 2$, and let

$$\mathcal{Q}_N = \{0\} \cup \{\lfloor N/k \rfloor : 1 \leq k \leq N\}.$$

Store f by $F(v) = \sum_{n \leq v} f_n$, $v \in \mathcal{Q}_N$.

Put $s = \lfloor \sqrt{N} \rfloor$, $t = \lfloor N/(s+1) \rfloor$, and $K = s + t$. The positive arguments are $1, \ldots, s$, followed by t larger quotients.

For a stored positive argument,

$$\mathrm{idx}(v) = \begin{cases} v, & v \leq s, \\ K - \lfloor N/v \rfloor + 1, & v > s. \end{cases}$$

For an arbitrary positive threshold a, the first stored argument at least a has index

$$\mathrm{first}(a) = \begin{cases} a, & a \leq s, \\ K - \lfloor N/a \rfloor + 1, & a > s. \end{cases}$$

Index $K + 1$ indicates that no stored argument is large enough. Thus suffix updates at arbitrary thresholds take constant time.

The quotient arguments are closed under division:

$$\left\lfloor \frac{\lfloor N/a \rfloor}{b} \right\rfloor = \left\lfloor \frac{N}{ab} \right\rfloor.$$

Also $v \mapsto \lfloor N/v \rfloor$ is an involution on the positive quotient grid. Indeed, if $v = \lfloor N/k \rfloor$ and $w = \lfloor N/v \rfloor$, then $w \geq k$ and $vw \leq N$, so $v \leq \lfloor N/w \rfloor \leq \lfloor N/k \rfloor = v$. The large arguments are the distinct values $\lfloor N/j \rfloor$ for $1 \leq j \leq t$, listed in reverse order of j. For $a > s$, exactly $\lfloor N/a \rfloor$ of them are at least a, proving the formula for $\mathrm{first}(a)$.

Compatibility with convolution.

Lemma 562 (Compression respects convolution)**.** *If two Dirichlet series have the same prefix table on $\mathcal{Q}_N$, they are interchangeable in any sum, scalar multiple, or convolution whose output is another such table. The same holds for normalized logarithms, exponentials, inverses, and powers whenever these are defined through index N.*

Proof. Only convolution needs proof. For $h = f * g$,

$$H(v) = \sum_{b \leq v} g_b F(\lfloor v/b \rfloor).$$

All arguments $\lfloor v/b \rfloor$ belong to $\mathcal{Q}_N$, so replacing f by a series with the same stored prefixes changes no output. By commutativity, the same holds for replacing g. The formal functions in the statement are finite convolution expressions through index N, after normalizing their coefficient at one. That coefficient is itself stored as $F(1)$. □

Compressed prefix tables do not determine ∂. For $N = 10$, $e_6 - e_7$ has zero prefix sum on $\{0, 1, 2, 3, 5, 10\}$, but its derivative $2e_6 - e_7$ has prefix sum one at 10. The full-array logarithmic-derivative algorithm therefore cannot be used directly on compressed tables.

15.11.3 Compressed Multiplication

Write

$$J(z) = H(\lfloor N/z \rfloor) = \sum_{\substack{x,y \geq 1 \\ xyz \leq N}} f_x g_y, \qquad z \in \mathcal{Q}_N \setminus \{0\}.$$

This indexes the output table by the reversed quotient grid.

Partition the triples by their largest coordinate, breaking ties in the order x, y, z:

$$x \geq y,\ x \geq z; \qquad y > x,\ y \geq z; \qquad z > x,\ z > y.$$

Fix the two smaller coordinates and sum over the interval of possible values of the largest. For an input coordinate this uses a prefix sum; for the output coordinate it uses a range addition.

For an ordered pair (a, b) with $ab \max\{a, b\} \leq N$, put $m = \max\{a, b\}$ and $r = \lfloor N/(ab) \rfloor$.

When x is largest, set $y = a, z = b$, and add

$$g_a(F(r) - F(m-1))$$

to $J(b)$.

When y is largest, set $x = a, z = b$. Its interval starts at $\ell = \max\{a+1, b\}$. If $\ell \leq r$, add

$$f_a(G(r) - G(\ell - 1))$$

to $J(b)$.

When z is largest, set $x = a, y = b$. Range-add $f_a g_b$ to stored $J(z)$ with $m + 1 \leq z \leq r$.

The fixed coordinates are at most $\sqrt{N}$, so their coefficients are differences of stored prefixes. The upper endpoint r is a quotient argument, and every lower endpoint minus one is at most s.

Theorem 563 (Compressed Dirichlet multiplication)**.** *Given arbitrary compressed prefix tables F, G, the product table can be computed in*

$$\boxed{O(N^{2/3})}$$

operations and $O(\sqrt{N})$ space.

Proof. The three regions partition all triples. Write $u = \min\{a, b\}$, $v = \max\{a, b\}$. Then $u \leq N^{1/3}$ and $u \leq v \leq \sqrt{N/u}$. The number of fixed pairs is

$$O\left(\sum_{u \leq N^{1/3}} \sqrt{\frac{N}{u}}\right) = O(N^{2/3}).$$

Each pair gives a constant number of point or range updates to a difference array on the quotient grid. Prefix-sum it and undo the quotient reindexing. □

Algorithm 129 Compressed Dirichlet multiplication

Input: Prefix tables F, G on $\mathcal{Q}_N$
Output: Prefix table H of $f * g$

```
Recover f_a, g_a for 1 ≤ a ≤ s by prefix differences
Initialize a difference array for J(z) on the positive quotient grid
for u = 1, ..., ⌊N^{1/3}⌋ do
    for v = u, ..., ⌊√(N/u)⌋ do
        ProcessPair(u, v)
        if u < v then
            ProcessPair(v, u)
Prefix-sum the difference array
H(v) ← J(⌊N/v⌋), H(0) ← 0
return H

function ProcessPair(a, b)
    m ← max{a, b}, r ← ⌊N/(ab)⌋
    Point-add g_a(F(r) − F(m − 1)) to J(b)
    ℓ ← max{a + 1, b}
    if ℓ ≤ r then
        Point-add f_a(G(r) − G(ℓ − 1)) to J(b)
    if m + 1 ≤ r then
        Range-add f_a g_b on stored arguments in [m + 1, r]
```

A range $[\ell, r]$, with r stored, uses difference-array endpoints $\mathrm{first}(\ell)$ and $\mathrm{idx}(r) + 1$.

15.11.4 Compressed Logarithms and Exponentials

The defining logarithm and exponential use $O(\log N)$ convolution powers. Eliminating an initial range of nonconstant coefficients reduces this to a constant number of products.

Removing one elementary factor costs $O(\sqrt{N})$. Choose B so that preprocessing costs no more than an $O(N^{2/3})$ compressed product: the balance $B\sqrt{N} = N^{2/3}$ gives $B = \lfloor N^{1/6} \rfloor$. The remaining nonconstant support starts at $B + 1$, whose sixth power exceeds N, so only five convolution powers contribute.

Elementary factors. Multiplication by $\delta - ce_m$, $m \geq 2$, changes prefixes by

$$H(v) = F(v) - cF(\lfloor v/m \rfloor).$$

Perform this in decreasing order of v.

Division satisfies

$$H(v) = F(v) + cH(\lfloor v/m \rfloor),$$

and is performed in increasing order. Each operation costs $O(\sqrt{N})$.

Eliminating small coefficients. For a general unit, first divide its prefix table by f_1; this does not change its normalized logarithm. With $f_1 = 1$, process $m = 2, \ldots, B$. Let c_m be the current coefficient at m, and multiply by $\delta - c_m e_m$. This kills coefficient m without changing smaller coefficients.

The result is $R = \delta + u$, with $u_n = 0$ for $n \leq B$, and

$$f = R * \prod_{m=2}^{B} (\delta - c_m e_m)^{-1}.$$

Hence

$$\boxed{\operatorname{Log}_* f = \sum_{j=1}^{5} \frac{(-1)^{j+1}}{j} u^{*j} + \sum_{m=2}^{B} \sum_{k:\, m^k \le N} \frac{c_m^k}{k} e_{m^k}.}$$

The last sum has $O(B \log N)$ explicitly listed terms. Generate m^k and c_m^k successively for each m. A term γe_a contributes γ to precisely the stored prefixes with argument at least a, so it is one suffix update at $\operatorname{first}(a)$, without storing the full N-coefficient array.

Exponential. Given $a_1 = 0$, choose $c_2, \ldots, c_B$ so that

$$L = \sum_{m=2}^{B} \sum_{k:\, m^k \le N} \frac{c_m^k}{k} e_{m^k}$$

agrees with a through index B. This is triangular: the contribution of c_m at index m is c_m, and its remaining contributions occur at larger indices.

Then $r = a - L$ vanishes through B, and

$$\boxed{\exp_* a = \left(\delta + \sum_{j=1}^{5} \frac{r^{*j}}{j!}\right) * \prod_{m=2}^{B} (\delta - c_m e_m)^{-1}.}$$

Theorem 564 (Compressed formal operations)**.** *Normalized logarithms, exponentials, inverses, quotients, and fixed normalized powers can be computed on $\mathcal{Q}_N$ in*

$$\boxed{O(N^{2/3})}$$

operations and $O(\sqrt{N})$ space.

Proof. The B elementary-factor updates cost $O(B\sqrt{N}) = O(N^{2/3})$. The truncated logarithm or exponential uses a constant number of compressed products, and constructing the correction costs $O(B \log N + \sqrt{N})$.

For a unit, use $f^{-1} = f_1^{-1} \exp_*(-\operatorname{Log}_* f)$. Normalized powers are $(f/f_1)^{*\alpha} = \exp_*(\alpha \operatorname{Log}_* f)$; a prescribed scalar factor can be restored afterward. A quotient needs one additional compressed product. □

Algorithm 130 Compressed Dirichlet logarithm

Input: Prefix table F, with $f_1 \neq 0$
Output: Prefix table of $\operatorname{Log}_* f$

```
B ← ⌊N^{1/6}⌋, R ← F/F(1)
for m = 2, ..., B do
    c_m ← R(m) − R(m − 1)
    for positive v ∈ Q_N in decreasing order do
        R(v) ← R(v) − c_m R(⌊v/m⌋)
Build L by suffix-adding c_m^k/k at threshold m^k, for all m^k ≤ N
u ← R − δ, P ← u, A ← L
for j = 1, ..., 5 do
    A ← A + (−1)^{j+1} P/j
    if j < 5 then
        P ← P ∗ u by compressed multiplication
return A
```

Algorithm 131 Compressed Dirichlet exponential

Input: Prefix table A, with $a_1 = 0$
Output: Prefix table of $\exp_* a$

```
B ← ⌊N^{1/6}⌋
b_m ← A(m) − A(m − 1) for 2 ≤ m ≤ B
Initialize a difference array for L
for m = 2, ..., B do
    c_m ← b_m
    for k ≥ 1 with m^k ≤ N do
        Suffix-add c_m^k/k to L at threshold m^k
        if m^k ≤ B then
            b_{m^k} ← b_{m^k} − c_m^k/k
Prefix-sum the difference array to obtain L
r ← A − L, P ← δ, E ← δ
for j = 1, ..., 5 do
    P ← P ∗ r by compressed multiplication
    E ← E + P/j!
for m = 2, ..., B do
    for positive v ∈ Q_N in increasing order do
        E(v) ← E(v) + c_m E(⌊v/m⌋)
return E
```

The prefix table of δ is one at every positive argument and zero at zero. Individual coefficients are used only in the small range where consecutive prefixes are stored.

15.11.5 Prime Counting

Let $Z = \sum_{n \ge 1} e_n$. Unique prime factorization gives

$$Z = \prod_{p \text{ prime}} (\delta - e_p)^{-1}, \qquad \boxed{\operatorname{Log}_* Z = \sum_{p} \sum_{k \ge 1} \frac{1}{k} e_{p^k}.}$$

The logarithm is supported only on prime powers.

If $J(x)$ is its prefix sum, then

$$J(x) = \sum_{k=1}^{\lfloor \log_2 x \rfloor} \frac{1}{k} \pi(\lfloor x^{1/k} \rfloor).$$

Therefore

$$\boxed{\pi(N) = J(N) - \sum_{k=2}^{\lfloor \log_2 N \rfloor} \frac{1}{k} \pi(\lfloor N^{1/k} \rfloor).}$$

Theorem 565 (Prime counting by Dirichlet logarithms)**.** *The value $\pi(N)$ can be computed in $O(N^{2/3})$ arithmetic operations and $O(\sqrt{N})$ space.*

Proof. The prefix table of Z is $Z(v) = v$. Compute its logarithm table to obtain $J(N)$. Every correction argument is at most $\sqrt{N}$, so a sieve through $\lfloor \sqrt{N} \rfloor$ supplies the remaining prime counts. Compute the roots as exact integer roots. □

Algorithm 132 Prime counting by a compressed Dirichlet logarithm

Input: Integer $N \geq 2$
Output: $\pi(N)$

Construct $\mathcal{Q}_N$, and set $Z(v) \leftarrow v$
Compute the prefix table J of $\mathrm{Log}_* Z$ using Algorithm 130
Sieve prime counts through $\lfloor \sqrt{N} \rfloor$
$a \leftarrow J(N)$
for $k = 2, \ldots, \lfloor \log_2 N \rfloor$ **do**
    $r \leftarrow \max\{j \in \mathbb{Z}_{\geq 0} : j^k \leq N\}$
    $a \leftarrow a - \pi(r)/k$
return a

For Z, the small-factor coefficients c_m are one at primes and zero at composites. Thus the logarithm algorithm is a partial sieve followed by a short formal logarithm.

For a completely multiplicative weight w,

$$\mathrm{Log}_* \left(\sum_{n \geq 1} w(n) e_n \right) = \sum_p \sum_{k \geq 1} \frac{w(p)^k}{k} e_{p^k}.$$

Whenever the weighted prefix table is available, the same method computes weighted prime sums after subtracting higher prime powers. For instance, $w(n) = n$ gives the sum of primes up to N.

15.11.6 Logarithmic Binning and Boundary Correction

Instead of the exact interval grouping used in the $N^{2/3}$ algorithm, approximate the product comparison by adding logarithmic indices. Ordinary convolution computes these approximate sums; exact correction over a short boundary interval gives $N^{1/2+o(1)}$ time.

Large coordinates. Put $s = \lfloor \sqrt{N} \rfloor$. Compute the product coefficients and their prefixes through s by the divisor-sieve algorithm in $O(s \log(s+2))$ operations. These are all the small quotient outputs. The remaining outputs have the form $H(\lfloor N/z \rfloor)$ with $1 \leq z \leq s$.

For each such z, the terms with $x > s$ contribute

$$\sum_{y \leq N/(z(s+1))} g_y \big(F(\lfloor N/(yz) \rfloor) - F(s) \big).$$

The terms with $y > s$ are symmetric, and the two cases are disjoint because $(s+1)^2 > N$. All required coefficients and prefixes are stored. Over $z \leq s$, these sums cost $\sum_{z \leq s} O(N/(z(s+1))) = O(\sqrt{N} \log N)$. It remains to find

$$T_z = \sum_{\substack{x,y \leq s \\ xyz \leq N}} f_x g_y \qquad (1 \leq z \leq s).$$

Logarithmic bins. At logarithmic scale b, the arrays have length $O(b \log N)$ and rounding affects an interval of length $O(N/b)$. Balance the two by choosing $b = \lceil \sqrt{N} \rceil$.

Choose nonnegative integers $\ell(j)$, for $1 \leq j \leq s$ and for $j = N$, satisfying

$$|\ell(j) - b \log_2 j| \leq 1.$$

Define the ordinary polynomials

$$A_{\mathrm{bin}}(u) = \sum_{x=1}^{s} f_x u^{\ell(x)}, \qquad B_{\mathrm{bin}}(u) = \sum_{y=1}^{s} g_y u^{\ell(y)},$$

and compute $C(u) = A_{\mathrm{bin}}(u)B_{\mathrm{bin}}(u)$. The multiplication uses ordinary convolution of the bin indices. Let $\widehat{C}(t) = \sum_{r\le t}[u^r]C$, with value zero for $t < 0$ and the full coefficient sum for $t \ge \deg C$.

Set $L = \ell(N) + 4$. The provisional answer

$$\widehat{T}_z = \widehat{C}(L - \ell(z))$$

is the sum over all $x, y \le s$ for which $\ell(x) + \ell(y) + \ell(z) \le L$. Every triple satisfying the original product bound is included, since

$$xyz \le N \quad\Longrightarrow\quad \ell(x) + \ell(y) + \ell(z) \le b\log_2 N + 3 \le L.$$

Conversely, an included triple satisfies

$$b\log_2(xyz) \le \ell(x) + \ell(y) + \ell(z) + 3 \le b\log_2 N + 8.$$

For $b \ge 8$, the convexity inequality $2^u \le 1 + u$ on $0 \le u \le 1$ therefore gives

$$\boxed{N < xyz \le N2^{8/b} \le N + 8N/b} \tag{61}$$

for every false positive. It remains to subtract these excess terms.

Boundary correction. Put $W = \lceil 8N/b\rceil$. Factor all integers in $(N, N + W]$ by a segmented sieve. Explicitly, initialize a residual entry equal to each integer in the interval, sieve the primes through $\sqrt{N+W}$, and divide each interval multiple repeatedly by the current prime, recording its exponent. Any final residual greater than one is prime. The prime sieve and the divisions take $O(\sqrt{N}\log(N+2))$ elementary operations.

For every factored integer m, enumerate its ordered factorizations $xyz = m$. Retain those with $x, y, z \le s$ and $\ell(x) + \ell(y) + \ell(z) \le L$, and subtract $f_x g_y$ from $\widehat{T}_z$. By (61), these are exactly the spurious contributions. This argument does not require positive coefficients.

If $m = \prod_p p^{a_p}$, the number of ordered factorizations is

$$d_3(m) = \prod_p \binom{a_p + 2}{2} = m^{o(1)}.$$

To prove the estimate, fix $\varepsilon > 0$. For sufficiently large p, $\binom{a+2}{2} \le 3^a \le p^{\varepsilon a}$. For each of the finitely many smaller primes, the polynomial in a is at most a constant times $p^{\varepsilon a}$. Multiplying gives $d_3(m) = O_\varepsilon(m^\varepsilon)$.

To enumerate the factorizations, recursively distribute each prime exponent among the three coordinates, multiplying their partial values by the corresponding powers. The recursion has at most $O(d_3(m)\log(m+2))$ nodes. Consequently, correcting all $O(\sqrt{N})$ boundary integers costs $N^{1/2+o(1)}$.

Theorem 566 (Square-root-scale compressed multiplication)**.** *Compressed Dirichlet multiplication on $\mathcal{Q}_N$ can be performed in*

$$\boxed{N^{1/2+o(1)}}$$

field and index-arithmetic operations, using $O(\sqrt{N}\log(N+2))$ stored coefficients and index data. No coefficient-sign restriction is needed.

Proof. The small outputs and large-coordinate contributions cost $O(\sqrt{N}\log N)$. The bin polynomials have degree $O(\sqrt{N}\log N)$, so their ordinary product costs $O(\sqrt{N}\log^2 N)$. The segmented sieve and the exact boundary correction have the cost proved above. All large quotient outputs are then obtained by adding back the separately computed large-coordinate contributions.

To compute the logarithmic indices, approximate $\log_2 j$ to error at most $1/(2b)$, multiply by b, and round to a nearest integer. The resulting error is at most one. To obtain the approximation, separate off the integer binary exponent and reduce the argument to $[1, 2]$. On that interval,

$$\log u = 2\sum_{k\ge0}\frac{z^{2k+1}}{2k+1}, \qquad z = \frac{u-1}{u+1}, \quad |z| \le \frac{1}{3}.$$

The tail decreases geometrically. Using $O(\log N)$ terms and polylogarithmic-bit arithmetic, including an approximation to $\log 2$, gives the required accuracy. The bound counts field operations on coefficients and bit operations for the logarithmic indices. □

Algorithm 133 Compressed multiplication by logarithmic binning

Input: Prefix tables F, G on $\mathcal{Q}_N$
Output: Prefix table H of $f * g$

if $N < 64$ **then**
  return the product from Algorithm 129
$s \leftarrow \lfloor \sqrt{N} \rfloor$, $b \leftarrow \lceil \sqrt{N} \rceil$
Recover $f_1, \dots, f_s$ and $g_1, \dots, g_s$
Compute $H(1), \dots, H(s)$ by a divisor sieve
Compute admissible $\ell(j)$ for $j \le s$ and $j = N$
Form $A_{\mathrm{bin}}(u), B_{\mathrm{bin}}(u)$; multiply them as ordinary polynomials
Let $\widehat{C}$ be the prefix sums of the product coefficients
$L \leftarrow \ell(N) + 4$, $W \leftarrow \lceil 8N/b \rceil$
$T_z \leftarrow \widehat{C}(L - \ell(z))$ for $1 \le z \le s$
Factor every integer in $(N, N + W]$ by the segmented sieve described above
for each such integer m **do**
  for each allocation of its prime exponents among x, y, z **do**
    if $x, y, z \le s$ and $\ell(x) + \ell(y) + \ell(z) \le L$ **then**
      $T_z \leftarrow T_z - f_x g_y$
for $z = 1, \dots, s$ **do**
  $R \leftarrow T_z$
  for $a = 1, \dots, \lfloor N/(z(s+1)) \rfloor$ **do**
    $R \leftarrow R + g_a(F(\lfloor N/(az) \rfloor) - F(s))$
    $R \leftarrow R + f_a(G(\lfloor N/(az) \rfloor) - G(s))$
  $H(\lfloor N/z \rfloor) \leftarrow R$
$H(0) \leftarrow 0$
return H

Using this multiplication routine, the defining logarithm and exponential series need only $O(\log N)$ products, absorbed into $N^{o(1)}$. Normalized logarithms, exponentials, inverses, powers, and prime counting therefore also admit $N^{1/2+o(1)}$-time algorithms.

16 Set Power Series

16.1 Subset Convolution

Given two functions $f, g : 2^V \to \mathbb{F}$ over a set V of size n, their subset convolution $h = f * g$ is defined as:

$$h(S) = \sum_{A \subseteq S} f(A) g(S \setminus A)$$

Computing this naively requires $O(3^n)$ time. We can optimize this to $O(n^2 2^n)$ using the Zeta and Möbius transforms.

Define the ranked representations $\hat{f}$ and $\hat{g}$:

$$\hat{f}_k(S) = \begin{cases} f(S) & \text{if } |S| = k \\ 0 & \text{otherwise} \end{cases}$$

The Zeta transform computes: $\mathcal{Z}(f)(S) = \sum_{A \subseteq S} f(A)$. We have that point-wise multiplication after the Zeta transform computes:

$$\mathcal{Z}^{-1}\big(\mathcal{Z}(u) \cdot \mathcal{Z}(v)\big)(S) = \sum_{A \cup B = S} u(A)v(B)$$

By performing a standard linear convolution over the rank dimension and a point-wise multiplication in the Zeta domain, we compute a new ranked function $\hat{h}$:

$$\hat{h}_i(S) = \mathcal{Z}^{-1}\left(\sum_{j=0}^{i} \mathcal{Z}(\hat{f}_j) \cdot \mathcal{Z}(\hat{g}_{i-j})\right)(S) = \sum_{A \cup B = S} \hat{f}_j(A)\hat{g}_{i-j}(B)$$

To extract the final answer for a set S, we evaluate the rank slice corresponding to its cardinality: $h(S) = \hat{h}_{|S|}(S)$.
By substituting $i = |S|$, the non-zero terms mandate that $|A| = j$ and $|B| = i - j$, which means $|A| + |B| = i = |S|$ so $A \cap B = \emptyset$.

Complexity

- **Time:** The Forward and Inverse Zeta Transforms process $n + 1$ arrays of size 2^n, taking $O(n \cdot n2^n)$ time. The ranked point-wise multiplication takes $O(n^2)$ for each of the 2^n sets. Total Time: $O(n^2 2^n)$.
- **Space:** Allocating the ranked arrays requires $(n + 1) \times 2^n$ memory. Total Space: $O(n2^n)$.

Algorithm 134 Subset Convolution

```
function ZetaTransform(A, n)
    for i ← 0 to n − 1 do
        for S ← 0 to 2^n − 1 do
            if (S & (1 ≪ i)) ≠ 0 then
                A[S] ← A[S] + A[S ⊕ (1 ≪ i)]
function MobiusTransform(A, n)
    for i ← 0 to n − 1 do
        for S ← 0 to 2^n − 1 do
            if (S & (1 ≪ i)) ≠ 0 then
                A[S] ← A[S] − A[S ⊕ (1 ≪ i)]
function SubsetConvolution(f, g, n)
    Initialize f̂, ĝ, ĥ as arrays of size (n + 1) × 2^n filled with 0
    for S ← 0 to 2^n − 1 do
        f̂[|S|][S] ← f[S]
        ĝ[|S|][S] ← g[S]
    for k ← 0 to n do
        ZetaTransform(f̂[k], n)
        ZetaTransform(ĝ[k], n)
    for i ← 0 to n do
        for j ← 0 to i do
            for S ← 0 to 2^n − 1 do
                ĥ[i][S] ← ĥ[i][S] + f̂[j][S] · ĝ[i − j][S]
    for k ← 0 to n do
        MobiusTransform(ĥ[k], n)
    Initialize h as an array of size 2^n
    for S ← 0 to 2^n − 1 do
        h[S] ← ĥ[|S|][S]
    return h
```

16.2 Transposed Subset Convolution

Given two functions $f, g : 2^V \to \mathbb{F}$ over a set V of size n, the Transposed Subset Convolution is equivalent to Superset Convolution, and computes h defined as:

$$h(S) = \sum_{Y \supseteq S} f(Y)g(Y \setminus S)$$

We reverse f to get $\tilde{f}$, then compute the subset convolution with g, then reverse the output.

To see why this yields the superset convolution, we expand the definition of $h(S)$ from the three steps above by substituting $K = U \setminus S$:

$$h(S) = \tilde{h}(U \setminus S) = \sum_{X \subseteq U \setminus S} f(U \setminus X)g((U \setminus S) \setminus X)$$

Letting $Y = U \setminus X$, the summation condition $X \subseteq U \setminus S$ becomes $S \subseteq Y$. The sum becomes:

$$h(S) = \sum_{S \subseteq Y} f(Y)g(Y \setminus S)$$

16.3 Inversion and Quotient

Given a subset power series $g : 2^V \to \mathbb{F}$ where $g(\emptyset) \neq 0$, its inverse $h = g^{-1}$ is the unique series satisfying $h * g = \varepsilon$, where $\varepsilon(\emptyset) = 1$ and $\varepsilon(S) = 0$ for all $S \neq \emptyset$. More generally, the quotient $h = f/g$ is the series

satisfying $h * g = f$.

We can compute the quotient layer-by-layer (by rank) in $O(n^2 2^n)$ time by a DP directly in the Ranked Zeta Transform domain.

Expanding the definition of the subset convolution evaluated at a specific set X, we isolate the contribution of $h(X)$:

$$f(X) = \sum_{A \subseteq X} h(A)g(X \setminus A) = h(X)g(\emptyset) + \sum_{A \subsetneq X} h(A)g(X \setminus A)$$

Let $E(X) = \sum_{A \subsetneq X} h(A)g(X \setminus A)$. If we process the sets in increasing order of cardinality $l = |X|$, all required values of $h(A)$ for $|A| < l$ will have already been computed.

This allows us to inductively construct the ranked components $\hat{h}_l$ of the quotient. For a fixed rank l, the transformed accumulation of the smaller ranks is:

$$\mathcal{Z}(\hat{E}_l) = \sum_{i=1}^{l} \mathcal{Z}(\hat{h}_{l-i}) \cdot \mathcal{Z}(\hat{g}_i)$$

We recover E in the subset domain by applying the inverse Zeta (Möbius) transform. Then, for every set X with $|X| = l$, we extract the value of $h(X)$:

$$h(X) = \big(f(X) - E(X)\big) \cdot g(\emptyset)^{-1}$$

Finally, we apply the forward Zeta transform to $\hat{h}_l$ so that its transformed values $\mathcal{Z}(\hat{h}_l)$ are available for computing the subsequent ranks $l + 1, \ldots, n$.

Algorithm The computation proceeds iteratively over the rank dimension:

1. **Initialization:** Compute the inverse scalar $g(\emptyset)^{-1}$. Initialize $(n+1) \times 2^n$ arrays $\hat{g}$ and $\hat{h}$. Fill $\hat{g}$ by placing $g(S)$ into $\hat{g}_{|S|}[S]$ and apply the Fast Zeta Transform to every rank slice of $\hat{g}$.
2. **Layered DP:** For each rank l from 0 to n:
   (a) **Accumulate:** For each element S, compute $\hat{E}_l[S] = \sum_{i=1}^{l} \hat{h}_{l-i}[S] \cdot \hat{g}_i[S]$.
   (b) **Möbius Inversion:** Apply the Fast Möbius Transform to $\hat{E}_l$.
   (c) **Extract:** For every set X where $|X| = l$, compute $\hat{h}_l[X] = \big(f(X) - \hat{E}_l[X]\big) \cdot g(\emptyset)^{-1}$.
   (d) **Zeta Transform:** Apply the Fast Zeta Transform to $\hat{h}_l$.

For SPS inversion, this identical algorithm is used by setting $f(\emptyset) = 1$ and $f(X) = 0$ for all $|X| > 0$.

Algorithm 135 SPS Quotient

```
function SPS_Quotient(f, g, n)
    g_inv ← g[0]^{-1}
    Initialize ĝ, ĥ as arrays of size (n + 1) × 2^n filled with 0
    for S ← 0 to 2^n − 1 do
        ĝ[|S|][S] ← g[S]
    for k ← 0 to n do
        FastZetaTransform(ĝ[k], n)
    for l ← 0 to n do
        for i ← 1 to l do
            for S ← 0 to 2^n − 1 do
                ĥ[l][S] ← ĥ[l][S] + ĥ[l − i][S] · ĝ[i][S]
        FastMobiusTransform(ĥ[l], n)
        for X ← 0 to 2^n − 1 do
            if |X| == l then
                ĥ[l][X] ← (f[X] − ĥ[l][X]) · g_inv
            else
                ĥ[l][X] ← 0                          ▷ Clear side-effects from the Möbius transform
        FastZetaTransform(ĥ[l], n)
    Initialize h as an array of size 2^n
    for S ← 0 to 2^n − 1 do
        h[S] ← ĥ[|S|][S]
    return h
```

16.4 Exponential and Logarithm

Given a subset power series $f : 2^V \to \mathbb{F}$, we can compute its exponential $h = \exp(f)$ and logarithm $h = \log(f)$ in $O(n^2 2^n)$ time. For the exponential to exist, we require $f(\emptyset) = 0$. For the logarithm to exist, we require $f(\emptyset) = 1$.

The coefficient of the partial derivative $\frac{\partial}{\partial x_i}$ on a subset S (where $i \notin S$) yields the coefficient of $S \cup \{i\}$ in the original series.

By the chain rule, the exponential $h = \exp(f)$ satisfies the differential equation:

$$\frac{\partial h}{\partial x_i} = h * \frac{\partial f}{\partial x_i}$$

We build the series h iteratively, adding elements from 0 to $n-1$. Let $h_{<i}$ denote the series restricted to subsets drawn from $\{0, \ldots, i-1\}$. When we introduce the i-th element, we want to compute the slice of h for subsets where i is the maximum element. We denote this slice as $h_{\text{slice}(i)}$.

By restricting the equation above to the domain of the first i elements, we have:

$$h_{\text{slice}(i)} = h_{<i} * f_{\text{slice}(i)}$$

This is a subset convolution of size 2^i. Since $f(\emptyset) = 0$, the i-th half of the output is determined by the i-th half of f and the already-computed lower half of h.

Similarly, for the logarithm $h = \log(f)$, the chain rule dictates:

$$\frac{\partial h}{\partial x_i} = \frac{\partial f}{\partial x_i} * f^{-1} \implies \frac{\partial f}{\partial x_i} = \frac{\partial h}{\partial x_i} * f$$

Restricting this to the new element i, we obtain:

$$f_{\text{slice}(i)} = h_{\text{slice}(i)} * f_{<i}$$

This means the new slice of the logarithm can be directly computed using the SPS quotient:

$$h_{\text{slice}(i)} = f_{\text{slice}(i)} / f_{<i}$$

Algorithm The algorithm iteratively computes the exact slices of size 2^i for $i = 0$ to $n-1$.

- **Exp:** We initialize $h[\emptyset] = 1$. At step i, we extract the previously computed half $h[0 \dots 2^i - 1]$ and multiply it by $f[2^i \dots 2^{i+1} - 1]$ using a Subset Convolution of size 2^i. We write the output into $h[2^i \dots 2^{i+1} - 1]$.
- **Log:** We initialize $h[\emptyset] = 0$. At step i, we compute the Subset Quotient of $f[2^i \dots 2^{i+1} - 1]$ divided by $f[0 \dots 2^i - 1]$ and write the output into $h[2^i \dots 2^{i+1} - 1]$.

Algorithm 136 SPS Exponential and Logarithm

```
function SPS_Exp(f, n)
    if f[0] ≠ 0 then
        return Error
    Initialize h as an array of size 2^n filled with 0
    h[0] ← 1
    for i ← 0 to n − 1 do
        h[2^i ... 2^{i+1} − 1] ← h[0 ... 2^i − 1] * f[2^i ... 2^{i+1} − 1]
    return h
function SPS_Log(f, n)
    if f[0] == 0 then
        return Error
    Initialize h as an array of size 2^n filled with 0
    for i ← 0 to n − 1 do
        h[2^i ... 2^{i+1} − 1] ← f[2^i ... 2^{i+1} − 1] * f[0 ... 2^i − 1]^{−1}
    return h
```

Complexity For both operations, at the i-th iteration, we perform a subset convolution (or quotient) on arrays of length 2^i, which requires exactly $O(i^2 2^i)$ time. The total complexity is the sum:

$$T(n) = \sum_{i=0}^{n-1} O(i^2 2^i) = O(n^2 2^n)$$

16.5 Composition

Given a formal power series $F(x) = \sum_{k=0}^{m-1} f_k x^k$ and a subset power series $G : 2^V \to \mathbb{F}$ over n variables, their composition $H(S) = F(G(S))$ can be computed in $O(n^2 2^n)$ time.

1. Handling $G(\emptyset) \neq 0$ Subset composition requires $G(\emptyset) = 0$ to converge. We first apply the Inverse Borel transform to F, which maps $f_i \to i! f_i = F^{(i)}(0)$.

If the constant term is non-zero (let $a = G(\emptyset)$), we must shift the power series. We have $F^{(i)}(a) = \sum_{j \geq i} F^{(j)}(0) \frac{a^{j-i}}{(j-i)!}$. This shift is computed as a semi-correlation of the transformed coefficients with $\exp(ax)$. Once shifted, we set $G(\emptyset) = 0$.

2. Taylor Expansion We evaluate $F(G)$ by introducing the elements of the set one by one. Let $G_{<k}$ denote the subset power series restricted to the first k elements, and let $G_{\text{slice}(k)}$ be the slice of G where the k-th element is the strict maximum (with x_k factored out). We can iteratively decompose G:

$$G_{\leq k} = G_{<k} + x_k G_{\text{slice}(k)}$$

We use the Taylor expansion of F evaluated at $A + B$, where $A = G_{<k}$ and $B = x_k G_{\text{slice}(k)}$:

$$F(A + B) = F(A) + B * F'(A) + \frac{B * B}{2!} * F''(A) + \dots$$

In subset convolution, elements must be disjoint to yield a non-zero product and every term in B contains the element x_k, thus we only need to consider the first derivative:

$$F(G_{\leq k}) = F(G_{<k}) + x_k G_{\text{slice}(k)} * F'(G_{<k})$$

Notice that computing $F(G_{\leq k})$ requires $F'(G_{<k})$. Consequently, computing $F'(G_{<k})$ will require $F''(G_{<k-1})$, etc. The transition for evaluating the i-th derivative is:

$$F^{(i)}(G_{\leq k}) = F^{(i)}(G_{<k}) + x_k G_{\text{slice}(k)} * F^{(i+1)}(G_{<k})$$

3. DP Optimization We maintain a single working array C of length 2^n. We iterate the derivative order i downwards from n to 0. For a fixed i, our goal is to transform the state of C from representing $F^{(i+1)}(G)$ into $F^{(i)}(G)$.

We iterate the introduced variable index j downwards. At the start of step j, the lower prefix $C[0 \dots 2^j - 1]$ holds the evaluation of $F^{(i+1)}(G_{<j})$. We compute its subset convolution with $G_{\text{slice}(j)}$ and write the result into the upper half $C[2^j \dots 2^{j+1} - 1]$.

Because we iterate j downwards, we compute the upper halves first, preventing the overwriting of the smaller prefixes. We then set the base case (the empty set evaluated at the i-th derivative) by assigning $C[\emptyset] = f_i$. This is correct as our coefficients were mapped to $F^{(i)}(a)$.

Algorithm 137 SPS Composition

```
function SPS_Compose(F, G, n, m)
    F ← InverseBorel(F)
    a ← G[0]
    if a ≠ 0 then
        F ← SemiCorr(F, exp(ax))
        G[0] ← 0
    Initialize C as an array of size 2^n filled with 0
    for i ← n to 0 step −1 do
        for j ← n − i − 1 to 0 step −1 do
            X ← C[0 ... 2^j − 1]
            G_slice(j) ← G[2^j ... 2^(j+1) − 1]
            C[2^j ... 2^(j+1) − 1] ← X * G_slice(j)
        if i < m then
            C[0] ← F[i]
        else
            C[0] ← 0
    return C
```

Complexity The Inverse Borel Transform takes $O(m)$ time, and the Taylor shift takes $O(m \log m)$ time. The outer loop executes $O(n)$ times, and the inner loop performs a subset convolution on slices of size 2^j, requiring $O(j^2 2^j)$ operations. Summing this over i and j gives $O(n^2 2^n)$. Thus, the complexity is $O(n^2 2^n + m \log m)$.

16.6 Power Projection

Given a subset power series $G : 2^V \to \mathbb{F}$ over n variables and a weight function $W : 2^V \to \mathbb{F}$, the Power Projection problem asks us to compute the sequence of scalars c_k for $k = 0 \dots m-1$, defined as the inner product of W with the convolution powers of G:

$$c_k = \langle W, G^k \rangle = \sum_{S \subseteq V} W(S)[x^S]G^k$$

This is the transpose of the subset composition algorithm. Let $L_G(F) = F(G)$ be the operator defined by subset composition. Its adjoint L_G^T maps the vector W to the polynomial $C(x) = \sum c_k x^k$ such that $\langle W, L_G(F) \rangle = \langle L_G^T(W), F \rangle$.

Assume $G(\emptyset) = 0$. In the forward composition DP, the state vector $C \in \mathbb{F}^{2^n}$ is updated iteratively by nesting loops: i downwards from n to 0, and j downwards from $n-i-1$ to 0, we reverse this in the transposed algorithm.

Let X denote the lower half of the state $(0 \dots 2^j - 1)$ and Y denote the upper half $(2^j \dots 2^{j+1} - 1)$. The forward transition at step j is the map:

$$Y \leftarrow G_{\text{slice}(j)} * X$$

Let X^* and Y^* be the corresponding halves in the transposed algorithm. The adjoint of reading from X to overwrite Y is to project the dual values from Y^* through the transposed operator and accumulate them into X^*, then to clear Y^*.

$$X^* \leftarrow X^* + G_{\text{slice}(j)} *^T Y^*$$
$$Y^* \leftarrow 0$$

Finally, the forward DP puts the input coefficient f_i into $C[\emptyset]$ at the conclusion of the j iterations. The transposed operation extracts this value into the output $c_i = W[\emptyset]$ and clears $W[\emptyset] = 0$ at the start of the transposed iterations.

If the original series had a non-zero constant term $a = G(\emptyset)$, the forward composition shifted $F^{(i)}(0)$ to $F^{(i)}(a)$. The algorithm computes the semicorrelation with $\exp(ax)$, the transpose of which is multiplication with $\exp(ax)$.

In the forward algorithm, we applied the Inverse Borel transform, since this is diagonal, it's self-adjoint.

Algorithm 138 SPS Power Projection

```
function SPS_PowerProj(G, W, n, m)
    a ← G[0]
    G[0] ← 0
    Initialize C as an array of size n + 1 filled with 0
    Initialize S as an intermediate array of size 2^n
    for i ← 0 to n do
        C[i] ← W[0]
        W[0] ← 0
        for j ← 0 to n − i − 1 do
            X ← W[0 ... 2^j − 1]
            Y ← W[2^j ... 2^{j+1} − 1]
            G_slice(j) ← G[2^j ... 2^{j+1} − 1]
            S ← Y *^T G_slice(j)
            X ← X + S
            Y ← 0
    if a ≠ 0 then
        C ← C · exp(ax) (mod x^m)
    C ← InverseBorel(C)
    return C
```

16.7 Power

Given a subset power series $f : 2^V \to \mathbb{F}$ over n variables, computing the k-th power $h = f^k$ can be done in $O(n^2 2^n)$ time.

Case 1: $f(\emptyset) \neq 0$ If the constant term $a_0 = f(\emptyset)$ is non-zero, we can compute the power using the exponential and logarithm.

$$f^k = a_0^k \exp\left(k \log\left(\frac{f}{a_0}\right)\right)$$

Case 2: $f(\emptyset) = 0$ If the constant term is zero, every non-zero term in the series corresponds to a subset of size ≥ 1. Consequently, convolving the series with itself k times means every term in the result must have a subset size of at least k. Thus, if $k > n$, f^k is identically zero.

If $k \leq n$, we compute the power as a composition: $h(S) = P(f(S))$, where $P(x) = x^k$.

We can specialize our SPS Composition algorithm. We have:

$$P^{(i)}(0) = \begin{cases} k! & \text{if } i = k \\ 0 & \text{if } i < k \end{cases}$$

We C with $C[\emptyset] = k!$, which represents $P^{(k)}(f)$. We then run the DP downwards from $i = k - 1$ to 0. At the end of each derivative step, we set the base case $C[\emptyset] = P^{(i)}(0) = 0$.

Algorithm 139 SPS Power

```
function SPS_Power(f, k, n)
    a_0 ← f[0]
    if a_0 ≠ 0 then
        a_0^inv ← (a_0)^{-1}
        f_norm ← f · a_0^inv
        L ← SPS_Log(f_norm, n)
        L ← L · k
        E ← SPS_Exp(L, n)
        return E · (a_0^k)
    else
        if k > n then
            return Array of 0s
        Initialize C as an array of size 2^n filled with 0
        C[0] ← k!
        for i ← k − 1 to 0 step −1 do
            for j ← n − i − 1 to 0 step −1 do
                X ← C[0 ... 2^j − 1]
                f_slice(j) ← f[2^j ... 2^{j+1} − 1]
                C[2^j ... 2^{j+1} − 1] ← SubsetConvolution(X, f_slice(j))
            C[0] ← 0
        return C
```

Complexity If $f(\emptyset) \neq 0$, we perform one subset logarithm and one subset exponential, taking $O(n^2 2^n)$ time. If $f(\emptyset) = 0$, the DP matches the complexity of composition, and thus is also $O(n^2 2^n)$ time.